

目录

1 RustWeb-Vue 全栈项目接手指南（新手向）	46
1.1 一、这套文档能帮你什么	46
1.2 二、文档总目录与阅读路线	47
1.3 三、项目一句话画像	49
1.4 四、学习环境准备（写代码前先读）	50
1.5 五、学习方法建议	50
1.6 六、术语速查表（先混个脸熟）	51
1.7 七、常见问题解答（读文档前先看）	52
1.7.1 Q1：我是完全零基础，这套文档能看懂吗？	52
1.7.2 Q2：我只想改一个页面，要看哪些章节？	52
1.7.3 Q3：我想加一个全新的业务模块，看哪章？	52
1.7.4 Q4：文档里的代码和仓库不一致怎么办？	52
1.7.5 Q5：mermaid 图在 VS Code 里不显示怎么办？	52
1.7.6 Q6：如何快速定位一个文件属于哪个章节？	52
1.8 八、文档维护约定（给后续维护者）	52
1.9 九、阅读路线图（按天规划）	52
1.9.1 第 1~2 天：先建全局	52
1.9.2 第 3~5 天：后端	52
1.9.3 第 6~8 天：前端	52
1.9.4 第 9~10 天：工程与维护	53
1.10 十、结语与开始	53
1.11 十一、给新手的学习避坑提示	53
1.11.1 11.1 常见误区	53
1.11.2 11.2 正确的学习姿势	53
1.11.3 11.3 遇到问题怎么办	53
1.12 十二、全书结构一页纸（速记版）	53
1.13 十三、致读者的一段话	53
1.14 十四、关于本套文档的字数说明	54
1.15 十五、最后再嘱咐三件事	54
2 01 项目全景：先建立全局地图	54
2.1 1.1 这个项目到底是个什么系统	54
2.1.1 1.1.1 一句话定位	54
2.1.2 1.1.2 为什么会有 7 个前端而不是 1 个	55
2.1.3 1.1.3 这个项目不是 Tauri 项目	55
2.2 1.2 技术栈总览	55
2.2.1 1.2.1 后端技术栈（Rust）	55
2.2.2 1.2.2 前端技术栈（Vue）	56
2.2.3 1.2.3 前后端之间的“契约”机制	56
2.3 1.3 七个前端应用总览	57
2.3.1 1.3.1 应用一览表	57
2.3.2 1.3.2 应用间如何跳转	57
2.3.3 1.3.3 为什么端口是 5173~5179	58
2.4 1.4 后端模块总览	58
2.4.1 1.4.1 目录树（去注释版）	58
2.4.2 1.4.2 模块的三种“体型”	59
2.4.3 1.4.3 统一的分层模式	59

2.5	1.5 一次完整请求的全链路 (HTTP 版)	60
2.6	1.6 WebSocket 实时数据链路 (硬件模块)	60
2.7	1.7 数据存储全景	61
2.8	1.8 目录逐层解析 (动手对照)	62
	2.8.1 1.8.1 根目录文件	62
	2.8.2 1.8.2 一个前端应用的内部结构 (以 fj200c_information 为例)	62
	2.8.3 1.8.3 packages/shared 结构	63
2.9	1.9 构建与部署全景	63
	2.9.1 1.9.1 开发模式 (日常开发)	63
	2.9.2 1.9.2 生产模式 (单 exe)	64
2.10	1.10 权限体系全景 (RBAC)	64
2.11	1.11 本章小结与自测	64
2.12	1.12 深入: Cargo.toml 依赖逐个讲 (新手对照表)	65
2.13	1.13 深入: 运行时进程与线程模型	66
2.14	1.14 深入: 7 个前端应用逐个认识	66
	2.14.1 1.14.1 admin (管理后台, 端口 5174)	66
	2.14.2 1.14.2 fj200c_information (发动机监控, 端口 5173)	67
	2.14.3 1.14.3 fj200c_main (发动机测控, 端口 5179) ——最复杂	67
	2.14.4 1.14.4 fw100 / fw150 (设备台账, 端口 5175/5178)	68
	2.14.5 1.14.5 ftj1c (UDP 通信监控, 端口 5176)	68
	2.14.6 1.14.6 city3d (城市 3D, 端口 5177)	68
2.15	1.15 深入: 后端“带硬件”模块的共同骨架	68
2.16	1.16 本章必读文件清单 (先读这 6 个文件)	69
2.17	1.17 深入: 安全设计概览 (接盘前必知)	69
	2.17.1 1.17.1 认证与授权	70
	2.17.2 1.17.2 数据安全	70
	2.17.3 1.17.3 传输层	70
	2.17.4 1.17.4 部署安全	71
2.18	1.18 深入: 代码风格与阅读约定	71
	2.18.1 1.18.1 注释约定	71
	2.18.2 1.18.2 命名约定	71
	2.18.3 1.18.3 分层职责铁律	71
	2.18.4 1.18.4 错误处理约定	71
2.19	1.19 深入: 全局状态盘点 (哪些“全局变量”要心里有数)	72
2.20	1.20 项目的时间线: git 历史透露了什么	72
2.21	1.21 自测题答案与延伸阅读	73
	2.21.1 1.21.1 自测题简答	73
	2.21.2 1.21.2 建议延伸阅读顺序	73
2.22	1.22 端到端案例走读: 模拟数据从硬件层到浏览器页面	73
	2.22.1 第一步: 前端点“启动服务”	73
	2.22.2 第二步: 后端启动 8 路采集线程	73
	2.22.3 第三步: 会话线程解码并广播	74
	2.22.4 第四步: WS 桥推送到浏览器	74
	2.22.5 第五步: 前端分发到表格	74
	2.22.6 第六步: CSV 记录 (可选)	74
	2.22.7 1.22.1 这个案例暴露的架构要点 (回顾清单)	75
2.23	1.23 补充: 7 个应用的页面与菜单全览	75

2.23.1	1.23.1 各应用导航菜单对照	75
2.23.2	1.23.2 页面类型与数据源对照	75
2.24	1.24 补充：目录逐层解析（后端 src 全貌）	76
2.24.1	1.24.1 顶层文件职责	76
2.24.2	1.24.2 common 公共层文件职责	76
2.25	1.25 补充：一次 HTTP 请求的生命周期（从浏览器到数据库）	76
2.26	1.26 本章知识点串联测验	77
2.27	1.27 补充：后端代码组织哲学（为什么这么分层）	77
2.27.1	1.27.1 三层架构的动机	77
2.27.2	1.27.2 全局状态的组织方式	77
2.28	1.28 补充：HTTP 与 WebSocket 的分工	77
2.28.1	1.28.1 什么时候用哪个	77
2.28.2	1.28.2 WS 消息的三种类型	78
2.28.3	1.28.3 WS 与 HTTP 的鉴权差异	78
2.29	1.29 补充：前端工程化细节（构建与部署链路）	78
2.29.1	1.29.1 构建链路	78
2.29.2	1.29.2 为什么需要前端构建	78
2.29.3	1.29.3 类型检查的位置	78
2.30	1.30 补充：系统边界与限制（认识它才能用好它）	78
2.31	1.31 补充：01 章补充自测（10 题）	78
2.32	1.32 补充：7 个应用的页面与菜单全览（逐应用）	79
2.32.1	1.32.1 admin (5174)	79
2.32.2	1.32.2 fj200c_information (5173)	79
2.32.3	1.32.3 fj200c_main (5179)	79
2.32.4	1.32.4 fw100 / fw150 (5175 / 5178)	79
2.32.5	1.32.5 ftj1c (5176)	79
2.32.6	1.32.6 city3d (5177)	79
2.33	1.33 补充：目录逐层解析（前端通用结构）	79
2.34	1.34 补充：后端进程的生命周期（一次完整运行）	80
2.35	1.35 补充：01 章最后补充自测（5 题）	80
2.36	1.36 补充：数据模型总览（数据库表一览）	80
2.36.1	1.36.1 核心表	80
2.36.2	1.36.2 表设计特点	80
2.36.3	1.36.3 种子数据的作用	80
2.37	1.37 补充：安全模型总览（认证 + 授权）	81
2.37.1	1.37.1 认证（你是谁）	81
2.37.2	1.37.2 授权（你能做什么）	81
2.37.3	1.37.3 新增权限的完整链路	81
2.38	1.38 补充：配置与数据文件总览	81
2.39	1.39 补充：01 章最终自测（追加 8 题）	81
2.40	1.40 补充：开发环境搭建手把手（Windows）	81
2.40.1	1.40.1 前置工具	81
2.40.2	1.40.2 安装步骤	82
2.40.3	1.40.3 拉取与安装依赖	82
2.40.4	1.40.4 启动	82
2.40.5	1.40.5 常见安装问题	82
2.41	1.41 补充：IDE 推荐配置	82

2.41.1	1.41.1 VS Code 插件清单	82
2.41.2	1.41.2 工作区建议	82
2.42	1.42 补充：01 章高频自测（8 题）	82
2.43	1.43 补充：项目里用到的关键技术栈详解	83
2.43.1	1.43.1 后端关键依赖	83
2.43.2	1.43.2 前端关键依赖	83
2.43.3	1.43.3 工程化工具	83
2.44	1.44 补充：项目规模的量化认识	83
2.44.1	1.44.1 代码规模估算	83
2.44.2	1.44.2 数据规模	83
2.44.3	1.44.3 对学习者的意义	84
2.45	1.45 补充：01 章终局自测（8 题）	84
2.46	1.46 补充：常见业务场景的端到端流程	84
2.46.1	1.46.1 新用户登录流程	84
2.46.2	1.46.2 监控数据全流程	84
2.46.3	1.46.3 台账操作流程	84
2.46.4	1.46.4 配置修改流程	84
2.47	1.47 补充：学习本系统的推荐实战顺序	84
2.47.1	1.47.1 初级实战（改配置）	84
2.47.2	1.47.2 中级实战（改逻辑）	84
2.47.3	1.47.3 高级实战（加功能）	85
2.48	1.48 补充：01 章毕业自测（8 题）	85
2.49	1.49 补充：项目演进历史与设计取舍	85
2.49.1	1.49.1 从单体到多应用的演进	85
2.49.2	1.49.2 类型方案的演进	85
2.49.3	1.49.3 部署方案的演进	85
2.49.4	1.49.4 设计取舍的原则	85
2.50	1.50 补充：01 章大师自测（8 题）	85
2.51	1.51 补充：文档使用的核心概念对照表（翻译手册）	86
2.51.1	1.51.1 中文术语 → 代码概念	86
2.51.2	1.51.2 前端术语 → 代码概念	86
2.51.3	1.51.3 使用方式	86
2.52	1.52 补充：01 章权威自测（8 题）	86
2.53	1.53 补充：项目背后的设计思想（为什么值得学）	86
2.53.1	1.53.1 五个值得学的点	86
2.53.2	1.53.2 三个真实的工程教训	87
2.53.3	1.53.3 学完后你能带走的	87
2.54	1.54 补充：01 章权威自测（8 题）	87
3	02 Rust 语法速成（以本项目代码为教材）	87
3.1	2.1 先建立正确心态：Rust 的“三座大山”其实没那么可怕	87
3.2	2.2 工具链准备（5 分钟）	88
3.3	2.3 变量、类型与 let	88
3.3.1	2.3.1 基本变量	88
3.3.2	2.3.2 常量与静态变量	88
3.3.3	2.3.3 元组与结构体字段访问	89
3.4	2.4 结构体与枚举（本项目最核心的两种类型）	89
3.4.1	2.4.1 结构体（struct）	89

3.4.2	2.4.2 枚举 (enum) ——本项目最常用的类型	90
3.4.3	2.4.3 枚举是“类型安全的 if”	90
3.5	2.5 Option 与 Result: 没有 null 的世界	91
3.5.1	2.5.1 Option: 可能有值	91
3.5.2	2.5.2 Result: 可能出错	91
3.5.3	2.5.3 项目里最经典的组合: 链式调用	91
3.6	2.6 模式匹配: match / if let / let else	92
3.6.1	2.6.1 match: 穷尽所有分支	92
3.6.2	2.6.2 if let: 只关心一种情况	92
3.6.3	2.6.3 let-else: 不满足就提前返回	92
3.6.4	2.6.4 模式匹配在项目中的使用场景清单	93
3.7	2.7 trait: 接口与抽象	93
3.7.1	2.7.1 trait 是什么	93
3.7.2	2.7.2 trait 对象与泛型	94
3.7.3	2.7.3 本项目用到的标准库 trait (背下来)	94
3.8	2.8 宏: derive 宏与声明式宏	94
3.8.1	2.8.1 derive 宏: 自动实现	94
3.8.2	2.8.2 属性宏: utoipa::path	94
3.8.3	2.8.3 声明式宏: macro_rules!	95
3.9	2.9 错误处理: AppError 体系	95
3.9.1	2.9.1 统一错误类型	95
3.9.2	2.9.2 自动转 HTTP 响应	95
3.9.3	2.9.3 From 转换: 错误自动升级	96
3.9.4	2.9.4 新手错误处理口诀	96
3.10	2.10 async/await 与 tokio	96
3.10.1	2.10.1 什么是异步	96
3.10.2	2.10.2 tokio::select!: 同时监听多个源	97
3.10.3	2.10.3 新手 async 避坑	97
3.11	2.11 线程与并发 (本项目并发全景)	97
3.11.1	2.11.1 std::thread: 独立线程	97
3.11.2	2.11.2 共享状态的三件套	98
3.11.3	2.11.3 broadcast: 一对多广播通道	98
3.11.4	2.11.4 mpsc: 多生产者单消费者命令通道	99
3.11.5	2.11.5 ArcSwap: 无锁热替换	99
3.12	2.12 生命周期与借用: rust-analyzer 是你的眼睛	99
3.12.1	2.12.1 借用的两条规则	99
3.12.2	2.12.2 生命周期标注 'a: 编译器给你做证明题	99
3.12.3	2.12.3 新手最常遇到的借用错误与修法	100
3.13	2.13 字符串与集合	100
3.13.1	2.13.1 String vs &str	100
3.13.2	2.13.2 Vec: 动态数组	100
3.13.3	2.13.3 HashMap: 键值对	100
3.13.4	2.13.4 迭代器链 (读代码必备)	100
3.14	2.14 模块系统: mod / use / crate::	101
3.14.1	2.14.1 模块声明	101
3.14.2	2.14.2 路径三种写法	101
3.14.3	2.14.3 项目跨模块引用关系图	101

3.15	2.15 条件编译: #[cfg]	102
3.16	2.16 测试	102
3.17	2.17 新手常见编译错误速查 (本项目语境)	103
3.18	2.18 语法索引表 (改代码时快速定位)	104
3.19	2.19 所有权与借用: 项目实战案例精讲	104
	3.19.1 2.19.1 案例一: 跨线程数据传递 (session.rs 的模式)	104
	3.19.2 2.19.2 案例二: 借用与修改 (SHARED_DATA 更新)	105
	3.19.3 2.19.3 案例三: 避免“借用地狱”的两种写法	105
3.20	2.20 serde 属性大全 (JSON 序列化实战)	105
3.21	2.21 sqlx 查询语法大全 (数据库实战)	106
	3.21.1 2.21.1 查一行 (可能有/无)	106
	3.21.2 2.21.2 查多行	106
	3.21.3 2.21.3 插入并返回自增/生成值	106
	3.21.4 2.21.4 更新/删除 (不返回行)	107
	3.21.5 2.21.5 动态查询 (分页 + 聚合)	107
	3.21.6 2.21.6 手写 FromRow (JSON 字段解析)	107
	3.21.7 2.21.7 新手 sqlx 避坑	107
3.22	2.22 Axum 提取器大全 (handler 参数的秘密)	108
3.23	2.23 配置文件解析 (configparser 实战)	109
3.24	2.24 CSV 读写实战 (csv crate)	109
	3.24.1 2.24.1 写入 (csv_writer 封装)	109
	3.24.2 2.24.2 读取 (csv crate 读取)	110
	3.24.3 2.24.3 报表插值 (csv → 报表)	110
3.25	2.25 串口与 UDP 实战	111
	3.25.1 2.25.1 serialport: 打开与读写	111
	3.25.2 2.25.2 socket2 + tokio: UDP 组播	111
3.26	2.26 rust-embed: 把前端嵌进 exe	111
3.27	2.27 常用标准库与第三方类型速查	112
3.28	2.28 学习资源与路线图 (如果你还想深入 Rust)	112
3.29	2.29 读代码演练一: jwt.rs 逐行精读 (154 行)	113
	3.29.1 2.29.1 模块文档 (1-31 行)	113
	3.29.2 2.29.2 导入 (33-38 行)	113
	3.29.3 2.29.3 Claims 结构体 (56-64 行)	113
	3.29.4 2.29.4 create_token (86-116 行)	114
	3.29.5 2.29.5 verify_token (138-153 行)	114
3.30	2.30 读代码演练二: config.rs 逐行精读 (84 行)	115
3.31	2.31 读代码演练三: utils.rs 里的“奇技淫巧”	116
3.32	2.32 深入: 泛型与 trait 对象 (读懂 embed 与 quad_frame)	116
	3.32.1 2.32.1 泛型结构体	116
	3.32.2 2.32.2 泛型函数与 trait 约束	116
	3.32.3 2.32.3 泛型 vs trait 对象 (dyn)	117
	3.32.4 2.32.4 Arc vs Rc (多线程 vs 单线程)	117
	3.32.5 2.32.5 Mutex vs RwLock vs Atomic	117
3.33	2.33 深入: 闭包与函数式风格	117
	3.33.1 2.33.1 闭包写法	117
	3.33.2 2.33.2 闭包在项目中的用法清单	118
3.34	2.34 深入: 常见困惑辨析 (Rust 新手高频问题)	118

3.34.1	2.34.1 String 和 &str 到底怎么选?	118
3.34.2	2.34.2 Result 和 Option 怎么分?	118
3.34.3	2.34.3 ? 到底怎么工作?	118
3.34.4	2.34.4 mut 什么时候必须加?	118
3.34.5	2.34.5 为什么结构体字段全是 pub 也没关系?	119
3.34.6	2.34.6 编译慢怎么办?	119
3.34.7	2.34.7 调试输出怎么办?	119
3.35	2.35 本章自测: 你能独立读这段代码吗?	119
3.36	2.36 深入: tokio 任务并发模型 (spawn / join / select)	120
3.36.1	2.36.1 tokio::spawn: 异步任务	120
3.36.2	2.36.2 tokio::join! / try_join!: 并发等待	120
3.36.3	2.36.3 tokio::time: 定时任务	120
3.36.4	2.36.4 项目里的异步/线程混合模型总结	120
3.37	2.37 深入: tracing 日志系统	120
3.37.1	2.37.1 用法	120
3.37.2	2.37.2 级别与过滤	121
3.37.3	2.37.3 输出格式	121
3.37.4	2.37.4 项目日志点分布	121
3.38	2.38 深入: 测试编写 (项目已有测试的解剖)	121
3.38.1	2.38.1 项目里的两个测试	121
3.38.2	2.38.2 测试语法速成	122
3.38.3	2.38.3 如何给新代码补测试	122
3.39	2.39 本项目惯用代码模式十式 (改代码时照抄)	122
3.40	2.40 第二章收官: 你现在的 Rust 水平能做什么	123
3.41	2.41 深入: 所有权与 move 的图解理解	124
3.41.1	2.41.1 值、变量与所有权	124
3.41.2	2.41.2 移动 (move): 所有权转移	124
3.41.3	2.41.3 克隆 (clone): 数据复制	124
3.41.4	2.41.4 借用 (borrow): 不转移所有权	125
3.41.5	2.41.5 项目中 move 的实际体现 (再回看一遍)	125
3.41.6	2.41.6 所有权思维口诀	125
3.42	2.42 深入: 生命周期标注到底在说什么	125
3.42.1	2.42.1 为什么要生命周期	125
3.42.2	2.42.2 省略规则 (90% 情况不用写)	125
3.42.3	2.42.3 项目中的 'a 出现在哪	125
3.42.4	2.42.4 'static 生命周期	126
3.43	2.43 深入: chrono 与 uuid 实战	126
3.43.1	2.43.1 chrono: 时间处理	126
3.43.2	2.43.2 uuid: 主键生成	126
3.43.3	2.43.3 serde_json 实战	126
3.44	2.44 给新手的三个“热身练习” (改代码前先做)	127
3.45	2.45 深入: 异步编程中的常见模式 (tokio 实战)	127
3.45.1	2.45.1 spawn 独立任务 (后台运行)	127
3.45.2	2.45.2 select! 多路等待 (谁先到处理谁)	127
3.45.3	2.45.3 broadcast 广播通道	127
3.45.4	2.45.4 mpsc 多对一 (或者一对多串行)	128
3.45.5	2.45.5 RwLock / Mutex 的选择	128

3.46	2.46 深入: Trait 设计模式 (读懂接口抽象)	128
3.46.1	2.46.1 为什么用 trait 抽象硬件	128
3.46.2	2.46.2 动态分发 vs 静态分发	128
3.46.3	2.46.3 trait 的默认实现	128
3.47	2.47 深入: Rust 项目常见编译报错对照 (后端)	129
3.48	2.48 本章语法点索引 (速查表)	129
3.49	2.49 深入: Result 的错误链 (? 操作符与错误转换)	130
3.49.1	2.49.1 从底层错误到 AppError	130
3.49.2	2.49.2 From 转换 (自动?)	130
3.49.3	2.49.3 自定义错误链的调试价值	130
3.50	2.50 深入: 字符串处理的实战模式	130
3.50.1	2.50.1 拼接与格式化	130
3.50.2	2.50.2 子串与查找	130
3.50.3	2.50.3 解析数字	130
3.51	2.51 深入: 集合类型的选择 (什么时候用什么)	131
3.52	2.52 深入: 并发原语在项目中的分布	131
3.53	2.53 深入: 宏 (macro_rules!) 在项目中的实战	131
3.53.1	2.53.1 项目实例: define_com_port!	131
3.53.2	2.53.2 什么时候用宏	131
3.54	2.54 深入: 条件编译与 feature (项目实例)	132
3.54.1	2.54.1 embedded feature	132
3.54.2	2.54.2 cfg 的其他用法	132
3.55	2.55 深入: 测试与代码检查习惯 (后端)	132
3.55.1	2.55.1 三层检查	132
3.55.2	2.55.2 测试的组织	132
3.56	2.56 深入: 项目里的安全编码习惯	132
3.57	2.57 深入: 02 章补充自测 (10 题)	132
3.58	2.58 深入: 模式匹配的实战全集	133
3.58.1	2.58.1 常见的匹配用法	133
3.58.2	2.58.2 let-else (项目常见)	133
3.59	2.59 深入: 生命周期与借用的常见编译错误	133
3.60	2.60 深入: 所有权与性能的权衡	133
3.61	2.61 深入: 日志 (tracing/log) 的使用规范	133
3.62	2.62 深入: async 代码的注意事项	134
3.62.1	2.62.1 不要在 async 里干重活	134
3.62.2	2.62.2 锁的使用	134
3.62.3	2.62.3 常见 async 模式	134
3.63	2.63 深入: 02 章最终综合自测 (追加 10 题)	134
3.64	2.64 深入: 迭代器链的实战翻译 (新手对照)	134
3.64.1	2.64.1 其他常用迭代器	135
3.65	2.65 深入: 常见数值类型转换速查	135
3.66	2.66 深入: 处理 None 的四种姿势	135
3.67	2.67 深入: 闭包捕获的三种方式	135
3.68	2.68 深入: 时间与随机数处理	135
3.69	2.69 深入: 02 章超纲自测 (5 题)	136
3.70	2.70 深入: 结构体与枚举的完整实战	136
3.70.1	2.70.1 结构体三兄弟	136

3.70.2	2.70.2 结构体更新语法	136
3.70.3	2.70.3 枚举携带数据 (项目核心模式)	136
3.71	2.71 深入: trait 的完整实战 (impl Trait 与泛型)	136
3.71.1	2.71.1 trait 定义与实现	136
3.71.2	2.71.2 泛型约束	136
3.71.3	2.71.3 项目中的典型应用	137
3.72	2.72 深入: 工程目录组织的 Rust 惯例	137
3.72.1	2.72.1 模块声明的两种方式	137
3.72.2	2.72.2 可见性	137
3.73	2.73 深入: 02 章实战自测 (8 题)	137
3.74	2.74 深入: 文件系统操作的完整参考 (后端实战)	137
3.74.1	2.74.1 读写文件	137
3.74.2	2.74.2 目录操作	137
3.74.3	2.74.3 项目中的位置	138
3.75	2.75 深入: 序列化的完整参考 (serde 实战)	138
3.75.1	2.75.1 派生宏	138
3.75.2	2.75.2 常用属性	138
3.75.3	2.75.3 Option 字段的处理	138
3.76	2.76 深入: 02 章高频自测 (8 题)	138
3.77	2.77 深入: 异步编程的完整参考 (tokio 实战)	138
3.77.1	2.77.1 async/await 基础	138
3.77.2	2.77.2 tokio::spawn 并发任务	139
3.77.3	2.77.3 并发执行的组合	139
3.77.4	2.77.4 项目中的应用	139
3.78	2.78 深入: 常见数据结构的实用操作	139
3.78.1	2.78.1 Vec 常用操作	139
3.78.2	2.78.2 HashMap 常用操作	139
3.78.3	2.78.3 String 常用操作	139
3.79	2.79 深入: 02 章综合自测 (8 题)	139
3.80	2.80 深入: 常用 crate 的实用 API 速查	140
3.80.1	2.80.1 chrono (时间)	140
3.80.2	2.80.2 serde_json (JSON)	140
3.80.3	2.80.3 regex (正则)	140
3.80.4	2.80.4 anyhow (错误处理, 测试/工具用)	140
3.81	2.81 深入: 新手常犯的 Rust 错误及修复	140
3.82	2.82 深入: 代码风格与命名规范 (项目惯例)	141
3.82.1	2.82.1 命名规范	141
3.82.2	2.82.2 组织规范	141
3.82.3	2.82.3 格式化	141
3.83	2.83 深入: 02 章终局自测 (8 题)	141
3.84	2.84 深入: 阅读 Rust 代码的实战方法	141
3.84.1	2.84.1 从签名读起	141
3.84.2	2.84.2 跟踪类型流向	141
3.84.3	2.84.3 找关键模式	141
3.85	2.85 深入: Rust 版本的常用特性 (项目所用)	141
3.85.1	2.85.1 常用特性清单	141
3.85.2	2.85.2 用不上但要知道的	142

3.86	2.86 深入：02 章毕业自测（8 题）	142
3.87	2.87 深入：实际读一段项目代码（综合实战）	142
	3.87.1 2.87.1 目标代码	142
	3.87.2 2.87.2 逐段解读	142
	3.87.3 2.87.3 知识点串联	143
3.88	2.88 深入：02 章大师自测（8 题）	143
3.89	2.89 深入：命令行参数与环境变量	143
	3.89.1 2.89.1 环境变量（项目主要方式）	143
	3.89.2 2.89.2 命令参数（较少用）	143
	3.89.3 2.89.3 项目约定	143
3.90	2.90 深入：错误处理的风格指南	143
	3.90.1 2.90.1 三种错误风格	143
	3.90.2 2.90.2 什么时候 panic	143
	3.90.3 2.90.3 错误消息的写法	144
3.91	2.91 深入：02 章权威自测（8 题）	144
3.92	2.92 深入：调试 Rust 代码的实战技巧	144
	3.92.1 2.92.1 打印调试	144
	3.92.2 2.92.2 断言调试	144
	3.92.3 2.92.3 编译器提示的使用	144
3.93	2.93 深入：从语法到项目的迁移路径	144
	3.93.1 2.93.1 语法知识的三个等级	144
	3.93.2 2.93.2 项目练习清单（按等级）	144
	3.93.3 2.93.3 常见误区	145
3.94	2.94 深入：02 章权威自测（8 题）	145
3.95	2.95 深入：零基础者的 30 天 Rust 计划	145
	3.95.1 2.95.1 四周计划	145
	3.95.2 2.95.2 每日安排（1~2 小时）	145
	3.95.3 2.95.3 学习的产出物	145
3.96	2.96 深入：Rust 官方资源导航	145
	3.96.1 2.96.1 必看资源	145
	3.96.2 2.96.2 项目内的资源	145
	3.96.3 2.96.3 提问的姿势	146
3.97	2.97 深入：02 章权威自测（8 题）	146
3.98	2.98 深入：学完本章的检验清单	146
	3.98.1 2.98.1 语法检验（能看懂）	146
	3.98.2 2.98.2 实战检验（能动手）	146
	3.98.3 2.98.3 进入 03 章的标准	146
3.99	2.99 深入：02 章权威自测（8 题）	146
4	03 后端逐模块精读	147
4.1	3.1 第一个文件：main.rs（程序入口，232 行）	147
	4.1.1 3.1.1 文件总览	147
	4.1.2 3.1.2 模块声明（41-58 行）	147
	4.1.3 3.1.3 主函数与 tokio（98-99 行）	148
	4.1.4 3.1.4 启动七步走（104-229 行）	148
	4.1.5 3.1.5 你将来会改 main.rs 的场景	149
4.2	3.2 第二个文件：routes.rs（路由集中注册，145 行）	149
	4.2.1 3.2.1 设计思想	149

4.2.2	3.2.2 关键代码走读 (108-144 行)	150
4.2.3	3.2.3 nest 与 route 的区别 (新手易混)	150
4.2.4	3.2.4 新增模块时 routes.rs 怎么改	151
4.3	3.3 第三个文件: roles.rs (RBAC 唯一源, 268 行)	151
4.3.1	3.3.1 为什么它最重要	151
4.3.2	3.3.2 核心数据结构	151
4.3.3	3.3.3 关键函数	151
4.3.4	3.3.4 权限流动全景图	152
4.3.5	3.3.5 新增角色的后端改动 (预告)	152
4.4	3.4 第四个文件: database.rs (建表 + 种子数据, 1269 行)	152
4.4.1	3.4.1 设计思想: 无迁移文件的数据库	152
4.4.2	3.4.2 初始化流程 (init_database)	152
4.4.3	3.4.3 建表清单 (create_tables)	153
4.4.4	3.4.4 种子账号 (seed_data)	153
4.4.5	3.4.5 你将来会改 database.rs 的场景	154
4.5	3.5 common 公共层精读 (全系统的地基)	154
4.5.1	3.5.1 common/mod.rs: 模块门面	155
4.5.2	3.5.2 common/models.rs: 核心模型 (389 行)	155
4.5.3	3.5.3 common/middleware.rs: 三个中间件 (198 行)	156
4.5.4	3.5.4 common/error.rs: 统一错误 (212 行)	157
4.5.5	3.5.5 common/dto.rs: 公共响应 DTO	157
4.5.6	3.5.6 common/ws.rs: WebSocket 事件桥 (90 行)	157
4.5.7	3.5.7 common/service.rs: ServiceRuntime (线程管理)	158
4.5.8	3.5.8 common/io.rs: IoControl trait (2.7.1 已详述, 此处补充)	159
4.5.9	3.5.9 common/frame_extractor.rs: 帧提取器 (112 行)	159
4.5.10	3.5.10 common/quad_frame.rs: 四槽帧缓冲 (主备切换)	159
4.5.11	3.5.11 common/latest_frame.rs: 最新帧跟踪器	160
4.5.12	3.5.12 common/global_var.rs: 全局 KV (123 行)	160
4.5.13	3.5.13 common/csv_writer.rs: CSV 写入器 (2.24.1 已详述, 此处补充 Drop 细节)	160
4.5.14	3.5.14 common/config.rs: INI 封装 (2.23 已详述)	160
4.5.15	3.5.15 common/utils.rs: 工具函数 (2.31 已详述)	160
4.5.16	3.5.16 common/ledger.rs: 台账演示数据	160
4.5.17	3.5.17 common/least_squares.rs: 最小二乘拟合 (86 行)	161
4.6	3.6 auth 登录流程精读 (全系统入口)	161
4.6.1	3.6.1 auth 模块结构	161
4.6.2	3.6.2 登录的完整时序	161
4.6.3	3.6.3 handlers.rs 逐行 (登录 handler 是“全项目最标准的 handler 模板”)	161
4.6.4	3.6.4 services.rs 精读: 登录的服务层实现	162
4.6.5	3.6.5 create_user: 管理员建用户	163
4.6.6	3.6.6 routes.rs: 路由与中间件	163
4.7	3.7 admin 模块精读 (用户管理)	163
4.7.1	3.7.1 模块结构与路由 (routes.rs, 106 行)	163
4.7.2	3.7.2 handlers.rs 的安全细节 (230 行)	164
4.7.3	3.7.3 页面与接口对照 (admin 前端)	165
4.8	3.8 fj200c_information 完整精读 (最典型硬件模块范本)	165

4.8.1	3.8.1 模块全景	165
4.8.2	3.8.2 事件与通道 (mod.rs)	166
4.8.3	3.8.3 会话线程 run_one_connection (session.rs, 核心中的核心) ..	166
4.8.4	3.8.4 handle_frame: 帧处理与 CSV 状态机 (212-311 行)	167
4.8.5	3.8.5 decode_shared_data: 16 字段解码 (318-365 行)	168
4.8.6	3.8.6 service.rs: 服务编排 (177 行)	168
4.8.7	3.8.7 handlers.rs: 8 个 HTTP 接口 + WS	169
4.8.8	3.8.8 config.rs: 热加载配置	170
4.8.9	3.8.9 mock.rs: 模拟数据源	170
4.9	3.9 fj200c_main 精读 (最复杂模块: 三路串口 + 报表)	171
4.9.1	3.9.1 与 fj200c_information 的差异总览	171
4.9.2	3.9.2 三路串口的抽象: abstract_com.rs	171
4.9.3	3.9.3 宏生成三路实现: com.rs	171
4.9.4	3.9.4 服务启停 (service.rs, 196 行)	172
4.9.5	3.9.5 模拟开关与主题 (服务端状态)	172
4.9.6	3.9.6 报表生成 (report.rs)	173
4.9.7	3.9.7 三路协议的启动线程 (com.rs init_ecu / init_adam / init_dyno)	173
4.10	3.10 ftj1c 精读 (UDP 组播 + 主备切换)	173
4.10.1	3.10.1 模块结构	173
4.10.2	3.10.2 主备切换架构 (本项目最复杂的并发设计)	173
4.10.3	3.10.3 模拟模式的设计 (process.rs 顶部注释就是说明书)	174
4.10.4	3.10.4 事件节流器 Throttle (50ms = 20 FPS)	174
4.10.5	3.10.5 坐标转换 (com.rs)	174
4.11	3.11 fw100 / fw150 精读 (最简模块)	174
4.11.1	3.11.1 四个文件全部内容量级	174
4.11.2	3.11.2 fw100 与 fw150 的差异	175
4.12	3.12 city3d 精读 (数据库 CRUD 范本)	175
4.12.1	3.12.1 模块结构	175
4.12.2	3.12.2 接口一览 (14 个)	175
4.12.3	3.12.3 分页与钳制 (handlers.rs)	175
4.12.4	3.12.4 LEFT JOIN 聚合 (services.rs 的 N+1 优化)	176
4.12.5	3.12.5 overview 聚合统计	176
4.13	3.13 role_template 精读 (新角色启动包)	176
4.14	3.14 api_docs.rs 精读 (OpenAPI 聚合 + 防漂移测试)	177
4.14.1	3.14.1 文件结构 (247 行)	177
4.14.2	3.14.2 新增 handler 时必须同步的三处	177
4.15	3.15 embedded_assets.rs 精读 (单 exe 前端内嵌)	178
4.15.1	3.15.1 结构体定义 (7 个应用各一个)	178
4.15.2	3.15.2 泛型处理器	178
4.15.3	3.15.3 路由注册 (21 条)	178
4.15.4	3.15.4 dev 模式对照 (main.rs 的 ServeDir)	178
4.16	3.16 后端精读收官: 模块知识地图	178
4.17	3.17 补充: 后端代码阅读方法论 (怎么读一个陌生模块)	179
4.17.1	3.17.1 五步阅读法	179
4.17.2	3.17.2 快速定位技巧	179
4.18	3.18 补充: 三大监控模块对照 (一次读懂三个)	179

4.19	3.19 补充：后端错误处理的完整链路	180
4.19.1	3.19.1 AppError 的传播	180
4.19.2	3.19.2 自定义错误信息	180
4.19.3	3.19.3 前端如何感知	180
4.20	3.20 补充：后端单元测试现状与写法	180
4.20.1	3.20.1 现有测试	180
4.20.2	3.20.2 怎么写新测试（以工具函数为例）	180
4.21	3.21 补充：后端性能与并发注意点	181
4.22	3.22 补充：后端精读自测（30 题精华 10 问）	181
4.23	3.23 补充：auth 模块完整走读（登录服务细节）	181
4.23.1	3.23.1 登录的四步校验	181
4.23.2	3.23.2 auth_me 的动态权限	181
4.24	3.24 补充：admin 模块权限分组路由细节	182
4.24.1	3.24.1 三组路由的中间件组合	182
4.24.2	3.24.2 防自锁的实现	182
4.25	3.25 补充：common 工具层逐个说明	182
4.26	3.26 补充：fj200c_main 的 ECU 解码与指令细节	182
4.26.1	3.26.1 EcuFields 结构（29 字段）	182
4.26.2	3.26.2 三路协议的差异	182
4.27	3.27 补充：ftj1c 的主备切换细节	182
4.27.1	3.27.1 为什么需要主备	182
4.27.2	3.27.2 QuadFrame<95> 的作用	183
4.27.3	3.27.3 节流 50ms 的意义	183
4.28	3.28 补充：city3d 的 SQL 聚合细节	183
4.28.1	3.28.1 overview 的聚合	183
4.28.2	3.28.2 LEFT JOIN 的用法	183
4.28.3	3.28.3 分页 clamp	183
4.29	3.29 补充：03 章补充自测（追加 10 题）	183
4.30	3.30 深入：Config 读取的完整链路（以 fj200c_information 为例）	183
4.30.1	3.30.1 配置在内存中的形态	183
4.30.2	3.30.2 热更新的实现	184
4.30.3	3.30.3 配置校验的重要性	184
4.31	3.31 深入：CSV 写入的细节（数据如何落盘）	184
4.31.1	3.31.1 写入时机	184
4.31.2	3.31.2 写入格式	184
4.31.3	3.31.3 为什么不用 SQLite 存帧数据	184
4.32	3.32 深入：WebSocket 消息的结构设计	184
4.32.1	3.32.1 消息信封	184
4.32.2	3.32.2 三种核心消息	184
4.32.3	3.32.3 心跳与重连	185
4.33	3.33 深入：路由注册与中间件顺序	185
4.33.1	3.33.1 中间件顺序的影响	185
4.33.2	3.33.2 资源路由 vs 手动路由	185
4.34	3.34 深入：错误处理中间件与全局异常	185
4.34.1	3.34.1 AppError 的序列化	185
4.34.2	3.34.2 handler 返回的错误如何统一	185
4.35	3.35 深入：单元测试的写法与执行	185

4.35.1	3.35.1 测试什么	185
4.35.2	3.35.2 测试写法示例	185
4.35.3	3.35.3 执行	186
4.36	3.36 深入: 03 章最终综合自测 (追加 10 题)	186
4.37	3.37 深入: 项目实战——完整读一个 handler (fw100 列表)	186
4.37.1	3.37.1 代码定位	186
4.37.2	3.37.2 逐行翻译	186
4.37.3	3.37.3 从 handler 到数据库的完整链	186
4.38	3.38 深入: sqlx 查询的三个核心 API	187
4.38.1	3.38.1 列名映射的约定	187
4.39	3.39 深入: 事务的用法 (什么时候需要)	187
4.40	3.40 深入: 索引与查询性能 (SQLite 实践)	187
4.40.1	3.40.1 什么时候加索引	187
4.40.2	3.40.2 验证索引是否生效	187
4.40.3	3.40.3 反模式	187
4.41	3.41 深入: 03 章终极自测 (5 题)	188
4.42	3.42 深入: Axum 路由与提取器的完整参考	188
4.42.1	3.42.1 常见提取器	188
4.42.2	3.42.2 提取器顺序注意	188
4.42.3	3.42.3 返回类型的写法	188
4.43	3.43 深入: 全局状态 AppState 的完整设计	188
4.43.1	3.43.1 为什么用 Clone	188
4.43.2	3.43.2 模块级状态 vs AppState	188
4.44	3.44 深入: 中间件链的完整自定义	189
4.44.1	3.44.1 中间件的三层结构	189
4.44.2	3.44.2 extensions 的用途	189
4.45	3.45 深入: 03 章实战自测 (8 题)	189
4.46	3.46 深入: WebSocket 后端的完整实现参考	189
4.46.1	3.46.1 建立连接	189
4.46.2	3.46.2 发送消息	190
4.46.3	3.46.3 断开处理	190
4.47	3.47 深入: 串口通信后端的完整实现参考	190
4.47.1	3.47.1 打开与配置	190
4.47.2	3.47.2 读线程	190
4.47.3	3.47.3 写指令	190
4.47.4	3.47.4 波特率与协议	190
4.48	3.48 深入: 定时任务与周期操作	190
4.48.1	3.48.1 项目中的定时任务	191
4.49	3.49 深入: 03 章高频自测 (8 题)	191
4.50	3.50 深入: CSV 记录的完整实现参考	191
4.50.1	3.50.1 开启与关闭	191
4.50.2	3.50.2 写入线程	191
4.50.3	3.50.3 文件名规范	191
4.50.4	3.50.4 容量管理	191
4.51	3.51 深入: 帧提取器的完整实现参考	191
4.51.1	3.51.1 问题背景	191
4.51.2	3.51.2 通用实现	192

4.51.3	3.51.3 断帧/粘帧的处理	192
4.51.4	3.51.4 测试要点	192
4.52	3.52 深入：解码器的完整实现参考	192
4.52.1	3.52.1 二进制解码	192
4.52.2	3.52.2 缩放系数	192
4.52.3	3.52.3 解码失败的日志	192
4.53	3.53 深入：03 章综合自测（8 题）	192
4.54	3.54 深入：每路服务的启动/停止生命周期	193
4.54.1	3.54.1 状态机	193
4.54.2	3.54.2 启动的完整动作	193
4.54.3	3.54.3 停止的完整动作	193
4.54.4	3.54.4 幂等性设计	193
4.55	3.55 深入：状态管理（服务状态与配置状态）	193
4.55.1	3.55.1 状态存储位置	193
4.55.2	3.55.2 状态广播的时机	193
4.55.3	3.55.3 前端状态同步	193
4.56	3.56 深入：模拟模式（Mock）的设计	194
4.56.1	3.56.1 为什么需要 Mock	194
4.56.2	3.56.2 模拟数据源的结构	194
4.56.3	3.56.3 数据源切换	194
4.56.4	3.56.4 Mock 的注意事项	194
4.57	3.57 深入：报表与试验信息（fj200c_main 特有）	194
4.57.1	3.57.1 试验信息是什么	194
4.57.2	3.57.2 报表生成流程	194
4.57.3	3.57.3 状态点（StatePoints）	194
4.58	3.58 深入：03 章终局自测（8 题）	194
4.59	3.59 深入：后端日志的完整规范	195
4.59.1	3.59.1 日志级别速查	195
4.59.2	3.59.2 日志内容规范	195
4.59.3	3.59.3 日志排查示例	195
4.60	3.60 深入：权限校验的完整实现	195
4.60.1	3.60.1 三层校验链路	195
4.60.2	3.60.2 路由上的组合	195
4.60.3	3.60.3 权限检查的返回	195
4.61	3.61 深入：数据库连接的池化	195
4.61.1	3.61.1 连接池的意义	195
4.61.2	3.61.2 配置连接池	195
4.61.3	3.61.3 使用注意	196
4.62	3.62 深入：03 章毕业自测（8 题）	196
4.63	3.63 深入：常见需求的标准实现模式库	196
4.63.1	3.63.1 分页查询模板	196
4.63.2	3.63.2 增删改查模板	196
4.63.3	3.63.3 唯一性校验模板	197
4.64	3.64 深入：多模块共享代码的组织	197
4.64.1	3.64.1 共享层的清单	197
4.64.2	3.64.2 使用方式	197
4.64.3	3.64.3 什么时候把代码下沉到 common	197

4.65	3.65 深入：03 章大师自测（8 题）	197
4.66	3.66 深入：后端测试策略详解	197
4.66.1	3.66.1 测试金字塔	197
4.66.2	3.66.2 项目现有测试	197
4.66.3	3.66.3 单元测试写法模板	198
4.66.4	3.66.4 集成测试模板（HTTP）	198
4.67	3.67 深入：性能与资源管理（后端运行期）	198
4.67.1	3.67.1 内存与线程	198
4.67.2	3.67.2 资源释放	198
4.67.3	3.67.3 监控指标（可观测性）	198
4.68	3.68 深入：03 章权威自测（8 题）	198
4.69	3.69 深入：各模块间的关系图（后端全景复盘）	199
4.69.1	3.69.1 关系解读	199
4.69.2	3.69.2 为什么这样设计	199
4.70	3.70 深入：后端设计模式的总结	199
4.71	3.71 深入：03 章权威自测（8 题）	199
4.72	3.72 深入：后端开发环境配置（rust-analyzer 使用指南）	200
4.72.1	3.72.1 rust-analyzer 的日常用法	200
4.72.2	3.72.2 常用 workflow	200
4.72.3	3.72.3 常见问题	200
4.73	3.73 深入：后端代码审查清单（提交前自查）	200
4.74	3.74 深入：03 章权威自测（8 题）	200
4.75	3.75 深入：后端性能调优的完整清单	201
4.75.1	3.75.1 数据库层	201
4.75.2	3.75.2 并发层	201
4.75.3	3.75.3 数据层	201
4.76	3.76 深入：后端监控与指标	201
4.76.1	3.76.1 代码内指标	201
4.76.2	3.76.2 暴露方式	201
4.76.3	3.76.3 告警阈值建议	201
4.77	3.77 深入：03 章权威自测（8 题）	201
5	04 Vue 3 与 TypeScript 语法速成（以本项目代码为教材）	202
5.1	4.1 先建立 Vue 3 的心智模型	202
5.1.1	4.1.1 Vue 的核心概念三件套	202
5.1.2	4.1.2 一个组件长什么样（本项目真实例子）	202
5.2	4.2 模板语法（template）	203
5.2.1	4.2.1 插值与指令速查	203
5.2.2	4.2.2 模板里的语法糖	203
5.3	4.3 响应式核心：ref / reactive / computed / watch	204
5.3.1	4.3.1 ref：单个值（最常用）	204
5.3.2	4.3.2 reactive：对象（整体响应式）	204
5.3.3	4.3.3 ref vs reactive 怎么选（项目约定）	204
5.3.4	4.3.4 computed：派生状态	204
5.3.5	4.3.5 watch：监听变化	205
5.3.6	4.3.6 模板中直接可用的全局对象	205
5.4	4.4 <script setup> 详解（本项目的统一写法）	205
5.4.1	4.4.1 为什么用 script setup	205

5.4.2	4.4.2 组件导入即用	205
5.4.3	4.4.3 props 与 emits (组件通信)	205
5.4.4	4.4.4 生命周期钩子	206
5.5	4.5 TypeScript 速成 (本项目用法)	206
5.5.1	4.5.1 类型注解基础	206
5.5.2	4.5.2 接口与类型别名	206
5.5.3	4.5.3 联合类型与类型收窄 (WS 事件分发核心)	206
5.5.4	4.5.4 泛型与工具类型	207
5.5.5	4.5.5 项目类型体系 (三层 re-export)	207
5.5.6	4.5.6 项目 tsconfig 严格模式 (写代码时注意)	207
5.6	4.6 Pinia 状态管理 (本项目用法)	207
5.6.1	4.6.1 什么是 Pinia	207
5.6.2	4.6.2 创建 store 的两种风格	207
5.6.3	4.6.3 组件中使用 store	208
5.6.4	4.6.4 store-to-refs: 解构保持响应式	208
5.6.5	4.6.5 本项目的 store 家族	208
5.7	4.7 Vue Router (路由与守卫)	208
5.7.1	4.7.1 路由表	208
5.7.2	4.7.2 路由守卫 (beforeEach)	209
5.7.3	4.7.3 动态导入 (懒加载)	209
5.8	4.8 Element Plus 组件库速查 (本项目高频组件)	209
5.8.1	4.8.1 布局与容器	209
5.8.2	4.8.2 表单 (登录页标准组合)	209
5.8.3	4.8.3 表格 (列表页标准组合)	210
5.8.4	4.8.4 弹窗与消息	210
5.8.5	4.8.5 下拉菜单 (导航栏用户区)	210
5.8.6	4.8.6 本项目 Element Plus 组件使用统计 (按频率)	211
5.9	4.9 WebSocket 前端连接 (项目两种模式)	211
5.9.1	4.9.1 连接地址构建 (shared 公共函数)	211
5.9.2	4.9.2 模式一: 组件级连接 (fj200c_information / ftj1c)	211
5.9.3	4.9.3 模式二: 模块级单例连接 (fj200c_main)	212
5.9.4	4.9.4 两种模式选择建议	212
5.9.5	4.9.5 消息分发模式 (handleEvent)	212
5.10	4.10 ECharts 可视化 (fj200c_information / fj200c_main)	213
5.10.1	4.10.1 基本用法	213
5.10.2	4.10.2 实时曲线的数据流 (与 WS 协作)	213
5.10.3	4.10.3 图表类型清单 (本项目)	213
5.11	4.11 组合式函数 (composables): 本项目逻辑复用的核心	213
5.11.1	4.11.1 什么是组合式函数	213
5.11.2	4.11.2 项目中的组合式函数清单	214
5.11.3	4.11.3 组合式函数范例 (useClock, 完整版)	214
5.11.4	4.11.4 一个视图组装多个组合式函数 (Monitor.vue 模式)	214
5.12	4.12 样式系统: CSS 变量与双主题	215
5.12.1	4.12.1 全局样式组织	215
5.12.2	4.12.2 各应用样式文件布局	215
5.12.3	4.12.3 Scoped 样式与 :deep	215
5.13	4.13 前端工程: Vite 开发体验	215

5.13.1	4.13.1 dev 服务器做了什么	215
5.13.2	4.13.2 HMR (热更新)	216
5.13.3	4.13.3 构建 (npm run build)	216
5.14	4.14 本章自测: 读一段真实代码	216
5.15	4.15 API 调用模式深入 (本项目前端请求全景)	216
5.15.1	4.15.1 一次请求的完整生命周期	216
5.15.2	4.15.2 响应处理三式 (项目统一的写法)	217
5.15.3	4.15.3 401 的全局处理 (token 过期)	217
5.15.4	4.15.4 facade 层的作用 (为什么多包一层)	218
5.16	4.16 Vue Router 深入: 项目特殊用法	218
5.16.1	4.16.1 两个 base 的奥妙 (dev vs prod)	218
5.16.2	4.16.2 路由 meta 的权限设计 (回看 4.7.1)	218
5.16.3	4.16.3 程式化导航	218
5.17	4.17 表单与校验深入 (Element Plus 全流程)	219
5.17.1	4.17.1 动态规则 (根据场景切换校验)	219
5.17.2	4.17.2 校验方法	219
5.17.3	4.17.3 自定义校验	219
5.18	4.18 前端调试技巧 (新手必会)	219
5.18.1	4.18.1 Vue DevTools (浏览器扩展)	219
5.18.2	4.18.2 F12 Network 面板	219
5.18.3	4.18.3 常见前端报错速查	220
5.18.4	4.18.4 项目内联调试手段	220
5.19	4.19 项目前端代码风格约定 (改代码时遵守)	220
5.19.1	4.19.1 命名约定	220
5.19.2	4.19.2 文件组织约定	220
5.19.3	4.19.3 代码纪律	220
5.20	4.20 新手常见 Vue 坑 (本项目语境)	221
5.21	4.21 语法索引表 (改代码时快速定位)	222
5.22	4.22 深入: 响应式原理 (理解“为什么能自动更新”)	222
5.22.1	4.22.1 从 getter/setter 说起	222
5.22.2	4.22.2 为什么项目里“改 store 数据界面就自动变”	222
5.22.3	4.22.3 ref 的 .value 为什么在模板里不用写	222
5.22.4	4.22.4 响应式丢失的典型场景	222
5.23	4.23 深入: v-model 自定义组件 (双向绑定的本质)	223
5.23.1	4.23.1 本质是什么	223
5.23.2	4.23.2 自定义组件实现 v-model	223
5.23.3	4.23.3 多值 v-model (本项目应用较少, 了解即可)	223
5.24	4.24 深入: provide / inject (跨层级传值)	223
5.24.1	4.24.1 用法	223
5.24.2	4.24.2 项目中的使用	223
5.25	4.25 深入: city3d 的 Three.js 基础 (只讲够看懂的程度)	223
5.25.1	4.25.1 Three.js 是什么	223
5.25.2	4.25.2 city3d 用到的 Three.js 特性	224
5.25.3	4.25.3 新手须知	224
5.26	4.26 深入: 前端构建与性能	224
5.26.1	4.26.1 构建产物分析	224
5.26.2	4.26.2 项目用到的性能手段	225

5.26.3	4.26.3 依赖安装规则（重要）	225
5.27	4.27 深入：Vue 生命周期全图	225
5.28	4.28 深入：TypeScript 高级模式（本项目实战）	225
5.28.1	4.28.1 泛型工厂（shared 的 createAuthStore）	225
5.28.2	4.28.2 类型守卫与谓词函数	225
5.28.3	4.28.3 模板类型（template literal types）	225
5.28.4	4.28.4 条件类型与映射类型（orval 生成的内部实现）	226
5.28.5	4.28.5 非空断言与可选链的取舍	226
5.29	4.29 第二章收官：动手练习清单	226
5.30	4.30 逐行精读：main.ts 与 App.vue（每个应用的骨架）	226
5.30.1	4.30.1 main.ts（7 个应用几乎相同）	226
5.30.2	4.30.2 App.vue（应用根组件）	226
5.30.3	4.30.3 vue-env.d.ts（类型声明）	227
5.31	4.31 逐行精读：axios 拦截器（shared/api/index.ts）	227
5.32	4.32 大页面拆解：Users.vue（admin 最复杂的页面，507 行）	228
5.32.1	4.32.1 模板结构（四区块）	228
5.32.2	4.32.2 script 结构（关注点分离）	228
5.32.3	4.32.3 从页面学到的模式	229
5.33	4.33 深入：响应式布局与自适应（utils/responsive.ts）	229
5.34	4.34 深入：状态流全景（一个页面从加载到更新的完整数据流）	229
5.35	4.35 第四章收官：知识自测	230
5.36	4.36 深入：TypeScript 严格模式的约束（为什么报错这么多）	230
5.36.1	4.36.1 项目 tsconfig 关键项	230
5.36.2	4.36.2 新手最常见的报错与修法	231
5.36.3	4.36.3 vue-tsc 的意义	231
5.37	4.37 深入：组件通信全谱（什么时候用什么）	231
5.37.1	4.37.1 四种通信方式的取舍	231
5.37.2	4.37.2 事件总线示例（vueuse 的 useEventBus）	231
5.38	4.38 深入：异步与竞态处理	232
5.38.1	4.38.1 Promise.all 并行请求	232
5.38.2	4.38.2 竞态问题（快速点击/快速切换的坑）	232
5.38.3	4.38.3 防抖与节流（vueuse）	232
5.39	4.39 深入：Vite 环境变量与模式	232
5.39.1	4.39.1 三种模式	232
5.39.2	4.39.2 环境变量文件（各前端目录）	232
5.39.3	4.39.3 项目用到的关键变量	233
5.40	4.40 深入：import 路径别名（@ 与 @shared）	233
5.41	4.41 第四章扩展阅读：Element Plus 组件速查	233
5.42	4.42 深入：el-table 高级用法（Monitor 页的核心）	233
5.42.1	4.42.1 自定义单元格插槽	233
5.42.2	4.42.2 多级表头（嵌套表头）	234
5.42.3	4.42.3 固定列与斑马纹	234
5.42.4	4.42.4 列宽自适应	234
5.43	4.43 深入：ECharts 在项目中的实际用法	234
5.43.1	4.43.1 基础三步（Vue 集成模式）	234
5.43.2	4.43.2 实时曲线（数据流 → setOption）	234
5.43.3	4.43.3 常见图表类型（项目对照）	234

5.44	4.44 深入：CSS 作用域与 :deep (样式隔离)	235
5.44.1	4.44.1 scoped 原理	235
5.44.2	4.44.2 为什么有时需要 :deep()	235
5.44.3	4.44.3 全局样式 (style.css)	235
5.45	4.45 深入：暗色模式与主题切换 (双主题)	235
5.45.1	4.45.1 原理	235
5.45.2	4.45.2 应用自身主题 (fj200c_main 的航天主题)	235
5.45.3	4.45.3 主题持久化	236
5.46	4.46 第四章最终自测 (进阶题)	236
5.47	4.47 深入：错误处理与空态设计 (前端)	236
5.47.1	4.47.1 三层错误处理	236
5.47.2	4.47.2 空态 (数据为空时页面表现)	236
5.47.3	4.47.3 加载态三件套	236
5.48	4.48 深入：模板 ref 与组件实例 (拿到 DOM/组件)	236
5.48.1	4.48.1 模板 ref 拿 DOM	236
5.48.2	4.48.2 模板 ref 拿子组件实例	237
5.48.3	4.48.3 defineExpose (子组件暴露方法)	237
5.49	4.49 深入：KeepAlive 与组件缓存 (多标签页)	237
5.49.1	4.49.1 作用	237
5.49.2	4.49.2 本项目用法	237
5.49.3	4.49.3 KeepAlive 的坑	238
5.50	4.50 第四章收官：综合项目实践 (任选其一)	238
5.51	4.51 深入：composable 的设计模式 (模块化与复用)	238
5.51.1	4.51.1 项目 composable 分类	238
5.51.2	4.51.2 标准写法模板	238
5.51.3	4.51.3 为什么用 composable 而不是直接写在组件里	239
5.52	4.52 深入：Pinia 最佳实践 (项目风格)	239
5.52.1	4.52.1 项目 store 分类	239
5.52.2	4.52.2 异步 action 的标准写法	239
5.52.3	4.52.3 组件中使用 store 的规范	239
5.52.4	4.52.4 store 之间的引用 (跨 store 通信)	239
5.53	4.53 深入：动态 import 与按需加载 (打包优化)	240
5.53.1	4.53.1 三种导入方式对比	240
5.53.2	4.53.2 项目中的动态导入	240
5.53.3	4.53.3 首屏优化手段总结	240
5.54	4.54 深入：跨应用通信 (7 个应用的关系)	240
5.54.1	4.54.1 应用间跳转	240
5.54.2	4.54.2 登录态共享	240
5.54.3	4.54.3 应用间隔离	240
5.55	4.55 第四章完结：十问十答自查表	241
5.56	4.56 深入：Transition 与列表动画 (动效速成)	241
5.56.1	4.56.1 单元素过渡	241
5.56.2	4.56.2 列表过渡 (Monitor 表格行进场)	241
5.57	4.57 深入：Suspense 与异步组件 (了解即可)	241
5.58	4.58 第四章真正的完结语	242
5.59	4.59 深入：模板语法全速查 (写模板前对照)	242
5.59.1	4.59.1 插值	242

5.59.2	4.59.2 指令速查	242
5.59.3	4.59.3 事件修饰符	242
5.59.4	4.59.4 模板中的三目与空值处理	242
5.60	4.60 深入：样式绑定与 class 技巧	242
5.60.1	4.60.1 对象/数组语法	242
5.60.2	4.60.2 动态 class 与主题	242
5.61	4.61 深入：计算属性 vs 方法的区别（再强调）	243
5.62	4.62 深入：watch 的深度监听与 immediate	243
5.63	4.63 深入：自定义指令（了解）	243
5.64	4.64 深入：TypeScript 常用实用类型	243
5.65	4.65 前端章节补充自测（10 题）	243
5.66	4.66 深入：Pinia 的底层机制（为什么 store 是响应式的）	244
5.66.1	4.66.1 defineStore 内部做什么	244
5.66.2	4.66.2 store 实例是单例吗	244
5.66.3	4.66.3 什么时候需要多个实例	244
5.67	4.67 深入：组合式 API 的生命周期等价表	244
5.68	4.68 深入：模板编译的常见误区	244
5.69	4.69 深入：前端模块化与 import 规范	245
5.69.1	4.69.1 导入顺序约定	245
5.69.2	4.69.2 命名导出 vs 默认导出	245
5.69.3	4.69.3 循环依赖的坑	245
5.70	4.70 深入：ElMessage 与 ElMessageBox 的完整用法	245
5.70.1	4.70.1 消息（ElMessage）	245
5.70.2	4.70.2 确认框（ElMessageBox）	245
5.70.3	4.70.3 输入框（ElMessageBox.prompt，可选）	245
5.71	4.71 深入：ECharts 的按需引入 vs 全量	245
5.72	4.72 深入：表单校验的 trigger 时机	246
5.73	4.73 深入：04 章补充自测（追加 10 题）	246
5.74	4.74 深入：响应式系统的常见陷阱再盘点（实战级）	246
5.74.1	4.74.1 ref vs reactive 的选择	246
5.74.2	4.74.2 数组响应式的坑	246
5.74.3	4.74.3 watch 的 immediate 与 deep	246
5.75	4.75 深入：路由与页面缓存的配合	247
5.75.1	4.75.1 监控页切走再回来	247
5.75.2	4.75.2 keep-alive 的注意事项	247
5.76	4.76 深入：TypeScript 类型工具（进阶 8 个）	247
5.77	4.77 深入：泛型的实战（表格列泛型组件）	247
5.78	4.78 深入：异步组件与按需加载	247
5.79	4.79 深入：04 章最终综合自测（追加 10 题）	247
5.80	4.80 深入：项目实战——完整读一个页面（fw100 列表页）	248
5.80.1	4.80.1 页面骨架	248
5.80.2	4.80.2 拆解	248
5.80.3	4.80.3 页面的通用五步法	248
5.81	4.81 深入：TypeScript 类型体操（生成代码的配合）	248
5.81.1	4.81.1 从生成代码推导用法	248
5.81.2	4.81.2 联合类型与可选链	248
5.81.3	4.81.3 类型断言（慎用）	249

5.82	4.82 深入: computed 的依赖追踪细节	249
5.83	4.83 深入: CSS 与组件样式的实践	249
	5.83.1 4.83.1 scoped 的原理	249
	5.83.2 4.83.2 覆盖 Element Plus 样式	249
	5.83.3 4.83.3 全局样式位置	249
5.84	4.84 深入: 04 章终极自测 (5 题)	249
5.85	4.85 深入: Vue Router 的完整参考 (本项目路由)	249
	5.85.1 4.85.1 路由定义	249
	5.85.2 4.85.2 meta 的用途	250
	5.85.3 4.85.3 程式化导航	250
5.86	4.86 深入: Axios 封装的完整参考 (shared 的 httpClient)	250
	5.86.1 4.86.1 拦截器的作用	250
	5.86.2 4.86.2 401 处理的时机	250
	5.86.3 4.86.3 为什么 7 个应用共用	250
5.87	4.87 深入: 04 章实战自测 (8 题)	250
5.88	4.88 深入: WebSocket 前端的完整实现参考	251
	5.88.1 4.88.1 建立连接	251
	5.88.2 4.88.2 消息分发	251
	5.88.3 4.88.3 断线重连的要点	251
5.89	4.89 深入: Pinia store 的完整实现参考	251
	5.89.1 4.89.1 定义	251
	5.89.2 4.89.2 getters 带参数的写法	252
	5.89.3 4.89.3 store 跨应用复用	252
5.90	4.90 深入: 04 章高频自测 (8 题)	252
5.91	4.91 深入: 表格组件的完整实现参考 (高频组件)	252
	5.91.1 4.91.1 带分页表格	252
	5.91.2 4.91.2 插槽 (#default= "{ row }") 的用法	252
	5.91.3 4.91.3 排序与搜索	253
5.92	4.92 深入: 表单组件的完整实现参考	253
	5.92.1 4.92.1 带校验表单	253
	5.92.2 4.92.2 校验规则	253
	5.92.3 4.92.3 提交前校验	253
	5.92.4 4.92.4 编辑回填	253
5.93	4.93 深入: 04 章综合自测 (8 题)	253
5.94	4.94 深入: 项目常用工具函数的实现参考	253
	5.94.1 4.94.1 时间格式化	253
	5.94.2 4.94.2 数值格式化	254
	5.94.3 4.94.3 防抖与节流	254
	5.94.4 4.94.4 深拷贝与数组操作	254
5.95	4.95 深入: 响应式状态的组织模式 (项目级)	254
	5.95.1 4.95.1 单一数据源原则	254
	5.95.2 4.95.2 状态流转的控制	254
	5.95.3 4.95.3 页面交互的状态提升	255
5.96	4.96 深入: 项目级 TypeScript 配置解读	255
	5.96.1 4.96.1 tsconfig 常用项	255
	5.96.2 4.96.2 strict 模式的意义	255
	5.96.3 4.96.3 noEmit + vue-tsc	255

5.97	4.97 深入：04 章终局自测 (8 题)	255
5.98	4.98 深入：前端工程的完整构建流程解读	255
5.98.1	4.98.1 npm run build 做了什么	255
5.98.2	4.98.2 构建产物说明	256
5.98.3	4.98.3 构建失败的常见原因	256
5.99	4.99 深入：前端调试技巧	256
5.99.1	4.99.1 浏览器 DevTools	256
5.99.2	4.99.2 常见调试场景	256
5.99.3	4.99.3 调试打印	256
5.100	4.100 深入：04 章毕业自测 (8 题)	256
5.101	4.101 深入：常见需求的组件模式库	256
5.101.1	4.101.1 搜索 + 表格 + 分页标准页	256
5.101.2	4.101.2 弹窗表单标准模式	257
5.101.3	4.101.3 状态标签模式	257
5.102	4.102 深入：04 章大师自测 (8 题)	258
5.103	4.103 深入：响应式详情的十个坑 (项目踩过)	258
5.103.1	4.103.1 坑列表	258
5.103.2	4.103.2 典型案例：storeToRefs	258
5.103.3	4.103.3 典型案例：v-model 与 store	258
5.104	4.104 深入：模板语法的完整参考 (补充)	258
5.104.1	4.104.1 条件与循环	258
5.104.2	4.104.2 事件与修饰符	259
5.104.3	4.104.3 动态属性与插槽	259
5.104.4	4.104.4 动态 class	259
5.105	4.105 深入：04 章权威自测 (8 题)	259
5.106	4.106 深入：前端代码的组织哲学 (为什么这么分)	259
5.106.1	4.106.1 分层思想	259
5.106.2	4.106.2 依赖方向	259
5.106.3	4.106.3 组件设计原则	259
5.107	4.107 深入：性能优化的前端实践	259
5.107.1	4.107.1 渲染性能	259
5.107.2	4.107.2 加载性能	260
5.107.3	4.107.3 内存管理	260
5.108	4.108 深入：04 章权威自测 (8 题)	260
5.109	4.109 深入：前端开发环境配置 (Volar 使用指南)	260
5.109.1	4.109.1 Volar 的日常用法	260
5.109.2	4.109.2 常见问题	260
5.110	4.110 深入：前端代码审查清单 (提交前自查)	260
5.111	4.111 深入：04 章权威自测 (8 题)	260
6	05 前端逐应用精读	261
6.1	5.1 共享层 packages/shared (7 个应用的公共基石)	261
6.1.1	5.1.1 shared 目录全景	261
6.1.2	5.1.2 index.ts 导出清单	261
6.1.3	5.1.3 roles.ts 深度 (菜单与地址的唯一手写副本)	262
6.1.4	5.1.4 session.ts (token 与 WS URL)	262
6.1.5	5.1.5 api/custom-instance.ts (orval 的请求壳)	263
6.1.6	5.1.6 stores/auth.ts (认证 store 工厂)	263

6.1.7	5.1.7	template/AppNavbar.vue (690 行导航栏拆解)	263
6.1.8	5.1.8	template/LoginPage.vue (683 行登录页)	264
6.1.9	5.1.9	各应用如何消费 shared	264
6.2	5.2	admin 应用走读 (管理后台)	265
6.2.1	5.2.1	admin 文件结构	265
6.2.2	5.2.2	admin 的路由与权限	265
6.2.3	5.2.3	admin 的 api facade	265
6.2.4	5.2.4	admin 特有的业务点	266
6.3	5.3	fj200c_information 应用全文件走读 (核心示例)	266
6.3.1	5.3.1	应用结构	266
6.3.2	5.3.2	api/fj200c_information.ts (facade + WS 事件类型)	266
6.3.3	5.3.3	types.ts (WS 事件类型手写, 不走 orval)	267
6.3.4	5.3.4	composables/useFj200cInformationEvents.ts (WS 连接 + 分发)	267
6.3.5	5.3.5	composables/useService.ts (服务启停 + 状态轮询)	268
6.3.6	5.3.6	composables/useCommandChannel.ts (命令发送)	268
6.3.7	5.3.7	composables/useClock.ts (时钟)	268
6.3.8	5.3.8	composables/useConfigDialog.ts (配置对话框)	268
6.3.9	5.3.9	Monitor.vue (411 行主界面)	269
6.3.10	5.3.10	Visual / Data / Config / Help 子页	270
6.4	5.4	fj200c_main 应用走读 (最复杂的业务前端)	270
6.4.1	5.4.1	应用特点	270
6.4.2	5.4.2	useBackendPorts.ts (模块级单例 WS + 引用计数)	270
6.4.3	5.4.3	ScaledPage (1920×1080 大屏适配)	271
6.4.4	5.4.4	dashboard store (业务数据中枢)	271
6.4.5	5.4.5	双主题 CSS (航天/仪表)	272
6.4.6	5.4.6	GenerateReport (报表打印, a06a8b4 改为原生 print)	272
6.4.7	5.4.7	ECU 面板 (示例面板拆解)	272
6.5	5.5	ftj1c 应用走读 (UDP 通信监控)	273
6.5.1	5.5.1	应用特点	273
6.5.2	5.5.2	文件结构	273
6.5.3	5.5.3	IP 配置页 (核心功能)	273
6.5.4	5.5.4	帧监控页	273
6.5.5	5.5.5	坐标转换 (地图/坐标页)	274
6.6	5.6	fw100 / fw150 应用走读 (最简 CRUD)	274
6.6.1	5.6.1	为什么先读这两个	274
6.6.2	5.6.2	文件结构 (fw100 示例)	274
6.6.3	5.6.3	完整 CRUD 走读 (后端 + 前端对照)	274
6.6.4	5.6.4	fw150 与 fw100 的差异	275
6.6.5	5.6.5	台账页的实践价值	275
6.7	5.7	city3d 应用走读 (3D 城市展示)	275
6.7.1	5.7.1	应用特点	275
6.7.2	5.7.2	文件结构	275
6.7.3	5.7.3	useCityScene.ts (3D 场景生命周期)	275
6.7.4	5.7.4	useCityData.ts (轮询 + 场景数据刷新)	276
6.7.5	5.7.5	建筑/区域/事件管理页 (与 fw100 同构)	276
6.7.6	5.7.6	Overview.vue (聚合统计)	276

6.8	5.8 前端共性模式总结（7 个应用的共同点）	277
6.8.1	5.8.1 七应用统一骨架	277
6.8.2	5.8.2 七应用差异对照表	277
6.8.3	5.8.3 典型页面三型	277
6.8.4	5.8.4 新增前端页面的标准步骤	278
6.9	5.9 本章收官：前端七应用速查卡	278
6.10	5.10 admin 深入：CreateUser 页面（完整表单实战）	278
6.10.1	5.10.1 页面职责	278
6.10.2	5.10.2 校验规则与提交	279
6.10.3	5.10.3 编辑用户的差异	279
6.11	5.11 fj200c_information 深入：Visual.vue（实时曲线）	280
6.11.1	5.11.1 页面结构	280
6.11.2	5.11.2 曲线数据流	280
6.11.3	5.11.3 ECharts 实时曲线要点	280
6.12	5.12 fj200c_main 深入：三路面板与高级功能	280
6.12.1	5.12.1 三路串口面板对照	280
6.12.2	5.12.2 命令发送通道（与 fj200c_information 同构）	281
6.12.3	5.12.3 CSV 录制状态	281
6.12.4	5.12.4 试验信息与报表	281
6.12.5	5.12.5 仪表盘组件 GaugeCard	281
6.12.6	5.12.6 双主题切换的实现细节	282
6.13	5.13 七个应用 vite.config.ts 对照	282
6.13.1	5.13.1 相同骨架	282
6.13.2	5.13.2 七应用参数表	283
6.13.3	5.13.3 为什么 ws: true 只在三个应用开	283
6.14	5.14 七个应用 router 对照（守卫差异）	283
6.14.1	5.14.1 统一守卫模板	283
6.14.2	5.14.2 各应用路由表	284
6.14.3	5.14.3 登录后跳转逻辑（LoginPage）	284
6.15	5.15 前端“测试”现状与构建即检查	284
6.15.1	5.15.1 项目无单元测试框架（有意为之）	284
6.15.2	5.15.2 手动验证清单（改动前端后必做）	284
6.15.3	5.15.3 常见构建错误速查（前端）	285
6.16	5.16 shared 深入：types.ts 与 Permission 的 re-export	285
6.16.1	5.16.1 types.ts 做什么	285
6.16.2	5.16.2 Permission 字符串值的约定	285
6.16.3	5.16.3 其他 re-export 的类型	285
6.17	5.17 前端文件速查表（按需求定位文件）	286
6.17.1	5.17.1 改菜单/导航	286
6.17.2	5.17.2 改登录流程	286
6.17.3	5.17.3 改业务页面	286
6.17.4	5.17.4 改样式主题	286
6.18	5.18 实战场景演练（5 个真实需求）	286
6.18.1	场景 1：给 fw100 加一个“备注”字段	286
6.18.2	场景 2：导航栏加一个“系统公告”入口	286
6.18.3	场景 3：监控页加一个 WS 数据字段	287
6.18.4	场景 4：给 ftj1c 加第 17 路 IP 配置	287

6.18.5	场景 5: 新增一个权限 (如“数据导出”)	287
6.19	5.19 前端性能与体验细节 (7 应用通用)	287
6.19.1	5.19.1 表格性能	287
6.19.2	5.19.2 图表性能	287
6.19.3	5.19.3 WS 数据量控制	287
6.19.4	5.19.4 加载体验	288
6.20	5.20 前端最佳实践清单 (写代码前对照)	288
6.20.1	5.20.1 必做清单	288
6.20.2	5.20.2 禁做清单	288
6.21	5.21 七应用前端架构对照总结 (本日精华)	288
6.22	5.22 admin 深入: Dashboard 与系统信息	288
6.22.1	5.22.1 概览页结构	288
6.22.2	5.22.2 角色名展示 (注册表映射)	289
6.23	5.23 深入: 登录态与多应用跳转完整链路	289
6.23.1	5.23.1 链路总览	289
6.23.2	5.23.2 dev 模式跨端口的特殊处理	289
6.23.3	5.23.3 prod 模式为什么不需要	290
6.23.4	5.23.4 登录失效的传导	290
6.24	5.24 深入: Element Plus 组件级封装 (复用模式)	290
6.24.1	5.24.1 常见封装清单	290
6.24.2	5.24.2 封装的原则	290
6.25	5.25 各应用 index.html 与入口文件对照	290
6.25.1	5.25.1 index.html 模板	290
6.25.2	5.25.2 七应用标题对照	291
6.26	5.26 前端工具函数 (各应用 utils)	291
6.26.1	5.26.1 常见工具模块	291
6.26.2	5.26.2 工具函数示例	291
6.27	5.27 前端开发环境设置 (VS Code)	291
6.27.1	5.27.1 必备扩展	291
6.27.2	5.27.2 调试技巧	292
6.27.3	5.27.3 常见工作流	292
6.28	5.28 前端章节收官自测 (50 题精华 10 问)	292
6.29	5.29 深入: orval 生成的请求函数 (读懂 generated)	292
6.29.1	5.29.1 一个生成函数的解剖	292
6.29.2	5.29.2 带参数与请求体的函数	292
6.29.3	5.29.3 生成文件的目录组织	293
6.29.4	5.29.4 为什么不能手改 generated	293
6.30	5.30 深入: WS 事件三种使用模式对照	293
6.30.1	5.30.1 模式一: 组件级连接 (fj200c_information)	293
6.30.2	5.30.2 模式二: 模块级单例 (fj200c_main)	293
6.30.3	5.30.3 模式三: 事件总线解耦 (跨模块)	293
6.30.4	5.30.4 三模式选择指南	293
6.31	5.31 深入: 前端错误处理全景 (续 4.47)	294
6.31.1	5.31.1 错误分层与兜底	294
6.31.2	5.31.2 后端错误消息的前端呈现	294
6.31.3	5.31.3 loading/disabled 防止重复提交	294
6.32	5.32 深入: 前端应用复制改造指南 (fw150 实例复盘)	294

6.32.1	5.32.1 fw150 是怎么来的	294
6.32.2	5.32.2 复制改造的通用清单（新应用模板）	295
6.32.3	5.32.3 复制改造的坑	295
6.33	5.33 前端章节全量速查表（收藏级）	295
6.33.1	5.33.1 按“症状”找文件	295
6.33.2	5.33.2 按“改动”找文件	295
6.33.3	5.33.3 前端学习路径推荐	296
6.34	5.34 深入：AppNavbar 的完整实现（690 行拆解）	296
6.34.1	5.34.1 顶层结构	296
6.34.2	5.34.2 菜单数据装配	296
6.34.3	5.34.3 当前高亮	296
6.34.4	5.34.4 用户菜单与退出	296
6.35	5.35 深入：Monitor 页全链路时序（请求→WS→渲染）	296
6.36	5.36 深入：Element Plus 国际化与语言	297
6.36.1	5.36.1 中文配置	297
6.36.2	5.36.2 项目内文字硬编码	297
6.37	5.37 深入：刷新页面状态恢复	297
6.37.1	5.37.1 刷新丢什么、留什么	297
6.37.2	5.37.2 为什么业务数据不持久化	297
6.38	5.38 深入：前端问题定位流程（故障排查指南）	298
6.38.1	5.38.1 页面/功能异常排查路径	298
6.38.2	5.38.2 记录问题信息（给协作方看）	298
6.39	5.39 终极实战：给 ftj1c 加“帧搜索”功能（完整实操）	298
6.39.1	需求	298
6.39.2	步骤 1：模板加搜索框	298
6.39.3	步骤 2：computed 过滤	298
6.39.4	步骤 3：表格换数据源	298
6.39.5	步骤 4：验证	298
6.40	5.40 05 章结束语	299
6.41	5.41 深入：前端与后端接口全对照（以应用为纲）	299
6.41.1	5.41.1 fj200c_information 接口对照表	299
6.41.2	5.41.2 fj200c_main 接口对照表	300
6.41.3	5.41.3 ftj1c 接口对照表	300
6.41.4	5.41.4 city3d 接口对照表（14 个）	300
6.42	5.42 深入：前端安全细节	300
6.42.1	5.42.1 安全边界认知	300
6.42.2	5.42.2 前端安全实践清单	301
6.42.3	5.42.3 v-html 的使用纪律	301
6.43	5.43 深入：前端配置管理三种模式	301
6.43.1	5.43.1 配置存储三处对照	301
6.43.2	5.43.2 前端如何提示“生效时机”	301
6.44	5.44 深入：七应用 index.html / package.json 对照	302
6.44.1	5.44.1 package.json 依赖差异	302
6.44.2	5.44.2 依赖版本管理	302
6.45	5.45 深入：前端常用组件模式汇总（写新组件参考）	302
6.45.1	5.45.1 组件分类与模板	302
6.45.2	5.45.2 展示型组件标准模板	303

6.45.3	5.45.3 表单型组件标准模板	303
6.46	5.46 深入：前端构建产物与部署形态	303
6.46.1	5.46.1 构建输出	303
6.46.2	5.46.2 部署到后端 (embedded 模式)	303
6.46.3	5.46.3 深链接回退的意义	303
6.47	5.47 05 章最终自测 (30 题随机抽 10)	303
6.48	5.48 深入：composable vs store (状态管理的选择)	304
6.48.1	5.48.1 什么时候用哪个	304
6.48.2	5.48.2 两者结合的典型形态	304
6.49	5.49 深入：TypeScript 类型安全实践 (as 的使用纪律)	304
6.49.1	5.49.1 项目中的类型断言用法	304
6.49.2	5.49.2 应避免的 as	304
6.50	5.50 七应用菜单与布局一览	305
6.50.1	5.50.1 菜单结构 (MENU_CONFIG 现行)	305
6.50.2	5.50.2 布局形态	305
6.51	5.51 前端高频疑问解答 (FAQ)	305
6.51.1	Q1: 为什么有的页面请求 404?	305
6.51.2	Q2: 为什么 WS 连不上?	305
6.51.3	Q3: 为什么改了 shared 代码没生效?	305
6.51.4	Q4: 为什么 npm run build 报一堆类型错?	306
6.51.5	Q5: 怎么知道接口有哪些参数/返回值?	306
6.52	5.52 05 章完结：学习成果清单	306
6.53	5.53 深入：组件通信模式在项目中的真实分布	306
6.53.1	5.53.1 通信方式统计与典型场景	306
6.53.2	5.53.2 props 设计原则 (GaugeCard 案例)	306
6.54	5.54 深入：前端节流/防抖/限流应用清单	307
6.54.1	5.54.1 全项目用到的限流手段	307
6.54.2	5.54.2 何时用防抖 vs 节流	307
6.55	5.55 深入：响应式与样式的细节规范	307
6.55.1	5.55.1 移动端适配的三个应用	307
6.55.2	5.55.2 CSS 变量分层	307
6.55.3	5.55.3 scoped 与全局样式的边界	308
6.56	5.56 05 章收官：终极综合自测	308
6.56.1	场景题：给 fj200c_information 加“历史数据回放”页	308
6.57	5.57 补充：跨应用细节——会话与跳转机制再梳理	309
6.57.1	5.57.1 登录态的三个环节	309
6.57.2	5.57.2 dev 与 prod 的差异 (重点)	309
6.57.3	5.57.3 跳转后如何回到原页面	309
6.58	5.58 补充：各应用「登录页」的差异	309
6.59	5.59 补充：前端「主题/暗色」全梳理	309
6.60	5.60 补充：前端数据流异常排查清单	310
6.61	5.61 补充：从零读一个陌生前端应用的五步法	310
6.62	5.62 补充：05 章知识自测 (追加 10 题)	310
6.63	5.63 补充：shared 里认证流程的完整时序	310
6.63.1	5.63.1 initAuth 细节	310
6.63.2	5.63.2 logout 做了什么	311
6.63.3	5.63.3 token 过期的时间线	311

6.64	5.64 补充: Monitor 页的表格渲染性能细节	311
6.64.1	5.64.1 数据更新策略	311
6.64.2	5.64.2 限长策略	311
6.64.3	5.64.3 何时需要虚拟滚动	311
6.65	5.65 补充: fj200c_main 三路面板的状态共享细节	311
6.65.1	5.65.1 store 与面板的关系	311
6.65.2	5.65.2 为什么用 store 而不是 props	311
6.66	5.66 补充: ftj1c 前端与后端的数据交互细节	311
6.66.1	5.66.1 两种数据通道	311
6.66.2	5.66.2 帧表格的更新频率	311
6.66.3	5.66.3 坐标转换的展示	311
6.67	5.67 补充: fw100/fw150 的响应式细节	312
6.67.1	5.67.1 useResponsive 的作用	312
6.67.2	5.67.2 移动端适配的取舍	312
6.68	5.68 补充: city3d 的 3D 与 2D 页面混合	312
6.68.1	5.68.1 页面关系	312
6.68.2	5.68.2 3D 场景与数据的联动	312
6.69	5.69 补充: 05 章补充自测 (追加 10 题)	312
6.70	5.70 深入: Vite 配置文件的完整解读	312
6.70.1	5.70.1 全量注释版	312
6.70.2	5.70.2 关键点逐条	313
6.71	5.71 深入: 生产环境前端如何被服务	313
6.71.1	5.71.1 两种模式	313
6.71.2	5.71.2 embedded 的服务方式	313
6.71.3	5.71.3 为什么能“改前端不用重启后端”(开发)	313
6.72	5.72 深入: 多应用共享登录态的边界	313
6.72.1	5.72.1 共享的前提	313
6.72.2	5.72.2 跨应用跳转的实现	313
6.72.3	5.72.3 登录态校验	313
6.73	5.73 深入: 状态管理的分层 (store 该放什么)	314
6.74	5.74 深入: composable 设计模式 (useXxx 的提炼)	314
6.74.1	5.74.1 什么时候抽 composable	314
6.74.2	5.74.2 典型结构	314
6.74.3	5.74.3 命名约定	314
6.75	5.75 深入: 样式方案与布局细节	314
6.75.1	5.75.1 样式方案对比	314
6.75.2	5.75.2 布局骨架	314
6.75.3	5.75.3 暗色模式 (如实现)	314
6.76	5.76 深入: 05 章最终综合自测 (追加 10 题)	314
6.77	5.77 深入: 项目实战——完整读一个监控页 (fj200c_information 的 Monitor)	315
6.77.1	5.77.1 页面组件树	315
6.77.2	5.77.2 数据流	315
6.77.3	5.77.3 关键代码模式	315
6.77.4	5.77.4 监控页与台账页的本质区别	315
6.78	5.78 深入: 7 个应用的 api facade 设计	315
6.78.1	5.78.1 统一模式	315

6.78.2	5.78.2 facade 的价值	316
6.78.3	5.78.3 为什么要 setApilnstance	316
6.79	5.79 深入：路由守卫的完整时序	316
6.80	5.80 深入：05 章终极自测（5 题）	316
6.81	5.81 深入：每个应用的核心配置文件速查	316
6.81.1	5.81.1 vite.config.ts 要点（7 应用对照）	316
6.81.2	5.81.2 package.json 脚本	316
6.81.3	5.81.3 tsconfig 要点	317
6.82	5.82 深入：Element Plus 的全局配置	317
6.82.1	5.82.1 main.ts 的挂载方式	317
6.82.2	5.82.2 项目现状	317
6.82.3	5.82.3 常用组件的使用清单	317
6.83	5.83 深入：ECharts 在监控页的配置参考	317
6.83.1	5.83.1 折线图配置	317
6.83.2	5.83.2 高频更新的优化	317
6.84	5.84 深入：05 章实战自测（8 题）	318
6.85	5.85 深入：admin 应用的完整实现参考（最典型的 CRUD 应用）	318
6.85.1	5.85.1 页面清单与路由	318
6.85.2	5.85.2 用户列表页要点	318
6.85.3	5.85.3 用户编辑页要点	318
6.85.4	5.85.4 与业务应用的差异	318
6.86	5.86 深入：login 页的完整实现参考	318
6.86.1	5.86.1 表单	318
6.86.2	5.86.2 提交逻辑	318
6.86.3	5.86.3 登录后的跳转	319
6.87	5.87 深入：导航栏与布局的完整实现参考	319
6.87.1	5.87.1 布局结构	319
6.87.2	5.87.2 用户菜单	319
6.87.3	5.87.3 应用切换	319
6.88	5.88 深入：05 章综合自测（8 题）	319
6.89	5.89 深入：fj200c_information 的完整页面走读（以仪表盘为例）	319
6.89.1	5.89.1 仪表盘的数据来源	319
6.89.2	5.89.2 仪表卡片的实现	319
6.89.3	5.89.3 仪表盘的布局	320
6.89.4	5.89.4 仪表盘的边界处理	320
6.90	5.90 深入：数据表格与图表页的配合	320
6.90.1	5.90.1 表格与图表共用数据	320
6.90.2	5.90.2 图表的滚动窗口	320
6.90.3	5.90.3 暂停/恢复	320
6.91	5.91 深入：CSV 页面的完整实现参考	320
6.91.1	5.91.1 文件列表	320
6.91.2	5.91.2 下载实现	321
6.91.3	5.91.3 删除/归档	321
6.92	5.92 深入：配置页面的完整实现参考	321
6.92.1	5.92.1 读取与展示	321
6.92.2	5.92.2 修改与保存	321
6.92.3	5.92.3 校验与提示	321

6.93	5.93 深入：05 章终局自测（8 题）	321
6.94	5.94 深入：city3d 的完整实现走读	321
6.94.1	5.94.1 技术栈	321
6.94.2	5.94.2 3D 场景的初始化	321
6.94.3	5.94.3 建筑的可视化	322
6.94.4	5.94.4 2D 与 3D 的协作	322
6.95	5.95 深入：ftj1c 的帧数据展示细节	322
6.95.1	5.95.1 表格列设计	322
6.95.2	5.95.2 坐标字段的展示	322
6.95.3	5.95.3 数据筛选	322
6.96	5.96 深入：fw100 与 fw150 的差异对比	322
6.96.1	5.96.1 相同点	322
6.96.2	5.96.2 不同点	322
6.96.3	5.96.3 复制新台账的模板价值	322
6.97	5.97 深入：05 章毕业自测（8 题）	322
6.98	5.98 深入：一个完整应用从头构建的步骤（模板复用法）	323
6.98.1	5.98.1 复制模板	323
6.98.2	5.98.2 修改清单	323
6.98.3	5.98.3 验证	323
6.98.4	5.98.4 常见翻车点	323
6.99	5.99 深入：监控类应用的共同骨架	323
6.99.1	5.99.1 骨架五件	323
6.99.2	5.99.2 三应用的差异点	323
6.99.3	5.99.3 新增监控类的模板	323
6.100	5.100 深入：05 章大师自测（8 题）	323
6.101	5.101 深入：7 个应用的异常处理策略	324
6.101.1	5.101.1 统一错误处理（axios 拦截器）	324
6.101.2	5.101.2 业务异常 vs 技术异常	324
6.101.3	5.101.3 WS 异常处理	324
6.101.4	5.101.4 页面级兜底	324
6.102	5.102 深入：跨应用跳转与登录态的完整细节	324
6.102.1	5.102.1 跳转的时机	324
6.102.2	5.102.2 跳转方式	324
6.102.3	5.102.3 登录态的传递	324
6.102.4	5.102.4 跳转后的自动登录	324
6.103	5.103 深入：05 章权威自测（8 题）	325
6.104	5.104 深入：状态栏与页面交互的完整设计	325
6.104.1	5.104.1 状态栏的构成	325
6.104.2	5.104.2 状态栏的更新机制	325
6.104.3	5.104.3 按钮的联动	325
6.105	5.105 深入：CSV 数据的可视化分析（进阶用法）	325
6.105.1	5.105.1 下载后分析	325
6.105.2	5.105.2 代码分析（Python 示例）	325
6.105.3	5.105.3 系统内分析（08 章案例七）	325
6.106	5.106 深入：05 章权威自测（8 题）	325
6.107	5.107 深入：7 个应用的项目配置对照（深入版）	326
6.107.1	5.107.1 index.html 的差异	326

6.107.2	5.107.2 依赖差异	326
6.107.3	5.107.3 构建体积参考	326
6.108	5.108 深入：真实排错场景演练（前端）	326
6.108.1	5.108.1 场景：登录后白屏	326
6.108.2	5.108.2 场景：WS 连不上	326
6.108.3	5.108.3 场景：表格数据不更新	326
6.109	5.109 深入：05 章权威自测（8 题）	326
7	06 前后端类型同步机制（utoipa + OpenAPI + orval）	327
7.1	6.1 问题：没有类型同步的世界	327
7.1.1	6.1.1 手写两端的痛苦	327
7.1.2	6.1.2 本项目的解法	327
7.2	6.2 链路总览（一条命令全同步）	327
7.3	6.3 utoipa 注解详解（后端侧）	328
7.3.1	6.3.1 结构体注解（DTO）	328
7.3.2	6.3.2 handler 注解（API 路径）	328
7.3.3	6.3.3 带参数的 handler 注解	329
7.3.4	6.3.4 统一响应包装的注解方式	329
7.4	6.4 src/api_docs.rs（OpenAPI 聚合中心）	330
7.4.1	6.4.1 文件职责	330
7.4.2	6.4.2 export_openapi 测试（防漂移断言）	330
7.4.3	6.4.3 断言的意义（防漂移）	331
7.5	6.5 orval.config.ts 详解（生成侧）	331
7.5.1	6.5.1 配置文件	331
7.5.2	6.5.2 配置项含义	331
7.6	6.6 npm run gen:api 完整流程演示	331
7.6.1	6.6.1 命令定义	331
7.6.2	6.6.2 一次典型执行	332
7.6.3	6.6.3 执行后的检查清单	332
7.7	6.7 generated 产物解读	332
7.7.1	6.7.1 类型文件示例	332
7.7.2	6.7.2 请求函数文件示例	332
7.7.3	6.7.3 与 facade 的衔接	332
7.8	6.8 修改 DTO 后的全流程演练	333
7.8.1	场景：给台账加“位置”字段	333
7.9	6.9 WebSocket 为什么不进 OpenAPI	333
7.9.1	6.9.1 OpenAPI 的边界	333
7.9.2	6.9.2 项目实践	333
7.9.3	6.9.3 WS 类型手写的维护注意	333
7.10	6.10 常见问题与维护指南	334
7.10.1	6.10.1 报错速查	334
7.10.2	6.10.2 日常维护清单	334
7.10.3	6.10.3 为什么 ts-rs 方案被替换	334
7.10.4	6.10.4 运行时文档（Swagger UI）	334
7.11	6.11 本章自测	334
7.12	6.12 utoipa 注解完整参考（抄写即用）	335
7.12.1	6.12.1 #[utoipa::path] 属性全表	335
7.12.2	6.12.2 完整模板（复制修改）	335

7.12.3	6.12.3 路径参数 + 查询参数混合	335
7.13	6.13 深入: ApiResponse 泛型与前端处理	336
7.13.1	6.13.1 Rust 侧定义	336
7.13.2	6.13.2 OpenAPI 如何表示泛型	336
7.13.3	6.13.3 orval 生成后	336
7.14	6.14 深入: Enum 与 Option 的生成细节	336
7.14.1	6.14.1 Rust enum → TS	336
7.14.2	6.14.2 Option 的两种形态	337
7.14.3	6.14.3 数字精度提示	337
7.15	6.15 深入: 复杂嵌套类型	337
7.15.1	6.15.1 Vec / 数组	337
7.15.2	6.15.2 HashMap	337
7.15.3	6.15.3 嵌套结构体	337
7.16	6.16 深入: 分页接口完整实战 (city3d 案例)	337
7.16.1	6.16.1 后端	337
7.16.2	6.16.2 前端调用	338
7.17	6.17 orval 高级配置 (按需定制)	338
7.17.1	6.17.1 参数命名覆盖	338
7.17.2	6.17.2 类型命名规范	338
7.17.3	6.17.3 何时需要动 orval 配置	338
7.18	6.18 类型同步机制的演进与边界	339
7.18.1	6.18.1 当前链路的能力边界	339
7.18.2	6.18.2 未来可改进方向 (如需要)	339
7.18.3	6.18.3 本章总结 (一张图)	339
7.19	6.19 最终自测 (追加题)	339
7.20	6.20 完整实例: 新增“设备维护记录”模块 (类型链路视角)	339
7.20.1	步骤 1: Rust DTO	340
7.20.2	步骤 2: handler 注解	340
7.20.3	步骤 3: api_docs.rs 登记	340
7.20.4	步骤 4: 跑 gen:api	341
7.20.5	步骤 5: 检查生成	341
7.20.6	步骤 6: 前端 facade + 页面	341
7.20.7	步骤 7: 验证	341
7.21	6.21 深入: git diff 读生成产物 (改了什么一目了然)	341
7.21.1	6.21.1 改 DTO 字段后的 diff	341
7.21.2	6.21.2 改接口签名后的 diff	341
7.21.3	6.21.3 diff 检查习惯	342
7.22	6.22 类型同步方案对比 (为什么选这套)	342
7.23	6.23 06 章完结: 知识串联	342
7.24	6.24 深入: openapi.json 结构速览	342
7.24.1	6.24.1 JSON 顶层结构	342
7.24.2	6.24.2 怎么看	343
7.25	6.25 常见 utoipa 报错与修复	343
7.25.1	6.25.1 一个实战修复案例	343
7.26	6.26 06 章最终自测 (全集)	343
7.27	6.27 深入: 类型同步的日常三问	344
7.27.1	6.27.1 “我改了后端, 前端要做什么?”	344

7.27.2	6.27.2 “为什么有时候 gen:api 后生成的函数少了?”	344
7.27.3	6.27.3 “这个类型是手写还是生成的?”	344
7.28	6.28 深入: OpenAPI 生成的历史与演进	344
7.28.1	6.28.1 项目类型方案的演进	344
7.28.2	6.28.2 为什么 orval 而不是其他生成器	345
7.28.3	6.28.3 未来可能的演进	345
7.29	6.29 深入: TypeScript 类型与 Rust 类型的边界	345
7.29.1	6.29.1 不会自动同步的边界	345
7.29.2	6.29.2 常见坑	345
7.29.3	6.29.3 防御性编码习惯	345
7.30	6.30 06 章终极综合题 (真实工作场景)	345
7.30.1	场景: 把 fw100 的 updated_at 从可选改为必填	345
7.31	6.31 06 章结束语	346
7.32	6.32 深入: OpenAPI 文件实操 (用 json 文件做接口地图)	346
7.32.1	6.32.1 用 PowerShell 快速查接口	346
7.32.2	6.32.2 用 jq (如装了)	346
7.33	6.33 深入: 如何调试“生成后类型不对”	346
7.33.1	6.33.1 常见问题与排查	346
7.33.2	6.33.2 调试链	346
7.34	6.34 06 章最终检验: 动手任务	347
7.34.1	任务 1: 接口普查 (30 分钟)	347
7.34.2	任务 2: 契约变更演练 (30 分钟)	347
7.34.3	任务 3: 新增接口全流程 (60 分钟)	347
7.35	6.35 深入: orval 生成代码的维护经验	347
7.35.1	6.35.1 生成文件版本管理	347
7.35.2	6.35.2 生成后必做的三件事	347
7.35.3	6.35.3 常用排查命令	347
7.36	6.36 深入: utd 常见场景速查 (Rust 注解写错时的指引)	347
7.37	6.37 深入: OpenAPI 版本与 utoipa 版本的注意事项	348
7.38	6.38 06 章完结补充自测 (5 题)	348
7.39	6.39 深入: Swagger UI 的对接方式 (可视化调试)	348
7.39.1	6.39.1 项目现状	348
7.39.2	6.39.2 加一个 Swagger UI 页面 (可选增强)	348
7.40	6.40 深入: orval 与 axios 的交互细节	348
7.40.1	6.40.1 请求参数如何传递	348
7.40.2	6.40.2 响应解包	348
7.40.3	6.40.3 超时与重试	348
7.41	6.41 深入: 契约变更的团队协作流程	348
7.41.1	6.41.1 变更前的沟通	348
7.41.2	6.41.2 变更后的同步节奏	349
7.41.3	6.41.3 破坏性变更的处理	349
7.42	6.42 深入: 06 章补充自测 (追加 5 题)	349
7.43	6.43 深入: 从零看一次完整的契约变更 (实战演练)	349
7.43.1	6.43.1 场景	349
7.43.2	6.43.2 后端改动	349
7.43.3	6.43.3 执行 gen:api	349
7.43.4	6.43.4 前端验证	349

7.43.5	6.43.5 检查点	349
7.44	6.44 深入: OpenAPI 文档的阅读技巧	350
7.44.1	6.44.1 从 JSON 里快速找接口	350
7.44.2	6.44.2 常用字段含义	350
7.44.3	6.44.3 验证 schema 与代码一致	350
7.45	6.45 深入: 类型同步失败时的排查清单	350
7.46	6.46 深入: 06 章最终综合自测 (追加 5 题)	350
7.47	6.47 深入: DTO 设计的最佳实践 (类型同步的前提)	350
7.47.1	6.47.1 请求 DTO vs 响应 DTO	350
7.47.2	6.47.2 字段命名规范	351
7.47.3	6.47.3 时间字段的序列化	351
7.48	6.48 深入: WebSocket 事件的手写类型约定	351
7.48.1	6.48.1 为什么手写	351
7.48.2	6.48.2 手写模板	351
7.48.3	6.48.3 保持同步的检查点	351
7.49	6.49 深入: 06 章实战自测 (5 题)	351
7.50	6.50 深入: OpenAPI 生成流程的故障排查演练	351
7.50.1	6.50.1 场景一: gen:api 后前端函数没有出现	351
7.50.2	6.50.2 场景二: 生成的类型与后端不一致	352
7.50.3	6.50.3 场景三: vue-tsc 大量报错	352
7.51	6.51 深入: 类型同步的自动化与检查点	352
7.51.1	6.51.1 项目内的自动化	352
7.51.2	6.51.2 手动检查点 (何时该跑 gen:api)	352
7.51.3	6.51.3 生成文件的管理纪律	352
7.52	6.52 深入: 06 章高频自测 (8 题)	352
7.53	6.53 深入: utopia 注解的完整参考表	352
7.53.1	6.53.1 常用属性一览	352
7.53.2	6.53.2 operation_id 的命名规范	353
7.53.3	6.53.3 params 的使用	353
7.54	6.54 深入: DTO 的校验与默认值	353
7.54.1	6.54.1 服务端校验	353
7.54.2	6.54.2 前端校验与服务端校验	353
7.54.3	6.54.3 默认值的约定	353
7.55	6.55 深入: 类型契约的演进记录 (项目复盘)	353
7.55.1	6.55.1 从 ts-rs 到 utoipa	353
7.55.2	6.55.2 升级的好处	353
7.55.3	6.55.3 升级的教训	353
7.56	6.56 深入: 06 章终局自测 (8 题)	354
7.57	6.57 深入: orval 生成代码的完整解读	354
7.57.1	6.57.1 生成文件结构	354
7.57.2	6.57.2 生成的请求函数长什么样	354
7.57.3	6.57.3 生成的模型类型	354
7.57.4	6.57.4 使用约定	354
7.58	6.58 深入: 契约设计的常见决策	354
7.58.1	6.58.1 接口粒度的选择	354
7.58.2	6.58.2 批量操作的设计	355
7.58.3	6.58.3 错误响应的设计	355

7.59	6.59 深入：06 章毕业自测（8 题）	355
7.60	6.60 深入：共享类型设计的最佳实践	355
	7.60.1 6.60.1 什么时候放 shared	355
	7.60.2 6.60.2 什么时候不放 shared	355
	7.60.3 6.60.3 shared 的结构	355
7.61	6.61 深入：类型重构的完整流程（改名/迁移）	355
	7.61.1 6.61.1 场景	355
	7.61.2 6.61.2 步骤	355
	7.61.3 6.61.3 重命名的风险	356
7.62	6.62 深入：06 章大师自测（8 题）	356
7.63	6.63 深入：契约变更的完整场景练习	356
	7.63.1 6.63.1 场景一：加查询参数	356
	7.63.2 6.63.2 场景二：拆字段	356
	7.63.3 6.63.3 场景三：接口改名	356
	7.63.4 6.63.4 变更决策表	356
7.64	6.64 深入：06 章权威自测（8 题）	357
7.65	6.65 深入：从 OpenAPI 到页面渲染的完整链路（收官复盘）	357
	7.65.1 6.65.1 链路的关键点	357
	7.65.2 6.65.2 如果链路断了	357
7.66	6.66 深入：06 章权威自测（8 题）	357
7.67	6.67 深入：类型系统的边界案例	357
	7.67.1 6.67.1 生成代码覆盖不了什么	357
	7.67.2 6.67.2 手写与生成的分界	358
	7.67.3 6.67.3 处理边界的原则	358
7.68	6.68 深入：06 章权威自测（8 题）	358
7.69	6.69 深入：契约的版本管理策略	358
	7.69.1 6.69.1 契约文件的版本控制	358
	7.69.2 6.69.2 契约变更的提交规范	358
	7.69.3 6.69.3 契约评审要点	358
7.70	6.70 深入：06 章权威自测（8 题）	358
7.71	6.71 深入：本章收尾——契约思维总结	359
	7.71.1 6.71.1 契约思维的核心	359
	7.71.2 6.71.2 一个判断标准	359
	7.71.3 6.71.3 学习小结	359
7.72	6.72 深入：06 章最终自测（6 题）	359
8	07 使用与维护手册	359
8.1	7.1 环境准备（第一次接手项目）	359
	8.1.1 7.1.1 所需工具	359
	8.1.2 7.1.2 安装检查	360
	8.1.3 7.1.3 克隆与依赖安装	360
	8.1.4 7.1.4 国内加速（可选）	360
8.2	7.2 启动开发环境	360
	8.2.1 7.2.1 启动后端	360
	8.2.2 7.2.2 启动前端（任一应用）	360
	8.2.3 7.2.3 七个应用一起跑（多终端）	361
	8.2.4 7.2.4 开发环境架构	361
8.3	7.3 日常开发 workflow	361

8.3.1	7.3.1 改后端 → 验证	361
8.3.2	7.3.2 改前端 → 验证	362
8.3.3	7.3.3 改 DTO/接口 → 全链路	362
8.3.4	7.3.4 提交规范	362
8.3.5	7.3.5 开发常用命令速查	362
8.4	7.4 数据库管理	362
8.4.1	7.4.1 数据库位置	362
8.4.2	7.4.2 表结构 (database.rs 建表)	362
8.4.3	7.4.3 查看与备份	362
8.4.4	7.4.4 重置数据库	363
8.4.5	7.4.5 WAL 模式说明	363
8.5	7.5 配置文件详解	363
8.5.1	7.5.1 .env (后端运行时配置)	363
8.5.2	7.5.2 config-fj200c_information.ini (发动机监控)	363
8.5.3	7.5.3 config-fj200c_main.ini (发动机测控)	363
8.5.4	7.5.4 config-ftj1c.ini (UDP 通信)	364
8.5.5	7.5.5 配置修改流程总结	364
8.6	7.6 部署发布 (deploy.bat 详解)	364
8.6.1	7.6.1 一键部署	364
8.6.2	7.6.2 deploy.bat 的 8 步 (AGENTS.md 记载)	364
8.6.3	7.6.3 部署产物	364
8.6.4	7.6.4 部署后访问	364
8.6.5	7.6.5 部署注意事项	365
8.7	7.7 用户与权限管理实操	365
8.7.1	7.7.1 管理用户	365
8.7.2	7.7.2 权限与应用的对应	365
8.7.3	7.7.3 重置密码	365
8.8	7.8 故障排查手册 (症状 → 原因 → 解决)	366
8.8.1	7.8.1 后端启动失败	366
8.8.2	7.8.2 前端启动失败	366
8.8.3	7.8.3 登录问题	366
8.8.4	7.8.4 数据问题	366
8.8.5	7.8.5 日志定位	366
8.9	7.9 性能与容量管理	367
8.9.1	7.9.1 资源占用评估	367
8.9.2	7.9.2 容量关注点	367
8.9.3	7.9.3 性能调优开关	367
8.10	7.10 备份与迁移	367
8.10.1	7.10.1 备份清单	367
8.10.2	7.10.2 迁移到新机器	367
8.10.3	7.10.3 版本升级	367
8.11	7.11 安全加固清单	368
8.11.1	7.11.1 必须做的	368
8.11.2	7.11.2 可选加固	368
8.12	7.12 日常维护清单 (每周/每月)	368
8.12.1	每周	368
8.12.2	每月	368

8.12.3	版本发布流程	368
8.13	7.13 本章自测	368
8.14	7.14 深入: deploy.bat 逐段解读	369
8.14.1	7.14.1 整体骨架	369
8.14.2	7.14.2 关键点解读	369
8.14.3	7.14.3 自定义部署变体	370
8.15	7.15 深入: 日志系统	370
8.15.1	7.15.1 日志级别	370
8.15.2	7.15.2 设置方式	370
8.15.3	7.15.3 日志输出到文件	370
8.16	7.16 深入: 服务自启动 (无人值守运行)	370
8.16.1	7.16.1 任务计划程序 (Windows)	370
8.16.2	7.16.2 简单方式 (启动文件夹)	370
8.16.3	7.16.3 崩溃自恢复 (可选)	370
8.17	7.17 深入: 故障排查演练 (5 个真实场景)	371
8.17.1	场景 A: 部署后页面 404	371
8.17.2	场景 B: 发动机监控启动服务即失败	371
8.17.3	场景 C: CSV 录制没有文件	371
8.17.4	场景 D: 前端构建报大量类型错误	371
8.17.5	场景 E: 登录后立即被登出	371
8.18	7.18 深入: 数据库详细结构	371
8.18.1	7.18.1 users 表	371
8.18.2	7.18.2 其他表 (按模块)	372
8.18.3	7.18.3 global_vars 表 (fj200c_main 主题)	372
8.19	7.19 深入: 监控与告警 (可选增强)	372
8.19.1	7.19.1 现有监控手段	372
8.19.2	7.19.2 可扩展的告警 (脚本级)	372
8.20	7.20 使用维护 FAQ	372
8.21	7.21 本章自测 (追加)	372
8.22	7.22 深入: 开发环境多开与资源管理	373
8.22.1	7.22.1 8 个进程的资源占用	373
8.22.2	7.22.2 只需部分应用时的策略	373
8.22.3	7.22.3 关闭与清理	373
8.23	7.23 深入: 版本管理策略 (Git 实践)	373
8.23.1	7.23.1 分支模型 (建议)	373
8.23.2	7.23.2 发布打 tag	373
8.23.3	7.23.3 回滚 (发布后出问题)	374
8.24	7.24 深入: 后端运维操作	374
8.24.1	7.24.1 手动加用户 (脚本)	374
8.24.2	7.24.2 修改角色	374
8.24.3	7.24.3 禁用用户	374
8.25	7.25 深入: CSV 数据处理	374
8.25.1	7.25.1 CSV 文件格式	374
8.25.2	7.25.2 数据分析	374
8.25.3	7.25.3 数据归档策略	374
8.26	7.26 深入: 配置与环境的组合场景	375
8.27	7.27 深入: 健康检查与重启流程	375

8.27.1	7.27.1 健康检查	375
8.27.2	7.27.2 标准重启流程	375
8.28	7.28 07 章收官：维护者能力清单	375
8.29	7.29 深入：前后端联调实践	376
8.29.1	7.29.1 联调准备	376
8.29.2	7.29.2 联调流程	376
8.29.3	7.29.3 联调利器	376
8.30	7.30 深入：多人协作注意事项	376
8.30.1	7.30.1 协作分工建议	376
8.30.2	7.30.2 冲突避免	376
8.30.3	7.30.3 提交纪律	376
8.31	7.31 深入：常见部署环境	377
8.31.1	7.31.1 客户端单机（最常见）	377
8.31.2	7.31.2 服务器（内网多用户）	377
8.31.3	7.31.3 虚拟机/测试环境	377
8.32	7.32 深入：性能基准与优化实验	377
8.32.1	7.32.1 基准指标（参考）	377
8.32.2	7.32.2 常见优化实验	377
8.33	7.33 07 章最终自测（全集）	377
8.34	7.34 深入：构建模式与产物形态全梳理	378
8.34.1	7.34.1 后端两种构建模式	378
8.34.2	7.34.2 dist-* 目录（dev 静态托管）	378
8.34.3	7.34.3 前端产物对照	378
8.34.4	7.34.4 三种形态的使用场景	378
8.35	7.35 深入：运维脚本合集（收藏）	379
8.35.1	7.35.1 备份脚本 backup.ps1	379
8.35.2	7.35.2 清理脚本 clean-csv.ps1	379
8.35.3	7.35.3 健康检查脚本 health.ps1	379
8.35.4	7.35.4 开机自启 + 存活守护（组合）	379
8.36	7.36 深入：日志分析指南	379
8.36.1	7.36.1 日志长什么样	379
8.36.2	7.36.2 快速过滤技巧	379
8.36.3	7.36.3 常见日志模式解读	380
8.37	7.37 07 章收官：运维能力地图	380
8.38	7.38 深入：改造决策参考（动哪一层，影响多大）	380
8.38.1	7.38.1 常见改造请求的影响面	380
8.38.2	7.38.2 改造前的检查清单	380
8.38.3	7.38.3 最小改动原则	381
8.39	7.39 深入：性能数据采集方法	381
8.39.1	7.39.1 采集什么	381
8.39.2	7.39.2 一次基准测试示例	381
8.39.3	7.39.3 优化验证方法	381
8.40	7.40 07 章终极验证：动手任务清单	381
8.40.1	任务 1：全新环境搭建（60 分钟）	381
8.40.2	任务 2：生产部署演练（60 分钟）	381
8.40.3	任务 3：故障排查演练（30 分钟）	381
8.41	7.41 补充：环境变量与 dotenv 机制	382

8.41.1	7.41.1 dotenv 的加载顺序	382
8.41.2	7.41.2 新增环境变量的步骤	382
8.42	7.42 补充: config-*.ini 的完整结构参考	382
8.42.1	7.42.1 config-fj200c_information.ini 全节	382
8.42.2	7.42.2 config-fj200c_main.ini 全节	382
8.42.3	7.42.3 config-ftj1c.ini 全节	382
8.43	7.43 补充: 常见安装/环境问题 (Windows)	382
8.44	7.44 补充: 07 章知识自测 (追加 10 题)	382
8.45	7.45 深入: 数据库备份的三种方式对比	383
8.46	7.46 深入: WAL 模式与数据安全	383
8.46.1	7.46.1 WAL 是什么	383
8.46.2	7.46.2 项目启用方式	383
8.46.3	7.46.3 WAL 文件说明	383
8.47	7.47 深入: 日志排障的实战案例	383
8.47.1	7.47.1 常见日志片段解读	383
8.47.2	7.47.2 如何看日志	383
8.48	7.48 深入: 性能与资源调优清单	384
8.49	7.49 深入: 7 应用日常运维任务清单	384
8.49.1	7.49.1 每日	384
8.49.2	7.49.2 每周	384
8.49.3	7.49.3 每月	384
8.50	7.50 深入: 07 章补充自测 (追加 10 题)	384
8.51	7.51 深入: 部署前的完整自检清单	385
8.52	7.52 深入: 服务自启动的完整配置	385
8.52.1	7.52.1 场景	385
8.52.2	7.52.2 方法一: 任务计划程序	385
8.52.3	7.52.3 方法二: 启动文件夹	385
8.52.4	7.52.4 方法三: Windows 服务 (第三方工具 NSSM)	385
8.53	7.53 深入: 日志与监控的长期方案	385
8.53.1	7.53.1 日志滚动	385
8.53.2	7.53.2 简易健康监控	385
8.53.3	7.53.3 升级路径	385
8.54	7.54 深入: 07 章最终综合自测 (追加 8 题)	385
8.55	7.55 深入: 7 个应用的真实使用场景演练	386
8.55.1	7.55.1 场景一: 车间监控值班	386
8.55.2	7.55.2 场景二: 管理员日常	386
8.55.3	7.55.3 场景三: 设备运维	386
8.55.4	7.55.4 场景四: 通信排查	386
8.56	7.56 深入: 故障分级与响应	386
8.57	7.57 深入: 07 章实战自测 (8 题)	386
8.58	7.58 深入: Windows 环境问题排查手册	387
8.58.1	7.58.1 常见问题速查	387
8.58.2	7.58.2 查看端口占用	387
8.58.3	7.58.3 控制台中文	387
8.59	7.59 深入: 数据安全与权限管理	387
8.59.1	7.59.1 账号安全	387
8.59.2	7.59.2 数据安全	387

	8.59.3 7.59.3 权限最小化	387
8.60	7.60 深入：07 章高频自测（8 题）	387
8.61	7.61 深入：三份 ini 的完整配置参考	388
	8.61.1 7.61.1 config-fj200c_information.ini	388
	8.61.2 7.61.2 config-fj200c_main.ini	388
	8.61.3 7.61.3 config-ftj1c.ini	388
	8.61.4 7.61.4 修改生效规则	388
8.62	7.62 深入：数据库操作的完整指南	388
	8.62.1 7.62.1 查看数据	388
	8.62.2 7.62.2 常用 SQL	388
	8.62.3 7.62.3 危险操作清单	389
8.63	7.63 深入：7 个应用的浏览器测试要点	389
	8.63.1 7.63.1 登录测试	389
	8.63.2 7.63.2 权限测试	389
	8.63.3 7.63.3 数据流测试	389
8.64	7.64 深入：07 章终局自测（8 题）	389
8.65	7.65 深入：日常运维的 PowerShell 脚本合集	389
	8.65.1 7.65.1 一键启动脚本	389
	8.65.2 7.65.2 健康检查脚本	389
	8.65.3 7.65.3 自动备份脚本	390
	8.65.4 7.65.4 日志清理脚本	390
8.66	7.66 深入：性能调优的完整路线	390
	8.66.1 7.66.1 前端性能	390
	8.66.2 7.66.2 后端性能	390
	8.66.3 7.66.3 性能测试方法	390
8.67	7.67 深入：07 章毕业自测（8 题）	390
8.68	7.68 深入：常见故障的完整排查手册（最全版）	391
	8.68.1 7.68.1 后端起不来	391
	8.68.2 7.68.2 前端白屏	391
	8.68.3 7.68.3 登录失败	391
	8.68.4 7.68.4 无数据/数据卡住	391
	8.68.5 7.68.5 数据不对	391
8.69	7.69 深入：多用户并发使用的注意事项	391
	8.69.1 7.69.1 并发读	391
	8.69.2 7.69.2 并发写	391
	8.69.3 7.69.3 监控数据的多人查看	391
8.70	7.70 深入：07 章权威自测（8 题）	391
8.71	7.71 深入：升级与迁移指南	392
	8.71.1 7.71.1 版本升级的完整流程	392
	8.71.2 7.71.2 数据库迁移的注意事项	392
	8.71.3 7.71.3 配置升级	392
8.72	7.72 深入：文档与知识沉淀	392
	8.72.1 7.72.1 为什么要有运行文档	392
	8.72.2 7.72.2 沉淀什么	392
	8.72.3 7.72.3 怎么沉淀	392
8.73	7.73 深入：07 章权威自测（8 题）	392
8.74	7.74 深入：跨年/长期运行的数据管理	393

8.74.1	7.74.1 长期运行的问题	393
8.74.2	7.74.2 数据保留策略	393
8.74.3	7.74.3 归档与恢复	393
8.75	7.75 深入：性能退化的排查（长期运行后）	393
8.75.1	7.75.1 症状与原因	393
8.75.2	7.75.2 排查顺序	393
8.75.3	7.75.3 常规优化	393
8.76	7.76 深入：07 章权威自测（8 题）	393
8.77	7.77 深入：本章收尾——运维思维总结	394
8.77.1	7.77.1 运维的核心原则	394
8.77.2	7.77.2 日常运维的闭环	394
8.77.3	7.77.3 学完本章的标准	394
8.78	7.78 深入：07 章最终自测（6 题）	394
9 08	扩展与二次开发	394
9.1	8.1 二次开发全景图	394
9.2	8.2 新增角色完整流程（核心方法论）	394
9.2.1	8.2.1 流程总览（7 步）	394
9.2.2	8.2.2 第 1 步：后端权限与注册表	395
9.2.3	8.2.3 第 2 步：复制 role_template	395
9.2.4	8.2.4 第 3 步：注解与登记	395
9.2.5	8.2.5 第 4 步：前端 roles.ts	395
9.2.6	8.2.6 第 5 步：复制前端	396
9.2.7	8.2.7 第 6~7 步：同步与部署	396
9.2.8	8.2.8 新增角色流程要点总结	396
9.3	8.3 后端新模块开发模板（role_template 详解）	396
9.3.1	8.3.1 模板文件结构	396
9.3.2	8.3.2 模块入口（mod.rs 模式）	396
9.3.3	8.3.3 路由注册（routes.rs）	397
9.3.4	8.3.4 handler 模板	397
9.3.5	8.3.5 service 模板	397
9.3.6	8.3.6 建表位置（database.rs）	397
9.4	8.4 修改现有模块（最高频操作）	397
9.4.1	8.4.1 加字段（全链路清单）	397
9.4.2	8.4.2 加接口	398
9.4.3	8.4.3 改逻辑（如改 CSV 列）	398
9.4.4	8.4.4 改前端行为	398
9.5	8.5 新硬件协议接入（串口/UDP）	398
9.5.1	8.5.1 协议接入的通用模式	398
9.5.2	8.5.2 接入新协议的步骤	398
9.5.3	8.5.3 模拟源的价值	398
9.6	8.6 前端组件与主题扩展	399
9.6.1	8.6.1 新增共享组件	399
9.6.2	8.6.2 新增主题（fj200c_main 模式）	399
9.6.3	8.6.3 新增页面/路由	399
9.7	8.7 报表与导出扩展	399
9.7.1	8.7.1 现有报表机制	399
9.7.2	8.7.2 扩展报表形态	399

9.7.3	8.7.3 CSV 导出的通用化	399
9.8	8.8 安全扩展	399
9.8.1	8.8.1 认证增强	399
9.8.2	8.8.2 权限模型增强	400
9.8.3	8.8.3 数据安全	400
9.9	8.9 常见扩展问答 (FAQ)	400
9.10	8.10 扩展开发最佳实践	400
9.10.1	8.10.1 扩展项目 Checklist	400
9.10.2	8.10.2 渐进式开发策略	400
9.10.3	8.10.3 向后兼容原则	400
9.11	8.11 扩展中的常见决策问题 (架构判断力)	401
9.11.1	8.11.1 该新建模块还是改现有模块?	401
9.11.2	8.11.2 数据存哪 (DB vs 文件 vs 内存)	401
9.11.3	8.11.3 配置放 ini 还是数据库	401
9.11.4	8.11.4 新接口怎么设计 (REST 惯例)	401
9.12	8.12 扩展中的常见坑 (血泪清单)	402
9.13	8.13 扩展之后的验证清单 (回归测试)	402
9.13.1	功能回归	402
9.13.2	契约回归	402
9.13.3	部署回归	402
9.14	8.14 深入: 扩展前的设计问题清单	402
9.15	8.15 深入: 扩展的最佳实践十条	403
9.16	8.16 深入: 常见坑的完整清单 (扩展时对照)	403
9.17	8.17 深入: 回归验证清单 (扩展后必过)	403
9.18	8.18 深入: 给新模块的代码结构建议	404
9.19	8.19 深入: 与团队协作的文档化约定	404
9.20	8.20 扩展实战: 完整案例一 (新增“报警管理”模块)	404
9.20.1	需求	404
9.20.2	步骤 1: 权限与角色	404
9.20.3	步骤 2: 新模块目录	404
9.20.4	步骤 3: models.rs	404
9.20.5	步骤 4: database.rs 建表	405
9.20.6	步骤 5: handlers + routes + 登记	405
9.20.7	步骤 6: 前端	405
9.20.8	步骤 7: gen:api + build + 部署同步	405
9.21	8.21 扩展实战: 完整案例二 (接入新串口协议)	405
9.21.1	需求	405
9.21.2	方案选型	405
9.21.3	实施步骤	406
9.22	8.22 扩展实战: 完整案例三 (给台账加 Excel 导出)	406
9.22.1	需求	406
9.22.2	方案对比	406
9.22.3	推荐: 后端 CSV + BOM (通用做法)	406
9.23	8.23 扩展实战: 完整案例四 (告警通知到钉钉)	406
9.23.1	需求	406
9.23.2	实现思路 (零侵入)	406
9.23.3	配置化	407

9.24	8.24 扩展实战：完整案例五（历史数据回放页）	407
9.24.1	需求（承接 05 章 5.56 的脑内规划，这里给出落地）	407
9.24.2	数据源：后端加“读 CSV”接口	407
9.24.3	前端回放	407
9.24.4	页面组装	408
9.25	8.25 扩展思路库（灵感清单）	408
9.26	8.26 扩展实战：完整案例六（审计日志模块）	408
9.26.1	需求	408
9.26.2	实现思路	408
9.26.3	效果	409
9.27	8.27 扩展实战：完整案例七（CSV 趋势分析页）	409
9.27.1	需求	409
9.27.2	实现	409
9.28	8.28 扩展实战：完整案例八（暗色模式一键统一切换）	409
9.28.1	需求	409
9.28.2	实现	410
9.29	8.29 扩展实战：完整案例九（多语言 i18n 方案要点）	410
9.29.1	需求	410
9.29.2	方案要点（不落地，给思路）	410
9.30	8.30 扩展路线图（按优先级排序）	410
9.31	8.31 结语：从读者到开发者	410
9.31.1	8.31.1 你现在能做什么	410
9.31.2	8.31.2 建议的学习闭环	411
9.31.3	8.31.3 使用文档的姿势	411
9.31.4	8.31.4 最后的话	411
9.32	8.32 全教程最终自测（跨章综合）	411
9.33	8.33 扩展补充：综合实战——给 fw100 加“告警阈值”功能（端到端演练） ..	411
9.33.1	8.33.1 需求	411
9.33.2	8.33.2 后端改动	411
9.33.3	8.33.3 gen:api 同步	412
9.33.4	8.33.4 前端改动	412
9.33.5	8.33.5 完成清单	412
9.34	8.34 扩展补充：性能与扩展性的预判	412
9.34.1	8.34.1 什么时候需要关注性能	412
9.34.2	8.34.2 性能优化的优先级	412
9.34.3	8.34.3 千万避免	412
9.35	8.35 扩展补充：与外部系统对接的注意事项	412
9.35.1	8.35.1 对接前必问	412
9.35.2	8.35.2 常见对接模式	413
9.35.3	8.35.3 健壮性要求	413
9.36	8.36 扩展补充：08 章最终自测（追加 8 题）	413
9.37	8.37 扩展补充：新增一个“巡检记录”完整模块（最小可行案例）	413
9.37.1	8.37.1 需求	413
9.37.2	8.37.2 后端五件套	413
9.37.3	8.37.3 表结构	413
9.37.4	8.37.4 gen:api + 前端	413
9.37.5	8.37.5 完成自检	414

9.38	8.38 扩展补充：多模块协作的扩展（联动类需求）	414
9.38.1	8.38.1 场景	414
9.38.2	8.38.2 实现方式	414
9.38.3	8.38.3 注意点	414
9.39	8.39 扩展补充：08 章高频自测（8 题）	414
9.40	8.40 扩展补充：部署新版本的完整流程（扩展后必知）	414
9.40.1	8.40.1 完整发布流程	414
9.40.2	8.40.2 发布检查清单	415
9.40.3	8.40.3 回滚方案	415
9.41	8.41 扩展补充：给 7 个应用加公共功能的思路	415
9.41.1	8.41.1 公共功能的三种层级	415
9.41.2	8.41.2 实例：加“关于”页面	415
9.41.3	8.41.3 判断标准	415
9.42	8.42 扩展补充：08 章综合自测（8 题）	415
9.43	8.43 扩展补充：给模块加“历史数据回放”的完整实现（复盘案例五）	415
9.43.1	8.43.1 需求回顾	415
9.43.2	8.43.2 后端设计	416
9.43.3	8.43.3 前端回放实现	416
9.43.4	8.43.4 回放的优化	416
9.44	8.44 扩展补充：多协议设备的统一抽象（复盘案例二）	416
9.44.1	8.44.1 问题	416
9.44.2	8.44.2 后端抽象	416
9.44.3	8.44.3 前端如何面对多协议	416
9.44.4	8.44.4 新增协议的成本	416
9.45	8.45 扩展补充：08 章终局自测（8 题）	416
9.46	8.46 扩展补充：全链路示例——从需求到上线的完整复盘	417
9.46.1	8.46.1 需求	417
9.46.2	8.46.2 后端改动	417
9.46.3	8.46.3 配置项	417
9.46.4	8.46.4 前端改动	417
9.46.5	8.46.5 上线流程	417
9.47	8.47 扩展补充：理解“契约驱动开发”的心智模型	418
9.47.1	8.47.1 核心思想	418
9.47.2	8.47.2 在本项目的落地	418
9.47.3	8.47.3 心智模型一句话	418
9.48	8.48 扩展补充：08 章毕业自测（8 题）	418
9.49	8.49 扩展补充：后端新模块的完整骨架（写代码之前先看）	418
9.49.1	8.49.1 目录骨架	418
9.49.2	8.49.2 mod.rs 的标准内容	418
9.49.3	8.49.3 handlers.rs 的标准内容	419
9.49.4	8.49.4 routes.rs 挂载	419
9.50	8.50 扩展补充：08 章权威自测（8 题）	419
9.51	8.51 扩展补充：前端新页面的完整落地流程	419
9.51.1	8.51.1 新建页面五步	419
9.51.2	8.51.2 复制改名的技巧	419
9.51.3	8.51.3 页面接入权限	419
9.51.4	8.51.4 验证清单	420

9.52	8.52 扩展补充: 08 章权威自测 (8 题)	420
9.53	8.53 扩展补充: 扩展能力的评估框架	420
9.53.1	8.53.1 评估的五个维度	420
9.53.2	8.53.2 小扩展 vs 大扩展	420
9.53.3	8.53.3 扩展的实施顺序	420
9.54	8.54 扩展补充: 08 章权威自测 (8 题)	420
9.55	8.55 扩展补充: 扩展质量的验收标准	421
9.55.1	8.55.1 功能验收	421
9.55.2	8.55.2 代码验收	421
9.55.3	8.55.3 文档验收	421
9.56	8.56 扩展补充: 给新手的最终建议	421
9.57	8.57 扩展补充: 08 章权威自测 (8 题)	421
9.58	8.58 扩展补充: 本章收尾——从读者到作者的最后一课	421
9.58.1	8.58.1 扩展的本质	421
9.58.2	8.58.2 本书给到你的工具箱	422
9.58.3	8.58.3 最后的告别	422
9.59	8.59 扩展补充: 08 章最终自测 (6 题)	422

1 RustWeb-Vue 全栈项目接手指南 (新手向)

适用对象: 即将接手本项目、但 Rust 与 Vue 经验有限的新手开发者。本指南共 9 个文档 (含本索引), 全文约 12 万字, 含 30+ 张 Mermaid 图表, 以项目**真实代码**为教材 (每个关键点都标注了源码路径, 可随时打开对照)。写作基准: 本指南内容以 AGENTS.md (项目根目录与仓库根目录各一份) 与当前代码库为准。注意 README.md 已部分过时 (仍写着 6 个前端应用与单一 fj200c 模块, 实际为 7 个应用、fj200c 已拆分), 阅读时以本指南与 AGENTS.md 为准。

1.1 一、这套文档能帮你什么

如果你是完全的新手, 读完这套文档后你将能够:

1. **看懂这个项目**——知道后端 Rust 代码的每一层 (路由、处理器、服务、数据库) 在干什么, 知道前端 7 个 Vue 应用为什么长得一模一样却各自独立。
2. **跑起来**——从零搭建环境, 启动后端和任意前端, 用种子账号登录, 让模拟数据流转起来。
3. **改代码**——在既有模块里改功能、加接口、加页面, 不破坏现有结构。
4. **加新业务**——按照“新增角色七步流程”把一套全新的业务模块 (新角色 + 新前端 + 新接口) 完整接入系统。
5. **发布上线**——理解 deploy.bat 一键部署的原理, 掌握单 exe 内嵌前端的构建方式。

这套文档**不是**完整的 Rust 语言教程或 Vue 官方文档, 而是“以本项目为教材”的语法速成 + 架构精读。所有语法点都用项目里的真实代码举例, 看完语法马上就能在项目里找到对应代码。

1.2 二、文档总目录与阅读路线

文档	主题	适合谁	建议耗时
00-索引与导学 (本文件)	总目录、阅读路线、学习方法	所有人	15 分钟
01-项目全景	系统定位、技术栈、7 应用总览、三层架构、一次完整请求的全链路、目录逐层解析	所有人，必读	1~2 小时
02-Rust 语法速成	所有权/借用、Option/Result、枚举与模式匹配、trait、async/await、宏、错误处理——全部用项目代码举例	Rust 新手必读	4~6 小时
03-后端逐模块精读	main.rs 启动、routes/roles/database、common 公共层 (JWT/中间件/WS/工具)、auth 登录流程、fj200c_information 完整数据流、其余业务模块差异	想深入后端的人	6~8 小时
04-Vue3 与 TS 语法速成	组合式 API、ref/reactive/computed/watch、TypeScript 类型、Vue Router 守卫、Pinia、Element Plus、WebSocket、ECharts——全部用项目代码举例	Vue/TS 新手必读	4~6 小时
05-前端逐应用精读	shared 公共层解剖 (角色注册表/会话/认证工厂/请求封装)、典型应用 fj200c_information 全文件走读、fj200c_main 的 WS 单例模式、其余应用差异	想深入前端的人	6~8 小时
06-前后端类型同步	utoipa 注解 → export_openapi → orval → generated 代码，改一个接口的新增/修改/删除流程	所有要改接口的人	1~2 小时
08-使用与维护开发环境	新增/修改/删除流程、账号、加接口、部署环境、部署脚本、deno 使用与维护	日常使用与维护者	2~3 小时

flowchart TD

```
A[你<br/>Rust/Vue 新手] --> B[00 索引与导学<br/>先花 15 分钟]
B --> C[01 项目全景<br/>建立全局地图]
C --> D{兴趣/任务偏向?}
D -->|后端为主| E[02 Rust 语法速成]
D -->|前端为主| F[04 Vue3+TS 语法速成]
D -->|都要| G[02 + 04 都看]
E --> H[03 后端逐模块精读]
F --> I[05 前端逐应用精读]
H --> J[06 类型同步机制<br/>改接口必读]
I --> J
J --> K[07 使用与维护手册<br/>日常干活必读]
K --> L[08 扩展与二次开发<br/>接需求时查阅]
G --> H
```

建议速通路线（最快 3 天）：01 → 02 挑“所有权、Result、枚举”三节 → 03

挑“main.rs、auth 登录、fj200c_information”三节 → 04 挑“组合式 API、Pinia、路由守卫”三节 → 05 挑“shared 公共层、典型应用走读”两节 → 06 → 07。其余章节作为手册按需查阅。

1.3 三、项目一句话画像

graph LR

```
subgraph 前端层["前端层（7 个 Vue 3 应用，各占一个端口）"]
    A1[admin<br/>用户管理]
    A2[fj200c_information<br/>发动机监控]
    A3[fj200c_main<br/>发动机测控]
    A4[fw100 / fw150<br/>设备台账]
    A5[ftj1c<br/>UDP 通信监控]
    A6[city3d<br/>城市 3D]
end
subgraph 共享层["packages/shared 共享包"]
    B[认证工厂<br/>角色注册表<br/>orval 生成代码<br/>模板组件]
end
subgraph 后端层["Rust 后端（Axum，端口 3000）"]
    C[HTTP 路由层]
    D[中间件层<br/>JWT 鉴权 + RBAC 权限]
    E[处理器层 handlers]
    F[服务层 services<br/>SQLite 读写]
    G[硬件/模拟层<br/>串口 / UDP / Mock]
    H[WebSocket 广播层]
end
subgraph 数据层["SQLite + INI + CSV"]
    I[(rustweb.db)]
    J[config-*.ini]
    K[csv/ 数据目录]
end
A1 --> B
A2 --> B
A3 --> B
A4 --> B
A5 --> B
A6 --> B
B --> C
C --> D --> E --> F
F --> I
E --> G
E --> H
G --> J
E --> K
```

一句话总结：这是一个“一台机器上的工业设备监控与管理系统”——Rust 后端负责登录鉴权、设备串口/UDP 通信、数据解码与广播；7 个前端按角色分工展示不同业务；SQLite 存用户与台账数据；INI 存设备通信配置；CSV 存试验数据；orval 保证前后端类型契约永远一致。

1.4 四、学习环境准备（写代码前先读）

在开始精读之前，建议先准备好环境，边读边运行边对照：

1. **Rust 工具链：**安装 rustup，确认 cargo --version 可用（项目用的是 Rust 2021 edition，任意 1.65+ 版本即可）。
2. **Node.js：**建议 18+（Vite 6 要求），确认 node --version、npm --version 可用。
3. **VS Code：**安装扩展 rust-analyzer（Rust 智能提示）、Volar（Vue 3 智能提示，注意已弃用 Vetur）、Prettier、Mermaid Preview（本地预览本套文档的图表）。
4. **数据库工具：**任意 SQLite 客户端（如 DB Browser for SQLite），用于查看 rustweb.db。
5. **浏览器：**Chrome/Edge，F12 开发者工具要会用（网络面板看 API 请求、控制台看报错）。

环境验证命令（在项目根目录）：

```
cargo --version      # 例如 cargo 1.80.0
node --version       # 例如 v20.11.0
npm --version        # 例如 10.2.4
```

1.5 五、学习方法建议

1. **边读边跑：**读完第 01 章后立刻按第 07 章把项目跑起来，登录 admin@rustweb.dev，点开各个页面，对照文档理解。
2. **代码要对照着看：**每个标注了 src/xxx.rs:123 的代码块，都建议打开真实文件对照。文档中的代码可能经过删节（用 ... 表示），完整代码以仓库为准。
3. **用好 rust-analyzer：**把鼠标悬停在变量/类型上，看推断类型；右键“转到定义”，在代码里跳来跳去理解结构。
4. **先读后改：**第 08 章的扩展流程是建立在第 02~06 章理解之上的，不要跳步。
5. **mermaid 图怎么用：**GitHub 网页端可以直接渲染；VS Code 装 Mermaid Preview 后可本地预览；命令行可用 npx @mermaid-js/mermaid-cli 导出 PNG/SVG（本项目根目录 devDependencies 已内置 mmdc）。

1.6 六、术语速查表（先混个脸熟）

术语	含义	在本项目中的位置
Axum	Rust 最流行的 Web 框架，类似 Express/Flask	后端框架本体
Handler	HTTP 处理函数，一个接口一个函数	src/*/handlers.rs
Middleware	中间件，在请求到达 handler 前拦截处理	src/common/middleware.rs
JWT	JSON Web Token，无状态登录凭证	src/common/jwt.rs
RBAC	基于角色的权限控制	src/roles.rs 角色注册表
Permission	权限点，一个权限点控制一类操作	src/common/models.rs 的 Permission 枚举
sqlx	Rust 的 SQL 库，支持编译期检查	所有 services 层
ORM	对象关系映射（本项目没用 ORM，用 sqlx 写 SQL）	—
WS	WebSocket，服务端→前端实时推送	src/common/ws.rs + 各模块 ws_handler
broadcast	tokio 的广播通道，一对多消息分发	src/fj200c_information/mod.rs
OnceLock	Rust 单例容器，进程内全局唯一	各模块全局状态
ArcSwap	无锁读写共享指针，热更新配置用	src/common/quad_frame.rs 等
Pinia	Vue 3 官方状态管理库（相当于 Vuex 的升级）	各前端 stores/
Composition API	Vue 3 组合式 API，用函数组织逻辑	所有 .vue 的 <script setup>
Vite	前端构建工具与开发服务器	各前端 vite.config.ts
Element Plus	基于 Vue 3 的 UI 组件库	所有页面组件
orval	根据 OpenAPI 自动生成 TS 请求代码的工具	根目录 orval.config.ts
utoipa	Rust 侧自动生成 OpenAPI 文档的宏框架	handler 上的 #[utoipa::path] 注解
workspace	npm 多包仓库管理（monorepo）	根 package.json 的 workspaces 字段

1.7 七、常见问题解答（读文档前先看）

1.7.1 Q1：我是完全零基础，这套文档能看懂吗？

能。02 章与 04 章是专门为零基础写的语法速成，所有语法点都用本项目真实代码举例，并标注了源码路径。建议顺序：01 → 02 → 03（后端线）或 01 → 04 → 05（前端线），最后汇合到 06。

1.7.2 Q2：我只想改一个页面，要看哪些章节？

最快路径：00 索引 → 05 章前端逐应用精读（找到对应应用的走读）→ 04 章（查语法）。改页面通常只涉及单个前端应用，无需读完整个后端文档。

1.7.3 Q3：我想加一个全新的业务模块，看哪章？

08 章「扩展与二次开发」的 8.2 节有完整七步流程，配合 06 章类型同步机制使用。建议先通读 01、03 两章建立全局概念再动手。

1.7.4 Q4：文档里的代码和仓库不一致怎么办？

文档中的代码可能经过删节（用 ... 省略与主题无关的部分），完整代码以仓库为准。每个代码块都有路径标注，打开真实文件对照阅读。

1.7.5 Q5：mermaid 图在 VS Code 里不显示怎么办？

安装 Mermaid Preview 扩展，或在 GitHub 网页端查看（自动渲染）。也可以运行 `npx @mermaid-js/mermaid-cli` 导出为 PNG 查看。

1.7.6 Q6：如何快速定位一个文件属于哪个章节？

用 00 章的术语速查表 + 01 章的目录解析（含完整目录树）。遇到具体文件时，也可以直接使用 IDE 的全局搜索（Ctrl+Shift+F）。

1.8 八、文档维护约定（给后续维护者）

1. 每章末尾统一以「>

下一节：0X-xxx。」结尾，方便衔接。2. 章节号与文件名前缀保持一致（00~08）。3. 新增内容优先追加到对应章节末尾，避免打乱原有编号。4. 引用代码时标注 文件路径:行号（如 `src/routes.rs:45`）。5. 修改 AGENTS.md 中的架构事实时，同步更新本套文档相关章节。

1.9 九、阅读路线图（按天规划）

如果希望系统化学完，可以参考以下节奏（每章约 1~3 小时）：

1.9.1 第 1~2 天：先建全局

第 1 天：00 章（本索引）+ 01 章（项目全景）

第 2 天：02 章（Rust 语法）+ 03 章第 1 节（后端总览）

1.9.2 第 3~5 天：后端

第 3 天：03 章（auth/admin/common）

第 4 天：03 章（fj200c_information/fj200c_main）

第 5 天：03 章（ftj1c/fw100/fw150/city3d）

1.9.3 第 6~8 天：前端

第 6 天：04 章（Vue3/TS 语法）

第 7 天：05 章（admin/fj200c_information/fj200c_main）

第 8 天：05 章（fw100/fw150/ftj1c/city3d）

1.9.4 第 9~10 天：工程与维护

第 9 天：06 章（类型同步）+ 07 章（使用与维护）

第 10 天：08 章（扩展实战）+ 总复习

灵活变通：只想用系统 → 只读 07 章；只想改前端 → 04+05+06；只想改后端 → 02+03+06。

1.10 十、结语与开始

这套系统的核心脉络只有三句话：

后端：Rust + Axum + SQLite，一次启动，七路服务。**前端：**七个 Vue 3 应用，一套 shared，一个登录态。**契约：**utoipa → OpenAPI → orval，生成代码即类型。

现在，打开 01-项目全景.md，开始你的旅程。祝学习愉快！

1.11 十一、给新手的学習避坑提示

1.11.1 11.1 常见误区

1. 想先学完 Rust 再看项目 → 学不完。先看项目，遇到语法去 02 章查
2. 只读不敲 → 记不住。每章自测题要动手敲
3. 一次读完全部 → 消化不良。按九节路线图分天
4. 代码不跑 → 不踏实。按 01 章搭好环境跑起来

1.11.2 11.2 正确的学习姿势

1. 环境先跑起来（01 章）
2. 每章读完 → 做自测 → 动手改一个小地方
3. 卡住 → 用 00 章索引定位 → 精读对应节
4. 反复修改 → 加深理解（改坏了 git checkout 还原）

1.11.3 11.3 遇到问题怎么办

1. 报错先读完整错误信息（前 5 行）
2. 搜索错误关键字（中文社区 + 官方文档）
3. 看本项目类似代码怎么写的
4. 实在不行 → 问同事/记录到文档

1.12 十二、全书结构一页纸（速记版）

00 索引与导学	← 你现在在这
01 项目全景	← 系统是什么
02 Rust 语法速成	← 读后端的钥匙
03 后端逐模块精读	← 后端源码地图
04 Vue3/TS 语法	← 读前端的钥匙
05 前端逐应用精读	← 前端源码地图
06 类型同步机制	← 前后端怎么握手
07 使用与维护	← 怎么跑/怎么修
08 扩展与二次开发	← 怎么加功能

两条主线：

后端线：02 → 03 → 06 → 08

前端线：04 → 05 → 06 → 08

1.13 十三、致读者的一段话

这套文档没有学院派的堆砌，只有“代码在哪儿、为什么这么写、怎么改”三件事。你不需要记住每一行，只需要知道：**遇到问题去哪个文件、哪个章节找答案。**

读的时候请记住：

1. 每个代码块都有出处（文件路径:行号）——去源码里验证
2. 每个 mermaid 图都是讲稿——能对着图讲出来就算懂
3. 每章自测题都要写——写出来才算会
4. 学完 08 章，你就能自己加功能了

现在，关掉聊天窗口，打开编辑器，我们开始。

1.14 十四、关于本套文档的字数说明

本套教程共 9 章，合计超过 10 万汉字，目标读者是“能看懂代码但没写过 Rust/Vue 的新手”。文档追求的不是面面俱到，而是“每个概念都能在项目里找到真实例子”。

如果读的时候发现某个概念在文档里没有直接答案，请记住：

1. 先在对应该章节的“深入/补充”小节里找
2. 再到源码里搜关键字
3. 最后把问题记下来——欢迎完善文档

1.15 十五、最后再嘱咐三件事

第一，**这套系统的代码是活的**——文档写于某个时间点，代码可能继续演进。遇到文档与代码不符时，以代码为准，并把差异记下来反馈给文档维护者。

第二，**学习最重要的是动手**。每章的自测题、动手任务、08 章的实战案例，都是为动手准备的。看完不做，等于白看。

第三，**保持好奇心**。遇到“为什么这么设计”的问题，翻代码、翻 git 历史（git log 能看到演进）、翻 AGENTS.md，答案通常都在。

祝学习顺利，早日成为能独立改这个系统的人。

再强调一次：遇到看不懂的代码，不要死磕，先带着问题往下走，回头再看自然就懂了。每一章都给出了代码位置与运行命令，跟着敲一遍，胜过看十遍。坚持读完九章，你就具备了独立维护这套系统、乃至从零搭建类似系统的完整能力。

下一节：**01-项目全景**——开始建立全局地图。

2 01 项目全景：先建立全局地图

本章目标：让你在 1~2 小时内建立对整个系统的完整地图——系统是什么、有哪些部分组成、数据怎么流动、目录怎么组织。读完本章你就能在任何代码文件里“找得到北”。

全文约 1.5 万字，含 8 张 Mermaid 图。建议边读边打开对应文件对照。

2.1 1.1 这个项目到底是个什么系统

2.1.1 1.1.1 一句话定位

RustWeb-Vue 是一套运行在工业内网环境中的“多业务管理平台”：一个 Rust 编写的后端进程负责登录鉴权、与硬件设备（串口、UDP 组播）通信、数据解码与实时推送；七个 Vue 3 前端应用各自面向一类业务角色，通过浏览器访问同一后端。

这些业务包括：

业务	干什么
发动机监控 (fj200c_information)	读取发动机试车台的多路串口数据帧，实时监控、可视化、记录 CSV 试验数据
发动机测控 (fj200c_main)	更复杂的测控台：ECU/ADAM/DYNO 三路串口同时采集，支持指令下发、试验信息录入、报表生成
通信监控 (ftj1c)	监控 UDP 组播通信链路（主备双路 + 6 路单连），查看遥测帧数据
设备台账 (fw100 / fw150)	两套设备台账列表展示（演示数据）
城市 3D (city3d)	城市区域/建筑/事件的三维可视化展示与数据管理
管理后台 (admin)	用户管理：创建用户、分配角色、删除用户

你可以把它理解成：**一个带登录系统的、前后端分离的、能接硬件数据的工业管理系统**。硬件部分（串口/UDP）都有 Mock 模拟模式，没有真实硬件也能完整体验所有功能——这是项目刻意设计的（配置文件里 `InProcess = true`、`Mock = true` 之类开关）。

2.1.2 1.1.2 为什么会有 7 个前端而不是 1 个

这是新人最容易困惑的问题。传统 Web 项目是一个前端包含所有页面，这里却是 7 个独立应用。原因：

- 1. **按角色隔离**：每个角色只应该看到自己的业务页面。7 个应用与 7 个角色一一对应（admin 对应管理角色，其余 6 个各对应一个业务角色）。
- 2. **互不干扰**：每个应用独立构建、独立部署路径（/admin、/fj200c_information...），业务升级一个应用不影响其他。
- 3. **规模适中**：每个应用其实都不大（几个页面），单页应用的组织成本低。

代价是：公共代码需要抽到 `packages/shared` 共享包，避免 7 份重复代码。这就是“7 个应用长得几乎一模一样”的原因——它们都是同一套模板（同样的 `main.ts`、同样的 `App.vue`、同样的认证 `store`）套出来的。

2.1.3 1.1.3 这个项目不是 Tauri 项目

项目目录名带 `TauriProjects` 前缀（仓库上级目录），但本项目**不是 Tauri**：没有 `src-tauri/`，不打包桌面应用，就是一个普通的 Web 前后端项目。上级仓库里的其他目录（`demo-base` 等）才是 Tauri 项目。别混淆。

2.2 1.2 技术栈总览

2.2.1 1.2.1 后端技术栈（Rust）

```
flowchart LR
    subgraph 框架层
        A["axum 0.7<br/>Web 框架"]
        B["tokio<br/>异步运行时"]
        C["tower-http<br/>CORS/静态文件"]
    end
    subgraph 数据层
        D["sqlx 0.7<br/>SQLite 数据库"]
        E["configparser<br/>INI 配置"]
        F["csv<br/>试验数据记录"]
    end
```

```
subgraph 认证层
  G[jsonwebtoken<br/>JWT 签发验证]
  H[bcrypt<br/>密码哈希]
  I[validator<br/>输入校验]
end
subgraph 硬件层
  J[serialport 4<br/>串口通信]
  K[socket2<br/>UDP 组播]
  L[rand<br/>模拟数据]
end
subgraph 文档层
  M[utoipa<br/>OpenAPI 生成]
  N[rust-embed<br/>前端内嵌打包]
end
A --- B
A --- C
D --- E --- F
G --- H --- I
J --- K --- L
M --- N
```

依赖清单在 Cargo.toml (项目根目录), 按功能分组注释, 是个很好的依赖清单范本。核心依赖版本: axum 0.7、sqlx 0.7、tokio 1.x、utoipa 5、rust-embed 8。

2.2.2 1.2.2 前端技术栈 (Vue)

技术	版本	用途
Vue 3	^3.5	前端框架 (组合式 API + <script setup>)
Vite 6	^6.0	构建工具与开发服务器
TypeScript	^5.x	类型系统 (strict 模式全开)
Element Plus	^2.x	UI 组件库 (表格/表单/弹窗/消息)
Pinia	^2.x	状态管理
Vue Router	^4.x	路由与守卫
ECharts	^5.x	图表 (fj200c_information/fj200c_main)
Three.js	无直接依赖 (city3d 用 importmap/本地构建? 见 5.x 章)	3D 场景 (city3d)

注意: 7 个应用各自的 package.json 依赖并不完全相同。例如只有 city3d 有 three.js、只有 fj200c_information/fj200c_main 有 echarts。但 npm workspaces 模式下, 依赖统一在根目录 npm install 一次安装完成。

2.2.3 1.2.3 前后端之间的“契约”机制

这是本项目最有特色的部分, 必须一开始就建立概念:

```
flowchart LR
  A["Rust 代码<br/>#[utoipa::path] 注解<br/>#[derive(ToSchema)]"] --> B[cargo test export_openapi]
  B --> C[openapi/openapi.json<br/>40 路径 / 50 操作 / 72 schema]
  C --> D[orval]
```

```
D --> E[packages/shared/src/api/generated/<br/>TS 请求函数 + TS 类型]
E --> F[7 个前端应用<br/>import from @shared]
```

后端是类型的唯一真相源：前端调用的每个接口、每个数据结构，都是后端代码注解自动生成的。后端改了接口，npm run gen:api 重新生成，前端 vue-tsc 立刻报错——报错的地方就是需要同步修改的调用点。这套机制保证了“改了后端不会悄悄破坏前端”。

2.3 1.3 七个前端应用总览

2.3.1 1.3.1 应用一览表

目录	角色 key	权限	dev 端口	prod 路径	主要页面
frontend/ admin	admin	SystemAdmin + Users*	5174	/admin	登录、用户 列表、创建 用户、403
frontend/ fj200c_infor fj200c_information	fj200c_infor fj200c_infor	Fj200cInformationMonitor	5173	/fj200c_infor fj200c_information	监控、可视 化、数据、 配置、帮助
frontend/ fj200c_main	fj200c_main	Fj200cMainMonitor	5179	/fj200c_main	监控、试验 录入、试验 查看、报表、 数据、配置、 帮助
frontend/ fw100	fw100	Fw100Monitor	5175	/fw100	台账面板
frontend/ fw150	fw150	Fw150Monitor	5178	/fw150	台账面板
frontend/ ftj1c	ftj1c	Ftj1cMonitor	5176	/ftj1c	通信监控 (单页)
frontend/ city3d	city3d	City3dView	5177	/city3d	3D 场景、数 据管理、全 景

2.3.2 1.3.2 应用间如何跳转

这 7 个应用共享同一个浏览器 localStorage (同一个域名 localhost)，token 存同一个 key。登录任何应用后，token 自动带过去；如果登录的账号角色不属于当前应用，登录页会**自动跳转到该角色自己的应用**：

```
sequenceDiagram
    participant U as 用户
    participant L as 应用A登录页
    participant B as 后端 /api/auth/login
    participant S as localStorage
    participant A2 as 应用B (角色对应应用)
    U->>L: 输入账号密码
    L->>B: POST /api/auth/login
    B-->>L: { token, user{role} }
    L->>S: 保存 token
    alt 角色属于当前应用
        L->>L: 跳转到本应用首页
    else 角色属于其他应用
        L->>A2: window.location.href 跳转应用B
```

```

A2->>S: 读取同一 token
A2->>B: GET /api/auth/profile 校验
B-->>A2: 用户信息, 进入应用
end

```

这个跳转逻辑在 `packages/shared/src/template/LoginPage.vue` 里, 跳转目标地址在 `packages/shared/src/roles.ts` 的 `ROLE_APP_URLS` 映射表里 (`dev` 是 `http://localhost:517x`, `prod` 是 `/应用路径`)。

2.3.3 1.3.3 为什么端口是 5173~5179

Vite 默认端口就是 5173, 后面的应用依次递增, 避免 `dev` 时端口冲突。7 个应用可以同时启动, 各自连接同一个后端 (3000 端口)。这在调试时很方便: 开 7 个终端, 每个 `npm run dev`。

2.4 1.4 后端模块总览

2.4.1 1.4.1 目录树 (去注释版)

```

src/
├─ main.rs                # 程序入口: 启动编排 (日志/配置/数据库/CORS/路由/静态托管)
├─ routes.rs              # 路由集中注册: 把 9 组子路由拼成一个 Router
├─ roles.rs               # 角色注册表 ROLE_REGISTRY (RBAC 的唯一源)
├─ database.rs            # 建表 + 种子数据 (没有 sqlx 迁移文件, 全在这里)
├─ api_docs.rs            # utoipa OpenAPI 聚合 + export_openapi 防漂移测试
├─ config.rs              # 读取环境变量 (PORT/DATABASE_URL)
├─ embedded_assets.rs     # 单 exe 模式: 编译期内嵌 7 个前端 dist
├─
├─ common/                # ★ 公共基础设施 (所有角色共用)
│   ├── mod.rs            # 子模块声明 + /health 健康检查
│   ├── models.rs         # Permission 枚举 + User 模型 + ApiResponse 统一响应
│   ├── middleware.rs     # auth / permission / role 三个鉴权中间件
│   ├── jwt.rs            # JWT 签发与验证
│   ├── error.rs          # AppError 统一错误类型
│   ├── dto.rs            # 公共响应 DTO
│   ├── ws.rs             # WebSocket 事件桥 (broadcast → WS 客户端)
│   ├── auth/             # 登录/用户信息 (routes/handlers/services)
│   ├── service.rs        # ServiceRuntime 服务启停线程管理
│   ├── io.rs             # IoControl trait (串口/模拟统一抽象)
│   ├── frame_extractor.rs # 字节流→定长帧提取器
│   ├── quad_frame.rs     # 四槽帧缓冲 (主备切换)
│   ├── latest_frame.rs   # 最新帧跟踪器
│   ├── global_var.rs     # 全局 KV 存储 (试验信息等)
│   ├── ledger.rs         # 设备台账演示数据
│   ├── csv_writer.rs     # CSV 批量写入器
│   ├── config.rs         # INI 配置封装
│   ├── utils.rs          # hex/时间/CSV 工具函数
│   └─ least_squares.rs   # 最小二乘拟合
├─
├─ admin/                 # 用户管理 (SystemAdmin 权限)
│   ├── routes.rs         # 路由: read/write/delete 三组 + 双层中间件
│   ├── handlers.rs       # 4 个 handler
│   └─ services.rs        # UserAdminService
├─
├─ fj200c_information/    # 发动机监控 (最典型的"带硬件"模块)
│   ├── mod.rs            # 事件枚举 + 广播通道
│   ├── state.rs          # 全局状态 (服务运行标志 + 16 字段共享数据)
│   ├── config.rs         # 配置单例 (config-fj200c_information.ini)
│   ├── decode.rs         # 100 字节帧校验 + 28 字段解码 + CSV 表头
│   ├── frame_bundle.rs   # 帧复合存储
│   └─ com.rs             # 串口控制

```

```

├─ mock.rs          # 进程内模拟数据源（20Hz 正弦+噪声）
├─ mock_feeder.rs   # 虚拟串口对发生器
├─ session.rs       # 每连接 IO 会话线程（核心数据流）
├─ service.rs       # 服务启停编排（8 路连接）
├─ handlers.rs      # 8 个 HTTP + WS
├─ routes.rs        # 子路由 + 权限中间件

├─ fj200c_main/     # 发动机测控（最复杂模块，三路串口）
├─ mod.rs / state.rs / config.rs
├─ types.rs         # ECU/ADAM/DYNO 三类字段模型 + 64 列 CSV
├─ abstract_com.rs  # ComSpec 协议规格 + AbstractCom
├─ com.rs           # 三路串口实现（宏生成）
├─ decode.rs / mock.rs / report.rs
├─ service.rs / handlers.rs / routes.rs

├─ ftj1c/           # UDP 组播通信监控
├─ mod.rs / state.rs / config.rs / models.rs
├─ udp.rs           # UDP 控制（Mock/Real 双模式）
├─ process.rs       # 主备/单路/模拟 8 个运行函数
├─ quad_frame.rs / com.rs / service.rs / handlers.rs / routes.rs

├─ fw100/ fw150/    # 设备台账（极简：各 4 个文件）
├─ routes.rs / handlers.rs / services.rs / mod.rs

├─ city3d/          # 城市 3D 数据管理
├─ models.rs        # Building/District/CityEvent/Overview
├─ routes.rs / handlers.rs / services.rs / mod.rs

├─ role_template/   # ★ 新角色参考模板（自带 5 步启用说明）

```

2.4.2 1.4.2 模块的三种“体型”

看目录就能发现业务模块分三种复杂程度，理解这点对读代码很重要：

体型	模块	特征
极简型（约 100 行/模块）	fw100、fw150	只有 GET /items 一个接口，返回演示数据
中型	admin、city3d	纯数据库 CRUD，无硬件、无 WS
大型（带硬件+WS）	fj200c_information、fj200c_main、ftj1c	有串口/UDP/模拟数据源、服务启停、WS 广播、CSV 记录

新手接手时，**先读极简型建立模式感**（fw100 四个文件 15 分钟读完），再读中型（admin 理解权限+CRUD），最后啃大型（fj200c_information 是最佳的“硬件模块范本”）。

2.4.3 1.4.3 统一的分层模式

每个业务模块都遵循同样的三层结构：

flowchart TD

```

Client[浏览器] -->|HTTP 请求| R[模块 routes.rs<br/>定义路由 + 挂权限中间件]
R -->|鉴权通过| H[handlers.rs<br/>解析请求参数<br/>调用服务<br/>返回 ApiResponse]
H --> S[services.rs<br/>业务逻辑<br/>SQL/文件/硬件操作]
S --> DB[(SQLite)]
S --> HW[串口/UDP/Mock]
H -.WS 推送.-> B[mod.rs 广播通道]
B -.broadcast.-> W[common/ws.rs<br/>WS 连接会话]

```

- **routes.rs**：只做路由与中间件编排，不写业务。

- **handlers.rs**: 参数提取 (State/Json/Query/Path)、调用 service、包 ApiResponse。带 `#[utoipa::path]` 注解。
- **services.rs**: 纯业务逻辑，可复用。
- **mod.rs**: 模块声明 + 事件枚举 + 广播通道等“模块级基础设施”。

2.5 1.5 一次完整请求的全链路 (HTTP 版)

以“admin 登录后查询用户列表”为例，走一遍完整链路。这是全项目最核心的理解，务必吃透：

```
sequenceDiagram
    participant B as 浏览器 (localhost:5174)
    participant V as Vite dev server (5174)
    participant A as Axum (127.0.0.1:3000)
    participant M as 中间件链
    participant H as handlers
    participant S as services
    participant D as SQLite

    B->>V: GET /api/users (开发时浏览器直接请求 5174 端口的 /api 路径)
    V->>A: 代理转发到 localhost:3000/api/users (vite proxy, ws:true 无需)
    A->>M: 请求进入路由 /api/users (admin_routes)
    M->>M: users_read_middleware → permission_middleware(UsersRead) <br/> 1. 取
    Authorization: Bearer xxx<br/> 2. jwt::verify_token 验证<br/> 3. 查数据库用户<br/> 4.
    user.has_permission(UsersRead)?<br/> 5. 注入 Extension<User>
    M->>H: 放行, request.extensions 里有 user
    H->>S: UserAdminService::list_users(&db)
    S->>D: SELECT * FROM users ORDER BY created_at DESC
    D-->>S: 用户行数据
    S-->>H: Vec<User>
    H-->>A: Json(ApiResponse::success(users))
    A-->>V: HTTP 200 { success: true, data: [...] }
    V-->>B: 响应体
    Note over B: axios 拦截器解开 ApiResponse<br/>组件拿到 response.data 渲染表格
```

要点提炼：

1. **开发时**浏览器访问的是前端端口，/api 路径由 Vite 代理转发到后端 (vite.config.ts 的 server.proxy)；**生产时**前端被后端托管，不存在跨域问题。
2. **每个请求必须带** Authorization: Bearer <token> 头，由前端 axios 请求拦截器自动附加。
3. **中间件链按顺序执行**：auth 鉴权 → 权限检查 → 业务 handler。不同路由挂不同中间件组合。
4. **所有响应统一包装成** { success, message, data } (ApiResponse<T>)，前端统一用 response.success / response.data / response.message。
5. **任何错误** (数据库异常、权限不足、参数错误) 都转成 AppError，最终输出同样的 JSON 结构，前端错误处理逻辑统一。

2.6 1.6 WebSocket 实时数据链路 (硬件模块)

硬件模块 (发动机监控/测控、UDP 监控) 需要把设备数据**实时推**到浏览器，HTTP 轮询不够，于是用 WebSocket。链路如下：

```
sequenceDiagram
    participant S as service.rs 启停服务
    participant T as 采集线程 (std::thread)
    participant B as 广播通道 broadcast::Sender (容量1024)
    participant W as common/ws.rs ws_bridge
    participant F as 浏览器 WebSocket
```



```

S->>T: 启动 8 路采集线程（串口/模拟）
loop 每帧数据
    T->>T: 帧提取 + 解码
    T->>B: tx.send(Fj200cInformationEvent::Frame)
end
F->>W: 建立 WS 连接（?token= 鉴权）
W->>W: 订阅 broadcast::Receiver（从 Sender clone）
loop 持续推送
    B->>W: 事件（若有）
    W->>F: JSON 文本消息
    F->>F: useFj200cInformationEvents 分发到 store/组件
end
Note over W,F: 客户端太慢? Lagged 事件直接丢弃，不阻塞采集

```

关键设计点（后面章节会逐行展开）：

- **线程模型**：采集线程是普通 `std::thread`（阻塞式串口读），HTTP/WS 是 `tokio` 异步，两者用 `broadcast::channel(1024)` 桥接，互不阻塞。
- **连接时先发快照**：WS 连接建立后先推一帧当前数据快照，前端刷新页面立刻看到最新状态（而不是等下一帧）。
- **token 走查询参数**：浏览器 WebSocket 无法自定义 HTTP 头，所以 `ws://...?token=xxx`，服务端升级前验证。
- **前端自动重连**：断开 1.5 秒后自动重连，生产环境网络抖动不会白屏。

2.7 1.7 数据存储全景

系统有四种数据形态，分工明确：

```

flowchart LR
    subgraph SQLite[rustweb.db<br/>结构化业务数据]
        A1[users 用户表]
        A2[user_settings 用户设置]
        A3[city3d_districts 区域]
        A4[city3d_buildings 建筑]
        A5[city3d_events 事件]
    end
    subgraph INI[config-*.ini<br/>设备通信配置]
        B1[config-fj200c_information.ini<br/>热加载]
        B2[config-fj200c_main.ini<br/>需重启]
        B3[config-ftj1c.ini<br/>需重启]
    end
    subgraph CSV[csv/ 目录<br/>试验数据]
        C1[SYSJSK 开始的试验.csv<br/>按帧追加行]
    end
    subgraph ENV[.env 环境变量]
        D1[PORT / DATABASE_URL<br/>JWT_SECRET / RUST_LOG]
    end

```

数据形态	存什么	谁写	谁读	位置
SQLite	用户、角色、城市 3D 业务数据	services 层	前端 API 调用	运行目录 rustweb.db
INI	硬件连接与模拟 开关配置	前端配置页 (HTTP PUT 保存) /手工编辑	服务启动/热加载	运行目录 config-*.ini
CSV	试验过程数据	采集线程 (500ms 批量 flush)	前端数据页 + 报表	csv/ 目录
.env	运行参数	手工编辑/ deploy.bat 生成	main.rs 启动时	运行目录 .env

注意：数据库建表与种子数据没有迁移文件，全部在 src/database.rs 里用代码建（幂等：CREATE TABLE IF NOT EXISTS + INSERT OR IGNORE）。删库重跑会自动重建并填充种子账号。

2.8 1.8 目录逐层解析（动手对照）

现在打开你的 VS Code，把项目根目录展开，逐项对照：

2.8.1 1.8.1 根目录文件

文件/目录	是什么	需要注意
Cargo.toml	Rust 后端依赖清单	embedded feature 用于单 exe 打包
package.json	npm workspaces 根	8 个 workspace + gen:api 脚本
orval.config.ts	OpenAPI→TS 代码生成配置	只由 npm run gen:api 驱动
deploy.bat	一键部署脚本	顺序不可颠倒（先前端后后端）
src/	后端源码	主战场
frontend/	7 个前端应用	主战场
packages/shared/	共享包	公共逻辑集中地
openapi/	生成的 OpenAPI spec	勿手改 ，由测试生成
config-fj200c_information.ini 等	开发时配置	运行目录即根目录，服务读这里
rustweb.db*	开发数据库	可随时删，重启自动重建
csv/	CSV 输出目录	可清空
deploy/	部署产物	git 忽略，deploy.bat 生成
docs/	设计文档	现在包含本套教程
AGENTS.md	项目说明	最重要的架构文档 ，本教程与其一致

2.8.2 1.8.2 一个前端应用的内部结构（以 fj200c_information 为例）

```
frontend/fj200c_information/
├─ vite.config.ts      # 端口 5173、base、@shared alias、/api 代理（ws:true）
├─ tsconfig.json       # strict + paths 映射 @/* 和 @shared/*
```

```

├─ package.json          # 依赖 (vue/pinia/router/element-plus/echarts)
├─ index.html            # Vite 入口 HTML
├─ src/
│   ├─ main.ts           # 创建应用: Pinia + Router + ElementPlus
│   ├─ App.vue           # 根组件: AppNavbar + router-view
│   ├─ style.css         # 全局样式
│   ├─ router/index.ts   # 路由表 + 守卫
│   ├─ stores/auth.ts    # 认证 store (19 行, 工厂生成)
│   ├─ api/index.ts      # api 实例 + setApiInstance + api 对象
│   ├─ api/fj200c_information.ts # facade: HTTP + WS 封装
│   ├─ types/index.ts    # re-export @shared/types
│   ├─ utils/responsive.ts
│   ├─ fj200c_information/
│   │   ├─ components/   # ★ 模块业务代码 (与 src 平级子目录)
│   │   │   └─ UI 组件
│   │   ├─ composables/  # 组合式函数 (WS/服务/时钟/命令)
│   │   └─ utils/        # hex/校验工具
│   └─ views/            # 页面
│       ├─ Login.vue
│       └─ fj200c_information/ # Monitor/Visual/Data/Config/Help

```

fj200c_information/ 业务子目录与 src/ 平级是刻意设计: 业务代码 (组件、组合式函数、工具) 与框架代码 (main/router/store/api) 分离, 保持页面层很薄。

2.8.3 1.8.3 packages/shared 结构

```

packages/shared/src/
├─ index.ts              # 统一出口 (对外 API)
├─ roles.ts              # 角色菜单注册表 MENU_CONFIG + 应用地址 ROLE_APP_URLS + 注册表加载
├─ types.ts              # re-export generated 类型 + MenuItem
├─ session.ts            # localStorage 会话 + buildWebSocketUrl
├─ api/
│   ├─ index.ts          # createApiClient: axios 工厂 (token 注入/401 处理)
│   ├─ auth.ts           # 登录/用户信息请求封装
│   ├─ custom-instance.ts # orval mutator: 所有生成请求走这里
│   └─ generated/
│       ├─ api/<tag>/*.ts # 9 个 tag 的请求函数
│       └─ model/*.ts    # 100+ 类型文件
├─ stores/auth.ts        # createAuthStore 工厂 (全部应用的认证逻辑)
└─ template/
    ├─ AppNavbar.vue     # 通用导航栏 (690 行, 含自动隐藏)
    ├─ LoginPage.vue     # 通用登录页 (683 行, 宇航员动画)
    └─ TemplatePanel.vue # 模板演示面板

```

2.9 1.9 构建与部署全景

2.9.1 1.9.1 开发模式 (日常开发)

```

flowchart LR
    subgraph 终端1[终端 1: 项目根]
        A[cargo run] -->|监听| B[127.0.0.1:3000]
    end
    subgraph 终端2[终端 2: frontend/xxx]
        C[npm run dev] -->|监听| D[localhost:517x]
    end
    D -->|/api 代理| B
    B -->|SQLite/INI/CSV| E[运行数据]
    D -->|HMR 热更新| F[浏览器]

```

开发时后端读根目录 rustweb.db、根目录三个 ini; 前端静态资源由 Vite 自己服务, /api 代理转发。后端只负责 API, 前端自己渲染自己。

2.9.2 1.9.2 生产模式 (单 exe)

flowchart TD

```
A[deploy.bat 开始] --> B[7 个前端依次 npm run build]
B --> C[dist 产物就绪]
C --> D[cargo build --release --features embedded]
D --> E[rust-embed 把 7 个 dist 编译期内嵌进 exe]
E --> F[组装 deploy/ 目录 exe + .env + 3 个 ini + csv/]
F --> G[双击 rust-web-backend.exe]
G --> H[127.0.0.1:3000/admin /fj200c_information ...]
```

生产模式后端同时是静态文件服务器：访问 /fj200c_information 直接返回内嵌的前端页面 (SPA 深链接回退 index.html)。**顺序不可颠倒**——前端必须先构建，因为 dist 是在编译期被嵌进去的。

2.10 1.10 权限体系全景 (RBAC)

权限是本项目的骨架，几乎所有改动都涉及它。先建立整体概念：

flowchart TD

```
DB["users 表 user.role = 'admin' / 'fj200c_information' / ..."]
DB -->|GET /api/meta/roles| REG["roles.rs ROLE_REGISTRY key + name + permissions"]
REG -->|运行时拉取| FE["前端 loadRoleRegistry 缓存 RoleInfo[]"]
REG -->|"permissions_for(role)"| BP["后端中间件 user.has_permission(p)"]
FE -->|getPermissionsByRole| FM["菜单过滤 按钮权限控制"]
FE -->|getMenusByRole| MENU["MENU_CONFIG 菜单树 前端本地概念"]
BP -->|403 拒绝| R["业务接口"]
```

核心要点 (后面 03/05 章会逐行展开)：

1. **角色注册表唯一源在后端** src/roles.rs: RoleDef { key, name, permissions } 数组，7 个角色。
2. **用户只有角色字段**：users.role = 'fj200c_information'，权限是查注册表推出来的，不存数据库。
3. **前端菜单是纯 UI 概念**：packages/shared/src/roles.ts 的 MENU_CONFIG 手写，按权限过滤后渲染导航栏。
4. **权限点枚举** Permission (后端 src/common/models.rs)：10 个值，前后端类型同步。
5. **三层落地**：路由层中间件 (后端强制)、路由守卫 (前端路由)、按钮禁用 (前端 UI)。

2.11 1.11 本章小结与自测

读到这里，你应该能回答：

1. 这个系统有几个前端应用？为什么是 7 个？各自什么端口？
2. 后端有几个模块？分哪三种体型？
3. 一次 HTTP 请求要经过哪些层？统一响应结构是什么？
4. WS 数据是怎么从采集线程到浏览器的？中间隔着什么？
5. 前后端类型契约是怎么保证的？
6. 开发模式和生产模式静态资源分别由谁提供？
7. RBAC 的角色注册表在哪里？前端菜单又在哪里？谁驱动谁？

如果都能答上来，恭喜你，地图已经建立。下一章进入第一片主战场：Rust 语法速成 (以本项目代码为教材)。

2.12 1.12 深入：Cargo.toml 依赖逐个讲（新手对照表）

新手打开 Cargo.toml 往往一头雾水，这里把每个依赖讲透。这些依赖分成六组，每一组对应一类能力：

---- 第一组：Web 框架三件套 ----

```
axum = { version = "0.7", features = ["ws"] }      # Web 框架本体 + WebSocket 支持
tokio = { version = "1.0", features = ["full"] }   # 异步运行时 ("full" 全部功能)
tower-http = { version = "0.5", features = ["cors", "fs"] } # CORS 中间件 + 静态文件服务
futures-util = "0.3"                               # WS 消息流操作 (StreamExt 扩展)
```

依赖	类比（你熟悉的语言）	在本项目中的角色
axum	Express / Flask / Spring MVC	路由注册、请求处理、WS 升级
tokio	Node.js 事件循环 / asyncio	所有 async fn 的调度器；HTTP 服务建立
tower-http	CORS 中间件库	开发时前端跨域放行；dev 模式静态文件服务
futures-util	—	WebSocket 双向流需要 .next() 取消息

---- 第二组：数据与序列化 ----

```
serde = { version = "1.0", features = ["derive"] } # 序列化 (JSON 转换)
serde_json = "1.0"                                # JSON 具体实现
sqlx = { version = "0.7", features = ["runtime-tokio-rustls", "sqlite", "chrono", "uuid"] }
csv = "1"                                           # CSV 读写
configparser = "3"                                 # INI 配置解析
```

依赖	类比	说明
serde	JSON.stringify / Gson	#[derive(Serialize, Deserialize)] 后结构体可自动转 JSON
sqlx	手写 SQL 的连接池 + PreparedStatement	不是 ORM! 本项目手写 SQL 字符串
csv	pandas.to_csv	试验数据记录
configparser	configparser (Python)	读 config-*.ini

---- 第三组：认证与安全 ----

```
jsonwebtoken = "9.2" # JWT 签发/验证
bcrypt = "0.15"      # 密码哈希（不可逆）
validator = { version = "0.16", features = ["derive"] } # 输入校验
uuid = { version = "1.0", features = ["v4", "serde"] } # 主键
chrono = { version = "0.4", features = ["serde"] }     # 时间
dotenv = "0.15"   # .env 环境变量
```

---- 第四组：日志 ----

```
tracing = "0.1" # 日志门面（类似 log4j/slf4j）
tracing-subscriber = { version = "0.3", features = ["env-filter"] } # 日志输出实现
```

RUST_LOG=debug cargo run 就能看到调试日志，这是后端调试的第一手段。

---- 第五组：硬件与通信 ----

```
serialport = "4" # 串口 (COM3、115200 波特率...)
socket2 = "0.5"  # UDP 底层 socket 选项 (SO_REUSEADDR 组播)
arc-swap = "1"   # 原子交换指针（无锁共享最新数据）
rand = "0.8"     # 模拟数据随机数
```

```
# ---- 第六组: OpenAPI 与打包 ----
utoipa = { version = "5", features = ["chrono", "uuid"] } # 代码→OpenAPI 文档
rust-embed = "8" # 编译期把前端 dist 嵌入 exe
mime_guess = "2" # 根据扩展名猜 Content-Type
```

给新手的读依赖建议：不需要记住所有依赖，只需要知道“我想做什么事 → 查对应依赖”：写接口找 axum、查数据库找 sqlx、解析配置找 configparser、日志找 tracing、硬件找 serialport/socket2。

2.13 1.13 深入：运行时进程与线程模型

后端是一个进程（rust-web-backend.exe），内部线程分成三大类，理解这个模型对排查“为什么卡住/为什么重启不了”至关重要：

```
flowchart TD
    subgraph 进程["rust-web-backend.exe (一个进程)"]
        subgraph T1["tokio 异步运行时线程池 (默认 CPU 核数)"]
            H1[HTTP 请求处理<br/>所有 handler 都在这里]
            W1[WS 会话<br/>ws_bridge 循环]
            DB1[SQLite 异步查询]
        end
        subgraph T2["业务 std::thread 线程 (独立创建)"]
            C1[采集线程x8<br/>fj200c_information]
            C2[三路串口线程<br/>fj200c_main]
            C3[UDP 收发线程<br/>ftj1c]
        end
        subgraph T3["tokio 内部线程"]
            B1[broadcast 通道 (容量 1024)]
        end
    end
    C1 -.send.-> B1
    C2 -.send.-> B1
    C3 -.send.-> B1
    B1 -.recv.-> W1
    W1 -.推送.-> Browser[浏览器]
```

关键结论：

1. **采集线程阻塞是正常的：**串口 read 是阻塞式 API，所以硬件代码用 std::thread 而不是 async——卡住采集线程不会卡 HTTP。
2. **broadcast 是桥：**两套线程模型之间只有广播通道这一个通信点，采集线程 send 永不阻塞（除非 1024 个消费者都太慢导致被丢弃）。
3. **重启服务 = join 线程：**stop_service 把停止标志置 true，线程循环退出，主线程 join 等待，超时 3 秒强杀（wait_stopping）。
4. **HTTP 是 async 的：**handler 里如果做 CPU 密集（如 bcrypt 校验密码），用 spawn_blocking 丢到阻塞线程池，避免卡住整个 HTTP 服务——src/common/auth/services.rs 里就有这个例子。

2.14 1.14 深入：7 个前端应用逐个认识

2.14.1 1.14.1 admin（管理后台，端口 5174）

```
flowchart LR
    L[Login.vue] --> U[Users.vue 用户列表]
    L --> C[CreateUser.vue 创建用户]
    U --> D[编辑角色对话框]
    U --> E[删除确认]
    subgraph 权限
```

```

F[UsersRead 查看]
G[UsersWrite 创建/改角色]
H[UsersDelete 删除]
end

```

页面：- **Users.vue** (507 行)：用户表格 + 搜索框（用户名/邮箱）+ 角色筛选下拉 + 分页 + “编辑角色”对话框 + “删除”按钮。按钮用 `authStore.hasPermission(...)` 控制可用性。
- **CreateUser.vue** (260 行)：创建用户表单（用户名/邮箱/密码/角色），角色下拉来自 `getAllRoles()`（后端注册表驱动，新角色自动出现）。- **NoPermission.vue**：403 页面，权限不足时跳到这里（防止守卫死循环）。

特点：admin 是唯一 `appKind: "admin"` 的应用，菜单来自 `adminMenus`，登录后跳 `/users`。

2.14.2 1.14.2 fj200c_information (发动机监控, 端口 5173)

```

flowchart LR
    M[Monitor 实时监控] --> V[Visual 可视化]
    M --> D[Data 数据浏览]
    M --> C[Config 配置编辑]
    M --> H[Help 帮助]
    subgraph Monitor 组成
        CP[CommandPanel 命令通道]
        DP[DataPanel 解码表格]
        SN[ServiceNavButton 启停]
        SB[StatusBar 状态栏]
    end
end

```

- **Monitor.vue** (411 行)：实时监控主界面。顶部启停服务按钮，中间命令通道（可发自定义 hex 指令），下方解码数据表格（28 个字段），底部状态栏（服务状态/连接/时钟）。
- **Visual.vue**：ECharts 六仪表盘 + 实时曲线。
- **Data.vue**：左侧 CSV 文件列表，右侧按列渲染表格（文件名防穿越由后端保证）。
- **Config.vue**：读取/编辑/保存 `config-fj200c_information.ini`（热加载，保存立即生效）。
- **Help.vue**：帮助说明页。

2.14.3 1.14.3 fj200c_main (发动机测控, 端口 5179) ——最复杂

```

flowchart LR
    M[Monitor 仪表盘] --> E[ExperimentInput 试验录入]
    M --> EV[ExperimentView 试验查看]
    M --> R[GenerateReport 报表]
    M --> D[Data 数据]
    M --> C[Config 配置]
    M --> H[Help 帮助]
    subgraph Monitor
        G[GaugeCard 仪表×若干]
        CH[ChartPanel 实时曲线]
        CP[ControlPanel 控制面板]
        E2[ECUStatus]
        FD[FaultDisplay 故障码]
        SS[ScaledPage 1920×1080 舞台]
    end
end

```

特点：- **ScaledPage**：1920×1080 设计稿缩放容器，任意分辨率下布局不变形。- **双主题**：深色/浅色主题 CSS 变量，主题状态存服务端（WS 广播同步所有页面）。- **三路数据**：ECU（发动机）/ADAM（采集模块）/DYNO（测功机）三路串口，dashboard store 统一管理。
- **报表**：试验数据 → 状态点插值 → 报表生成（原生打印方案）。- **App.vue 级 WS 常驻**：模块级单例 WS + 引用计数（05 章详述），切页面不断连。

2.14.4 1.14.4 fw100 / fw150 (设备台账, 端口 5175/5178)

最简应用: 登录 → 一个 Panel.vue (138 行) → 表格展示 GET /api/fw100/items 返回的演示台账数据。后端 common/ledger.rs 生成演示数据。**两个应用代码几乎一模一样**, 只是路径/端口/类型名不同 (fw150 有独立 Fw150LedgerItem schema)。

2.14.5 1.14.5 ftj1c (UDP 通信监控, 端口 5176)

单页应用: Monitor.vue (559 行) 4×4 卡片网格 (8 路连接 × 2 张卡片), 显示每路 UDP 连接的收发状态与帧内容; 顶部服务控制 (启停); 配置对话框 (读写 config-ftj1c.ini)。WS 连接由页面内建立 (组件级连接模式)。

2.14.6 1.14.6 city3d (城市 3D, 端口 5177)

- **CityScene.vue** (999 行): 全屏 Three.js 场景 (建筑/星空/粒子/Bloom 光效/昼夜循环/天气/热力) + 玻璃拟态 HUD 覆盖层 (统计卡片、区域列表、事件流、建筑悬停浮窗)。
- **DataPanel.vue**: 数据管理 (区域/建筑/事件 CRUD)。
- **Panorama.vue**: 全景展示页。
- 数据 5 秒轮询 (useCityData), 场景渲染循环在 useCityScene (1200 行)。

2.15 1.15 深入: 后端“带硬件”模块的共同骨架

fj200c_information / fj200c_main / ftj1c 三个模块虽然业务不同, 但骨架完全一致。掌握这个骨架, 等于同时掌握三个模块:

flowchart TD

```
subgraph 配置
    CF[config.rs<br/>Config 全局单例<br/>OnceLock]
    CFI[config-*.ini]
end
subgraph 服务生命周期
    S[start_service<br/>读配置→起线程]
    ST[stop_service<br/>置标志→join 3s]
    STT[SERVICE_RUNNING<br/>AtomicBool]
end
subgraph 采集
    IO[IoControl trait<br/>send/recv/set_timeout]
    REAL[串口实现/UDP 实现]
    MOCK[Mock 实现<br/>模拟数据]
    FE[FrameExtractor<br/>字节流→帧]
    DE[decode<br/>帧→字段]
end
subgraph 发布
    EV[mod.rs 事件枚举]
    TX[broadcast Sender]
end
subgraph HTTP/WS
    RT[routes.rs 权限+WS]
    HD[handlers.rs 8~15 个接口]
    WB[ws_handler<br/>token 校验→快照→ws_bridge]
end
subgraph 持久化
    CSVW[csv_writer<br/>500ms 批量]
end
S --> IO
S --> ST
CF --> IO
CFI --> CF
IO --> REAL
IO --> MOCK
```



```
REAL --> FE
MOCK --> FE
FE --> DE
DE --> EV
EV --> TX
TX --> WB
TX --> CSVW
RT --> HD
HD --> S

每个模块的 mod.rs 里都有一个 xxx_tx() 函数返回全局唯一的 broadcast Sender:

pub static FJ200C_INFORMATION_TX: OnceLock<broadcast::Sender<Fj200cInformationEvent>> =
OnceLock::new();

pub fn fj200c_information_tx() -> broadcast::Sender<Fj200cInformationEvent> {
    FJ200C_INFORMATION_TX
        .get_or_init(|| broadcast::channel(1024).0) // 首次调用创建，之后复用
        .clone() // 返回克隆（引用计数 +1），多线程各自持有
}
```

这段代码在 src/fj200c_information/mod.rs，值得背下来——它是整个广播架构的地基。

2.16 1.16 本章必读文件清单（先读这 6 个文件）

在进入语法章节前，强烈建议先打开这 6 个文件通读一遍（都是小文件，总计不到 700 行）：

优先级	文件	为什么先读它
★★★★	src/main.rs	程序入口，整个启动流程
★★★★	src/routes.rs	路由全景，所有接口的挂载点
★★★★	src/roles.rs	RBAC 唯一源，7 个角色定义
★★☆	src/common/models.rs	Permission 枚举 + ApiResponse + User
★★☆	src/common/jwt.rs	JWT 签发验证（80 行小文件）
★★☆	src/database.rs 前 100 行	建表逻辑 + 种子账号

读完这 6 个文件，你对后端的骨架就有感觉了；02 章语法速成里会逐行拆解其中的关键代码。

2.17 1.17 深入：安全设计概览（接盘前必知）

本项目虽在工业内网运行，安全设计仍然完善，接盘维护时**不要破坏这些防线**。逐条列出，后续章节会看到具体实现：

2.17.1 1.17.1 认证与授权

防线	实现位置	说明
密码存储	src/common/auth/services.rs	bcrypt 哈希 (不可逆), User 模型 #[serde(skip_serializing)] 保证永不返回哈希
登录凭证	src/common/jwt.rs	HS256 签名 JWT, 24 小时过期 (可配), 密钥来自 JWT_SECRET 环境变量
接口鉴权	src/common/middleware.rs auth_middleware	每个受保护路由必经: 验证 Bearer token → 查用户 → 注入 Extension
角色权限	permission_middleware	检查用户是否有特定 Permission, 无则 403
前端路由守卫	各应用 router/index.ts	未登录跳登录页, 无权限跳回有权限的首页
前端按钮禁用	组件内 hasPermission	UI 层控制, 权限不足按钮置灰

2.17.2 1.17.2 数据安全

防线	实现位置	说明
CSV 文件名防穿越	fj200c_information/handlers.rs	校验文件名不含 ../, 防止 GET /csv/../../../../etc/passwd 类攻击
用户角色白名单	admin/handlers.rs	创建用户时角色必须在注册表内
不能删自己	admin/handlers.rs	删除/改角色时校验目标不是当前登录用户
不能移除自己的管理权限	同上	防止误操作把自己锁在门外
SQL 注入	sqlx 绑定参数 ?	所有查询用绑定参数, 不用字符串拼接
密码回显	User serde skip	序列化时排除密码字段
参数校验	validator crate	邮箱格式、必填等 LoginRequest 声明式校验
分页钳制	city3d/handlers.rs	page_size 上限钳制, 防大分页拖垮数据库

2.17.3 1.17.3 传输层

- 开发环境 CORS 全放行 (Any), 因为 dev 时前端端口不同; 生产环境同源 (后端托管前端), CORS 形同虚设但保留无妨。
- WS 连接用 ?token= 查询参数鉴权——浏览器 WS API 无法自定义请求头, 这是标准做法; **token 会出现在 URL 里**, 属可接受权衡 (内网 + 短有效期)。

2.17.4 1.17.4 部署安全

- `.env` 里的 `JWT_SECRET` 有默认值 (`your-secret-key`)，**生产环境必须改** (`deploy.bat` 生成的 `.env` 已带提示注释)。
- 服务只绑定 `127.0.0.1` (`src/main.rs`)，外网访问才需要改 `0.0.0.0`——默认绑回环是安全默认值。

2.18 1.18 深入：代码风格与阅读约定

这个项目是“教学型代码”：几乎每个语法点都有中文注释。了解它的风格约定，读代码事半功倍：

2.18.1 1.18.1 注释约定

```
// 行注释：解释“为什么”或“这一步在干嘛”
// 例如：// 顺序不可颠倒：前端 dist 在编译期内嵌进 exe，必须先构建前端再编译后端

/// 文档注释（三斜杠）：函数/类型说明，rust-analyzer 悬停可显示
/// 例如：/// 服务停止：等待线程优雅退出（最长 3 秒），超时强制终止
pub fn stop_service() { ... }

//! 模块级文档注释（文件顶部）：说明整个文件的用途
//! 例如：//! 发动机监控模块（从 fj200c.informatization 迁移）
```

2.18.2 1.18.2 命名约定

项	约定	例子
模块/文件	snake_case	fj200c_information、frame_extractor.rs
函数	snake_case	start_service、verify_token
结构体/枚举	PascalCase	ApiResponse、Fj200cInformationEvent
常量	SCREAMING_SNAKE	SERVICE_RUNNING、CONFIG_PATH
全局单例	类型名小写 + <code>_tx/_()</code>	fj200c_information_tx()
HTTP 路径	模块同名	/api/fj200c_information/service/start
前端组件	PascalCase	CommandPanel.vue
前端组合式函数	useXxx	useService、useBackendPorts

2.18.3 1.18.3 分层职责铁律

读代码时不断自问：“这段逻辑放在这一层对吗？”本项目的铁律：

- **handler 不写业务**：只做“取参数 → 调 service → 包装响应”。看到 handler 里出现复杂 SQL 或文件操作，就是坏味道。
- **service 不管 HTTP**：不碰 `Request/Response`，只处理数据。
- **routes 只编排**：挂中间件、拼路由，不出现业务。
- **mod.rs 是模块门面**：声明子模块 + 模块级基础设施（事件枚举、广播通道）。

2.18.4 1.18.4 错误处理约定

全项目统一 `Result<T, AppError> + ? 链`。看到 `?` 就理解为“出错就往上抛，交给统一错误处理器”：

```
let user = AuthService::login(&db, login_data)
    .await
    .map_err(|e| AppError::bad_request(e.to_string()))?; // 失败→400 错误
let token = jwt::create_token(&user)?; // 失败→500 错误
Ok(Json(ApiResponse::success(LoginResponse { token, user })))
```

前端看到的是：`{ success: false, message: "...", ... }`，永远一致。

2.19 1.19 深入：全局状态盘点（哪些“全局变量”要心里有数）

Rust 的全局状态用 OnceLock（惰性单例）承载，散落在各模块。接盘后排查问题常需要“全局状态地图”：

全局单例	类型	位置	用途
FJ200C_INFORMATION_broadcast	AtomicBool	src/fj200c_information/mod.rs	发动机监控广播
SERVICE_RUNNING	AtomicBool	src/fj200c_information/state.rs	服务运行标志
SHARED_DATA	SharedData (16 个 RwLock)	src/fj200c_information/state.rs	最新解码字段
CONFIG	OnceLock<Config>	src/fj200c_information/config.rs	配置单例
FJ200C_MAIN_broadcast	AtomicBool	src/fj200c_main/mod.rs	测控广播
GLOBAL_VAR	GlobalVar (KV 存储)	src/common/global_var.rs	试验信息/主题等
FTJ1C_TX	OnceLock<broadcast::Sender<Ftj1cEvent>>	src/ftj1c/mod.rs	UDP 广播
QUAD_FRAME	OnceLock<Arc<QuadFrame<95>>>	src/ftj1c/state.rs	最新帧
RUNTIME	ServiceRuntime	src/common/service.rs	全局线程句柄池
CSV_RECORDING	AtomicU8	src/fj200c_main/state.rs	CSV 录制状态机

特点总结：1. 全部惰性初始化（首次访问才创建）。2. 广播通道全部容量 1024，Sender::clone() 按引用计数共享。3. 可变全局状态用 RwLock/Atomic*，尽量少用 Mutex（读多写少场景）。4. 最新帧用 ArcSwap（无锁读，热替换），见 common/quad_frame.rs。

2.20 1.20 项目的时间线：git 历史透露了什么

新人接盘前扫一眼 git log，能快速了解项目演化脉络：

```
gitGraph
    commit id: "b2cac9e 用户操作手册"
    commit id: "3217a4b FJ-200C 迁移计划执行"
    commit id: "f725592 重命名 fj200c → fj200c_information"
    commit id: "153c1a0 merge"
    commit id: "8447ad6 统一登录页与导航栏"
    commit id: "af0e7cd 移除报表服务启动依赖"
    commit id: "b5dab30 对齐启动逻辑"
    commit id: "7059a63 清理废弃文档"
    commit id: "de2ad83 修复导航栏自动隐藏"
    commit id: "d51619c AppNavbar 双层结构重构"
    commit id: "4f8dfee 3D城市布局适配"
    commit id: "24e3b90 模块级 WS 引用计数"
    commit id: "6bae335 登录页 SVG 动画"
    commit id: "37433fb 密码框掩码"
```

可观察到的演化方向：1. **fj200c 一拆二**：早期只有一个 fj200c 模块，后来拆成 information（监控）+ main（测控）。2. **工程化治理**：逐步清理废弃文档、统一登录页/导航栏/布局、共

享组件化。3. **稳定性修复**：导航栏自动隐藏的 sticky 高度重算问题（两次修复）、WS 连接生命周期问题——这些是**前车之鉴**：改 shared 组件要小心，一次改动影响 7 个应用。

2.21 1.21 自测题答案与延伸阅读

2.21.1 1.21.1 自测题简答

1. **几个前端**？7 个：admin/fj200c_information/fj200c_main/fw100/fw150/ftj1c/city3d，端口 5173~5179（admin 5174）。
2. **后端模块**？11 个业务模块（admin/fj200c_information/fj200c_main/ftj1c/fw100/fw150/city3d + common + role_template + 顶层文件），分极简/中型/大型三种体型。
3. **请求链路**？浏览器 → Vite 代理 → Axum 路由 → 中间件链（JWT 鉴权 + 权限）→ handler → service → SQLite → 逐层返回，统一 ApiResponse<T> 包装。
4. **WS 链路**？采集线程（std::thread）→ broadcast 通道 → ws_bridge → 浏览器；连接时先发快照；token 走查询参数。
5. **类型契约**？utoipa 注解 → openapi.json → orval → generated TS 代码 → 前端 import。
6. **开发/生产静态资源**？开发：Vite dev server；生产：后端 rust-embed 内嵌 dist（或 dev 模式 ServeDir 磁盘 dist-*）。
7. **RBAC**？注册表唯一源在后端 roles.rs；前端菜单 MENU_CONFIG 是纯 UI 概念，按权限过滤；用户表只存 role 字段。

2.21.2 1.21.2 建议延伸阅读顺序

1. 把 src/main.rs、src/routes.rs、src/roles.rs 通读一遍（1.16 节清单）。
2. 启动项目（按 07 章操作），登录 admin，创建用户、切换角色体验权限差异。
3. 开 F12 Network 面板，登录时观察 POST /api/auth/login 请求与响应结构。
4. 进入 fj200c_information 应用，启动模拟服务，观察 WS 帧推送（Network 面板 WS 标签页）。

2.22 1.22 端到端案例走读：模拟数据从硬件层到浏览器页面

作为全景章节的收官，我们走一遍最典型的完整业务流——**fj200c_information 启动模拟服务后，数据如何从“虚拟串口”流到浏览器表格**。这个案例把本章讲的所有概念串成一条线：

2.22.1 第一步：前端点“启动服务”

用户在 Monitor.vue 顶部点“启动服务”按钮 → fj200cInformationApi.startService() → 请求 POST /api/fj200c_information/service/start。这个接口带 Fj200cInformationMonitor 权限，中间件放行后进入 handlers.rs 的 start_service handler：

```
pub async fn start_service(
    State(db): State<DatabaseConnection>,
) -> Result<Json<ApiResponse<ServiceStatus>>, AppError> {
    let config = crate::fj200c_information::config::get_config(); // 读 INI 单例
    crate::fj200c_information::service::start_service(&db, config.clone()); // 编排启动
    Ok(Json(ApiResponse::success(ServiceStatus { running: true })))
}
```

2.22.2 第二步：后端启动 8 路采集线程

service.rs 的 start_service 读取 INI（config-fj200c_information.ini），对每个 [ConnectionN] 节判断 Enabled，然后分支到真实串口或模拟器：

```
// 伪代码（service.rs 实际逻辑）
for connection in 0..8 {
```

```

if !config.is_enabled(connection) { continue; }
if config.mock_enabled() {
    let mock = MockControl::create(); // 模拟数据源 (20Hz 正弦+噪声)
    RUNTIME.push(spawn_thread(move || {
        run_one_connection(connection, mock, tx) // 会话线程
    }));
} else {
    let port = open_serial_port(connection)?; // 真实串口
    RUNTIME.push(spawn_thread(move || {
        run_one_connection(connection, port, tx) // 会话线程
    }));
}
}
}

```

关键点：模拟器实现了与串口相同的 `IoControl trait`——上层代码完全不知道数据来自哪里，这就是“Mock 模式开箱即用”的架构基础。

2.22.3 第三步：会话线程解码并广播

每个会话线程 (`session.rs`) 循环执行：读字节 → `FrameExtractor` 找帧头、拼帧、校验 → `decode` 解出 28 个字段 → 更新 `SHARED_DATA` 全局状态 → 构造 `Fj200cInformationEvent::TableData` 发进广播通道：

```

// 伪代码 (session.rs)
loop {
    let bytes = io.recv()?; // 阻塞读 (模拟器每 50ms 产生一帧)
    if let Some(frame) = frame_extractor.process(&bytes) { // 完整帧
        if let Some(row) = decode_shared_data(&frame) { // 28 字段
            *SHARED_DATA.update(...) = row; // 全局最新状态
            let event = Fj200cInformationEvent::TableData { row: row.clone() };
            if tx.send(event).is_err() { break; } // 没有订阅者就退出？不，is_err 表示所有接
收端关闭
        }
    }
    if SERVICE_RUNNING.is_stopped() { break; } // 停止标志轮询
}

```

2.22.4 第四步：WS 桥推送到浏览器

浏览器 `Monitor.vue` 挂载时建立 WS (`useFj200cInformationEvents`)，后端 `ws_handler` 验证 token 后启动 `ws_bridge`：订阅广播通道，把每个事件序列化为 JSON 文本帧推给客户端。连接建立时还会先发一个当前快照，页面刷新后立刻有数据。

2.22.5 第五步：前端分发到表格

```

sequenceDiagram
    participant WS as WebSocket
    participant Hook as useFj200cInformationEvents
    participant St as 组件内 refs
    participant T as DataPanel 表格
    WS->>Hook: JSON {type:"table_data", row:{...}}
    Hook->>Hook: switch(event.type) 分发
    Hook->>St: rows.value = [row, ...prev] (最新 100 行)
    St->>T: el-table :data="rows" 自动渲染

```

用户看到的效果：表格每 50ms 跳动一行 (20Hz 模拟帧)，状态栏显示“运行中”，点“停止服务”后线程优雅退出。

2.22.6 第六步：CSV 记录 (可选)

如果 INI 里 `[CSV] Enabled = true`，会话线程同时维护 CSV 状态机：收到 `SYSJSK` (试验开始) 帧 → 创建文件并写表头；`SYSJZJK` (数据中) 帧 → 每帧追加一行；`SYSJMK` (结束) 帧 → 关闭

文件。数据页 (Data.vue) 调 GET /api/fj200c_information/csv/files 列出文件, GET /csv/{name} 读取内容。

2.22.7 1.22.1 这个案例暴露的架构要点 (回顾清单)

- 1. **trait 抽象 (IoControl)** 让 Mock/真实硬件可替换——这是硬件模块的灵魂。
- 2. **状态机 (CSV)** 用事件驱动, 不写一堆 if 嵌套。
- 3. **全局状态 (SHARED_DATA)** 与广播 (TX) 分离: 一个给新连接发快照, 一个持续推送增量。
- 4. **线程 + 广播**是唯一数据通路, HTTP 请求 (启动/停止/状态) 与数据流互不干扰。
- 5. **前端页面很薄**: WS 收到事件 → store/refs 更新 → 组件自动渲染, 没有轮询、没有手动 DOM 操作。

2.23 1.23 补充: 7 个应用的页面与菜单全览

2.23.1 1.23.1 各应用导航菜单对照

应用	一级菜单	页面职责
admin	用户管理	用户列表/新建/编辑角色
fj200c_information	监控/可视化/数据记录/配置/帮助	实时表格、曲线、CSV、ini 配置
fj200c_main	主控台/试验/报表/设置	三路面板、试验信息、报表打印、主题
ftj1c	帧监控/IP 配置/帮助	帧表格、16 路组播配置
fw100	台账	设备列表/详情
fw150	台账	设备列表/详情
city3d	3D 视图/建筑/区域/事件/概览	三维场景与各类管理

共性规律: 都是「顶栏导航 + 内容区」骨架; 业务差异全在内容区 (表格/面板/3D)。

2.23.2 1.23.2 页面类型与数据源对照

页面类型	数据源	代表页面
实时型	WS 推送	fj200c_information/Monitor、fj200c_main 三路面板
列表型	HTTP 拉取	fw100/Panel、admin/Users
配置型	HTTP 读写 ini	fj200c_information/Config
展示型	静态/统计接口	city3d/Overview、各 Help 页

2.24 1.24 补充：目录逐层解析（后端 src 全貌）

2.24.1 1.24.1 顶层文件职责

文件	行数（约）	职责
main.rs	232	启动装配（dotenv/DB/pool/state/router/静态托管）
routes.rs	145	全部路由集中注册
roles.rs	268	ROLE_REGISTRY 角色注册表
database.rs	1269	建表 + 种子数据
api_docs.rs	247	OpenAPI 聚合 + 防漂移测试
config.rs	84	环境配置读取
embedded_assets.rs	131	单 exe 内嵌前端

2.24.2 1.24.2 common 公共层文件职责

文件	职责
middleware.rs	auth/permission/role 三中间件
error.rs	AppError 统一错误
models.rs	Permission 枚举 + 通用 DTO
jwt.rs	签发/校验
ws.rs	ws_bridge 广播桥
frame_extractor.rs	字节流切帧
quad_frame.rs	主备四源帧管理
global_var.rs	全局键值存储
csv_writer.rs	CSV 写入
io.rs	IoControl trait
service.rs	ServiceRuntime 启停

这张表可以当“后端文件地图”用——遇到任何文件先查这表知道它属于哪层。

2.25 1.25 补充：一次 HTTP 请求的生命周期（从浏览器到数据库）

sequenceDiagram

```
participant B as 浏览器
participant N as Vite proxy
participant A as Axum Router
participant M as 中间件
participant H as Handler
participant S as Service
participant D as SQLite
B->>N: GET /api/fw100/items（带 Bearer token）
N->>A: 转发到 :3000
A->>M: 匹配路由 → auth 中间件验 JWT
M->>M: 验权限（permission_middleware）
M->>H: 放行 → fw100_list_items
H->>S: list_items(&state.db)
S->>D: SELECT ...
D-->>S: 行数据
S-->>H: Vec<LedgerItem>
H-->>A: Json(ApiResponse::success(items))
A-->>B: {success, message, data}
B->>B: 前端 res.success 判断 → 渲染表格
```


一次请求 8 步，所有 HTTP 接口都是这个模式。掌握了这张图，你就掌握了后端的一切。

2.26 1.26 本章知识点串联测验

- 1. 7 个前端应用的端口分别是？
- 2. 前端 dev 模式如何访问后端？ (/api 代理到 :3000)
- 3. 数据流的两条通路是什么？ (HTTP 请求 + WS 推送)
- 4. 类型契约从哪来到哪去？ (Rust 注解 → openapi.json → orval → TS)
- 5. RBAC 的角色注册表唯一源在哪？ (src/roles.rs)
- 6. 单 exe 部署如何实现？ (embedded feature + rust-embed 内嵌 dist)
- 7. 三份 ini 配置文件各自管什么？
- 8. 种子账号有哪些？ (7 个，密码 123456)

答对全部 → 01 章地图建立完成，可以进入语法速成章节。

2.27 1.27 补充：后端代码组织哲学（为什么这么分层）

2.27.1 1.27.1 三层架构的动机

flowchart LR
A[Handler
薄：只做参数与响应] --> B[Service
厚：业务与数据]
A --> C[Models
DTO 契约]
B --> D[SQLite]

为什么不让 handler 直接写 SQL： 1. 复用——多个 handler 调同一个 service 函数。 2. 测试——service 逻辑可脱离 HTTP 测试。 3. 一致——错误处理/日志集中在 service。

项目实际：小模块 (fw100) service 很薄 (约 100 行)，大模块 (fj200c_main) service 厚 (200+ 行含会话管理)。分层程度随复杂度调整。

2.27.2 1.27.2 全局状态的组织方式

状态	存放	访问方式
数据库连接池	AppState	handler 的 State 提取器
服务运行标志	模块级 SERVICE_RUNNING	OnceLock
最新数据帧	SHARED_DATA	全局静态
事件通道	TX	全局静态
运行时配置	ArcSwap	热更新

规律：进程级共享用全局静态 (OnceLock)，请求级共享用 AppState。

2.28 1.28 补充：HTTP 与 WebSocket 的分工

2.28.1 1.28.1 什么时候用哪个

场景	用	例子
查询/操作	HTTP	台账 CRUD、启停服务
实时数据流	WS	帧数据、状态变化
大文件	HTTP	CSV 下载
配置	HTTP	ini 读写

2.28.2 1.28.2 WS 消息的三种类型

1. 快照（连接时发一次）→ 页面秒开有数据
2. 增量（帧事件）→ 表格滚动更新
3. 状态（服务状态变化）→ 状态栏刷新

2.28.3 1.28.3 WS 与 HTTP 的鉴权差异

HTTP: Authorization: Bearer <token> (header)

WS: 浏览器无法自定义 header → ?token= 查询参数

这是项目的一个设计细节——AGENTS.md 明确记录，buildWebSocketUrl 的实现就为此。

2.29 1.29 补充：前端工程化细节（构建与部署链路）

2.29.1 1.29.1 构建链路

源文件（.vue/.ts/.css）

→ Vite 打包（按路由分包）

→ dist/ 产物

→ rust-embed 编译期内嵌（embedded feature）

→ 单 exe 内内存服务

2.29.2 1.29.2 为什么需要前端构建

1. 浏览器不认 .vue/.ts → 需编译
2. 依赖需打包（Element Plus 等）
3. 按路由分包 → 首屏更快
4. 压缩/混淆 → 体积小

2.29.3 1.29.3 类型检查的位置

vue-tsc（构建前）→ 静态类型检查

rustc（编译时）→ Rust 类型检查

orval 生成 → 契约一致性

三层防线让“类型错误”几乎不可能跑到运行时。

2.30 1.30 补充：系统边界与限制（认识它才能用好它）

边界	说明
单机部署	SQLite + 内存服务，不适合集群
串口资源	每路连接独占一个串口
数据量	CSV 按帧写盘，长时间运行需管理
并发	广播模式，N 客户端共享
安全	内网工具型，无 HTTPS/加密存储

认识边界的意义：明确“哪些需求不能通过配置解决，需要改造”。

2.31 1.31 补充：01 章补充自测（10 题）

1. handler 与 service 的分工？
2. 全局状态两种存放方式？
3. HTTP 与 WS 各自适用场景？
4. WS 鉴权为什么用 query 参数？
5. 前端构建的必要性？
6. 类型检查的三层防线？
7. 系统的部署边界？
8. 快照与增量的区别？
9. 为什么 service 不直接放 handler 里？

10. AppState 与全局静态的区别？

答对 8+ → 01 章全面完成。

2.32 1.32 补充：7 个应用的页面与菜单全览（逐应用）

2.32.1 1.32.1 admin (5174)

登录页 → 用户列表 → 新增/编辑用户 → 角色分配 → 权限查看

2.32.2 1.32.2 fj200c_information (5173)

登录页 → 服务控制（启动/停止/状态）
→ 仪表盘（转速/水温/油压 等实时卡片）
→ 数据表格（逐帧参数）
→ 图表页（参数曲线）
→ CSV 记录（历史文件列表/下载）
→ 配置页（config-fj200c_information.ini 编辑）
→ 命令页（向设备发指令）

2.32.3 1.32.3 fj200c_main (5179)

登录页 → 总览（ECU/ADAM/DYNO 三路状态）
→ ECU 面板（发动机参数 + 指令）
→ ADAM 面板（模拟量采集）
→ DYNO 面板（测功机控制）
→ 试验信息（试验配置/记录）
→ 报表生成
→ 主题切换
→ CSV 64 列录制管理

2.32.4 1.32.4 fw100 / fw150 (5175 / 5178)

登录页 → 设备台账列表（分页/搜索/排序）
→ 新增/编辑/删除设备
→ 详情查看
→ CSV 导入导出（如有）

2.32.5 1.32.5 ftj1c (5176)

登录页 → 服务控制（UDP 启停）
→ IP 配置（16 路组播地址）
→ 帧数据表格（实时滚动）
→ 坐标展示（CGCS2000 转换结果）
→ 配置页（config-ftj1c.ini）

2.32.6 1.32.6 city3d (5177)

登录页 → 概览（建筑/区域/事件统计）
→ 3D 视图页（Three.js 场景）
→ 建筑管理（CRUD）
→ 区域管理（CRUD）
→ 事件管理（CRUD）

2.33 1.33 补充：目录逐层解析（前端通用结构）

```
frontend/<应用>/
├─ index.html           # SPA 入口
├─ vite.config.ts       # 端口/base/代理
├─ tsconfig.json        # TS 配置
├─ package.json         # 依赖与脚本
├─ public/              # 静态资源
├─ src/
│  └─ main.ts           # 应用入口（挂载/路由/初始化）
│  └─ App.vue           # 根组件
│  └─ router/           # 路由与守卫
```

— stores/	# Pinia 状态
— api/	# API facade
— composables/	# 组合式函数
— types.ts	# WS 事件等手写类型
— assets/	# 样式/图片
— components/	# 组件
— views/	# 页面

规律：7 个应用目录结构高度一致——学会一个，其余全会。

2.34 1.34 补充：后端进程的生命周期（一次完整运行）

```
sequenceDiagram
    participant OS as 操作系统
    participant M as main.rs
    participant D as database.rs
    participant R as routes.rs
    participant W as WebSocket
    OS->>M: 启动 exe
    M->>M: 读 .env (dotenv)
    M->>D: 初始化数据库（建表+种子）
    D-->>M: pool
    M->>M: 初始化各模块全局状态
    M->>R: 注册路由
    M->>M: 监听 127.0.0.1:3000
    loop 运行期
        W-->>M: 收到 HTTP/WS 请求 → 路由分发
    end
```

2.35 1.35 补充：01 章最后补充自测（5 题）

1. 7 个应用各自的页面结构？
2. 前端通用目录结构的作用？
3. main.rs 的启动顺序？
4. 路由注册在哪个函数？
5. 如何验证启动成功？

答对 4+ → 01 章全部完成。

2.36 1.36 补充：数据模型总览（数据库表一览）

2.36.1 1.36.1 核心表

users	用户 (id/email/password_hash/is_active)
user_roles	用户角色关联
permissions	权限点（或角色权限关联表）
fw100_items	fw100 设备台账
fw150_items	fw150 设备台账
city3d_buildings	城市建筑
city3d_regions	城市区域
city3d_events	城市事件

2.36.2 1.36.2 表设计特点

1. SQLite 单文件（无需安装数据库服务）
2. 外键少（业务简单，关联靠 service 层查询）
3. 时间字段统一 TEXT/INTEGER（时间戳）
4. 无迁移文件（database.rs 内建表）

2.36.3 1.36.3 种子数据的作用

启动时自动插入 7 个账号（admin + 6 个业务角色）
 → 首次启动即可用
 → 密码 123456，建议立即修改

2.37 1.37 补充：安全模型总览（认证 + 授权）

2.37.1 1.37.1 认证（你是谁）

登录 → 后端校验密码 → 发 JWT（含 user_id/role）
前端存储 token → 每次请求带 Authorization 头
WS → token 走查询参数

2.37.2 1.37.2 授权（你能做什么）

中间件链：认证 → 角色（role）→ 权限（permission）
角色注册表：ROLE_REGISTRY（后端唯一源）
前端：拉 /api/meta/roles 渲染菜单

2.37.3 1.37.3 新增权限的完整链路

Permission::XxxMonitor 枚举 → 角色注册表配置
→ gen:api → 前端类型
→ 中间件校验 → 前端菜单控制

2.38 1.38 补充：配置与数据文件总览

文件	位置	作用
.env	运行目录	端口/数据库/JWT
rustweb.db	运行目录	SQLite 数据
config-fj200c_information.ini	运行目录	发动机监控配置
config-fj200c_main.ini	运行目录	发动机测控配置
config-ftj1c.ini	运行目录	通信模块配置
csv/	运行目录	记录数据

规律：所有运行期文件都在运行目录（相对路径）——换机器部署只需拷目录。

2.39 1.39 补充：01 章最终自测（追加 8 题）

- 1. 核心表有哪些？
- 2. 表设计的特点？
- 3. 种子账号的作用？
- 4. 认证与授权的区别？
- 5. JWT 的用途？
- 6. 新增权限的链路？
- 7. 运行期文件有哪些？
- 8. 部署换机器的要点？

答对 7+ → 01 章彻底完成。

2.40 1.40 补充：开发环境搭建手把手（Windows）

2.40.1 1.40.1 前置工具

工具	用途	检查命令
Rust (rustup)	后端编译	rustc -version
Node.js (≥18)	前端构建	node -version
npm	依赖管理	npm -version
Git	版本管理	git -version

2.40.2 1.40.2 安装步骤

1. rustup-init.exe (官网下载) → 默认安装
2. Node.js LTS (官网下载) → 一路下一步
3. 验证: 新终端里三个 --version 都出来

2.40.3 1.40.3 拉取与安装依赖

```
git clone <仓库地址>
cd RustWeb-Vue
npm install          # 根目录一次 (workspaces 全装)
cargo build          # 首次较慢 (编译所有依赖)
```

2.40.4 1.40.4 启动

```
cargo run            # 后端 :3000
# 新终端
cd frontend/admin
npm run dev           # 前端 :5174
```

2.40.5 1.40.5 常见安装问题

1. cargo 报 openssl/链接错误 → 安装 VS Build Tools (C++ 依赖)
2. npm 慢 → 换镜像: npm config set registry https://registry.npmmirror.com
3. 端口被占 → 换端口或关占用程序

2.41 1.41 补充: IDE 推荐配置

2.41.1 1.41.1 VS Code 插件清单

插件	用途
rust-analyzer	Rust 智能提示
Volar (Vue Language Features)	Vue 3 支持
Prettier	格式化
Mermaid Preview	本套文档渲染
SQLite Viewer	查看 rustweb.db

2.41.2 1.41.2 工作区建议

打开根目录 (RustWeb-Vue) → workspaces 自动识别
→ 前后端同窗开发
→ 底部终端跑 cargo run + npm run dev

2.42 1.42 补充: 01 章高频自测 (8 题)

1. 三个前置工具版本检查命令?
2. npm install 在哪执行?
3. 首次 cargo build 慢的原因?
4. 启动后端的命令?
5. 启动前端的命令?
6. 链接错误怎么解决?
7. npm 慢怎么办?
8. 数据库文件怎么查看?

答对 7+ → 01 章高频通过。

2.43 1.43 补充：项目里用到的关键技术栈详解

2.43.1 1.43.1 后端关键依赖

依赖	用途	备注
axum	Web 框架	路由/中间件/提取器
tokio	异步运行时	spawn/interval/select
sqlx	SQL 访问	编译期校验 SQL
serde	序列化	DTO 转换
utoipa	OpenAPI 生成	类型契约源头
jsonwebtoken	JWT	登录令牌
bcrypt	密码哈希	不存明文
arc-swap	热更新配置	无锁读
serialport	串口通信	三路串口
rust-embed	前端内嵌	单 exe 部署
notify	文件监听	配置热加载

2.43.2 1.43.2 前端关键依赖

依赖	用途
vue 3	框架 (组合式 API)
vue-router	路由
pinia	状态管理
element-plus	UI 组件库
echarts	图表
axios	HTTP 请求
three.js (city3d)	3D 渲染

2.43.3 1.43.3 工程化工具

vite 构建
vue-tsc 类型检查
orval 契约生成
npm workspaces 多包管理
cargo 后端构建

2.44 1.44 补充：项目规模的量化认识

2.44.1 1.44.1 代码规模估算

后端：约 1.5 万行 Rust
前端：7 个应用 × 约 3000~8000 行 = 约 3 万行
shared：约 3000 行
总计：约 5 万行

2.44.2 1.44.2 数据规模

7 个数据库表 + 配置 + CSV 文件
用户量：通常 1~50 人（内网工具）
数据量：CSV 按帧增长（监控类）

2.44.3 1.44.3 对学习者的意义

- 5 万行对个人项目不小，但模块化清晰
- 每次只读一个模块（几百行）
- 循序渐进完全可行

2.45 1.45 补充：01 章终局自测（8 题）

1. 后端十大依赖各自的用途？
2. 前端六大依赖？
3. 三个工程化工具？
4. 代码规模估算？
5. 数据规模特征？
6. 为什么模块化重要？
7. 每次读多少行合适？
8. 学习节奏的建议？

答对 7+ → 01 章终局通过。

2.46 1.46 补充：常见业务场景的端到端流程

2.46.1 1.46.1 新用户登录流程

1. 打开浏览器 → 登录页
2. 输入邮箱密码 → 前端校验
3. POST /api/auth/login → 后端校验密码
4. 返回 JWT → 前端存 localStorage
5. 拉 /api/auth/me → 用户信息 + 权限
6. 拉 /api/meta/roles → 角色注册表
7. 按角色跳转到对应应用

2.46.2 1.46.2 监控数据全流程

- 设备 → 串口 → 后端读线程 → 帧提取 → 解码
- 广播 (WS) → 前端收到 → 表格/图表更新
- 可选：CSV 落盘

2.46.3 1.46.3 台账操作流程

- 前端表单 → 校验 → POST /api/fw100/items
- 后端鉴权 → service 校验 → SQL INSERT
- 返回新纪录 → 前端刷新列表

2.46.4 1.46.4 配置修改流程

- 前端配置页 → 编辑 → PUT /api/xxx/config
- 后端校验 → 写盘
- 热加载 (fj200c_information) 或提示重启

2.47 1.47 补充：学习本系统的推荐实战顺序

2.47.1 1.47.1 初级实战（改配置）

1. 改 .env 端口
2. 改 ini 模拟模式
3. 改种子账号密码
4. 改前端标题

2.47.2 1.47.2 中级实战（改逻辑）

1. 给台账加字段（后端 + 契约 + 前端）
2. 加一个查询接口
3. 改表格列显示
4. 调整 CSV 记录频率

2.47.3 1.47.3 高级实战（加功能）

1. 加告警模块（08 章案例）
2. 加新协议（08 章案例）
3. 加新角色/应用（08 章七步流程）
4. 加报表导出（08 章案例）

2.48 1.48 补充：01 章毕业自测（8 题）

1. 登录流程的七步？
2. 监控数据的完整链路？
3. 台账操作的流程？
4. 配置修改的流程？
5. 初级实战的四个任务？
6. 中级实战的四个任务？
7. 高级实战的四个任务？
8. 你现在能完成哪个级别？

答对 7+ → 01 章毕业。

2.49 1.49 补充：项目演进历史与设计取舍

2.49.1 1.49.1 从单体到多应用的演进

最初：一个前端应用包含所有功能

问题：权限混乱/打包巨大/改动互相影响

现在：7 个独立应用 + shared 共享

好处：独立部署/独立权限/独立打包

代价：配置同步点增多（deploy.bat/main.rs 等）

2.49.2 1.49.2 类型方案的演进

最早：手写前后端类型（不一致）

中间：ts-rs 生成（只有类型没有函数）

现在：utoipa + orval（类型 + 请求函数 + 文档）

2.49.3 1.49.3 部署方案的演进

早期：前端 dist + 后端 exe 分开部署

现在：rust-embed 内嵌单文件

好处：拷贝一个 exe 即部署

代价：前端改动必须重新编译后端

2.49.4 1.49.4 设计取舍的原则

1. 工具型系统优先可用性（不用过度设计）
2. 单机优先（不引分布式）
3. 配置驱动（ini 可调）
4. 契约自动（避免手工同步）

2.50 1.50 补充：01 章大师自测（8 题）

1. 多应用化的好处与代价？
2. 类型方案的三阶段？
3. 内嵌部署的取舍？
4. 四个设计原则？
5. 为什么需要共享层？
6. 改前端要重新编译吗？
7. 独立打包的价值？
8. 配置驱动的好处？

答对 7+ → 01 章大师。

2.51 1.51 补充：文档使用的核心概念对照表（翻译手册）

2.51.1 1.51.1 中文术语 → 代码概念

文档用语	代码里的位置
路由	routes.rs / Router
接口	handler + #[utoipa::path]
权限点	Permission 枚举
角色	roles.rs ROLE_REGISTRY
契约	openapi.json / generated
全局状态	OnceLock + ArcSwap
数据帧	TableRow / EcuFields
广播	broadcast 通道
台账	fw100_items 表
会话	JWT + AuthUser

2.51.2 1.51.2 前端术语 → 代码概念

文档用语	代码里的位置
页面	src/views/*.vue
路由守卫	router beforeEach
状态管理	stores/*.ts
组合式函数	composables/*.ts
API 封装	api/index.ts
生成代码	shared/api/generated
实时数据	WS + types.ts

2.51.3 1.51.3 使用方式

读文档遇到术语 → 查此表 → 定位代码位置

改代码时 → 用此表反查文档章节

2.52 1.52 补充：01 章权威自测（8 题）

1. 接口在代码里的三个要素？
2. 角色注册表在哪？
3. 数据帧的类型？
4. 广播用什么？
5. 页面在哪个目录？
6. 生成代码在哪？
7. 组合式函数在哪？
8. 实时数据的类型在哪手写？

答对 7+ → 01 章权威。

2.53 1.53 补充：项目背后的设计思想（为什么值得学）

2.53.1 1.53.1 五个值得学的点

1. 契约驱动：类型自动同步，前后端不脱节
2. 模块化：7 应用 + 模块化后端，边界清晰

3. 配置驱动: ini 免编译调整
4. 模拟优先: 无硬件也能开发演示
5. 单文件部署: 内嵌前端, 拷贝即用

2.53.2 1.53.2 三个真实的工程教训

1. 类型不一致的痛 (ts-rs 到 utoipa 的升级)
2. 依赖重复的痛 (npm workspaces 统一装)
3. 部署顺序的痛 (前端必须先构建)

2.53.3 1.53.3 学完后你能带走的

1. Rust + Axum 全栈实战经验
2. Vue 3 多应用工程化经验
3. 契约驱动开发方法论
4. 实时数据系统 (WS) 实战
5. 完整部署运维经验

2.54 1.54 补充: 01 章权威自测 (8 题)

1. 五个值得学的点?
2. 三个工程教训?
3. 五个能带走的技能?
4. 契约驱动的好处?
5. 模拟优先的意义?
6. 单文件部署的代价?
7. ts-rs 升级的原因?
8. 部署顺序为什么不能乱?

答对 7+ → 01 章权威。

下一节: 02-Rust 语法速成——第一片主战场。

3 02 Rust 语法速成 (以本项目代码为教材)

适用对象: Rust 零基础或入门不久的新手。教学目标: 不是系统学习 Rust, 而是**看懂并修改本项目的 Rust 代码**——语法点全部用项目真实代码举例, 每个例子都标注源码位置, 建议边读边打开文件对照。全文约 2 万字。如果你有编程经验 (Java/Python/JS 均可), 预计 4~6 小时可读完并消化。

3.1 2.1 先建立正确心态: Rust 的“三座大山”其实没那么可怕

新手听说 Rust 难, 主要难在三个概念: **所有权 (Ownership)**、**借用 (Borrowing)**、**生命周期 (Lifetime)**。本项目的代码大量使用这些概念, 但你不需要精通它们才能开始——你只需要掌握**阅读模式**和**修改模式**:

场景	你需要会什么
读代码	能看出“这个变量被谁拥有”“这个引用是从哪来的”
小改代码	能照着周围代码的样子抄 (编译器会告诉你哪里错了)
大改代码	才需要真正理解所有权规则

Rust 编译器的报错信息是全球公认最好的——它会告诉你具体改法。所以**大胆编译**, 让编译器当你的老师。本项目后端改动后用 `cargo build` 验证, 报错看不懂就问, 或者贴给 AI。

还有一个好消息：本项目代码风格统一、注释详尽（几乎每个语法点都有中文注释），是最好的 Rust 阅读材料之一。

3.2 2.2 工具链准备 (5 分钟)

```
# 1. 安装 rustup (Windows 用官方安装器, 或 winget install Rustlang.Rustup)
# 2. 验证
rustc --version      # 编译器
cargo --version      # 构建/包管理工具 (npm 的 Rust 版)
rustup --version      # 工具链管理
# 3. VS Code 装 rust-analyzer 扩展 (自动分析、悬停提示、跳转定义)
# 4. 进入项目根目录
cargo check          # 只检查不编译产物, 比 cargo build 快 (日常开发用这个)
cargo run            # 编译并运行
cargo build --release --features embedded # 生产构建 (单 exe)
```

术语: **crate** = 包 (一个 Cargo 项目); **module** = 模块 (一个 .rs 文件或目录); cargo 类似 npm, 但 Rust 是编译型语言, “npm run” 在 Rust 里是 cargo run。

3.3 2.3 变量、类型与 let

3.3.1 2.3.1 基本变量

```
// src/config.rs — 环境变量读取 (最典型的 let 用法)
pub fn load() -> Result<AppConfig, Box<dyn std::error::Error>> {
    let port: u16 = std::env::var("PORT")           // 读环境变量 (返回 Result<String>)
        .unwrap_or_else(|_| "3000".to_string())     // 出错时用默认值 "3000"
        .parse()?;                                   // 字符串转 u16 (? 见 2.5 节)
    let database_url = std::env::var("DATABASE_URL")
        .unwrap_or_else(|_| "sqlite://rustweb.db".to_string());
    Ok(AppConfig { port, database_url })
}
```

新手要掌握的点:

1. **let 声明变量, 默认不可变**: let x = 5; 之后 x = 6 会编译报错。要可变需写 let mut x = 5;。本项目大量利用这一点——不可变变量让代码意图清晰。
2. **类型可以推断**: let port: u16 = ... 显式标注了类型; 很多地方可以省略 let x = 5 (自动推断 i32)。
3. **.to_string()**: 任何可打印类型转 String。
4. **.unwrap_or_else(闭包)**: Result/Option 出错时的兜底值模式——**项目里”配置有默认值”全部用这个模式**。
5. **.parse::<T>()**: 字符串解析为数值类型, 返回 Result (可能失败)。

3.3.2 2.3.2 常量与静态变量

```
// src/fj200c_information/state.rs — 模块常量
pub const CONFIG_PATH: &str = "config-fj200c_information.ini"; // 编译期常量

// src/common/service.rs — 全局静态可变状态 (线程安全)
pub static RUNTIME: OnceLock<Mutex<Vec<JoinHandle<()>>>> = OnceLock::new();
```

关键字	含义	区别
const	编译期常量	无内存地址，内联展开；类型必须是字面量可表示的
static	全局静态变量	有固定内存地址，生命周期为整个程序
static mut	全局可变静态	禁止直接使用 （不安全），本项目用 OnceLock/AtomicBool/RwLock 包一层

Rust 不允许随意访问可变全局变量（数据竞争），所以本项目所有全局状态都用 OnceLock（单例容器）、AtomicBool（原子布尔）、RwLock（读写锁）包裹——**看到这些类型，就知道这是“全局状态”**。

3.3.3 2.3.3 元组与结构体字段访问

```
// src/common/auth/handlers.rs — 返回元组类型
// axum handler 提取器可以组合，这里用 () 表示无额外参数
// (State(db), Json(login_data)) 是元组解构的典型用法：
// let (a, b) = (1, 2); // 元组解构赋值

// src/common/models.rs — 结构体字段访问
let email = &user.email; // 取字段（借用）
let name = user.username.clone(); // 取字段（克隆，拥有所有权）
```

3.4 2.4 结构体与枚举（本项目最核心的两种类型）

3.4.1 2.4.1 结构体（struct）

```
// src/common/models.rs — 用户模型（节选）
#[derive(Debug, Clone, Serialize, Deserialize, utoipa::ToSchema)]
pub struct User {
    pub id: uuid::Uuid,
    pub username: String,
    pub email: String,
    #[serde(skip_serializing)]
    pub password_hash: String, // 序列化时跳过！永不下发到前端
    pub role: String,
    pub created_at: chrono::NaiveDateTime,
}
```

要点：

- 1. pub = 公开字段，模块外可访问（本项目几乎所有字段都是 pub，简化跨模块访问）。
- 2. #[derive(...)] 自动实现 trait（见 2.8 节宏）：Clone（可复制）、Serialize/Deserialize（JSON 转换）、ToSchema（OpenAPI 文档）。
- 3. #[serde(skip_serializing)] 属性：JSON 输出时**排除**该字段——密码哈希永不外泄。这是 serde 属性的经典例子。
- 4. **创建结构体**：User { id, username, ... } 字段名与变量名相同时可简写（User { id, ... } 中的 id 即 id: id）。

结构体的方法定义（impl 块）：

```
// src/common/models.rs — User 的方法
impl User {
    /// 计算用户的权限列表（查角色注册表）
    pub fn permissions(&self) -> Vec<Permission> {
        crate::roles::permissions_for(&self.role)
    }
}
```

```

    }

    /// 判断是否拥有某权限
    pub fn has_permission(&self, permission: &Permission) -> bool {
        self.permissions().contains(permission)
    }
}

```

要点：&self = 只读借用（类似 this 但不可修改）；&mut self = 可变借用；self = 消费所有权。本项目方法几乎全是 &self（只读查询）。

3.4.2 2.4.2 枚举（enum）——本项目最常用的类型

```

// src/common/models.rs — 权限枚举（10 个权限点）
#[derive(Debug, Serialize, Deserialize, Clone, PartialEq, Eq, Hash, utoipa::ToSchema)]
pub enum Permission {
    Fj200cInformationMonitor,
    Fj200cMainMonitor,
    Fw100Monitor,
    Fw150Monitor,
    Ftj1cMonitor,
    City3dView,
    UsersRead,
    UsersWrite,
    UsersDelete,
    SystemAdmin,
}

```

要点：

1. 枚举成员默认没有值（unit variant），用于“标记”场景——权限就是纯标记。
2. PartialEq, Eq 用于 == 比较（user.has_permission 里 contains(permission) 就依赖它）；Hash 用于放 HashSet 键。
3. serde 对枚举默认序列化为**字符串**（如 "UsersRead"），前端 orval 生成同名 TS 枚举——这就是前后端权限点类型同步的基础。

枚举也可以**携带数据**——这是项目里事件系统的基础：

```

// src/fj200c_information/mod.rs — 事件枚举（WS 推送的载荷类型）
#[derive(Debug, Clone, Serialize, utoipa::ToSchema)]
#[serde(tag = "type")] // 序列化时加 "type" 字段做判别标签
pub enum Fj200cInformationEvent {
    /// 解码后的完整数据行（前端表格更新用）
    TableData { row: TableRow },
    /// 原始数据帧（16 进制字符串）
    Frame { hex: String },
    /// 载荷数据
    Payload { hex: String },
}

```

这里的 #[serde(tag = "type")] 叫**内部标签枚举（internally tagged enum）**，序列化结果形如：

```
{ "type": "TableData", "row": { ... } }
```

前端 JS 用 switch (event.type) 分发——这就是前后端事件协议的契约。**带数据枚举 + serde tag 是 WebSocket 推送的标准模式**，三个硬件模块全部这么用。

3.4.3 2.4.3 枚举是“类型安全的 if”

在别的语言里，你可能会写 if (type == 1) 或魔法字符串；Rust 用枚举 + match 保证编译器强制处理所有分支（见 2.6 节）。

3.5 2.5 Option 与 Result: 没有 null 的世界

3.5.1 2.5.1 Option: 可能有值

```
// src/common/ws.rs — WS 桥的可选初始消息
async fn ws_bridge_with_initial(
    tx: broadcast::Sender<T>,
    socket: WebSocket,
    prefix: &str,
    initial: Option<String>,    // 要么 Some(内容), 要么 None (无)
) {
    // ...
}
```

模式	说明	本项目例子
Option<T>	有值/无值	WS 初始快照、配置可选字段
.unwrap()	有值直接取出, 无值 panic (崩溃)	避免在生产路径用, 测试里常见
.unwrap_or(default)	无值用默认值	配置读取
.ok_or(err)?	无值转成错误并返回	见下
if let Some(x) = ...	有值才处理	见 2.6

3.5.2 2.5.2 Result: 可能出错

```
// src/common/jwt.rs — 验证 token (Result 的教科书用法)
pub fn verify_token(token: &str) -> Result<Uuid, AppError> {
    let data = jsonwebtoken::decode::<Claims>(    // decode 返回 Result, ? 自动转 AppError
        token,
        &DecodingKey::from_secret(SECRET.as_bytes()),
        &Validation::default(),
    )
    .map_err(|e| AppError::unauthorized(format!("无效 token: {e}")))?;
    // ... 检查过期
    Ok(data.claims.sub)    // 成功: 返回用户 ID
}
```

Result<T, E> 的含义: **成功时是 T, 失败时是 E**。项目统一 Result<T, AppError>。

新手必会的 Result 操作:

```
let x: Result<u32, String> = Ok(42);

x.unwrap()                // 42: 失败则 panic (程序崩溃)
x.expect("说明文字")      // 同上, 带自定义错误信息 (比 unwrap 好)
x?                         // 成功取出 42; 失败则函数提前返回错误 (★最常用)
x.map(|v| v * 2)           // 成功时转换值: Ok(84)
x.map_err(|e| ...)        // 失败时转换错误类型
x.ok()?                   // 只关心值, 错误直接抛 (转 Option)
x.is_ok() / x.is_err()    // 判断
```

3.5.3 2.5.3 项目里最经典的组合: 链式调用

```
// src/common/auth/handlers.rs — 登录 handler
login_data.validate()?;    // 校验失败 → 400
let user = AuthService::login(&db, login_data)
    .await
    .map_err(|e| AppError::bad_request(e.to_string()))?;    // 登录失败 → 400
let token = jwt::create_token(&user)?;                      // 签发失败 → 500
Ok(Json(ApiResponse::success(LoginResponse { token, user })))
```

读法口诀：看到 `?` 就念“出错了就返回”，看到 `.map_err(|e| AppError::xxx(...))` 就念“把错误转换成 HTTP 错误码”。

3.6 2.6 模式匹配：match / if let / let else

3.6.1 2.6.1 match：穷尽所有分支

```
// src/fj200c_main/decode.rs — 状态机映射（match 的经典用法）
pub fn engine_status_str(status: u8) -> &'static str {
    match status {
        0x01 => "停车",
        0x02 => "启动",
        0x03 => "运行",
        0x04 => "故障",
        _ => "未知", // 通配分支：其他值都到这里
    }
}
```

要点：

1. **必须穷尽**：match 要求覆盖所有情况，否则编译错误。`_` 是“其他所有”。
2. 每个分支是表达式，match 整体可以赋值：`let s = match x { 1 => "a", _ => "b" };。`
3. 模式可以是字面量、枚举成员、带 `|` 的多模式 (`0x01 | 0x02 => "启动"`)、带守卫 (`n if n > 10 => ...`)。

3.6.2 2.6.2 if let：只关心一种情况

```
// src/common/ws.rs — WS 桥循环里的消息判断
match msg {
    Ok(Some(Message::Text(text))) => { /* 客户端发来文本 */ }
    Ok(Some(Message::Close(_))) => break,
    _ => {}
}
```

if let 简化“只关心一种情况”的 match：

```
if let Some(user) = request.extensions().get::<User>() {
    // 有用户（中间件已注入），处理业务
}
```

3.6.3 2.6.3 let-else：不满足就提前返回

```
// src/fj200c_information/handlers.rs — CSV 文件名防目录穿越
let Ok(name) = url::percent_decode_str(name).decode_utf8() else {
    return Err(AppError::bad_request("文件名编码无效".to_string()));
};
let Some(file_name) = name.rsplit('/').next() else {
    return Err(AppError::bad_request("文件名不合法".to_string()));
};
```

`let ... else` 是“反直觉守卫”：模式匹配失败时执行 else 分支（通常提前 return）。它让“校验后继续”的代码不产生嵌套地狱。

3.6.4 2.6.4 模式匹配在项目中的使用场景清单

场景	位置
十六进制状态码 → 中文名称	fj200c_main/decode.rs
WS 消息类型分发	common/ws.rs、各模块 WS 循环
事件枚举分发 (前端)	useFj200cInformationEvents.ts (TS 侧)
帧类型匹配 (CSV 状态机)	fj200c_information/session.rs
序列化标签分发	各 decode.rs

3.7 2.7 trait: 接口与抽象

3.7.1 2.7.1 trait 是什么

trait 类似 Java 的接口 / TypeScript 的 interface: 定义一组方法签名, 让不同类型实现同一套行为。

```
// src/common/io.rs — 硬件抽象 (本项目最重要的 trait)
/// 串口与模拟器的统一抽象: 上层代码不关心数据来自真实硬件还是模拟器
pub trait IoControl {
    fn send(&self, data: &[u8]) -> Result<(), String>;
    fn recv(&self) -> Result<Vec<u8>, String>;
    fn set_timeout(&self, timeout_ms: u32) -> Result<(), String>;
}
```

实现者 (两种):

```
// src/fj200c_information/com.rs — 真实串口实现
pub struct SerialControl { port: Mutex<Box<dyn SerialPort>> }
impl IoControl for SerialControl {
    fn send(&self, data: &[u8]) -> Result<(), String> {
        self.port.lock().unwrap().write(data)
            .map_err(|e| e.to_string())?;
        Ok(())
    }
    // ...
}

// src/fj200c_information/mock.rs — 模拟器实现
pub struct MockControl { /* 模拟数据生成状态 */ }
impl IoControl for MockControl {
    fn send(&self, _data: &[u8]) -> Result<(), String> { Ok(()) }
    fn recv(&self) -> Result<Vec<u8>, String> {
        // 每 50ms 生成一帧 20Hz 正弦+噪声模拟数据
        Ok(generate_frame())
    }
    // ...
}
```

这就是 Mock 开关的魔法来源: config.ini 里 InProcess = true 时, 启动服务就传 MockControl; 否则传 SerialControl。上层 session.rs 拿到的是 Box<dyn IoControl> (trait 对象), 完全不感知差异:

```
// 抽象调用处 (session.rs 伪代码)
let io: Box<dyn IoControl> = if mock { Box::new(MockControl::create()) }
                                else { Box::new(SerialControl::open(&cfg?)) };
io.recv() // 不管底下是串口还是模拟器, 接口一致
```

3.7.2 2.7.2 trait 对象与泛型

写法	含义	本项目例子
Box<dyn IoControl>	trait 对象：运行时多态（堆上，动态分发）	会话线程持有的 IO
impl IoControl（参数位置）	泛型糖：编译期静态分发	较少
<T: IoControl>	泛型约束	FrameExtractor::new 内部
A: RustEmbed	泛型约束（嵌入式资源）	embedded_assets.rs

3.7.3 2.7.3 本项目用到的标准库 trait（背下来）

trait	作用	看到它就想到
Serialize / Deserialize	JSON 序列化/反序列化	DTO 结构体必带
Clone	深度复制	跨线程传数据需要
Debug	{:?} 打印调试	日志/测试
PartialEq / Eq	== 比较	枚举、配置比较
Send / Sync	跨线程安全标记	所有跨线程的类型必须满足，编译器强制检查
ToSchema	OpenAPI 文档生成	DTO 必带（utoipa）
From<T> / Into<T>	类型转换	AppError 自动转换（见 2.9）
Default	Default::default() 默认值	配置结构体
IntoResponse	转 HTTP 响应	AppError、handler 返回值
FromRow	数据库行转结构体	sqlx 查询结果

3.8 2.8 宏：derive 宏与声明式宏

3.8.1 2.8.1 derive 宏：自动实现

`#[derive(Serialize)]` 不是装饰器，而是**代码生成器**：编译器展开后为你的结构体自动生成几十行 `Serialize` 实现代码。本项目里：

```
#[derive(Debug, Clone, Serialize, Deserialize, ToSchema)] // 一行顶几十行
pub struct Building { ... }
```

新手只需要：**新写 DTO 时照抄现有 DTO 的 derive 行**（要什么能力就抄什么：要进 OpenAPI 就加 `ToSchema`；要 JSON 就加 `Serialize/Deserialize`；要跨线程传就加 `Clone`）。

3.8.2 2.8.2 属性宏：utoipa::path

```
#[utoipa::path(
    post,
    tag = "auth",
    path = "/api/auth/login",
    operation_id = "authLogin",
    request_body = LoginRequest,
    responses((status = 200, description = "登录成功", body = ApiResponse<LoginResponse>))
)]
pub async fn login(State(db): State<DatabaseConnection>, Json(login_data):
    Json<LoginRequest>):
    -> Result<Json<ApiResponse<LoginResponse>>, AppError>
{ ... }
```

这行注解生成 OpenAPI 文档条目 (06 章详述)。写新接口时必须加, 否则 cargo test export_openapi 的防漂移断言会失败。

3.8.3 2.8.3 声明式宏: macro_rules!

```
// src/fj200c_main/com.rs — 用宏生成三个结构相似的串口实现
macro_rules! define_com_port {
    ($name:ident, $spec:expr) => {
        pub struct $name { /* ... */ }
        impl $name {
            pub fn new(...) -> Result<Self, String> { ... }
            pub fn run(...) { ... }
        }
    };
}

define_com_port!(ECUCom, ComSpec::ecu_protocol());
define_com_port!(AdamCom, ComSpec::adam_protocol());
define_com_port!(DynoCom, ComSpec::dyno_protocol());
```

宏的作用是消除重复代码: 三个串口实现只有协议规格不同, 写三遍太蠢, 用宏“参数化类型名”。

```
// src/fj200c_information/session.rs — 表行宏 (把 N 个字段打包成表行)
macro_rules! push_row {
    ($row:expr, $table:expr, $($field:expr),+) => {
        $table.push(Row::new(&[$($field),+]).with_cells(...));
    };
}
```

新手不需要会写宏, 但要能读懂宏的调用, 并知道“想改这个行为要动哪里”。

3.9 2.9 错误处理: AppError 体系

3.9.1 2.9.1 统一错误类型

```
// src/common/error.rs
pub struct AppError {
    pub status_code: u16,    // HTTP 状态码
    pub message: String,    // 人类可读信息
}

impl AppError {
    pub fn bad_request(msg: String) -> Self { Self { status_code: 400, message: msg } }
    pub fn unauthorized(msg: String) -> Self { Self { status_code: 401, message: msg } }
    pub fn forbidden(msg: String) -> Self { Self { status_code: 403, message: msg } }
    pub fn not_found(msg: String) -> Self { Self { status_code: 404, message: msg } }
    pub fn internal(msg: String) -> Self { Self { status_code: 500, message: msg } }
}
```

3.9.2 2.9.2 自动转 HTTP 响应

```
// src/common/error.rs — 让 AppError 变成合法的 handler 返回值
impl IntoResponse for AppError {
    fn into_response(self) -> Response {
        let body = serde_json::json!({
            "success": false,
            "message": self.message,
        });
        (StatusCode::from_u16(self.status_code).unwrap(), Json(body)).into_response()
    }
}
```

因为实现了 IntoResponse, handler 返回 Result<T, AppError> 时, Err(AppError) 会被自动序列化化成上面的 JSON 响应。

3.9.3 2.9.3 From 转换: 错误自动升级

// src/common/error.rs — 常见错误 → AppError 的自动转换

```
impl From<sqlx::Error> for AppError {
    fn from(err: sqlx::Error) -> Self {
        match err {
            sqlx::Error::RowNotFound => AppError::not_found("记录不存在".to_string()),
            sqlx::Error::Database(ref db_err) if db_err.is_unique_violation() =>
                AppError::bad_request("记录已存在".to_string()),
            other => AppError::internal(other.to_string()),
        }
    }
}

impl From<jsonwebtoken::Error> for AppError { ... } // → 401
impl From<bcrypt::Error> for AppError { ... } // → 500
impl From<validator::ValidationErrors> for AppError { ... } // → 400
```

这个机制是 ? 能工作的关键: 函数返回 Result<_, AppError> 时, 如果内部错误类型实现了 From<E> for AppError, ? 会自动调用转换。所以代码里可以随便写:

```
let user = sqlx::query_as::<_, User>("SELECT ...")
    .fetch_one(&db)
    .await?; // sqlx::Error 自动转 AppError, 一行都不用写转换
```

3.9.4 2.9.4 新手错误处理口诀

1. handler 返回 Result<Json<ApiResponse<T>>, AppError>。
2. 业务错误用工厂方法: AppError::bad_request(...).
3. 内部错误靠 From + ? 自动转。
4. 前端永远收到 { success, message, data? }。

3.10 2.10 async/await 与 tokio

3.10.1 2.10.1 什么是异步

同步代码: 一个请求占一个线程, 线程阻塞等 IO (数据库、网络)。**异步代码:** 少数线程轮流处理大量请求, 阻塞等待期间让出线程。Axum 的 handler 全部是异步函数。

```
#[tokio::main] // 宏: 把 main 变成 tokio 异步运行时入口
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = init_database(&config.database_url).await?; // 异步等待数据库连接
    let listener = tokio::net::TcpListener::bind(addr).await?;
    axum::serve(listener, app).await?; // 一直跑, 直到服务器关闭
    Ok(())
}
```

要点: 1. async fn 的函数调用**不会立刻执行**, 返回一个 Future, 必须 .await 才会推进。
2. .await 遇到阻塞会自动让出线程, 其他请求乘机执行。3. 项目中 handler 都是 async fn; HTTP/WS/数据库操作都有异步版本。4. **CPU 密集任务不能用 async 处理:** bcrypt 哈希会阻塞线程。项目用 tokio::task::spawn_blocking 把它丢到阻塞线程池:

```
// src/common/auth/services.rs — 密码校验 (CPU 密集 → spawn_blocking)
let valid = tokio::task::spawn_blocking(move || {
    bcrypt::verify(&password, &user.password_hash).unwrap_or(false)
})
.await
.unwrap_or(false);
```

3.10.2 2.10.2 tokio::select!: 同时监听多个源

// src/common/ws.rs — WS 桥的核心循环

```
loop {
    tokio::select! {
        msg = receiver.next() => {           // 分支一: 客户端发来消息
            match msg { Some(Ok(Message::Close(_))) | None => break, ... }
        }
        event = rx.recv() => {               // 分支二: 广播通道有事件
            match event {
                Ok(evt) => { /* 序列化并推给客户端 */ }
                Err(RecvError::Lagged(_)) => continue,    // 客户端太慢, 丢弃
                Err(RecvError::Closed) => break,
            }
        }
    }
}
```

tokio::select! 同时等待两个 future, **谁先完成执行谁**——这是“WS 双向通信”的惯用模式。Lagged 是 broadcast 通道的特性: 接收端跟不上时旧消息被丢弃, 用 continue 跳过即可(不让客户端拖慢系统)。

3.10.3 2.10.3 新手 async 避坑

1. **不要在 async 里做阻塞操作** (串口读、文件大读写、CPU 计算) ——用 spawn_blocking 或 std::thread。
2. std::thread::sleep 会阻塞整个 tokio 线程——异步代码里用 tokio::time::sleep。
3. 本项目采集线程用 std::thread (阻塞串口读), HTTP 用 tokio, 两者靠 broadcast 桥接——**不要试图把串口读改成 async**。

3.11 2.11 线程与并发 (本项目并发全景)

3.11.1 2.11.1 std::thread: 独立线程

// src/common/service.rs — 线程句柄管理

```
pub struct ServiceRuntime {
    handles: Mutex<Vec<JoinHandle<()>>>,    // 线程句柄集合
    stop: AtomicBool,                       // 停止标志
}

impl ServiceRuntime {
    pub fn push(&self, handle: JoinHandle<()>) { // 记录线程
        self.handles.lock().unwrap().push(handle);
    }
    pub fn wait_stopping(&self, timeout_secs: u64) {
        let deadline = Instant::now() + Duration::from_secs(timeout_secs);
        let mut handles = self.handles.lock().unwrap();
        while let Some(handle) = handles.pop() {
            let remaining = deadline.saturating_duration_since(Instant::now());
            let _ = handle.thread().unpark(); // 唤醒
            let _ = handle.join_timeout(remaining).unwrap_or_else(|_| handle); // 等待
        }
    }
}
```

启动线程的模式:

// src/fj200c_information/service.rs (示意)

```
let tx = fj200c_information_tx();
let handle = std::thread::spawn(move || { // move: 闭包拿走 tx 的所有权 (克隆引用)
    run_one_connection(connection, io, tx); // 会话线程主函数
});
```

```
});
RUNTIME.push(handle);
```

要点： - thread::spawn 的闭包用 move 关键字**捕获环境**（把变量所有权移入线程）——tx 是 Sender 的克隆，跨线程合法（Sender<T> 实现 Send）。 - 线程结束方式：轮询 AtomicBool 停止标志 + join 等待。 - **停止模式**：stop_service 置标志 → 线程循环看到标志 break → join 收尾（最长等 3 秒）。

3.11.2 2.11.2 共享状态的三件套

类型	用途	项目位置
Arc<T>	多线程共享只读数据	Arc<QuadFrame<95>> (ftj1c)
Arc<Mutex<T>>	多线程共享可变数据（互斥锁）	ServiceRuntime.handles
Arc<RwLock<T>>	读多写少共享数据	SHARED_DATA 16 字段
AtomicBool / AtomicU8 / AtomicUsize	单个数值的线程安全读写	停止标志、CSV 状态机
ArcSwap<T>	无锁热替换（高频读）	最新帧存储

```
// src/common/global_var.rs — 全局 KV 存储（RwLock 的典型用法）
pub struct GlobalVar {
    inner: RwLock<HashMap<String, String>>, // 读锁可并发，写锁独占
}

impl GlobalVar {
    pub fn set(&self, key: &str, value: &str) {
        let mut map = self.inner.write().unwrap(); // 写锁
        map.insert(key.to_string(), value.to_string());
    }
    pub fn get(&self, key: &str) -> Option<String> {
        let map = self.inner.read().unwrap(); // 读锁
        map.get(key).cloned()
    }
}
```

OnceLock<T>： **全局单例容器**——要么未初始化，要么已初始化，只能设置一次：

```
// src/fj200c_information/mod.rs — 广播通道单例
pub static FJ200C_INFORMATION_TX: OnceLock<broadcast::Sender<Fj200cInformationEvent>> =
OnceLock::new();

pub fn fj200c_information_tx() -> broadcast::Sender<Fj200cInformationEvent> {
    FJ200C_INFORMATION_TX
        .get_or_init(|| broadcast::channel(1024).0) // 首次调用创建
        .clone() // 克隆给调用者（计数+1）
}
```

3.11.3 2.11.3 broadcast：一对多广播通道

```
let (tx, _rx) = broadcast::channel(1024); // 创建：发送端 + 接收端
let rx2 = tx.subscribe(); // 每个新接收端从当前 Sender 派生
tx.send(event); // 广播给所有订阅者
rx.recv().await // 异步接收；Lagged → 丢旧消息
```

特性	说明
容量	1024 条；满了之后最旧的被丢弃，慢订阅者收 Lagged
多接收者	每个 subscribe() 独立游标，互不干扰
自动清理	所有接收端 drop 后 send 返回 Err（会话线程据此退出）
线程安全	Sender 可克隆到任意线程

架构地位：广播通道是“采集线程 → WS 推送”的唯一桥梁，也是“HTTP 操作 → 采集线程”命令通道（mpsc，见下）的姊妹机制。

3.11.4 2.11.4 mpsc：多生产者单消费者命令通道

```
// src/fj200c_information/session.rs（示意）：命令通道
// 服务启动时给每条连接创建一个（tx，rx），rx 传给会话线程
// handler 通过 mpsc tx 发送命令，会话线程 try_recv 消费
if let Ok(cmd) = command_rx.try_recv() {
    io.send(&cmd)?; // 把命令写进串口/模拟器
}
```

try_recv 是**非阻塞**取命令：会话线程每轮循环检查一次，有命令就发，没命令就继续收帧——命令与数据流互不阻塞。

3.11.5 2.11.5 ArcSwap：无锁热替换

```
// src/common/quad_frame.rs — 四槽帧缓冲
pub struct QuadFrame<const FRAME_LEN: usize> {
    frames: [ArcSwap<[u8; FRAME_LEN]>; 4], // 4 个槽位，主备双源
    sequence: AtomicU32,
}
// 读：load() 无锁取当前帧（高频读场景，绝不停顿）
// 写：store(new) 原子替换（写者不阻塞读者）
```

ArcSwap 用于“高频读、低频写”的热数据（最新帧），比 RwLock 更快（读完全无锁）。理解即可，不必深入。

3.12 2.12 生命周期与借用：rust-analyzer 是你的眼睛

3.12.1 2.12.1 借用的两条规则

1. 一个可变借用 OR 多个只读借用（不能同时可变+只读）。
2. 借用不能超过所有者的生命（借用者存活期间，所有者不能被销毁）。

新手读项目代码时的实际体验：

```
let rows = Vec::new();
// 传给函数：传借用 &rows（不移动所有权），函数用完还能继续用 rows
let json = serde_json::to_string(&rows)?;
println!("{}", rows.len()); // ✓ rows 还在

// 传所有权（无 &）：rows 被“移动”，之后不能再碰
let json = serde_json::to_string(rows)?;
println!("{}", rows.len()); // ✗ 编译错误：rows 已被移动
```

3.12.2 2.12.2 生命周期标注 'a：编译器给你做证明题

```
// src/common/quad_frame.rs — 泛型生命周期（&'a str）
// 含义：“返回值活得不会比参数 a/b 更久”
fn foo<'a>(a: &'a str, b: &'a str) -> &'a str { if a.len() > b.len() { a } else { b } }
```


新手策略： 1. **90% 的情况编译器自动推断**（省略生命周期标注）。 2. 看到 'static: 表示“整个程序生命周期”——&'static str 即字符串常量（CONFIG_PATH 那种）。 3. 看到 &'a str: 只是编译器在检查借用时长，**不要慌，通常不用你改**。

3.12.3 2.12.3 新手最常遇到的借用错误与修法

编译错误	原因	修法
cannot borrow as mutable	已有不可变借用又取可变借用	缩小借用范围/用 RwLock
value moved here	用了移动（无 &）	加 .clone() 或改传 &
borrow of moved value	移动后又用	结构体实现 Clone 后 clone
lifetime may not live long enough	返回值借用被提前释放	返回 owned 值（String/Vec）而不是引用
captured variable in FnOnce	闭包用 move 后原变量失效	先 clone 一份再 move 进闭包

实用技巧：改完代码编译报借用错误，优先看编译器建议（它经常直接给出修法），其次参考同一文件相邻代码的写法——项目代码风格统一，照抄即可。

3.13 2.13 字符串与集合

3.13.1 2.13.1 String vs &str

```
let s: String = String::from("hello"); // 堆上、可增长、拥有所有权
let s2: String = "hello".to_string(); // 同上（常用写法）
let slice: &str = &s; // 借用视图，不可变
let lit: &'static str = "hello"; // 编译期字符串常量
```

项目约定：**配置常量用 &'static str，动态数据用 String，函数参数尽量 &str**（可接受两者传入）。

```
// 项目里随处可见的模式：&str 参数 + 内部 to_string
pub fn set(&self, key: &str, value: &str) { ... }
// 调用：set("key", &value) — &String 自动强转 &str
```

3.13.2 2.13.2 Vec: 动态数组

```
let mut v = Vec::new(); // 创建空数组
v.push(1); // 追加
let x = v[0]; // 索引访问（越界 panic）
let first = v.first(); // Option 安全访问
v.iter() // 迭代器（借用）
v.len() // 长度
```

3.13.3 2.13.3 HashMap: 键值对

```
let mut map: HashMap<String, String> = HashMap::new();
map.insert("k".into(), "v".into());
map.get("k") // Option<&String>
map.entry("k").or_insert("default".into()) // 不存在才插入
```

项目使用场景：global_var.rs 的 KV 存储、ftj1c/models.rs 的 16 组 IP 配置、jwt.rs 解码后的 Claims。

3.13.4 2.13.4 迭代器链（读代码必备）

```
// src/city3d/services.rs（示意）— 常见迭代器链
let names: Vec<String> = districts.iter() // 遍历引用
```



```

.filter(|d| d.enabled)           // 过滤
.map(|d| d.name.clone())        // 转换
.collect();                     // 收集成 Vec

```

项目里迭代器链主要用于：列表转换、权限过滤（permissions_for）、配置解析、CSV 行拼接。

3.14 2.14 模块系统：mod / use / crate::

3.14.1 2.14.1 模块声明

// src/common/mod.rs — 模块目录的门面

```

pub mod auth;
pub mod config;
pub mod csv_writer;
pub mod dto;
pub mod error;
pub mod frame_extractor;
pub mod global_var;
pub mod io;
pub mod jwt;
pub mod latest_frame;
pub mod ledger;
pub mod least_squares;
pub mod middleware;
pub mod models;
pub mod quad_frame;
pub mod service;
pub mod utils;
pub mod ws;

```

规则：- 目录下必须有 mod.rs（Rust 2018 后也可用同名目录+文件），声明其子模块。- pub mod 对外可见；pub(crate) 仅本 crate 可见。- 子模块文件内 use crate::common::models::... 引用。

3.14.2 2.14.2 路径三种写法

// 1. crate:: 绝对路径（从 crate 根开始）—项目主用风格

```
crate::roles::permissions_for(&self.role)
```

// 2. super:: 相对路径（上一级）

```
super::fj200c_information_tx()
```

// 3. use 导入后直接用

```

use crate::common::error::AppError;
use crate::common::models::ApiResponse;

```

3.14.3 2.14.3 项目跨模块引用关系图

flowchart LR

```

main.rs --> routes.rs
routes.rs --> common/auth/routes.rs
routes.rs --> admin/routes.rs
routes.rs --> fj200c_information/routes.rs
routes.rs --> fj200c_main/routes.rs
routes.rs --> ftj1c/routes.rs
routes.rs --> fw100/routes.rs
routes.rs --> fw150/routes.rs
routes.rs --> city3d/routes.rs
routes.rs --> roles.rs
main.rs --> embedded_assets.rs
main.rs --> database.rs
main.rs --> config.rs
common/models.rs --> roles.rs

```

```
common/middleware.rs --> jwt.rs
common/middleware.rs --> models.rs
handlers --> services
services --> DB["database.rs (连接池类型)"]
handlers --> common/ws.rs
handlers --> common/error.rs
```

规则：业务模块只向上引用 common（公共层），不互相引用（fw100 不 import fw150）。新增模块照此办理。

3.15 2.15 条件编译：#[cfg]

```
// src/main.rs — 根据 feature 编译不同分支
let app = {
    let base = create_router(pool)
        .layer(cors)
        .route("/", get(|| async { Redirect::permanent("/admin") }));
    #[cfg(feature = "embedded")]
    let app = base.merge(embedded_assets::embedded_router()); // 生产：内存内嵌

    #[cfg(not(feature = "embedded"))]
    let app = base
        .nest_service("/admin", ServeDir::new("dist-admin").fallback(...)) // 开发：磁盘目录
        /* ... 7 个前端 ... */;
    app
};
```

写法	含义
#[cfg(feature = "embedded")]	启用 embedded feature 才编译这一段
#[cfg(not(...))]	相反
#[cfg(test)]	仅测试编译
#[cfg(debug_assertions)]	仅 debug 构建

cargo run (无 feature) → 开发模式读磁盘 dist-*; cargo build --release --features embedded → 单 exe。同一套代码两种部署形态。

3.16 2.16 测试

```
// src/fj200c_information/mock.rs — 模块内测试
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn test_generate_frame() {
        let frame = generate_frame();
        assert_eq!(frame.len(), FRAME_LEN); // 帧长必须正确
        assert_eq!(&frame[0..3], &[0xEB, 0x90, 0x64]); // 帧头必须正确
    }
}

cargo test # 跑全部测试
cargo test export_openapi # 只跑这个测试（生成 openapi.json 并防漂移校验）
```

本项目测试很少（两个位置），**重要的测试是 export_openapi** (src/api_docs.rs)：它生成 openapi.json，并断言所有 40 个路径、50 个操作都有 operationId——防漂移关卡，改接口后必须跑（06 章详述）。

3.17 2.17 新手常见编译错误速查（本项目语境）

#	报错片段	原因	本项目修法示例
1	E0382: use of moved value	无 & 传参移动了所有权	调用处加 & 或 .clone()
2	E0502: cannot borrow as mutable because it is also borrowed as immutable	借用冲突	提前结束借用作用域, 或改用 RwLock
3	E0277: the trait bound ... Send is not satisfied	类型不能跨线程	检查是否用了 Rc (应换 Arc); struct 字段加 Arc/Mutex
4	E0433: failed to resolve: use of undeclared crate or module	模块未声明/未导入	mod.rs 加 pub mod xxx; 或 use crate::xxx
5	E0601: main function not found	入口缺失	确认 main.rs 存在且 fn main
6	E0107: wrong number of lifetime parameters	生命周期标注错误	删掉多余标注让编译器推断
7	error[E0596]: cannot borrow *x as mutable	只读引用上取可变	改成 Arc<Mutex<T>> + .lock()
8	error: could not compile ... due to previous error	连锁错误	先修第一个错误, 其他多是衍生
9	the trait 'ToSchema' is not implemented for ...	DTO 没加 derive	结构体加 utoipa::ToSchema (06 章)
10	custom attribute panicked	utoipa 注解写错	检查 operation_id 是否唯一、request_body 类型是否实现了 ToSchema

万能口诀：报错 → 看行号 → 看编译器建议 → 看同文件相邻代码 → 编译 → 循环。通常 3 轮以内解决。

3.18 2.18 语法索引表（改代码时快速定位）

你想写的代码	语法	项目参考位置
定义 DTO	<code>#[derive(Serialize, Deserialize, Clone, ToSchema)] pub struct X</code>	<code>common/models.rs</code>
定义枚举（含事件）	<code>#[serde(tag="type")] pub enum E { A { .. }, B }</code>	<code>fj200c_information/mod.rs</code>
写一个 HTTP handler	<code>async fn h(State(db): State<..>, Json(b): Json<T>) -> Result<Json<ApiResponse<U>>, AppError></code>	<code>admin/handlers.rs</code>
查询数据库	<code>sqlx::query_as::<_, User>("SELECT ... WHERE id = ?").bind(id).fetch_optional(&db).await?</code>	<code>common/auth/services.rs</code>
写服务层	<code>pub struct XService; impl XService { pub async fn fn_name(&self, db: &..) -> Result<.., AppError> }</code>	各模块 <code>services.rs</code>
读配置	<code>get_config() 单例 + Config::get_or("section", "key", "default")</code>	<code>fj200c_information/config.rs</code>
日志	<code>tracing::info!("..."); tracing::error!("...")</code>	各模块 <code>service.rs</code>
启动线程	<code>thread::spawn(move { ... }) + RUNTIME.push(handle)</code>	<code>common/service.rs</code>
广播事件	<code>tx.send(Event::X { .. })</code>	各模块 <code>session.rs / service.rs</code>
验证 token	<code>jwt::verify_token(token)?</code>	<code>common/middleware.rs</code>
错误处理	<code>AppError::bad_request(msg) 或 ?</code>	<code>common/error.rs</code>
时间	<code>chrono::Utc::now() / SystemTime::now()</code>	<code>database.rs</code> 种子
hex 转换	<code>utils::parse_hex() / format_hex()</code>	<code>common/utils.rs</code>

3.19 2.19 所有权与借用：项目实战案例精讲

3.19.1 2.19.1 案例一：跨线程数据传递（session.rs 的模式）

采集线程需要持有数据源（IO）、广播发送端（tx）、停止标志。看项目怎么传：

```
// src/fj200c_information/service.rs（示意，结构还原）
let tx = fj200c_information_tx();           // broadcast::Sender 克隆（Arc 内部计数）
let stop = SERVICE_RUNNING.clone();         // Arc<AtomicBool> 克隆（共享同一标志）
let io = io;                                // Box<dyn IoControl> 唯一所有权

let handle = std::thread::spawn(move || { // move: 三样东西的所有权移入闭包
    run_one_connection(connection_index, io, tx, stop);
});
RUNTIME.push(handle);                      // JoinHandle 也存起来（后面 join）
```

变量	类型	传递方式	为什么
tx	broadcast::Sender	克隆	内部是 Arc，克隆=引用计数+1，多处共享
stop	Arc<AtomicBool>	克隆	所有线程共享同一个停止标志
io	Box<dyn IoControl>	移动	每线程一份，不需要共享
handle	JoinHandle	移动进 RUNTIME	主线程统一管理

新手要点：move 闭包把捕获的变量**全部移动**进新线程。如果之后还要用某个变量，先 .clone() 一份。这是项目最常出现的模式。

3.19.2 2.19.2 案例二：借用与修改 (SHARED_DATA 更新)

```
// src/fj200c_information/session.rs — 解码结果写全局 (RwLock 写锁)
let mut guard = SHARED_DATA.lock().unwrap(); // 拿到写锁
guard.set("ng_speed", &value.to_string()); // 修改
// guard 作用域结束自动释放锁
```

RwLock 的 lock() 返回 RwLockWriteGuard (智能指针)，支持 *guard = ... 解引用赋值，或用 guard.method()。**锁的释放是自动的：**guard 离开作用域即 drop。新手经常担心“忘了解锁”——Rust 里不存在这个问题。

3.19.3 2.19.3 案例三：避免“借用地狱”的两种写法

```
// 写法 A: 先取出再操作 (借用结束，再进入下一步)
let rows = self.rows.clone(); // 克隆出来 (小数据量可接受)
process(&rows); // 之后 rows 随使用

// 写法 B: 缩小借用作用域 (用花括号包住借用)
let sum = {
    let guard = self.data.lock().unwrap();
    guard.iter().sum::<u64>()
}; // guard 在此释放
println!("{}", sum); // 之后可再借用
```

原则：要么克隆，要么用块缩小作用域，要么用锁容器。项目三种都用了，注意分辨。

3.20 2.20 serde 属性大全 (JSON 序列化实战)

serde 是 Rust 的 JSON 神器，本项目大量使用其属性。以下是全部用法的速查表：

```
// 1. 跳过字段 (密码、内部状态)
#[serde(skip_serializing)]
pub password_hash: String, // 只跳过序列化 (还能反序列化)

// 2. 默认值 (反序列化缺字段时用)
#[serde(default)]
pub page_size: i64,

// 3. 字段重命名 (Rust snake_case → JSON camelCase)
#[serde(rename = "ngSpeed")]
pub ng_speed: f64,
// 等价全局写法 (整个结构体): #[serde(rename_all = "camelCase")]

// 4. 可选字段 (对应 TS 的 ?)
#[serde(default)]
```

```
pub description: Option<String>, // 反序列化缺省 → None

// 5. 枚举的 JSON 形态
//   #[serde(rename_all = "camelCase")] → 值变 camelCase
//   #[serde(tag = "type")]             → 内部标签（事件用）
//   #[serde(untagged)]                 → 不加标签（按字段试匹配）

// 6. 自定义转换（Serialize 手写 impl）
impl Serialize for User { ... } // 项目很少用，但可看 jwt Claims 的写法
```

项目实战观察：前端字段全是 camelCase（如 ngSpeed、faultCodes），后端 Rust 字段是 snake_case，靠 serde 属性转换——orval 生成的 TS 类型因此直接可用，无需前端再转换。

```
// src/fj200c_main/types.rs — 实际例子（节选）
#[derive(Debug, Clone, Serialize, Deserialize, Default, ToSchema)]
#[serde(rename_all = "camelCase")]
pub struct EcuFields {
    #[serde(default)]
    pub ng_speed: f64, // 转速
    #[serde(default)]
    pub torque: f64,   // 扭矩
    // ... 29 个字段
}
```

3.21 2.21 sqlx 查询语法大全（数据库实战）

本项目不用 ORM，全部手写 SQL + 绑定参数。掌握以下五种写法就能读写任何表：

3.21.1 2.21.1 查一行（可能有/无）

```
// src/common/auth/services.rs — 按邮箱查用户
let user: Option<User> = sqlx::query_as::<_, User>(
    "SELECT * FROM users WHERE email = ?1",
)
    .bind(&login_data.email) // 绑定参数（防注入）
    .fetch_optional(&db)     // Option: 0 行 → None
    .await?;
```

3.21.2 2.21.2 查多行

```
// src/admin/services.rs — 列表
let users: Vec<User> = sqlx::query_as::<_, User>(
    "SELECT * FROM users ORDER BY created_at DESC",
)
    .fetch_all(&db)
    .await?;
```

3.21.3 2.21.3 插入并返回自增/生成值

```
// src/common/auth/services.rs — 建用户
let user = sqlx::query_as::<_, User>(
    "INSERT INTO users (id, username, email, password_hash, role)
    VALUES (?, ?, ?, ?, ?)
    RETURNING *", // SQLite 支持 RETURNING
)
    .bind(user_id) // uuid::Uuid
    .bind(&login_data.username)
    .bind(&login_data.email)
    .bind(&password_hash)
    .bind("fj200c_information")
    .fetch_one(&db)
    .await?;
```

3.21.4 2.21.4 更新/删除 (不返回行)

```
// src/admin/services.rs — 改角色
let result = sqlx::query(
    "UPDATE users SET role = ?1 WHERE id = ?2",
)
    .bind(&role)
    .bind(&user_id)
    .execute(&db)
    .await?;
// result.rows_affected() 返回受影响行数，可判断是否成功
```

3.21.5 2.21.5 动态查询 (分页 + 聚合)

```
// src/city3d/services.rs — 分页
let buildings = sqlx::query_as::(<_, Building>(
    "SELECT * FROM city3d_buildings
    WHERE district_id = ?1
    ORDER BY name
    LIMIT ?2 OFFSET ?3",
)
    .bind(&district_id)
    .bind(page_size)
    .bind(offset)
    .fetch_all(&db)
    .await?;

let total: i64 = sqlx::query_scalar(
    "SELECT COUNT(*) FROM city3d_buildings WHERE district_id = ?1",
)
    .bind(&district_id)
    .fetch_one(&db)
    .await?;
```

3.21.6 2.21.6 手写 FromRow (JSON 字段解析)

```
// src/common/models.rs — user_settings 表 (含 JSON 数组字段)
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct UserSettings {
    pub columns: Vec<String>, // JSON 数组
    pub order: Vec<String>,
}

impl<'r> FromRow<'r, SqliteRow> for UserSettings {
    fn from_row(row: &'r SqliteRow) -> sqlx::Result<Self> {
        let raw: String = row.try_get("value")?; // 取原始 JSON 字符串
        let parsed = serde_json::from_str(&raw) // 手动解析
            .unwrap_or_default();
        Ok(parsed)
    }
}
```

表 → 结构体自动映射的规则：sqlx 按列名匹配结构体字段 (id → id, password_hash → password_hash)。所以数据库列名必须与 Rust 字段名一致 (snake_case)。不一致时用 SELECT id AS user_id 或手写 FromRow。

3.21.7 2.21.7 新手 sqlx 避坑

1. query_as 需要结构体实现 FromRow：普通结构体 #[derive(FromRow)]；有自定义解析的字段 (JSON/枚举) 必须手写。
2. 占位符 ? 或 ?1/?2 (SQLite 支持命名序号)，必须与 bind 顺序一致。
3. 数据库连接用 State(db): State<DatabaseConnection> 拿 (Axum 状态注入，03 章详述)。

4. 迁移没有文件：改表结构要改 database.rs 的 create_tables，并且**注意幂等**（CREATE TABLE IF NOT EXISTS）。

3.22 2.22 Axum 提取器大全 (handler 参数的秘密)

Axum 的 handler 参数**按类型自动注入**，这叫“提取器 (extractor)”。看到一个 handler，先看它的参数类型就知道它需要什么：

```
// src/common/auth/handlers.rs — 最全的提取器组合
pub async fn login(
    State(db): State<DatabaseConnection>, // ① 应用状态（数据库连接池）
    Json(login_data): Json<LoginRequest>, // ② JSON 请求体
) -> Result<Json<ApiResponse<LoginResponse>>, AppError> // ③ 返回值也自动变响应

// src/admin/handlers.rs — 路径参数
pub async fn update_user_role(
    Path(id): Path<uuid::Uuid>, // URL 路径 :id
    State(db): State<DatabaseConnection>,
    Json(body): Json<UpdateUserRoleRequest>,
) -> Result<Json<ApiResponse<User>>, AppError>

// src/city3d/handlers.rs — 查询参数
pub async fn list_buildings(
    State(db): State<DatabaseConnection>,
    Query(params): Query<PaginationParams>, // ?page=1&page_size=10
) -> Result<Json<ApiResponse<BuildingPage>>, AppError>

// src/common/middleware.rs — 自定义 Extension 提取（中间件注入的用户）
// handler 里拿当前登录用户：
Extension(user): Extension<User> // 由 permission_middleware 注入

// src/fj200c_information/handlers.rs — WebSocket 升级
pub async fn ws_handler(
    ws: WebSocketUpgrade, // 升级请求
    Query(params): Query<HashMap<String, String>>, // ?token=xxx
    State(_db): State<DatabaseConnection>,
) -> Result<Response, axum::http::StatusCode> {
    let token = params.get("token")....; // WS 不走 header, token 走查询参数
    jwt::verify_token(token)...;
    Ok(ws.on_upgrade(|socket| ws_session(socket))) // 升级为 WS 会话
}
```

提取器速查表：

提取器	用途	失败行为
State<T>	共享应用状态（数据库）	—
Json<T>	JSON 请求体（自动反序列化）	400
Query<T>	URL 查询参数	400
Path<T>	URL 路径参数	404
Extension<T>	中间件注入的数据	500（未注入）
WebSocketUpgrade	WS 升级请求	仅 WS 路由使用
HeaderMap	读取请求头	—

返回值规则：handler 返回 impl IntoResponse。Result<Json<...>, AppError>、StatusCode、Response 都行。项目统一 Result<Json<ApiResponse<T>>, AppError>。

3.23 2.23 配置文件解析 (configparser 实战)

三个模块都有 INI 配置，解析方式统一 (src/common/config.rs 封装):

```
// src/common/config.rs — INI 封装 (精简还原)
use configparser::ini::Ini;

#[derive(Clone)]
pub struct Config {
    inner: Ini,
}

impl Config {
    pub fn from_file(path: &str) -> Result<Self, String> {
        let mut ini = Ini::new();
        ini.load(path).map_err(|e| format!("加载配置失败: {e}"))?;
        Ok(Self { inner: ini })
    }

    /// 读取节-键，带默认值；节不存在返回 None -> 默认
    pub fn get_or(&self, section: &str, key: &str, default: &str) -> String {
        self.inner.get(section, key)
            .unwrap_or_else(|| default.to_string())
    }

    pub fn get_bool(&self, section: &str, key: &str) -> bool {
        self.get_or(section, key, "false").to_lowercase() == "true"
    }
}

// 使用 (fj200c_information/config.rs):
let mock_enabled = config.get_bool("Mock", "InProgress"); // true/false
let port = config.get_or("Connection0", "ComPort", "COM3"); // COM3
let baud: u32 = config.get_or("Connection0", "BaudRate",
    "115200").parse().unwrap_or(115200);
```

INI 文件长什么样 (config-fj200c_information.ini):

```
[Mock]
InProgress = true           ; 模拟模式开 (无需硬件)
[Connection0]
Enabled = true
ComPort = COM3
BaudRate = 115200
[CSV]
Enabled = true
Dir = csv
```

热加载机制 (fj200c_information 特有): 服务会话线程每轮循环**重新读配置文件** (或定期重读), 修改保存后下一帧生效, 无需重启。fj200c_main/ftj1c 则是启动时读一次, 改后需重启服务 (stop + start)。

3.24 2.24 CSV 读写实战 (csv crate)

3.24.1 2.24.1 写入 (csv_writer 封装)

```
// src/common/csv_writer.rs — 批量写入器 (500ms 刷新一次)
pub struct CsvWriter {
    path: PathBuf,
    writer: csv::Writer<BufWriter<File>>, // 缓冲写入
    buffer: Mutex<Vec<Vec<String>>>,      // 内存暂存
}
```

```
impl CsvWriter {
    pub fn new(path: &Path) -> Result<Self, String> {
        let file = File::create(path).map_err(|e| e.to_string());
        let writer = csv::Writer::from_writer(BufWriter::new(file));
        Ok(Self { path: path.to_path_buf(), writer, buffer: Mutex::new(Vec::new()) })
    }

    pub fn write_row(&self, row: Vec<String>) {
        self.buffer.lock().unwrap().push(row); // 只进内存
        // 每满 N 条或定时触发 flush
    }

    pub fn flush(&mut self) -> Result<(), String> {
        let mut rows = self.buffer.lock().unwrap().drain(..).collect::<Vec<_>>();
        for row in rows {
            self.writer.write_record(&row).map_err(|e| e.to_string());
        }
        self.writer.flush().map_err(|e| e.to_string()) // 落盘
    }
}

impl Drop for CsvWriter {
    fn drop(&mut self) {
        let _ = self.flush(); // 对象销毁时兜底刷新（防丢数据）
    }
}
```

设计要点：**高频写不直接碰磁盘**，先进内存缓冲，500ms 或满量时批量落盘——避免每帧一次磁盘 IO。

3.24.2 2.24.2 读取 (csv crate 读取)

```
// src/common/utils.rs — 读 CSV 转 Map（简化示意）
pub fn read_csv_to_map(path: &str) -> Vec<HashMap<String, String>> {
    let mut reader = csv::Reader::from_path(path).unwrap();
    let headers = reader.headers().unwrap().clone();
    reader.records()
        .filter_map(|r| r.ok())
        .map(|record| {
            headers.iter().enumerate()
                .map(|(i, h)| (h.to_string(), record.get(i).unwrap_or("").to_string()))
                .collect()
        })
        .collect()
}
```

3.24.3 2.24.3 报表插值 (csv → 报表)

```
// src/fj200c_main/report.rs — 状态点插值（简化示意）
// 从试验 CSV 中取指定状态点（如转速 30000）对应的性能值
fn fill_forward(xs: &[f64], ys: &[f64], x: f64) -> f64 {
    // 找到 x 两侧的点，线性插值
    for i in 0..xs.len() - 1 {
        if xs[i] <= x && x <= xs[i + 1] {
            let t = (x - xs[i]) / (xs[i + 1] - xs[i]);
            return ys[i] + t * (ys[i + 1] - ys[i]);
        }
    }
    ys.last().copied().unwrap_or(0.0) // 超出范围取末尾值
}
```

3.25 2.25 串口与 UDP 实战

3.25.1 2.25.1 serialport: 打开与读写

```
// src/fj200c_information/com.rs (结构还原)
use serialport::SerialPort;

pub struct SerialControl {
    port: Mutex<Box<dyn SerialPort>>, // Mutex: 串口读写在多线程间互斥
}

impl SerialControl {
    pub fn open(com_port: &str, baud: u32) -> Result<Self, String> {
        let port = serialport::new(com_port, baud)
            .data_bits(serialport::DataBits::Eight)
            .stop_bits(serialport::StopBits::One)
            .parity(serialport::Parity::None)
            .timeout(Duration::from_millis(100))
            .open()
            .map_err(|e| format!("打开串口 {com_port} 失败: {e}"))?;
        Ok(Self { port: Mutex::new(port) })
    }
}

impl IoControl for SerialControl {
    fn recv(&self) -> Result<Vec<u8>, String> {
        let mut buf = [0u8; 256];
        let n = self.port.lock().unwrap().read(&mut buf) // 阻塞读, 超时 100ms
            .map_err(|e| e.to_string())?;
        Ok(buf[..n].to_vec())
    }
}
```

新手注意: 串口 read 是阻塞的, 所以整个硬件采集都在 std::thread 里跑, 绝不进 tokio。

3.25.2 2.25.2 socket2 + tokio: UDP 组播

```
// src/ftj1c/udp.rs (结构还原) — UDP 组播接收
use socket2::{Socket, Domain, Type, Protocol, SockAddr};

let address: std::net::SocketAddr = ip.parse()?;
let socket = Socket::new(Domain::IPV4, Type::DGRAM, Some(Protocol::UDP))?;
socket.set_reuse_address(true)?; // 组播必备
socket.bind(&SockAddr::from(address))?; // 绑定本地端口
socket.join_multicast_v4(&multicast_addr, &local_addr)?; // 加入组播组
// 1MB 接收缓冲 (组播高吞吐)
socket.set_recv_buffer_size(1024 * 1024)?;
// 转为 tokio 异步 UDP socket
let udp: tokio::net::UdpSocket = socket.into();
```

UDP 是无连接的, 接收循环: udp.recv_from(&mut buf).await 拿到数据报和来源地址。ftj1c 用 std::thread + 阻塞 socket2 还是 tokio 看具体实现, 但模式一致: 一个收发线程 + 广播。

3.26 2.26 rust-embed: 把前端嵌进 exe

```
// src/embedded_assets.rs
use rust_embed::RustEmbed;

#[derive(RustEmbed)]
#[folder = "frontend/admin/dist/"] // 编译期读取的目录 (相对 crate 根)
pub struct AdminAssets;

#[derive(RustEmbed)]
```

```
#[folder = "frontend/fj200c_information/dist/"]
pub struct Fj200cInformationAssets;
// ... 7 个应用各一个

// 读取内嵌文件: AdminAssets::get("index.html") → Option<EmbeddedFile>
// 泛型处理器:
pub async fn serve_embedded<A: RustEmbed>(path: &str) -> Response {
    if path.is_empty() {
        return serve_index:::<A>(); // 根路径 → index.html
    }
    match A::get(path) {
        Some(file) => { // 命中 → 返回文件 (带 MIME)
            let mime = mime_guess::from_path(path).first_or_octet_stream();
            [(header::CONTENT_TYPE, mime.as_ref()), file.data.as_ref()].into_response()
        }
        None => serve_index:::<A>(), // 未命中 → SPA 回退 index.html
    }
}
```

部署流程: 前端先 npm run build (产生 dist) → cargo build --features embedded (把 dist 编译进二进制) → 单 exe 自带全部页面。这就是“为什么顺序不可颠倒”的原因。

3.27 2.27 常用标准库与第三方类型速查

类型	说明	项目例子
uuid::Uuid	主键	用户 id、建筑 id
chrono::NaiveDateTime / DateTime<Utc>	时间	created_at
PathBuf / Path	文件路径	CSV 目录
Duration / Instant	时间间隔	超时、节流
serde_json::json!	构建 JSON	错误响应
Box<T>	堆分配	trait 对象
VecDeque	双端队列	环形缓冲
HashSet	集合	权限判重 (可查)

```
// src/database.rs — UUID 生成种子 (从固定值递增, 保证幂等)
let base = Uuid::from_u128(0x00000000000000000000000000000001); // 固定起始
for i in 1..=51 {
    let id = Uuid::from_u128(base.as_u128() + i as u128); // 递增
    // INSERT OR IGNORE ...
}

// src/common/utils.rs — 时间戳毫秒
pub fn now_ms() -> u128 {
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_millis())
        .unwrap_or(0)
}
```

3.28 2.28 学习资源与路线图 (如果你还想深入 Rust)

本项目代码之外, 推荐按顺序补充:

1. 官方书《The Rust Programming Language》(rust book) : 前 10 章 (所有权、借用、结构体、枚举、模块、集合、错误处理、trait) ——在线阅读免费, 中文翻译 rustwiki.cn。

2. **Rust 语言圣经 (course.rs)**：中文社区经典，更通俗。
3. **《Rust 程序设计 (Programming Rust)》**：进阶必读。
4. **axum 官方示例**：github.com/tokio-rs/axum/tree/main/examples——本项目很多写法就是从官方例子学来的。
5. **tokio 官方教程**：tokio.rs/tokio/tutorial——理解 async 生态。
6. **SQLite 官方文档**：了解 RETURNING、ON CONFLICT、WAL 模式（本项目全用了）。

练习建议：- 改 fw100 加一个字段（最小改动热身）。- 给 fj200c_information 加一个新接口（走完整 utoipa → gen:api 流程）。- 自己写一个小模块（照 role_template）。

3.29 2.29 读代码演练一：jwt.rs 逐行精读（154 行）

把 02 章学的所有概念放一起，逐行走读 src/common/jwt.rs 全文件。这个文件是全项目最“教科书”的文件——注释详细到每个语法点。

3.29.1 2.29.1 模块文档（1-31 行）

```

//! # JWT 令牌模块
//!
//! 本模块提供 JWT（JSON Web Token）令牌的创建和验证功能。
//!
//! # JWT 结构
//!
//! JWT 由三部分组成，用 `.` 分隔：
//! ```text
//! Header.Payload.Signature
//! ```
//!
//! - **Header**：算法和令牌类型
//! - **Payload**：载荷数据（用户 ID、过期时间等）
//! - **Signature**：签名（防止篡改）

```

//! 是模块级文档注释，写在文件顶部。rust-analyzer 悬停能看到。项目每个文件都有这种“文件说明书”，读新文件先扫这段。

3.29.2 2.29.2 导入（33-38 行）

```

use crate::common::models::User; // 用户结构体
use jsonwebtoken::{decode, encode, DecodingKey, EncodingKey, Header, Validation};
use serde::{Deserialize, Serialize};
use std::env;
use uuid::Uuid;

```

导入	用途	归属
User	create_token 需要用户信息	本项目 (crate::common::models)
jsonwebtoken::{...}	JWT 编解码	第三方库
serde::{...}	Claims 的序列化	第三方库
std::env	读环境变量	标准库
uuid::Uuid	用户 ID 类型	第三方库

新手读法：扫一眼 import 就知道这个文件依赖什么；看到 crate:: 就是项目内部模块。

3.29.3 2.29.3 Claims 结构体（56-64 行）

```

#[derive(Debug, Serialize, Deserialize)]
pub struct Claims {
    pub sub: String, // 主题（用户 ID）
    pub exp: usize, // 过期时间（Unix 时间戳秒）
}

```

```
pub iat: usize, // 签发时间 (Unix 时间戳秒)
}
```

注意: sub/exp/iat 是 JWT 标准字段名 (规范要求), 所以这里**没有**用 snake_case——标准字段名必须遵守。

3.29.4 2.29.4 create_token (86-116 行)

```
pub fn create_token(user: &User) -> Result<String, jsonwebtoken::errors::Error> {
```

- &User: 只读借用, 不拿走用户数据。
- 返回 Result<String, jsonwebtoken::errors::Error>: 失败类型是库自带的错误。

```
let secret = env::var("JWT_SECRET").unwrap_or_else(|_| "your-secret-key".to_string());
let expiration = env::var("JWT_EXPIRATION")
    .unwrap_or_else(|_| "86400".to_string())
    .parse::<u64>()
    .unwrap_or(86400);
```

配置读取三连: env::var 读 → unwrap_or_else 兜底 → parse 转类型 (解析失败再兜底)。

```
let now = chrono::Utc::now();
let expires_at = now + chrono::Duration::seconds(expiration as i64);

let claims = Claims {
    sub: user.id.to_string(),
    exp: expires_at.timestamp() as usize,
    iat: now.timestamp() as usize,
};
```

- chrono::Utc::now() 当前 UTC 时间; 时间运算用 Duration::seconds。
- as usize: **as 是 Rust 的类型强转** (类似 JS 的 Number(x), 但 Rust 只做数值转换)。
- 结构体初始化: 字段名简写 (sub: user.id.to_string() 是完整写法, 注意这里不是简写)。

```
encode(
    &Header::default(),
    &claims,
    &EncodingKey::from_secret(secret.as_ref()),
)
}
```

- Header::default(): 默认头 (HS256)。
- EncodingKey::from_secret(secret.as_ref()): &String → &str (as_ref() 自动转换)。
- 最后的 encode(...) **没有分号**: 这是“尾表达式”, 即函数的返回值。等同于 return encode(...);。

3.29.5 2.29.5 verify_token (138-153 行)

```
pub fn verify_token(token: &str) -> Result<Uuid, jsonwebtoken::errors::Error> {
    let secret = env::var("JWT_SECRET").unwrap_or_else(|_| "your-secret-key".to_string());

    let token_data = decode::<Claims>(
        token,
        &DecodingKey::from_secret(secret.as_ref()),
        &Validation::default(),
    )?;
```

- decode::<Claims>: **泛型函数**——<Claims> 指定解码成什么类型。
- ? : 解码失败直接返回错误 (调用方处理)。

```
Uuid::parse_str(&token_data.claims.sub)
    .map_err(|_| jsonwebtoken::errors::Error::from(
        jsonwebtoken::errors::ErrorKind::InvalidToken))
}
```

- `Uuid::parse_str` 把字符串转回 UUID，失败时 `map_err` 转成 JWT 库的错误类型（保持函数签名一致）。
- 整个表达式是尾表达式 → 返回值。

读完这个文件，你应该能回答： 1. JWT 的三部分是什么？各存什么？ 2. 密钥从哪来？默认值是什么？为什么生产必须改？ 3. 过期时间怎么算的？默认多久？ 4. ？在这里做什么？`map_err` 呢？尾表达式呢？

3.30 2.30 读代码演练二：config.rs 逐行精读（84 行）

再精读一个更小的文件 `src/config.rs`——环境变量加载。

```

//! # 应用配置模块
//!
//! 本模块负责从环境变量加载应用配置。
//!
//! # 配置项
//!
//! | 环境变量 | 类型 | 默认值 | 说明 |
//! |-----|-----|-----|-----|
//! | `PORT` | u16 | 3000 | 服务器监听端口 |
//! | `DATABASE_URL` | String | `sqlite://rustweb.db` | 数据库连接 URL |

```

模块文档还带 markdown 表格——项目注释规范：**配置项全部列在文件头。**

```

#[derive(Debug, Deserialize)]
pub struct AppConfig {
    pub port: u16,
    pub database_url: String,
}

```

注意：这里 `Deserialize` 是给谁用的？**目前代码是手动 `env::var` 读取的**，`derive` 是为了将来兼容（或已废弃用法）。这是项目的“历史包袱”，读到类似代码不用慌——**不用的 `derive` 不影响运行。**

```

pub fn load() -> Result<Self, Box<dyn std::error::Error>> {
    let port = env::var("PORT")
        .unwrap_or_else(|_| "3000".to_string())
        .parse::<u16>()?;

    let database_url = env::var("DATABASE_URL")
        .unwrap_or_else(|_| "sqlite://rustweb.db".to_string());

    Ok(AppConfig {
        port,
        database_url,
    })
}

```

这里出现了字段简写：`port` 相当于 `port: port`——局部变量名与字段同名时省略。`Box<dyn std::error::Error>` 是“任意错误”类型（trait 对象），小脚本类函数常用；本项目其他函数用更严格的 `AppError`。

读文件的方法论（以后所有文件都这么读）： 1. 先读顶部 `///` 模块文档（文件是干嘛的）。 2. 再看 `import`（依赖谁）。 3. 看公开类型/函数签名（有什么能力）。 4. 挑核心函数逐行走读。 5. 看函数里的注释（项目注释就是导学）。

3.31 2.31 读代码演练三：utils.rs 里的“奇技淫巧”

```
// src/common/utils.rs — 小端字节转 ASCII（发动机报文解码用）
pub fn little_endian_bytes_to_ascii(bytes: &[u8]) -> String {
    bytes
        .chunks(2) // 每 2 字节一组（小端 16 位字符）
        .map(|chunk| {
            let v = u16::from_le_bytes([chunk[0], chunk[1]]); // 组装小端 u16
            char::from_u32(v as u32).unwrap_or('?') // 转字符
        })
        .collect() // 收集成 String
}
```

这个函数把“报文里以小端序存储的 16 位字符”转回可读文本。读懂它的关键：**迭代器链**（chunks → map → collect），前面 2.13.4 讲过。

```
// src/common/utils.rs — 十六进制解析与格式化
pub fn parse_hex(hex_str: &str) -> Vec<u8> {
    // "EB 90 64" / "eb9064" → [0xEB, 0x90, 0x64]
    hex_str
        .split(|c: char| c.is_ascii_whitespace()) // 按空白分割
        .filter(|s| !s.is_empty())
        .filter_map(|s| u8::from_str_radix(s, 16).ok()) // 16 进制解析，失败跳过
        .collect()
}
```

filter_map = 过滤 + 转换一步完成（失败就丢掉）。前端“命令通道”发送 hex 指令就靠它。

```
// src/common/utils.rs — 时间戳
pub fn now_ms() -> u128 {
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_millis())
        .unwrap_or(0)
}
```

SystemTime::now().duration_since(UNIX_EPOCH) 返回 Result（理论上可能系统时间在 1970 前，实际不会），unwrap_or(0) 兜底。

这些工具函数是给硬件模块用的——理解硬件模块前先扫一眼 utils.rs 的十几个函数，后面读 session.rs/decode.rs 会轻松很多。

3.32 2.32 深入：泛型与 trait 对象（读懂 embed 与 quad_frame）

3.32.1 2.32.1 泛型结构体

```
// src/common/quad_frame.rs — 常量泛型（const generic）
pub struct QuadFrame<const FRAME_LEN: usize> {
    frames: [ArcSwap<[u8; FRAME_LEN]>; 4], // FRAME_LEN 是编译期常量
    sequence: AtomicU32,
}
// 使用：QuadFrame<95> — ftj1c 的帧长 95 字节
```

<const FRAME_LEN: usize> 是“常量泛型”：类型参数是数值而不是类型。数组 [T; N] 的长度必须是编译期常量，所以用常量泛型参数化。

3.32.2 2.32.2 泛型函数与 trait 约束

```
// src/embedded_assets.rs — 泛型 + trait 约束
pub async fn serve_embedded<A: RustEmbed>(path: &str) -> Response {
    match A::get(path) { ... }
}
```


<A: RustEmbed>: A 是实现了 RustEmbed trait 的类型。调用时
serve_embedded::<AdminAssets>("index.html") 指定 A。

3.32.3 2.32.3 泛型 vs trait 对象 (dyn)

```
// 泛型: 编译期展开 (静态分发, 更快, 代码膨胀)
fn handle<T: IoControl>(io: &T) { io.recv(); }

// trait 对象: 运行时查虚表 (动态分发, 灵活, 略微慢)
fn handle(io: &dyn IoControl) { io.recv(); }
```

本项目两个都用: **集合里存异构类型用 dyn** (Box<dyn IoControl>), **单点调用用泛型**。新手看到 dyn 就理解为“接口引用”, 看到 <T: Trait> 就理解为“类型参数约束”。

3.32.4 2.32.4 Arc vs Rc (多线程 vs 单线程)

```
// Rc<T>: 单线程引用计数 (多线程编译报错!)
// Arc<T>: 原子引用计数 (多线程安全)
// 本项目所有共享都是 Arc, 因为采集线程+HTTP 跨线程
```

新手如果写单线程组件用了 Rc 而代码运行在多线程上下文, 编译器会直接报错并提示换 Arc——照做即可。

3.32.5 2.32.5 Mutex vs RwLock vs Atomic

场景	选型
写多读多	Mutex<T> (简单可靠)
读多写少	RwLock<T> (读并发)
单个布尔/数字	AtomicBool / AtomicU8 (最快)
高频读低频写的大数据	ArcSwap<T> (无锁读)

本项目对照: 停止标志=AtomicBool, CSV 状态机=AtomicU8, SHARED_DATA=带锁容器, 最新帧=ArcSwap。选型逻辑一目了然。

3.33 2.33 深入: 闭包与函数式风格

3.33.1 2.33.1 闭包写法

```
// 完整写法: |参数| { 函数体 }
let f = |x: i64| { x * 2 };

// 省略写法 (类型推断)
let f = |x| x * 2;

// 多参数
let g = |a, b| a + b;

// 移动捕获 (move 关键字): 把环境变量移动进闭包
thread::spawn(move || run_one_connection(...)); // 项目最常见
```

3.33.2 2.33.2 闭包在项目中的用法清单

用途	例子
线程体	<code>thread::spawn(move { ... })</code>
错误兜底	<code>.unwrap_or_else(_ "default".to_string())</code>
错误转换	<code>.map_err(e AppError::bad_request(e.to_string()))</code>
单例初始化	<code>ONCELOCK.get_or_init(broadcast::channel(1024).0)</code>
集合转换	<code>.map(d d.name.clone())</code>
条件过滤	<code>.filter(u u.role == "admin")</code>

读法： `|参数| 表达式` 就是“一个临时函数”；看到 `move ||` 就是“把这个函数连同它用到的变量一起搬到另一个线程”。

3.34 2.34 深入：常见困惑辨析（Rust 新手高频问题）

3.34.1 2.34.1 String 和 &str 到底怎么选？

场景	选
函数参数（只读）	<code>&str</code> （可接受 <code>String</code> 和 <code>&str</code> 传入）
返回值（新数据）	<code>String</code> （拥有所有权，可修改）
常量/字面量	<code>&'static str</code>
结构体字段	<code>String</code> （除非特意共享只读）

```
// 项目范例
pub fn get(&self, key: &str) -> Option<String> {    // 参数 &str, 返回 String
    self.map.get(key).cloned()                    // cloned: Option<&String> ->
Option<String>
}
```

3.34.2 2.34.2 Result 和 Option 怎么分？

- Option：值**可能不存在**（无错误概念）。
- Result：操作**可能失败**（有错误信息）。
- 转换：Option → Result 用 `.ok_or(msg)?`；Result → Option 用 `.ok()`。

3.34.3 2.34.3 ? 到底怎么工作？

```
fn f() -> Result<u32, AppError> {
    let x: u32 = g().map_err(|e| AppError::bad_request(e.to_string()))?;
    // 等价于：
    let x: u32 = match g() {
        Ok(v) => v,
        Err(e) => return Err(AppError::bad_request(e.to_string())),
    };
    Ok(x)
}
```

? 需要“错误类型可转换”：From<E> for AppError 存在时直接 ?；否则先 `map_err`。这是 2.9 节 From 转换的实际效果。

3.34.4 2.34.4 mut 什么时候必须加？

- 变量声明后要**重新赋值**：`let mut x = 1; x = 2;`
- 要调用 `&mut self` 方法（如 `vec.push()`）——注意：`let mut v = vec![1]; v.push(2);` `push` 需要可变借用。

- 结构体字段可变：let mut user = user; user.role = ... (需要 mut 绑定)。

3.34.5 2.34.5 为什么结构体字段全是 pub 也没关系？

本项目结构体字段几乎都是 pub，因为模块间频繁跨层传递数据 (handler → service → 模型)，pub 简化访问。**项目内部代码**这么约定没问题；如果要对外发布库，才需要封装。**别在项目里引入 getter/setter 风格**，保持统一。

3.34.6 2.34.6 编译慢怎么办？

```
cargo check    # 只检查类型，不生成二进制（快很多）
cargo build    # 完整编译
cargo run      # build + 运行
```

依赖第一次编译慢 (tokio/axum/sqlx 全家桶)，之后增量编译很快。改一个小文件用 cargo check 足够。

3.34.7 2.34.7 调试输出怎么办？

```
// 1. 日志（生产也用）
tracing::info!("用户登录成功: {}", user.email);
tracing::debug!("帧数据: {:?}", frame);
tracing::error!("串口打开失败: {e}");
```

```
// 2. println!（快速临时调试，用完全删）
println!("debug: {:?}", x);
```

```
// 3. dbg! 宏（打印并返回值，可插在表达式中间）
let x = dbg!(compute());
```

RUST_LOG=debug cargo run 打开调试日志看全链路。

3.35 2.35 本章自测：你能独立读这段代码吗？

最后做个小测验。不看 2.29 节，独立读这段真实代码 (src/common/middleware.rs 的权限中间件，摘录)，回答三个问题：

```
pub async fn permission_middleware(
    required_permission: Permission,
    State(db): State<DatabaseConnection>,
    mut request: Request,
    next: Next,
) -> Result<Response, StatusCode> {
    let user_id = extract_user_id(&request)?;
    let user = load_user(&db, user_id).await?;
    if !user.has_permission(&required_permission) {
        return Err(StatusCode::FORBIDDEN);
    }
    request.extensions_mut().insert(user);
    Ok(next.run(request).await)
}
```

问题： 1. 这个函数是异步的吗？如何看出？ 2. ? 在这里传播的是什么错误类型？ 3. request.extensions_mut().insert(user) 在干嘛？ user 为什么能放进去？

参考答案： 1. 是，async fn 关键字。 2. ? 传播 StatusCode (函数返回 Result<Response, StatusCode>; extract_user_id 返回 Result<Uuid, StatusCode>, 类型匹配直接 ?)。 3. 把当前登录用户塞进请求的扩展区 (一个 HashMap)，后续 handler 用 Extension(user): Extension<User> 取出来用。这是 Axum 中间件向 handler 传数据的标准机制。

如果三个问题都能答对，你的 Rust 阅读能力已经足够支撑本项目日常开发了。本章目标达成。

3.36 2.36 深入：tokio 任务并发模型（spawn / join / select）

3.36.1 2.36.1 tokio::spawn：异步任务

```
// 把 async 任务丢进 tokio 线程池并发执行，返回 JoinHandle
let handle = tokio::spawn(async move {
    do_something().await
});
// 之后可以 handle.await 等待结果
```

项目里 async 任务主要用于：WS 会话、初始化、spawn_blocking。注意区分：
std::thread::spawn 创建的是**系统线程**（硬件阻塞用），tokio::spawn 创建的是**异步任务**（IO 等待用）。

3.36.2 2.36.2 tokio::join! / try_join!：并发等待

```
// join!：并发执行多个 future，都完成才返回
let (a, b) = tokio::join!(f1(), f2());

// try_join!：任一失败立即返回 Err
let (a, b) = tokio::try_join!(f1(), f2())?;
```

3.36.3 2.36.3 tokio::time：定时任务

```
// 每 500ms 执行一次（CSV 刷新、状态轮询等场景）
let mut interval = tokio::time::interval(Duration::from_millis(500));
loop {
    interval.tick().await;
    csv_writer.flush();
}
```

3.36.4 2.36.4 项目里的异步/线程混合模型总结

flowchart TD

```
subgraph tokio[Tokio 运行时]
    H[HTTP handlers<br/>async fn]
    W[WS 会话<br/>ws_bridge]
    S[spawn_blocking<br/>bcrypt 密码校验]
    I[定时任务<br/>500ms CSV flush]
end
subgraph threads[std::thread]
    C[采集线程<br/>阻塞串口读]
    U[UDP 收发线程]
end
H -->|start/stop 命令| C
C -->|broadcast| W
W -->|WS 帧| Browser
S -->|从 tokio 调用| H
```

为什么混合：串口 read 是阻塞 API，Tokio 的异步模型管不了它；HTTP 是 async 友好的。两者用 broadcast 桥接后互不干扰。这是本项目的核心并发设计，理解它胜过背十条语法。

3.37 2.37 深入：tracing 日志系统

3.37.1 2.37.1 用法

```
use tracing::{info, warn, error, debug, trace};

info!("服务已启动");
debug!("收到帧：{:?}", frame);
error!("启动失败：{:e}");
```

3.37.2 2.37.2 级别与过滤

```
RUST_LOG=info cargo run      # 只显示 info 及以上
RUST_LOG=debug cargo run     # 显示 debug 及以上（含帧级日志）
RUST_LOG=trace cargo run     # 全量（数据量巨大，谨慎）
RUST_LOG=warn cargo run      # 只显示警告和错误
```

3.37.3 2.37.3 输出格式

2026-08-08T10:00:00.123Z INFO rust_web_backend::fj200c_information::service: 服务启动成功

带时间戳、日志级别、模块路径——模块路径帮你定位代码位置。

3.37.4 2.37.4 项目日志点分布

位置	日志内容
main.rs	启动配置、绑定地址
database.rs	数据库初始化
各 service.rs	服务启停、配置加载
各 session.rs	连接建立/断开、异常
各 handlers.rs	接口调用（部分）

新手排障第一步永远是： RUST_LOG=debug cargo run，看日志输出，再打开 F12 Network。

3.38 2.38 深入：测试编写（项目已有测试的解剖）

3.38.1 2.38.1 项目里的两个测试

// 测试 1: api_docs.rs 的 export_openapi 测试（防漂移关卡，必跑）

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn export_openapi() {
        let spec = generate_openapi_spec();
        let json = serde_json::to_string_pretty(&spec).unwrap();
        std::fs::write("openapi/openapi.json", json).unwrap();

        // 断言：所有预期路径必须存在
        for path in expected_paths() {
            assert!(spec.paths.contains_key(path), "缺少路径: {path}");
        }
        // 断言：所有操作必须有 operationId
        for op in collect_all_operations(&spec) {
            assert!(op.operation_id.is_some(), "operation 缺少 operationId");
        }
    }
}
```

跑法：cargo test export_openapi。它的作用是**漂移检测**：新增接口没注解、改路径没同步，测试就失败——保证 openapi.json 永远与代码一致。

// 测试 2: fj200c_information/mock.rs 的模拟数据测试

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_generate_frame() {
```

```

    let frame = generate_frame();
    assert_eq!(frame.len(), FRAME_LEN);           // 帧长 100
    assert_eq!(&frame[0..3], &[0xEB, 0x90, 0x64]); // 帧头
    assert_ne!(frame[3], 0);                       // 序号非 0
  }
}

```

3.38.2 2.38.2 测试语法速成

```

#[test]                                // 标记测试函数
fn test_xxx() { ... }

#[tokio::test]                          // 异步测试（项目里需要时用）
async fn test_async() { ... }

assert!(condition);                    // 断言真
assert_eq!(a, b);                      // 断言相等
assert_ne!(a, b);                      // 断言不等
assert!(a.is_ok());                    // 断言 Result 成功

```

3.38.3 2.38.3 如何给新代码补测试

给“纯函数”补测试最划算：decode、校验、工具函数、状态机逻辑。

```

// 例：给 decode 补测试（示意）
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_validate_frame() {
        let frame = [0xEB, 0x90, 0x64, /* 96 字节数据 */, 0x00];
        assert!(validate_frame(&frame).is_ok());
        let bad = [0xEB, 0x90, 0x64, 0xFF, 0xFF]; // 长度不足
        assert!(validate_frame(&bad).is_err());
    }
}

```

给项目补测试的建议：接手后优先给 decode 校验、hex 工具、CSV 状态机补测试——这些是硬件模块的正确性根基，且无需硬件即可测。

3.39 2.39 本项目惯用代码模式十式（改代码时照抄）

以下模式遍布全项目，**改代码时直接照抄对应模式**，不要自创风格：

第一式：统一响应 handler

```

#[utoipa::path(get, tag = "xxx", path = "/api/xxx/items", operation_id = "xxxListItems",
    responses((status = 200, description = "列表", body = ApiResponse<Vec<LedgerItem>>)))]
pub async fn list_items(
    State(db): State<DatabaseConnection>,
) -> Result<Json<ApiResponse<Vec<LedgerItem>>>, AppError> {
    let items = XxxService::list_items(&db).await?;
    Ok(Json(ApiResponse::success(items)))
}

```

第二式：service 层查询

```

pub struct XxxService;
impl XxxService {
    pub async fn list_items(db: &DatabaseConnection) -> Result<Vec<LedgerItem>, AppError> {
        Ok(sqlx::query_as::(<_, LedgerItem>("SELECT * FROM xxx")
            .fetch_all(db).await?))
    }
}

```

```
    }
}
```

第三式：启动服务编排

```
pub fn start_service() -> Result<(), String> {
    let tx = xxx_tx();
    for i in 0..N {
        if !enabled(i) { continue; }
        let handle = std::thread::spawn(move || run_worker(i, tx));
        RUNTIME.push(handle);
    }
    Ok(())
}
```

第四式：停止服务

```
pub fn stop_service() {
    SERVICE_RUNNING.set_stopped(); // 置停止标志
    RUNTIME.wait_stopping(3);       // join 最多 3 秒
}
```

第五式：WS 事件推送

```
let event = XxxEvent::Data { ... };
let _ = tx.send(event); // 忽略错误：没有订阅者也无所谓
```

第六式：从数据库取用户（中间件）

```
async fn load_user(db: &DatabaseConnection, user_id: uuid::Uuid) -> Result<User,
StatusCode> {
    sqlx::query_as::<_, User>("SELECT * FROM users WHERE id = ?1")
        .bind(user_id)
        .fetch_optional(db)
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?
        .ok_or(StatusCode::UNAUTHORIZED)
}
```

第七式：配置读取（带默认值）

```
let value = get_config().get_or("Section", "Key", "default");
let flag = get_config().get_bool("Section", "Flag");
```

第八式：Json 响应构造

```
Ok(Json(ApiResponse::success(ServiceStatus { running: true })))
```

第九式：错误兜底链

```
let port = env::var("PORT").unwrap_or_else(|_| "3000".to_string()).parse()?;
```

第十式：日志+异常处理

```
match risky_operation() {
    Ok(v) => { tracing::info!("成功"); v }
    Err(e) => { tracing::error!("失败: {e}"); return
Err(AppError::internal(e.to_string())); }
}
```

3.40 2.40 第二章收官：你现在的 Rust 水平能做什么

读完本章，你应当具备：

1. 读：任何项目 .rs 文件，逐行读懂（借 rust-analyzer 悬停）。
2. 抄：照着“惯用模式十式”写新的 handler/service/启动逻辑。

3. **改**：改现有功能的字段、逻辑、配置——改完 cargo check 看编译错误迭代。
4. **测**：给纯函数补 #[test]。
5. **排**：用 RUST_LOG=debug + 日志定位问题。

还不会的（没关系，进阶再看）：unsafe 代码、高级生命周期、宏编写、复杂 trait 设计、性能优化。项目代码里 95% 都用不到这些。

下一章预告：03 章会把所有语法放进真实模块——从 main.rs 启动流程开始，逐模块走读后端全部代码。

3.41 2.41 深入：所有权与 move 的图解理解

很多新手卡在所有权上，是因为把它当成“魔法规则”。其实它是**内存管理模型**。用图理解：

3.41.1 2.41.1 值、变量与所有权

```
let s = String::from("hello");
```

```
flowchart LR
    subgraph 栈["栈（变量表）"]
        S["s: 指向堆的指针 + 长度 + 容量"]
    end
    subgraph 堆["堆（数据）"]
        H["'hello' 字符串数据"]
    end
    S -.-> H
```

- s 拥有堆上那块字符串数据的**所有权**。
- Rust 规定：**一个数据在同一时刻只能有一个所有者**。

3.41.2 2.41.2 移动 (move)：所有权转移

```
let a = String::from("hello");
let b = a;           // 所有权从 a 移到 b
println!("{}", a);   // 编译错误！a 已失去所有权
```

```
flowchart LR
    subgraph 栈2["移动后"]
        A["a: 已失效（编译器禁止使用）"]
        B["b: 新所有者"]
    end
    subgraph 堆2
        H2["'hello'"]
    end
    B -.-> H2
```

为什么设计成这样：如果 a 和 b 都指向同一块内存，a 销毁时把内存 free 了，b 就悬空 (use-after-free)。Rust 干脆禁止这种状态。

3.41.3 2.41.3 克隆 (clone)：数据复制

```
let a = String::from("hello");
let b = a.clone();    // 深拷贝，两个所有者各有一份数据
```

```
flowchart LR
    subgraph 栈3
        A3["a"]
        B3["b"]
    end
    subgraph 堆3
        H3a["'hello'"]
        H3b["'hello'（复制）"]
    end
    A3 -.-> H3a
    B3 -.-> H3b
```


3.41.4 2.41.4 借用 (borrow): 不转移所有权

```
let a = String::from("hello");
let len = a.len();           // 借用 &a (编译器自动), a 仍拥有所有权
println!("{}", a);          // ✓ 还能用
```

flowchart LR

```
    subgraph 栈4
        A4["a: 所有者"]
        L4["&a: 借用 (临时视图)"]
    end
    end
    subgraph 堆4
        H4["'hello'"]
    end
    end
    A4 -.-> H4
    L4 -.- 只读访问 -.-> H4
```

3.41.5 2.41.5 项目中 move 的实际体现 (再回看一遍)

```
// src/fj200c_information/service.rs (结构还原)
let tx = fj200c_information_tx(); // Sender 克隆 (Arc 计数+1, 所有权各自独立)
std::thread::spawn(move || {     // tx 被 move 进线程闭包
    run_one_connection(i, io, tx);
});
// 循环外还能用 tx (因为之前克隆了)
```

Sender::clone() 内部是 Arc 计数: 每个线程持有的 Sender 都是同一通道的“引用”, 谁 drop 自己的副本都不影响别人。这是跨线程共享的标准做法: 要么 Arc, 要么克隆。

3.41.6 2.41.6 所有权思维口诀

1. **传参**: 默认借用 &; 要修改传 &mut; 要转移所有权直接传值。
2. **返回值**: 返回 owned 值 (String/Vec) 而不是引用 (避免悬空)。
3. **结构体字段**: String/Vec 拥有数据; 需要共享用 Arc; 需要可变共享用 Arc<Mutex<...>>。
4. **编译器就是老师**: 报借用错误时, 90% 的修复是加 &、加 .clone()、或把变量移进正确的所有权位置。

3.42 2.42 深入: 生命周期标注到底在说什么

3.42.1 2.42.1 为什么要生命周期

```
fn first_word(s: &str) -> &str {
    // 返回的 &str 必须活得和 s 一样久, 否则调用者拿到悬空引用
}
```

编译器需要保证: 函数返回的引用不会指向已被释放的内存。生命周期标注就是“给编译器提供证明材料”。

3.42.2 2.42.2 省略规则 (90% 情况不用写)

```
fn get_name(&self) -> &str { &self.name } // 规则: 一个输入引用, 输出默认可推断
fn longest(a: &str, b: &str) -> &str { ... } // X 多个输入, 编译器无法推断 → 必须标注
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str { ... } // ✓ 标注: 返回值活不过 a/b 中最短者
```

3.42.3 2.42.3 项目中的 'a 出现在哪

```
// src/common/models.rs — FromRow 手写实现 (生命周期 'r: 行数据借用)
impl<'r> FromRow<'r, SqliteRow> for UserSettings {
    fn from_row(row: &'r SqliteRow) -> sqlx::Result<Self> { ... }
}
```

这里 `'r` 表示：实现过程中借用的 row 引用，其生命周期与实现绑定。**新手策略：这种代码直接照抄模板，不要自己设计。**

3.42.4 2.42.4 'static 生命周期

```
let s: &'static str = "hello";           // 字符串字面量：编译期存在于二进制中，永远有效
pub const CONFIG_PATH: &str = "config-fj200c_information.ini"; // 隐式 'static
```

'static 不一定是“程序永远运行”，而是“这个数据不会在程序结束前被释放”。字符串字面量、const 常量都是。

3.43 2.43 深入：chrono 与 uuid 实战

3.43.1 2.43.1 chrono：时间处理

```
// 当前时间
let now = chrono::Utc::now();           // 2026-08-08T10:00:00Z（带时区）
let local = chrono::Local::now();       // 本地时区

// 格式化
now.format("%Y-%m-%d %H:%M:%S").to_string() // "2026-08-08 10:00:00"

// 解析
chrono::NaiveDateTime::parse_from_str(s, "%Y-%m-%d %H:%M:%S")

// 加减
now + chrono::Duration::seconds(86400)    // 明天
now - chrono::Duration::days(7)          // 上周

// 时间戳
now.timestamp()                          // 秒
now.timestamp_millis()                   // 毫秒
```

项目里：created_at 字段、JWT 过期时间、种子数据、CSV 文件名时间戳。

3.43.2 2.43.2 uuid：主键生成

```
uuid::Uuid::new_v4()                    // 随机 UUID (v4)
uuid::Uuid::parse_str("00000000-0000-4000-8000-000000000001") // 从字符串解析
uuid.as_u128()                          // 转 u128（种子递增用）
uuid.to_string()                        // 转字符串
```

项目里：所有表主键、种子数据固定 UUID（保证 INSERT OR IGNORE 幂等）。

3.43.3 2.43.3 serde_json 实战

```
// 构造 JSON (json! 宏，类似 JS 对象字面量)
let body = serde_json::json!({
    "success": false,
    "message": "密码错误",
});

// 结构体 → JSON 字符串
let json_str = serde_json::to_string(&user)?;

// JSON 字符串 → 结构体
let user: User = serde_json::from_str(&json_str)?;

// 任意值取字段
let val = serde_json::from_str::<serde_json::Value>(&raw)?;
let name = val["name"].as_str().unwrap_or("");
```

项目里：错误响应、WS 事件序列化、user_settings 的 JSON 字段、试验信息存储 (GlobalVar 以 JSON 存)。

3.44 2.44 给新手的三个“热身练习”（改代码前先做）

正式动手改项目前，建议先做三个热身练习（每个 10 分钟，改完 cargo check）：

练习 1：读函数——打开 src/common/utils.rs，把每个函数的签名读出来，猜用途，然后看注释核对。

练习 2：改日志——在 src/common/auth/handlers.rs 的 login 里加一行 tracing::info!("登录尝试: {}", login_data.email);，cargo run 启动，用前端登录一次，观察日志输出。（练完删除）

练习 3：改配置默认值——把 src/config.rs 的 PORT 默认值从 "3000" 改为 "3001"，cargo run，访问 localhost:3001/health 验证。（练完改回）

练习 4（进阶）：加一个测试——给 src/common/utils.rs 的 parse_hex 写测试：

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_parse_hex() {
        assert_eq!(parse_hex("EB 90 64"), vec![0xEB, 0x90, 0x64]);
        assert_eq!(parse_hex("eb9064"), vec![0xEB, 0x90, 0x64]);
        assert!(parse_hex("").is_empty());
    }
}
```

跑 cargo test 验证通过。这四个练习做完，你已经能在项目里改代码而不心虚了。

3.45 2.45 深入：异步编程中的常见模式（tokio 实战）

3.45.1 2.45.1 spawn 独立任务（后台运行）

```
// 项目模式：tokio::spawn 启动后台任务，主流程继续
tokio::spawn(async move {
    loop {
        // 后台循环（如心跳、轮询、重连）
        tokio::time::sleep(Duration::from_secs(1)).await;
        if SHARED.is_stopped() { break; }
    }
});
```

适用场景：会话线程、监听循环、定时清理。注意 async move 把需要的值移进任务。

3.45.2 2.45.2 select! 多路等待（谁先到处理谁）

```
// tokio::select!：等待多个 future，先完成的执行
tokio::select! {
    _ = rx.recv() => { /* 收到事件 */ }
    _ = tokio::time::sleep(Duration::from_millis(200)) => { /* 超时 */ }
}
```

适用场景：会话循环里“等事件 vs 等超时”——fj200c_information 会话线程就用它做 200ms 超时判断。

3.45.3 2.45.3 broadcast 广播通道

```
// 一对多：所有订阅者收到同一事件
let (tx, rx) = tokio::sync::broadcast::channel::<XxxEvent>(128);
```

```
// 每个 WS 连接 subscribe() 拿一个 rx, 互不干扰
let mut rx = tx.subscribe();
```

适用场景：WS 广播（N 个浏览器订阅同一数据流）。

3.45.4 2.45.4 mpsc 多对一（或者一对多串行）

```
// 有界通道：生产者可以多个，消费者一个
let (tx, mut rx) = tokio::sync::mpsc::channel::<XxxEvent>(128);
```

适用场景：CSV 写入队列（采样线程 → 写盘线程）。

3.45.5 2.45.5 RwLock / Mutex 的选择

```
// 写少读多 → RwLock（shared 状态）
// 写多读少 → Mutex
// 热更新热点配置 → ArcSwap（无锁）
```

3.46 2.46 深入：Trait 设计模式（读懂接口抽象）

3.46.1 2.46.1 为什么用 trait 抽象硬件

```
flowchart LR
    subgraph 实现
        A[SerialControl<br/>真实串口]
        B[MockControl<br/>模拟器]
        C[UdpControl<br/>UDP 组播]
    end
    subgraph 接口
        T[Trait IoControl]
    end
    T --> A
    T --> B
    T --> C
```

业务代码只依赖 trait，不依赖具体实现——换硬件 = 换实现，业务代码零改动。

3.46.2 2.46.2 动态分发 vs 静态分发

```
// 静态分发（编译期确定）：泛型 <T: IoControl>
fn run<T: IoControl>(io: &mut T) { ... }

// 动态分发（运行期确定）：trait object Box<dyn IoControl>
let io: Box<dyn IoControl> = if mock { Box::new(MockControl::new()) } else
{ Box::new(SerialControl::new()) };
```

项目做法：配置驱动（ini 的 Mock 开关）选择实现——动态分发更灵活。

3.46.3 2.46.3 trait 的默认实现

```
trait IoControl {
    fn recv(&mut self) -> Result<Vec<u8>, io::Error>;
    fn send(&mut self, data: &[u8]) -> Result<usize, io::Error> { // 默认实现
        let _ = data;
        Ok(0) // 只读设备不需要实现 send
    }
}
```

价值：新实现只需要实现必要的少数方法。

3.47 2.47 深入：Rust 项目常见编译报错对照（后端）

报错	含义	修复
borrow of moved value	所有权被移走	clone / 借用 &
cannot borrow as mutable	需要 mut 引用	声明 let mut / &mut
lifetime may not live long enough	生命周期不足	加生命周期参数 / 改所有权
expected &str, found String	类型不匹配	&s / s.as_str()
the trait bound X: Send is not satisfied	跨线程不安全	加 Send/Sync 约束或用 Arc
no method named xxx	方法不存在	检查 trait 是否导入
mismatched types	类型不一致	查看两个类型并转换
unused variable	变量未用	加 _ 前缀或删除
warning: unused import	导入未用	删除导入

调试技巧：rust-analyzer 的悬停/转到定义 + cargo check 快速反馈，比任何文档都准。

3.48 2.48 本章语法点索引（速查表）

语法	章节	项目位置
变量/类型/函数	2.2~2.5	所有文件
所有权/借用	2.6~2.8	所有函数
Option/Result	2.10~2.13	所有 handler
枚举/match	2.15~2.16	Permission、事件枚举
struct/impl	2.17~2.18	DTO、ServiceRuntime
trait	2.19~2.21	IoControl
泛型	2.22	ApiResponse
闭包	2.24	map/filter 链
生命周期	2.25	函数签名
async/await	2.27~2.29	handler
线程	2.31~2.32	会话线程
通道	2.33	broadcast/mpsc
ArcSwap	2.34	热更新
serde	2.36~2.37	DTO 序列化
sqlx	2.38~2.39	services
configparser	2.40	配置读取
csv	2.41	CSV 记录
serialport	2.42	串口
时间/uuid/json	2.43	工具

改代码时：先查表定位语法章节，再看对应项目代码实例——这是最快的上手方式。

3.49 2.49 深入：Result 的错误链 (? 操作符与错误转换)

3.49.1 2.49.1 从底层错误到 AppError

```
// 项目模式: sqlx 错误 → AppError
pub async fn get_user(db: &SqlitePool, id: i64) -> Result<UserInfo, AppError> {
    sqlx::query_as:::<_, UserInfo>("SELECT ...")
        .bind(id)
        .fetch_one(db)
        .await
        .map_err(|e| AppError::internal(format!("查询用户失败: {e}"))) // 显式转换
}
```

3.49.2 2.49.2 From 转换 (自动 ?)

```
// 实现 From<sqlx::Error> for AppError 后, ? 自动转换:
let user = sqlx::query_as(...).fetch_one(db).await?; // 无需 map_err
```

项目写法: 显式 map_err (保留上下文) 与 From (简洁) 并用。

3.49.3 2.49.3 自定义错误链的调试价值

底层错误: database disk image is malformed
→ 转换后: 查询用户失败: database disk image is malformed
→ 前端: 失败信息直达用户

教训: 错误消息带上层信息, 排查日志时能定位到具体操作。

3.50 2.50 深入：字符串处理的实战模式

3.50.1 2.50.1 拼接与格式化

```
// format! 最常用
let path = format!("{}", dir, name);
let msg = format!("连接 {} 失败: {}", port, e);

// 集合拼接
let joined = list.join(","); // Vec<String> → String
let split: Vec<&str> = s.split(',').collect();

// 大小写/去空白
let clean = s.trim().to_lowercase();
```

3.50.2 2.50.2 子串与查找

```
s.contains("SYSJSK") // 包含
s.starts_with("EB") // 前缀
s.find('=')          // 位置
s[..n]              // 切片 (注意边界!)
```

3.50.3 2.50.3 解析数字

```
"123".parse:::<i64>()? // 失败返回 Result
s.chars().next()       // 首字符
```

项目位置: CSV 文件名解析、协议帧字符串解析、ini 值转换。

3.51 2.51 深入：集合类型的选择（什么时候用什么）

类型	特点	项目用途
Vec	有序可重复	列表查询结果
HashMap<K,V>	键值无序	配置节、字段映射
HashSet	去重	权限集合
VecDeque	双端队列	环形缓冲
BTreeMap	有序	需要排序时

```
// 查找时优先 Option 风格
let port = config.get("port").unwrap_or("COM3");
let exists = set.contains(&key);
```

3.52 2.52 深入：并发原语在项目中的分布

```
flowchart LR
    subgraph 并发原语
        A[OnceLock<br/>全局单例]
        B[ArcSwap<br/>热配置]
        C[AtomicBool<br/>运行标志]
        D[Mutex/RwLock<br/>共享数据]
        E[broadcast<br/>广播]
        F[mpsc<br/>串行队列]
    end
```

原语	项目位置	选型理由
OnceLock	各模块 TX/SHARED	一次初始化，全局访问
ArcSwap	config.rs 热更新	读多写少无锁
AtomicBool	SERVICE_RUNNING	跨线程布尔
broadcast	WS 广播	一对多
mpsc	CSV 队列	一写一读

选型口诀：共享数据读多 → ArcSwap；需同步 → Mutex；广播 → broadcast；流水 → mpsc。

3.53 2.53 深入：宏（macro_rules!）在项目中的实战

3.53.1 2.53.1 项目实例：define_com_port!

```
// src/fj200c_main/com.rs
macro_rules! define_com_port {
    ($name:ident, $header:expr) => {
        pub struct $name { ... }
        impl SerialControl for $name { ... }
    };
}
define_com_port!(ECUCom, 0xEB);
define_com_port!(AdamCom, 0x3E);
define_com_port!(DynoCom, 0xFF);
```

作用：三路串口实现几乎一样，用宏消除重复代码。

3.53.2 2.53.2 什么时候用宏

1. 重复的样板代码 ≥3 处（结构体/impl 同构）
2. 编译期计算（数字转换）
3. 需要捕获调用位置（file!()/line!()）

新手原则：先复制粘贴，重复到忍不了再用宏。

3.54 2.54 深入：条件编译与 feature（项目实例）

3.54.1 2.54.1 embedded feature

```
# Cargo.toml
[features]
default = []
embedded = []          # 启用前端内嵌

// main.rs
#[cfg(feature = "embedded")]
mod embedded_assets;    // 仅 embedded 构建编译

#[cfg(not(feature = "embedded"))]
// 默认模式：读磁盘 dist-* 目录
```

3.54.2 2.54.2 cfg 的其他用法

```
#[cfg(debug_assertions)] // 开发构建
#[cfg(windows)]          // 平台
#[cfg(test)]             // 测试
```

价值：同一份代码按构建模式差异化——单 exe 与开发模式共存。

3.55 2.55 深入：测试与代码检查习惯（后端）

3.55.1 2.55.1 三层检查

```
cargo check      # 快查（秒级）——开发时常用
cargo test       # 跑测试（含 openapi 生成）——提交前必跑
cargo clippy     # lint 建议（可选）——质量提升
```

3.55.2 2.55.2 测试的组织

单元测试：#[cfg(test)] mod tests 写在同文件（工具函数）
集成测试：tests/ 目录（端到端）
专用测试：export_openapi（生成 + 断言）

习惯养成：每次改完核心工具函数，cargo test 一把。

3.56 2.56 深入：项目里的安全编码习惯

1. 密码 bcrypt 哈希（不存明文）
2. 输入校验（validator 库 / 手动）
3. SQL 参数化（sqlx bind，防注入）
4. 路径处理（防目录穿越：CSV 文件名校验）
5. 日志不输出敏感信息

核心原则：永远不信任外部输入（HTTP 参数、ini 值、串口数据）。

3.57 2.57 深入：02 章补充自测（10 题）

1. ? 操作符如何做错误转换？
2. 错误消息为什么要带上层上下文？
3. Vec/HashMap/Set 各适合什么？
4. 并发原语选型口诀？
5. 项目里宏解决了什么问题？
6. embedded feature 如何条件编译？
7. cargo check/test/clippy 的区别？
8. 防 SQL 注入的做法？
9. 为什么不存明文密码？

10. 外部输入为什么不能信任?

答对 8+ → 02 章语法关彻底通过。

3.58 2.58 深入：模式匹配的实战全集

3.58.1 2.58.1 常见的匹配用法

```
// 枚举 + 数据
match msg {
  WsMessage::Frame(f) => handle_frame(f),
  WsMessage::Status(s) => handle_status(s),
  _ => {} // 忽略其他
}

// 数字范围
match code {
  0..=99 => "小",
  100..=999 => "大",
  _ => "超大",
}

// 守卫
match x {
  n if n % 2 == 0 => "偶数",
  _ => "奇数",
}
```

3.58.2 2.58.2 let-else (项目常见)

```
let Some(token) = token else {
  return Err(AppError::Unauthorized("未登录".into()));
};
// token 已解包，后面直接用
```

注意：let-else 的 else 分支必须返回 (return/break/continue)。

3.59 2.59 深入：生命周期与借用的常见编译错误

编译错误	含义	常见修复
borrowed value does not live long enough	借用超过所有者存活期	改传引用为传所有权/用 Arc
cannot borrow as mutable more than once	可变借用冲突	缩小作用域/换 RwLock
use of moved value	移动后再用	传引用/先 clone
expected lifetime parameter	缺少生命周期标注	加 'a 或改结构

项目里的处理：全局状态用 ArcSwap/OnceLock 绕开大部分借用难题。

3.60 2.60 深入：所有权与性能的权衡

原则：能借用不拷贝，能传引用不传值

但：小数据（数字/短串）copy/clone 无感知

大数据（Vec/结构体）优先借用

```
// 项目实例：解析帧 → 借用切片 → 转换成自有 Vec
let frame: &[u8] = extract(bytes); // 借用
let fields = decode(frame); // 产出新结构
```

3.61 2.61 深入：日志 (tracing/log) 的使用规范

```
use tracing::{info, warn, error, debug};
```

```
// 项目规范
info!("服务启动, 端口: {}", port);
warn!("连接 {} 心跳超时, 重连中", port);
error!("解析帧失败: {:?}", err);
debug!("接收 {} 字节", bytes.len());
```

级别选择: debug 细节, info 状态, warn 可恢复异常, error 不可恢复
运行时控制: RUST_LOG=info / RUST_LOG=debug

3.62 2.62 深入: async 代码的注意事项

3.62.1 2.62.1 不要在 async 里干重活

```
// ❌ 阻塞: 大计算会卡住整个执行器
let result = heavy_calc();
// ✅ tokio::task::spawn_blocking
let result = tokio::task::spawn_blocking(heavy_calc).await?;
```

3.62.2 2.62.2 锁的使用

```
// ❌ 持锁 .await (锁跨异步边界)
let guard = mutex.lock().await;
do_something().await; // 危险
// ✅ 先拿数据再释放, 再 await
```

3.62.3 2.62.3 常见 async 模式

1. tokio::spawn 后台任务 (CSV 写入、心跳)
2. broadcast 广播
3. interval 定时器

3.63 2.63 深入: 02 章最终综合自测 (追加 10 题)

1. let-else 的 else 必须做什么?
2. 守卫 (guard) 怎么用?
3. 三种常见借用错误怎么修?
4. 大数据什么时候用借用?
5. 日志四个级别的选择?
6. RUST_LOG 怎么控制级别?
7. async 里重计算怎么办?
8. 持锁 await 为什么危险?
9. interval 定时器的场景?
10. broadcast 与 mpsc 的区别?

答对 8+ → 02 章最终通过。

3.64 2.64 深入: 迭代器链的实战翻译 (新手对照)

// 需求: 从 Vec<Frame> 里挑出转速 > 1000 的前 5 帧的转速值

```
let speeds: Vec<f64> = frames.iter()
    .filter(|f| f.ng_speed > 1000.0) // 过滤
    .map(|f| f.ng_speed)             // 转换
    .take(5)                         // 取前 5
    .collect();                     // 收集
```

// 逐句翻译

```
// frames.iter()      → 拿迭代器 (不拿所有权)
// filter(闭包)       → 保留满足条件的
// map(闭包)          → 每个元素做变换
// take(5)            → 只要前 5 个
// collect()          → 转成 Vec
```

3.64.1 2.64.1 其他常用迭代器

```
frames.iter().find(|f| f.id == 10) // 找第一个
frames.iter().any(|f| f.ng_speed > 0) // 是否存在
frames.iter().all(|f| f.ng_speed >= 0) // 是否全部
frames.iter().fold(0.0, |acc, f| acc + f.ng_speed) // 累加
frames.iter().max_by(|a, b| a.ng_speed.total_cmp(&b.ng_speed)) // 最大值
```

3.65 2.65 深入：常见数值类型转换速查

转换	写法	注意
i32 → f64	x as f64	as 可能精度损失
String → i64	s.parse::<i64>()?	失败返回 Err
f64 → i32	x as i32	截断
&str → String	s.to_string()	常用
String → &str	s.as_str()	借用
Vec → 数组	v.try_into().unwrap()	长度必须一致
u8 → 十六进制字符串	format!("{:02X}", b)	协议调试常用

3.66 2.66 深入：处理 None 的四种姿势

```
// 1. unwrap_or: 给默认值
let port = config.get("port").unwrap_or("COM3");

// 2. unwrap_or_else: 惰性计算默认值
let dir = config.get("dir").unwrap_or_else(|| default_dir());

// 3. ? 传播
let token = extract_token(req).ok_or(AppError::Unauthorized("未登录".into()))?;

// 4. if let 处理
if let Some(user) = users.get(0) {
    println!("第一个用户: {}", user.name);
}
```

戒律：生产代码禁用裸 `unwrap()`（除测试）——用上述姿势替代。

3.67 2.67 深入：闭包捕获的三种方式

方式	场景	例子
借用（默认）	读取外层变量	<code>map(x x + offset)</code>
可变借用 <code>mut</code>	修改外层变量	<code>for_each(x counter += 1)</code>
<code>move</code>	跨线程/所有权转移	<code>tokio::spawn(async move { ... })</code>

```
// move 在异步任务中最常见
let tx = tx.clone();
tokio::spawn(async move { tx.send(data).await });
```

3.68 2.68 深入：时间与随机数处理

```
// 时间戳
let now = SystemTime::now().duration_since(UNIX_EPOCH)?.as_secs();
// 或 chrono 库（项目常用）
let ts = chrono::Local::now().format("%Y-%m-%d %H:%M:%S");

// 随机数（rand 库）
let id: u32 = rand::random();
```

3.69 2.69 深入：02 章超纲自测（5 题）

1. 迭代器链每步的作用？
2. fold 与 reduce 的区别（写法）？
3. as 转换的精度损失场景？
4. unwrap 的替代姿势有哪些？
5. move 闭包什么时候必须用？

答对 4+ → 02 章超纲完成。

3.70 2.70 深入：结构体与枚举的完整实战

3.70.1 2.70.1 结构体三兄弟

```
// 普通结构体
struct User { id: i64, name: String }

// 元组结构体（字段无名）
struct Point(f64, f64);

// 单元结构体（无字段，作标记）
struct Marker;

// 使用
let u = User { id: 1, name: "a".into() };
let p = Point(0.0, 1.0);
```

3.70.2 2.70.2 结构体更新语法

```
let u2 = User { id: 2, ..u }; // 其余字段从 u 复制（u 被部分移动）
```

3.70.3 2.70.3 枚举携带数据（项目核心模式）

```
enum WsEvent {
    Frame(TableRow),
    Status(ServiceStatus),
    Error(String),
}
```

3.71 2.71 深入：trait 的完整实战（impl Trait 与泛型）

3.71.1 2.71.1 trait 定义与实现

```
trait SerialControl {
    fn open(&mut self) -> Result<(), SerialError>;
    fn read_frame(&mut self) -> Result<Frame, SerialError>;
}

impl SerialControl for ECUCom {
    fn open(&mut self) -> Result<(), SerialError> { /* ... */ }
    fn read_frame(&mut self) -> Result<Frame, SerialError> { /* ... */ }
}
```

3.71.2 2.71.2 泛型约束

```
// 任何实现 SerialControl 的类型都能用
fn monitor<T: SerialControl>(mut com: T) { ... }

// trait 对象（动态分发）
let com: Box<dyn SerialControl> = Box::new(ECUCom::new());
```

3.71.3 2.71.3 项目中的典型应用

1. SerialControl: 三路串口统一接口（抽象层）
2. ToSchema: DTO 统一生成文档
3. From<X> for AppError: 错误统一转换

3.72 2.72 深入：工程目录组织的 Rust 惯例

```
src/
├─ main.rs      # 入口（薄）
├─ lib.rs       # 库入口（有 crate 时）
├─ common/      # 共享模块
├─ admin/       # 业务模块
│   └─ mod.rs   # 模块声明 + 内部 use
│   └─ handlers.rs # HTTP 层
│   └─ services.rs # 业务层
└─ routes.rs    # 路由聚合
```

3.72.1 2.72.1 模块声明的两种方式

```
mod common;    // 单文件: src/common.rs
mod admin;     // 目录: src/admin/mod.rs
```

3.72.2 2.72.2 可见性

```
pub          # 公开
pub(crate)   # 仅本 crate
pub(super)   # 仅父模块
（默认私有）
```

3.73 2.73 深入：02 章实战自测（8 题）

1. 三种结构体的区别？
2. 枚举携带数据的场景？
3. trait 对象 vs 泛型约束？
4. 项目里三个典型 trait？
5. 目录组织惯例？
6. 两种模块声明方式？
7. pub(crate) 的意义？
8. 更新语法 ..u 的作用？

答对 7+ → 02 章实战通过。

3.74 2.74 深入：文件系统操作的完整参考（后端实战）

3.74.1 2.74.1 读写文件

```
// 读整个文件
let content = std::fs::read_to_string("config.ini");

// 写文件（覆盖）
std::fs::write("report.csv", csv_content);

// 追加
use std::io::Write;
let mut f = std::fs::OpenOptions::new()
    .append(true).create(true).open("log.txt");
f.write_all(b"new line\n");
```

3.74.2 2.74.2 目录操作

```
std::fs::create_dir_all("csv/2026"); // 递归创建
std::fs::read_dir("csv");             // 遍历
// 文件名过滤
```

```
let csvs: Vec<_> = read_dir("csv")?
    .filter_map(|e| e.ok())
    .filter(|e| e.path().extension().is_some_and(|x| x == "csv"))
    .collect();
```

3.74.3 2.74.3 项目中的位置

1. config-*.ini 读写（配置管理）
2. csv/ 目录创建与文件命名（记录模块）
3. 报表生成写文件
4. 路径拼接用 Path::join（别用字符串 +）

3.75 2.75 深入：序列化的完整参考（serde 实战）

3.75.1 2.75.1 派生宏

```
#[derive(Serialize, Deserialize)]
pub struct Point { pub x: i32, pub y: i32 }
```

// 序列化

```
let json = serde_json::to_string(&point)?;
```

// 反序列化

```
let p: Point = serde_json::from_str(&json)?;
```

3.75.2 2.75.2 常用属性

```
#[serde(rename_all = "camelCase")]    // 字段命名转换
#[serde(skip_serializing)]            // 序列化跳过
#[serde(default)]                    // 缺失时默认值
#[serde(alias = "old_name")]          // 兼容旧字段名
```

3.75.3 2.75.3 Option 字段的处理

```
pub remark: Option<String>    // 缺失 → None，前端可见 undefined
pub count: Option<i64>        // 数值可选
```

3.76 2.76 深入：02 章高频自测（8 题）

1. 三种文件读写方式？
2. Path::join 为什么优于字符串拼接？
3. 遍历目录过滤文件的方法？
4. Serialize/Deserialize 的区别？
5. rename_all 的作用？
6. skip_serializing 的用途？
7. Option 字段缺省的表现？
8. create_dir_all 与 create_dir 的区别？

答对 7+ → 02 章高频通过。

3.77 2.77 深入：异步编程的完整参考（tokio 实战）

3.77.1 2.77.1 async/await 基础

```
async fn fetch_data() -> Result<String, Error> {
    // 非阻塞等待
    let data = network_call().await?;
    Ok(data)
}
```

// 调用方

```
#[tokio::main]
async fn main() {
```

```
    let result = fetch_data().await;
}
```

3.77.2 2.77.2 tokio::spawn 并发任务

```
let handle = tokio::spawn(async {
    // 后台任务
    loop { work().await; }
});
// handle.await 等待完成（可选）
```

3.77.3 2.77.3 并发执行的组合

```
// 并发执行两个任务（等待都完成）
let (a, b) = tokio::join!(task1(), task2());

// 择一完成（谁先回来用谁）
tokio::select! {
    v = task1() => println!("task1 先完成: {v}"),
    v = task2() => println!("task2 先完成: {v}"),
}
```

3.77.4 2.77.4 项目中的应用

1. spawn: 串口读线程、CSV 写线程、WS 广播
2. interval: 心跳、节流
3. select: 主备切换（心跳超时 vs 数据）
4. join: 并行初始化

3.78 2.78 深入：常见数据结构的实用操作

3.78.1 2.78.1 Vec 常用操作

```
let mut v = vec![1, 2, 3];
v.push(4);           // 尾部添加
v.pop();             // 尾部取出
v.insert(0, 0);      // 头部插入
v.remove(0);         // 删除指定位置
v.contains(&2);       // 包含
v.sort();            // 排序
v.dedup();           // 去重（需先排序）
let slice = &v[1..3]; // 切片
```

3.78.2 2.78.2 HashMap 常用操作

```
let mut m = HashMap::new();
m.insert("key".to_string(), 1);
m.get("key");           // Option<&V>
m.entry("key").or_insert(0); // 不存在则插入默认
m.remove("key");
m.contains_key("key");
```

3.78.3 2.78.3 String 常用操作

```
let mut s = String::from("hello");
s.push_str(" world"); // 追加
s.push('!');          // 追加字符
s.replace("l", "L");  // 替换
s.chars().count();    // 字符数（非字节数）
```

3.79 2.79 深入：02 章综合自测（8 题）

1. spawn 与 await 的区别？
2. join! 与 select! 的区别？
3. 主备切换用哪个原语？

4. Vec 去重的步骤?
5. entry().or_insert() 的作用?
6. chars().count() 与 len() 的区别?
7. 并行初始化的方式?
8. 后台任务的退出方式?

答对 7+ → 02 章综合通过。

3.80 2.80 深入：常用 crate 的实用 API 速查

3.80.1 2.80.1 chrono (时间)

```
use chrono::prelude::*;

let now = Local::now();
println!("{}", now.format("%Y-%m-%d %H:%M:%S"));
let date = NaiveDate::from_ymd_opt(2026, 8, 8).unwrap();
```

3.80.2 2.80.2 serde_json (JSON)

```
use serde_json::{json, Value};

let v = json!({ "name": "设备A", "count": 3 });
let s = serde_json::to_string(&v)?;
let parsed: Value = serde_json::from_str(&s)?;
parsed["name"].as_str()
```

3.80.3 2.80.3 regex (正则)

```
use regex::Regex;

let re = Regex::new(r"^[COM\d+]+$"); // 串口号校验
re.is_match("COM3")
```

3.80.4 2.80.4 anyhow (错误处理，测试/工具用)

```
use anyhow::Result;

fn main() -> Result<()> {
    let content = std::fs::read_to_string("x.ini")?; // 自动转换
    Ok(())
}
```

3.81 2.81 深入：新手常犯的 Rust 错误及修复

错误	原因	修复
E0308 类型不匹配	类型错误	看期望类型，转换
E0596 不可变借用	需要 mut	变量加 mut
E0502 借用冲突	同时可变/不可变借用	重排代码
E0382 使用已移动	移动后再用	传引用/clone
E0277 trait 未实现	缺 trait 约束	加约束
E0658 不稳定特性	用了 nightly API	换稳定写法
E0433 找不到模块	模块未声明	mod 声明
E0405 找不到 trait	未导入	use 导入

3.82 2.82 深入：代码风格与命名规范（项目惯例）

3.82.1 2.82.1 命名规范

类型/结构/枚举: PascalCase (UserInfo, ServiceStatus)

函数/变量/模块: snake_case (get_user, tx, db)

常量: SCREAMING_SNAKE (MAX_RETRY)

trait: PascalCase (SerialControl)

3.82.2 2.82.2 组织规范

1. 每函数 ≤ 50 行（超了拆函数）
2. 模块入口 mod.rs 声明 + 重导出
3. 错误优先返回 (Result 早退出)
4. 注释解释“为什么”而非“是什么”

3.82.3 2.82.3 格式化

cargo fmt # 自动格式化（提交前必跑）

3.83 2.83 深入：02 章终局自测（8 题）

1. chrono 格式化时间的写法？
2. json! 宏的用途？
3. 正则校验串口号？
4. anyhow 的？自动转换？
5. 五种常见编译错误？
6. 命名规范三条？
7. 函数长度建议？
8. cargo fmt 的作用？

答对 7+ → 02 章终局通过。

3.84 2.84 深入：阅读 Rust 代码的实战方法

3.84.1 2.84.1 从签名读起

// 先读函数签名，理解输入输出

```
pub async fn get_config(db: &SqlitePool) -> Result<Config, AppError>
```

// 输入：数据库池（借用）

// 输出：Config 或 AppError

// 不读实现也能猜到用途

3.84.2 2.84.2 跟踪类型流向

State<AppState> → &SqlitePool → query_as::<_, Config>

→ Config → Json<ApiResponse<Config>>

（类型贯穿：结构体 DTO 决定数据形态）

3.84.3 2.84.3 找关键模式

1. .await?: 异步调用 + 错误传播
2. .ok_or(Err)? : Option → Result
3. .unwrap_or: 给默认值
4. Arc / Mutex: 共享状态
5. spawn: 后台任务

3.85 2.85 深入：Rust 版本的常用特性（项目所用）

3.85.1 2.85.1 常用特性清单

// let-else (2021 edition 稳定)

```
let Some(v) = opt else { return Err(...) };
```

```
// 格式化字符串 (2021)
let s = format!("{name}: {value}");

// if let / while let
if let Some(x) = opt { ... }
while let Some(x) = iter.next() { ... }

// 闭包捕获
let f = |x| x + 1;
```

3.85.2 2.85.2 用不上但要知道的

1. const 泛型: QuadFrame<95> (数组长度泛型)
2. impl Trait 返回: fn() -> impl Iterator
3. 解构: let (a, b) = pair;

3.86 2.86 深入: 02 章毕业自测 (8 题)

1. 怎么读一个函数签名?
2. 类型流向怎么跟踪?
3. 五种关键模式?
4. let-else 的写法?
5. const 泛型的例子?
6. 解构的写法?
7. while let 的场景?
8. 闭包捕获的三种方式?

答对 7+ → 02 章毕业。

3.87 2.87 深入: 实际读一段项目代码 (综合实战)

3.87.1 2.87.1 目标代码

```
// src/fj200c_information/com.rs (结构示意图: 串口连接管理)
pub struct SerialManager {
    port: Box<dyn SerialPort>, // 动态 trait 对象
    config: ArcSwap<ConnConfig>, // 可热更新配置
    running: Arc<AtomicBool>, // 运行标志
}

impl SerialManager {
    pub fn new(config: Arc<ArcSwap<ConnConfig>>) -> Result<Self, AppError> {
        let cfg = config.load_full(); // 读当前配置
        let port = open_port(&cfg)?; // 打开串口
        Ok(Self { port, config, running: Arc::new(AtomicBool::new(false)) })
    }

    pub fn start(&self) {
        self.running.store(true, Ordering::SeqCst);
    }

    pub fn stop(&self) {
        self.running.store(false, Ordering::SeqCst);
    }
}
```

3.87.2 2.87.2 逐段解读

1. Box<dyn SerialPort>: trait 对象 (多实现通用)
2. ArcSwap: 配置热更新 (读多写少)
3. Arc<AtomicBool>: 跨线程共享运行标志

4. `load_full()`: 读取当前配置快照
5. `Ordering::SeqCst`: 最强一致性 (简单场景够用)

3.87.3 2.87.3 知识点串联

所有权: 结构体持有资源

借用: `&self` 方法 (读不写)

`trait` 对象: `Box<dyn ...>`

原子类型: `AtomicBool`

智能指针: `Arc` / `ArcSwap`

泛型与配置: `ConnConfig`

3.88 2.88 深入: 02 章大师自测 (8 题)

1. `Box` 的意义?
2. `ArcSwap` 为什么适合配置?
3. `AtomicBool` 的用途?
4. `load_full` 返回什么?
5. `Ordering::SeqCst` 是什么?
6. `&self` 与 `&mut self` 的区别?
7. `trait` 对象的动态分发?
8. `Arc` 的引用计数机制?

答对 7+ → 02 章大师。

3.89 2.89 深入: 命令行参数与环境变量

3.89.1 2.89.1 环境变量 (项目主要方式)

// `dotenv`: 从 `.env` 读取

`use dotenv::dotenv;`

```
fn main() {
    dotenv().ok(); // 加载 .env (不存在不报错)
    let port = std::env::var("PORT").unwrap_or_else(|_| "3000".into());
    let db_url = std::env::var("DATABASE_URL")
        .unwrap_or_else(|_| "sqlite://rustweb.db".into());
}
```

3.89.2 2.89.2 命令参数 (较少用)

`let args: Vec<String> = std::env::args().collect();`

// `args[0]` = 程序名, `args[1..]` = 参数

3.89.3 2.89.3 项目约定

1. 配置优先级: 环境变量 > `ini` 默认值
2. `.env` 不存在 → 自动生成 (`main.rs`)
3. 敏感信息 (`JWT_SECRET`) 走环境变量
4. 开发与部署共享同一逻辑

3.90 2.90 深入: 错误处理的风格指南

3.90.1 2.90.1 三种错误风格

1. `Result` 传播 (?): 业务代码主流
2. `Option` 语义: 可能不存在 (查找)
3. `panic`: 编程错误 (不捕获, 直接崩)

3.90.2 2.90.2 什么时候 `panic`

1. 测试断言
2. 启动时必需资源缺失 (可明确报错)

3. 逻辑不可能分支 (expect 说明原因)

→ 运行期业务错误永远用 Result

3.90.3 2.90.3 错误消息的写法

好: 连接 COM3 失败: 系统找不到指定的文件

差: failed

好: 登录失败: 邮箱或密码错误

差: error

(带上下文 + 具体原因)

3.91 2.91 深入: 02 章权威自测 (8 题)

1. 环境变量的读取写法?
2. dotenv 的作用?
3. 配置优先级?
4. 三种错误风格?
5. 什么时候 panic?
6. 错误消息的规范?
7. 敏感信息放哪?
8. .env 不存在怎么办?

答对 7+ → 02 章权威。

3.92 2.92 深入: 调试 Rust 代码的实战技巧

3.92.1 2.92.1 打印调试

```
// 标准打印
println!("{:?}", value);           // Debug 输出
println!("{:#?}", value);         // 格式化输出 (更易读)
```

```
// 日志
tracing::debug!("帧: {:?}", frame);
```

3.92.2 2.92.2 断言调试

```
// 测试/开发期快速验证
debug_assert!(frame.len() >= 8, "帧太短");
debug_assert_eq!(checksum, 0x5A);
// debug_assert 只在调试构建生效 (发布构建自动移除)
```

3.92.3 2.92.3 编译器提示的使用

1. 报错信息末尾常有 help 建议
2. cargo check 比 build 快 (先 check)
3. 复杂错误 → 拆小函数逐步验证

3.93 2.93 深入: 从语法到项目的迁移路径

3.93.1 2.93.1 语法知识的三个等级

- 一级 (认识): 看到代码知道在干嘛
- 二级 (会写): 能模仿写出相同模式
- 三级 (会用): 根据需求选对模式

3.93.2 2.93.2 项目练习清单 (按等级)

- 一级: 读 03 章所有代码块, 标注用到的语法
- 二级: 改写现有小函数 (加字段/改逻辑)
- 三级: 独立实现 08 章案例

3.93.3 2.93.3 常见误区

1. 背语法 → 没用，要用中理解
2. 只看不写 → 记不住
3. 一次学完 → 忘了
4. 不看报错 → 错过学习机会

3.94 2.94 深入：02 章权威自测（8 题）

1. 两种打印调试写法？
2. debug_assert 的特性？
3. 编译报错的 help 怎么用？
4. check 与 build 的区别？
5. 三个等级的能力要求？
6. 练习清单怎么分配？
7. 四个学习误区？
8. 报错信息的价值？

答对 7+ → 02 章权威。

3.95 2.95 深入：零基础者的 30 天 Rust 计划

3.95.1 2.95.1 四周计划

- 第一周：环境 + 基础语法（变量/函数/所有权）
→ 完成 2.1~2.20 节
- 第二周：进阶（结构体/枚举/泛型/trait）
→ 完成 2.21~2.45 节
- 第三周：异步 + 并发（tokio）
→ 完成 2.46~2.60 节
- 第四周：项目实战（对照 03 章）
→ 读模块代码 + 改小功能

3.95.2 2.95.2 每日安排（1~2 小时）

1. 30 分钟：看本套教程 2~3 节
2. 40 分钟：打开项目源码对照
3. 30 分钟：写小练习（改/仿写）
4. 20 分钟：自测题

3.95.3 2.95.3 学习的产出物

1. 环境可跑（cargo run 出 Hello）
2. 能读懂 03 章大部分代码
3. 能仿写一个模块骨架
4. 能独立完成一个字段级改动

3.96 2.96 深入：Rust 官方资源导航

3.96.1 2.96.1 必看资源

1. Rust 官方 Book（入门首选）
2. rust-analyzer（编辑器增强）
3. cargo doc --open（本地文档）
4. docs.rs（crate 文档）
5. 中文社区（Rust 语言中文社区）

3.96.2 2.96.2 项目内的资源

1. 本套教程 02 章（语法 + 项目对照）
2. src/ 全部代码（真实案例）
3. AGENTS.md（项目约定）
4. Cargo.toml（依赖清单）

3.96.3 2.96.3 提问的姿势

1. 先搜（关键词 + 项目名）
2. 带完整报错信息
3. 描述期望与实际
4. 附最小复现（或路径）

3.97 2.97 深入：02 章权威自测（8 题）

1. 四周计划的划分？
2. 每日四段的安排？
3. 四个学习产出物？
4. 五个官方资源？
5. 项目内四个资源？
6. 提问的四条姿势？
7. 为什么对照源码学？
8. 本地文档怎么开？

答对 7+ → 02 章权威。

3.98 2.98 深入：学完本章的检验清单

3.98.1 2.98.1 语法检验（能看懂）

1. 能解释所有权三规则
2. 能区分 `String` 与 `&str`
3. 能写出 `Result` 与 `Option` 的处理
4. 能说出 `trait` 与泛型的关系
5. 能理解 `async/await` 与 `spawn`
6. 能读懂宏调用与派生
7. 能看懂并发原语的选型
8. 能处理编译报错并修复

3.98.2 2.98.2 实战检验（能动手）

1. 用 `cargo` 新建一个二进制项目
2. 写一个含结构体 + 方法的模块
3. 用 `sqlx` 完成一次查询（或模拟）
4. 用 `tokio` 写一个并发任务
5. 用 `serde` 序列化一个结构体
6. 给项目里一个小函数加日志

3.98.3 2.98.3 进入 03 章的标准

能读懂 03 章 80% 的代码块

→ 可以进入 03 章

（遇到不懂的语法随时回 02 章查）

3.99 2.99 深入：02 章权威自测（8 题）

1. 语法检验的八条？
2. 实战检验的六条？
3. 进入 03 章的标准？
4. 所有权三规则？
5. `String` 与 `&str` 的区别？
6. 派生宏的作用？
7. 并发原语的选型依据？
8. 遇到不懂语法怎么办？

答对 7+ → 02 章权威。

下一节：03-后端逐模块精读。

4 03 后端逐模块精读

适用对象：已完成 02 章语法速成，准备深入后端源码。本章将**逐模块、逐文件、逐行走**读后端全部核心代码，从启动到业务数据流，每个文件标注源码位置与行号，建议全程打开真实文件对照。全文约 2.5 万字，含 10+ 张 Mermaid 图。

4.1 3.1 第一个文件：main.rs (程序入口, 232 行)

4.1.1 3.1.1 文件总览

src/main.rs 是后端的唯一入口。虽然只有 232 行，但它是全项目的“总指挥”。整个启动过程分 7 步，与文件顶部 `///` 文档注释一一对应：

步骤	代码行	做什么
1. 加载环境变量	104	读取 .env 文件
2. 初始化日志	115-120	tracing 日志系统
3. 加载配置	130	PORT / DATABASE_URL
4. 初始化数据库	138	建连接池 + 建表 + 种子数据
5. 配置 CORS	148-151	跨域放行
6. 创建路由	175-216	API 路由 + 静态托管
7. 启动服务器	222-229	绑定 127.0.0.1:3000

flowchart TD

```
A[dotenv 加载 .env] --> B[初始化 tracing 日志]
B --> C[AppConfig::load<br/>PORT/DATABASE_URL]
C --> D[init_database<br/>连接池+建表+种子]
D --> E[CorsLayer 全放行]
E --> F[create_router 注册 API]
F --> G[embedded feature?]
G -->|是| H[merge embedded_assets::embedded_router<br/>内存内嵌 7 个前端]
G -->|否| I[7 个 ServeDir 磁盘目录<br/>fallback index.html]
H --> J[绑定 127.0.0.1:PORT]
I --> J
J --> K[axum::serve 常驻运行]
```

4.1.2 3.1.2 模块声明 (41-58 行)

```
mod admin;           // 管理员模块：用户管理、角色管理
mod api_docs;        // OpenAPI 文档定义与导出
mod city3d;          // city3d 角色模块：城市 3D 展示
mod common;          // 公共模块：认证、中间件、数据模型、错误处理
mod config;          // 配置模块：从环境变量加载应用配置
mod database;        // 数据库模块：SQLite 连接、表创建、种子数据
mod fj200c_information; // fj200c_information 角色模块：发动机监控
mod fj200c_main;     // fj200c_main 角色模块：发动机测控
mod ftj1c;           // ftj1c 角色模块：UDP 组播通信监控
mod fw100;           // fw100 角色模块：设备台账管理
mod fw150;
mod role_template;   // 角色模板：新角色开发的参考模板
mod roles;           // 角色注册表：全系统角色定义的单一事实来源
mod routes;          // 路由模块：集中注册所有 API 路由
```

```
#[cfg(feature = "embedded")]
mod embedded_assets; // 单 exe 打包时才编译
```

这是整个模块树的根。每个 `mod xxx;` 对应 `src/xxx/` 目录（或 `src/xxx.rs`）。新增业务模块时，第一步就是在这里加一行 `mod xxx;`（第 08 章流程的第一步）。

注意 `embedded_assets` 用了 `#[cfg(feature = "embedded")]`——只有带 `feature` 编译时才存在这个模块，否则不编译（条件编译，见 02 章 2.15 节）。

4.1.3 3.1.3 主函数与 tokio (98-99 行)

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
```

`#[tokio::main]` 是过程宏：它把你的 `async fn main` 包进一个同步 `fn main`，内部创建 Tokio 运行时再执行你的异步代码。**这是 axum 异步生态的入口**，所有 `.await` 都靠它运行。

`Result<(), Box<dyn std::error::Error>>`：函数可以失败；`()` 是空值；`Box<dyn Error>` 是“任意错误”（启动阶段还没引入 `AppError`，用最宽泛的错误类型）。

4.1.4 3.1.4 启动七步走 (104-229 行)

第一步：环境变量 (104 行)

```
dotenv::dotenv().ok();
```

读取运行目录的 `.env`。`.ok()` 表示“失败也没关系”（`.env` 不存在时静默）。

第二步：日志 (115-120 行)

```
tracing_subscriber::registry()
    .with(tracing_subscriber::EnvFilter::new(
        std::env::var("RUST_LOG").unwrap_or_else(|_| "info".into()),
    ))
    .with(tracing_subscriber::fmt::layer())
    .init();
```

日志级别由环境变量 `RUST_LOG` 控制，默认 `info`。调试用 `RUST_LOG=debug` 重启。

第三步：配置 (130 行)

```
let config = AppConfig::load()?;
```

读取 `PORT`（默认 3000）和 `DATABASE_URL`（默认 `sqlite://rustweb.db`）。`?` 传播错误——配置错了直接启动失败（比如 `PORT` 不是数字）。

第四步：数据库 (138 行)

```
let pool = init_database(&config.database_url).await?;
```

建连接池 + 建表 + 插种子数据（3.4 节详述）。返回 `SqlitePool`（连接池），克隆成本极低，会被注入到所有 handler。

第五步：CORS (148-151 行)

```
let cors = CorsLayer::new()
    .allow_methods([Method::GET, Method::POST, Method::PUT, Method::DELETE])
    .allow_headers(Any)
    .allow_origin(Any);
```

开发时前端在 5173~5179，后端在 3000，跨域全靠它放行。**全放行**是开发便利；内网生产环境同源后形同虚设。

第六步：路由与静态托管 (175-216 行) ——最值得细看的部分


```

let app = {
  let base = create_router(pool)
    .layer(cors)
    .route("/", get(|| async { Redirect::permanent("/admin") }));

  #[cfg(feature = "embedded")]
  let app = base.merge(embedded_assets::embedded_router());

  #[cfg(not(feature = "embedded"))]
  let app = base
    .nest_service("/admin", ServeDir::new("dist-admin")
      .fallback(ServeFile::new("dist-admin/index.html")))
    // ... 其余 6 个前端同理
  app
};

```

三个设计点：

1. / 重定向到 /admin：根路径直接进管理后台，方便演示。
2. 双模式静态托管：--features embedded 时前端内嵌内存；默认 dev 模式读磁盘 dist-<app>/ 目录。
3. fallback(index.html) 是 SPA 深链接的关键：Vue Router 用 history 模式，浏览器直接访问 /fj200c_information/data（刷新页面）时服务器没有这个文件，fallback 返回 index.html，前端路由接管渲染。**没有这行，刷新就 404。**

注意第 173-174 行的注释：.layer() 在 axum 0.7 中只作用于调用时**已注册**的路由，所以 CORS 必须加在 create_router(pool) 之后、merge/nest_service 之前，确保 API 与静态路由都被包裹——这是踩过坑后留下的注释，改动时别把它破坏。

第七步：启动（222-229 行）

```

let addr = SocketAddr::from(([127, 0, 0, 1], config.port));
let listener = tokio::net::TcpListener::bind(addr).await?;
axum::serve(listener, app).await?;
Ok(())

```

127.0.0.1 绑定回环地址——**只允许本机访问**。要允许局域网访问，改成 0.0.0.0 (AGENTS.md 有注明)。axum::serve 是常驻调用，程序在这里一直跑。

4.1.5 3.1.5 你将来会改 main.rs 的场景

场景	改哪里
改端口	第 130 行配置（或 .env 的 PORT）
允许外网访问	第 222 行 127.0.0.1 → 0.0.0.0
新增前端应用	第 183-213 行加一段 nest_service + 第 56-58 行加嵌入结构体
改根路径重定向	第 178 行 /admin
调整 CORS	第 148-151 行

4.2 3.2 第二个文件：routes.rs（路由集中注册，145 行）

4.2.1 3.2.1 设计思想

src/routes.rs 是**路由的中央集线器**：所有模块的 xxx_router(db) 子路由在这里拼装成一颗完整路由树，最后 with_state(db) 注入数据库连接池。

flowchart TD

```
RT["create_router(db)"] --> H["/health"]
RT --> OD["/api-docs/openapi.json"]
RT --> MR["/api/meta/roles"]
RT --> A["nest /api/auth → auth_router"]
RT --> U["nest /api/users → admin_router"]
RT --> FI["nest /api/fj200c_information → fj200c_information_router"]
RT --> FM["nest /api/fj200c_main → fj200c_main_router"]
RT --> F1["nest /api/fw100 → fw100_router"]
RT --> F2["nest /api/fw150 → fw150_router"]
RT --> T["nest /api/ftj1c → ftj1c_router"]
RT --> C["nest /api/city3d → city3d_router"]
RT --> S["with_state(db) 注入连接池"]
```

4.2.2 3.2.2 关键代码走读 (108-144 行)

```
Router::<DatabaseConnection>::new()
  .route("/health", get(crate::common::health_check))
  .route("/api-docs/openapi.json", get(crate::api_docs::openapi_json))
  .route("/api/meta/roles", get(crate::roles::list_roles))
  .nest("/api/auth", auth_routes)
  .nest("/api/users", admin_routes)
  .nest("/api/fj200c_information", fj200c_information_routes)
  .nest("/api/fj200c_main", fj200c_main_routes)
  .nest("/api/fw100", fw100_routes)
  .nest("/api/ftj1c", ftj1c_routes)
  .nest("/api/city3d", city3d_routes)
  .nest("/api/fw150", fw150_routes)
  .with_state(db)
```

逐个解释：

1. `Router::<DatabaseConnection>::new()`：显式声明状态类型为数据库连接池。所有 handler 都可以用 `State(db): State<DatabaseConnection>` 拿它。
2. 三个公开端点（无需登录）：
 - `/health`：健康检查（负载均衡/监控用）。
 - `/api-docs/openapi.json`：实时 OpenAPI 文档。
 - `/api/meta/roles`：角色注册表（前端运行时拉取）。
3. `.nest(prefix, router)`：挂载子路由，自动加前缀。`auth_router` 里的 `/login` 挂到 `/api/auth/login`。
4. `.with_state(db)`：最后注入状态。所有子路由共享同一个连接池。

4.2.3 3.2.3 nest 与 route 的区别 (新手易混)

方法	用途	例子
<code>.route(path, handler)</code>	注册单个路由	<code>.route("/health", get(health))</code>
<code>.nest(path, router)</code>	挂载整棵子路由树	<code>.nest("/api/auth", auth_router)</code>
<code>.nest_service(path, service)</code>	挂载静态文件服务	<code>.nest_service("/admin", ServeDir::new(...))</code>
<code>.layer(mw)</code>	给已注册的路由添加中间件	<code>.layer(cors)</code>
<code>.route_layer(mw)</code>	只给该路由加中间件	各模块内部用
<code>.with_state(s)</code>	注入共享状态	<code>.with_state(db)</code>
<code>.merge(router)</code>	合并另一棵树（平级）	<code>embedded_router</code>

4.2.4 3.2.4 新增模块时 routes.rs 怎么改

1. let xxx_routes = crate::xxx::routes::xxx_router(db.clone());
2. .nest("/api/xxx", xxx_routes)
3. 模块内部实现 xxx_router() (第 08 章详述)。

4.3 3.3 第三个文件: roles.rs (RBAC 唯一源, 268 行)

4.3.1 3.3.1 为什么它最重要

src/roles.rs 是**全系统角色定义的单一事实来源**: 前端菜单怎么显示、后端接口放行谁、用户列表角色下拉有哪些——最终都由这里驱动。

4.3.2 3.3.2 核心数据结构

```
// 角色定义
pub struct RoleDef {
    pub key: String,           // 角色标识 (存数据库用, 如 "admin")
    pub name: String,         // 角色显示名 (前端展示)
    pub permissions: &'static [Permission], // 权限点列表
}

// 注册表: 静态数组, 全系统唯一的角色清单
pub static ROLE_REGISTRY: &[RoleDef] = &[
    RoleDef { key: "admin", name: "管理员",
        permissions: &[Permission::UsersRead, Permission::UsersWrite,
            Permission::UsersDelete, Permission::SystemAdmin] },
    RoleDef { key: "fj200c_information", name: "fj200c_information",
        permissions: &[Permission::Fj200cInformationMonitor] },
    RoleDef { key: "fw100", name: "fw100", permissions: &[Permission::Fw100Monitor] },
    RoleDef { key: "ftj1c", name: "ftj1c", permissions: &[Permission::Ftj1cMonitor] },
    RoleDef { key: "city3d", name: "city3d", permissions: &[Permission::City3dView] },
    RoleDef { key: "fw150", name: "fw150", permissions: &[Permission::Fw150Monitor] },
    RoleDef { key: "fj200c_main", name: "fj200c_main",
        permissions: &[Permission::Fj200cMainMonitor] },
];
```

注意: permissions: &'static [Permission] 是**静态切片**——编译期内嵌的权限列表, 不可变。注册表本身不可变, 这就是“唯一源”的保证。

4.3.3 3.3.3 关键函数

```
// 根据角色 key 查权限列表
pub fn permissions_for(role: &str) -> Vec<Permission> {
    ROLE_REGISTRY.iter()
        .find(|r| r.key == role)           // 找到角色
        .map(|r| r.permissions.to_vec())   // 权限列表克隆
        .unwrap_or_default()               // 没找到 → 空权限 (无权限用户)
}

// 角色是否存在于注册表 (创建用户时白名单校验)
pub fn is_registered_role(role: &str) -> bool {
    ROLE_REGISTRY.iter().any(|r| r.key == role)
}

// 前端需要的 DTO: key + name + permissions
#[derive(Serialize, ToSchema)]
pub struct RoleInfo {
    pub key: String,
    pub name: String,
    pub permissions: Vec<Permission>,
}
```

```
// 公开端点: GET /api/meta/roles
pub async fn list_roles() -> Json<ApiResponse<Vec<RoleInfo>>> {
    let roles = ROLE_REGISTRY.iter()
        .map(|r| RoleInfo {
            key: r.key.to_string(),
            name: r.name.to_string(),
            permissions: r.permissions.to_vec(),
        })
        .collect();
    Json(ApiResponse::success(roles))
}
```

4.3.4 3.3.4 权限流动全景图

flowchart LR

```
REG[ROLE_REGISTRY 静态数组] -->|permissions_for| U1["users 表 user.role<br/>→ 权限列表"]
REG -->|list_roles| FE["前端 loadRoleRegistry<br/>RoleInfo[]"]
U1 -->|has_permission| MW[permission_middleware<br/>403 拦截]
FE -->|getPermissionsByRole| FM[菜单过滤/按钮禁用]
```

重要理解： 1. 数据库 users 表只存角色字符串 (role = 'admin')，不存权限列表。 2. 权限是推导出来的：permissions_for(role) 查注册表。 3. 改权限 → 改注册表 → 重新生成 API (npm run gen:api) → 前端自动同步。 4. is_registered_role 是创建用户的白名单：管理员不能给用户编造一个不存在的角色。

4.3.5 3.3.5 新增角色的后端改动 (预告)

// 只需加一行：

```
RoleDef { key: "xxx", name: "xxx", permissions: &[Permission::XxxMonitor] },
```

加上后，GET /api/meta/roles 自动返回新角色，前端注册表加载自动同步，用户列表的角色下拉自动出现。**但注意：**光有注册表，新角色的接口/前端还不存在——第 08 章讲完整流程。

4.4 3.4 第四个文件：database.rs (建表 + 种子数据, 1269 行)

4.4.1 3.4.1 设计思想：无迁移文件的数据库

一般项目用迁移工具 (如 Diesel、Flyway) 管理表结构；本项目**没有迁移文件**——建表 SQL 全在 database.rs 的 create_tables 函数里，启动时执行。特点是：

1. **幂等：**CREATE TABLE IF NOT EXISTS，重复执行无害。
2. **自动升级：**加了新表/新字段，重启即生效 (老代码兼容旧表)。
3. **可重置：**删掉 rustweb.db 重启，自动重建 + 重新种种子。

4.4.2 3.4.2 初始化流程 (init_database)

```
pub async fn init_database(database_url: &str) -> Result<SqlitePool, sqlx::Error> {
    // 1. 配置 SQLite 选项
    let options = SqliteConnectOptions::from_str(database_url)?
        .create_if_missing(true) // 文件不存在自动创建
        .foreign_keys(true) // 启用外键
        .journal_mode(SqliteJournalMode::Wal); // WAL 模式 (并发读写优化)

    // 2. 创建连接池
    let pool = SqlitePool::connect_with(options).await?;

    // 3. 建表
    create_tables(&pool).await?;

    // 4. 种子数据
    seed_data(&pool).await?;
}
```

```
Ok(pool)
}
```

三个选项值得理解：

选项	作用	为什么
create_if_missing(true)	库文件不存在自动创建	开箱即用，无需手动建库
foreign_keys(true)	外键约束生效	city3d 建筑引用区域
journal_mode(Wal)	预写日志模式	读写不互相阻塞，性能更好

4.4.3 3.4.3 建表清单 (create_tables)

```
async fn create_tables(pool: &SqlitePool) -> Result<(), sqlx::Error> {
    // users 表
    sqlx::query(
        "CREATE TABLE IF NOT EXISTS users (
            id TEXT PRIMARY KEY,          -- UUID 存 TEXT
            username TEXT NOT NULL UNIQUE,
            email TEXT NOT NULL UNIQUE,
            password_hash TEXT NOT NULL,
            role TEXT NOT NULL DEFAULT 'fj200c_information',
            created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
        )",
    )
    .execute(pool)
    .await?;

    // user_settings 表（列配置 JSON）
    sqlx::query(
        "CREATE TABLE IF NOT EXISTS user_settings (
            user_id TEXT PRIMARY KEY,
            value TEXT NOT NULL,          -- JSON 字符串
            updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
        )",
    )
    .execute(pool)
    .await?;

    // city3d 三张表 + 索引
    // ...（省略，模式相同）
    Ok(())
}
```

新手注意： 1. 主键用 TEXT 存 UUID 字符串（SQLite 无原生 UUID 类型）。2. UNIQUE 约束配合 INSERT OR IGNORE 实现幂等种子。3. 数据库列名 snake_case 与 Rust 结构体字段一致，sqlx 才能自动映射。4. database.rs 后半部分有**旧表清理**逻辑（DROP TABLE IF EXISTS 老表名）和**角色迁移**逻辑（UPDATE users SET role = ...）——项目演进留下的痕迹，改表结构时注意保持幂等。

4.4.4 3.4.4 种子账号 (seed_data)

7 个种子账号，密码全是 123456：

账号 (email)	角色	用途
admin@rustweb.dev	admin	管理后台
fj200c_information@rustweb.dev	fj200c_information	发动机监控
fj200c_main@rustweb.dev	fj200c_main	发动机测控
fw100@rustweb.dev	fw100	设备台账
fw150@rustweb.dev	fw150	设备台账
ftj1c@rustweb.dev	ftj1c	通信监控
city3d@rustweb.dev	city3d	城市 3D

种子逻辑:

```
// 固定 UUID (幂等关键): 重复运行 INSERT OR IGNORE 不冲突
const SEED_UUIDS: &[(&str, &str, &str, &str)] = &[
    ("00000000-0000-4000-8000-000000000001", "admin", "admin@rustweb.dev", "admin"),
    // ... 7 个
];

for (id, username, email, role) in SEED_UUIDS {
    let hash = bcrypt::hash("123456", 10).unwrap(); // 每次启动都重算? 一不, 见下
    sqlx::query(
        "INSERT OR IGNORE INTO users (id, username, email, password_hash, role) VALUES
        (?, ?, ?, ?, ?)",
    )
    .bind(id)
    .bind(username)
    .bind(email)
    .bind(hash)
    .bind(role)
    .execute(pool)
    .await?;
}
```

注: 实际实现会把哈希计算放在循环外或只算一次 (bcrypt 代价参数 10 约 100ms, 启动时一次性代价可接受)。INSERT OR IGNORE 保证已存在的用户 (改过密码的) 不被覆盖——种子只在首次创建时生效。

city3d 种子: 5 个区域、51 座建筑 (UUID 从基值递增)、8 个事件——都是演示数据, 删库重启即恢复。

4.4.5 3.4.5 你将来会改 database.rs 的场景

场景	改法
新增业务表	create_tables 加 CREATE TABLE IF NOT EXISTS
加字段	老表加列用 ALTER TABLE ... ADD COLUMN (注意幂等: 先查是否已存在) 或改建表语句
换种子密码	改 123456
加演示数据	参照 city3d 种子写法

4.5 3.5 common 公共层精读 (全系统的地基)

src/common/ 是所有角色共用的基础设施, 17 个子模块。这是全项目价值密度最高的目录, 逐个精读。

4.5.1 3.5.1 common/mod.rs: 模块门面

```
pub mod auth;
pub mod config;
pub mod csv_writer;
pub mod dto;
pub mod error;
pub mod frame_extractor;
pub mod global_var;
pub mod io;
pub mod jwt;
pub mod latest_frame;
pub mod ledger;
pub mod least_squares;
pub mod middleware;
pub mod models;
pub mod quad_frame;
pub mod service;
pub mod utils;
pub mod ws;
```

```
/// 健康检查: GET /health → 200
pub async fn health_check() -> &'static str {
    "OK"
}
```

health_check 返回 &'static str (常量字符串), Axum 自动转 text/plain 响应——最简单的 handler 形态。

4.5.2 3.5.2 common/models.rs: 核心模型 (389 行)

Permission 枚举 (2.4.2 已讲, 跳过) → 直接看 User:

```
#[derive(Debug, Clone, Serialize, Deserialize, utoipa::ToSchema)]
pub struct User {
    pub id: uuid::Uuid,
    pub username: String,
    pub email: String,
    #[serde(skip_serializing)]
    pub password_hash: String,
    pub role: String,
    pub created_at: chrono::NaiveDateTime,
}

impl User {
    pub fn permissions(&self) -> Vec<Permission> {
        crate::roles::permissions_for(&self.role)
    }
    pub fn has_permission(&self, permission: &Permission) -> bool {
        self.permissions().contains(permission)
    }
}
```

ApiResponse 统一响应 (前端所有接口的标准包装):

```
#[derive(Debug, Clone, Serialize, Deserialize, utoipa::ToSchema)]
pub struct ApiResponse<T> {
    pub success: bool,          // 是否成功
    pub message: String,        // 提示信息 (失败时是错误描述)
    pub data: Option<T>,        // 业务数据 (成功时 Some)
}

impl<T> ApiResponse<T> {
    pub fn success(data: T) -> Self {
```

```

        Self { success: true, message: "ok".to_string(), data: Some(data) }
    }
    pub fn error(message: impl Into<String>) -> Self {
        Self { success: false, message: message.into(), data: None }
    }
}

```

LoginRequest 与校验:

```

#[derive(Debug, Deserialize, ToSchema)]
pub struct LoginRequest {
    #[validate(email)] // validator 属性: 邮箱格式校验
    pub email: String,
    pub password: String,
}

```

4.5.3 3.5.3 common/middleware.rs: 三个中间件 (198 行)

这是权限体系的执行层。三个中间件一条链:

```

/// ① 从请求头提取 Bearer token 并验证, 返回用户 ID
fn extract_user_id(request: &Request) -> Result<uuid::Uuid, StatusCode> {
    let auth_header = request.headers()
        .get("Authorization")
        .and_then(|h| h.to_str().ok())
        .and_then(|s| s.strip_prefix("Bearer ")); // "Bearer xxx" → "xxx"
    let token = auth_header.ok_or(StatusCode::UNAUTHORIZED)?;
    jwt::verify_token(token).map_err(|_| StatusCode::UNAUTHORIZED)
}

/// ② 从数据库加载用户 (未找到 → 401)
async fn load_user(db: &DatabaseConnection, user_id: uuid::Uuid) -> Result<User,
StatusCode> {
    sqlx::query_as:::<_, User>("SELECT * FROM users WHERE id = ?1")
        .bind(user_id)
        .fetch_optional(db)
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?
        .ok_or(StatusCode::UNAUTHORIZED)
}

/// ③ 鉴权中间件: 验证 token → 加载用户 → 注入 Extension
pub async fn auth_middleware(
    State(db): State<DatabaseConnection>,
    mut request: Request,
    next: Next,
) -> Result<Response, StatusCode> {
    let user_id = extract_user_id(&request)?;
    let user = load_user(&db, user_id).await?;
    request.extensions_mut().insert(user); // ★ 把用户塞进请求扩展区
    Ok(next.run(request).await) // 放行, handler 用 Extension 取
}

/// ④ 权限中间件: 额外检查指定权限
pub async fn permission_middleware(
    required_permission: Permission,
    State(db): State<DatabaseConnection>,
    mut request: Request,
    next: Next,
) -> Result<Response, StatusCode> {
    let user_id = extract_user_id(&request)?;
    let user = load_user(&db, user_id).await?;
    if !user.has_permission(&required_permission) {
        return Err(StatusCode::FORBIDDEN); // 有身份但无权限 → 403
    }
}

```



```

    }
    request.extensions_mut().insert(user);
    Ok(next.run(request).await)
}

```

/// ⑤ 角色中间件: SystemAdmin 专用 (admin 模块)

```

pub async fn role_middlewares(* ... 检查 Permission::SystemAdmin */) -> Result<Response,
StatusCode> { ... }

```

中间件模式总结 (新手必记): - 签名固定: (State, Request, Next) -> Result<Response, StatusCode> (额外参数如 required_permission 需要函数包装, 因为 from_fn 不能传参——见 3.7 节 admin/routes.rs)。- 三件事: **验证** → **加载** → **注入**。- 放行 = next.run(request).await; **拦截** = 返回 StatusCode。- request.extensions_mut().insert(user): Axum 的“请求背包”, 中间件放进什么, handler 用 Extension<T> 取什么。

4.5.4 3.5.4 common/error.rs: 统一错误 (212 行)

已在本章 2.9 精读, 这里补一张“错误转换关系图”:

flowchart LR

```

    SQLX[sqlx::Error<br/>RowNotFound → 404<br/>唯一冲突 → 400<br/>其他 → 500] -->|From|
    AE[AppError]
    JWT[jsonwebtoken::Error] -->|From → 401| AE
    BCrypt[bcrypt::Error] -->|From → 500| AE
    VALID[validator::ValidationErrors] -->|From → 400| AE
    AE -->|IntoResponse| R["{success:false, message}"]

```

4.5.5 3.5.5 common/dto.rs: 公共响应 DTO

```

#[derive(Debug, Clone, Serialize, ToSchema)]
pub struct ServiceStatus { pub running: bool } // 服务启停状态

#[derive(Debug, Clone, Serialize, ToSchema)]
pub struct SentResult { pub sent: bool } // 命令发送结果

#[derive(Debug, Clone, Serialize, ToSchema)]
pub struct SavedResult { pub saved: bool } // 配置保存结果

#[derive(Debug, Clone, Serialize, ToSchema)]
pub struct ConfigContent { pub content: String } // INI 文件内容

#[derive(Debug, Clone, Serialize, ToSchema)]
pub struct CsvFileList { pub files: Vec<String>, pub dir: String } // CSV 文件列表

#[derive(Debug, Clone, Serialize, ToSchema)]
pub struct CsvFileContent { pub content: String } // CSV 文件内容

```

这些“小响应体”是硬件的公共返回形态, 三个硬件模块共用。**新增通用响应体放这里。**

4.5.6 3.5.6 common/ws.rs: WebSocket 事件桥 (90 行)

/// 把 broadcast 通道的事件推给一个 WS 连接 (无初始快照版本)

```

pub async fn ws_bridge<T>(
    tx: broadcast::Sender<T>,
    socket: WebSocket,
    log_prefix: &str,
) where T: Serialize + Clone + Send + Sync + 'static
{
    ws_bridge_with_initial(tx, socket, log_prefix, None).await;
}

```

/// 带初始快照: 连接建立时先推一条 (前端刷新页面立刻有数据)

```

pub async fn ws_bridge_with_initial<T>(
    tx: broadcast::Sender<T>,

```

```

socket: WebSocket,
log_prefix: &str,
initial: Option<String>,          // 初始消息（已序列化好的 JSON）
) where T: Serialize + Clone + Send + Sync + 'static
{
    let (mut sender, mut receiver) = socket.split(); // 拆成发送/接收两半
    let mut rx = tx.subscribe(); // 订阅广播

    if let Some(initial) = initial {
        if sender.send(Message::Text(initial)).await.is_err() { return; }
    }

    loop {
        tokio::select! {
            // 客户端 → 服务端（本项目一般只用 Close 消息做退出）
            msg = receiver.next() => {
                match msg {
                    Some(Ok(Message::Close(_))) | None | Some(Err(_)) => break,
                    _ => {}
                }
            }
            // 服务端 → 客户端（广播事件推送）
            event = rx.recv() => {
                match event {
                    Ok(evt) => {
                        let text = serde_json::to_string(&evt).unwrap_or_default();
                        if sender.send(Message::Text(text)).await.is_err() {
                            break; // 客户端断开
                        }
                    }
                    Err(RecvError::Lagged(_)) => continue, // 慢客户端丢事件
                    Err(RecvError::Closed) => break, // 广播通道关闭
                }
            }
        }
    }
    tracing::info!("{log_prefix} WebSocket 连接关闭");
}

```

这是全项目 WS 的公共实现——所有硬件模块的 ws_handler 最后都调用它。注意 where T: Serialize + Clone + Send + Sync + 'static: 泛型约束说明事件类型需要能序列化、能克隆、线程安全——事件枚举都满足。

4.5.7 3.5.7 common/service.rs: ServiceRuntime (线程管理)

```

/// 全局线程句柄池：管理所有业务线程的启停
pub struct ServiceRuntime {
    handles: Mutex<Vec<JoinHandle<()>>>,
    stop: AtomicBool,
}

impl ServiceRuntime {
    /// 登记线程句柄
    pub fn push(&self, handle: JoinHandle<()>) { ... }

    /// 请求停止：置标志 + 等所有线程 join（最长 timeout_secs 秒）
    pub fn stop_and_wait(&self, timeout_secs: u64) { ... }

    /// 检查停止标志
    pub fn is_stopped(&self) -> bool { ... }
}

```

```
// 全局单例
pub static RUNTIME: OnceLock<Mutex<ServiceRuntime>> = ...;
```

服务生命周期四步曲（三个硬件模块统一）：

flowchart LR

```
A[start_service<br/>读配置+起线程] --> B[运行中<br/>线程循环轮询停止标志]
B --> C[stop_service<br/>置标志]
C --> D[线程退出<br/>join ≤3s]
```

4.5.8 3.5.8 common/io.rs: IoControl trait (2.7.1 已详述, 此处补充)

```
pub trait IoControl {
    fn send(&self, data: &[u8]) -> Result<(), String>;
    fn recv(&self) -> Result<Vec<u8>, String>;
    fn set_timeout(&self, timeout_ms: u32) -> Result<(), String>;
}
```

4.5.9 3.5.9 common/frame_extractor.rs: 帧提取器 (112 行)

串口是**字节流**，没有消息边界。帧提取器负责从字节流里“攒”出完整的一帧：

```
pub struct FrameExtractor {
    header: Vec<u8>,           // 帧头标记（如 [0xEB, 0x90, 0x64]）
    frame_size: usize,         // 帧总长（如 100）
    validator: Option<fn(&[u8]) -> bool>, // 校验函数（累加和）
    decoder: Option<fn(&[u8]) -> Option<Vec<String>>>, // 解码函数
    buffer: Vec<u8>,           // 字节缓冲
}

impl FrameExtractor {
    /// 喂入新字节，产出完整帧（可能有 0/1/多帧）
    pub fn process(&mut self, bytes: &[u8]) -> Vec<Vec<u8>> {
        self.buffer.extend_from_slice(bytes);
        let mut frames = Vec::new();
        loop {
            // 1. 找帧头：跳过帧头前的脏数据
            let Some(pos) = find_header(&self.buffer, &self.header) else { break };
            if pos > 0 { self.buffer.drain(..pos); } // 丢弃脏数据
            // 2. 攒够一帧？
            if self.buffer.len() < self.frame_size { break; }
            // 3. 取出候选帧
            let frame: Vec<u8> = self.buffer.drain(..self.frame_size).collect();
            // 4. 校验（累加和/固定字段）
            if self.validator.map_or(true, |v| v(&frame)) {
                frames.push(frame); // 合法帧
            } else {
                // 校验失败：丢弃第 1 字节，重新对齐（防卡死）
                self.buffer.drain(..1);
            }
        }
        frames
    }
}
```

状态机逻辑：**找帧头** → **丢脏数据** → **攒帧** → **校验** → **解码**。校验失败时“逐字节丢弃”是防呆设计——不会因为一帧坏数据卡死整个流。

4.5.10 3.5.10 common/quad_frame.rs: 四槽帧缓冲（主备切换）

```
/// 四槽帧缓冲：主备双数据源的最新帧管理
pub struct QuadFrame<const FRAME_LEN: usize> {
    frames: [ArcSwap<[u8; FRAME_LEN]>; 4], // 槽位 0-3
    sequence: AtomicU32, // 全局序号（CAS 去重）
}
```

```
impl<const FRAME_LEN: usize> QuadFrame<FRAME_LEN> {
    /// 尝试更新：序号大于当前才生效（去旧帧）
    pub fn try_update(&self, slot: usize, frame: [u8; FRAME_LEN], seq: u32) -> bool {
        // CAS：只有 seq 更大才写入
        // 主源心跳超时后，备源自动接管
    }
}
```

ftj1c 用它实现**主备双路 UDP**：主链路断流（心跳超时 1000ms）后备链路自动接管，主链路恢复自动切回。ArcSwap 保证读者永远无锁读到最新帧。

4.5.11 3.5.11 common/latest_frame.rs: 最新帧跟踪器

```
/// 简单版"最新帧"：ArcSwap 存最新帧 + CAS 序号去重
pub struct LatestFrame<N> { ... }
```

与 quad_frame 的区别：单槽、用于其他模块（fj200c_information 的 frame_bundle 类似）。

4.5.12 3.5.12 common/global_var.rs: 全局 KV (123 行)

```
/// 进程内全局键值存储（OnceLock + RwLock + HashMap）
pub struct GlobalVar { inner: RwLock<HashMap<String, String>> }
```

```
impl GlobalVar {
    pub fn init() -> &'static GlobalVar { ... } // 初始化单例
    pub fn set(&self, key: &str, value: &str) { ... }
    pub fn get(&self, key: &str) -> Option<String> { ... }
    pub fn get_or(&self, key: &str, default: &str) -> String { ... }
    pub fn delete(&self, key: &str) { ... }
    pub fn snapshot(&self) -> HashMap<String, String> { ... }
    pub fn clear(&self) { ... }
}
```

```
pub static GLOBAL_VAR: OnceLock<GlobalVar> = OnceLock::new();
pub fn global_var() -> &'static GlobalVar { ... }
```

用途：fj200c_main 的试验信息、主题状态等”轻量服务端状态”以 JSON 字符串存这里。**特点**：不落数据库，重启即失——适合临时状态。

4.5.13 3.5.13 common/csv_writer.rs: CSV 写入器 (2.24.1 已详述，此处补 Drop 细节)

```
impl Drop for CsvWriter {
    fn drop(&mut self) {
        let _ = self.flush(); // 无论何时销毁，把残余数据写盘
    }
}
```

Drop trait：对象销毁时自动执行——防止”进程被杀丢数据”。这是 Rust 的资源管理精髓 (RAII)。

4.5.14 3.5.14 common/config.rs: INI 封装 (2.23 已详述)

4.5.15 3.5.15 common/utils.rs: 工具函数 (2.31 已详述)

4.5.16 3.5.16 common/ledger.rs: 台账演示数据

```
pub struct LedgerItem {
    pub id: String,
    pub name: String,
    pub category: String,
    pub status: String,
    pub location: String,
    pub updated_at: String,
```

```
}
```

```
/// 按用户名生成不同的演示数据（fw100/fw150 共用）
```

```
pub fn demo_ledger_items(username: &str) -> Vec<LedgerItem> { ... }
```

fw100/fw150 的“设备台账”就是它生成的演示数据——没有数据库表，纯函数生成。最简模块的真相。

4.5.17 3.5.17 common/least_squares.rs: 最小二乘拟合 (86 行)

```
/// 多段线性拟合（最小二乘）：xjc 数据校正用
```

```
pub struct LeastSquareEstimation;
```

```
impl LeastSquareEstimation {
```

```
    /// 构造法方程并解（列主元 Gauss 消元）
```

```
    pub fn multi_line(xs: &[Vec<f64>], ys: &[f64], degree: usize) -> Vec<f64> { ... }
```

```
}
```

数值计算工具，发动机数据校正用，新手了解即可。

4.6 3.6 auth 登录流程精读（全系统入口）

4.6.1 3.6.1 auth 模块结构

```
src/common/auth/
```

```
├─ mod.rs          # 模块声明
├─ routes.rs       # auth_router: POST /login + GET /profile
├─ handlers.rs     # login / get_profile 两个 handler (156 行)
└─ services.rs     # AuthService: 登录/查用户/建用户/默认设置 (249 行)
```

4.6.2 3.6.2 登录的完整时序

```
sequenceDiagram
```

```
    participant F as 前端 LoginPage.vue
```

```
    participant H as auth/handlers.rs login()
```

```
    participant S as auth/services.rs AuthService::login
```

```
    participant D as SQLite
```

```
    participant J as common/jwt.rs
```

```
    F->>H: POST /api/auth/login {email, password}
```

```
    H->>H: login_data.validate()? (validator 邮箱校验)
```

```
    H->>S: AuthService::login(&db, login_data)
```

```
    S->>D: SELECT * FROM users WHERE email = ?
```

```
    D-->>S: 用户行 (含 password_hash)
```

```
    S->>S: spawn_blocking(bcrypt::verify(password, hash))
```

```
    alt 密码正确
```

```
        S-->>H: Ok(user)
```

```
        H->>J: jwt::create_token(&user)
```

```
        J-->>H: Ok(token)
```

```
        H-->>F: {success:true, data:{token, user}}
```

```
    else 密码错误
```

```
        S-->>H: Err("密码错误")
```

```
        H-->>F: {success:false, message:"密码错误"} (400)
```

```
    end
```

4.6.3 3.6.3 handlers.rs 逐行（登录 handler 是“全项目最标准的 handler 模板”）

```
#[utoipa::path(
```

```
    post,
```

```
    tag = "auth",
```

```
    path = "/api/auth/login",
```

```
    operation_id = "authLogin",
```

```
    request_body = LoginRequest,
```

```
    responses((status = 200, description = "登录成功", body = ApiResponse<LoginResponse>)),
```

```
)]
```

```
pub async fn login(
```

```

    State(db): State<DatabaseConnection>,
    Json(login_data): Json<LoginRequest>,
) -> Result<Json<ApiResponse<LoginResponse>>, AppError> {
    login_data.validate()?; // ① 输入校验
    let user = AuthService::login(&db, login_data) // ② 服务层
        .await
        .map_err(|e| AppError::bad_request(e.to_string()))?;
    let token = jwt::create_token(&user)?; // ③ 签发 JWT
    Ok(Json(ApiResponse::success(LoginResponse { token, user }))) // ④ 统一响应
}

```

四步模板 (写任何 handler 都照抄这个骨架): 1. **校验输入**: xxx.validate()? (validator crate) 2. **调服务**: Service::xxx(&db, data).await? (错误转换) 3. **附加逻辑**: 如签发 token 4. **统一响应**: Ok(Json(ApiResponse::success(data)))

get_profile 是“零参数 handler”的范例——用户信息由中间件注入, handler 只是取出来返回:

```

pub async fn get_profile(
    Extension(user): Extension<User>, // ← 中间件塞进去的
) -> Result<Json<ApiResponse<User>>, AppError> {
    Ok(Json(ApiResponse::success(user)))
}

```

4.6.4 3.6.4 services.rs 精读: 登录的服务层实现

```

pub async fn login(
    pool: &DatabaseConnection,
    login_data: LoginRequest,
) -> Result<User, Box<dyn std::error::Error>> {
    // ① 按邮箱查用户 (找不到 → "用户不存在")
    let user = sqlx::query_as::<_, User>("SELECT * FROM users WHERE email = $1")
        .bind(&login_data.email)
        .fetch_optional(pool)
        .await?
        .ok_or("用户不存在");

    // ② 验证密码 (bcrypt, CPU 密集 → spawn_blocking)
    let password = login_data.password;
    let expected_hash = user.password_hash.clone();
    let is_valid = tokio::task::spawn_blocking(move || {
        verify(password.as_bytes(), &expected_hash)
    })
    .await??; // 两层 ?: 外层是 JoinHandle 错误, 内层是 verify 错误
    if !is_valid {
        return Err("密码错误".into());
    }
    Ok(user)
}

```

值得学习的点:

1. `ok_or("用户不存在")`: 把 Option 转成 Result, 错误信息是字符串, 靠 `From<&str> for Box<dyn Error>` 自动转。
2. `.await??` **双层问号**: `spawn_blocking` 返回 `JoinHandle<Result<bool, bcrypt::Error>>`, 第一个 ? 解 `JoinHandle` (线程 panic 等), 第二个 ? 解 `bcrypt` 错误。
3. **安全细节**: 密码错误返回“密码错误”、用户不存在返回“用户不存在”——严格说存在**用户枚举**风险 (攻击者可探测邮箱是否存在), 内网系统权衡可接受; 高安全需求时应统一为“邮箱或密码错误”。

4. 服务层错误用 `Box<dyn Error>`, handler 层统一转 `AppError`——服务层不依赖 HTTP 概念, 可复用 (测试友好)。

4.6.5 3.6.5 create_user: 管理员建用户

```
pub async fn create_user(
    pool: &DatabaseConnection,
    username: &str, email: &str, password: &str, role: &str,
) -> Result<User, Box<dyn std::error::Error>> {
    // ① 查重: 邮箱或用户名已存在 → 报错
    let existing_user = sqlx::query_as::<_, User>(
        "SELECT * FROM users WHERE email = $1 OR username = $2",
    ).bind(email).bind(username).fetch_optional(pool).await?;
    if existing_user.is_some() {
        return Err("用户名或邮箱已存在".into());
    }

    // ② bcrypt 加密 (spawn_blocking)
    let password_hash = tokio::task::spawn_blocking(move || {
        hash(password_owned.as_bytes(), DEFAULT_COST)
    }).await??;

    // ③ 插入 + RETURNING * (SQLite 3.35+ 支持)
    let user = sqlx::query_as::<_, User>(
        r#"INSERT INTO users (id, username, email, password_hash, role, created_at,
updated_at)
VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING *"#,
    ).bind(Uuid::new_v4()).bind(username).bind(email)
    .bind(&password_hash).bind(role).bind(now).bind(now)
    .fetch_one(pool).await?;

    // ④ 建默认设置
    Self::create_default_settings(pool, user.id).await?;
    Ok(user)
}
```

注意这里 SQL 用了 `r#"..."#` 原始字符串——多行 SQL 不需要转义, 是 Rust 写长 SQL 的标准姿势。

4.6.6 3.6.6 routes.rs: 路由与中间件

```
pub fn auth_router(db: DatabaseConnection) -> Router {
    Router::new()
        // 登录: 公开 (不需要任何中间件)
        .route("/login", post(crate::common::auth::handlers::login))
        // 用户信息: 需要登录 (auth_middleware)
        .route(
            "/profile",
            get(crate::common::auth::handlers::get_profile)
                .route_layer(axum::middleware::from_fn(auth_middleware)),
        )
        .with_state(db)
}
```

关键区分: - `/login` 无中间件——登录前没有 token。 - `/profile` 挂 `auth_middleware`——必须登录。 - `.route_layer()` 只作用于该路由; `.layer()` 作用于所有已注册路由。

4.7 3.7 admin 模块精读 (用户管理)

4.7.1 3.7.1 模块结构与路由 (routes.rs, 106 行)

admin 的路由设计值得细看: 按权限分三组, 每组中间件不同:


```

use crate::common::middleware::{auth_middleware, permission_middleware, role_middleware};

// * from_fn 不能传参，所以用闭包包装固定权限的中间件
fn users_read_middleware() -> axum::middleware::FromFn<...> {
    axum::middleware::from_fn(move |request, next| {
        permission_middleware(Permission::UsersRead, request, next)
    })
}

pub fn admin_router(db: DatabaseConnection) -> Router {
    Router::new()
        // 读组: GET /api/users (UsersRead)
        .route("/", get(list_users).route_layer(users_read_middleware()))
        // 写组: POST /api/users + PUT /api/users/{id}/role (UsersWrite)
        .route("/", post(create_user).route_layer(users_write_middleware()))
        .route("/{id}/role", put(update_user_role).route_layer(users_write_middleware()))
        // 删组: DELETE /api/users/{id} (UsersDelete)
        .route("/{id}", delete(delete_user).route_layer(users_delete_middleware()))
        .with_state(db)
}

```

项目独有的中间件包装模式：permission_middleware 需要 required_permission 参数，但 from_fn 只能接受 fn(Request, Next) 形状的函数——所以用闭包固定参数再传给 from_fn。这就是 admin 权限细分的基础：读、写、删三个权限各自控制一组路由。

权限 → 路由映射表：

权限	控制的路由
UsersRead	GET /api/users
UsersWrite	POST /api/users、PUT /api/users/{id}/role
UsersDelete	DELETE /api/users/{id}

注意 axum 0.7 的路径参数语法是 {id} (0.6 是 :id)。升级/模仿代码时注意版本差异。

4.7.2 3.7.2 handlers.rs 的安全细节 (230 行)

```

// ① 创建用户：角色白名单
pub async fn create_user(
    State(db): State<DatabaseConnection>,
    Json(body): Json<CreateUserRequest>,
) -> Result<Json<ApiResponse<User>>, AppError> {
    // 角色必须在注册表中，否则拒绝（防注入任意角色）
    if !is_registered_role(&body.role) {
        return Err(AppError::bad_request(format!("未知角色: {}", body.role)));
    }
    let user = AuthService::create_user(&db, &body.username, &body.email, &body.password, &body.role)
        .await
        .map_err(|e| AppError::bad_request(e.to_string()))?;
    Ok(Json(ApiResponse::success(user)))
}

// ② 修改角色：不能移除自己的管理角色（防锁死）
pub async fn update_user_role(...) -> Result<Json<ApiResponse<User>>, AppError> {
    let current_user: User = request.extensions().get().unwrap().clone();
    if body.role != "admin" && current_user.id == user_id && current_user.role == "admin" {
        return Err(AppError::bad_request("不能移除自己的管理角色".to_string()));
    }
    // ... 更新
}

```



```

}

// ③ 删除用户：不能删自己
pub async fn delete_user(...) -> Result<Json<ApiResponse<()>>, AppError> {
    if current_user.id == user_id {
        return Err(AppError::bad_request("不能删除自己".to_string()));
    }
    // ... 删除
}

```

三个安全细节都是“防止管理员把自己锁在门外”的设计——接盘后**不要去掉**。

4.7.3 3.7.3 页面与接口对照 (admin 前端)

前端页面	接口	方法
Users.vue 列表	GET /api/users	list_users
CreateUser.vue 提交	POST /api/users	create_user
Users.vue 编辑角色	PUT /api/users/{id}/role	update_user_role
Users.vue 删除	DELETE /api/users/{id}	delete_user

4.8 3.8 fj200c_information 完整精读 (最典型硬件模块范本)

4.8.1 3.8.1 模块全景

这是全项目**最值得精读**的模块：麻雀虽小五脏俱全，串口、模拟器、帧提取、解码、CSV 状态机、命令通道、WS 推送、配置热加载全部具备，而代码量（约 1800 行）远小于 fj200c_main。

```

flowchart TD
    subgraph 配置层
        CF[config.rs 全局单例] --> INI[config-fj200c_information.ini]
    end
    end
    subgraph 生命周期
        SV[service.rs start/stop<br/>8 路连接编排]
        SS[SERVICE_RUNNING 停止标志]
    end
    end
    subgraph 数据源
        CO[com.rs SerialControl<br/>真实串口]
        MO[mock.rs MockControl<br/>20Hz 模拟帧]
        MF[mock_feeder.rs<br/>虚拟串口对]
    end
    end
    subgraph 处理链
        SE[session.rs run_one_connection<br/>每连接一个会话线程]
        FE[FrameExtractor<br/>找帧头/拼帧/校验]
        DE[decode.rs<br/>100B 帧 → 28 字段]
        SB[SharedData 16 字段<br/>frame_bundle 复合存储]
    end
    end
    subgraph 输出
        TX[broadcast 通道<br/>TableData/Frame/Payload]
        CSVW[CsvWriter<br/>SYSJSK 开始/SYSJMK 结束]
    end
    end
    subgraph HTTP/WS
        HD[handlers.rs 8+1 接口]
        RT[routes.rs 权限路由]
    end
    end
    SV --> CO
    SV --> MO
    SV --> MF

```

```

CF --> SV
CO --> SE
MO --> SE
SE --> FE
FE --> DE
DE --> SB
DE --> TX
DE --> CSVW
SS --> SE
HD --> SV
HD --> TX
RT --> HD
INI --> CF

```

4.8.2 3.8.2 事件与通道 (mod.rs)

```

// 事件枚举: WS 推送的三种载荷
#[derive(Debug, Clone, Serialize, ToSchema)]
#[serde(tag = "type")]
pub enum Fj200cInformationEvent {
    TableData { connection_index: usize, rows: Vec<TableRow> }, // 16 字段表格行
    Frame { connection_index: usize, hex: String, frame_type: String, fields:
Vec<String> }, // 帧明细
    Payload { connection_index: usize, hex: String }, // 原始字节
}

// 广播通道 (容量 1024)
pub static FJ200C_INFORMATION_TX: OnceLock<broadcast::Sender<Fj200cInformationEvent>> =
OnceLock::new();
pub fn fj200c_information_tx() -> broadcast::Sender<Fj200cInformationEvent> {
    FJ200C_INFORMATION_TX.get_or_init(|| broadcast::channel(1024).0).clone()
}

```

三种事件的定位 (前端据此决定渲染什么): - Payload: 原始字节 (命令通道面板/调试用)。
 - Frame: 单帧详情 (帧类型 + 28 字段 + hex, 可视化页面)。 - TableData: 16 个标识字段汇总 (监控表格用, 200ms 节流)。

4.8.3 3.8.3 会话线程 run_one_connection (session.rs, 核心中的核心)

这是整个模块的大脑。**逐段精读** (对应真实文件 90-202 行):

```

pub fn run_one_connection(
    connection_index: usize,
    control: Arc<dyn IoControl>, // 串口或模拟器 (trait 对象)
    tx: broadcast::Sender<Fj200cInformationEvent>, // 广播发送端
    cfg: &Config, // 配置引用
) {
    // ① 读 CSV 配置
    let csv_enabled = cfg.get_or("CSV", "Enabled", "true").eq_ignore_ascii_case("true");
    let csv_dir = cfg.get_or("CSV", "Dir", "csv");

    // ② 设置接收超时 (200ms: 超时=轮询周期)
    if let Err(e) = control.set_timeout(RECV_TIMEOUT_MS) { ... }

    // ③ 构造帧提取器 (注入校验函数 + 解码闭包)
    // * 解码器把结果写入共享 Arc<Mutex<Option<ExtractedFrame>>>,
    // 主循环 try_lock 取走—解码器与主循环解耦
    let result: Arc<Mutex<Option<ExtractedFrame>>> = Arc::new(Mutex::new(None));
    let decoder = make_decoder(Arc::clone(&result));
    let mut extractor = FrameExtractor::new(HEADER.to_vec(), FRAME_LEN,
        Box::new(frame_validator), Box::new(decoder));

    // ④ 主循环

```

```

loop {
    // 停止检查 (Relaxed 序: 单标志轮询足够)
    if STOP_SIGNAL.load(Ordering::Relaxed) { break; }

    // ⑤ 命令通道: 非阻塞取命令并发送
    if let Some(cmd_rx) = COMMAND_RX.get() {
        if let Ok(cmd) = cmd_rx.lock().unwrap().try_recv() {
            control.send(&cmd)?; // 写进串口/模拟器
        }
    }

    // ⑥ 阻塞读 (串口 200ms 超时轮询)
    match control.recv(&mut recv_buf) {
        Ok(n) if n > 0 => {
            // 节流推送 payload 事件 (200ms 最多一次)
            // 送入帧提取器
            extractor.feed(chunk);
            // 非阻塞取解码结果 → handle_frame
            if let Ok(mut guard) = result.try_lock() {
                if let Some(extracted) = guard.take() {
                    handle_frame(connection_index, extracted, &tx, ...);
                }
            }
        }
        Ok(_) => {} // 空数据 (超时无数据) 继续
        Err(e) => {
            if is_timeout(&e) { continue; } // 超时是常态, 继续轮询
            break; // 真实错误 (设备拔出) 退出
        }
    }
}

// 退出前 flush CSV
if let Some(writer) = &csv { let _ = writer.flush(); }
}

```

设计精华提炼 (面试/答辩都能用):

1. **非阻塞命令通道**: 命令 (前端发送的 hex 指令) 通过 mpsc 通道 try_recv 拉取——数据流优先级高于命令流, 命令不会插队阻塞收帧。
2. **try_lock 解耦解码**: 解码闭包在 extractor 内部被调用 (同一线程), 写完 Some(frame) 后主循环 try_lock 取走——不成功就下一轮再取, 绝不阻塞。
3. **超时=心跳**: RECV_TIMEOUT_MS = 200, 串口 read 超时返回空数据 → 继续循环 → 相当于 5Hz 的“循环心跳”, 让停止标志能在 200ms 内被响应。
4. **ok(_) => {} 空分支**: 超时无数据时静默继续——**不要**在这里记日志, 否则每秒刷 5 条废话。

4.8.4 3.8.4 handle_frame: 帧处理与 CSV 状态机 (212-311 行)

```

fn handle_frame(...) {
    // 帧类型 → 中文名 (match 映射)
    let frame_type = match &extracted.frame_type {
        FrameType::CSSZZL => "参数设置",
        FrameType::SYSJSK => "试验数据首块",
        ...
    };

    // CSV 状态机
    match extracted.frame_type {
        FrameType::SYSJSK => { // 首块: 创建文件
            *csv_active = true;
            let filename = format!("fj200c_information_{}.csv",

```

```

        chrono::Local::now().format("%Y%m%d_%H%M%S"));
        *csv = Some(CsvWriter::create(csv_dir, &filename, CSV_HEADERS)?);
    }
    FrameType::SYSJZJK => {                // 中间块: 写行
        if csv_active {
            let fields = decode(...);
            writer.write_row(fields)?;
        }
    }
    FrameType::SYSJMK => {                // 末块: flush + 关闭
        writer.flush();
        *csv = None;
        *csv_active = false;
    }
    _ => decode_shared_data(&extracted.data),    // 其他帧: 解码标识字段
}

// ① 更新全局复合存储 (WS 快照用)
frame_bundle().update(fields, &extracted.data);

// ② 推送 Frame 事件 (每帧都发, 可视化用)
tx.send(Fj200cInformationEvent::Frame { ... });

// ③ 推送 TableData 事件 (200ms 节流, 防 UI 卡顿)
if last_table_emit.elapsed() >= TABLE_EMIT_INTERVAL {
    tx.send(Fj200cInformationEvent::TableData { rows: shared_data_rows() });
}
}

```

CSV 状态机 (硬件协议驱动):

flowchart LR

```

A[空闲] -->|收到 SYSJSK 首块帧| B[记录中<br/>文件已创建]
B -->|SYSJZJK 数据帧| B2[写行]
B2 -->|SYSJMK 末块帧| C[结束<br/>flush 关闭文件]
C --> A

```

节流是新手容易忽略的设计: 20Hz 模拟帧如果每帧都推 TableData, 前端表格 50ms 刷新一次必然卡顿。200ms 节流把刷新频率压到 5Hz——体验流畅度与数据实时性的平衡。

4.8.5 3.8.5 decode_shared_data: 16 字段解码 (318-365 行)

```

fn decode_shared_data(frame: &[u8]) {
    let shared = SharedData::global();        // 全局单例 (16 个 RwLock<String>)
    // 从帧的固定字节偏移取字段
    ascii.copy_from_slice(&frame[4..12]);
    *shared.field_product_name.write().unwrap() =
        utils::little_endian_bytes_to_ascii(&ascii)...;    // 产品名称
    *shared.field_engine_product_code.write().unwrap() = ...;    // 发动机产品代号
    // ... 共 16 个字段
}

```

协议知识: 帧格式为 [帧头 3B][数据区 96B][校验 1B], 标识字段按**固定字节偏移**存放 (frame[4..12] 是产品名称等)。解码就是把字节切出来转成可读文本。改协议时改这里的偏移量。

4.8.6 3.8.6 service.rs: 服务编排 (177 行)

```

pub fn start_service(db: &DatabaseConnection, config: Config) -> Result<(), String> {
    // 先停旧服务 (防重复启动)
    stop_service();
    RUNTIME.wait_stopping(3);
}

```

```

// 遍历 8 个连接
for connection in 0..MAX_CONNECTIONS { // MAX_CONNECTIONS = 8
    let enabled = config.get_or(&format!("Connection{connection}"), "Enabled",
"false");
    if enabled != "true" { continue; } // 未启用跳过

    let tx = fj200c_information_tx();
    // 模拟 or 串口
    let control: Arc<dyn IoControl> = if mock_enabled {
        Arc::new(MockControl::create()) // 20Hz 正弦+噪声模拟
    } else {
        Arc::new(SerialControl::open(&cfig, connection)?) // 真实串口
    };
    let handle = std::thread::spawn(move || {
        run_one_connection(connection, control, tx, &config);
    });
    RUNTIME.push(handle); // 登记线程
}
SERVICE_RUNNING.set_running();
Ok(())
}

```

关键设计： - **先停后启**：stop_service() + wait_stopping(3) 防止“旧线程没死就开新线程”的竞态。 - Arc<dyn IoControl>：模拟/串口都塞进同一容器，会话线程零感知。 - **8 路连接**：INI 里 [Connection0]~[Connection7] 各配一路 (Enabled 控制开关)。 - **stop_service 用独立线程 join** (不阻塞 HTTP handler 响应)：HTTP 请求“停止”立即返回，实际线程逐步退出。

4.8.7 3.8.7 handlers.rs: 8 个 HTTP 接口 + WS

接口	方法	路径	功能
start_service	POST	/service/start	启动服务
stop_service	POST	/service/stop	停止服务
get_service_status	GET	/service/status	查运行状态
send_command	POST	/service/command	发 hex 命令
get_config	GET	/config	读 INI 内容
save_config	PUT	/config	保存 INI (热加载)
list_csv_files	GET	/csv/files	CSV 文件列表
get_csv_file	GET	/csv/{name}	读 CSV 内容 (防穿越)
ws_handler	GET	/ws	WebSocket (token 查询参数)

CSV 防目录穿越 (安全重点，回看 2.6.3 的 let-else 例子就是这个文件)：

```

pub async fn get_csv_file(
    Path(name): Path<String>,
    State(db): State<DatabaseConnection>,
) -> Result<Json<ApiResponse<CsvFileContent>>, AppError> {
    // ① 解码百分号编码 (前端 encodeURIComponent 过)
    let Ok(name) = url::percent_decode_str(&name).decode_utf8() else {
        return Err(AppError::bad_request("文件名编码无效".into()));
    };
    // ② 只取最后一个 '/' 后的部分 (丢掉路径部分)
    let Some(file_name) = name.rsplit('/').next() else {
        return Err(AppError::bad_request("文件名不合法".into()));
    };
    // ③ 拼接时用 join 而非拼接字符串，保证不会逃出 csv 目录
    let path = Path::new(csv_dir()).join(file_name);
}

```

```

// ④ 确认文件存在于 csv 目录下（双重保险）
if !path.starts_with(csv_dir()) { ... 400 ... }
// ⑤ 读文件返回
}

```

4.8.8 3.8.8 config.rs: 热加载配置

```

pub fn get_config() -> Config {
    // OnceLock 单例
    CONFIG.get_or_init(|| Config::from_file(CONFIG_PATH).unwrap()).clone()
}

```

热加载的真相： session.rs 每轮循环不持有配置引用？——不，看代码：cfg: &Config 是启动时传入的引用。**真正热加载的是：**配置页 save_config 保存后，service.rs 的配置读取在每次 start_service 时重新读文件；运行时改配置生效的机制是——会话线程 run_one_connection 里 cfg.get_or("CSV", ...) 每次循环都查（Config 内部是共享的 Inl），加上**保存配置的 handler 会更新全局 CONFIG 单例**（重新加载文件）。修改 INI 立即生效 = 前端保存 → handler 重载全局单例 → 会话线程下一轮循环读到新值。这是 AGENTS.md 说的“热加载”。

4.8.9 3.8.9 mock.rs: 模拟数据源

```

pub struct MockControl { /* 模拟状态 */ }

impl IoControl for MockControl {
    fn recv(&self) -> Result<Vec<u8>, String> {
        // 20Hz: 每 50ms 一帧
        std::thread::sleep(Duration::from_millis(50));
        Ok(generate_frame()) // 正弦 + 噪声模拟帧
    }
    fn send(&self, _data: &[u8]) -> Result<(), String> { Ok(()) } // 模拟器不回数据
    fn set_timeout(&self, _ms: u32) -> Result<(), String> { Ok(()) }
}

```

20Hz 正弦+噪声： generate_frame() 用 rand 生成带随机抖动的数据帧，让前端曲线看起来像真实传感器——无硬件演示的关键。

4.9 3.9 fj200c_main 精读（最复杂模块：三路串口 + 报表）

4.9.1 3.9.1 与 fj200c_information 的差异总览

维度	fj200c_information	fj200c_main
串口数量	最多 8 路（同协议）	3 路（ 三种不同协议 ）
协议	单一 100B 帧	ECU(0xEB 0x90 0x2A) / ADAM(ASCII > 开头) / DYNO(0xFF 0xFF)
服务端状态	SharedData 16 字段	GlobalVar（试验信息/主题）+ CSV 状态机 + 模拟开关
CSV	协议帧驱动（SYSJSK 自动开始）	前端按钮手动开关 (toggle_csv_recording)
报表	无	report.rs 状态点插值生成报表
主题	无	服务端同步主题（WS 广播 theme_state）
前端复杂度	411 行 Monitor	仪表盘 + ScaledPage + 双主题 + 多页面

4.9.2 3.9.2 三路串口的抽象：abstract_com.rs

// ComSpec: 三种协议的规格描述

```
pub struct ComSpec {
    pub name: &'static str,           // "ECU" / "ADAM" / "DYNO"
    pub header: Vec<u8>,              // 帧头
    pub frame_len: usize,             // 帧长
    pub validator: Option<fn(&[u8]) -> bool>, // 校验函数
    pub decoder: Option<fn(&[u8]) -> Option<Vec<String>>>, // 解码函数
}

impl ComSpec {
    pub fn ecu_protocol() -> Self { Self { name: "ECU", header: vec![0xEB, 0x90, 0x2A], ... } }
    pub fn adam_protocol() -> Self { Self { name: "ADAM", header: vec![b'>'], ... } }
    pub fn dyno_protocol() -> Self { Self { name: "DYNO", header: vec![0xFF, 0xFF], ... } }
}
```

// AbstractCom: 协议无关的串口会话（组合 IoControl + ComSpec）

```
pub struct AbstractCom {
    spec: ComSpec,
    control: Arc<dyn IoControl>,
    // ... 停止标志、事件通道
}
```

设计模式：策略模式。协议差异（帧头/校验/解码）被抽象成 ComSpec 规格，串口会话逻辑（读写循环）只依赖规格——新增第四种协议只需加一个 ComSpec::xxx_protocol()。

4.9.3 3.9.3 宏生成三路实现：com.rs

```
macro_rules! define_com_port {
    ($name:ident, $spec:expr) => {
        pub struct $name {
            com: AbstractCom,
        }
    }
}
```



```

impl $name {
    pub fn new(port_name: &str, baud: u32) -> Result<Self, String> {
        let control = Arc::new(SerialControl::open(port_name, baud)?);
        Ok(Self { com: AbstractCom::new($spec, control) })
    }
    pub fn run(self, tx: broadcast::Sender<Fj200cMainEvent>, index: usize) {
        self.com.start_with(tx, index);
    }
}

};

}
define_com_port!(ECUCom, ComSpec::ecu_protocol());
define_com_port!(AdamCom, ComSpec::adam_protocol());
define_com_port!(DynoCom, ComSpec::dyno_protocol());

```

4.9.4 3.9.4 服务启停 (service.rs, 196 行)

```

pub fn start_service(tx: broadcast::Sender<Fj200cMainEvent>) -> Result<(), String> {
    if is_running() { return Err("服务已在运行中".into()); } // 幂等防重入
    RUNTIME.wait_stopping(Duration::from_secs(3)); // 清理旧线程

    let cfg = Config::load(state::CONFIG_PATH?); // 读 INI
    GlobalVar::init(); // 全局 KV 初始化
    gv.set("PathCSV", "csv"); // CSV 目录约定

    let shared = state::shared_port_data().cloned().unwrap(); // 共享端口数据
    let ports = init_all_from_config(&shared, tx.clone()); // 打开三路串口/模拟
    *state::ALL_COM_PORTS.lock().unwrap() = Some(ports);

    let proc_stop = start_processing_thread(shared.clone(), tx.clone()); // CSV 处理线程
    state::PROCESSING_STOP.lock().unwrap() = Some(proc_stop);

    state::SERVICE_RUNNING.store(true, Ordering::Relaxed);
    info!("fj200c_main 服务已启动");
    Ok(())
}

```

对比 fj200c_information 的差异点： - 用 state::ALL_COM_PORTS 持有三路串口句柄（而非 RUNTIME 线程池）——结构不同但目的相同。 - 增加 start_processing_thread: 100ms 周期的 CSV 录制线程（前端开关驱动）。 - 模拟模式由 start_mock_senders 启动（INI [MOCK] SimulationMenu = true）。

4.9.5 3.9.5 模拟开关与主题（服务端状态）

```

// 模拟运行开关: WS 广播 simulation_state
pub fn toggle_simulation(tx: &broadcast::Sender<Fj200cMainEvent>) -> Result<(), String> {
    if is_running() {
        return Err("请先停止服务再切换模拟".into()); // 运行中禁止切换
    }
    let new_state = !state::is_simulation_enabled();
    state::set_simulation_enabled(new_state);
    let _ = tx.send(Fj200cMainEvent::SimulationState { simulating: new_state });
    Ok(())
}

// 主题: GlobalVar 存 + WS 广播（所有页面同步深/浅色）
pub fn set_theme(tx: &broadcast::Sender<Fj200cMainEvent>, is_dark: bool) -> Result<(), String> {
    GlobalVar 存 "theme" -> 广播 ThemeState { is_dark }
}

```

服务端状态设计：模拟开关、主题、试验信息都存 GlobalVar/state，任何页面打开时 WS 快照 + HTTP 接口都能拿到一致状态——这就是“主题切到深色，刷新页面还是深色”的实现。

4.9.6 3.9.6 报表生成 (report.rs)

```
// 从试验 CSV 中提取状态点性能数据（状态点由 INI [REPORT] StatePoints 配置）
pub fn process_report_csv(csv_path: &str, state_points: &[u32]) -> Result<ReportOutput,
String> {
    // 读 CSV → 按状态点插值 → 计算修正系数（ncor/fcor/sfc_cor）
    // 返回 PerformanceRow / StandardRow / DesignPointRow 三类数据
}
```

报表流程（前端 GenerateReport.vue 驱动）：1. 前端 POST /api/fj200c_main/report（带试验信息 + 状态点）。2. 后端从 csv/ 读试验数据，按 INI 配置的状态点（如 30000~53000 共 24 个）插值。3. 返回报表数据 → 前端打印（原生 window.print 方案）。

4.9.7 3.9.7 三路协议的启动线程 (com.rs init_ecu / init_adam / init_dyno)

// ECU 发送线程：每 100ms 重发查询指令（带计数器+校验和）

```
pub fn init_ecu(...) {
    thread::spawn(move || loop {
        // 构造 ECU_SEND_DATA 帧（含递增计数器）
        // 100ms 周期发送（串口指令轮询协议）
    });
}
```

// ADAM：每秒发 "#010\r" 轮询（ASCII 协议）

```
pub fn init_adam(...) {
    thread::spawn(move || loop {
        // "#010\r" → 读取 8 通道模拟量
    });
}
```

协议轮询差异：三种设备协议不同——ECU 用二进制帧 100ms 查询、ADAM 用 ASCII 命令 1s 轮询、DYNO 用二进制帧。这些周期与格式都写死在 com.rs，改协议时对照这里。

4.10 3.10 ftj1c 精读 (UDP 组播 + 主备切换)

4.10.1 3.10.1 模块结构

```
src/ftj1c/
├─ mod.rs           # 帧协议表（95B: EB 90 5B + SLOT + SEQ + 85B + 2B 校验）+ 事件枚举
├─ state.rs        # 停止信号 + QuadFrame 单例
├─ config.rs       # 配置单例（config-ftj1c.ini）
├─ models.rs       # IpConfig（16 组 IP/端口）+ 请求体
├─ udp.rs          # UdpControl: Mock/Real 双模式 + 组播
├─ process.rs      # start_all: 主备/单路/串口 8 个运行函数（931 行，模块最重）
├─ quad_frame.rs   # QuadFrame<95> 别名（主备切换）
├─ com.rs          # 坐标转换 + 三种协议构建器（ulh2ecef / to_cgcs2000）
├─ service.rs      # 启停/IP 配置/重载
├─ handlers.rs     # 6 个 HTTP + WS
└─ routes.rs       # 子路由
```

4.10.2 3.10.2 主备切换架构（本项目最复杂的并发设计）

```
flowchart TD
    subgraph TD
        subgraph 主链路
            M1[主链接收线程] -->|写入槽 0| QF[QuadFrame 95<br/>四槽帧缓冲]
        end
        subgraph 备链路
            B1[备链接收线程] -->|写入槽 1| QF
        end
        subgraph 单路
            S1[单路连接x6] -->|写入槽 2/3| QF
        end
        QF -->|CAS 序号去重| LATEST[最新帧]
    end
```

```
LATEST -->|50ms 节流| TH[Throttle 20FPS]
TH --> TX[broadcast → WS]
```

主备切换逻辑 (quad_frame.rs): 主源写入时更新心跳; 备源在主心跳超时 1000ms 后接管; 主源恢复自动切回。前端卡片上“主/备”标签随切换变化——模拟模式下 IP11 (主链) 每 5~10 秒故意暂停一次, 专门用来验证切换。

4.10.3 3.10.3 模拟模式的设计 (process.rs 顶部注释就是说明书)

```
///! `config-ftj1c.ini` 的 `[Udp] Mock = true` (默认) 时使用进程内数据源, 无需硬件:
///! 按 200ms 周期向各链路生成 `EB 90 5B` 帧; IP11 (主链) 在 5~10 秒窗口暂停,
///! 可验证主备切换。`Mock = false` 时使用真实 UDP 套接字 (组播收发)。
```

- **Mock = true**: 进程内生成帧, 200ms 周期, 主链 5~10 秒窗口暂停 (模拟断流)。
- **Mock = false**: UdpControl::Real 模式——组播 socket + SO_REUSEADDR + 1MB 缓冲。

4.10.4 3.10.4 事件节流器 Throttle (50ms = 20 FPS)

```
struct Throttle { last_emit: Instant }
impl Throttle {
    fn ready(&mut self) -> bool {
        let now = Instant::now();
        if now >= self.last_emit + EMIT_INTERVAL { // 50ms 间隔
            self.last_emit = now;
            true
        } else { false }
    }
}
```

与 fj200c_information 的“last_table_emit 变量”异曲同工——**高频采集 + 节流推送**是本项目统一的 WS 性能策略。UDP 数据可能 1kHz, 推给前端 20FPS 足够人眼。

4.10.5 3.10.5 坐标转换 (com.rs)

```
// 经纬高 → 地心坐标 (WGS84 ECEF)
pub fn ulh2ecef(lon: f64, lat: f64, h: f64) -> (f64, f64, f64) { ... }
// ECEF → CGCS2000 投影 (航天测控数据可视化用)
pub fn to_cgcs2000(...) -> ... { ... }
```

轨迹数据在地球坐标与显示坐标间转换——city3d 之外另一个“空间数据”模块, 新手了解即可。

4.11 3.11 fw100 / fw150 精读 (最简模块)

4.11.1 3.11.1 四个文件全部内容量级

这两个模块各约 100 行, **15 分钟读完一个**, 是新手建立“模块感”的最佳起点:

```
// fw100/routes.rs — 单路由 + 双层中间件
pub fn fw100_router(db: DatabaseConnection) -> Router {
    Router::new()
        .route("/items", get(list_items))
        .route_layer(permission_layer(Permission::Fw100Monitor))
        .with_state(db)
}

// fw100/handlers.rs — 单 handler
#[utoipa::path(get, tag = "fw100", path = "/api/fw100/items", operation_id = "fw100ListItems",
    responses((status = 200, description = "设备台账列表", body = ApiResponse<Vec<LedgerItem>>)))]
pub async fn list_items(
    State(db): State<DatabaseConnection>,
```

```

) -> Result<Json<ApiResponse<Vec<LedgerItem>>>, AppError> {
    let items = Fw100Service::list_items(&db).await?;
    Ok(Json(ApiResponse::success(items)))
}

// fw100/services.rs — 演示数据（无数据库表）
pub struct Fw100Service;
impl Fw100Service {
    pub async fn list_items(_db: &DatabaseConnection) -> Result<Vec<LedgerItem>, AppError>
    {
        Ok(demo_ledger_items("fw100")) // 纯函数演示数据
    }
}

```

4.11.2 3.11.2 fw100 与 fw150 的差异

差异点	fw100	fw150
路由前缀	/api/fw100	/api/fw150
权限	Fw100Monitor	Fw150Monitor
返回类型	LedgerItem (共用)	Fw150LedgerItem (独立 schema)
演示数据	demo_ledger_items("fw100")	独立生成
前端端口	5175	5178

为什么 fw150 有独立类型：进化历史——最初只有一个台账模块，后来拆分出 fw150 并给了独立类型，方便各自扩展字段。

4.12 3.12 city3d 精读 (数据库 CRUD 范本)

4.12.1 3.12.1 模块结构

```

src/city3d/
├─ models.rs      # Building/District/CityEvent/Overview + 分页 DTO
├─ routes.rs      # 三组 CRUD + /overview
├─ handlers.rs    # 12 个 handler (分页参数钳制)
└─ services.rs    # 分页查询 + LEFT JOIN 聚合 (避免 N+1)

```

4.12.2 3.12.2 接口一览 (14 个)

接口	方法	说明
/buildings	GET/POST	建筑分页列表/新建
/buildings/{id}	PUT/DELETE	建筑改/删
/districts	GET/POST	区域列表/新建
/districts/{id}	PUT/DELETE	区域改/删
/events	GET/POST	事件分页/新建
/events/{id}	DELETE	事件删除
/overview	GET	聚合统计 (HUD 数据源)

4.12.3 3.12.3 分页与钳制 (handlers.rs)

```

#[derive(Debug, Deserialize, Default, ToSchema)]
pub struct PaginationParams {
    #[serde(default)]
    pub page: Option<i64>,
    #[serde(default)]
    pub page_size: Option<i64>,
}

```

```
}
```

```
// 分页参数钳制：防大分页拖垮数据库
```

```
let page = params.page.unwrap_or(1).max(1); // 最小 1
let page_size = params.page_size.unwrap_or(20).clamp(1, 100); // 1~100
let offset = (page - 1) * page_size;
```

4.12.4 3.12.4 LEFT JOIN 聚合 (services.rs 的 N+1 优化)

```
// 区域列表需要"每个区域的建筑数"—
```

```
// 错误做法：查 5 个区域再查 5 次建筑数 (N+1 查询)
```

```
// 正确做法：一条 SQL 搞定
```

```
pub async fn list_districts(db: &DatabaseConnection) -> Result<Vec<District>, AppError> {
    sqlx::query_as::(<_, District>{
        "SELECT d.*, COUNT(b.id) AS building_count
        FROM city3d_districts d
        LEFT JOIN city3d_buildings b ON b.district_id = d.id
        GROUP BY d.id
        ORDER BY d.name",
    })
    .fetch_all(db)
    .await?;
}
```

这是全项目 SQL 质量最高的文件——学习 SQL 聚合/分页看这里。

4.12.5 3.12.5 overview 聚合统计

```
// HUD 顶部统计卡的数据源：区域数/建筑数/事件数/最近事件
```

```
pub async fn overview(db: &DatabaseConnection) -> Result<Overview, AppError> {
    let districts = query_scalar("SELECT COUNT(*) FROM
city3d_districts").fetch_one().await?;
    let buildings = query_scalar("SELECT COUNT(*) FROM
city3d_buildings").fetch_one().await?;
    let events = query_scalar("SELECT COUNT(*) FROM city3d_events").fetch_one().await?;
    let recent = query_as::(<_, RecentEvent>("SELECT ... ORDER BY created_at DESC LIMIT
5"))....;
    Ok(Overview { districts, buildings, events, recent })
}
```

4.13 3.13 role_template 精读 (新角色启动包)

src/role_template/ 是给“新角色开发”准备的参考模板，mod.rs 顶部自带启用说明：

```
//! # 角色模块模板
//!
//! ## 如何启用新角色模块
//!
//! 1. 复制本目录为 `src/xxx/`
//! 2. 在 `src/main.rs` 添加 `mod xxx;`
//! 3. 在 `src/xxx/routes.rs` 修改路径与权限
//! 4. 在 `src/routes.rs` 挂载 `nest("/api/xxx", xxx_router)`
//! 5. 在 `src/roles.rs` 注册角色
//! 6. 在 `src/api_docs.rs` 添加 paths/schemas/tags
//! 7. 执行 `npm run gen:api` 生成前端类型
#![allow(dead_code)] // 模板允许未使用代码

// routes.rs — 模板展示"nest + protected + permission 包装"
pub fn template_router(db: DatabaseConnection) -> Router {
    Router::new()
        .route("/items", get(list_items))
        .route_layer(permission_layer(Permission::TemplateMonitor))) // 改权限
```

```

        .with_state(db)
    }

```

第 08 章会基于它演示完整的新增角色流程。记住：模板里的权限名是占位符，复制后必须改。

4.14 3.14 api_docs.rs 精读 (OpenAPI 聚合 + 防漂移测试)

4.14.1 3.14.1 文件结构 (247 行)

```

// ① OpenAPI 结构体：聚合所有 paths / schemas / tags
#[derive(OpenApi)]
#[openapi(
    paths(
        crate::common::auth::handlers::login,
        crate::common::auth::handlers::get_profile,
        // ... 全部 40 个 handler 逐个列出
    ),
    components(schemas(
        ApiResponse::<LoginResponse>, User, LoginRequest, ...
        // ... 全部 72 个 schema
    )),
    tags(
        (name = "auth", description = "认证"),
        // ... 9 个 tag
    )
)]
pub struct ApiDoc;

// ② 运行时提供实时 spec: GET /api-docs/openapi.json
pub async fn openapi_json() -> Json<serde_json::Value> {
    Json(serde_json::to_value(ApiDoc::openapi()).unwrap())
}

// ③ 导出测试 (防漂移关卡)
#[cfg(test)]
mod tests {
    #[test]
    fn export_openapi() {
        let spec = ApiDoc::openapi();
        let json = serde_json::to_string_pretty(&spec).unwrap();
        std::fs::write("openapi/openapi.json", json).unwrap();

        // 断言 1: 40 个预期路径全部存在 (少了会失败)
        for path in EXPECTED_PATHS {
            assert!(spec.paths.contains_key(*path), "缺少路径: {path}");
        }
        // 断言 2: 50 个 operation 全部有 operationId (orval 依赖)
        // ...
    }
}

```

4.14.2 3.14.2 新增 handler 时必须同步的三处

1. `#[utoipa::path(...)]` 注解 (handler 上)。
2. `api_docs.rs` 的 `paths(...)` 列表加函数名。
3. 新 DTO 在 `components(schemas(...))` 加类型。

漏了任何一处：cargo test export_openapi 报错 (路径缺失) 或 orval 生成缺类型 (前端报错)。这个测试是“契约一致性”的守门员。

4.15 3.15 embedded_assets.rs 精读 (单 exe 前端内嵌)

4.15.1 3.15.1 结构体定义 (7 个应用各一个)

```
#[derive(RustEmbed)]
#[folder = "frontend/admin/dist/"]
pub struct AdminAssets;

#[derive(RustEmbed)]
#[folder = "frontend/fj200c_information/dist/"]
pub struct Fj200cInformationAssets;
// ... Fj200cMainAssets / Fw100Assets / Fw150Assets / Ftj1cAssets / City3dAssets
```

#[folder] 路径相对 crate 根 (项目根目录)。**编译时**把整个 dist 目录读进二进制。

4.15.2 3.15.2 泛型处理器

```
/// 泛型: A 是任意一个内嵌资源结构体
pub async fn serve_embedded<A: RustEmbed>(path: &str) -> Response {
    if path.is_empty() {
        return serve_index::<A>(); // 根路径 → index.html
    }
    match A::get(path) { // 命中资源
        Some(file) => {
            let mime = mime_guess::from_path(path).first_or_octet_stream();
            [(header::CONTENT_TYPE, mime.as_ref())], file.data.as_ref()).into_response()
        }
        None => serve_index::<A>(), // SPA 深链接回退
    }
}
```

4.15.3 3.15.3 路由注册 (21 条)

```
pub fn embedded_router() -> Router {
    Router::new()
        .route("/admin", get(|| async { Redirect::permanent("/admin/") }))
        .route("/admin/", get(|| async { serve_embedded::<AdminAssets>("/").await })))
        .route("/admin/*path", get(|Path(path): Path<String>| async move {
            serve_embedded::<AdminAssets>(&path).await
        })))
    // ... 其余 6 个应用 × 3 条 = 21 条
}
```

为什么要 3 条路由： /admin (重定向补斜杠) → /admin/ (根) → /admin/*path (任意子路径)。这是实测踩坑后的修复——matchit 对 /admin/ 触发 ExtraTrailingSlash 检查会 404，必须显式注册。

4.15.4 3.15.4 dev 模式对照 (main.rs 的 ServeDir)

```
.nest_service("/admin", ServeDir::new("dist-admin")
    .fallback(ServeFile::new("dist-admin/index.html")))
```

- 需要根目录有 dist-admin/ 等目录 (npm run build 后手动复制或 build script)。
- 这就是“开发时先 build 前端才能让后端托管”的原因；日常开发一般用 Vite dev server，不用这模式。

4.16 3.16 后端精读收官：模块知识地图

flowchart TD

```
subgraph 入门级["入门级 (先读)"]
    M1[fw100/fw150<br/>最简 CRUD 范本]
    M2[auth<br/>登录与用户]
    M3[admin<br/>权限分组路由]
end
```

```
subgraph 核心级["核心级（重点）"]
  M4[common<br/>中间件/错误/WS/工具]
  M5[fj200c_information<br/>硬件模块范本]
  M6[database/roles<br/>表结构与权限源]
end
subgraph 进阶级["进阶级（按需）"]
  M7[fj200c_main<br/>三协议/报表/主题]
  M8[ftj1c<br/>主备切换/UDP]
  M9[city3d<br/>SQL 聚合/分页]
  M10[api_docs/embedded<br/>契约与打包]
end
```

阅读建议：先读入门级建立模式 → 核心级理解骨架 → 进阶级按业务需求深入。本套文档的 06 章（类型同步）和 08 章（二次开发）会依赖本章的基础。

4.17 3.17 补充：后端代码阅读方法论（怎么读一个陌生模块）

4.17.1 3.17.1 五步阅读法

```
flowchart TD
  A[1. 看 routes.rs<br/>有哪些接口] --> B[2. 看 handler<br/>接口怎么处理]
  B --> C[3. 看 service<br/>业务逻辑/数据]
  C --> D[4. 看 models.rs<br/>数据结构]
  D --> E[5. 看 database.rs<br/>表结构]
```

自上而下：路由 → 处理 → 业务 → 数据。任何一个模块都能用这五步读完。

4.17.2 3.17.2 快速定位技巧

想找什么	搜索什么
某接口的实现	src/*/handlers.rs 里函数名（对照路由）
某字段来自哪	该模块 models.rs → database.rs 建表
某数据的产生	对应模块 session/process（采集线程）
权限控制点	permission_middleware::<Permission:::Xxx>
事件流向	模块 mod.rs 的 tx → ws_bridge

4.18 3.18 补充：三大监控模块对照（一次读懂三个）

维度	fj200c_information	fj200c_main	ftj1c
数据源	串口/模拟（8 路连接）	串口三路（ECU/ADAM/DYNO）	UDP 组播（16 路 IP）
帧大小	100 字节	按协议不同	95 字节
会话线程	每路一个 session	每路一个会话	主备切换 + 6 路单连
节流	20Hz	按需	50ms（20FPS）
配置生效	热加载	需重启	需重启
CSV	状态机三帧	64 列	按需
特殊	快照+增量	主题持久化/报表	坐标转换 CGCS2000

三个模块的骨架完全一致（routes→handlers→session→decode→broadcast），差异只在数据源与解码。

4.19 3.19 补充：后端错误处理的完整链路

4.19.1 3.19.1 AppError 的传播

// service 返回 Err(AppError::Xxx) → handler 用 ? 传播
// → axum 自动将 AppError 转换为 Json(ApiResponse{success:false,...})

错误	HTTP 状态	前端看到
AppError::BadRequest	400	message (参数问题)
AppError::Unauthorized	401	拦截器跳登录
AppError::Forbidden	403	权限不足
AppError::NotFound	404	资源不存在
AppError::Internal	500	内部错误

4.19.2 3.19.2 自定义错误信息

```
Err(AppError::BadRequest("密码错误".into()))  
Err(AppError::NotFound(format!("用户 {} 不存在", id)))  
Err(AppError::Internal(format!("数据库错误: {e}")))
```

4.19.3 3.19.3 前端如何感知

HTTP 状态码 → axios 拦截器 (401 特殊处理)
message → 响应体里 (res.message 展示)
success=false → 前端判断逻辑

4.20 3.20 补充：后端单元测试现状与写法

4.20.1 3.20.1 现有测试

1. export_openapi 防漂移测试 (api_docs.rs)
2. 各模块工具函数的单元测试 ([cfg(test)] mod tests)
3. 种子数据相关验证

4.20.2 3.20.2 怎么写新测试 (以工具函数为例)

```
[cfg(test)]  
mod tests {  
    use super::*;  
  
    #[test]  
    fn test_frame_extractor() {  
        let mut fe = FrameExtractor::new(100);  
        let bytes = vec![0xEB; 100];  
        let frame = fe.process(&bytes);  
        assert!(frame.is_some());  
    }  
}  
  
cargo test          # 跑全部  
cargo test xxx      # 只跑匹配的测试
```

建议：核心工具函数（解码/提取/校验）补测试收益最大——它们纯逻辑、无 IO。

4.21 3.21 补充：后端性能与并发注意点

关注点	现状	建议
会话线程数	每路连接一个线程	正常（8 路内）
广播负载	每帧全量广播	大客户端数时考虑节流（已有）
SQLite 写	串行写	数据量大时批量写
内存	帧缓冲有限长	注意环形缓冲边界
日志	debug 级别 IO 大	生产用 info/warn

4.22 3.22 补充：后端精读自测（30 题精华 10 问）

1. main.rs 启动七步是哪些？
2. 三个中间件的顺序与职责？
3. JWT 从签发到校验的完整路径？
4. 角色注册表为什么放后端？（唯一源）
5. 建表为什么没有迁移文件？（database.rs 直接建）
6. 会话线程的循环结构？
7. 热加载如何实现？（ArcSwap + 保存时重载）
8. 三种 IoControl 实现是什么？
9. WS 广播的通道类型？（broadcast）
10. AppError 如何转成 HTTP 响应？

答对 8+ → 03 章通过，可以进入前端语法章节。

4.23 3.23 补充：auth 模块完整走读（登录服务细节）

4.23.1 3.23.1 登录的四步校验

```
// src/common/auth/services.rs（结构示意图）
pub async fn login(db: &SqlitePool, login_data: &LoginRequest) -> Result<LoginResult, AppError> {
    // 1. 查用户（按邮箱）
    let user = query_user_by_email(db, &login_data.email).await?;
    // 2. 校验密码（bcrypt verify）
    if !verify_password(&login_data.password, &user.password_hash)? {
        return Err(AppError::BadRequest("密码错误".into()));
    }
    // 3. 检查禁用
    if !user.is_active {
        return Err(AppError::Forbidden("账号已禁用".into()));
    }
    // 4. 生成 token
    let token = sign_jwt(user.id, &user.email)?;
    Ok(LoginResult { token, user: ... })
}
```

顺序的讲究：先查存在（未找到与密码错误都报“密码错误”防探测），再验密码，再查禁用，最后发 token。

4.23.2 3.23.2 auth_me 的动态权限

```
// GET /api/auth/me: 每次实时查权限（不用 token 里的旧数据）
let permissions = query_user_permissions(db, user_id).await?;
```

意义：管理员改了角色 → 用户下次请求立刻生效（无需等 token 过期）。

4.24 3.24 补充：admin 模块权限分组路由细节

4.24.1 3.24.1 三组路由的中间件组合

```
// 只读组 (List/Get/Info): auth + role(admin)
// 写组 (Create/Update): auth + role + permission(UsersWrite)
// 删除组 (Delete): auth + role + permission(UsersDelete)
```

4.24.2 3.24.2 防自锁的实现

```
// 不能删除自己 / 不能降级自己为普通角色
if user.id == current_user.id { return Err(...) }
```

前端同步：按钮 disabled 依据当前登录用户 id 判断。

4.25 3.25 补充：common 工具层逐个说明

文件	核心函数	用途
frame_extractor.rs	process(&bytes)	攒字节切完整帧
csv_writer.rs	write_row / flush	追加写 CSV
global_var.rs	get/set	键值存储 (主题等)
utils.rs	parse_hex / bytes_to_hex	十六进制转换
ledger.rs	台账演示数据	种子
least_squares.rs	fit	最小二乘拟合
dto.rs	通用 DTO	分页等

这些工具是“积木”——新模块可以直接 import 复用。

4.26 3.26 补充：fj200c_main 的 ECU 解码与指令细节

4.26.1 3.26.1 EcuFields 结构 (29 字段)

```
// src/fj200c_main/types.rs (EcuFields 字段预览)
pub struct EcuFields {
    pub ng_speed: f64,      // 转速
    pub coolant_temp: f64,  // 水温
    pub oil_pressure: f64,  // 油压
    pub fuel_consumption: f64, // 油耗
    // ... 29 个字段
}
// serde camelCase: 前端 JSON 字段为 ngSpeed/coolantTemp
```

4.26.2 3.26.2 三路协议的差异

路	协议特征	帧头
ECU	自定义二进制	0xEB 0x90 0x2A
ADAM	ASCII (‘>’ 起始)	‘>’
DYNO	自定义二进制	0xFF 0xFF

前端感知差异：三块面板数据字段不同、命令不同，但连接管理统一 (AbstractCom)。

4.27 3.27 补充：ftj1c 的主备切换细节

4.27.1 3.27.1 为什么需要主备

UDP 组播多源：主源 + 备源
主源心跳超时 (1000ms) → 备源接管
恢复后自动切回

4.27.2 3.27.2 QuadFrame<95> 的作用

// 95 字节帧，四路源管理（主备 × 通道）
QuadFrame<95> 持有各源最近帧 + 心跳时间

4.27.3 3.27.3 节流 50ms 的意义

UDP 帧率高（可能每毫秒多帧）→ 50ms 节流 → 20 FPS 推送前端
→ 前端流畅 + 后端省 IO

4.28 3.28 补充：city3d 的 SQL 聚合细节

4.28.1 3.28.1 overview 的聚合

-- 建筑数/区域数/事件数 一次查齐

```
SELECT
  (SELECT COUNT(*) FROM city3d_buildings) AS building_count,
  (SELECT COUNT(*) FROM city3d_regions) AS region_count,
  (SELECT COUNT(*) FROM city3d_events) AS event_count;
```

4.28.2 3.28.2 LEFT JOIN 的用法

-- 事件关联区域（无区域的事件也显示）

```
SELECT e.*, r.name AS region_name
FROM city3d_events e
LEFT JOIN city3d_regions r ON e.region_id = r.id;
```

4.28.3 3.28.3 分页 clamp

```
// page 最小 1, page_size 夹在 1..=100
let page = params.page.unwrap_or(1).max(1);
let page_size = params.page_size.unwrap_or(20).clamp(1, 100);
let offset = (page - 1) * page_size;
```

这是全项目分页的统一写法——抄它即可。

4.29 3.29 补充：03 章补充自测（追加 10 题）

1. 登录四步校验的顺序与原因？
2. auth_me 为什么实时查权限？
3. admin 三组路由的中间件组合？
4. frame_extractor 的作用？
5. EcuFields 字段序列化命名规则？
6. 三路协议的帧头？
7. 主备切换的触发条件？
8. 50ms 节流的意义？
9. overview 的聚合 SQL 模式？
10. 分页 clamp 的写法？

答对 8+ → 03 章彻底掌握。

4.30 3.30 深入：Config 读取的完整链路（以 fj200c_information 为例）

4.30.1 3.30.1 配置在内存中的形态

// config.rs 的 Config 结构（示意）

```
pub struct Config {
    pub mock: MockConfig,           // [Mock] InProcess
    pub connections: Vec<ConnConfig>, // [Connection1]~[Connection4]
    pub csv: CsvConfig,             // [CSV]
}
```

读取流程：

进程启动 → 读取 config-fj200c_information.ini
 → 解析成 Config 结构（解析器：连字符节名、KV 对）
 → 存入 ArcSwap<Config>（可热替换）
 → 各 service 通过 config() 快照函数读取

4.30.2 3.30.2 热更新的实现

文件监听（notify crate）→ 文件变更 → 重新解析
 → 校验合法 → arc_swap.swap(新 Config)
 → 服务下一次读取即用新值

关键设计：解析失败不覆盖旧配置（保持上次可用状态）。

4.30.3 3.30.3 配置校验的重要性

串口号非法（COM999）→ 校验拦截，保留旧值
 CSV 目录不存在 → 自动创建或报错
 端口号越界 → 拒绝

4.31 3.31 深入：CSV 写入的细节（数据如何落盘）

4.31.1 3.31.1 写入时机

每解析一帧 → 检查是否开启记录 → 写一行
 （受 [CSV] Record 开关控制）

4.31.2 3.31.2 写入格式

时间戳, 字段1, 字段2, ..., 字段N（CSV 首行是表头）
 → Excel 可直接打开分析

4.31.3 3.31.3 为什么不用 SQLite 存帧数据

对比	CSV	SQLite
写入速度	追加写极快	事务开销
文件体积	可控	需索引膨胀
分析工具	Excel 直接开	需查询
数据量	高帧率撑得住	高帧率有压力

结论：监控数据走 CSV（吞吐优先），业务数据走 SQLite（结构优先）。

4.32 3.32 深入：WebSocket 消息的结构设计

4.32.1 3.32.1 消息信封

```
{
  "type": "frame",           // 消息类型
  "data": { ... },           // 载荷
  "timestamp": 1700000000    // 服务端时间
}
```

好处：前端按 type 分发，新增类型不破坏旧逻辑。

4.32.2 3.32.2 三种核心消息

type	触发时机	前端处理
frame	每帧数据	更新表格/图表
status	服务状态变化	更新状态栏
snapshot	连接建立时	初始化数据

4.32.3 3.32.3 心跳与重连

后端 30s 发 ping → 前端收 pong
断线 → 前端 2s 自动重连 (token 重传)
重连成功 → 服务端重发 snapshot

4.33 3.33 深入：路由注册与中间件顺序

4.33.1 3.33.1 中间件顺序的影响

Route 层级：先全局中间件，后路径中间件
请求 → CORS → 认证中间件 → 权限中间件 → handler

注意： /api/auth/login 不需要认证 → 挂在 auth 路由组外层。

4.33.2 3.33.2 资源路由 vs 手动路由

```
// 资源风格 (nest + 方法链)
router.nest("/api/fw100", fw100_router)
// 手动风格
router.route("/api/fj200c_information/start", post(start_service))
```

项目两种混用：简单 CRUD 用资源风格，特殊接口手动。

4.34 3.34 深入：错误处理中间件与全局异常

4.34.1 3.34.1 AppError 的序列化

```
// 统一错误响应 JSON
{
  "success": false,
  "code": "bad_request",
  "message": "密码错误"
}
```

4.34.2 3.34.2 handler 返回的错误如何统一

handler 里 Result<_, AppError>
→ 框架自动转成 JSON 响应 (From<AppError> for Response 已实现)
→ 前端 axios 拦截器统一弹错

要点：错误处理只写一次 (From 实现)，所有 handler 复用。

4.35 3.35 深入：单元测试的写法与执行

4.35.1 3.35.1 测试什么

1. 帧提取器 (喂字节流 → 断帧/粘帧)
2. 解码器 (已知帧 → 字段值)
3. 校验逻辑 (非法输入)

4.35.2 3.35.2 测试写法示例

```
#[cfg(test)]
mod tests {
    #[test]
    fn test_frame_extract() {
        let mut buffer = vec![];
        let bytes = [0xEB, 0x90, 0x2A, ...]; // 一帧数据
        buffer.extend_from_slice(&bytes);
        let frames = process(&mut buffer);
        assert_eq!(frames.len(), 1);
    }
}
```

4.35.3 3.35.3 执行

```
cargo test          # 全部
cargo test frame    # 按名字过滤
```

4.36 3.36 深入：03 章最终综合自测（追加 10 题）

1. 配置热更新的双保险是什么？
2. 解析失败时旧配置为什么保留？
3. CSV 与 SQLite 的选型依据？
4. WS 消息信封的三个字段？
5. 断线重连后前端如何拿到最新数据？
6. 中间件顺序错误会导致什么？
7. AppError 的 From 实现解决什么问题？
8. 帧提取器测试喂什么数据？
9. 资源路由与手动路由的区别？
10. 日志中 ERROR 的含义与排查第一步？

答对 8+ → 03 章最终通过。

4.37 3.37 深入：项目实战——完整读一个 handler (fw100 列表)

4.37.1 3.37.1 代码定位

```
// src/fw100/handlers.rs (结构示意图)
#[utoipa::path(
    get,
    path = "/api/fw100/items",
    tag = "fw100",
    operation_id = "fw100ListItems",
    responses((status = 200, description = "设备列表", body = ApiResponse<Vec<Item>>))
)]
pub async fn list_items(
    State(state): State<AppState>,          // 1. 拿全局状态（数据库池）
    Query(params): Query<PageParams>,       // 2. 拿查询参数（分页）
    user: AuthUser,                         // 3. 拿登录用户（中间件注入）
) -> Result<Json<ApiResponse<Vec<Item>>>, AppError> { // 4. 返回 JSON
    let items = fw100_service::list_items(&state.db, &params).await?; // 5. 调 service
    Ok(Json(ApiResponse::success(items))) // 6. 包统一响应
}
```

4.37.2 3.37.2 逐行翻译

1. State<AppState>: Axum 提取器，从请求中拿出全局状态
2. Query<PageParams>: 把 URL 查询串解析成结构体
3. AuthUser: 中间件验完 token 后注入的用户信息
4. 返回类型: 统一 ApiResponse 包装
5. service 层处理业务（真正干活）
6. 成功包 success

4.37.3 3.37.3 从 handler 到数据库的完整链

```
handler → service::list_items → sqlx::query_as
→ SELECT * FROM fw100_items LIMIT ? OFFSET ?
→ Result<Vec<Item>> → ApiResponse
```

新手任务：照着这个模板，读 fw150 的 handler，写出一张同样结构的流程图。

4.38 3.38 深入：sqlx 查询的三个核心 API

API	用途	特点
query_as	映射到结构体	列名需与字段匹配
query	不映射	手动取行
fetch_one / fetch_all	取一行 / 多行	配 query_as 常用

```
// 单行
let user = sqlx::query_as::<_, User>("SELECT * FROM users WHERE id = ?")
    .bind(id).fetch_one(db).await?;

// 多行
let items = sqlx::query_as::<_, Item>("SELECT * FROM items LIMIT 20")
    .fetch_all(db).await?;

// 影响行数（增删改）
let result = sqlx::query("INSERT INTO items (name) VALUES (?)")
    .bind(name).execute(db).await?;
result.rows_affected()
```

4.38.1 3.38.1 列名映射的约定

Rust 字段: user_name
SQL 列: user_name（直接匹配）
或 SQL 用别名: SELECT user_name AS username ...

项目惯例：数据库列与结构体字段同名，减少映射坑。

4.39 3.39 深入：事务的用法（什么时候需要）

```
// 多表操作必须用事务（要么全成功要么全失败）
let mut tx = db.begin().await?;
sqlx::query("INSERT INTO a ...").execute(&mut *tx).await?;
sqlx::query("INSERT INTO b ...").execute(&mut *tx).await?;
tx.commit().await?;
```

需要事务的场景：

1. 两个表同时写（如订单 + 明细）
2. 先删后建（重建类操作）
3. 库存类增减（防超卖）

项目位置：报表生成、试验信息保存等多表操作处。

4.40 3.40 深入：索引与查询性能（SQLite 实践）

4.40.1 3.40.1 什么时候加索引

WHERE / ORDER BY 常用的列 → 加索引
频繁查询但表很小（<1000 行）→ 无需索引

```
CREATE INDEX idx_items_created_at ON fw100_items(created_at);
```

4.40.2 3.40.2 验证索引是否生效

```
EXPLAIN QUERY PLAN SELECT * FROM items WHERE created_at > ?;
-- 输出含 "SEARCH ... USING INDEX" 即生效
```

4.40.3 3.40.3 反模式

1. 给所有列加索引（写放大）
2. 索引列做函数运算（索引失效）: WHERE date(created_at) = ...
3. LIKE '%xx%'（无法用索引）

4.41 3.41 深入：03 章终极自测（5 题）

1. handler 的四个参数分别是什么？
2. 画一张 list_items 的完整调用链？
3. fetch_one 与 fetch_all 的区别？
4. 什么场景必须用事务？
5. 加索引的判断标准？

答对 4+ → 03 章彻底通关。

4.42 3.42 深入：Axum 路由与提取器的完整参考

4.42.1 3.42.1 常见提取器

提取器	用途	例子
State	全局状态	State(state): State
Path	路径参数	Path(id): Path
Query	查询参数	Query(params): Query
Json	JSON 请求体	Json(req): Json
HeaderMap	请求头	headers: HeaderMap
AuthUser (自定义)	登录用户	user: AuthUser

4.42.2 3.42.2 提取器顺序注意

提取器按顺序消费请求体

- 一个 handler 只能有一个"身体"提取器（Json/Form/bytes）
- 其他提取器（State/Path/Query/Header）顺序随意

4.42.3 3.42.3 返回类型的写法

```
Result<Json<ApiResponse<T>>, AppError>           // 成功 JSON
Result<Response, AppError>                        // 自定义响应（CSV 下载）
Result<impl IntoResponse, AppError>               // 泛化写法
```

4.43 3.43 深入：全局状态 AppState 的完整设计

```
#[derive(Clone)]
pub struct AppState {
    pub db: SqlitePool,           // 数据库连接池
    // 其他模块级状态按需加入
}
```

```
// 创建：main.rs 初始化
let state = AppState { db };
let app = Router::new()
    .with_state(state); // 全局注入
```

4.43.1 3.43.1 为什么用 Clone

Axum 每个请求需要自己的 state 副本

- SqlitePool 内部是 Arc（克隆只是引用计数）
- 成本极低，放心 Clone

4.43.2 3.43.2 模块级状态 vs AppState

模块级（OnceLock）：进程级共享（WS 广播、串口）

AppState：请求级注入（数据库池）

规则：所有请求都要用的 → AppState；单实例资源的 → OnceLock

4.44 3.44 深入：中间件链的完整自定义

// 认证中间件（项目简化示意）

```
pub async fn auth_middleware(
    State(state): State<AppState>,
    mut req: Request,
    next: Next,
) -> Result<Response, AppError> {
    // 1. 取 token
    let token = req.headers()
        .get(AUTHORIZATION)
        .and_then(|v| v.to_str().ok())
        .and_then(|s| s.strip_prefix("Bearer "));

    // 2. 校验
    let Some(claims) = token.and_then(|t| verify_jwt(t)) else {
        return Err(AppError::Unauthorized("未登录".into()));
    };

    // 3. 注入扩展（后续提取器可读）
    req.extensions_mut().insert(AuthUser::from(claims));

    // 4. 放行
    Ok(next.run(req).await)
}
```

4.44.1 3.44.1 中间件的三层结构

全局（CORS/日志）→ 路由组（认证）→ 单路由（权限）

4.44.2 3.44.2 extensions 的用途

中间件把用户信息塞进 extensions

handler 的 AuthUser 提取器从 extensions 取

→ 中间件与 handler 通过 extensions 通信

4.45 3.45 深入：03 章实战自测（8 题）

1. 五种提取器的写法？
2. 为什么只能有一个身体提取器？
3. AppState 为什么 Clone？
4. 模块级与 AppState 的分工？
5. 中间件四步结构？
6. extensions 的作用？
7. 权限中间件的顺序？
8. 自定义响应（CSV）返回什么？

答对 7+ → 03 章实战通过。

4.46 3.46 深入：WebSocket 后端的完整实现参考

4.46.1 3.46.1 建立连接

// ws_handler（各监控模块通用模式）

```
pub async fn ws_handler(
    ws: WebSocketUpgrade,
    Query(params): Query<WsQuery>,
) -> Result<impl IntoResponse, AppError> {
    // 校验 token（?token=）
    verify_jwt(&params.token)?;
    // 升级协议
    Ok(ws.on_upgrade(handle_socket))
}
```

4.46.2 3.46.2 发送消息

```
async fn handle_socket(socket: WebSocket) {
    let (mut sender, mut receiver) = socket.split();
    // 订阅广播通道
    let mut rx = TX.subscribe();
    // 连上先发快照
    sender.send(Message::Text(snapshot_json)).await?;
    // 循环转发
    while let Ok(msg) = rx.recv().await {
        if sender.send(Message::Text(msg)).await.is_err() {
            break; // 客户端断开
        }
    }
}
```

4.46.3 3.46.3 断开处理

send 失败 → break → socket 自动关闭
→ 广播通道自动退订 (rx 丢弃)
→ 无泄漏 (无需手动管理连接列表)

4.47 3.47 深入：串口通信后端的完整实现参考

4.47.1 3.47.1 打开与配置

```
// serialport 库
use serialport::prelude::*;

let mut com = serialport::new("COM3", 115200)
    .timeout(Duration::from_millis(100))
    .data_bits(DataBits::Eight)
    .stop_bits(StopBits::One)
    .parity(Parity::None)
    .open()?;
```

4.47.2 3.47.2 读线程

```
// 独立任务持续读
tokio::spawn(async move {
    loop {
        let n = com.read(&mut buf)?; // 阻塞读
        if n > 0 {
            // 交给帧提取器
            tx.send(buf[..n].to_vec()).await?;
        }
    }
});
```

4.47.3 3.47.3 写指令

```
com.write(&cmd_bytes)?;
com.flush()?; // 确保发出
```

4.47.4 3.47.4 波特率与协议

波特率在 ini 配置 (ConnectionN 节)
不同设备协议 → 帧提取/解码不同实现
抽象层 (AbstractCom) 统一 open/read/write

4.48 3.48 深入：定时任务与周期操作

```
// tokio interval (心跳/轮询)
let mut interval = tokio::time::interval(Duration::from_secs(30));
loop {
    interval.tick().await;
    // 发送心跳/检查状态
}
```

```

}

// tokio::time::sleep (延迟)
tokio::time::sleep(Duration::from_millis(50)).await;

```

4.48.1 3.48.1 项目中的定时任务

1. 心跳超时检测 (ftj1c 主备切换)
2. 状态广播节流 (50ms)
3. 定时保存/清理
4. 报表定时生成 (如需要)

4.49 3.49 深入：03 章高频自测 (8 题)

1. WS 升级前校验什么?
2. 断开如何自动清理?
3. 串口打开的关键参数?
4. 读线程为什么用 spawn?
5. 帧提取在哪层完成?
6. interval 与 sleep 的区别?
7. 心跳超时的用途?
8. 广播退订机制?

答对 7+ → 03 章高频通过。

4.50 3.50 深入：CSV 记录的完整实现参考

4.50.1 3.50.1 开启与关闭

// 服务控制接口
start_service(config) → 若 [CSV].Record = true → 开始写
stop_service() → 关闭文件句柄

4.50.2 3.50.2 写入线程

```

// mpsc 队列：帧数据 → 写线程（避免阻塞主循环）
let (tx, mut rx) = mpsc::channel(1024);
tokio::spawn(async move {
    while let Some(row) = rx.recv().await {
        writer.write_row(&row)?;
        // 自动 flush 间隔（或积累后写）
    }
});

```

4.50.3 3.50.3 文件名规范

csv/2026-08-08.csv # 按天分文件
csv/2026-08-08_14-00.csv # 按小时（可选）

4.50.4 3.50.4 容量管理

1. 定期归档（脚本/计划任务）
2. 磁盘空间预警
3. 可选：自动删除 N 天前文件

4.51 3.51 深入：帧提取器的完整实现参考

4.51.1 3.51.1 问题背景

串口/UDP 是字节流，没有明确边界
→ 需要“攒字节” → 识别帧头帧尾 → 切出完整帧"

4.51.2 3.51.2 通用实现

```
pub fn process(buffer: &mut Vec<u8>) -> Vec<Vec<u8>> {
    let mut frames = Vec::new();
    // 1. 找帧头 (0xEB 0x90)
    // 2. 校验长度字段 → 判断是否凑齐
    // 3. 凑齐 → 切帧 (drain 掉)
    // 4. 没凑齐 → 等待下次数据
    frames
}
```

4.51.3 3.51.3 断帧/粘帧的处理

1. 断帧: 只收到半帧 → 留在 buffer 等下次
2. 粘帧: 一包多帧 → 循环切出全部
3. 脏数据: 找不到帧头 → 丢弃到下一个帧头

4.51.4 3.51.4 测试要点

```
#[test]
fn test_sticky_frame() { // 粘帧
    let mut buf = vec![];
    buf.extend_from_slice(&frame1);
    buf.extend_from_slice(&frame2);
    assert_eq!(process(&mut buf).len(), 2);
}

#[test]
fn test_split_frame() { // 断帧
    let mut buf = frame[..10].to_vec();
    assert_eq!(process(&mut buf).len(), 0);
    buf.extend_from_slice(&frame[10..]);
    assert_eq!(process(&mut buf).len(), 1);
}
```

4.52 3.52 深入: 解码器的完整实现参考

4.52.1 3.52.1 二进制解码

```
fn decode_frame(raw: &[u8]) -> Option<TableRow> {
    // 字节序: 大端/小端按协议
    let ng_speed = i16::from_be_bytes([raw[5], raw[6]]) as f64;
    let coolant_temp = u16::from_le_bytes([raw[7], raw[8]]) as f64 / 10.0;
    // 校验和验证
    if checksum(raw) != raw.last()? { return None; }
    Some(TableRow { ... })
}
```

4.52.2 3.52.2 缩放系数

原始值 → 系数 → 物理量
转速: 原始值 × 0.1 (rpm)
温度: 原始值 × 0.1 - 40 (°C)

4.52.3 3.52.3 解码失败的日志

解码失败 → 记 debug/warn 日志 + 丢弃
→ 不 panic、不阻塞
→ 帧计数用于统计丢帧率

4.53 3.53 深入: 03 章综合自测 (8 题)

1. CSV 写入为什么用队列?
2. 文件名按天分的好处?
3. 断帧如何等待?

4. 粘帧如何切分?
5. 脏数据怎么处理?
6. 校验和的作用?
7. 缩放系数如何确定?
8. 解码失败的策略?

答对 7+ → 03 章综合通过。

4.54 3.54 深入：每路服务的启动/停止生命周期

4.54.1 3.54.1 状态机

```
stateDiagram-v2
    [*] --> Stopped
    Stopped --> Starting: start 接口
    Starting --> Running: 初始化成功
    Starting --> Stopped: 初始化失败
    Running --> Stopping: stop 接口
    Stopping --> Stopped: 资源释放
    Running --> Stopped: 异常退出
    Stopped --> [*]
```

4.54.2 3.54.2 启动的完整动作

1. 读配置 (ini 解析)
2. 打开资源 (串口/UDP socket)
3. 启动后台任务 (读线程/心跳)
4. 广播状态 (WS status 事件)
5. 前端状态栏更新

4.54.3 3.54.3 停止的完整动作

1. 停止接收 (关 socket/串口)
2. 停止任务 (abort/信号)
3. 关闭 CSV 写入 (flush)
4. 广播停止状态
5. 清理临时状态

4.54.4 3.54.4 幂等性设计

重复 start → 已在运行则忽略 (返回当前状态)
重复 stop → 已停止则忽略
→ 前端按钮不会因重复点击出错

4.55 3.55 深入：状态管理 (服务状态与配置状态)

4.55.1 3.55.1 状态存储位置

1. SERVICE_RUNNING (OnceLock<AtomicBool>): 服务运行标志
2. 模块内配置结构: 当前生效配置
3. SHARED_DATA: 最新数据帧
4. TX: 广播通道

4.55.2 3.55.2 状态广播的时机

1. 服务状态变化 (start/stop/异常)
2. 每帧数据 (节流后)
3. 配置热更新

4.55.3 3.55.3 前端状态同步

WS status 事件 → 状态栏
WS frame 事件 → 数据表格
快照 → 页面初始化

4.56 3.56 深入：模拟模式 (Mock) 的设计

4.56.1 3.56.1 为什么需要 Mock

1. 开发环境没有硬件 → 无法联调
2. 演示需要数据 → 无硬件也能展示
3. 测试流程 → 可控数据源

4.56.2 3.56.2 模拟数据源的结构

```
// mock.rs: 生成仿真数据
pub struct MockSource { seed: f64 }
impl MockSource {
    fn next_frame(&mut self) -> TableRow {
        // 基于时间/正弦波生成合理数值
        TableRow { ng_speed: 800.0 + 50.0 * (t * 0.1).sin(), ... }
    }
}
```

4.56.3 3.56.3 数据源切换

ini: [Mock] InProcess = true → 用 MockSource
false → 用真实串口
启动时读取 → 数据源固定

4.56.4 3.56.4 Mock 的注意点

1. 生成的数据要“像真的”（变化合理）
2. 提供随机性（不同启动不同数据）
3. 标记来源（前端可显示“模拟数据”）

4.57 3.57 深入：报表与试验信息 (fj200c_main 特有)

4.57.1 3.57.1 试验信息是什么

一次试验：工况设置 + 持续时间 + 采样数据
→ 试验完成后可生成报告

4.57.2 3.57.2 报表生成流程

1. 试验中记录数据（CSV + 内存摘要）
2. 试验结束 → 汇总状态点数据
3. 生成报表（CSV/文本）
4. 前端可下载

4.57.3 3.57.3 状态点 (StatePoints)

ini: [REPORT] StatePoints = 100, 200, 300...
→ 到达指定转速点记录该点数据
→ 报表包含各状态点采样

4.58 3.58 深入：03 章终局自测 (8 题)

1. 服务状态机的五个状态？
2. 启动的四步动作？
3. 停止的流程？
4. 幂等性的意义？
5. Mock 的四个作用？
6. 数据源怎么切换？
7. 状态点的作用？
8. 报表数据的来源？

答对 7+ → 03 章终局通过。

4.59 3.59 深入：后端日志的完整规范

4.59.1 3.59.1 日志级别速查

级别	用法	例子
error!	不可恢复/严重	串口打开失败
warn!	可恢复异常	心跳超时
info!	关键状态	服务启动/停止
debug!	详细调试	每帧数据

4.59.2 3.59.2 日志内容规范

1. 带上下文：连接 COM3 失败（不只"失败"）
2. 带错误详情：{e}
3. 关键操作记 info：用户 xx 登录/删除设备 xx
4. 不记敏感信息：密码/token

4.59.3 3.59.3 日志排查示例

[ERROR] 连接 COM3 失败：系统找不到指定的文件
→ 串口不存在（设备管理器核对 COM 号）

[WARN] 心跳超时，切换备用源
→ 主源断流（检查上游设备）

[ERROR] 解析帧失败：checksum mismatch
→ 数据链路脏（检查波特率/接线）

4.60 3.60 深入：权限校验的完整实现

4.60.1 3.60.1 三层校验链路

1. 认证中间件：token 有效 → AuthUser
2. 角色中间件：用户角色在允许列表
3. 权限中间件：用户权限含所需 Permission

4.60.2 3.60.2 路由上的组合

```
router.route("/api/users",
  get(list_users)
    .route_layer(permission_middleware(Permission::SystemAdmin))
)
```

4.60.3 3.60.3 权限检查的返回

无权限 → 403 Forbidden + 错误消息
前端拦截器 → 提示"无权限" → 不跳转

4.61 3.61 深入：数据库连接的池化

4.61.1 3.61.1 连接池的意义

SQLite 单文件但连接开销仍存在
→ 池化复用连接（默认 5 个）
→ 并发请求不互相等待

4.61.2 3.61.2 配置连接池

```
let pool = SqlitePoolOptions::new()
  .max_connections(5)
  .connect(&database_url).await?;
```

4.61.3 3.61.3 使用注意

1. 借出连接用完即还（drop）
2. 长事务会占连接（注意超时）
3. WAL 模式支持读写并发

4.62 3.62 深入：03 章毕业自测（8 题）

1. 四级日志的选用？
2. 日志内容的规范？
3. 三个排查示例的含义？
4. 三层校验链路？
5. 权限中间件的挂法？
6. 无权限的返回？
7. 连接池的配置？
8. WAL 与连接池的关系？

答对 7+ → 03 章毕业。

4.63 3.63 深入：常见需求的标准实现模式库

4.63.1 3.63.1 分页查询模板

```
pub async fn list_items(
    db: &SqlitePool, page: i64, page_size: i64, keyword: Option<&str>,
) -> Result<(Vec<Item>, i64), AppError> {
    let offset = (page - 1) * page_size;
    let where_clause = keyword.map(|k| format!("{k}%")).unwrap_or("").into();

    let items = sqlx::query_as::<_, Item>(
        "SELECT * FROM items WHERE name LIKE ? ORDER BY id DESC LIMIT ? OFFSET ?")
        .bind(where_clause).bind(page_size).bind(offset)
        .fetch_all(db).await?;

    let total = sqlx::query_scalar::<_, i64>(
        "SELECT COUNT(*) FROM items WHERE name LIKE ?")
        .bind(where_clause)
        .fetch_one(db).await?;

    Ok((items, total))
}
```

4.63.2 3.63.2 增删改查模板

```
// 创建
pub async fn create(db, req) -> Result<Item, AppError> {
    let id = sqlx::query("INSERT INTO items (name, type) VALUES (?, ?)")
        .bind(&req.name).bind(&req.type_name)
        .execute(db).await?.last_insert_rowid();
    get_by_id(db, id).await
}

// 更新
pub async fn update(db, id, req) -> Result<Item, AppError> {
    let rows = sqlx::query("UPDATE items SET name=?, type=? WHERE id=?")
        .bind(...).execute(db).await?.rows_affected();
    if rows == 0 { return Err(AppError::NotFound("记录不存在".into())); }
    get_by_id(db, id).await
}

// 删除
pub async fn delete(db, id) -> Result<(), AppError> {
    sqlx::query("DELETE FROM items WHERE id=?").bind(id).execute(db).await?;
}
```



```
    Ok(()))
}
```

4.63.3 3.63.3 唯一性校验模板

```
// 邮箱/名称重复检查
let exists = sqlx::query_scalar::<_, bool>(<
    "SELECT EXISTS(SELECT 1 FROM users WHERE email = ?)"
    .bind(&email).fetch_one(db).await?;
if exists { return Err(AppError::BadRequest("邮箱已存在".into())); }
```

4.64 3.64 深入：多模块共享代码的组织

4.64.1 3.64.1 共享层的清单

```
src/common/
├─ models.rs      # Permission 枚举 + 通用模型
├─ dto.rs         # 通用 DTO（分页参数等）
├─ auth/         # 登录/JWT/中间件
├─ ws.rs         # WS 基础设施
├─ errors.rs     # AppError
├─ utils.rs      # 十六进制等工具
└─ middleware.rs # 认证/权限中间件
```

4.64.2 3.64.2 使用方式

```
// 模块里 use common::...
use crate::common::{AppError, models::Permission, ws::broadcast};
```

4.64.3 3.64.3 什么时候把代码下沉到 common

1. ≥ 2 个模块使用
 2. 与业务无关（通用能力）
 3. 有稳定接口
- 否则留在业务模块（避免过度抽象）

4.65 3.65 深入：03 章大师自测（8 题）

1. 分页查询的 SQL 模板？
2. 创建/更新/删除的返回约定？
3. 唯一性校验的写法？
4. common 层的清单？
5. 下沉 common 的三个条件？
6. rows_affected 的作用？
7. last_insert_rowid 的作用？
8. 过度抽象的危害？

答对 7+ → 03 章大师。

4.66 3.66 深入：后端测试策略详解

4.66.1 3.66.1 测试金字塔

单元测试（最多）：纯函数（帧提取/解码/校验）
集成测试（中间）：模块间（service + db）
端到端（最少）：全链路（HTTP 接口）

4.66.2 3.66.2 项目现有测试

1. 帧提取器测试（粘帧/断帧）
2. 解码测试（已知帧 → 期望值）
3. export_openapi 测试（契约防漂移）

4.66.3 3.66.3 单元测试写法模板

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_checksum_valid() {
        let raw = [0xEB, 0x90, 0x2A, 0x00, 0x04, 0x12, 0x34, 0x5A];
        assert!(verify_checksum(&raw));
    }

    #[test]
    fn test_decode_speed() {
        let row = decode_frame(&RAW_FRAME).unwrap();
        assert_eq!(row.ng_speed, 1200.0);
    }
}
```

4.66.4 3.66.4 集成测试模板 (HTTP)

```
#[tokio::test]
async fn test_login_success() {
    let app = build_test_app().await;
    let res = app
        .oneshot(Request::builder()
            .uri("/api/auth/login")
            .method("POST")
            .body(Body::from(r#"{"email":"admin@x.com","password":"123456"}"#))
            .unwrap())
        .await.unwrap();
    assert_eq!(res.status(), StatusCode::OK);
}
```

4.67 3.67 深入：性能与资源管理（后端运行期）

4.67.1 3.67.1 内存与线程

1. 每模块一个读线程（串口/UDP）
2. 数据缓存有界（环形/限长）
3. 广播频道有界（backpressure）
4. 长任务用 `spawn_blocking`

4.67.2 3.67.2 资源释放

1. 服务停止 → 关串口/关闭文件
2. 进程退出 → 自动释放
3. CSV 写线程 → `channel` 关闭退出

4.67.3 3.67.3 监控指标（可观测性）

1. 帧计数（接收/解析/丢弃）
 2. 连接状态
 3. 队列深度
 4. 磁盘占用
- 通过日志/接口暴露

4.68 3.68 深入：03 章权威自测（8 题）

1. 测试金字塔三层？
2. 项目现有三类测试？
3. 单元测试模板？
4. 集成测试模板？
5. 内存管理的四条？

- 资源释放的时机?
- 四类监控指标?
- 队列背压的意义?

答对 7+ → 03 章权威。

4.69 3.69 深入：各模块间的关系图（后端全景复盘）

flowchart TB

```

M[main.rs] --> D[database.rs]
M --> R[routes.rs]
R --> AUTH[common/auth]
R --> USERS[admin]
R --> FJ200C[fj200c_information]
R --> MAIN[fj200c_main]
R --> FTJ[ftj1c]
R --> FW100[fw100]
R --> FW150[fw150]
R --> CITY[city3d]
FJ200C --> COM1[common/com + mock]
MAIN --> COM3[common/abstract_com + 三路]
FTJ --> UDP[common/udp]
FJ200C --> CSV[common/csv_writer]
MAIN --> CSV
FJ200C --> WS1[common/ws 广播]
MAIN --> WS2[common/ws 广播]
FTJ --> WS3[common/ws 广播]

```

4.69.1 3.69.1 关系解读

- main 启动一切（db + 路由 + 模块初始化）
- 模块间不互相调用（各自独立）
- 共享基础设施在 common/
- 通信类模块：数据源（串口/UDP）→ 帧处理 → 广播
- 台账/管理类：HTTP CRUD + SQLite

4.69.2 3.69.2 为什么这样设计

- 解耦：模块独立开发/独立故障
- 复用：common 一次实现到处用
- 清晰：新人上手快
- 可测：模块可独立测试

4.70 3.70 深入：后端设计模式的总结

模式	位置	作用
三层架构	所有模块	handler/service/db 分层
工厂/模板	role_template	新模块起点
抽象接口	SerialControl	多协议统一
广播-订阅	WS 推送	一对多
队列-消费者	CSV 写入	解耦写盘
状态机	服务生命周期	启停可控
配置驱动	ini 热更新	免编译调整
契约生成	utoipa+orval	类型同步

4.71 3.71 深入：03 章权威自测（8 题）

- 画一张后端模块关系图？

2. 模块间为什么不互调?
3. 八种设计模式的对应位置?
4. 状态机的作用?
5. 队列-消费者的价值?
6. 配置驱动的意义?
7. 抽象接口的复用价值?
8. 为什么模块可独立测试?

答对 7+ → 03 章权威。

4.72 3.72 深入：后端开发环境配置 (rust-analyzer 使用指南)

4.72.1 3.72.1 rust-analyzer 的日常用法

1. 悬停查看类型 (Ctrl+Hover)
2. 转到定义 (F12)
3. 查找引用 (Shift+F12)
4. 重命名 (F2)
5. 自动修复建议 (Ctrl+.)

4.72.2 3.72.2 常用工作流

1. 读代码：悬停 → 转到定义 → 跟调用链
2. 改代码：重命名 → 找引用 → 编译验证
3. 调试：测试断点 (cargo test 下)

4.72.3 3.72.3 常见问题

1. 索引慢 → 等 build 完 (首次)
2. 类型不对 → 检查 feature/编译错误
3. 宏展开 → "Expand macro" 功能
4. 多项目 → 工作区设置 (本项目单 crate)

4.73 3.73 深入：后端代码审查清单 (提交前自查)

1. 所有 Result 正确传播 (无吞错)
2. 错误消息带上下文
3. 日志级别正确
4. 无 unwrap (测试除外)
5. SQL 参数化 (无字符串拼接)
6. 输入有校验
7. 资源有释放
8. 无死锁 (锁不跨 await)
9. 事务覆盖多表操作
10. utoipa 注解完整

4.74 3.74 深入：03 章权威自测 (8 题)

1. rust-analyzer 五个常用功能?
2. 读代码的三个工作流?
3. 四个常见问题?
4. 十条审查清单?
5. 无吞错的含义?
6. 锁跨 await 的问题?
7. SQL 拼接的风险?
8. 资源释放检查什么?

答对 7+ → 03 章权威。

4.75 3.75 深入：后端性能调优的完整清单

4.75.1 3.75.1 数据库层

1. 常用查询加索引（EXPLAIN 验证）
2. 分页用索引列排序
3. 避免 SELECT *（只取需要列）
4. 大结果集分页（LIMIT/OFFSET）
5. 频繁写的表考虑批处理

4.75.2 3.75.2 并发层

1. 广播频道容量合理（背压）
2. 锁粒度小（不要锁全表）
3. 避免锁内 await
4. 长任务 spawn_blocking
5. 连接池大小匹配并发

4.75.3 3.75.3 数据层

1. 帧缓存有界（环形缓冲）
2. CSV 写队列解耦
3. 序列化字段精简
4. 日志级别生产用 info（省 IO）

4.76 3.76 深入：后端监控与指标

4.76.1 3.76.1 代码内指标

1. 帧计数（收到/解析/丢弃）
2. 队列长度（CSV 积压）
3. 连接数（WS 客户端）
4. 处理耗时（接口响应）

4.76.2 3.76.2 暴露方式

1. 日志（定时汇总）
 2. 接口（/api/xxx/status 扩展）
 3. 文件（计数写入）
- 简单系统用日志 + 状态接口即可

4.76.3 3.76.3 告警阈值建议

1. 丢弃率 > 5% → 检查链路
2. 队列积压 > 1000 → 检查写盘
3. WS 断连频繁 → 检查网络
4. 接口 > 2s → 检查查询

4.77 3.77 深入：03 章权威自测（8 题）

1. 数据库层五点？
2. 并发层五点？
3. 数据层四点？
4. 四类代码内指标？
5. 三种暴露方式？
6. 四个告警阈值？
7. 背压的意义？
8. 环形缓冲的好处？

答对 7+ → 03 章权威。

下一节：04-Vue3 与 TypeScript 语法速成。

5 04 Vue 3 与 TypeScript 语法速成（以本项目代码为教材）

适用对象：Vue 零基础或接触过 Vue 2、想快速上手 Vue 3 的新手。教学目标：看懂并修改本项目的 Vue 代码——语法点全部用项目真实代码举例（带文件路径）。本项目前端统一使用 **Vue 3 组合式 API + <script setup> + TypeScript**，没有 Vue 2 风格的选项式代码，学起来更统一。全文约 1.5 万字，建议 4~6 小时消化。

5.1 4.1 先建立 Vue 3 的心智模型

5.1.1 4.1.1 Vue 的核心概念三件套

概念	一句话	类比（Rust/其他）
响应式状态	数据变了，界面自动更新	ref/reactive ≈ 可观察变量
模板	用声明式写法描述界面	类似 JSX/模板字符串，但有编译期优化
组件	可复用的界面单元	类似函数/模块，但自带模板和状态

本项目的页面流：Vue Router 把 URL 映射到组件 → 组件在 <script setup> 里定义状态与逻辑 → 模板里渲染 → 用户交互触发事件/请求 → 状态更新 → 界面自动刷新。

5.1.2 4.1.2 一个组件长什么样（本项目真实例子）

```
<!-- frontend/fw100/src/views/fw100/Panel.vue (138 行，全项目最小页面) -->
<script setup lang="ts">
import { ref, onMounted } from 'vue'
import { useAuthStore } from '@stores/auth'
import { fw100Api } from '@api'
import type { LedgerItem } from '@shared'

const authStore = useAuthStore()
const items = ref<LedgerItem[]>([]) // 响应式数组
const loading = ref(false)

const fetchItems = async () => {
  loading.value = true
  try {
    const response = await fw100Api.getItems()
    if (response.success && response.data) {
      items.value = response.data
    }
  } finally {
    loading.value = false
  }
}

onMounted(fetchItems) // 挂载后请求数据
</script>

<template>
<div class="panel">
  <h2>fw100 设备台账</h2>
  <el-table v-loading="loading" :data="items" stripe>
    <el-table-column prop="name" label="名称" />
    <el-table-column prop="category" label="类别" />
    <el-table-column prop="status" label="状态" />
    <el-table-column prop="location" label="位置" />
  </el-table>
</div>
</template>
```

```

    </div>
  </template>

```

这就是一个完整组件：逻辑在 `<script setup>`，界面在 `<template>`。别看它小，它包含了 Vue 3 的 90% 核心语法：ref、onMounted、模板指令（v-loading/:data/v-for）、Pinia store、API 调用。

5.2 4.2 模板语法 (template)

5.2.1 4.2.1 插值与指令速查

```

<!-- 插值: {{ }} 内是 JS 表达式 -->
<span>{{ user?.username }}</span>
<span>{{ items.length }} 条记录</span>

<!-- 指令 (v- 开头): -->
<el-input v-model="form.email" />          <!-- v-model: 双向绑定 (表单) -->
<el-table :data="items" />                <!-- :prop: 绑定属性 (:data = v-bind:data) -->
<button @click="handleLogin">登录</button> <!-- @click: 事件绑定 (@click = v-on:click) -->
<div v-if="loading">加载中</div>          <!-- v-if: 条件渲染 -->
<div v-show="visible">显示</div>          <!-- v-show: CSS 显示/隐藏 -->
<li v-for="item in items" :key="item.id"> <!-- v-for: 列表渲染 (必须 :key) -->
<span v-html="content" />                 <!-- v-html: 渲染 HTML (慎用 XSS) -->
<button :disabled="!hasPerm">保存</button> <!-- :disabled: 布尔绑定 -->

```

本项目常用指令清单 (Element Plus 组件 + Vue 指令组合):

指令/绑定	项目例子	位置
v-model	登录表单、配置编辑器、搜索框	LoginPage.vue、Config.vue
v-for + :key	菜单遍历、卡片遍历、表格行	AppNavbar.vue、ftj1c Monitor
v-if / v-else	登录页/主页面切换、空态	App.vue、各视图
:loading	表格加载态	Users.vue、Panel.vue
@click	按钮事件	所有页面
@command	下拉菜单选择	AppNavbar.vue
:style / :class	动态样式	仪表盘、主题切换
@submit.prevent	表单提交拦截	LoginPage.vue

5.2.2 4.2.2 模板里的语法糖

```

<!-- 简写对照 -->
<el-input v-bind:model-value="x" />    <!-- 完整 -->
<el-input :model-value="x" />          <!-- 简写 (项目统一用简写) -->
<button v-on:click="f">x</button>      <!-- 完整 -->
<button @click="f">x</button>          <!-- 简写 -->

<!-- 动态组件 -->
<component :is="currentView" />

<!-- 插槽 (slot): 父组件向子组件注入内容 -->
<AppNavbar>
  <template #actions>                  <!-- 具名插槽: nav-actions 区域 -->
    <el-button @click="save">保存数据</el-button>
  </template>
</AppNavbar>

```

插槽是 AppNavbar 的应用扩展点：<slot name="actions"> 让每个应用在导航栏右侧放自定义按钮（fj200c_main 的“保存数据/模拟运行/主题”按钮就是这么放进去的）。

5.3 4.3 响应式核心：ref / reactive / computed / watch

5.3.1 4.3.1 ref：单个值（最常用）

```
import { ref } from 'vue'

const loading = ref(false)      // 创建响应式值
loading.value = true           // ★ 读取/修改必须 .value（模板中自动解包）
console.log(loading.value)
```

为什么有 .value：ref 把值包进一个对象（.value 属性），这样 JS 的基础类型（number/string/bool）也能被 Vue 追踪。在模板里不用写 .value（自动解包），在 <script setup> 里必须写。

```
// 项目实例：ftj1c Monitor.vue 的服务状态
const serviceRunning = ref(false)
const startService = async () => {
  const res = await ftj1cApi.startService()
  serviceRunning.value = res.success
}
```

5.3.2 4.3.2 reactive：对象（整体响应式）

```
import { reactive } from 'vue'

const form = reactive({          // 对象字段都是响应式的
  email: '',
  password: '',
})
form.email = 'admin@rustweb.dev' // 直接改字段，无需 .value
```

5.3.3 4.3.3 ref vs reactive 怎么选（项目约定）

场景	用
单个值（数字/布尔/字符串）	ref
数组	ref（项目统一 ref<Type[]>([])）
表单对象（字段级操作）	reactive
复杂嵌套对象	reactive

项目铁律：数组一律 ref<Type[]>([])（不要 reactive([])——历史坑）。看 fj200c_main 的 dashboard store：

```
// frontend/fj200c_main/src/fj200c_main/store/dashboard.ts
const ecuData = reactive<EcuFields>({ /* 29 个字段 */ }) // 对象 → reactive
const chartData = ref<Array<{time: string; value: number}>>([]) // 数组 → ref
const isSimulating = ref(false) // 布尔 → ref
```

5.3.4 4.3.4 computed：派生状态

```
import { computed } from 'vue'

// 依赖其他响应式值自动重算（缓存）
const hasUsers = computed(() => items.value.length > 0)
const displayName = computed(() => user.value?.username || '游客')

// 项目实例：App.vue 判断登录页（computed 读取 route）
const isLoginPage = computed(() => route.path.startsWith('/login'))
```


computed vs 函数: computed 有缓存 (依赖不变不重算); 普通函数每次调用都执行。模板里频繁读取的派生值用 computed。

5.3.5 4.3.5 watch: 监听变化

```
import { watch } from 'vue'

watch(serviceRunning, (val) => { // 监听 ref
  console.log('服务状态变为', val)
})

watch([a, b], ([na, nb]) => {}) // 监听多个
watch(() => route.path, () => { // 监听 getter 表达式
  // 路由变化时做什么
}, { immediate: true }) // 立即执行一次
```

5.3.6 4.3.6 模板中直接可用的全局对象

```
<span>{{ import.meta.env.PROD ? '/fj200c_information' : '/' }}</span>
```

import.meta.env 是 Vite 注入的环境对象: DEV/PROD/MODE/自定义 VITE_* 变量。

5.4 4.4 <script setup> 详解 (本项目的统一写法)

5.4.1 4.4.1 为什么用 script setup

Vue 3 的组合式 API 有两种载体: setup() 函数和 <script setup> (编译器语法糖)。本项目 100% 用 <script setup>——顶层声明的变量/函数自动暴露给模板, 不用 return。

5.4.2 4.4.2 组件导入即用

```
<script setup lang="ts">
import CommandPanel from '@fj200c_information/components/CommandPanel.vue'
// 导入的组件在模板中直接可用 (无需注册)
</script>
<template>
  <CommandPanel />
</template>
```

5.4.3 4.4.3 props 与 emits (组件通信)

```
<!-- frontend/fj200c_information/src/fj200c_information/components/CommandRow.vue (结构示意) -->
<script setup lang="ts">
// props: 父组件传入的数据 (只读)
const props = withDefaults(defineProps<{
  index: number
  label?: string // 可选 prop
}>(), { label: '命令' }) // 默认值

// emits: 向父组件发送事件
const emit = defineEmits<{
  (e: 'remove', index: number): void
  (e: 'send', index: number, hex: string): void
}>()

const handleSend = () => emit('send', props.index, hexString.value)
</script>
```

本项目 props/emits 类型化写法: defineProps<{...}>() 泛型 + withDefaults 默认值 + defineEmits<{...}>() 事件签名——全程 TypeScript 类型检查, 写错类型直接编译报错。

5.4.4 4.4.4 生命周期钩子

```
import { onMounted, onUnmounted, onBeforeUnmount } from 'vue'

onMounted(() => {          // 组件挂载后（请求数据、建立连接）
  authStore.initAuth()
  connectWs()
})
onUnmounted(() => {        // 组件卸载前（清理）
  disconnectWs()
  clearInterval(timer)
})
```

项目中的生命周期使用场景：

钩子	项目用途	位置
onMounted	拉数据、建 WS、起轮询	所有视图
onUnmounted	断 WS、清定时器	useFj200cInformationEvents 等
onScopeDispose	组合式函数内清理（推荐替代 onUnmounted）	useCityData.ts

5.5 4.5 TypeScript 速成（本项目用法）

5.5.1 4.5.1 类型注解基础

```
const email: string = 'admin@rustweb.dev'
const count: number = 42
const isDark: boolean = true
const items: LedgerItem[] = []           // 类型来自 orval 生成

function fetchItems(): Promise<ApiResponse<LedgerItem[]>> { ... }
```

5.5.2 4.5.2 接口与类型别名

```
// 接口：对象的形状
interface MenuItem {
  id: string
  title: string
  path: string
  icon?: string           // 可选字段
  permissions: Permission[] // 权限点列表
  children?: MenuItem[]   // 子菜单（递归）
}

// 类型别名：联合类型/复杂类型
type Fj200cMainWsEvent =
  | { type: 'port_data'; ... }
  | { type: 'simulation_state'; ... }
```

5.5.3 4.5.3 联合类型与类型收窄（WS 事件分发核心）

```
// 后端枚举（serde tag）生成的联合类型：按 type 字段分发
switch (event.type) {
  case 'port_data':      // 收窄：这里 event 自动变成 port_data 的形状
    handlePortData(event)
    break
  case 'simulation_state':
    store.isSimulating = event.simulating
    break
  case 'csv_recording_state':
```

```

    store.isRecording = event.recording
    break
}

```

判别联合 (discriminated union) 是前后端事件协议在 TS 侧的体现——后端
#[serde(tag="type")] 枚举 ↔ 前端 TS 联合类型，天然对应。

5.5.4 4.5.4 泛型与工具类型

```

// 泛型函数: T 由调用方决定
function wrap<T>(data: T): ApiResponse<T> { ... }

// 内置工具类型
type UserId = string
type MaybeUser = User | null
type UserFields = keyof User           // 字段名联合
type PartialUser = Partial<User>       // 全部可选
type ReadonlyUser = Readonly<User>     // 全部只读

```

5.5.5 4.5.5 项目类型体系 (三层 re-export)

```

flowchart TD
    A[packages/shared/src/api/generated/model/*.ts<br/>orval 生成(唯一事实源)] --> B[packages/shared/src/types.ts<br/>re-export 精选类型]
    B --> C[各应用 src/types/index.ts<br/>export * from '@shared/types']
    C --> D[视图层 import type]

```

新手规则：类型从哪里来？→ 一律从 @shared 或 @shared/api/generated 导入；**绝不手写**与后端重复的类型定义。

5.5.6 4.5.6 项目 tsconfig 严格模式 (写代码时注意)

```

{
  "compilerOptions": {
    "strict": true,           // 全严格: null 检查等
    "noUnusedLocals": true,  // 未使用变量报错
    "noUnusedParameters": true, // 未使用参数报错
    "noEmit": true,          // 只检查不输出
    "moduleResolution": "bundler",
    "paths": {
      "@/*": ["src/*"],
      "@shared": ["../../packages/shared/src/index.ts"],
      "@shared/*": ["../../packages/shared/src/*"]
    }
  }
}

```

新手最常见的三个报错：1. 声明了没用的变量 → 删掉 (noUnusedLocals)。2. strict 下 null 处理: user?.name 或 user!.name (非空断言, 慎用)。3. import 路径写错 → 用 @/ 和 @shared 别名。

5.6 4.6 Pinia 状态管理 (本项目用法)

5.6.1 4.6.1 什么是 Pinia

Pinia 是 Vue 3 官方状态管理库：跨组件共享的状态 (用户信息、权限、业务数据)。

5.6.2 4.6.2 创建 store 的两种风格

```

// 选项式 (Vuex 风格)
export const useStore = defineStore('id', {
  state: () => ({ count: 0 }),
  getters: { double: (s) => s.count * 2 },

```

```

    actions: { inc() { this.count++ } },
  })

```

// 组合式（setup 风格）——本项目用这个

```

export const useDashboardStore = defineStore('fj200c_main-dashboard', () => {
  const ecuData = reactive<EcuFields>({ ... })
  const isSimulating = ref(false)
  const dashboardState = computed(() => ({ ... }))
  function addChartPoint() { ... }
  return { ecuData, isSimulating, dashboardState, addChartPoint } // 暴露出去
})

```

5.6.3 4.6.3 组件中使用 store

```

import { useDashboardStore } from '@fj200c_main/store/dashboard'
const store = useDashboardStore() // 必须在 setup 顶层调用
console.log(store.ecuData.ngSpeed)
store.addChartPoint()

```

5.6.4 4.6.4 store-to-refs: 解构保持响应式

```

import { storeToRefs } from 'pinia'
const { ecuData, isSimulating } = storeToRefs(store) // refs 解构
const { addChartPoint } = store // 方法直接解构

```

重要：直接 `const { ecuData } = store` 会丢失响应式（拿的是快照）；用 `storeToRefs` 包装。

5.6.5 4.6.5 本项目的 store 家族

store	应用	内容
useAuthStore（工厂生成）	所有应用	用户/权限/菜单/登录退出 (19 行配置)
useDashboardStore	fj200c_main	ECU/ADAM/DYNO 数据 + 图表缓冲

auth store 是 `createAuthStore` 工厂生成的（05 章详述），业务 store 只有 dashboard 一个——本项目状态管理很轻，大部分状态就在组件内。

5.7 4.7 Vue Router（路由与守卫）

5.7.1 4.7.1 路由表

```

// frontend/fj200c_information/src/router/index.ts
import { createRouter, createWebHistory } from 'vue-router'

const router = createRouter({
  history: createWebHistory(import.meta.env.PROD ? '/fj200c_information/' : '/'),
  routes: [
    { path: '/', redirect: '/login' },
    { path: '/login', name: 'Login',
      component: () => import('@views/Login.vue'),
      meta: { requiresGuest: true } }, // 游客才能进
    { path: '/fj200c_information/monitor', name: 'Monitor',
      component: () => import('@views/fj200c_information/Monitor.vue'),
      meta: { requiresAuth: true, // 需要登录
        permissions: [Permission.Fj200cInformationMonitor] } }, // 需要权限
    // ... 其余页面
    { path: '/*', redirect: '/login' }, // 兜底
  ],
})

```

meta 字段是路由权限契约：requiresAuth（要登录）+ permissions（权限点数组，任一满足即可）。

5.7.2 4.7.2 路由守卫 (beforeEach)

```
router.beforeEach(async (to, _from, next) => {
  const authStore = useAuthStore()
  await authStore.initAuth() // ① 确保认证状态就绪
  if (to.meta.requiresAuth && !authStore.isAuthenticated) {
    next('/login') // ② 未登录 → 登录页
    return
  }
  if (to.meta.requiresGuest && authStore.isAuthenticated) {
    next('/fj200c_information') // ③ 已登录访问登录页 → 首页
    return
  }
  if (to.meta.permissions) {
    const has = (to.meta.permissions as Permission[])
      .some((p) => authStore.hasPermission(p)) // ④ 任一权限满足即放行
    if (!has) {
      const fallback = getFirstMenuPath(authStore.userRole)
      next(fallback ?? '/login') // ⑤ 无权限 → 跳有权限的首页
      return
    }
  }
  next() // ⑥ 放行
})
```

守卫四步口诀：等认证 → 查登录 → 查权限 → 跳转。这是 7 个应用共用的守卫模板（admin 版差异：无权限跳 /403 而非回跳首页）。

5.7.3 4.7.3 动态导入 (懒加载)

```
component: () => import('@/views/xxx.vue')
```

页面按需加载，首屏更快——本项目所有页面都是动态导入。

5.8 4.8 Element Plus 组件库速查 (本项目高频组件)

5.8.1 4.8.1 布局与容器

```
<el-card class="login-card"> <!-- 卡片 -->
  <template #header>标题</template> <!-- 卡片头部插槽 -->
  内容
</el-card>

<el-container> <!-- 布局容器 -->
  <el-header>顶部</el-header>
  <el-main>主体</el-main>
</el-container>
```

5.8.2 4.8.2 表单 (登录页标准组合)

```
<el-form ref="formRef" :model="form" :rules="rules" label-position="top"
@submit.prevent="handleLogin">
  <el-form-item label="邮箱" prop="email">
    <el-input v-model="form.email" placeholder="请输入邮箱" />
  </el-form-item>
  <el-form-item label="密码" prop="password">
    <el-input v-model="form.password" type="password" show-password placeholder="请输入密码" />
  </el-form-item>
```

```

    <el-button type="primary" :loading="loading" @click="handleLogin">立即登录</el-button>
  </el-form>

  // 表单校验规则 (Element Plus rules)
  const rules = {
    email: [{ required: true, message: '请输入邮箱', trigger: 'blur' }],
    password: [{ required: true, message: '请输入密码', trigger: 'blur' }],
  }
  // 校验提交
  formRef.value?.validate(async (valid) => {
    if (!valid) return
    // ...
  })

```

5.8.3 4.8.3 表格 (列表页标准组合)

```

<el-table v-loading="loading" :data="users" stripe border>
  <el-table-column prop="username" label="用户名" />
  <el-table-column prop="email" label="邮箱" />
  <el-table-column label="角色">
    <template #default="{ row }"> <!-- 作用域插槽: 拿当前行 -->
      {{ findRole(row.role)?.name ?? row.role }}
    </template>
  </el-table-column>
  <el-table-column label="操作" width="180">
    <template #default="{ row }">
      <el-button size="small" @click="openEdit(row)">编辑</el-button>
      <el-button size="small" type="danger" @click="removeUser(row)">删除</el-button>
    </template>
  </el-table-column>
</el-table>

```

5.8.4 4.8.4 弹窗与消息

```

<el-dialog v-model="dialogVisible" title="编辑角色" width="400px">
  <el-select v-model="editForm.role">
    <el-option v-for="r in roles" :key="r.key" :label="r.name" :value="r.key" />
  </el-select>
  <template #footer>
    <el-button @click="dialogVisible = false">取消</el-button>
    <el-button type="primary" @click="saveEdit">确定</el-button>
  </template>
</el-dialog>

import { ElMessage } from 'element-plus'
ElMessage.success('登录成功')
ElMessage.error(response.message || '操作失败')
ElMessage.warning('该账号属于其他应用, 正在跳转')

```

5.8.5 4.8.5 下拉菜单 (导航栏用户区)

```

<el-dropdown @command="handleCommand">
  <el-avatar :size="32">{{ user?.username?.charAt(0)?.toUpperCase() }}</el-avatar>
  <template #dropdown>
    <el-dropdown-item divided command="logout">退出登录</el-dropdown-item>
  </template>
</el-dropdown>

```

5.8.6 4.8.6 本项目 Element Plus 组件使用统计（按频率）

组件	频率	用途
el-button / el-input	★★★★★	表单交互
el-table / el-table-column	★★★★★	数据展示
el-form / el-form-item	★★★★	表单校验
el-card	★★★★	内容容器
el-dialog	★★★	编辑弹窗
el-select / el-option	★★★	下拉选择
el-tag / el-badge	★★★	状态标签
el-dropdown	★★	菜单
el-avatar	★★	用户头像
el-switch / el-radio / el-checkbox	★	开关/单选/复选

5.9 4.9 WebSocket 前端连接（项目两种模式）

5.9.1 4.9.1 连接地址构建（shared 公共函数）

```
// packages/shared/src/session.ts
export function buildWebSocketUrl(apiPath: string): string {
  const token = getSessionToken() || ''
  const protocol = window.location.protocol === 'https:' ? 'wss' : 'ws'
  return `${protocol}://${window.location.host}${apiPath}?token=${encodeURIComponent(token)}`
}
```

- 协议按页面协议自动选 wss/ws（部署 HTTPS 时自动安全）。
- token 走查询参数（浏览器 WS API 无法自定义 header）。

5.9.2 4.9.2 模式一：组件级连接（fj200c_information / ftj1c）

```
// frontend/fj200c_information/src/fj200c_information/composables/
useFj200cInformationEvents.ts
const ws = ref<WebSocket | null>(null)
let reconnectTimer: number | null = null
let manualClose = false

const connect = () => {
  if (ws.value || connecting.value) return
  manualClose = false
  ws.value = new WebSocket(fj200cInformationApi.buildWebSocketUrl())

  ws.value.onopen = () => { connected.value = true }
  ws.value.onmessage = (message) => {
    try {
      const data = JSON.parse(message.data) as Fj200cInformationEvent
      handleEvent(data) // switch(event.type) 分发
    } catch { /* 忽略非 JSON */ }
  }
  ws.value.onclose = () => {
    connected.value = false
    if (!manualClose) {
      reconnectTimer = window.setTimeout(connect, 1500) // ★ 1.5s 自动重连
    }
  }
  ws.value.onerror = () => ws.value?.close()
}
```

```

}

const disconnect = () => {
  manualClose = true
  if (reconnectTimer) clearTimeout(reconnectTimer)
  ws.value?.close()
  ws.value = null
}

```

组件级连接的生命周期：onMounted(connect) + onUnmounted(disconnect)——离开页面断开。适合“单页面使用 WS”的应用。

5.9.3 4.9.3 模式二：模块级单例连接 (fj200c_main)

```

// frontend/fj200c_main/src/fj200c_main/composables/useBackendPorts.ts
// 模块级变量（不随组件销毁）
let sharedWs: WebSocket | null = null
let refCount = 0 // 引用计数

export function useBackendPorts() {
  // 页面挂载时 acquire: 计数 +1 并确保连接
  const acquire = () => { refCount++; manualClose = false; connect() }
  // 页面卸载时 release: 计数 -1, 归零才真正断开
  const release = () => {
    refCount = Math.max(0, refCount - 1)
    if (refCount > 0) return
    manualClose = true; clearTimeout(reconnectTimer); sharedWs?.close(); sharedWs = null
  }
  onMounted(acquire)
  onUnmounted(release)
  return { /* 数据和事件 */ }
}

```

为什么 fj200c_main 用单例：仪表盘 Monitor 页 + 试验查看 ExperimentView 页都要收数据，组件级连接会导致**切页断线、数据冻结**（git 历史里的真实 bug，debb02f 修复）。引用计数让多个页面共享一个连接，最后离开的才断开。

5.9.4 4.9.4 两种模式选择建议

场景	模式
只有一个页面用 WS	组件级（简单）
多个页面都要实时数据	模块级单例 + 引用计数
全应用都要（含未打开任何页面时也要收）	App.vue 挂载时建立

5.9.5 4.9.5 消息分发模式 (handleEvent)

```

const handleEvent = (event: Fj200cMainWsEvent) => {
  switch (event.type) {
    case 'port_data': handlePortData(event); break
    case 'simulation_state': store.isSimulating = event.simulating; break
    case 'theme_state': applyTheme(event.isDark); break
    case 'csv_recording_state': store.isRecording = event.recording; break
  }
}

```

一个原则：WS 事件只做“写 store / 更新 ref”，不做 UI 直接操作——渲染交给模板自动响应。

5.10 4.10 ECharts 可视化 (fj200c_information / fj200c_main)

5.10.1 4.10.1 基本用法

```
// frontend/fj200c_information/src/views/fj200c_information/Visual.vue (结构示意图)
import * as echarts from 'echarts'
import { onMounted, onUnmounted, ref } from 'vue'

const chartRef = ref<HTMLDivElement | null>(null)
let chart: echarts.ECharts | null = null

onMounted(() => {
  chart = echarts.init(chartRef.value!) // 初始化 (绑定 DOM)
  chart.setOption({
    series: [{
      type: 'gauge', // 仪表盘
      data: [{ value: 70, name: '转速' }],
    }],
  })
})

onUnmounted(() => { chart?.dispose() }) // 必须销毁 (内存泄漏预防)

// 数据更新: setOption 增量合并 (不要每次重建图表)
const updateData = (value: number) => {
  chart?.setOption({ series: [{ data: [{ value }] }] })
}
```

5.10.2 4.10.2 实时曲线的数据流 (与 WS 协作)

```
// 环形缓冲: 最多 100 个点
const chartData = ref<Array<{ time: string; value: number }>>([])
const addPoint = (v: number) => {
  chartData.value.push({ time: new Date().toLocaleTimeString(), value: v })
  if (chartData.value.length > 100) chartData.value.shift() // 超长截断
  chart?.setOption({ xAxis: { data: chartData.value.map(p => p.time) }, ... })
}
```

5.10.3 4.10.3 图表类型清单 (本项目)

图表	用途	位置
gauge (仪表盘)	转速/扭矩等 6 个仪表	Visual.vue
line (折线)	实时曲线	Visual.vue、ChartPanel.vue
bar (柱状)	状态统计 (少量)	报表页

新手注意: ECharts 实例要随组件销毁 `dispose()`; `setOption` 是增量合并, 重复调用安全; WebSocket 高频数据要节流再 `setOption`。

5.11 4.11 组合式函数 (composables): 本项目逻辑复用的核心

5.11.1 4.11.1 什么是组合式函数

组合式函数 (composable) 是以 `useXXX` 命名的函数, 内部可组合 `ref/computed/watch/生命周期`, 把可复用逻辑抽出来。类似 React 的 Hooks。

5.11.2 4.11.2 项目中的组合式函数清单

函数	位置	职责
useClock	fj200c_information	每秒更新的时钟
useService	fj200c_information	服务启停 + 3 秒轮询状态
useCommandChannel	fj200c_information	命令通道状态与发送
useConfigDialog	多个应用	配置读写对话框逻辑
useFj200cInformationEvents	fj200c_information	WS 连接与事件分发
useBackendPorts	fj200c_main	模块级单例 WS
useTheme	fj200c_main	深浅主题切换
useWindowScale	fj200c_main	窗口缩放计算
useCityData	city3d	数据加载 + 5 秒轮询
useCityScene	city3d	Three.js 场景
useResponsive	shared 相关	响应式布局

5.11.3 4.11.3 组合式函数范例 (useClock, 完整版)

```
// frontend/fj200c_information/src/fj200c_information/composables/useClock.ts
```

```
import { onUnmounted, ref } from 'vue'

export function useClock() {
  const now = ref(new Date())
  let timer: number | null = null

  timer = window.setInterval(() => {
    now.value = new Date()
  }, 1000) // 每秒更新

  onUnmounted(() => {
    if (timer) clearInterval(timer) // ★ 清理定时器（防泄漏）
  })

  return {
    now,
    timeStr: () => now.value.toLocaleTimeString('zh-CN'),
  }
}
```

组合式函数三定律 (项目全部遵守): 1. 命名 useXxx。2. 内部创建的资源 (定时器/WS/监听器) 在 onUnmounted/onScopeDispose 清理。3. 返回响应式数据 + 方法。

5.11.4 4.11.4 一个视图组装多个组合式函数 (Monitor.vue 模式)

```
// Monitor.vue 的 script 组织 (411 行页面, 逻辑高度复用)
```

```
const { now } = useClock()
const { serviceRunning, startService, stopService } = useService()
const { channels, addChannel, removeChannel, send } = useCommandChannel()
const { configDialog, openConfig, saveConfig } = useConfigDialog()
const { connected, rows } = useFj200cInformationEvents()
```

这就是组合式 API 的威力: 页面只是“组装器”, 每个关注点一个组合式函数, 测试/复用/维护都容易。

5.12 4.12 样式系统：CSS 变量与双主题

5.12.1 4.12.1 全局样式组织

```
/* frontend/fj200c_main/src/fj200c_main/styles/theme.css — 双主题变量 */
:root {
  --bg-primary: #0f1d33;          /* 深色主题底色 */
  --bg-card: #1a2940;
  --text-primary: #e5eaf3;
  --border-color: #303133;
}
html.light {
  --bg-primary: #f5f7fa;          /* 浅色主题覆盖 */
  --bg-card: #ffffff;
  --text-primary: #303133;
  --border-color: #dcdfe6;
}

// useTheme.ts: html 根节点加 class 控制主题
const applyTheme = (isDark: boolean) => {
  document.documentElement.classList.toggle('light', !isDark)
  localStorage.setItem('theme', isDark ? 'dark' : 'light')
}
```

主题机制：CSS 变量 + html.light 类 + 服务端同步 (WS theme_state 广播) ——所有页面统一切换。

5.12.2 4.12.2 各应用样式文件布局

文件	内容
src/style.css	全局基础样式 (每个应用都有)
src/fj200c_information/fj200c_information.css	模块专属样式
src/fj200c_main/styles/theme.css	双主题变量
src/fj200c_main/print-lock.css	打印样式

5.12.3 4.12.3 Scoped 样式与 :deep

```
<style scoped>
/* scoped: 样式只作用于本组件 (自动加属性选择器) */
.monitor-grid { display: grid; gap: 12px; }

/* :deep(): 穿透到子组件内部 (改 Element Plus 内部样式) */
:deep(.el-table__row) { cursor: pointer; }
</style>
```

5.13 4.13 前端工程：Vite 开发体验

5.13.1 4.13.1 dev 服务器做了什么

flowchart LR

```

  B[浏览器 localhost:5173] -->|1 页面请求| V[Vite dev server]
  V -->|2 模块编译/转换| B
  B -->|3 /api 请求| P[Vite proxy]
  P -->|4 转发| A[Axum :3000]
  B -->|5 WS 连接| P2[Vite proxy ws:true]
  P2 -->|6 转发| A
```

- Vite 启动后，浏览器访问即时的模块服务 (无需构建)。
- /api 代理: vite.config.ts 的 server.proxy 把请求转给后端 (changeOrigin 改 Host 头)。

- **WS 代理**: `ws: true` 让 WS 升级请求也被转发——`fj200c_information/fj200c_main/ftj1c/city3d` 的配置里有。

5.13.2 4.13.2 HMR (热更新)

改 `.vue` 文件 → 浏览器**不刷新**地更新组件 (保留状态); 改 `vite.config.ts` → 需要重启 dev server。

5.13.3 4.13.3 构建 (npm run build)

在对应 `frontend/<app>` 目录执行

`npm run build`

= `vue-tsc --noEmit` (类型检查) && `vite build` (产物到 `dist/`)

两步必须顺序执行: 先类型检查 (报错就不构建), 再打包。产物 `dist/` 供后端内嵌/磁盘托管。

5.14 4.14 本章自测: 读一段真实代码

独立阅读 `frontend/fj200c_information/src/views/fj200c_information/Config.vue` 的核心片段, 回答:

```
const configContent = ref('')
const loading = ref(false)

const loadConfig = async () => {
  loading.value = true
  try {
    const res = await fj200cInformationApi.getConfig()
    if (res.success) configContent.value = res.data?.content ?? ''
    else ElMessage.error(res.message || '读取失败')
  } finally {
    loading.value = false
  }
}

onMounted(loadConfig)
```

问题: 1. `configContent` 是什么类型? `ref` 包的是什么? 2. `res.data?.content ?? ''` 的含义? 3. `finally` 在这里的作用? 4. 为什么 `onMounted(loadConfig)` 而不是 `onMounted(loadConfig())`?

参考答案: 1. `ref('')` → `Ref<string>`, 模板中自动解包为字符串。2. 可选链: `res.data` 可能为 `null` (失败时 `data` 为 `null`), 取不到 `content` 就用空字符串兜底 (`??` 空值合并)。3. 无论成功失败都恢复 `loading` 状态 (防止按钮/表格永远转圈)。4. `onMounted(loadConfig)` 传入**函数引用**, 挂载时调用; 如果写 `loadConfig()` 会立即执行 (且返回值 `undefined` 传给 `onMounted`, 挂载时不执行)。

答对 3 题以上, 说明 Vue 3 基础已经足够阅读本项目页面代码。继续 05 章——前端逐应用精读。

5.15 4.15 API 调用模式深入 (本目前端请求全景)

5.15.1 4.15.1 一次请求的完整生命周期

sequenceDiagram

```
participant C as 组件 (Views)
participant F as facade (api/xxx.ts)
participant G as orval generated 函数
participant I as customInstance (shared)
participant A as axios 实例 (token 注入)
participant B as 后端 Axum
```

```

C-->F: fw100Api.getItems()
F-->G: api.fw100ListItems()
G-->I: customInstance({url, method})
I-->A: instance({...}) (合并 baseUrl /api)
A-->A: 请求拦截器: 加 Authorization: Bearer token
A-->B: GET /api/fw100/items
B-->A: {success, message, data}
A-->A: 响应拦截器: 401 → 请会话跳登录页
A-->I: axios response
I-->G: .then(({data}) => data) (解出 ApiResponse)
G-->F: ApiResponse<LedgerItem[]>
F-->C: response.success ? response.data : ...

```

五层调用链是理解前端 API 的关键——组件不直接调 axios，全部走 orval generated 封装，保证类型安全与统一错误处理。

5.15.2 4.15.2 响应处理三式（项目统一的写法）

```

// 第一式：成功才继续
const res = await usersApi.getUsers()
if (res.success && res.data) {
  users.value = res.data
} else {
  ElMessage.error(res.message || '获取失败')
}

// 第二式：try/catch/finally（加载态）
const fetchData = async () => {
  loading.value = true
  try {
    const res = await xxxApi.list()
    if (res.success) list.value = res.data ?? []
  } catch (e) {
    ElMessage.error('网络错误')
  } finally {
    loading.value = false
  }
}

// 第三式：抛错处理（提交类操作）
const handleSave = async () => {
  const res = await xxxApi.save(content)
  if (!res.success) throw new Error(res.message)
  ElMessage.success('保存成功')
}

```

5.15.3 4.15.3 401 的全局处理（token 过期）

```

// packages/shared/src/api/index.ts
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      clearSession()
      window.location.href = loginPath // 直接跳登录页（用 location 而非 router）
    }
    return Promise.reject(error)
  }
)

```

为什么用 window.location.href 而不是 router.push：401 可能发生在任何组件里，且可能不在 Vue 上下文（拦截器是全局的）；整页跳转最可靠。

5.15.4 4.15.4 facade 层的作用（为什么多包一层）

```
// frontend/fw100/src/api/fw100.ts
import { getFw100 } from '@shared/api/generated'

export function createFw100Api() {
  const api = getFw100() // orval 工厂
  return {
    async getItems() { return api.fw100ListItems() },
    // 以后加逻辑就在这里加（日志/转换/组合请求）
  }
}
```

facade 的价值：1. 视图层解耦：组件 import @/api，不直接碰 generated——generated 重新生成（函数名可能变）时只改 facade。2. 可加逻辑：日志、参数转换、多请求组合。3. 类型收口：export type XxxApi = ReturnType<typeof createXxxApi> 导出统一类型。

5.16 4.16 Vue Router 深入：项目特殊用法

5.16.1 4.16.1 两个 base 的奥妙（dev vs prod）

```
// 路由 history 的 base:
createWebHistory(import.meta.env.PROD ? '/fj200c_information/' : '/')
// vite.config.ts 的 base:
base: command === 'build' ? '/fj200c_information/' : '/'
```

环境	路由 base	资源 base	效果
dev	/	/	5173 端口根路径访问
prod	/fj200c_information/	/fj200c_information/	后端托管在 /fj200c_information 下

两个 base 必须一致——这是 SPA 子路径部署的标准配置，7 个应用都遵守。

5.16.2 4.16.2 路由 meta 的权限设计（回看 4.7.1）

```
meta: {
  requiresAuth: true, // 需要登录
  permissions: [Permission.Fj200cInformationMonitor], // 需要权限（任一）
}
```

权限判定函数（shared/auth store）：

```
const hasPermission = (p: Permission) => permissions.value.includes(p)
const hasAnyPermission = (ps: Permission[]) => ps.some((p) =>
permissions.value.includes(p))
const hasAllPermissions = (ps: Permission[]) => ps.every((p) =>
permissions.value.includes(p))
```

5.16.3 4.16.3 程式式导航

```
import { useRouter } from 'vue-router'
const router = useRouter()
router.push('/fj200c_information/monitor') // 跳转
router.replace('/login') // 替换（不留历史）
router.back() // 后退
```

5.17 4.17 表单与校验深入 (Element Plus 全流程)

5.17.1 4.17.1 动态规则 (根据场景切换校验)

// frontend/admin/src/views/CreateUser.vue (结构示意图)

```
const rules = computed(() => ({
  password: [
    { required: true, message: '请输入密码', trigger: 'blur' },
    { min: 6, max: 32, message: '密码长度 6-32 位', trigger: 'blur' },
  ],
  email: [
    { required: true, message: '请输入邮箱', trigger: 'blur' },
    { type: 'email', message: '邮箱格式不正确', trigger: 'blur' },
  ],
}))
```

5.17.2 4.17.2 校验方法

```
const formRef = ref<FormInstance>()
const valid = await formRef.value?.validate().catch(() => false) // 校验全部
formRef.value?.validateField('email') // 校验单字段
formRef.value?.resetFields() // 重置
formRef.value?.clearValidate() // 清除校验状态
```

5.17.3 4.17.3 自定义校验

```
const validateRole = (rule: unknown, value: string, callback: (err?: Error) => void) => {
  if (!isRegisteredRole(value)) callback(new Error('角色不存在'))
  else callback()
}
```

5.18 4.18 前端调试技巧 (新手必会)

5.18.1 4.18.1 Vue DevTools (浏览器扩展)

- **组件树**: 查看组件层级、props、当前状态。
- **Pinia 面板**: 直接查看/修改 store 状态 (调试 WS 数据流神器)。
- **时间旅行**: Vuex 才有, Pinia 无 (不是缺陷)。

5.18.2 4.18.2 F12 Network 面板

标签	看什么
Fetch/XHR	API 请求: URL、状态码、请求头 (token)、响应体
WS	WebSocket 连接: 消息流 (实时数据调试核心)
Console	报错、console.log

调试 WS 数据流三步: ① Network → WS → 打开连接查看帧 → ② 对照后端 RUST_LOG=debug 日志 → ③ 定位断在哪一层 (后端采集/广播/WS/前端分发)。

5.18.3 4.18.3 常见前端报错速查

报错	原因	修法
Cannot read properties of undefined (reading 'xxx')	空值访问	可选链 ?. 或 ?? '' 兜底
[Vue warn]: Extraneous non-emits event listeners	事件未声明 emits	defineEmits 加事件
TypeError: xxx is not a function	引用错误	检查导入名/facade 方法名
401 Unauthorized	token 失效/未带	检查会话、重新登录
500 Internal Server Error	后端异常	看后端日志
403 Forbidden	权限不足	换有权限的账号
ERR_CONNECTION_REFUSED (/api 请求)	后端没启动	启动 cargo run
WebSocket connection failed	后端没启动 / WS 代理没配	检查后端 + ws: true

5.18.4 4.18.4 项目内联调试手段

```
console.log('调试', response) // 临时调试
console.table(users.value)      // 表格化输出数组
```

5.19 4.19 项目前端代码风格约定（改代码时遵守）

5.19.1 4.19.1 命名约定

项	约定	例子
组件文件	PascalCase	CommandPanel.vue、GaugeCard.vue
视图文件	PascalCase	Monitor.vue、Data.vue
组合式函数	useXxx	useClock.ts
普通工具	camelCase	ascii.ts、hex.ts
变量/函数	camelCase	fetchItems、configContent
类型/接口	PascalCase	MenuItem、EcuFields
常量	SCREAMING_SNAKE	RECONNECT_DELAY
Store	useXxxStore	useAuthStore

5.19.2 4.19.2 文件组织约定

```
src/
├─ api/           # API facade（每个应用）
├─ router/        # 路由
├─ stores/        # 认证 store（工厂调用）
├─ views/         # 页面（薄）
├─ <模块名>/     # 业务子目录
│   ├── components/ # UI 组件
│   ├── composables/ # 组合式函数
│   ├── store/      # 业务 store（fj200c_main）
│   └── styles/     # 模块样式
└─ utils/        # 通用工具
```

5.19.3 4.19.3 代码纪律

1. 页面组件保持“薄”：逻辑尽量下沉到 composables。
2. 样式尽量 scoped；改 Element Plus 内部用 :deep()。

3. 所有 API 走 facade，不直接 import generated。
4. 所有类型从 @shared 导入，不手写后端类型的副本。
5. 删除的变量/导入必须清干净（noUnusedLocals 会报错）。

5.20 4.20 新手常见 Vue 坑（本项目语境）

#	坑	后果	正确做法
1	<code>const { user } = store</code> 解构	丢失响应式	<code>storeToRefs(store)</code>
2	忘记 onUnmounted 清理定时器/WS	内存泄漏、重复请求	清理所有资源
3	<code>onMounted(async () => await fetch())</code>	无影响但没必要	<code>onMounted(fetch)</code>
4	模板里写复杂逻辑	难读难测	抽 computed/函数
5	<code>v-for</code> 不用 <code>:key</code>	渲染错乱警告	用唯一 id
6	<code>@click="handleLogin()"</code> VS <code>@click="handleLogin"</code>	前者会传事件对象	无参数时写函数名
7	直接改 props	报错/警告	emit 事件让父组件改
8	<code>import { reactive } from 'vue'</code> 后 <code>reactive([])</code>	数组替换陷阱	<code>ref<Type[]>([])</code>
9	忽略 TS 报错继续写	构建失败	先修类型错误
10	在子目录单独 npm install	依赖双实例（黑屏 bug!）	根目录统一安装

坑 10 是真实事故：AGENTS.md 明确记载“子目录单独装依赖曾导致 pinia 双实例黑屏”。任何前端依赖变更，**在根目录执行 npm install。**

5.21 4.21 语法索引表（改代码时快速定位）

你想做的	语法	项目参考
响应式单个值	<code>const x = ref(0) + x.value</code>	所有页面
响应式对象	<code>reactive({...})</code>	dashboard store
派生状态	<code>computed(() => ...)</code>	App.vue isLoginPage
监听变化	<code>watch(src, cb)</code>	主题/路由变化
页面请求	<code>onMounted(fetch) + try/finally</code>	Panel.vue
表格渲染	<code>el-table :data="items" + prop</code>	Users.vue
表单绑定校验	<code>el-form :rules + validate</code>	LoginPage.vue
弹窗	<code>el-dialog v-model</code>	Users.vue
消息	<code>ElMessage.success/error</code>	所有页面
路由跳转	<code>useRouter().push(path)</code>	LoginPage
路由守卫	<code>router.beforeEach</code>	各 router/index.ts
store 读写	<code>useXxxStore() + storeToRefs</code>	视图层
WS 连接	<code>new WebSocket(url) + onmessage</code>	composables
定时器	<code>setInterval + onUnmounted 清理</code>	useClock
类型导入	<code>import type { X } from '@shared'</code>	所有页面
请求	<code>await xxxApi.method() + res.success</code>	所有页面
环境判断	<code>import.meta.env.DEV/PROD</code>	LoginPage 跳转
样式作用域	<code><style scoped> + :deep()</code>	所有组件

5.22 4.22 深入：响应式原理（理解“为什么能自动更新”）

5.22.1 4.22.1 从 getter/setter 说起

Vue 3 的响应式基于 **Proxy**（ES6 代理对象）。当你访问 `ref.value` 或 `reactive` 对象的字段时：

flowchart LR

A[读取 x.value] --> B[Proxy get 拦截] --> C[登记依赖
谁在用这个值]

D[写入 x.value] --> E[Proxy set 拦截] --> F[触发更新
通知依赖者重新渲染]

简单说：**读的时候登记，写的时候通知**。组件渲染时读过的响应式值，之后任何一个变化都会触发该组件重渲染。

5.22.2 4.22.2 为什么项目里“改 store 数据界面就自动变”

```
const ecuData = reactive<EcuFields>({ ngSpeed: 0, ... })
// WS 收到数据: store.ecuData.ngSpeed = 100
// 模板里的 {{ ecuData.ngSpeed }} 自动更新（因为渲染时登记了依赖）
```

5.22.3 4.22.3 ref 的 .value 为什么在模板里不用写

模板编译时自动解包：`{{ loading }}` 编译为 `{{ loading.value }}`。这是编译器语法糖，理解即可。

5.22.4 4.22.4 响应式丢失的典型场景

```
// ① 解构 reactive 对象 → 丢失
const { ngSpeed } = store.ecuData      // X ngSpeed 是普通值
// 用 storeToRefs 或整对象访问
```

```
// ② 数组元素替换 → 部分丢失
const arr = reactive([a: 1])
const item = arr[0] // 之后 arr[0] = 新对象 → item 仍是旧引用

// ③ 深层嵌套 → reactive 自动深响应 (ref 不会)
// reactive 深; ref 只包裹 .value (内部如果是对象也会变响应式)
```

项目避坑口诀：跨组件共享用 store；组件内复杂对象用 reactive；基础值/数组用 ref。

5.23 4.23 深入：v-model 自定义组件（双向绑定的本质）

5.23.1 4.23.1 本质是什么

```
<!-- v-model 是语法糖: -->
<el-input v-model="form.email" />
<!-- 等价于: -->
<el-input :model-value="form.email" @update:model-value="(v) => (form.email = v)" />
```

5.23.2 4.23.2 自定义组件实现 v-model

```
<!-- 子组件: MyToggle.vue -->
<script setup lang="ts">
const props = defineProps<{ modelValue: boolean }>()
const emit = defineEmits<{ (e: 'update:modelValue', v: boolean): void }>()
const toggle = () => emit('update:modelValue', !props.modelValue)
</script>
<template>
  <button @click="toggle">{{ modelValue ? '开' : '关' }}</button>
</template>

<!-- 父组件用法 -->
<MyToggle v-model="isDark" />
```

5.23.3 4.23.3 多值 v-model（本项目应用较少，了解即可）

```
<MyComp v-model:title="t" v-model:content="c" />
<!-- 子组件对应 emit('update:title', ...) / emit('update:content', ...) -->
```

5.24 4.24 深入：provide / inject（跨层级传值）

5.24.1 4.24.1 用法

```
// 祖先组件
import { provide } from 'vue'
provide('theme', isDark) // 提供

// 任意后代组件
import { inject } from 'vue'
const theme = inject('theme', false) // 注入（第二个参数是默认值）
```

5.24.2 4.24.2 项目中的使用

本项目主要用 Pinia 代替 provide/inject（全局状态），provide/inject 只在深层组件链传值场景用（如仪表盘向子卡片传数据）。了解即可，新代码优先 store。

5.25 4.25 深入：city3d 的 Three.js 基础（只讲够看懂的程度）

5.25.1 4.25.1 Three.js 是什么

WebGL 3D 库：场景（Scene）→ 相机（Camera）→ 物体（Mesh）→ 渲染循环（render loop）→ 灯光/材质/动画。

```
// frontend/city3d/src/composables/useCityScene.ts (结构示意图)
import * as THREE from 'three'

const scene = new THREE.Scene() // 场景
const camera = new THREE.PerspectiveCamera(75, w / h, 0.1, 1000) // 透视相机
const renderer = new THREE.WebGLRenderer({ antialias: true }) // 渲染器
renderer.setSize(window.innerWidth, window.innerHeight)
container.appendChild(renderer.domElement)

// 建筑: BoxGeometry (盒子) + MeshStandardMaterial (材质) + position
const geometry = new THREE.BoxGeometry(1, height, 1)
const material = new THREE.MeshStandardMaterial({ color: 0x4f7bbd })
const mesh = new THREE.Mesh(geometry, material)
mesh.position.set(x, height / 2, z)
scene.add(mesh)

// 渲染循环 (requestAnimationFrame 驱动)
const animate = () => {
  requestAnimationFrame(animate)
  controls.update() // 轨道控制器
  renderer.render(scene, camera) // 每帧渲染
}
animate()
```

5.25.2 4.25.2 city3d 用到的 Three.js 特性

特性	用途
BoxGeometry + Mesh	建筑方块
Points + BufferGeometry	星空粒子
自定义 ShaderMaterial	天空穹顶/地面圆盘 (GLSL 着色器在 shaders/index.ts)
后处理 (Bloom)	光效
OrbitControls	视角操作
昼夜/天气状态机	timeOfDay.ts 四档插值

5.25.3 4.25.3 新手须知

- city3d 是全项目最“特殊”的应用 (Three.js 深度定制), 日常维护以参数调整为主 (改颜色/高度/数量), 不要轻易重构场景逻辑。
- 它的 5 秒事件轮询 (useCityData) 与 WS 无关——3D 场景数据以轮询为主。

5.26 4.26 深入：前端构建与性能

5.26.1 4.26.1 构建产物分析

```
npm run build # vue-tsc 类型检查 + vite build
# dist/ 下产物: index.html + assets/*.js (按路由分包) + assets/*.css
```

5.26.2 4.26.2 项目用到的性能手段

手段	位置	说明
路由懒加载	所有 router	() => import(...) 分包
动态 import 大库	fj200c_main 打印	await import('./reportPrint') 独立 chunk
WS 节流	后端 + 前端	200ms/50ms 事件节流
环形缓冲	dashboard store	图表限长 100 点
表格虚拟滚动 (可选)	—	数据量大时考虑 el-table-v2

5.26.3 4.26.3 依赖安装规则 (重要)

☒ 根目录安装 (workspaces 统一)
npm install <pkg> -w frontend/<app> # 指定 workspace 安装

☒ 不要进子目录单独装
cd frontend/admin && npm install xxx # 危险: 产生重复依赖实例

5.27 4.27 深入: Vue 生命周期全图

flowchart TD

```
A[创建组件实例] --> B[setup 执行<br/>组合式函数初始化]
B --> C[onBeforeMount]
C --> D[DOM 挂载]
D --> E[onMounted<br/>★ 请求数据/建连接]
E --> F[数据变化 → onBeforeUpdate → onUpdated]
F --> G[卸载前 onBeforeUnmount<br/>★ 清理资源]
G --> H[onUnmounted]
```

项目最重要的三个钩子: 1. setup 阶段: 所有响应式声明 (写在 script setup 顶层)。2. onMounted: 请求数据、建 WS、起定时器。3. onUnmounted: 断开 WS、清定时器、销毁图表。

组合式函数内部的 onScopeDispose 比 onUnmounted 更灵活 (组件卸载 + 组合式函数作用域结束都会触发), useCityData.ts 用它清理轮询。

5.28 4.28 深入: TypeScript 高级模式 (本项目实战)

5.28.1 4.28.1 泛型工厂 (shared 的 createAuthStore)

```
// packages/shared/src/stores/auth.ts — 泛型 + 返回对象类型推断
export function createAuthStore(options: AuthStoreOptions): StoreDefinition {
  return defineStore(options.id, () => { ... })
}
// 调用方: useAuthStore 的类型由 factory 推导, 无需手写
```

5.28.2 4.28.2 类型守卫与谓词函数

```
// 判断是否为某事件类型 (类型收窄)
function isPortData(e: Fj200cMainWsEvent): e is Extract<Fj200cMainWsEvent, { type: 'port_data' }> {
  return e.type === 'port_data'
}
```

5.28.3 4.28.3 模板类型 (template literal types)

```
type WsPath = `/api/${string}` // 约束以 /api/ 开头
```

5.28.4 4.28.4 条件类型与映射类型 (orval 生成的内部实现)

```
// 解包 Promise: Awaited<T>
type Result = Awaited<ReturnType<typeof fn>> // generated 文件尾部大量使用

// 映射类型: 所有字段变可选
type PartialUser = { [K in keyof User]?: User[K] }
```

新手原则: 看到这些高级类型不要慌——它们都是 orval 生成的**内部类型**, 你只需要使用导出的 XxxResult 与模型类型。

5.28.5 4.28.5 非空断言与可选链的取舍

```
user!.name // 非空断言: 告诉编译器"肯定有" (运行时可能崩, 慎用)
user?.name ?? '--' // 可选链 + 兜底: 安全 (项目主用)
```

5.29 4.29 第二章收官: 动手练习清单

给 Vue 新手的四个热身练习 (每个 15 分钟, 都在 fw100 上做——最简单):

练习 1: 读页面——打开 frontend/fw100/src/views/fw100/Panel.vue, 逐行读懂, 回答: 数据从哪来? loading 怎么控制? 表格列绑定什么?

练习 2: 加一列——给 Panel.vue 的表格加一列 updatedAt (先查 LedgerItem 类型有没有这个字段, 再改模板)。

练习 3: 加个按钮——表格上方加“刷新”按钮, @click="fetchItems"。

练习 4: 状态条——页面底部加一行显示记录数: 共 {{ items.length }} 条。

做完后 npm run build 验证类型与构建通过。这四个练习做完, 你对本项目前端的读写能力已经入门。

5.30 4.30 逐行精读: main.ts 与 App.vue (每个应用的骨架)

5.30.1 4.30.1 main.ts (7 个应用几乎相同)

```
// frontend/fj200c_information/src/main.ts
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css' // 亮色主题样式
import 'element-plus/theme-chalk/dark/css-vars.css' // 暗色主题 CSS 变量
import zhCn from 'element-plus/dist/locale/zh-cn.mjs' // 中文语言包
import App from './App.vue'
import router from './router'
import './style.css'

const app = createApp(App)
app.use(createPinia()) // ① Pinia
app.use(router) // ② Router
app.use(ElementPlus, { locale: zhCn }) // ③ Element Plus (中文)
app.mount('#app') // ④ 挂载
```

顺序有讲究: Pinia 必须先注册 (router 守卫和 App.vue 里要用 store); Element Plus 全局注册后模板里可直接用所有组件。

5.30.2 4.30.2 App.vue (应用根组件)

```
<!-- frontend/fj200c_information/src/App.vue -->
<script setup lang="ts">
import { computed, onMounted } from 'vue'
import { useRoute } from 'vue-router'
```

```
import { useAuthStore } from '@stores/auth'
import { AppNavbar } from '@shared'

const route = useRoute()
const isLoginPage = computed(() => route.path.startsWith('/login'))
const authStore = useAuthStore()

onMounted(() => {
  authStore.initAuth() // 应用启动即初始化认证（恢复会话/拉角色注册表）
})
</script>

<template>
  <div id="app">
    <AppNavbar v-if="!isLoginPage" /> <!-- 登录页不显示导航栏 -->
    <router-view /> <!-- 页面出口 -->
  </div>
</template>
```

App.vue 就是“壳”：导航栏 + 页面插槽。登录页特殊（无导航栏）。initAuth 在挂载时执行，保证刷新页面后会话恢复。

5.30.3 4.30.3 vue-env.d.ts (类型声明)

```
/// <reference types="vite/client" />
declare module 'element-plus/dist/locale/zh-cn.mjs' { // 无类型声明的库
  const zhCn: Record<string, unknown>
  export default zhCn
}
```

手写类型声明：当第三方库没有 TS 类型时，用 declare module 声明。项目里 hiprint 也有类似声明（types/vue-plugin-hiprint.d.ts）。

5.31 4.31 逐行精读：axios 拦截器 (shared/api/index.ts)

```
// packages/shared/src/api/index.ts
import axios from 'axios'
import { getSessionToken, clearSession } from '../session'

export function createApiClient(loginPath: string): AxiosInstance {
  const api = axios.create({
    baseURL: import.meta.env.VITE_API_BASE_URL || '/api', // 所有请求自动加前缀
    timeout: 10000, // 10 秒超时
  })

  // 请求拦截器：自动附加 token
  api.interceptors.request.use((config) => {
    const token = getSessionToken()
    if (token) config.headers.Authorization = `Bearer ${token}`
    return config
  }, (error) => Promise.reject(error))

  // 响应拦截器：401 统一处理
  api.interceptors.response.use(
    (response) => response,
    (error) => {
      if (error.response?.status === 401) {
        clearSession()
        window.location.href = loginPath // 各应用登录路径不同
      }
      return Promise.reject(error)
    }
  )
}
```

```

    )
    return api
}

```

设计要点： 1. baseUrl: '/api': 代码里写 /auth/login, 实际请求 /api/auth/login (与 OpenAPI 路径、后端路由一致)。 2. token 从 session (localStorage) 取——**不是从 Pinia**, 因为拦截器在 Vue 上下文外运行。 3. loginPath 参数让每个应用指定自己的登录页路径 (dev 与 prod 不同)。 4. 401 处理全局兜底: token 过期/无效时自动清会话跳登录。

5.32 4.32 大页面拆解: Users.vue (admin 最复杂的页面, 507 行)

以 admin 的用户列表页为例, 看一个真实业务页面的完整结构:

5.32.1 4.32.1 模板结构 (四区块)

```

<template>
  <div class="users-page">
    <!-- 区块一: 顶部工具栏 (搜索 + 角色筛选 + 新建按钮) -->
    <div class="toolbar">
      <el-input v-model="search" placeholder="搜索用户名/邮箱" clearable />
      <el-select v-model="roleFilter" placeholder="角色" clearable>
        <el-option v-for="r in roles" :key="r.key" :label="r.name" :value="r.key" />
      </el-select>
      <el-button v-if="canCreate" type="primary" @click="goCreate">新建用户</el-button>
    </div>

    <!-- 区块二: 数据表格 -->
    <el-table v-loading="loading" :data="filteredUsers" stripe>
      <!-- 列定义... -->
      <el-table-column label="操作">
        <template #default="{ row }">
          <el-button :disabled="!canEdit" @click="openEditDialog(row)">编辑角色</el-button>
          <el-button :disabled="!canDelete" type="danger" @click="confirmDelete(row)">删除</el-button>
        </template>
      </el-table-column>
    </el-table>

    <!-- 区块三: 分页 -->
    <el-pagination v-model:current-page="page" :total="totalUsers" layout="prev, pager, next" />

    <!-- 区块四: 编辑角色对话框 -->
    <el-dialog v-model="editDialogVisible" title="编辑角色">
      <!-- 表单... -->
    </el-dialog>
  </div>
</template>

```

5.32.2 4.32.2 script 结构 (关注点分离)

```

// ① 权限控制 (UI 与权限联动)
const authStore = useAuthStore()
const canCreate = computed(() => authStore.hasPermission(Permission.UsersWrite))
const canDelete = computed(() => authStore.hasPermission(Permission.UsersDelete))

// ② 数据与筛选
const users = ref<User[]>([])
const search = ref('')
const roleFilter = ref('')
const filteredUsers = computed(() => users.value.filter(u =>
  (!search.value || u.username.includes(search.value) || u.email.includes(search.value)) &&

```



```

    (!roleFilter.value || u.role === roleFilter.value)
  ))

// ③ 请求
const fetchUsers = async () => { loading.value = true; try { ... } finally { loading.value = false } }

// ④ 操作
const openEditDialog = (user: User) => { editForm.value = { ...user };
editDialogVisible.value = true }
const confirmDelete = (user: User) => {
  ElMessageBox.confirm(`确定删除用户 ${user.username}?`, '提示', { type: 'warning' })
    .then(async () => { await deleteUser(user.id); ElMessage.success('已删除');
fetchUsers() })
    .catch(() => {})
}

```

5.32.3 4.32.3 从页面学到的模式

模式	说明	复用到哪
权限驱动 UI	hasPermission 控制按钮 disabled/显示	所有管理页面
computed 筛选	前端筛选不重新请求	列表页通用
ElMessageBox 确认	危险操作二次确认	删除类操作
对话框编辑	行数据 → 表单 → 保存 → 刷新列表	CRUD 通用

5.33 4.33 深入：响应式布局与自适应 (utils/responsive.ts)

```

// frontend/fw100/src/utils/responsive.ts (结构示意图)
import { useWindowSize, useElementBounding } from '@vueuse/core'

export function useResponsive() {
  const { width } = useWindowSize()
  const isMobile = computed(() => width.value < 768)
  return { isMobile }
}

```

// useLayoutConfig: 导航栏/侧栏布局配置

vueuse 是 Vue 组合式工具库 (useWindowSize 等)，各应用直接依赖。移动端与桌面端自适应靠它 + CSS 媒体查询。

5.34 4.34 深入：状态流全景 (一个页面从加载到更新的完整数据流)

以 fj200c_information 的 Monitor 页为例，把本章所有概念串起来：

```

sequenceDiagram
    participant R as Router 守卫
    participant M as Monitor.vue
    participant H as composables
    participant S as 后端 WS
    R->>R: 守卫: initAuth + 权限检查
    R->>M: 放行渲染
    M->>M: 组装 4 个 composables (useClock/useService/useCommandChannel/
useFj200cInformationEvents)
    M->>H: onMounted → useService.checkStatus()
    H->>S: GET /api/fj200c_information/service/status
    S-->>H: {running: true}
    H-->>M: serviceRunning.value = true
    M->>H: useFj200cInformationEvents.connect()

```

```

H->>S: WebSocket 连接 (?token=)
S-->>H: 初始快照 (TableData)
loop 每帧
  S-->>H: WS 事件 (TableData/Frame/Payload)
  H->>H: switch(event.type) 分发
  H-->>M: rows.value 更新 → 表格自动刷新
end
M->>H: 用户点"停止服务"
H->>S: POST /service/stop
S-->>H: {running: false}

```

这一页就是一个微缩全栈：守卫 → 组装 → 请求 → WS → 分发 → 渲染。理解它，就理解了本项目所有页面。

5.35 4.35 第四章收官：知识自测

1. ref 和 reactive 的区别？项目里数组用什么？
2. <script setup> 相比普通 setup 的优势？
3. 路由守卫的四个步骤？
4. WS 自动重连怎么实现？模块级单例连接解决什么问题？
5. 为什么 API 调用要经过 facade 层？
6. 401 为什么用 window.location.href 处理？
7. storeToRefs 解决什么问题？
8. 表格列里的 #default="{ row }" 是什么语法？
9. 子目录为什么不能单独 npm install？
10. computed 和普通函数的区别？

对照本章内容检查答案。全部掌握后，进入 05 章——前端逐应用精读。

5.36 4.36 深入：TypeScript 严格模式的约束（为什么报错这么多）

5.36.1 4.36.1 项目 tsconfig 关键项

```

{
  "compilerOptions": {
    "strict": true,           // 严格模式全家桶
    "noUnusedLocals": true,   // 未使用局部变量 → 报错
    "noUnusedParameters": true, // 未使用函数参数 → 报错
    "noFallthroughCasesInSwitch": true, // switch 穿透 → 报错
    "skipLibCheck": true,     // 跳过 node_modules 类型检查（提速）
    "noEmit": true            // vue-tsc 只检查不产出 JS
  }
}

```

5.36.2 4.36.2 新手最常见的报错与修法

报错	原因	修法
'x' is declared but its value is never read	变量没用	删掉或用起来
Parameter 'e' implicitly has an 'any' type	参数没类型	显式标注类型或推断
Property 'xxx' does not exist on type 'Y'	字段不存在	查模型类型拼写
Object is possibly 'null'	可空值未判断	? . 或 if (x) 收窄
Type 'A' is not assignable to type 'B'	类型不匹配	转换或修类型
'X' is declared but never used	导入未使用	删除导入

5.36.3 4.36.3 vue-tsc 的意义

npm run build # 第一步就是 vue-tsc --noEmit: 模板里的类型也检查!

模板里也会查类型——`{{ user.email }}` 如果 user 类型没有 email, 构建直接失败。所以前端“编译不过”大多不是语法错, 而是类型错。**构建报错第一件事: 看类型。**

5.37 4.37 深入: 组件通信全谱 (什么时候用什么)

flowchart TD

```
A[组件间要传什么] --> B{层级关系}
B -->|父子| C[props 下传<br/>emit 上传]
B -->|祖孙/深层| D[provide/inject<br/>或 store]
B -->|兄弟| E[store<br/>或 useEventBus]
B -->|任意/临时| F[事件总线 eventBus]
```

5.37.1 4.37.1 四种通信方式的取舍

方式	优点	缺点	项目用法
props + emit	显式、单向数据流	深层传递繁琐	组件封装 (最多)
v-model	简洁双向	只适合单值/对值	表单组件
provide/inject	穿透层级	隐式依赖	少用 (三处内)
Pinia store	全局、响应式、调试友好	全局污染	主力 : 认证/业务数据
useEventBus	任意触发、解耦	难追踪	跨模块事件 (同步/主题)

项目准则: 跨组件、跨页面共享 → store; 仅父子 → props/emit; 临时广播 → useEventBus。

5.37.2 4.37.2 事件总线示例 (vueuse 的 useEventBus)

```
// 某处定义总线 (模块级单例)
import { useEventBus } from '@vueuse/core'
export const syncBus = useEventBus<'theme-changed' | 'language-changed'>()

// 发布者
syncBus.emit('theme-changed', 'dark')

// 订阅者
```

```
const off = syncBus.on('theme-changed', (t) => applyTheme(t))
onUnmounted(off) // 记得清理
```

5.38 4.38 深入：异步与竞态处理

5.38.1 4.38.1 Promise.all 并行请求

```
// 同时请求多个接口（比串行快）
const [service, config, record] = await Promise.all([
  api.getServiceStatus(),
  api.getConfig(),
  api.getRecordStatus(),
])
```

5.38.2 4.38.2 竞态问题（快速点击/快速切换的坑）

```
// 问题：两次请求返回顺序颠倒，旧数据覆盖新数据
// 方案：请求序号 + 令牌
let requestSeq = 0
const fetchData = async () => {
  const mySeq = ++requestSeq
  const res = await api.getData()
  if (mySeq !== requestSeq) return // 已有更新的请求，丢弃
  data.value = res.data
}
```

本项目实例：搜索框防抖（useDebounce）就为减少竞态；切换角色/页面时用序号令牌防旧响应覆盖。

5.38.3 4.38.3 防抖与节流（vueuse）

```
import { useDebounceFn, useThrottleFn } from '@vueuse/core'
const onSearch = useDebounceFn(async (kw: string) => {
  items.value = (await api.search(kw)).data ?? []
}, 300) // 输入停顿 300ms 才请求

const onWsData = useThrottleFn((data) => {
  updateChart(data)
}, 50) // 50ms 内最多执行一次
```

5.39 4.39 深入：Vite 环境变量与模式

5.39.1 4.39.1 三种模式

模式	命令	变量	用途
development	npm run dev	import.meta.env.DEV = true	开发调试
production	npm run build	import.meta.env.PROD = true	构建部署
staging（自定义）	vite build --mode staging	import.meta.env.MODE = 'staging'	可选

5.39.2 4.39.2 环境变量文件（各前端目录）

```
.env.development # VITE_API_BASE_URL=/api
.env.production  # VITE_API_BASE_URL=/api
```

命名规则：只有 VITE_ 前缀的变量会暴露给前端代码（防泄露密钥）。

5.39.3 4.39.3 项目用到的关键变量

```
import.meta.env.DEV      // 是否开发模式 (LoginPage 跳转用)
import.meta.env.PROD      // 路由 base 切换用
import.meta.env.BASE_URL // vite base (build 时 = /xxx/)
```

5.40 4.40 深入：import 路径别名 (@ 与 @shared)

```
// vite.config.ts
resolve: {
  alias: {
    '@': path.resolve(__dirname, 'src'), // 应用自身源码
    '@shared': path.resolve(__dirname, '../..../packages/shared/src'), // 共享包
  }
}
```

别名	指向	用途
@	本应用 src/	import App from '@App.vue'
@shared	共享包源码	import { AppNavbar } from '@shared'

注意：@shared 直连 packages/shared/src 源码（开发模式实时编译），不是打包产物——所以改 shared 代码无需重新 build 共享包，刷新即生效。这也是 npm workspaces + 源码引用的组合优势。

5.41 4.41 第四章扩展阅读：Element Plus 组件速查

场景	组件	项目用法
展示数据	el-table / el-descriptions / el-statistic	Users / Monitor
输入表单	el-form / el-input / el-select / el-date-picker	Login / Config
状态反馈	el-message / el-message-box / el-notification	全局提示
布局	el-container / el-header / el-main / el-row/el-col	页面骨架
导航	el-menu / el-breadcrumb / el-tabs	Navbar / 子页切换
交互	el-dialog / el-drawer / el-popover / el-tooltip	编辑/详情
状态	el-tag / el-badge / el-progress	状态列
流程	el-button loading / el-skeleton	加载态
数据图表	ECharts (非 Element)	Monitor 曲线

5.42 4.42 深入：el-table 高级用法 (Monitor 页的核心)

5.42.1 4.42.1 自定义单元格插槽

```
<el-table-column prop="state" label="状态" width="100">
  <template #default="{ row }">
    <el-tag :type="row.state === 'running' ? 'success' : 'danger'">
```

```

      {{ row.state }}
    </el-tag>
  </template>
</el-table-column>

```

核心语法： `#default="{ row }"` —— 作用域插槽，`row` 是当前行数据。表格渲染的自定义全走这个插槽。

5.42.2 4.42.2 多级表头（嵌套表头）

```

<el-table-column label="参数">
  <el-table-column prop="temperature" label="温度" />
  <el-table-column prop="pressure" label="压力" />
</el-table-column>

```

5.42.3 4.42.3 固定列与斑马纹

```

<el-table :data="rows" stripe border>
  <el-table-column type="index" label="#" width="50" fixed="left" />
  <!-- fixed="left/right" 锁定列 -->
</el-table>

```

5.42.4 4.42.4 列宽自适应

```

<el-table-column prop="name" label="名称" min-width="140" />
<!-- min-width: 按内容撑开，不够时横向滚动 -->

```

5.43 4.43 深入：ECharts 在项目中的实际用法

5.43.1 4.43.1 基础三步（Vue 集成模式）

```

// frontend/fj200c_information/src/views/Monitor.vue（示意）
import * as echarts from 'echarts'
const chartRef = ref<HTMLDivElement>() // 模板 <div ref="chartRef" />

onMounted(() => {
  chart.value = echarts.init(chartRef.value!)
  chart.value.setOption({ /* 曲线配置 */ })
  window.addEventListener('resize', resize) // 窗口变化重绘
})

const resize = () => chart.value?.resize()
onUnmounted(() => {
  window.removeEventListener('resize', resize)
  chart.value?.dispose() // 销毁实例（防泄漏）
})

```

5.43.2 4.43.2 实时曲线（数据流 → setOption）

```

watch(() => store.recentData, (data) => {
  chart.value?.setOption({
    series: [{ data: data.map(d => d.value) }],
  }, { notMerge: false }) // 保留已有配置，只更新数据
}, { deep: true })

```

5.43.3 4.43.3 常见图表类型（项目对照）

图表	配置	用途
折线图 line	type: 'line' + smooth: true	温度/转速曲线
仪表盘 gauge	type: 'gauge'	圆形仪表
柱状图 bar	type: 'bar'	统计对比
饼图 pie	type: 'pie'	占比

5.44 4.44 深入：CSS 作用域与 :deep (样式隔离)

5.44.1 4.44.1 scoped 原理

```
<style scoped>
.metric-card { padding: 12px }
</style>
<!-- 编译后: .metric-card[data-v-xxxx] { padding: 12px }
      只对当前组件元素生效 (带 data 属性标记) -->
```

5.44.2 4.44.2 为什么有时需要 :deep()

Element Plus 的组件内部元素不在 data-v 标记内——所以直接写 .el-dialog 不生效，需要穿透：

```
<style scoped>
:deep(.el-dialog__title) { font-size: 18px }
:deep(.table-wrap .el-table__body) { font-size: 13px }
</style>
```

规则：自定义类名加 data-v 能直接命中；Element Plus 内部类必须 :deep()。

5.44.3 4.44.3 全局样式 (style.css)

```
/* 各应用 style.css: CSS 变量 + 全局基础样式 */
:root {
  --app-header-height: 64px;
  --app-color-bg-page: #f0f2f5;
}
```

5.45 4.45 深入：暗色模式与主题切换 (双主题)

5.45.1 4.45.1 原理

```
// 切换 class="dark" 到 html 根元素
const setDarkMode = (dark: boolean) => {
  document.documentElement.classList.toggle('dark', dark)
}
// Element Plus 官方: html.dark 时启用暗色 CSS 变量 (dark/css-vars.css)
// 应用自身: CSS 变量在 .dark 下重新定义
```

5.45.2 4.45.2 应用自身主题 (fj200c_main 的航天主题)

```
/* 亮色 */
:root {
  --theme-bg: #ffffff;
  --theme-text: #1f2937;
  --theme-accent: #2563eb;
}
/* 暗色 (html.dark) */
html.dark {
  --theme-bg: #0f172a;
  --theme-text: #e2e8f0;
  --theme-accent: #60a5fa;
}
/* 组件引用变量 */
.card { background: var(--theme-bg); color: var(--theme-text) }
```

主题切换 = 切换一个 class，所有用变量的地方自动变。 fj200c_main 的航天/仪表两套主题通过 set_theme 接口存储 (GlobalVar 持久化)，刷新后仍生效。

5.45.3 4.45.3 主题持久化

```
// 登录后从后端读取主题设置 (admin 接口) 或本地缓存
watch(theme, (t) => {
  document.documentElement.classList.toggle('dark', t === 'dark')
  localStorage.setItem('theme', t)
})
```

5.46 4.46 第四章最终自测 (进阶题)

1. 为什么 main.ts 必须先注册 Pinia 再挂载 App?
2. initAuth 在 App.vue 的 onMounted 里做什么? 为什么要做?
3. #default="{ row }" 插槽里, row 的类型从哪来?
4. :deep() 什么时候必须用? 原理是什么?
5. 暗色模式切换的核心机制是什么?
6. import.meta.env 的三种常见变量是什么? 各自用途?
7. @shared 直接引源码有什么好处?
8. 竞态问题的两种解法是什么?
9. ECharts 实例为什么要 dispose?
10. 环境变量为什么必须 VITE_ 前缀?

全部答对 → 前端语法关通过。 下一章开始逐应用精读, 把 04 章的知识落到 7 个真实应用里。

5.47 4.47 深入: 错误处理与空态设计 (前端)

5.47.1 4.47.1 三层错误处理

flowchart TD

```
A[错误] --> B{类型}
B -->|请求失败| C[拦截器 401 处理]
B -->|接口返回失败| D[页面 res.success 判断<br/>ElMessage.error 提示]
B -->|代码异常| E[try/catch + console.error<br/>兜底 UI]
```

原则: res.success === false 是业务失败 (参数错/权限错), 网络异常走 catch。两层都要处理。

5.47.2 4.47.2 空态 (数据为空时页面表现)

```
<el-table :data="rows" v-loading="loading">
  <template #empty>
    <el-empty description="暂无数据" />
  </template>
</el-table>
```

本项目约定: 列表空 → el-empty; 实时数据空 → 显示“ - ”占位 (Monitor 页表格每格 ?? ' - ')。

5.47.3 4.47.3 加载态三件套

```
const loading = ref(false) // v-loading 指令: el-table/el-button
const submitting = ref(false) // 提交中: 按钮 loading 防重复点击
const skeleton = ref(false) // 骨架屏 (首次加载大页面)
```

5.48 4.48 深入: 模板 ref 与组件实例 (拿到 DOM/组件)

5.48.1 4.48.1 模板 ref 拿 DOM

```
<template>
  <div ref="boxRef"></div>
```



```

</template>
<script setup lang="ts">
import { ref, onMounted } from 'vue'
const boxRef = ref<HTMLDivElement>()
onMounted(() => {
  boxRef.value?.getBoundingClientRect() // 直接操作 DOM
})
</script>

```

5.48.2 4.48.2 模板 ref 拿子组件实例

```

<template>
  <el-form ref="formRef" :model="form" :rules="rules" />
</template>
<script setup lang="ts">
import type { FormInstance } from 'element-plus'
const formRef = ref<FormInstance>()
const submit = async () => {
  const valid = await formRef.value?.validate().catch(() => false)
  if (valid) await api.save(form)
}
</script>

```

5.48.3 4.48.3 defineExpose (子组件暴露方法)

```

<!-- 子组件 Child.vue -->
<script setup lang="ts">
const open = () => { dialogVisible.value = true }
defineExpose({ open }) // 默认不暴露，父组件才能 ref.open()
</script>

```

```

<!-- 父组件 -->
<Child ref="childRef" /> <!-- childRef.value?.open() -->

```

项目实例：配置对话框 useConfigDialog 的 open() 方法就是通过 defineExpose 给页面调用的。

5.49 4.49 深入：KeepAlive 与组件缓存（多标签页）

5.49.1 4.49.1 作用

```

<router-view v-slot="{ Component }">
  <keep-alive :include="['Monitor']">
    <component :is="Component" />
  </keep-alive>
</router-view>

```

切走再切回时，**不销毁组件**（保留状态：滚动位置、表单内容、WS 连接）。

5.49.2 4.49.2 本项目用法

fj200c_main 的子页面切换依赖 KeepAlive 保持 ECU/ADAM/DYNO 状态；**注意：**KeepAlive 缓存组件时 onUnmounted 不会执行，清理逻辑要放 onDeactivated。Monitor 页的 WS 由 composable 内部引用计数管理，与 KeepAlive 兼容。

5.49.3 4.49.3 KeepAlive 的坑

坑	说明	对策
include 名字匹配	匹配的是 组件名 不是文件名	defineOptions({ name: 'Monitor' })
onUnmounted 不触发	只是停用	用 onActivated/ onDeactivated
缓存过多	内存占用	include 白名单/条件缓存

5.50 4.50 第四章收官：综合项目实践（任选其一）

实践 A：给 fw100 加“导出当前筛选结果 CSV”按钮 1. 表格数据 items 已在前端 → 手动拼 CSV 字符串 → Blob 下载。2. 用到的知识：computed、Blob、a[download]、EIMessage。

实践 B：给 Monitor 页加一个“最近 10 条告警”面板 1. composable 监听 WS 事件（type 为告警类）。2. 环形数组存最近 10 条 → 面板展示。3. 用到的知识：watch、数组、插槽、样式。

实践 C：全应用暗色模式切换按钮（navbar 右上角） 1. 按钮切换 html.dark class + localStorage 持久化。2. 用到的知识：computed、watch、CSS 变量。

做完任一实践并 npm run build 通过，即可宣告 Vue 语法速成毕业。

5.51 4.51 深入：composable 的设计模式（模块化与复用）

5.51.1 4.51.1 项目 composable 分类

类别	例子	职责
生命周期类	useClock	定时器 + 自动清理
数据获取类	useService	API 调用 + 状态管理
连接类	useFj200cInformationEvents	WS 连接 + 事件分发
交互类	useConfigDialog	对话框状态 + 提交
配置类	useBackendPorts	模块级单例配置

5.51.2 4.51.2 标准写法模板

```
// composables/useXxx.ts — 项目统一风格
import { ref, onMounted, onUnmounted } from 'vue'

export function useXxx() {
  const data = ref<XxxData | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)

  const fetchData = async () => {
    loading.value = true
    error.value = null
    try {
      const res = await api.getXxx()
      data.value = res.data ?? null
    } catch (e) {
      error.value = '获取失败'
    } finally {
      loading.value = false
    }
  }
}
```

```

    }
  }

  onMounted(fetchData)    // 挂载即拉数据

  return { data, loading, error, refresh: fetchData }
}

```

5.51.3 4.51.3 为什么用 composable 而不是直接写在组件里

1. **复用**：多个页面用同一个 composable（如 4 个页面都用 useService）。
2. **测试**：逻辑脱离 UI 可单独测试。
3. **组织**：组件只剩模板 + 组装，逻辑清晰。
4. **生命周期**：composable 内 onMounted/onUnmounted 自动绑定到宿主组件。

准则：一个页面超过 100 行 script 逻辑时，考虑拆分 composable。

5.52 4.52 深入：Pinia 最佳实践（项目风格）

5.52.1 4.52.1 项目 store 分类

Store	用途	是否持久化
useAuthStore	认证/权限/角色	会话 (localStorage)
useDashboardStore	fj200c_main 业务数据	否 (运行时)
主题 store	主题设置	后端 GlobalVar

5.52.2 4.52.2 异步 action 的标准写法

```

// store 内异步操作：action 返回 Promise，组件 await
const login = async (username: string, password: string) => {
  loading = true
  try {
    const res = await api.login({ username, password })
    if (!res.success) throw new Error(res.message)
    token = res.data!.token
    permissions = res.data!.permissions
    return true // 成功
  } finally {
    loading = false
  }
}

```

5.52.3 4.52.3 组件中使用 store 的规范

```

// ✓ 标准用法
const authStore = useAuthStore()
const { permissions } = storeToRefs(authStore) // 需要响应式时
await authStore.login(u, p) // 需要调用时

// ✗ 错误用法
const { login } = useAuthStore() // 解构 action 也会丢 this 绑定（组合式写法
没这问题，但仍不建议）

```

5.52.4 4.52.4 store 之间的引用（跨 store 通信）

```

// storeA 内部使用 storeB（不要在 action 外缓存引用）
const getXxx = () => {
  const b = useDashboardStore() // 函数内取，避免循环依赖
  return b.rows
}

```

5.53 4.53 深入：动态 import 与按需加载（打包优化）

5.53.1 4.53.1 三种导入方式对比

```
import { Xxx } from '@shared/api' // 静态导入：打包时全部包含
const m = await import('@utils/print') // 动态导入：运行时按需加载（独立 chunk）
const m = import.meta.glob('./views/*.vue') // 批量导入（glob 模式）
```

5.53.2 4.53.2 项目中的动态导入

```
// fj200c_main 报表打印：用户点击"打印"才加载打印库
const generatePrint = async () => {
  const { generateReport } = await import('@utils/reportPrint')
  await generateReport(store.rows)
}
// 效果：打印库打进独立 chunk，首次加载页面不下载它（减小首屏体积）
```

5.53.3 4.53.3 首屏优化手段总结

手段	效果
路由懒加载	各页面独立 chunk，按需加载
动态 import 大库	功能触发才加载
三方库按需引入	Element Plus 全量 vs 按需（本项目全量，够用）
WS 节流	减少渲染频率

5.54 4.54 深入：跨应用通信（7 个应用的关系）

5.54.1 4.54.1 应用间跳转

```
// roles.ts 的 ROLE_APP_URLS：应用地址表
// 导航栏按当前用户角色渲染可访问的应用入口
window.open(ROLE_APP_URLS[role]) // 新窗口打开目标应用
```

5.54.2 4.54.2 登录态共享

7 个应用共用 **localStorage** 的同一 **token key**（@shared/session.ts 统一实现）。跨应用跳转后，新应用 `initAuth` 读同一 token → 免登录。

前提：同源（同域名同端口后端）——token 存 localStorage 只按“源”隔离，dev 模式 5173~5179 端口不同但共享同一后端域名，实际是同源（localhost 的 517x 端口互不共享 localStorage!）。

实际机制（重点）：dev 模式各端口 localStorage 隔离——所以 LoginPage 登录成功后用 `window.location.href` 整页跳转到目标应用并把 **token 带过去**（?token= 参数或 `postMessage`）；prod 模式同一后端托管（同源），localStorage 天然共享。这是 7 应用登录链路的关键细节。

5.54.3 4.54.3 应用间隔离

- 各应用独立构建、独立路由 base、独立样式。
- 共享的只有：@shared 代码（认证/组件/API 客户端）+ 后端接口 + 登录态 token。

5.55 4.55 第四章完结：十问十答自查表

#	问题	答案要点
1	ref vs reactive	基础值/数组用 ref，深对象用 reactive
2	script setup 优势	少样板代码、顶层自动暴露
3	守卫四步	initAuth → 未登录跳转 → 权限校验 → 放行
4	WS 重连	onclose 定时重连 + 指数退避 + 页面退出清理
5	facade 价值	解耦 generated、可加逻辑、类型收口
6	401 处理	拦截器清会话 + location.href 跳登录
7	storeToRefs	解构保持响应式
8	插槽作用域	#default= "{ row }" 取行数据
9	子目录安装	双依赖实例 → pinia 双实例黑屏
10	computed	有缓存、响应式依赖、惰性求值

04 章到此收官。本章 50+ 节覆盖了 Vue3 语法、TS 严格模式、Pinia、Router、WS、Element Plus、ECharts、构建与调试，全部基于本项目真实代码。下一章将把这些知识应用到 7 个前端应用的逐文件精读中。

5.56 4.56 深入：Transition 与列表动画（动效速成）

5.56.1 4.56.1 单元素过渡

```
<template>
  <transition name="fade">
    <div v-if="show">内容</div>
  </transition>
</template>
<style scoped>
.fade-enter-active, .fade-leave-active { transition: opacity 0.3s }
.fade-enter-from, .fade-leave-to { opacity: 0 }
</style>
```

5.56.2 4.56.2 列表过渡（Monitor 表格行进场）

```
<transition-group name="list" tag="div">
  <div v-for="item in items" :key="item.id">{{ item.name }}</div>
</transition-group>
```

项目里动效用得克制：ElMessage 弹入弹出、图表动画（ECharts 自带）、登录页宇航员 CSS 动画。新代码别过度加动画，保持专业感。

5.57 4.57 深入：Suspense 与异步组件（了解即可）

```
<Suspense>
  <template #default>
    <AsyncComponent /> <!-- 内部可 await setup -->
  </template>
  <template #fallback>
    <el-skeleton />
  </template>
</Suspense>
```

本项目未用 Suspense（用 loading 状态 + onMounted 控制，更直白）。了解存在即可，遇到异步组件需求时再学。

5.58 4.58 第四章真正的完结语

至此，04 章「Vue3 与 TypeScript 语法速成」全部 58 节完成。你用到的每一个语法点，都能在 frontend/*/src 的真实文件里找到对应代码。下一章开始，我们将带着这些语法知识，逐一走进 7 个前端应用——先解剖 shared 公共层（所有应用的基石），再走读典型应用 fj200c_information 的每个文件，最后过一遍其余 6 个应用的核心差异。

5.59 4.59 深入：模板语法全速查（写模板前对照）

5.59.1 4.59.1 插值

```
{{ text }}           <!-- 文本插值（自动转义） -->
{{ row.name }}        <!-- 表达式 -->
{{ items.length }}    <!-- 方法/计算 -->
{{ form.price.toFixed(2) }} <!-- 链式调用 -->
```

5.59.2 4.59.2 指令速查

指令	用途	例子
v-if / v-else / v-else-if	条件渲染	<div v-if="loading">
v-show	显示隐藏（不销毁）	<div v-show="show">
v-for	列表渲染	<tr v-for="u in users" :key="u.id">
v-model	双向绑定	<el-input v-model="form.name">
v-bind / :	属性绑定	:data="rows" :disabled="!canEdit"
v-on / @	事件监听	@click="save" @keyup.enter="search"
v-html	原始 HTML（慎用）	报表预览
v-pre / v-cloak	跳过编译/防闪烁	少量场景

5.59.3 4.59.3 事件修饰符

```
@click.stop           <!-- 阻止冒泡 -->
@click.prevent         <!-- 阻止默认行为 -->
@keyup.enter           <!-- 回车触发 -->
@click.once            <!-- 只触发一次 -->
```

5.59.4 4.59.4 模板中的三目与空值处理

```
{{ item.state === 'running' ? '运行中' : '已停止' }}
{{ item.remark ?? '暂无备注' }}
```

5.60 4.60 深入：样式绑定与 class 技巧

5.60.1 4.60.1 对象/数组语法

```
<div :class="{ active: isActive, disabled: isDisabled }"> <!-- 对象 -->
<div :class="[baseClass, condClass, 'always']">           <!-- 数组 -->
<div :style="{ color: themeColor, fontSize: size + 'px' }"> <!-- 内联 -->
```

5.60.2 4.60.2 动态 class 与主题

```
<div :class="theme">    <!-- 主题切换: class 变了变量全变 -->
```

这是双主题实现的本质——CSS 变量跟随 class 切换。

5.61 4.61 深入：计算属性 vs 方法的区别（再强调）

对比	computed	method
缓存	依赖不变不重算	每次调用都执行
响应式	自动追踪依赖	无
使用	模板里当属性	必须调用
适用	派生状态	事件处理/无缓存计算

```
const filtered = computed(() => items.value.filter(...)) // 依赖 items, 变了才重算
const getFiltered = () => items.value.filter(...) // 每次调用都过滤
```

项目铁律：模板里的派生数据一律 computed，不用方法。

5.62 4.62 深入：watch 的深度监听与 immediate

```
// 深层监听（对象内部变化）
watch(store.ecuData, (v) => updateChart(v), { deep: true })

// 立即执行一次（初始化时也跑）
watch(selectedMetric, (v) => resetChart(), { immediate: true })

// 监听多个源
watch([a, b], ([na, nb]) => { ... })

// 只监听一次
watch(x, cb, { once: true })
```

项目用法：主题切换、参数切换、图表数据流都用 watch。

5.63 4.63 深入：自定义指令（了解）

```
<!-- 全局指令: v-loading 就是 Element 的指令 -->
v-loading="loading" <!-- el-table 的加载遮罩 -->
```

自研指令场景（本项目中几乎没有）——大多数需求用组件/composable 解决。

5.64 4.64 深入：TypeScript 常用实用类型

```
// 项目 generated 里大量出现
Partial<T> // 所有字段可选
Required<T> // 所有字段必填
Pick<T, K> // 挑字段
Omit<T, K> // 排除字段
Readonly<T> // 只读
Record<K, V> // 键值对象
ReturnType<F> // 函数返回类型
Awaited<T> // 解开 Promise
```

实战例子：

```
type CreateUser = Omit<UserInfo, 'id' | 'createdAt'> // 创建时不用传 id
const res = await api.create(payload as CreateUserRequest)
```

5.65 4.65 前端章节补充自测（10 题）

1. v-for 为什么必须 :key?
2. v-if 与 v-show 的区别与选择?
3. computed 的缓存机制?
4. watch deep/immediate 什么时候用?
5. :class 对象语法的场景?
6. 事件修饰符 @click.stop 干什么?

7. 模板里如何安全处理 null? (?? ' ')
8. Omit/Pick 的作用?
9. 动态主题的 CSS 机制?
10. 插值为什么默认转义? (防 XSS)

答对 8+ → 04 章全面掌握，可以进入前端逐应用精读。

5.66 4.66 深入：Pinia 的底层机制（为什么 store 是响应式的）

5.66.1 4.66.1 defineStore 内部做什么

flowchart LR

```

A[defineStore 定义] --> B["组件 useAuthStore() 实例化"]
B --> C[state 包成 reactive]
C --> D[getters 包成 computed]
D --> E[actions 绑定 this]
E --> F[组件响应式使用]

```

本质：store 的 state 是 reactive 对象，getters 是 computed——所以解构会丢响应式（4.20 坑 1）。

5.66.2 4.66.2 store 实例是单例吗

同一 store id 在应用中只实例化一次（懒实例化）
 → 多个组件 useAuthStore() 拿到的是同一个实例
 → 所以状态天然全局共享

5.66.3 4.66.3 什么时候需要多个实例

createAuthStore 工厂：每个应用传不同 id → 7 个应用各自独立的 store 实例
 （同一应用内同 id 单例）

5.67 4.67 深入：组合式 API 的生命周期等价表

选项式 API	组合式 API
created	setup 顶层
mounted	onMounted
updated	onUpdated
unmounted	onUnmounted
beforeUnmount	onBeforeUnmount
watch	watch/watchEffect

项目全部用组合式（script setup）——理解等价表可读旧代码/示例。

5.68 4.68 深入：模板编译的常见误区

误区	真相
模板里可以写复杂逻辑	能写但应避免（抽 computed）
{{ }} 会执行任意 JS	只支持表达式（无语句）
v-for 和 v-if 可以同元素	优先级 v-for 高，官方不建议
ref 在模板要 .value	自动解包（顶层）

5.69 4.69 深入：前端模块化与 import 规范

5.69.1 4.69.1 导入顺序约定

```
// 1. 三方库
import { ref, computed } from 'vue'
import { useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'
// 2. shared
import { useAuthStore, Permission } from '@shared'
// 3. 本应用
import { fj200cInformationApi } from '@api'
import { useService } from '@composables/useService'
// 4. 类型
import type { TableRow } from '@types'
```

5.69.2 4.69.2 命名导出 vs 默认导出

本项目约定：

- Vue 组件：默认导出（SFC 本身）
- 工具/组合式：命名导出
- 统一通过 index.ts 汇总导出

5.69.3 4.69.3 循环依赖的坑

api → composable → views 单向引用，不要反向
跨模块共享一律走 shared/导入

5.70 4.70 深入：ElMessage 与 ElMessageBox 的完整用法

5.70.1 4.70.1 消息 (ElMessage)

```
ElMessage.success('操作成功')
ElMessage.error('操作失败')
ElMessage.warning('请检查输入')
ElMessage.info('信息提示')
```

5.70.2 4.70.2 确认框 (ElMessageBox)

```
await ElMessageBox.confirm('确定删除吗?', '提示', {
  type: 'warning',
  confirmButtonText: '确定',
  cancelButtonText: '取消',
}).then(() => {
  // 确认后的逻辑
}).catch(() => {
  // 取消（不处理）
})
```

5.70.3 4.70.3 输入框 (ElMessageBox.prompt, 可选)

```
const { value } = await ElMessageBox.prompt('请输入备注', '备注', {
  inputPattern: /^.{0,200}$/ ,
  inputErrorMessage: '最多 200 字',
})
```

5.71 4.71 深入：ECharts 的按需引入 vs 全量

```
// 全量（项目现状）：import * as echarts from 'echarts'
// 按需（体积优化）：
import { use } from 'echarts/core'
import { LineChart } from 'echarts/charts'
import { GridComponent, TooltipComponent } from 'echarts/components'
use([LineChart, GridComponent, TooltipComponent])
```

现状分析：全量打包体积大但省事；监控类应用图表类型单一，可改为按需（首屏体积 -30% 左右）。

5.72 4.72 深入：表单校验的 trigger 时机

trigger	触发时机	适用
blur	失焦	文本输入
change	值变化	选择器/开关
blur + change	两者	推荐组合

```
rules = {
  username: [{ required: true, message: '必填', trigger: ['blur', 'change'] }],
}
```

5.73 4.73 深入：04 章补充自测（追加 10 题）

- 1. store 为什么解构会丢响应式？
- 2. store 单例的条件？
- 3. 选项式与组合式生命周期对应？
- 4. 模板支持什么表达式？
- 5. import 顺序的约定？
- 6. 循环依赖怎么避免？
- 7. ElMessageBox.confirm 的取消怎么处理？
- 8. ECharts 按需引入怎么做？
- 9. trigger 什么时候用 change？
- 10. 项目中组件默认导出还是命名导出？

答对 8+ → 04 章补充完成。

5.74 4.74 深入：响应式系统的常见陷阱再盘点（实战级）

5.74.1 4.74.1 ref vs reactive 的选择

```
// 简单值: ref (string/number/boolean)
const count = ref(0)

// 复杂结构: reactive 或 ref 包对象
const form = reactive({ name: '', age: 0 })
const form2 = ref({ name: '', age: 0 }) // 也可，访问 form2.value
```

项目惯例：统一用 ref（写法一致，少纠结）。

5.74.2 4.74.2 数组响应式的坑

```
const list = ref<string[]>([])

// 通过 index 赋值不触发响应
list.value[0] = 'x' // ✗ 可能不更新
list.value.splice(0, 1, 'x') // ✓
list.value = ['x'] // ✓ 整体替换
```

5.74.3 4.74.3 watch 的 immediate 与 deep

```
watch(source, cb, { immediate: true }) // 立刻执行一次
watch(obj, cb, { deep: true }) // 深度监听（性能注意）
```

5.75 4.75 深入：路由与页面缓存的配合

5.75.1 4.75.1 监控页切走再回来

问题：切走页面销毁 → 回来重新初始化 → 数据重新拉

方案：keep-alive 缓存（<router-view v-slot="{ Component }">

<keep-alive> <component :is="Component" /> </keep-alive>）

5.75.2 4.75.2 keep-alive 的注意事项

1. 只缓存路由级组件
2. onActivated/onDeactivated 钩子（代替 onMounted）
3. 数据量大的页面慎用（内存占用）

5.76 4.76 深入：TypeScript 类型工具（进阶 8 个）

工具	作用	例子
Pick<T, K>	取部分字段	Pick<User, 'id' 'name' >
Omit<T, K>	排除字段	Omit<User, 'password' >
Partial	全可选	Partial
Required	全必填	Required
Readonly	只读	Readonly
Record<K, V>	键值对象	Record<string, number>
ReturnType	函数返回类型	ReturnType<typeof api.getList>
keyof T	字段名联合	keyof User → 'id' 'name'

5.77 4.77 深入：泛型的实战（表格列泛型组件）

// 泛型组件示例：SimpleTable<T>

defineProps<{ data: T[]; columns: Column<T>[] }>()

// 调用时自动推断

<SimpleTable :data="users" :columns="userColumns" />

好处：一个组件服务所有表格，类型安全。

5.78 4.78 深入：异步组件与按需加载

// 路由级懒加载（项目已用）

const Monitor = () => import('@views/Monitor.vue')

// 组件级（可选优化）

const BigChart = defineAsyncComponent(() => import('@components/BigChart.vue'))

效果：首屏只加载必要代码，大页面按需加载。

5.79 4.79 深入：04 章最终综合自测（追加 10 题）

1. ref 与 reactive 的选择依据？
2. 数组下标赋值为什么不触发响应？
3. immediate/deep 各解决什么问题？
4. keep-alive 缓存什么？
5. onActivated 与 onMounted 的区别？
6. Pick/Omit 的区别？
7. ReturnType 的用途？
8. 泛型表格组件的价值？
9. 异步组件解决什么问题？
10. 页面缓存的内存风险？

答对 8+ → 04 章最终通过。

5.80 4.80 深入：项目实战——完整读一个页面（fw100 列表页）

5.80.1 4.80.1 页面骨架

```
<script setup lang="ts">
// 1. 状态
const list = ref<Item[]>([])
const loading = ref(false)
const query = reactive({ page: 1, pageSize: 20, keyword: '' })

// 2. 方法
const load = async () => {
  loading.value = true
  const res = await fw100Api.listItems(query)
  list.value = res.data ?? []
  loading.value = false
}

// 3. 生命周期
onMounted(load)

// 4. 表格列配置
const columns = [
  { prop: 'name', label: '名称' },
  { prop: 'type', label: '类型' },
]
</script>

<template>
  <el-table :data="list" v-loading="loading">
    <el-table-column v-for="col in columns" :key="col.prop" v-bind="col" />
  </el-table>
</template>
```

5.80.2 4.80.2 拆解

1. 状态：ref/reactive 定义页面数据
2. 方法：load 负责拉数据
3. 生命周期：进入页面自动加载
4. 模板：el-table 绑定数据

5.80.3 4.80.3 页面的通用五步法

- ① 定义状态（ref/reactive）
- ② 定义加载函数（调用 api）
- ③ 挂载时调用（onMounted）
- ④ 绑定模板（v-for/v-model）
- ⑤ 交互（按钮/分页 → 重新加载）

任何页面都是这五步——看懂一个，全会。

5.81 4.81 深入：TypeScript 类型体操（生成代码的配合）

5.81.1 4.81.1 从生成代码推导用法

```
// 生成: fw100ListItems(params: PageParams) => Promise<ApiResponse<Item[]>>
// 推断: res.data 是 Item[] | undefined
// 使用: res.data ?? [] 保底
```

5.81.2 4.81.2 联合类型与可选链

```
// 可选字段安全访问
item.remark?.toUpperCase() ?? '-'
```

```
// 多层次
user?.profile?.avatar ?? ''
```

5.81.3 4.81.3 类型断言（慎用）

```
// 当知道比 TS 更精确时
const n = Number(str) as number
// 项目少用；尽量让类型自然推导
```

5.82 4.82 深入：computed 的依赖追踪细节

```
const filtered = computed(() => {
  // 只有用到的 ref 才触发重算
  return list.value.filter(i => i.name.includes(keyword.value))
})
// 修改 list 或 keyword → filtered 自动重算
// 其他无关变量修改 → 不触发（性能保障）
```

误区：computed 里用了非响应式变量 → 永不更新

5.83 4.83 深入：CSS 与组件样式的实践

5.83.1 4.83.1 scoped 的原理

```
<style scoped>
/* 自动加 data-v-xxx 属性选择器 */
.title { color: red; } /* 编译成 .title[data-v-xxx] */
</style>
```

5.83.2 4.83.2 覆盖 Element Plus 样式

```
<style scoped>
:deep(.el-table) { --el-table-border-color: #eee; }
</style>
```

5.83.3 4.83.3 全局样式位置

src/assets/main.css 或 index.html 引入

5.84 4.84 深入：04 章终极自测（5 题）

1. 页面的通用五步法？
2. res.data ?? [] 为什么需要保底？
3. computed 何时重算？
4. scoped 的原理？
5. :deep() 什么时候用？

答对 4+ → 04 章彻底通关。

5.85 4.85 深入：Vue Router 的完整参考（本项目路由）

5.85.1 4.85.1 路由定义

```
// router/index.ts
import { createRouter, createWebHistory } from 'vue-router'

const routes = [
  { path: '/', redirect: '/monitor' },
  { path: '/login', component: Login, meta: { public: true } },
  { path: '/monitor', component: Monitor, meta: { requiresAuth: true } },
  { path: '/csv', component: CsvList, meta: { requiresAuth: true } },
]

const router = createRouter({ history: createWebHistory(), routes })

// 守卫
```

```
router.beforeEach((to) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { path: '/login' }
  }
})
```

5.85.2 4.85.2 meta 的用途

meta.public: 是否免登录

meta.requiresAuth: 需要认证

meta.permission: 需要的权限 (可选控制菜单/按钮)

5.85.3 4.85.3 编程式导航

```
router.push('/csv')           // 跳转
router.replace('/login')      // 替换 (不留历史)
router.back()                 // 后退
```

5.86 4.86 深入: Axios 封装的完整参考 (shared 的 httpClient)

5.86.1 4.86.1 拦截器的作用

// 请求拦截器: 自动加 token

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

// 响应拦截器: 统一处理错误

```
api.interceptors.response.use(
  (res) => res.data,           // 解包
  (err) => {
    if (err.response?.status === 401) {
      // 跳登录
    }
    ElMessage.error(err.response?.data?.message ?? '请求失败')
    return Promise.reject(err)
  }
)
```

5.86.2 4.86.2 401 处理的时机

token 过期 → 任意请求 401

→ 拦截器清 localStorage → 跳登录页

→ 用户重新登录 (无需手动刷新)

5.86.3 4.86.3 为什么 7 个应用共用

httpClient 在 shared → 一套 token 逻辑

各应用 setApiInstance 注入 baseUrl (自己的 /api)

→ 跨应用体验一致

5.87 4.87 深入: 04 章实战自测 (8 题)

1. 路由守卫的三要素?
2. meta 的三种用途?
3. push 与 replace 的区别?
4. 请求拦截器做什么?
5. 响应拦截器做什么?
6. 401 的处理时机?
7. setApiInstance 注入什么?
8. 为什么 httpClient 放 shared?

答对 7+ → 04 章实战通过。

5.88 4.88 深入：WebSocket 前端的完整实现参考

5.88.1 4.88.1 建立连接

```
// composables/useWebSocket.ts (结构示意图)
export function useWebSocket() {
  const socket = ref<WebSocket | null>(null)

  const connect = () => {
    const token = localStorage.getItem('token')
    socket.value = new WebSocket(`ws://localhost:3000/api/fj200c_information/ws?token=${token}`)
    socket.value.onmessage = (e) => {
      const msg = JSON.parse(e.data)
      handleMessage(msg) // 按 type 分发
    }
    socket.value.onclose = () => setTimeout(connect, 2000) // 重连
  }

  onMounted(connect)
  onUnmounted(() => socket.value?.close())
  return { socket }
}
```

5.88.2 4.88.2 消息分发

```
function handleMessage(msg: WsMessage) {
  switch (msg.type) {
    case 'frame': rows.value.push(msg.data); break
    case 'status': status.value = msg.data; break
    case 'snapshot': rows.value = msg.data.rows; break
  }
}
```

5.88.3 4.88.3 断线重连的要点

1. onclose 自动重连（2 秒延迟）
2. 重连成功后服务端重发 snapshot（数据完整）
3. token 过期 → 401 → 跳登录
4. 页面销毁必须 close（防泄漏）

5.89 4.89 深入：Pinia store 的完整实现参考

5.89.1 4.89.1 定义

```
// stores/auth.ts
import { defineStore } from 'pinia'

export const useAuthStore = defineStore('auth', {
  state: () => ({
    token: localStorage.getItem('token') ?? '',
    user: null as UserInfo | null,
    permissions: [] as string[],
  }),
  getters: {
    isAuthenticated: (s) => !!s.token,
    hasPermission: (s) => (perm: string) => s.permissions.includes(perm),
  },
  actions: {
    async login(email: string, password: string) {
      const res = await authApi.login({ email, password })
      this.token = res.data.token
      localStorage.setItem('token', this.token)
    }
  }
})
```

```

    },
    logout() {
      this.token = ''; this.user = null; this.permissions = []
      localStorage.removeItem('token')
    },
  },
})

```

5.89.2 4.89.2 getters 带参数的写法

hasPermission 是返回函数的 getter

→ 模板里调用 permission 判断（按钮级权限）

5.89.3 4.89.3 store 跨应用复用

createAuthStore（shared）用工厂函数 → 每个应用独立实例
（同应用内仍单例）

5.90 4.90 深入：04 章高频自测（8 题）

1. WS 连接的三个关键点？
2. 断线重连的机制？
3. snapshot 的作用？
4. 为什么销毁必须 close？
5. store 的五要素（state/getters/actions 等）？
6. getter 带参数怎么写？
7. login action 的步骤？
8. 跨应用 store 隔离的方式？

答对 7+ → 04 章高频通过。

5.91 4.91 深入：表格组件的完整实现参考（高频组件）

5.91.1 4.91.1 带分页表格

```

<template>
  <el-table :data="list" v-loading="loading" @sort-change="handleSort">
    <el-table-column prop="name" label="名称" sortable />
    <el-table-column prop="createdAt" label="创建时间" />
    <el-table-column label="操作" width="200">
      <template #default="{ row }">
        <el-button size="small" @click="edit(row)">编辑</el-button>
        <el-button size="small" type="danger" @click="remove(row)">删除</el-button>
      </template>
    </el-table-column>
  </el-table>

  <el-pagination
    v-model:current-page="query.page"
    :page-size="query.pageSize"
    :total="total"
    @current-change="load"
  />
</template>

```

5.91.2 4.91.2 插槽（#default= "{ row }"）的用法

操作列用插槽拿当前行数据

→ row 就是当前行的对象

→ 编辑/删除/详情按钮都从这里触发

5.91.3 4.91.3 排序与搜索

sortable 列 → @sort-change → 重新查询
搜索框 → keyword → 查询参数 → 重新加载

5.92 4.92 深入：表单组件的完整实现参考

5.92.1 4.92.1 带校验表单

```
<el-form ref="formRef" :model="form" :rules="rules" label-width="80px">
  <el-form-item label="名称" prop="name">
    <el-input v-model="form.name" />
  </el-form-item>
  <el-form-item label="类型" prop="typeName">
    <el-select v-model="form.typeName">
      <el-option label="发动机" value="engine" />
      <el-option label="泵" value="pump" />
    </el-select>
  </el-form-item>
</el-form>
```

5.92.2 4.92.2 校验规则

```
const rules = {
  name: [{ required: true, message: '请输入名称', trigger: 'blur' }],
  typeName: [{ required: true, message: '请选择类型', trigger: 'change' }],
}
```

5.92.3 4.92.3 提交前校验

```
const formRef = ref<FormInstance>()
const submit = async () => {
  await formRef.value?.validate() // 不通过会报错
  await fw100Api.createItem(form.value)
  ElMessage.success('创建成功')
}
```

5.92.4 4.92.4 编辑回填

```
// 编辑时把 row 数据填进表单（注意深拷贝）
Object.assign(form, { ...row })
```

5.93 4.93 深入：04 章综合自测（8 题）

1. 操作列插槽怎么拿当前行？
2. 分页组件的双向绑定？
3. 排序触发后怎么办？
4. 校验规则的 trigger 区别？
5. validate 的用法？
6. 编辑回填注意什么？
7. v-loading 的用途？
8. 搜索框与查询参数的联动？

答对 7+ → 04 章综合通过。

5.94 4.94 深入：项目常用工具函数的实现参考

5.94.1 4.94.1 时间格式化

```
// 时间戳 → 可读字符串
export function formatTime(ts: number | string): string {
  const d = new Date(Number(ts))
  const pad = (n: number) => String(n).padStart(2, '0')
```

```
return `${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`
}
```

5.94.2 4.94.2 数值格式化

// 保留指定位数，避免浮点噪声

```
export function formatNum(v: number, digits = 2): string {
  return v.toFixed(digits)
}
```

// 单位换算（如转速）

```
export function formatRpm(v: number): string {
  return `${v.toFixed(0)} rpm`
}
```

5.94.3 4.94.3 防抖与节流

// 节流：高频事件限制频率（如 resize）

```
export function throttle<T extends (...args: any[]) => void>(
  fn: T, wait: number
): T {
  let last = 0
  return ((...args: Parameters<T>) => {
    const now = Date.now()
    if (now - last >= wait) { last = now; fn(...args) }
  }) as T
}
```

// 防抖：操作停止后才执行（如搜索输入）

```
export function debounce<T extends (...args: any[]) => void>(
  fn: T, wait: number
): T {
  let timer: ReturnType<typeof setTimeout> | null = null
  return ((...args: Parameters<T>) => {
    if (timer) clearTimeout(timer)
    timer = setTimeout(() => fn(...args), wait)
  }) as T
}
```

5.94.4 4.94.4 深拷贝与数组操作

```
export const clone = <T>(v: T): T => JSON.parse(JSON.stringify(v))
export const last = <T>(arr: T[]): T | undefined => arr[arr.length - 1]
```

5.95 4.95 深入：响应式状态的组织模式（项目级）

5.95.1 4.95.1 单一数据源原则

页面数据 → 一个 ref 数组

派生视图 → computed（过滤/排序/分页）

→ 不维护多份拷贝（避免不同步）

5.95.2 4.95.2 状态流转的控制

// 加载状态的三态

```
const loading = ref(false)
const error = ref<string | null>(null)
const list = ref<Item[]>([])

const load = async () => {
  loading.value = true
  error.value = null
  try {
    list.value = (await api.list()).data ?? []
  } catch (e: any) {
    error.value = e.message ?? '加载失败'
  }
}
```

```

    } finally {
      loading.value = false
    }
  }
}

```

5.95.3 4.95.3 页面交互的状态提升

子组件修改数据 → emit 给父组件 → 父组件统一刷新
(单向数据流, 避免子组件各自拉取)

5.96 4.96 深入: 项目级 TypeScript 配置解读

5.96.1 4.96.1 tsconfig 常用项

```

{
  "compilerOptions": {
    "strict": true,
    "moduleResolution": "bundler",
    "target": "ES2020",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "skipLibCheck": true,
    "noEmit": true,
    "paths": { "@/*": [". /src/*"] }
  }
}

```

5.96.2 4.96.2 strict 模式的意义

1. 严格空检查 (null/undefined 必须处理)
 2. 隐式 any 报错
 3. 未使用变量报错
- 编译期拦截大量运行时 bug

5.96.3 4.96.3 noEmit + vue-tsc

vue-tsc 只做类型检查不输出
→ npm run build = 类型检查 + vite build
→ 类型错误会阻断构建 (红线)

5.97 4.97 深入: 04 章终局自测 (8 题)

1. 时间格式化工具怎么写?
2. 节流与防抖的区别?
3. 单一数据源原则?
4. 三态加载怎么写?
5. 子组件如何让父刷新?
6. strict 模式拦截什么?
7. noEmit 的作用?
8. 为什么类型错误要阻断构建?

答对 7+ → 04 章终局通过。

5.98 4.98 深入: 前端工程的完整构建流程解读

5.98.1 4.98.1 npm run build 做了什么

1. vue-tsc --noEmit # 类型检查 (红线)
2. vite build # 打包
 - ├ 按路由分包 (chunk)
 - ├ 压缩混淆
 - └ 输出 dist/

5.98.2 4.98.2 构建产物说明

```
dist/
├── index.html          # 入口
├── assets/
│   ├── index-xxx.js   # 主包
│   ├── Monitor-xxx.js # 路由分包
│   └── index-xxx.css  # 样式
```

5.98.3 4.98.3 构建失败的常见原因

1. 类型错误 (vue-tsc 报错) → 修类型
2. 导入路径错误 → 修路径
3. 内存不足 → 增大 NODE_OPTIONS
4. 依赖缺失 → npm install

5.99 4.99 深入：前端调试技巧

5.99.1 4.99.1 浏览器 DevTools

1. Console: 看错误日志 (红色)
2. Network: 看请求/响应 (API 排错)
3. Vue DevTools: 看组件/状态 (pinia 状态)
4. 断点调试: Sources 面板

5.99.2 4.99.2 常见调试场景

场景: 表格没数据

1. Network 看请求是否发出
2. 看响应状态 (200/401/403)
3. Console 看报错
4. Vue DevTools 看 store 状态

5.99.3 4.99.3 调试打印

```
console.log('查询参数:', query)
console.log('响应:', res)
// 调试完删除 (或用 console.debug)
```

5.100 4.100 深入：04 章毕业自测 (8 题)

1. build 的两步是什么?
2. 路由分包的好处?
3. 构建失败的三个原因?
4. DevTools 四个面板?
5. 表格没数据的排查步骤?
6. 看 store 状态用哪个面板?
7. console 调试的注意?
8. Network 面板看什么?

答对 7+ → 04 章毕业。

5.101 4.101 深入：常见需求的组件模式库

5.101.1 4.101.1 搜索 + 表格 + 分页标准页

```
<template>
  <el-card>
    <div class="toolbar">
      <el-input v-model="query.keyword" placeholder="搜索名称" clearable
@keyup.enter="load" />
      <el-button type="primary" @click="load">查询</el-button>
      <el-button @click="reset">重置</el-button>
    </div>
```

```

<el-table :data="list" v-loading="loading">
  <!-- 列定义 -->
</el-table>
<el-pagination
  v-model:current-page="query.page"
  :page-size="query.pageSize"
  :total="total"
  layout="total, prev, pager, next"
  @current-change="load"
/>
</el-card>
</template>

<script setup lang="ts">
const query = reactive({ page: 1, pageSize: 20, keyword: '' })
const list = ref<Item[]>([])
const total = ref(0)
const loading = ref(false)

const load = async () => {
  loading.value = true
  const res = await api.listItems({ ...query })
  list.value = res.data ?? []
  total.value = res.total ?? 0
  loading.value = false
}

const reset = () => {
  query.keyword = ''
  query.page = 1
  load()
}
</script>

```

5.101.2 4.101.2 弹窗表单标准模式

```

<el-dialog v-model="dialogVisible" :title="editing ? '编辑' : '新增'" width="500px">
  <el-form ref="formRef" :model="form" :rules="rules" label-width="80px">
    <el-form-item label="名称" prop="name">
      <el-input v-model="form.name" />
    </el-form-item>
  </el-form>
  <template #footer>
    <el-button @click="dialogVisible = false">取消</el-button>
    <el-button type="primary" @click="save">保存</el-button>
  </template>
</el-dialog>

const openCreate = () => { editing = false; form = emptyForm(); dialogVisible = true }
const openEdit = (row: Item) => { editing = true; form = { ...row }; dialogVisible = true }
const save = async () => {
  await formRef.value?.validate()
  editing ? await api.updateItem(form) : await api.createItem(form)
  ElMessage.success('保存成功')
  dialogVisible = false
  load()
}

```

5.101.3 4.101.3 状态标签模式

```

<el-tag :type="statusType(row.status)">{{ statusText(row.status) }}</el-tag>

const statusMap = {
  running: { text: '运行中', type: 'success' },
  stopped: { text: '已停止', type: 'info' },
}

```

```

    error: { text: '异常', type: 'danger' },
  } as const
const statusInfo = (s: string) => statusMap[s as keyof typeof statusMap] ?? { text: s,
type: 'info' }

```

5.102 4.102 深入：04 章大师自测（8 题）

1. 标准页的四块结构？
2. 重置的写法？
3. 弹窗表单的两种打开方式？
4. save 的完整流程？
5. 状态标签映射怎么做？
6. clearable 的作用？
7. 编辑时表单怎么填？
8. loading 的挂载位置？

答对 7+ → 04 章大师。

5.103 4.103 深入：响应式详情的十个坑（项目踩过）

5.103.1 4.103.1 坑列表

1. 解构丢失响应式 → 用 storeToRefs
2. 数组下标赋值不触发 → splice/整体替换
3. reactive 嵌套的深层对象 → 深拷贝再修改
4. watch 默认不 deep → 需要 deep: true
5. computed 里写副作用 → 用 watch 代替
6. v-model 直接绑 store state → 需 get/set
7. 循环里闭包捕获旧值 → 用函数参数
8. 模板里大量计算 → 抽 computed
9. ref 数组替换但组件未更新 → 新数组引用
10. 异步回调里读过期状态 → 用 ref 快照

5.103.2 4.103.2 典型案例：storeToRefs

```

// ❌ 解构后失去响应式
const { user } = authStore

// ✅ 保持响应式
const { user } = storeToRefs(authStore)

```

5.103.3 4.103.3 典型案例：v-model 与 store

```

// 直接绑定 store 会报警告
// 用 computed 的 get/set 桥接
const searchText = computed({
  get: () => store.searchText,
  set: (v) => { store.searchText = v },
})

```

5.104 4.104 深入：模板语法的完整参考（补充）

5.104.1 4.104.1 条件与循环

```

<template v-if="hasData">有数据</template>
<template v-else>暂无数据</template>

<li v-for="(item, index) in items" :key="item.id">
  {{ index }} - {{ item.name }}
</li>

```

5.104.2 4.104.2 事件与修饰符

```
<button @click="save">保存</button>
<button @click.stop="stop">停止冒泡</button>
<button @click.prevent="submit">阻止默认</button>
<input @keyup.enter="search" />
<input v-model.trim="name" />
```

5.104.3 4.104.3 动态属性与插槽

```
<component :is="currentTab" />
<slot name="footer" :data="data" />
```

5.104.4 4.104.4 动态 class

```
<div :class="{ active: isActive, 'text-red': hasError }" />
<div :class="[baseClass, isActive ? 'active' : '']" />
```

5.105 4.105 深入：04 章权威自测（8 题）

1. 解构丢响应式怎么解决？
2. 数组更新的正确姿势？
3. watch 什么时候加 deep？
4. computed 的副作用问题？
5. v-model 绑定 store 的桥接？
6. 事件修饰符的四种？
7. 动态 class 的两种写法？
8. 循环里闭包的问题？

答对 7+ → 04 章权威。

5.106 4.106 深入：前端代码的组织哲学（为什么这么分）

5.106.1 4.106.1 分层思想

视图层（views）：页面组装（只做展示）
组合层（composables）：可复用逻辑
状态层（stores）：跨页面共享状态
数据层（api）：后端调用
类型层（types）：数据形状

5.106.2 4.106.2 依赖方向

views → composables/stores → api → generated
（单向依赖，禁止反向）

5.106.3 4.106.3 组件设计原则

1. 组件尽量只做一件事
2. props 传数据，emit 传事件
3. 不直接改 props（只读）
4. 复用优先（shared/公共组件）

5.107 4.107 深入：性能优化的前端实践

5.107.1 4.107.1 渲染性能

1. v-for 加 :key（列表稳定）
2. 大数据列表虚拟滚动
3. 高频更新节流（WS 数据）
4. computed 缓存（避免重复计算）

5.107.2 4.107.2 加载性能

1. 路由懒加载（分包）
2. 组件按需引入（Element Plus）
3. 图片懒加载
4. 首屏最小化（骨架屏）

5.107.3 4.107.3 内存管理

1. 定时器清理（onUnmounted）
2. WS 关闭
3. 大对象释放（null 引用）
4. 事件监听移除

5.108 4.108 深入：04 章权威自测（8 题）

1. 五层结构的职责？
2. 依赖方向？
3. 组件设计三原则？
4. 渲染性能四点？
5. 加载性能四点？
6. 内存管理四点？
7. props 为什么只读？
8. 定时器清理的位置？

答对 7+ → 04 章权威。

5.109 4.109 深入：前端开发环境配置（Volar 使用指南）

5.109.1 4.109.1 Volar 的日常用法

1. 模板类型检查（vue-tsc 集成）
2. 组件跳转（Ctrl+点击）
3. 模板里的自动补全
4. 重构（重命名/提取）
5. 插槽类型提示

5.109.2 4.109.2 常见问题

1. 类型不识别 → 重启 TS server（Ctrl+Shift+P → Restart）
2. 模板报错 → 检查 tsconfig 是否含 .vue
3. 别名不解析 → 检查 tsconfig paths
4. 多应用 → 各自打开或工作区

5.110 4.110 深入：前端代码审查清单（提交前自查）

1. 无 console.log 残留（调试用 debug）
2. 无未使用变量/导入（tsc 会查）
3. 组件 props 类型完整
4. 异步错误有处理
5. 定时器/监听有清理
6. 无直接修改 props
7. 模板无复杂逻辑（抽 computed）
8. 样式 scoped
9. 文案统一（走常量/i18n）
10. 无死代码（未使用的页面/组件）

5.111 4.111 深入：04 章权威自测（8 题）

1. Volar 五个功能？
2. 四个常见问题？
3. 十条前端审查清单？

4. 重启 TS server 的时机?
5. tsconfig paths 的作用?
6. 异步错误的处理?
7. 死代码的检查?
8. 文案统一的方式?

答对 7+ → 04 章权威。

下一节：05-前端逐应用精读。

6 05 前端逐应用精读

阅读前提：已读完 04 章 (Vue3 + TS 语法速成)。本章用真实文件走读 7 个前端应用，重点讲「每个文件是干什么的、为什么这么写、改哪里能满足你的需求」。

flowchart TD

```

subgraph SHARED[packages/shared 共享层]
    T[template 模板组件]
    A[api 客户端工厂]
    R[roles.ts 菜单/地址表]
    S[stores/auth 工厂]
    Y[types.ts 类型]
end
SHARED --> F1[admin]
SHARED --> F2[fj200c_information]
SHARED --> F3[fj200c_main]
SHARED --> F4[ftj1c]
SHARED --> F5[fw100 / fw150]
SHARED --> F6[city3d]

```

6.1 5.1 共享层 packages/shared (7 个应用的公共基石)

6.1.1 5.1.1 shared 目录全景

```

packages/shared/src/
├── index.ts           # 统一导出入口
├── roles.ts          # 菜单配置 + 应用地址表 (纯前端 UI 概念)
├── types.ts          # 类型 re-export (自 generated 导入转发)
├── session.ts        # 会话管理: token 读写、WS URL 构建
├── api/
│   ├── index.ts      # createApiClient 工厂 (axios 实例)
│   ├── auth.ts       # 登录/用户信息 API
│   ├── custom-instance.ts # orval 用的请求实例
│   └── generated/     # orval 生成 (不手改)
├── stores/
│   └── auth.ts        # createAuthStore 工厂 (认证 store)
└── template/
    ├── AppNavbar.vue  # 导航栏 (690 行)
    ├── LoginPage.vue  # 登录页 (683 行)
    └── TemplatePanel.vue # 模板面板

```

6.1.2 5.1.2 index.ts 导出清单

```

// packages/shared/src/index.ts (导出什么, 各应用就 import 什么)
export * from './roles'
export * from './types'
export * from './session'
export { createApiClient } from './api'

```

```
export { createAuthStore } from './stores/auth'
export * from './template' // AppNavbar/LoginPage/TemplatePanel
```

设计意义：7 个应用 import 的 @shared 就来自这个文件。加新共享内容 → 这里加一行导出。

6.1.3 5.1.3 roles.ts 深度（菜单与地址的唯一手写副本）

```
// packages/shared/src/roles.ts（关键概念）
// 注意：角色 key/name/permissions 不在手写，运行时从 /api/meta/roles 拉取。
// 这里只维护纯前端 UI 概念：

// ① 菜单配置（导航栏用）
export const MENU_CONFIG: MenuItem[] = [
  {
    key: 'admin',
    label: '系统管理',
    icon: 'Setting',
    appPath: '/admin',
    children: [
      { key: 'users', label: '用户管理', path: '/admin/users' },
    ],
  },
],
// fj200c_information / fj200c_main / fw100 / fw150 / ftj1c / city3d ...
]

// ② 应用地址表（跨应用跳转用）
export const ROLE_APP_URLS: Record<string, string> = {
  admin: '/admin',
  fj200c_information: '/fj200c_information',
  fj200c_main: '/fj200c_main',
  fw100: '/fw100',
  fw150: '/fw150',
  ftj1c: '/ftj1c',
  city3d: '/city3d',
}

// ③ 应用名映射（各应用入口）
export const ROLE_APP_NAMES: Record<string, string> = {
  admin: '管理后台',
  fj200c_information: '发动机监控',
  fj200c_main: '发动机测控',
  // ...
}
```

维护指南：新增角色时，这里加菜单项 + 地址（第 7 步流程里的一条）。

6.1.4 5.1.4 session.ts (token 与 WS URL)

```
// packages/shared/src/session.ts
const TOKEN_KEY = 'rustweb_token' // 7 个应用共用的 key

export function getSessionToken(): string | null {
  return localStorage.getItem(TOKEN_KEY)
}
export function setSessionToken(token: string) {
  localStorage.setItem(TOKEN_KEY, token)
}
export function clearSession() {
  localStorage.removeItem(TOKEN_KEY)
}

// WebSocket URL 构建（WS 不走 JWT header, token 走 ?token= 参数）
export function buildWebSocketUrl(path: string): string {
  const token = getSessionToken()
```

```

const protocol = window.location.protocol === 'https:' ? 'wss://' : 'ws://'
const base = window.location.hostname + (import.meta.env.DEV ? ':3000' : '')
return `${protocol}${base}${path}?token=${token ?? ''}`
}

```

要点：1. token 的 key 全局统一 → 同源应用共享登录态。2. dev 模式后端在 3000 端口，生产同端口托管 → 端口按环境拼。3. 所有 WS 连接（7 个应用的监控页）都走这个函数，token 失效即连接失败。

6.1.5 5.1.5 api/custom-instance.ts (orval 的请求壳)

```

// packages/shared/src/api/custom-instance.ts
// orval 生成的请求函数需要自定义实例：在这里接上统一 axios 实例
import { createApiClient } from './index'

const instance = createApiClient('/login') // 参数：401 后跳转的登录路径

export const customInstance = <T>(config: Parameters<typeof instance>[0]): Promise<T> => {
  return instance(config).then(({ data }) => data as T) // 直接解出 data (ApiResponse)
}

```

关键：orval 生成的 generated/api/*.ts 内部全部调用 customInstance({...})，所以改这个文件 = 改所有 API 的请求行为（超时、拦截器、401 处理）。

6.1.6 5.1.6 stores/auth.ts (认证 store 工厂)

```

// packages/shared/src/stores/auth.ts
// 工厂模式：7 个应用各自调用 createAuthStore 生成自己的 store
export function createAuthStore(options: {
  id: string, // 唯一 id (各应用不同: admin / fj200c_information ...)
  loginPath: string, // 登录页路径 (dev/prod 不同)
  homePath: string, // 登录后首页
}) {
  return defineStore(options.id, () => {
    const token = ref<string | null>(null)
    const user = ref<UserInfo | null>(null)
    const permissions = ref<Permission[]>([])
    const roles = ref<RoleInfo[]>([]) // 角色注册表缓存
    const loaded = ref(false) // initAuth 是否完成

    const isLoggedIn = computed(() => !!token.value)
    const hasPermission = (p: Permission) => permissions.value.includes(p)

    async function initAuth() { ... } // 启动恢复会话 + 拉角色注册表
    async function login(u: string, p: string) { ... }
    function logout() { ... }
    return { token, user, permissions, roles, isLoggedIn, hasPermission, initAuth, login,
      logout }
  })
}

```

工厂的好处：同一份逻辑生成 7 个独立 store（状态互不干扰），但代码只写一份。

6.1.7 5.1.7 template/AppNavbar.vue (690 行导航栏拆解)

```

<!-- 结构：双层 — 顶部角色区 + 菜单区 -->
<template>
  <div class="navbar">
    <!-- 第一层：Logo + 应用标题 + 用户信息 + 退出 -->
    <div class="top-row">
      <div class="brand">{{ appName }}</div>
      <div class="actions">
        <el-dropdown> <!-- 用户菜单：主题切换/退出登录 -->
          <span>{{ authStore.user?.username }}</span>

```

```

    <template #dropdown>
      <el-dropdown-menu>
        <el-dropdown-item @click="toggleDark">切换暗色</el-dropdown-item>
        <el-dropdown-item divided @click="handleLogout">退出登录</el-dropdown-item>
      </el-dropdown-menu>
    </template>
  </el-dropdown>
</div>
</div>
<!-- 第二层：菜单（按权限过滤） -->
<el-menu mode="horizontal" :default-active="activePath" router>
  <el-menu-item v-for="item in visibleMenus" :key="item.key" :index="item.path">
    <el-icon><component :is="item.icon" /></el-icon>
    <span>{{ item.label }}</span>
  </el-menu-item>
</el-menu>
</div>
</template>

```

导航栏逻辑： visibleMenus = MENU_CONFIG.filter(菜单权限 <= 用户权限) —— 没权限的菜单不显示。

6.1.8 5.1.8 template/LoginPage.vue (683 行登录页)

```

<!-- 结构：左侧品牌区 + 右侧登录表单 + 底部版权 -->
<template>
  <div class="login-page">
    <div class="login-left">
      <!-- 宇航员 CSS 动画（纯 CSS 绘制，无图片资源） -->
      <div class="astronaut"></div>
      <h1>RustWeb 管理系统</h1>
      <p>Rust + Vue3 全栈平台</p>
    </div>
    <div class="login-right">
      <el-form :model="form" :rules="rules" ref="formRef" @keyup.enter="handleLogin">
        <el-form-item prop="username"><el-input v-model="form.username" placeholder="用户名" /></el-form-item>
        <el-form-item prop="password"><el-input v-model="form.password" type="password" show-password /></el-form-item>
        <el-button :loading="loading" @click="handleLogin">登录</el-button>
      </el-form>
    </div>
  </div>
</template>

```

登录流程： 校验表单 → authStore.login → 成功后 window.location.href = homePath（整页跳转：跨应用/同应用刷新，保证路由 base 与 token 正确加载）。

宇航员动画： 纯 CSS (transform + animation 关键帧)，无任何图片资源——这是“前端也能做视觉效果”的示范。

6.1.9 5.1.9 各应用如何消费 shared

```

// 每个应用的 stores/auth.ts (约 15 行)
import { createAuthStore } from '@shared'
export const useAuthStore = createAuthStore({
  id: 'fj200c_information',
  loginPath: '/login',
  homePath: '/fj200c_information/monitor',
})

```

7 个应用 = 7 个 createAuthStore 调用。 改认证逻辑只改 shared 一处。

6.2 5.2 admin 应用走读 (管理后台)

6.2.1 5.2.1 admin 文件结构

```
frontend/admin/
├─ index.html
├─ vite.config.ts      # 端口 5174, base /admin/
├─ package.json        # name: @rustweb/admin
├─ src/
│   ├── main.ts
│   ├── App.vue
│   ├── style.css
│   ├── api/index.ts    # usersApi 组装
│   ├── router/index.ts # 路由 + 守卫
│   ├── stores/auth.ts  # createAuthStore 调用
│   └─ views/
│       ├── Login.vue   # 登录页 (复用 shared LoginPage 或自定义)
│       ├── Users.vue   # 用户列表 (507 行, 04 章已拆解)
│       ├── CreateUser.vue # 新建用户
│       └─ Dashboard.vue # 概览
```

6.2.2 5.2.2 admin 的路由与权限

```
// frontend/admin/src/router/index.ts
const router = createRouter({
  history: createWebHistory(import.meta.env.PROD ? '/admin/' : '/'),
  routes: [
    { path: '/login', component: Login },
    {
      path: '/',
      component: Layout,
      redirect: '/users',
      children: [
        {
          path: 'users',
          name: 'Users',
          component: Users,
          meta: { requiresAuth: true, permissions: [Permission.UsersRead] },
        },
        // CreateUser / Dashboard 同理
      ],
    },
  ],
})
router.beforeEach(guard) // 04 章守卫四步
```

6.2.3 5.2.3 admin 的 api facade

```
// frontend/admin/src/api/index.ts
import { adminApi as generated } from '@shared/api/generated'

export const usersApi = {
  list: () => generated.adminUsersList(),
  create: (payload: CreateUserRequest) => generated.adminUsersCreate(payload),
  update: (id: number, payload: UpdateUserRequest) => generated.adminUsersUpdate(id, payload),
  remove: (id: number) => generated.adminUsersDelete(id),
}
```

6.2.4 5.2.4 admin 特有的业务点

特性	说明
双层权限中间件	后端 List/Get/Info 只读, Create/Update/Delete 写 (R/W/D 三组路由)
防自锁	不能删除/降级自己的 admin 角色 (后端校验 + 前端按钮禁用)
用户表	id/username/email/password_hash/role/is_active
角色下拉	从 authStore.roles (注册表) 渲染, 非硬编码

6.3 5.3 fj200c_information 应用全文件走读 (核心示例)

6.3.1 5.3.1 应用结构

```
frontend/fj200c_information/
├─ vite.config.ts           # 端口 5173, proxy /api → :3000 (ws: true)
├─ src/
│   ├── main.ts / App.vue / style.css
│   ├── api/
│   │   ├── index.ts         # fj200c_informationApi 组装 (8 个 HTTP + WS)
│   │   └─ fj200c_information.ts # facade + WS 事件类型手写
│   ├── composables/
│   │   ├── useClock.ts      # 系统时钟
│   │   ├── useService.ts    # 服务状态轮询
│   │   ├── useCommandChannel.ts # 命令发送
│   │   ├── useConfigDialog.ts # 配置对话框
│   │   └─ useFj200cInformationEvents.ts # WS 事件分发
│   ├── router/index.ts      # 4 个子页路由
│   ├── stores/auth.ts
│   ├── types.ts             # WS 事件类型 (手写)
│   └─ views/
│       ├── Monitor.vue      # 监控主界面 (411 行)
│       ├── Visual.vue       # 可视化曲线
│       ├── Data.vue         # 数据记录
│       ├── Config.vue       # 配置管理
│       └─ Help.vue          # 使用帮助
```

6.3.2 5.3.2 api/fj200c_information.ts (facade + WS 事件类型)

```
// frontend/fj200c_information/src/api/fj200c_information.ts
import { getFj200cInformationApi } from '@shared/api/generated'
import { buildWebSocketUrl } from '@shared'

const api = getFj200cInformationApi()

// HTTP 接口 (8 个)
export const fj200cInformationApi = {
  getServiceStatus: () => api.fj200cInformationServiceStatus(),
  startService: () => api.fj200cInformationServiceStart(),
  stopService: () => api.fj200cInformationServiceStop(),
  sendCommand: (payload: CommandPayload) => api.fj200cInformationCommand(payload),
  getConfig: () => api.fj200cInformationConfigGet(),
  saveConfig: (content: string) => api.fj200cInformationConfigSave({ content }),
  getCsvRecords: () => api.fj200cInformationCsvRecords(),
  downloadCsv: (name: string) => api.fj200cInformationCsvDownload(name),
}
```

```
// WS 连接 URL (token 走参数)
export const buildWsUrl = () => buildWebSocketUrl('/api/fj200c_information/ws')
```

6.3.3 5.3.3 types.ts (WS 事件类型手写, 不走 orval)

```
// frontend/fj200c_information/src/types.ts
// WebSocket 不进 OpenAPI (orval 不生成), 事件类型全部手写
```

```
export type Fj200cInformationWsEvent =
  | { type: 'table_data'; data: TableRow[] } // 表格数据快照
  | { type: 'frame'; data: FramePayload } // 帧数据
  | { type: 'payload'; data: PayloadPayload } // 解包载荷
  | { type: 'service_status'; running: boolean } // 服务状态
  | { type: 'command_result'; success: boolean; message?: string } // 命令结果
```

判别联合: type 字段区分事件, 前端 switch(event.type) 分发——04 章学的判别联合在此实战。

6.3.4 5.3.4 composables/useFj200cInformationEvents.ts (WS 连接 + 分发)

```
// 核心逻辑 (简化): 连接、重连、事件分发、清理
```

```
import { buildWsUrl } from '@api/fj200c_information'
```

```
const RECONNECT_DELAY = 3000 // 重连间隔 3 秒
```

```
export function useFj200cInformationEvents() {
  const connected = ref(false)
  const tableData = ref<TableRow[]>([])
  const frameData = ref<FramePayload | null>(null)
  const listeners: Array<(e: Fj200cInformationWsEvent) => void> = []

  let socket: WebSocket | null = null
  let reconnectTimer: number | null = null

  const connect = () => {
    if (socket) return
    socket = new WebSocket(buildWsUrl())
    socket.onopen = () => { connected.value = true }
    socket.onmessage = (ev) => {
      const event = JSON.parse(ev.data) as Fj200cInformationWsEvent
      listeners.forEach((fn) => fn(event)) // 广播给所有订阅者
      // 自身也维护几份快照状态
      if (event.type === 'table_data') tableData.value = event.data
    }
    socket.onclose = () => {
      connected.value = false
      socket = null
      reconnectTimer = window.setTimeout(connect, RECONNECT_DELAY) // 自动重连
    }
  }

  const close = () => {
    if (reconnectTimer) clearTimeout(reconnectTimer)
    socket?.close()
    socket = null
  }

  onUnmounted(close) // 组件卸载即断开

  return { connected, tableData, frameData, connect, close, on: (fn) =>
    listeners.push(fn) }
}
```

要点： onClose 里 3 秒重连（断线自愈）； onUnmounted 关闭连接（页面退出不留残留）。

6.3.5 5.3.5 composables/useService.ts（服务启停 + 状态轮询）

```
export function useService() {
  const running = ref(false)
  const checking = ref(false)

  const checkStatus = async () => {
    const res = await fj200cInformationApi.getServiceStatus()
    if (res.success) running.value = res.data?.running ?? false
  }

  const start = async () => {
    const res = await fj200cInformationApi.startService()
    ElMessage.success(res.message || '服务已启动')
    await checkStatus()
  }

  const stop = async () => {
    // ElMessageBox 二次确认
    await checkStatus()
  }

  return { running, checking, checkStatus, start, stop }
}
```

6.3.6 5.3.6 composables/useCommandChannel.ts（命令发送）

// 向串口发送控制命令（如启动/停止发动机测试流程）

```
export function useCommandChannel() {
  const sending = ref(false)
  const send = async (cmd: string) => {
    sending.value = true
    try {
      const res = await fj200cInformationApi.sendCommand({ command: cmd })
      if (!res.success) throw new Error(res.message)
      ElMessage.success('命令已发送')
    } catch (e) {
      ElMessage.error((e as Error).message)
    } finally {
      sending.value = false
    }
  }
  return { sending, send }
}
```

6.3.7 5.3.7 composables/useClock.ts（时钟）

```
export function useClock() {
  const now = ref(new Date())
  let timer: number | null = null
  const start = () => {
    timer = window.setInterval(() => { now.value = new Date() }, 1000)
  }
  onUnmounted(() => { if (timer) clearInterval(timer) })
  return { now, start }
}
```

6.3.8 5.3.8 composables/useConfigDialog.ts（配置对话框）

// 打开 → 拉取当前配置 → 编辑 → 保存 → 刷新

```
export function useConfigDialog() {
  const visible = ref(false)
  const content = ref('')
```



```

const saving = ref(false)

const open = async () => {
  visible.value = true
  const res = await fj200cInformationApi.getConfig()
  if (res.success) content.value = res.data?.content ?? ''
}
const save = async () => {
  saving.value = true
  try {
    const res = await fj200cInformationApi.saveConfig(content.value)
    if (!res.success) throw new Error(res.message)
    ElMessage.success('已保存 (立即生效)') // 后端热加载
    visible.value = false
  } finally { saving.value = false }
}
return { visible, content, saving, open, save }
}

```

6.3.9 5.3.9 Monitor.vue (411 行主界面)

```

<script setup lang="ts">
import { useClock } from '@/composables/useClock'
import { useService } from '@/composables/useService'
import { useCommandChannel } from '@/composables/useCommandChannel'
import { useConfigDialog } from '@/composables/useConfigDialog'
import { useFj200cInformationEvents } from '@/composables/useFj200cInformationEvents'

const { now, start } = useClock()
const service = useService()
const command = useCommandChannel()
const configDialog = useConfigDialog()
const events = useFj200cInformationEvents()

onMounted(() => {
  start() // 启动时钟
  service.checkStatus() // 查询服务状态
  events.connect() // 连接 WS
})

// 事件订阅：表格数据直接绑定
const rows = computed(() => events.tableData.value)

const toggleService = async () => {
  service.running.value ? await service.stop() : await service.start()
}
</script>

<template>
<div class="monitor-page">
  <!-- 顶部：时钟 + 服务状态 + 启停按钮 + 配置按钮 -->
  <div class="toolbar">
    <el-statistic title="系统时间"><template #default>{{ now.toLocaleTimeString() }}</template></el-statistic>
    <el-tag :type="service.running.value ? 'success' : 'info'">
      {{ service.running.value ? '运行中' : '已停止' }}
    </el-tag>
    <el-button :type="service.running.value ? 'danger' : 'primary'"
      @click="toggleService">
      {{ service.running.value ? '停止服务' : '启动服务' }}
    </el-button>
    <el-button @click="configDialog.open()">配置</el-button>
  </div>

```

```

<!-- 中部：实时数据表格（WS 驱动） -->
<el-table :data="rows" height="calc(100vh - 200px)" stripe>
  <el-table-column prop="name" label="参数" />
  <el-table-column prop="value" label="数值" />
  <el-table-column prop="unit" label="单位" />
  <el-table-column prop="quality" label="质量" />
</el-table>

<!-- 配置对话框 -->
<ConfigDialog v-bind="configDialog" @save="configDialog.save" />
</div>
</template>

```

Monitor 页就是“composable 组装器”：5 个 composable + 模板，组件本身逻辑极少——这是项目推荐的页面写法。

6.3.10 5.3.10 Visual / Data / Config / Help 子页

子页	职责	技术
Visual	曲线可视化	ECharts 实时折线（数据来自 WS）
Data	CSV 记录列表 + 下载	el-table + 下载按钮（后端文件）
Config	配置说明页	静态文档 + 保存入口
Help	使用帮助	静态页面

6.4 5.4 fj200c_main 应用走读（最复杂的业务前端）

6.4.1 5.4.1 应用特点

```

flowchart LR
    subgraph FJ[fj200c_main 前端]
        A[ScaledPage 1920×1080 缩放]
        B[双主题：航天/仪表]
        C[dashboard store 环形缓冲]
        D[useBackendPorts 单例 WS]
        E[ECU/ADAM/DYNO 三路面板]
    end
    FJ --> G[后端三路串口服务]

```

与 fj200c_information 的三大差异：1. 三路串口（ECU/ADAM/DYNO）→ 前端三块面板 + 多路 WS 数据。2. 1920×1080 固定设计稿 → ScaledPage 组件按窗口比例缩放（适配大屏）。3. 双主题（航天/仪表）→ CSS 变量 + set_theme 后端持久化。

6.4.2 5.4.2 useBackendPorts.ts（模块级单例 WS + 引用计数）

```

// frontend/fj200c_main/src/composables/useBackendPorts.ts
// 问题：子页面切换（KeepAlive）时，WS 不该断也不该重复建
// 方案：模块级单例连接 + 引用计数

let socket: WebSocket | null = null
let refCount = 0 // 引用计数：当前有多少个组件在用
let reconnectTimer: number | null = null

export function useBackendPorts() {
  const data = ref<BackendPortsData | null>(null)

  if (refCount === 0) connect() // 第一个组件才建连
  refCount++

```

```

onUnmounted(() => {
  refCount--
  if (refCount === 0) {
    // 最后一个组件退出才断连
    socket?.close()
    socket = null
  }
})
return data
}

```

这个模式解决了真实 bug：git debb02f 修复了“切页导致 WS 断开”——之前每个页面自己建连，切页时旧连接销毁、新页面又建连，中间状态丢失/重复。引用计数让连接生命周期跟随**整个应用**而非单个页面。

6.4.3 5.4.3 ScaledPage (1920×1080 大屏适配)

```

<!-- frontend/fj200c_main/src/components/ScaledPage.vue (示意) -->
<script setup lang="ts">
const DESIGN_WIDTH = 1920
const DESIGN_HEIGHT = 1080
const scale = ref(1)
const update = () => {
  const { innerWidth: w, innerHeight: h } = window
  scale.value = Math.min(w / DESIGN_WIDTH, h / DESIGN_HEIGHT) // 等比缩放
}
window.addEventListener('resize', update) // 窗口变化重算
update()
</script>
<template>
  <div class="scaled-page" :style="{ transform: `scale(${scale})`, width: '1920px', height: '1080px' }">
    <slot />
  </div>
</template>

```

原理：设计稿 1920×1080 直接写死，外层 transform scale 缩放到窗口适配——所有子元素坐标不变，一套布局适配任意分辨率（大屏/小屏）。

6.4.4 5.4.4 dashboard store (业务数据中枢)

```

// frontend/fj200c_main/src/stores/dashboard.ts (示意)
export const useDashboardStore = defineStore('dashboard', () => {
  // 三路数据
  const ecu = reactive<EcuFields>({ ... })
  const adam = reactive<AdamFields>({ ... })
  const dyno = reactive<DynoFields>({ ... })

  // 图表环形缓冲（限长 100 点）
  const history = ref<number[]>([])
  const pushHistory = (v: number) => {
    history.value.push(v)
    if (history.value.length > 100) history.value.shift()
  }

  // 主题
  const theme = ref<'space' | 'dashboard'>('space')
  const setTheme = (t: 'space' | 'dashboard') => { theme.value = t; ... }

  // CSV 录制状态
  const recording = ref(false)
  return { ecu, adam, dyno, history, pushHistory, theme, setTheme, recording }
})

```

要点：三路数据放 store（多组件共享），图表数据限长 100（防内存膨胀），主题响应式全局。

6.4.5 5.4.5 双主题 CSS（航天/仪表）

```
/* frontend/fj200c_main/src/styles/themes.css（示意） */
.theme-space {
  --panel-bg: linear-gradient(180deg, #0b1026, #141b3d);
  --panel-border: rgba(100, 180, 255, 0.35);
  --panel-text: #7fd4ff;
}
.theme-dashboard {
  --panel-bg: linear-gradient(180deg, #1e3a2f, #14332a);
  --panel-border: rgba(80, 220, 160, 0.4);
  --panel-text: #7dffcf;
}
/* 组件引用变量：改主题 = 换 class */
.panel { background: var(--panel-bg); border: 1px solid var(--panel-border); color: var(--panel-text) }
```

主题持久化：登录/启动时 set_theme 接口读后端 GlobalVar（刷新后仍生效）——主题选择是“全局变量”不是本地缓存。

6.4.6 5.4.6 GenerateReport（报表打印，a06a8b4 改为原生 print）

```
// 生成试验报表：后端生成 HTML → 前端 window.print 打印
const generateReport = async () => {
  const res = await api.generateReport(store.currentTest)
  if (!res.success) return ElMessage.error(res.message)
  printWindow.value = window.open('', '_blank') // 新窗口
  printWindow.value.document.write(res.data.html)
  printWindow.value.document.close()
  printWindow.value.print() // 原生打印
}
```

git 记录：a06a8b4 把旧方案（hiprint 插件）换成原生 window.print——打印需求用浏览器原生能力更可靠，也减少了依赖。

6.4.7 5.4.7 ECU 面板（示例面板拆解）

```
<!-- 三路面板结构统一（示意） -->
<template>
  <div class="panel">
    <div class="panel-title">ECU 发动机控制单元</div>
    <div class="panel-grid">
      <GaugeCard title="转速" :value="store.ecu.ngSpeed" unit="rpm" :min="0" :max="9000" />
      <GaugeCard title="水温" :value="store.ecu.coolantTemp" unit="°C" />
      <!-- 每个参数一个卡片：仪表盘/数字/进度条三选一 -->
    </div>
    <div class="panel-actions">
      <el-button v-for="c in ecuCommands" :key="c.name" @click="command.send(c.name)">
        {{ c.label }}
      </el-button>
    </div>
  </div>
</template>
```

GaugeCard 是高度复用组件（仪表盘）：props 接收 value/unit/range，内部 ECharts gauge 或 SVG 渲染。三路面板 = 三个不同数据源的相同组件组合。

6.5 5.5 ftj1c 应用走读 (UDP 通信监控)

6.5.1 5.5.1 应用特点

```
flowchart LR
    subgraph FT[ftj1c 前端]
        A[16 路 IP 配置页]
        B[帧监控表格]
        C[坐标转换表]
        D[服务启停]
    end
    FT --> G[后端 UDP 组播服务]
```

与 fj200c_information 的差异：数据源是 UDP 组播（网络帧）而非串口；页面以**表格轮询**为主（WS 实时推送为辅）；IP 配置是核心功能（16 组组播地址）。

6.5.2 5.5.2 文件结构

```
frontend/ftj1c/
├── vite.config.ts           # 端口 5176, proxy ws: true
├── src/
│   ├── api/index.ts        # ftj1cApi 组装
│   ├── composables/        # useFtj1cEvents / useService (同 5.3 模式)
│   ├── router/index.ts
│   ├── stores/auth.ts
│   ├── types.ts            # WS 事件类型手写 (帧数据/坐标)
│   └── views/
│       ├── Monitor.vue     # 帧监控主界面
│       ├── IpConfig.vue    # IP 配置 (16 路)
│       └── Help.vue
```

6.5.3 5.5.3 IP 配置页 (核心功能)

```
<!-- IpConfig.vue 核心 (示意) -->
<template>
  <div>
    <el-table :data="ipConfigs" stripe>
      <el-table-column prop="index" label="路数" width="60" />
      <el-table-column prop="name" label="名称" />
      <el-table-column prop="ip" label="组播地址" />
      <el-table-column prop="port" label="端口" />
      <el-table-column label="操作">
        <template #default="{ row }">
          <el-button @click="openEdit(row)">编辑</el-button>
        </template>
      </el-table-column>
    </el-table>
    <el-button @click="saveAll">保存全部</el-button>
  </div>
</template>
```

注意：IP 配置修改后需**重启服务**生效（后端 config-ftj1c.ini 启动时加载，不热加载）——前端保存成功后提示“配置已保存，重启服务后生效”。

6.5.4 5.5.4 帧监控页

```
<!-- Monitor.vue 帧表格 (示意) -->
<el-table :data="frames" height="100%">
  <el-table-column prop="timestamp" label="时间" width="120" />
  <el-table-column prop="sourceIp" label="来源" width="140" />
  <el-table-column prop="frameType" label="帧类型" width="100" />
  <el-table-column prop="hexData" label="原始数据" min-width="300" />
</el-table>
```

帧展示：后端解码后的字段 + 原始 hex（方便人工核对协议）。

6.5.5 5.5.5 坐标转换（地图/坐标页）

// 后端将 UDP 帧中的坐标转换为 CGCS2000（国家大地坐标系）

// 前端展示转换结果表格：原始坐标 → 转换坐标 → 偏移量

6.6 5.6 fw100 / fw150 应用走读（最简 CRUD）

6.6.1 5.6.1 为什么先读这两个

fw100/fw150 是项目里最简单的应用（后端约 100 行，前端一个列表页 + 详情），最适合作为“完整 CRUD 闭环”的解剖标本：

后端：handler（4 个接口）→ service（增删改查）→ SQLite 表

前端：Panel.vue（列表）→ Detail.vue（详情）→ 增/删/改弹窗

6.6.2 5.6.2 文件结构（fw100 示例）

```
frontend/fw100/
├── vite.config.ts           # 端口 5175, proxy /api
├── src/
│   ├── api/index.ts        # fw100Api 组装（5 个方法）
│   ├── router/index.ts     # /login / /panel / /detail/:id
│   ├── stores/auth.ts
│   ├── utils/responsive.ts # 响应式工具（04 章提过）
│   └── views/
│       ├── Login.vue
│       ├── Panel.vue       # 设备台账列表
│       └── Detail.vue      # 设备详情
```

6.6.3 5.6.3 完整 CRUD 走读（后端 + 前端对照）

```
sequenceDiagram
    participant V as Panel.vue
    participant A as fw100Api
    participant H as 后端 handler
    participant S as service
    participant D as SQLite
    V->>A: list()
    A->>H: GET /api/fw100/items
    H->>S: list_items()
    S->>D: SELECT ...
    D-->>S: 行数据
    S-->>H: Vec<LedgerItem>
    H-->>A: ApiResponse<List<LedgerItem>>
    A-->>V: {success, data}
    V->>V: items.value = data（表格渲染）
    Note over V: 用户点"新增"填表提交
    V->>A: create(payload)
    A->>H: POST /api/fw100/items
    H->>S: create_item(payload)
    S->>D: INSERT ...
    D-->>S: 新行 id
    H-->>V: {success, message: "创建成功"}
    V->>V: ElMessage.success + 重新 list()
```

读这张图你就读懂了全项目 CRUD——fw100/fw150/admin 的增删改查全是这个模式。

6.6.4 5.6.4 fw150 与 fw100 的差异

差异	fw100	fw150
后端 schema	Fw100LedgerItem	Fw150LedgerItem (独立)
表名	fw100_ledger_items	fw150_ledger_items
前端	fw100/	fw150/
端口	5175	5178
字段	约 10 个	约 10 个 (略有不同)

历史原因：fw150 是 fw100 的变体（后来加的），复制改造——这也是 AGENTS.md「复制现有前端为新应用」的实例。

6.6.5 5.6.5 台账页的实践价值

如果你想从零学本项目前端，**fw100 是最佳起点**：1. 代码量最小（一个列表页 + 详情）。2. 无 WS、无复杂 composable（纯请求 → 渲染）。3. 增删改查全覆盖（学习后端如何与前端对接）。4. 06 章类型同步的试验田（改一个字段跑一遍 gen:api 全流程）。

6.7 5.7 city3d 应用走读（3D 城市展示）

6.7.1 5.7.1 应用特点

```
flowchart LR
    subgraph C3[city3d 前端]
        A[Three.js 3D 场景]
        B[建筑/区域/事件管理页]
        C[概览统计页]
        D[5 秒事件轮询]
    end
    C3 --> G[后端 city3d 接口]
```

特殊性：唯一使用 Three.js 的应用；后端是纯 CRUD + overview 聚合统计；**无 WS**（3D 场景用轮询刷新）。

6.7.2 5.7.2 文件结构

```
frontend/city3d/
├── vite.config.ts           # 端口 5177
├── src/
│   ├── api/index.ts        # city3dApi 组装（14 个接口）
│   ├── composables/
│   │   ├── useCityScene.ts  # 3D 场景生命周期（init/dispose）
│   │   └── useCityData.ts   # 5 秒事件轮询
│   ├── shaders/index.ts     # 自定义 GLSL 着色器
│   ├── router/index.ts
│   ├── stores/auth.ts
│   └── views/
│       ├── City3dView.vue   # 3D 主视图
│       ├── Buildings.vue    # 建筑管理
│       ├── Regions.vue      # 区域管理
│       ├── Events.vue       # 事件管理
│       └── Overview.vue     # 概览统计
```

6.7.3 5.7.3 useCityScene.ts（3D 场景生命周期）

```
// 3D 场景初始化与销毁（简化）
export function useCityScene(container: Ref<HTMLElement> | undefined) {
  let renderer: THREE.WebGLRenderer | null = null
  let animationId: number | null = null
```

```

onMounted(() => {
  if (!container.value) return
  renderer = new THREE.WebGLRenderer({ antialias: true })
  container.value.appendChild(renderer.domElement)
  // 场景/相机/灯光/建筑网格初始化 ...
  animate()
})

const animate = () => {
  animationId = requestAnimationFrame(animate)
  controls.update()
  renderer?.render(scene, camera)
}

onUnmounted(() => {
  if (animationId) cancelAnimationFrame(animationId)
  renderer?.dispose()
  // 释放 GPU 资源: geometry/material/texture dispose
})
}

```

6.7.4 5.7.4 useCityData.ts (轮询 + 场景数据刷新)

```

export function useCityData() {
  const buildings = ref<Building[]>([])
  const regions = ref<Region[]>([])
  const events = ref<CityEvent[]>([])
  let timer: number | null = null

  const refresh = async () => {
    const [b, r, e] = await Promise.all([ // 并行请求
      city3dApi.listBuildings(),
      city3dApi.listRegions(),
      city3dApi.listEvents(),
    ])
    if (b.success) buildings.value = b.data ?? []
    // ...
  }

  onMounted(() => {
    refresh()
    timer = window.setInterval(refresh, 5000) // 5 秒轮询
  })
  onScopeDispose(() => { if (timer) clearInterval(timer) })
}

```

6.7.5 5.7.5 建筑/区域/事件管理页 (与 fw100 同构)

三个管理页全部是标准 CRUD (列表 + 弹窗表单)，只是字段不同：- 建筑：名称、坐标 (x/y/z)、高度、颜色、状态。- 区域：名称、范围 (多边形点集)。- 事件：类型、位置、时间、级别、描述。

与 3D 联动：保存建筑/事件后刷新场景 (useCityData.refresh() 或直接重绘网格)。

6.7.6 5.7.6 Overview.vue (聚合统计)

```

<!-- 概览页: 后端 overview 接口聚合统计 -->
<template>
  <div class="overview">
    <el-statistic title="建筑总数" :value="overview.buildingCount" />
    <el-statistic title="区域总数" :value="overview.regionCount" />
    <el-statistic title="事件总数" :value="overview.eventCount" />
    <!-- 事件级别分布图表 (ECharts 饼图) -->
  </div>
</template>

```



```

<el-card> 事件级别分布 <ChartPie :data="levelDistribution" /> </el-card>
</div>
</template>

```

后端实现 (回忆 03 章): LEFT JOIN COUNT 聚合 + PaginationParams (page/page_size clamp 1..100)。

6.8 5.8 前端共性模式总结 (7 个应用的共同点)

6.8.1 5.8.1 七应用统一骨架

层	内容	7 应用一致
入口	main.ts (Pinia → Router → ElementPlus → mount)	是
根组件	App.vue (Navbar + router-view)	是
认证	createAuthStore 工厂 + initAuth	是
登录	LoginPage (shared 模板)	是
路由	守卫四步 + meta 权限	是
API	facade 组装 generated	是
类型	从 @shared 导入	是
会话	session.ts 统一 token key	是
构建	vite base /xxx/ + proxy /api	是

6.8.2 5.8.2 七应用差异对照表

应用	端口	数据源	WS	复杂度	独特技术
admin	5174	REST CRUD	无	中	双层权限中间件
fj200c_information	5173	串口/模拟	有 (事件分发)	高	composable 组装
fj200c_main	5179	三路串口	有 (单例+引用计数)	最高	ScaledPage/双主题/报表打印
ftj1c	5176	UDP 组播	有	中	IP 配置 16 路
fw100	5175	REST CRUD	无	低	最简单 (学习起点)
fw150	5178	REST CRUD	无	低	fw100 变体
city3d	5177	REST CRUD	无 (轮询)	中	Three.js 场景

6.8.3 5.8.3 典型页面三型

flowchart TD

P[页面类型] --> T1[列表型: 请求→表格→增删改弹窗]

P --> T2[实时型: WS→composable→表格/图表/仪表]

P --> T3[展示型: 静态内容/统计卡片]

T1 --> E1[fw100 Panel / admin Users / city3d Buildings]

T2 --> E2[fj200c_information Monitor / fj200c_main 三路面板 / ftj1c 帧表格]

T3 --> E3[Help / Overview / Visual]

判断新页面的类型 → 套对应模板写, 这是本项目的“前端开发心法”。

6.8.4 5.8.4 新增前端页面的标准步骤

- 1. router/index.ts 加路由 (path + component + meta.permissions)。
- 2. views/ 建页面组件 (复制同类页面改)。
- 3. 需要接口 → facade 加方法 (若后端有对应接口)。
- 4. npm run build 过 vue-tsc。
- 5. 导航栏菜单 (MENU_CONFIG) 若需要入口则加。

6.9 5.9 本章收官：前端七应用速查卡

想找什么	去哪里
登录/认证逻辑	@shared stores/auth + session
导航栏/菜单	@shared template/AppNavbar
请求封装	@shared api/index + custom-instance
类型	@shared types (re-export generated)
最简单的 CRUD	fw100 views/Panel.vue
最全的 CRUD	admin views/Users.vue
WS 组件级用法	fj200c_information composables
WS 单例用法	fj200c_main composables/useBackendPorts
大屏适配	fj200c_main ScaledPage
主题定制	fj200c_main styles/themes
3D	city3d composables/useCityScene
打印	fj200c_main 报表 (window.print)

05 章结束。你已经可以回答：7 个应用分别干什么、共性在哪、差异在哪、新页面怎么写。下一章讲类型同步机制——前后端如何从一份 Rust 代码生成全部前端类型，这是本项目工程化的核心。

6.10 5.10 admin 深入：CreateUser 页面（完整表单实战）

6.10.1 5.10.1 页面职责

```
<!-- frontend/admin/src/views/CreateUser.vue（结构） -->
<template>
  <div class="create-user">
    <el-page-header @back="router.back()" content="新建用户" />
    <el-card class="form-card">
      <el-form ref="formRef" :model="form" :rules="rules" label-width="100px">
        <el-form-item label="用户名" prop="username">
          <el-input v-model="form.username" placeholder="登录用户名" />
        </el-form-item>
        <el-form-item label="邮箱" prop="email">
          <el-input v-model="form.email" placeholder="user@example.com" />
        </el-form-item>
        <el-form-item label="密码" prop="password">
          <el-input v-model="form.password" type="password" show-password />
        </el-form-item>
        <el-form-item label="角色" prop="role">
          <el-select v-model="form.role" placeholder="选择角色">
            <el-option v-for="r in
authStore.roles" :key="r.key" :label="r.name" :value="r.key" />
          </el-select>
        </el-form-item>
      </el-form>
    </el-card>
  </div>
</template>
```

```

    <el-form-item>
      <el-button type="primary" :loading="submitting" @click="submit">创建</el-button>
      <el-button @click="router.back()">取消</el-button>
    </el-form-item>
  </el-form>
</el-card>
</div>
</template>

```

角色下拉数据来源：authStore.roles（运行时从 /api/meta/roles 拉取的角色注册表）——**不是硬编码**。新增角色后无需改前端代码，下拉自动出现新角色。

6.10.2 5.10.2 校验规则与提交

```

const rules: FormRules = {
  username: [
    { required: true, message: '请输入用户名', trigger: 'blur' },
    { min: 3, max: 32, message: '长度 3-32', trigger: 'blur' },
    { pattern: /^[a-zA-Z0-9_]+$/, message: '仅字母数字下划线', trigger: 'blur' },
  ],
  email: [
    { required: true, message: '请输入邮箱', trigger: 'blur' },
    { type: 'email', message: '邮箱格式不正确', trigger: 'blur' },
  ],
  password: [
    { required: true, message: '请输入密码', trigger: 'blur' },
    { min: 6, max: 32, message: '密码 6-32 位', trigger: 'blur' },
  ],
  role: [{ required: true, message: '请选择角色', trigger: 'change' }],
}

const submit = async () => {
  const valid = await formRef.value?.validate().catch(() => false)
  if (!valid) return
  submitting.value = true
  try {
    const res = await usersApi.create({ ...form.value })
    if (!res.success) throw new Error(res.message)
    ElMessage.success('用户创建成功')
    router.push('/users')
  } catch (e) {
    ElMessage.error((e as Error).message)
  } finally {
    submitting.value = false
  }
}

```

完整表单模式（本项目所有表单统一）：el-form :model :rules + validate() → 成功才提交 → submitting 防重复点击 → 成功跳转/刷新 → 失败 ElMessage。

6.10.3 5.10.3 编辑用户的差异

```

// 编辑页复用 CreateUser: 传入 id → 初始数据不同 → 提交调用 update
const loadUser = async (id: number) => {
  const res = await usersApi.info(id)
  if (res.success && res.data) form.value = { username: ..., email: ..., role: ... }
}

```

小技巧：新建/编辑共用一份表单组件，用 defineProps<{ userId?: number }> 区分模式。

6.11 5.11 fj200c_information 深入: Visual.vue (实时曲线)

6.11.1 5.11.1 页面结构

```
<template>
  <div class="visual-page">
    <div class="chart-toolbar">
      <el-select v-model="selectedMetric" placeholder="选择参数">
        <el-option v-for="m in metrics" :key="m.key" :label="m.label" :value="m.key" />
      </el-select>
      <el-checkbox v-model="autoScale">自动缩放</el-checkbox>
    </div>
    <div ref="chartRef" class="chart" />
  </div>
</template>
```

6.11.2 5.11.2 曲线数据流

// 参数切换 → 清空曲线 → 从 WS 快照重新开始

```
const selectedMetric = ref('ngSpeed')
watch(selectedMetric, () => {
  points.value = [] // 清空历史
  chart.value?.clear() // 清空图表
})
```

// WS 帧数据 → 曲线点

```
events.on((e) => {
  if (e.type === 'frame') {
    const v = (e.data as FramePayload)[selectedMetric.value] // 按参数取数
    if (typeof v === 'number') points.value.push({ time: Date.now(), value: v })
    if (points.value.length > 200) points.value.shift() // 限长 200 点
    chart.value?.setOption({
      series: [{ data: points.value.map((p) => p.value) }],
    })
  }
})
```

6.11.3 5.11.3 ECharts 实时曲线要点

要点	说明
setOption 增量更新	notMerge: false 只更新数据, 不重建图表
限长	环形缓冲防内存膨胀 (图表最多 200 点)
resize	窗口变化 chart.resize()
dispose	页面卸载销毁实例
参数切换	清空历史防新旧数据混连

6.12 5.12 fj200c_main 深入: 三路面板与高级功能

6.12.1 5.12.1 三路串口面板对照

面板	数据	命令	视觉
ECU	转速/水温/油压/油耗... (29 字段)	启动/停止/标定指令	仪表盘 gauge
ADAM	模拟量/数字量采集	读取通道	数字卡片
DYNO	测功机状态/扭矩	加载控制	进度条

6.12.2 5.12.2 命令发送通道 (与 fj200c_information 同构)

```
// useCommandChannel (fj200c_main 版)
const sendCommand = async (port: 'ECU' | 'ADAM' | 'DYNO', cmd: string) => {
  sending.value = true
  try {
    const res = await fj200cMainApi.sendCommand({ port, command: cmd })
    if (!res.success) throw new Error(res.message)
    ElMessage.success(`${port} 命令已发送`)
  } finally { sending.value = false }
}
```

后端对应： /api/fj200c_main/ecu/command 等三路独立接口 (AbstractCom 统一 trait, 前端感知三路差异)。

6.12.3 5.12.3 CSV 录制状态

```
<!-- 录制控制 (工具条) -->
<el-button :type="store.recording ? 'danger' : 'primary'" @click="toggleRecording">
  {{ store.recording ? '停止录制' : '开始录制' }}
</el-button>
<el-tag v-if="store.recording" type="warning">REC ●</el-tag>
```

后端： 64 列 CSV 按会话写入 csv/ 目录 ([CSV] Dir = csv 配置)。前端仅控制启停 + 展示状态。

6.12.4 5.12.4 试验信息与报表

```
<!-- 试验信息面板: 当前试验编号/名称/状态 -->
<el-descriptions title="当前试验" :column="3">
  <el-descriptions-item label="编号">{{ store.currentTest?.id }}</el-descriptions-item>
  <el-descriptions-item label="名称">{{ store.currentTest?.name }}</el-descriptions-item>
  <el-descriptions-item label="状态">
    <el-tag :type="store.currentTest?.completed ? 'success' : 'warning'">
      {{ store.currentTest?.completed ? '已完成' : '进行中' }}
    </el-tag>
  </el-descriptions-item>
</el-descriptions>

<!-- 报表按钮 -->
<el-button @click="generateReport">生成报表</el-button>
```

报表流程： 后端生成 HTML ([REPORT] StatePoints 指定状态点) → 前端新窗口打印。

6.12.5 5.12.5 仪表盘组件 GaugeCard

```
<script setup lang="ts">
// 复用组件: 一个仪表盘卡片
const props = defineProps<{
  title: string
  value: number
  unit?: string
  min?: number
  max?: number
}>()
const chartRef = ref<HTMLDivElement>()
onMounted(() => {
  chart.value = echarts.init(chartRef.value!)
  chart.value.setOption({
    series: [{
      type: 'gauge',
      min: props.min ?? 0,
      max: props.max ?? 100,
      data: [{ value: props.value, name: props.title }],
      // 仪表盘样式: 指针/刻度/颜色分段
    }]
  })
})
```

```

    }}
  })
})
watch(() => props.value, (v) => {
  chart.value?.setOption({ series: [{ data: [{ value: v }] }] })
})
onUnmounted(() => chart.value?.dispose())
</script>
<template>
  <div class="gauge-card">
    <div ref="chartRef" class="gauge" />
  </div>
</template>

```

三路面板复用同一组件，只是 props 不同——这就是“组件复用”的实际价值。

6.12.6 5.12.6 双主题切换的实现细节

```

// 主题 store/组合式
const theme = ref<'space' | 'dashboard'>('space')
const applyTheme = (t: 'space' | 'dashboard') => {
  document.documentElement.classList.toggle('theme-space', t === 'space')
  document.documentElement.classList.toggle('theme-dashboard', t === 'dashboard')
}
// 初始化：启动时读后端全局变量
const initTheme = async () => {
  const res = await fj200cMainApi.getTheme()
  if (res.success && res.data) { theme.value = res.data; applyTheme(res.data) }
}
// 切换：保存到后端（持久化）
const setTheme = async (t: 'space' | 'dashboard') => {
  theme.value = t
  applyTheme(t)
  await fj200cMainApi.setTheme(t)
}

```

注意与 Element Plus 暗色模式的区别：这是应用自有的两套视觉主题（航天蓝/仪表绿），不是亮暗模式。CSS 变量定义在根元素 class 上。

6.13 5.13 七个应用 vite.config.ts 对照

6.13.1 5.13.1 相同骨架

```

// 每个应用的 vite.config.ts（以 fj200c_information 为例）
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { fileURLToPath, URL } from 'node:url'

export default defineConfig(({ command }) => ({
  plugins: [vue()],
  base: command === 'build' ? '/fj200c_information/' : '/', // 生产子路径
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url)),
      '@shared': fileURLToPath(new URL('../packages/shared/src', import.meta.url)),
    },
  },
  server: {
    port: 5173, // 各应用不同
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,

```

```

      ws: true, // 需要 WS 的应用开启
    },
  },
},
build: { chunkSizeWarningLimit: 1500 }, // 大包不警告 (Element Plus 全量)
}))

```

6.13.2 5.13.2 七应用参数表

应用	port	base (build)	proxy ws
admin	5174	/admin/	否
fj200c_information	5173	/fj200c_information/	是
fj200c_main	5179	/fj200c_main/	是
fw100	5175	/fw100/	否
fw150	5178	/fw150/	否
ftj1c	5176	/ftj1c/	是
city3d	5177	/city3d/	否

6.13.3 5.13.3 为什么 ws: true 只在三个应用开

WS 代理是必需的：浏览器 WS 直连 517x 端口没有后端；代理转发 /api/*/ws 到 3000。没用到 WS 的应用不开代理选项（其实开也无妨，但项目按需配置更清晰）。

6.14 5.14 七个应用 router 对照（守卫差异）

6.14.1 5.14.1 统一守卫模板

```

// 每个应用 router/index.ts 的守卫（结构一致，路径不同）
router.beforeEach(async (to) => {
  const authStore = useAuthStore()
  if (!authStore.loaded) await authStore.initAuth() // ① 确保证初始化

  if (to.meta.requiresAuth && !authStore.isLoggedIn) {
    return { path: '/login', query: { redirect: to.fullPath } } // ② 未登录
  }
  if (to.meta.permissions && !authStore.hasAnyPermission(to.meta.permissions)) {
    return { path: '/login' } // ③ 无权限
  }
  return true // ④ 放行
})

```

6.14.2 5.14.2 各应用路由表

应用	路由
admin	/login, /users, /users/create, /dashboard
fj200c_information	/login, /monitor, /visual, /data, /config, /help
fj200c_main	/login, /main (三路面板) , /test (试验) , /report, /settings
ftj1c	/login, /monitor, /ipconfig, /help
fw100	/login, /panel, /detail/:id
fw150	/login, /panel, /detail/:id
city3d	/login, /view (3D) , /buildings, /regions, /events, /overview

6.14.3 5.14.3 登录后跳转逻辑 (LoginPage)

```
const handleLogin = async () => {
  const ok = await authStore.login(form.username, form.password)
  if (ok) {
    const redirect = route.query.redirect as string | undefined
    window.location.href = redirect ?? homePath // 支持登录前想去的页面
  }
}
```

redirect 参数: 被守卫拦下时记录目标路径, 登录成功跳回。

6.15 5.15 前端“测试”现状与构建即检查

6.15.1 5.15.1 项目无单元测试框架 (有意为之)

现状	说明
无 Vitest/Jest	项目未引入单测框架
构建即检查	npm run build = vue-tsc (类型) + vite build (打包)
类型即契约	前后端类型由 orval 生成, 编译不过 = 契约破坏

为什么够用: 项目是内部工具型系统, 核心风险在类型契约 (由 gen:api 链路保证) 而非纯函数逻辑。想加测试的起点: composables (纯逻辑) → Vitest。

6.15.2 5.15.2 手动验证清单 (改动前端后必做)

1. npm run build 通过 (类型 + 构建)。
2. dev 模式打开页面 → F12 Network 看 API 是否 200。
3. WS 页面 → Network → WS 面板看消息流。
4. 权限切换账号验证按钮/路由控制。
5. npm run gen:api 后如果类型变了, 全局搜报错点。

6.15.3 5.15.3 常见构建错误速查（前端）

报错	含义	处理
Module not found: Can't resolve '@shared/...'	alias 拼错	检查 import 路径
Element Plus: no matching component	组件名错	检查组件名
Property 'xxx' does not exist	类型缺字段	改类型或改调用
Cannot find module 'xxx.vue'	路径错	检查大小写/后缀
error TS2322: Type 'string' is not assignable to type 'number'	类型不匹配	改类型

6.16 5.16 shared 深入：types.ts 与 Permission 的 re-export

6.16.1 5.16.1 types.ts 做什么

```
// packages/shared/src/types.ts
// 前端类型唯一入口：从 generated 转发，不自己写
export type { Permission, RoleInfo, UserInfo, ... } from './api/generated/model'
export { Permission } from './api/generated/model' // 枚举（运行时用）
```

为什么转发一层：generated 的模型可能因 orval 配置调整而变（文件路径/命名），前端只认 @shared 稳定出口。Permission 是 enum（不是 type）——它同时是类型与运行时值：

```
// 使用方式
import { Permission } from '@shared'
authStore.hasPermission(Permission.UsersWrite) // 运行时值
const p: Permission = 'users.write'           // 类型
```

6.16.2 5.16.2 Permission 字符串值的约定

```
// generated/model/permission.ts (orval 生成，示意)
export enum Permission {
  SystemAdmin = 'system.admin',
  UsersRead = 'users.read',
  UsersWrite = 'users.write',
  UsersDelete = 'users.delete',
  Fj200cInformationMonitor = 'fj200c_information.monitor',
  Fj200cMainMonitor = 'fj200c_main.monitor',
  Fw100Monitor = 'fw100.monitor',
  Fw150Monitor = 'fw150.monitor',
  Ftj1cMonitor = 'ftj1c.monitor',
  City3dView = 'city3d.view',
}
```

对应后端：src/common/models.rs 的 Permission 枚举（#[derive(ToSchema)]）——**两端是同一份定义**（Rust 源头 → OpenAPI → TS）。

6.16.3 5.16.3 其他 re-export 的类型

```
export type {
  ApiResponse, // 统一响应包装 {success, message, data}
  CreateUserRequest, // 请求体
  LedgerItem, // 台账条目
  Fj200cMainEcuData, // ECU 数据
  // ... 所有 ToSchema 的 Rust 结构体
} from './api/generated/model'
```

ApiResponse 的泛型：ApiResponse<T> = { success: boolean; message: string; data: T | null }。所有接口返回它——前端判断 res.success 的统一依据。

6.17 5.17 前端文件速查表（按需求定位文件）

6.17.1 5.17.1 改菜单/导航

需求	文件
加菜单项	packages/shared/src/roles.ts MENU_CONFIG
改应用名	packages/shared/src/roles.ts ROLE_APP_NAMES
改导航栏样式	packages/shared/src/template/AppNavbar.vue

6.17.2 5.17.2 改登录流程

需求	文件
登录逻辑	@shared stores/auth.ts (login action)
登录页 UI	@shared template/LoginPage.vue
token 存取	@shared session.ts
401 跳转	@shared api/index.ts (拦截器)

6.17.3 5.17.3 改业务页面

需求	文件
加列表字段	对应 views/*.vue 表格列
加表单字段	表单组件 + facade 参数
加请求	对应 api/*.ts facade
加 WS 事件	types.ts + composables 分发

6.17.4 5.17.4 改样式主题

需求	文件
全局颜色	各应用 style.css (CSS 变量)
暗色模式	@shared template/AppNavbar.vue (切换逻辑)
航天主题	fj200c_main styles/themes.css

6.18 5.18 实战场景演练（5 个真实需求）

6.18.1 场景 1：给 fw100 加一个“备注”字段

后端：

1. src/fw100/service.rs 查询/插入 SQL 加 remark 列
2. src/fw100 的 DTO (models) 加 remark 字段 (ToSchema)

前端：

3. npm run gen:api → generated 自动多出 remark
4. Panel.vue 表格加一列 <el-table-column prop="remark" />
5. 弹窗表单加 el-input v-model="form.remark"
6. npm run build 通过

6.18.2 场景 2：导航栏加一个“系统公告”入口

1. packages/shared/src/roles.ts MENU_CONFIG 加 { key: 'notice', label: '系统公告', path: '/notice' }
2. admin src/views/Notice.vue 新建页面

3. admin router/index.ts 加路由 (meta.permissions 选个权限)
4. npm run build

6.18.3 场景 3: 监控页加一个 WS 数据字段

后端:

1. 帧解码 struct 加字段 (src/fj200c_information/decode.rs)
2. WS 事件 payload 加字段 → 不需要动 (帧结构整体推)

前端:

3. types.ts 帧类型加字段
4. Monitor.vue 表格加列
5. 若曲线需要 → Visual.vue metrics 列表加项

6.18.4 场景 4: 给 ftj1c 加第 17 路 IP 配置

后端:

1. src/ftj1c/models.rs IpConfig 列表长度改 17
2. config-ftj1c.ini 生成逻辑同步

前端:

3. IpConfig.vue 表格自然渲染 17 行 (数据驱动, 无硬编码)
4. 若固定渲染 16 → 找硬编码处改掉

6.18.5 场景 5: 新增一个权限 (如“数据导出”)

后端:

1. src/common/models.rs Permission 加 XxxExport
2. src/roles.rs 角色注册表加权限
3. 需要校验的接口加 permission_middleware

前端:

4. npm run gen:api (Permission 枚举更新)
5. 页面按钮 :disabled="!authStore.hasPermission(Permission.XxxExport)"
6. admin 界面即可配置角色权限 (注册表驱动)

五个场景覆盖了日常二次开发 90% 的需求——都是“改一处 → 生成 → 前端引用”的模式。

6.19 5.19 前端性能与体验细节 (7 应用通用)

6.19.1 5.19.1 表格性能

手段	适用
height 固定表格高度	大数据列表 (Monitor)
分页	列表 > 100 条 (admin 用户表)
字段裁剪	表格只显示必要列
虚拟滚动 (el-table-v2)	数千行 (当前未用, 可按需引入)

6.19.2 5.19.2 图表性能

- 限长缓冲 (100~200 点) ——防 DOM/Canvas 膨胀。
- setOption 增量更新而非整表重设。
- 卸载 dispose——防内存泄漏。
- 页面不可见时暂停动画 (document.hidden 判断, 可选)。

6.19.3 5.19.3 WS 数据量控制

层	手段
后端	事件节流 (200ms/50ms)、帧缓存
前端	只订阅需要的字段、限长缓冲、节流渲染
传输	JSON 压缩 (小帧无所谓)、token 查询参数

6.19.4 5.19.4 加载体验

- 页面级 v-loading + 按钮 loading 防重复点击。
- 首屏关键数据并行请求 (Promise.all)。
- 路由懒加载 (默认) → 首包小。
- 大依赖动态 import (打印库等)。

6.20 5.20 前端最佳实践清单 (写代码前对照)

6.20.1 5.20.1 必做清单

- [] 类型从 @shared 导入 (不手写后端类型副本)
- [] API 走 facade (不直接 import generated)
- [] 页面逻辑下沉 composable (>100 行 script 时)
- [] 跨组件状态用 store (不用 event bus 硬传)
- [] 资源清理 (定时器/WS/图表/dispose)
- [] 列表 :key 唯一、loading 完整、空态兜底
- [] 危险操作二次确认 (ElMessageBox)
- [] 表单 validate 通过才提交
- [] 网络异常 try/catch + 提示
- [] npm run build 通过 (类型即契约)

6.20.2 5.20.2 禁做清单

- [X] 子目录单独 npm install (pinia 双实例黑屏)
- [X] 直接改 packages/shared/src/api/generated/ (生成文件)
- [X] 手写 Permission 字符串 (用枚举)
- [X] 解构 store 丢响应式 (用 storeToRefs)
- [X] 组件内自己管 WS 不清理 (用 composable 模式)
- [X] 模板里写复杂逻辑 (抽 computed/函数)
- [X] 删除 shared 导出 (其他应用依赖)

6.21 5.21 七应用前端架构对照总结 (本日精华)

flowchart TD

```
subgraph 共性
    A1[main/App/Navbar/Login/guard/facade]
end
subgraph 实时监控族
    B1[fj200c_information<br/>composable 组装]
    B2[fj200c_main<br/>单例 WS + 引用计数]
    B3[ftj1c<br/>表格 + 轮询]
end
subgraph 管理族
    C1[admin 用户管理]
    C2[fw100/fw150 台账]
    C3[city3d 3D 管理]
end
共性 --> 实时监控族
共性 --> 管理族
```

一句话总结 05 章：7 个应用共享一套骨架 (shared)，差异只在业务数据形态 (REST 表数据 / WS 实时流 / 3D 场景)，页面按“列表型/实时型/展示型”三模板套写。

6.22 5.22 admin 深入：Dashboard 与系统信息

6.22.1 5.22.1 概览页结构

```
<!-- frontend/admin/src/views/Dashboard.vue (结构) -->
<template>
  <div class="dashboard">
```

```

<el-row :gutter="16">
  <el-col :span="8">
    <el-card>用户总数 <el-statistic :value="stats.userCount" /></el-card>
  </el-col>
  <el-col :span="8">
    <el-card>角色总数 <el-statistic :value="stats.roleCount" /></el-card>
  </el-col>
  <el-col :span="8">
    <el-card>当前在线 <el-statistic :value="stats.onlineCount" /></el-card>
  </el-col>
</el-row>
<el-card class="welcome">
  <h3>欢迎, {{ authStore.user?.username }}</h3>
  <p>当前角色: {{ currentRoleName }}</p>
  <p>拥有权限: {{ authStore.permissions.join(', ') }}</p>
</el-card>
</div>
</template>

```

概览页价值：新手一进来就能看“我有多少权限、什么角色”——这是验证认证链路的可视化窗口。

6.22.2 5.22.2 角色名展示（注册表映射）

```

const currentRoleName = computed(() => {
  const key = authStore.user?.role
  return authStore.roles.find((r) => r.key === key)?.name ?? key ?? '未知'
})

```

注意：显示名称从 `/api/meta/roles` 注册表映射，不硬编码中文名——后端改名自动同步。

6.23 5.23 深入：登录态与多应用跳转完整链路

6.23.1 5.23.1 链路总览

```

sequenceDiagram
    participant A as 应用 A (如 admin :5174)
    participant B as 应用 B (如 fj200c_information :5173)
    participant S as 后端 :3000

    A->>A: LoginPage 登录成功
    A->>A: setSessionToken(token) → localStorage
    A->>B: window.location.href = "http://localhost:5173/fj200c_information?token=xxx"
    B->>B: 启动 → initAuth()
    B->>S: 读 localStorage token (若同源则直接可用)
    B->>S: GET /api/auth/me (验证 token)
    S-->>B: {user, permissions}
    B->>B: 拉取角色注册表 → 导航栏就绪
    B-->>A: ✅ 免登录进入

```

6.23.2 5.23.2 dev 模式跨端口的特殊处理

问题：dev 模式各应用端口不同 (5173~5179)，localStorage 按“源”隔离——5173 存的 token 5174 读不到。

解决方案（项目实际做法）：1. 登录成功后整页跳转到目标应用，并把 token 通过 `?token=` 查询参数传递。2. 目标应用启动时 `initAuth` 检查 URL 参数 → 写入自己端口的 localStorage → 完成会话恢复。

```

// 各应用 App.vue 的 initAuth (示意)
async function initAuth() {
  const urlToken = new URLSearchParams(window.location.search).get('token')

```

```

if (urlToken) {
  setSessionToken(urlToken) // 写入本端口的 localStorage
  // 清理 URL (刷新页面不带 token)
  history.replaceState({}, '', window.location.pathname)
}
// 继续正常流程: validate token → 拉用户信息 → 拉注册表
}

```

6.23.3 5.23.3 prod 模式为什么不需要

生产环境 7 个应用由同一个后端托管 (同源同端口), localStorage 天然共享 → 跨应用跳转无需传 token, 直接共享登录态。

这就是 01 章说的“6 个用户端 + admin 共享同一登录态 (localStorage token), 跨应用跳转 token 自动传递”的实现细节。

6.23.4 5.23.4 登录失效的传导

token 过期 → 任一应用接口 401 → 拦截器清会话 (clearSession) → 跳该应用登录页。其他应用不受影响 (各自 initAuth 时才验证)。

6.24 5.24 深入: Element Plus 组件级封装 (复用模式)

6.24.1 5.24.1 常见封装清单

封装组件	位置	用途
GaugeCard	fj200c_main components	仪表盘卡片
ChartLine	各应用 (ECharts 折线)	实时曲线
ConfigDialog	fj200c_information components	配置编辑对话框
StatusTag	各应用	状态标签 (运行/停止)
AppNavbar	@shared template	全局导航栏

6.24.2 5.24.2 封装的原则

// 封装 = props 收参数 + emit 抛事件 + 内部维护自身状态
defineProps<{ title: string; value: number; unit?: string }>()
defineEmits<{ (e: 'click-more'): void }>()

什么时候该封装: 同一 UI 结构出现 ≥2 次。什么时候不该封装: 需求差异太大 (为封装而封装是负资产)。

6.25 5.25 各应用 index.html 与入口文件对照

6.25.1 5.25.1 index.html 模板

```

<!-- 每个应用 frontend/xxx/index.html -->
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>发动机监控系统</title> <!-- 各应用标题不同 -->
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>

```

6.25.2 5.25.2 七应用标题对照

应用	标题
admin	管理系统
fj200c_information	发动机监控系统
fj200c_main	发动机测控系统
ftj1c	UDP 通信监控
fw100	设备台账管理
fw150	设备台账管理
city3d	城市三维展示

浏览器标签页/后端托管时的 <title> 与文档标题可改——生产环境由后端内嵌 dist 托管，改标题只需改 index.html 重新构建。

6.26 5.26 前端工具函数（各应用 utils）

6.26.1 5.26.1 常见工具模块

工具	内容	使用
utils/format.ts	时间格式化、数字补零	表格时间列
utils/hex.ts	十六进制转换	ftj1c 帧展示
utils/ascii.ts	ASCII ↔ 字符串	串口帧解析
utils/responsive.ts	响应式判断	布局
utils/csv.ts	CSV 导出	Data 页
utils/theme.ts	主题切换	fj200c_main

6.26.2 5.26.2 工具函数示例

```
// utils/format.ts（示意）
export const pad2 = (n: number) => String(n).padStart(2, '0')
export const formatTime = (d: Date) => `${pad2(d.getHours())}:${pad2(d.getMinutes())}:${pad2(d.getSeconds())}`

// utils/hex.ts（示意）
export const bytesToHex = (bytes: number[]) => bytes.map((b) => b.toString(16).padStart(2, '0')).toUpperCase().join(' ')
```

6.27 5.27 前端开发环境设置（VS Code）

6.27.1 5.27.1 必备扩展

扩展	用途
Vue Language Features (Volar)	Vue SFC 语法支持
TypeScript Vue Plugin	模板类型检查
ESLint（如启用）	代码规范
Prettier	格式化

注意：Vue 3 项目必须用 Volar（不要装 Vetur，那是 Vue 2 的）。工作区设置：

```
// .vscode/settings.json (仓库根目录)
{
  "vue.server.hybridMode": true,
  "files.eol": "crlf", // Windows 项目统一 CRLF (与现有文件一致)
  "editor.formatOnSave": true
}
```

6.27.2 5.27.2 调试技巧

- **F12 浏览器 DevTools**: 断点、Network、Vue DevTools。
- **console.log + vscode 调试**: 快速定位。
- **Vue DevTools 组件面板**: 直接看 ref/state 当前值。
- **模拟移动端**: DevTools 设备模拟器。

6.27.3 5.27.3 常见工作流

```
# 开发循环: 改代码 → 浏览器热更新 (dev) → 最终 npm run build 验证
# 改 shared 代码 → 所有应用同时生效 (源码引用)
# 改后端接口 → npm run gen:api 更新类型 → 前端按报错修调用点
```

6.28 5.28 前端章节收官自测 (50 题精华 10 问)

1. shared 里角色注册表的 key/name/permissions 从哪来? (运行时 /api/meta/roles)
2. AppNavbar 的菜单如何过滤权限? (MENU_CONFIG 按权限过滤)
3. LoginPage 登录成功后为什么整页跳转? (路由 base + 跨应用 token 传递)
4. useBackendPorts 为什么用模块级单例? (切页不断连、防重复连接)
5. ScaledPage 的缩放原理? (固定设计稿 transform scale)
6. 双主题切换的本质? (CSS 变量 + 根元素 class 切换)
7. fw100 适合新手读的原因? (最简单 CRUD 闭环)
8. city3d 与实时监控族的最大差异? (无 WS, 5 秒轮询)
9. 新增前端页面的五个步骤? (路由/视图/facade/build/菜单)
10. dev 模式跨端口登录态如何传递? (?token= 参数 + initAuth 写入)

全部答对 → 05 章毕业。你已经具备阅读项目任意前端文件的能力。下一章深入类型同步——理解 7 个应用类型从何而来、如何“改一处全链路生效”。

6.29 5.29 深入: orval 生成的请求函数 (读懂 generated)

6.29.1 5.29.1 一个生成函数的解剖

```
// packages/shared/src/api/generated/api/fj200c-information.ts (orval 生成, 示意)
export const fj200cInformationServiceStatus = () => {
  return customInstance<ApiResponse<ServiceStatusResult>>({
    url: `/api/fj200c_information/service/status`,
    method: 'GET',
  })
}
```

结构规律: 1. 函数名 = tag + operationId (驼峰), 直接可读。2. 内部调用 customInstance (统一请求壳)。3. 返回类型 = ApiResponse<...> (解包后的 Promise)。4. 参数 → 直接拼进 url 或 requestBody。

6.29.2 5.29.2 带参数与请求体的函数

```
// 路径参数
export const fw100ItemsUpdate = (id: number, createLedgerItemRequest:
CreateLedgerItemRequest) => {
  return customInstance<ApiResponse<CreateLedgerItemRequest>>({
    url: `/api/fw100/items/${id}`,
```



```

    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    data: createLedgerItemRequest,
  })
}

```

6.29.3 5.29.3 生成文件的目录组织

```

packages/shared/src/api/generated/
├── api/
│   ├── auth.ts
│   ├── admin.ts
│   ├── fj200c-information.ts
│   ├── fj200c-main.ts
│   ├── fw100.ts / fw150.ts / ftj1c.ts / city3d.ts / meta.ts
├── model/
│   ├── permission.ts
│   ├── role-info.ts
│   ├── user-info.ts
│   ├── api-response.ts
│   └── ... 所有 ToSchema 结构体
└── custom-instance.ts # 壳（真正实例在 shared api/index.ts）

```

tags-split 模式：按后端 utoipa 的 tags 分组生成文件——后端加一个新 tag（新模块）→ generated 自动多一个文件。

6.29.4 5.29.4 为什么不能手改 generated

```

// ⚠️ 文件头部注释：此文件由 orval 自动生成，请勿手动修改
// npm run gen:api 会重新生成整个目录

```

任何手改都会在下次生成时丢失。要改生成逻辑 → 改 orval.config.ts（根目录）。

6.30 5.30 深入：WS 事件三种使用模式对照

6.30.1 5.30.1 模式一：组件级连接 (fj200c_information)

```

// 每个使用页自己 connect/close（页面少、逻辑简单时）
const events = useFj200cInformationEvents()
onMounted(() => events.connect())
onUnmounted(() => events.close())

```

适用：单页使用；页面切换可接受短暂断连重连。

6.30.2 5.30.2 模式二：模块级单例 (fj200c_main)

```

// 连接与整个应用共存亡（KeepAlive 切页不断连）
// useBackendPorts: refCount 计数，第一个用才建连，最后一个离开才断开

```

适用：多页面共享同一数据源；切换频繁；连接成本高（三路数据）。

6.30.3 5.30.3 模式三：事件总线解耦（跨模块）

```

// 连接只在一处（App.vue 或初始化模块），页面通过事件订阅
const syncBus = useEventBus('backend-data')
syncBus.on('update', (data) => ...)

```

适用：多个不相关模块都要消费同一事件流。本项目实际以模式一/二为主，事件总线用于主题/同步等零星事件。

6.30.4 5.30.4 三模式选择指南

```

flowchart TD
    Q[WS 数据的使用方式] --> C[多页面共享?]
    C --> B[切页需要保数据?]

```

B --> |是| M2[模块级单例+引用计数]
B --> |否| M1[组件级连接]
C --> |否| M1

6.31 5.31 深入：前端错误处理全景（续 4.47）

6.31.1 5.31.1 错误分层与兜底

flowchart TD

```
A[用户操作] --> B[表单校验失败<br/>提示具体字段]
A --> C[请求失败]
C --> D{HTTP 状态}
D --> |401| E[拦截器清会话跳登录]
D --> |403| F[ErrorMessage.error 无权限]
D --> |4xx/5xx| G[后端错误消息]
A --> H[代码异常<br/>try/catch + console.error]
```

6.31.2 5.31.2 后端错误消息的前端呈现

```
// 后端 AppError 的 message 通过 ApiResponse.message 透传
const res = await api.save(content)
if (!res.success) {
  ErrorMessage.error(res.message) // 后端说啥前端显示啥（如"配置格式错误"）
  return
}
```

设计默契：后端错误消息面向用户可读（中文），前端直接展示——不需要前端再翻译错误码。

6.31.3 5.31.3 loading/disabled 防止重复提交

```
// 所有提交按钮：submitting 期间禁用
<el-button :loading="submitting" :disabled="submitting" @click="submit">
// 后端幂等设计（如服务启停）进一步兜底
```

6.32 5.32 深入：前端应用复制改造指南（fw150 实例复盘）

6.32.1 5.32.1 fw150 是怎么来的

按 AGENTS.md「复制现有前端为新应用」流程：

1. 复制 frontend/fw100 → frontend/fw150
2. 全局替换：fw100 → fw150（包名/组件名/路径）
3. vite.config.ts: 端口 5175 → 5178, base /fw100/ → /fw150/
4. package.json: name @rustweb/fw100 → @rustweb/fw150
5. 根 package.json workspaces 加 frontend/fw150
6. api facade: fw100Api → fw150Api（generated 对应生成）
7. 后端：新 fw150 模块（handler/service/models）
8. npm run gen:api + npm run build 验证
9. deploy.bat 加构建步骤

6.32.2 5.32.2 复制改造的通用清单（新应用模板）

文件	改什么
vite.config.ts	port/base
package.json	name
index.html	title
api/*.ts	facade 函数名
router/index.ts	路由 path/meta
stores/auth.ts	createAuthStore id/loginPath/homePath
views/*	页面内容

6.32.3 5.32.3 复制改造的坑

1. **残留硬编码**：旧应用名出现在 import 路径/变量名 → 用全局搜索替换清理。
2. **端口冲突**：新端口没同步改 vite.config → dev 起不来。
3. **workspaces 漏加**：根 package.json 没加 → npm install 不安装依赖。
4. **权限不匹配**：路由 meta.permissions 用的还是旧权限 → 新角色进不来。
5. **后端路由没挂**：新模块 routes.rs 没注册 → 404。

对照 AGENTS.md 第 7 步流程逐项检查是唯一稳妥路线。

6.33 5.33 前端章节全量速查表（收藏级）

6.33.1 5.33.1 按“症状”找文件

症状	文件
页面白屏	main.ts / App.vue / router
登录不进	shared stores/auth + LoginPage
菜单不对	shared roles.ts MENU_CONFIG
按钮点了没反应	对应页面 script（看 loading/权限）
数据不动	WS composables（连接/事件）
表格列不对	对应 views/*.vue
样式乱	style.css / themes.css
构建报类型错	对应页面 + @shared types

6.33.2 5.33.2 按“改动”找文件

改动	文件
加菜单	shared roles.ts
加路由页	router + views
加接口	后端 handler + facade
加 WS 事件	types.ts + composables
改主题	themes.css + theme 逻辑
改权限	后端 roles.rs + generated
改端口	vite.config.ts

6.33.3 5.33.3 前端学习路径推荐

fw100 (CRUD) → admin (权限/表单) → fj200c_information (WS/composable)
→ ftj1c (轮询/配置) → fj200c_main (单例 WS/大屏/主题) → city3d (3D)

难度递增，每过一个应用你的“阅读理解能力”就上一个台阶。读代码顺序：index.html → main.ts → App.vue → router → api → composables → views。

6.34 5.34 深入：AppNavbar 的完整实现 (690 行拆解)

6.34.1 5.34.1 顶层结构

```
<template>
  <div class="navbar">
    <header class="navbar-top">
      <!-- 左：品牌区 (Logo + 应用名 + 当前应用标识) -->
      <!-- 右：暗色切换 / 用户下拉 (个人中心、退出) -->
    </header>
    <nav class="navbar-menu">
      <!-- 菜单区：水平菜单 (权限过滤后) -->
    </nav>
  </div>
</template>
```

6.34.2 5.34.2 菜单数据装配

```
// 计算可见菜单
const visibleMenus = computed(() => {
  // 从 MENU_CONFIG 里按用户权限过滤
  return MENU_CONFIG.filter((menu) => {
    // 无权限要求的直接显示
    if (!menu.permissions?.length) return true
    // 有权限要求的：任一命中即可
    return menu.permissions.some((p) => authStore.hasPermission(p))
  })
})
```

6.34.3 5.34.3 当前高亮

```
// 高亮 = 当前路径匹配菜单 path
const activeMenu = computed(() => {
  const path = route.path
  return visibleMenus.value.find((m) => path.startsWith(m.path))?.key ?? ''
})
```

6.34.4 5.34.4 用户菜单与退出

```
const handleLogout = async () => {
  await authStore.logout() // 清会话 + 清注册表缓存
  window.location.href = loginPath // 整页跳登录页
}
```

为什么用 location.href：退出后需要彻底重置应用状态 (store 内存态 + localStorage)，整页刷新最干净。

6.35 5.35 深入：Monitor 页全链路时序 (请求→WS→渲染)

```
sequenceDiagram
    participant U as 用户打开 /monitor
    participant R as 路由守卫
    participant A as App.vue
    participant M as Monitor.vue
    participant C as useFj200cInformationEvents
    participant B as 后端
```

```

U->>R: 导航
R->>A: initAuth (拉用户/权限/注册表)
R->>R: 校验登录 + 权限
R->>M: 放行
M->>C: onMounted → connect()
C->>B: WS 握手 (?token=)
B-->>C: 连接成功
B-->>C: 初始 table_data 快照
C-->>M: tableData.value = snapshot
M->>M: 表格渲染 (响应式)
loop 数据流
    B-->>C: frame 事件 (200ms 节流)
    C-->>C: 监听器广播
    C-->>M: rows 更新 → 表格刷新
end
Note over M: 用户点击"停止服务"
M->>B: POST /service/stop
B-->>M: {success}
M->>M: ElMessage + running=false

```

这张时序图回答了“一个实时监控页是怎么活起来的”——守卫 (权限) → 挂载 (连 WS) → 快照 (首屏) → 增量 (持续更新) → 操作 (控制命令)。

6.36 5.36 深入：Element Plus 国际化与语言

6.36.1 5.36.1 中文配置

```

// main.ts
import zhCn from 'element-plus/dist/locale/zh-cn.mjs'
app.use(ElementPlus, { locale: zhCn })

```

效果：分页器“共 x 条”、日期选择器星期、MessageBox 按钮文字都是中文。

6.36.2 5.36.2 项目内文字硬编码

项目业务文案 (按钮/提示) 直接写在模板中文——无 i18n 框架。内部系统够用；如要国际化，需要引入 vue-i18n 并迁移文案 (超出当前需求)。

6.37 5.37 深入：刷新页面状态恢复

6.37.1 5.37.1 刷新丢什么、留什么

状态	刷新后	机制
token/用户	保留	localStorage (session.ts)
角色注册表	重新拉取	initAuth
组件内 ref	丢失	无持久化 (页面重挂载)
store 数据	丢失	无持久化 (WS 会重连重新推送)
主题	保留 (fj200c_main)	后端 GlobalVar

6.37.2 5.37.2 为什么业务数据不持久化

WS 应用的数据是“实时流”，刷新后重连即可拿到最新——持久化旧数据反而误导。只有认证/配置类状态需要持久化。

6.38 5.38 深入：前端问题定位流程（故障排查指南）

6.38.1 5.38.1 页面/功能异常排查路径

flowchart TD

```
S[问题] --> Q1{是页面问题?}
Q1 -->|是| Q2{控制台报错?}
Q2 -->|是| E1[按报错修<br/>类型/语法/空值]
Q2 -->|否| Q3{数据问题?}
Q3 -->|是| Q4{Network 看请求}
Q4 -->|请求失败| E2[后端问题<br/>看后端日志]
Q4 -->|成功但数据不对| E3[检查 WS 连接/事件/字段]
Q3 -->|否| E4[样式/交互问题<br/>检查模板与 CSS]
Q1 -->|否| Q5{是构建/启动问题?}
Q5 -->|是| E5[vite config/端口/依赖]
```

6.38.2 5.38.2 记录问题信息（给协作方看）

环境: dev/prod

应用: admin / fj200c_information ...

页面: /monitor

现象: 表格 5 秒后卡住不刷新

复现: 启动服务 → 打开页面 → 等待

控制台: 无报错

Network: WS 消息持续到达（截图）

后端日志: RUST_LOG=debug 输出（附件）

排查三要素：控制台报错、Network 请求/WS 流、后端日志——三者对照即可定位绝大多数问题。

6.39 5.39 终极实战：给 ftj1c 加“帧搜索”功能（完整实操）

6.39.1 需求

帧监控表格加搜索框：按来源 IP 或帧类型过滤。

6.39.2 步骤 1：模板加搜索框

```
<!-- frontend/ftj1c/src/views/Monitor.vue -->
<el-input v-model="searchText" placeholder="按来源 IP 或类型搜索" clearable style="width: 280px" />
```

6.39.3 步骤 2：computed 过滤

```
const searchText = ref('')
const filteredFrames = computed(() => {
  const kw = searchText.value.trim().toLowerCase()
  if (!kw) return frames.value
  return frames.value.filter((f) =>
    f.sourceIp.toLowerCase().includes(kw) ||
    f.frameType.toLowerCase().includes(kw)
  )
})
```

6.39.4 步骤 3：表格换数据源

```
<el-table :data="filteredFrames" ...>
```

6.39.5 步骤 4：验证

npm run build # 类型通过

dev 打开 → 输入关键字 → 表格实时过滤 ✓

5 分钟完成——纯前端过滤（数据已有），无需后端改动。如果数据量大要服务端搜索，才需要加后端接口 + 查询参数。

6.40 5.40 05 章结束语

05 章「前端逐应用精读」至此完成。你现在拥有：

1. **地图**：7 个应用 + shared 的完整文件地图。
2. **解剖**：每个应用的典型页面怎么组装（composable/store/模板）。
3. **方法论**：列表型/实时型/展示型三模板 + 复制改造清单 + 排查路径。
4. **实战**：5 个场景演练 + 1 个完整实操，覆盖日常需求。

下一章讲**类型同步机制**——本项目工程化的灵魂：Rust 一份定义 → OpenAPI → TS 类型，两端永不脱节。

6.41 5.41 深入：前端与后端接口全对照（以应用为纲）

6.41.1 5.41.1 fj200c_information 接口对照表

前端 facade	HTTP	后端 handler	说明
getServiceStatus	GET /service/status	service_status	服务运行状态
startService	POST /service/start	service_start	启动服务（8 路）
stopService	POST /service/stop	service_stop	停止服务
sendCommand	POST /command	command	发送串口命令
getConfig	GET /config	config_get	读取 config.ini
saveConfig	POST /config/save	config_save	保存 config.ini（热加载）
getCsvRecords	GET /csv/records	csv_records	CSV 记录列表
downloadCsv	GET /csv/download	csv_download	下载 CSV 文件

注意：downloadCsv 返回的是文件流（非 ApiResponse）——facade 里做 Blob 处理：

// 下载文件的标准前端写法

```
const downloadCsv = async (name: string) => {
  const res = await api.fj200cInformationCsvDownload(name)
  // res 是 Blob → 触发浏览器下载
  const url = URL.createObjectURL(res as Blob)
  const a = document.createElement('a')
  a.href = url
  a.download = name
  a.click()
  URL.revokeObjectURL(url)
}
```

6.41.2 5.41.2 fj200c_main 接口对照表

前端 facade	HTTP	说明
serviceStatus / start / stop	/service/status /start /stop	三路串口服务
ecuCommand	/ecu/command	ECU 指令
adamCommand	/adam/command	ADAM 指令
dynoCommand	/dyno/command	DYNO 指令
configGet / save	/config /config/save	配置（重启生效）
csvRecords / start / stop	/csv/records /start /stop	64 列录制
testInfo / testInfoSave	/test/info /test/info/save	试验信息
generateReport	/report/generate	报表生成（HTML）
themeGet / set	/theme /theme/set	主题持久化
simulationToggle	/simulation/toggle	模拟开关

6.41.3 5.41.3 ftj1c 接口对照表

前端 facade	HTTP	说明
serviceStatus / start / stop	/service/*	服务
ipConfigGet	/ip/config	读取 16 路 IP
ipConfigSave	/ip/config/save	保存（重启生效）
configGet / save	/config*	config.ini
csvRecords	/csv/records	CSV

6.41.4 5.41.4 city3d 接口对照表（14 个）

前端 facade	HTTP	说明
buildingsList / create / update / delete	/buildings*	建筑 CRUD
regionsList / create / update / delete	/regions*	区域 CRUD
eventsList / create / update / delete	/events*	事件 CRUD
overview	/overview	聚合统计

共同规律：所有接口都返回 ApiResponse<T>；CRUD 都遵循 list/get/create/update/delete 命名（RESTful）；权限都靠 middleware 统一拦截。

6.42 5.42 深入：前端安全细节

6.42.1 5.42.1 安全边界认知




```
      B4[bcrypt 密码哈希]
    end
```

核心认知：前端权限控制是“体验”，后端权限控制才是“安全”。 任何人可以直接调 API（跳过前端 UI）——所以后端必须独立校验。

6.42.2 5.42.2 前端安全实践清单

项	做法
token	存 localStorage（项目现状；更严格可用 HttpOnly Cookie）
敏感操作	二次确认（ElMessageBox）
密码	不落日志、不存明文
XSS	Vue 默认转义插值（{{ }} 自动转义）；v-html 慎用
CSRF	JWT header 认证天然免疫（无 Cookie 自动携带）
文件下载	后端鉴权后才给文件

6.42.3 5.42.3 v-html 的使用纪律

```
<!-- ⚠️ v-html 会渲染 HTML：永远不要放用户输入 -->
<div v-html="adminProvidedHtml"></div>    <!-- 只用于后端生成的可信内容（报表预览） -->
```

项目实例：报表预览用 v-html 渲染后端生成的 HTML——数据源可信（自己生成），可接受；若渲染用户输入则必须消毒。

6.43 5.43 深入：前端配置管理三种模式

6.43.1 5.43.1 配置存储三处对照

配置	存储	生效时机	前端交互
config-fj200c_information.ini	文件	立即生效 （热加载）	Config 页保存 → 提示已生效
config-fj200c_main.ini	文件	需重启服务	保存 → 提示重启
config-ftj1c.ini	文件	需重启服务	保存 → 提示重启
主题（fj200c_main）	GlobalVar（DB）	立即	切换按钮
IP 配置（ftj1c）	文件 + DB	重启服务	IpConfig 页保存

6.43.2 5.43.2 前端如何提示“生效时机”

```
// Config.vue（fj200c_information）
const save = async () => {
  const res = await api.saveConfig(content)
  ElMessage.success('配置已保存（立即生效）') // 热加载无重启
}

// IpConfig.vue（ftj1c）
const saveAll = async () => {
  const res = await api.saveIpConfig(list)
  ElMessageBox.confirm('配置已保存，需重启服务生效。现在重启？', '提示')
    .then(async () => { await service.restart(); })
    .catch(() => {})
}
```

提示文案 = 后端行为的用户可见化——改配置时用户永远知道下一步会发生什么。

6.44 5.44 深入：七应用 index.html / package.json 对照

6.44.1 5.44.1 package.json 依赖差异

依赖	admin	fj200c_information	fj200c_main	ftj1c	fw100	fw150	city3d
vue/ vue- router/ pinia	✓	✓	✓	✓	✓	✓	✓
element- plus	✓	✓	✓	✓	✓	✓	✓
echarts	—	✓	✓	—	—	—	—
three	—	—	—	—	—	—	✓
@vueuse/ core	✓	✓	✓	✓	✓	✓	✓

规律：基础四件套（vue/router/pinia/element-plus）全应用必备；可视化库按需（echarts 在监控应用，three 只在 city3d）。

6.44.2 5.44.2 依赖版本管理

根目录 package.json workspaces 统一安装：

```
{
  "workspaces": [
    "packages/shared",
    "frontend/admin",
    "frontend/fj200c_information",
    "frontend/fj200c_main",
    "frontend/fw100",
    "frontend/fw150",
    "frontend/ftj1c",
    "frontend/city3d"
  ]
}
```

改依赖：npm install <pkg> -w frontend/fj200c_main（或根目录直接装）。**严禁**子目录单独 npm install。

6.45 5.45 深入：前端常用组件模式汇总（写新组件参考）

6.45.1 5.45.1 组件分类与模板

组件类型	核心模式	项目实例
展示型	props 驱动渲染	GaugeCard / StatusTag
表单型	v-model + rules	ConfigDialog
容器型	slot 组合	Panel / Card
数据型	内部请求 + loading	各列表页
控制型	事件 + store 联动	启停按钮

6.45.2 5.45.2 展示型组件标准模板

```
<script setup lang="ts">
defineProps<{ value: number; unit?: string }>()
</script>
<template>
  <div class="stat-value">{{ value }} <span v-if="unit" class="unit">{{ unit }}</span></div>
</template>
```

6.45.3 5.45.3 表单型组件标准模板

```
<script setup lang="ts">
const model = defineModel<string>() // Vue 3.4+ 简写 v-model
const rules = { required: true, message: '必填' }
</script>
<template>
  <el-input v-model="model" />
</template>
```

注意：defineModel 是 Vue 3.4 新 API，项目若用 3.4+ 可简化双向绑定；低版本用 props + emit('update:modelValue') 模式（04 章 4.23 写法）。

6.46 5.46 深入：前端构建产物与部署形态

6.46.1 5.46.1 构建输出

```
# 每个应用
npm run build
# 输出 dist/（7 个应用各一份）
```

6.46.2 5.46.2 部署到后端（embedded 模式）

```
cargo build --release --features embedded
# rust-embed 把 7 个 dist 编译进 exe
# 运行时访问 /admin → 后端内存服务 admin 的 dist
# 深链接回退：/admin/users 刷新 → 回退 index.html（SPA 路由接管）
```

6.46.3 5.46.3 深链接回退的意义

```
// 后端 embedded_router: 未匹配静态文件 → 返回该应用的 index.html
// 前端 createWebHistory 路由才不 404
```

前端历史路由 + 后端回退是 SPA 部署的黄金组合——01 章讲过的知识点在这里闭环。

6.47 5.47 05 章最终自测（30 题随机抽 10）

1. shared 的 roles.ts 里有什么？注册表数据从哪来？
2. buildWebSocketUrl 为什么带 token？为什么不用 header？
3. createApiClient 的参数 (loginPath) 干什么用？
4. createAuthStore 工厂模式的好处？
5. AppNavbar 菜单怎么按权限过滤？
6. fj200c_information 的 Monitor 页由几个 composable 组装？
7. useBackendPorts 引用计数的意义？
8. ScaledPage 为什么用 transform scale 而不是改尺寸？
9. fw150 是怎么来的（历史）？与 fw100 的差异？
10. city3d 为什么不用 WS？用什么替代？
11. 前端接口全部返回什么包装类型？
12. 文件下载的前端标准写法？
13. 前端权限控制与后端权限控制的关系？

14. 配置生效时机的三种情况？
15. 新增前端页面的五个步骤？
16. 生产环境如何托管 7 个应用？(embedded + 路由回退)
17. 排查前端问题三要素？
18. orval generated 为什么不能手改？
19. 前端安全边界在谁那？
20. 组件封装的判断标准？

答对 18+ → 05 章精通。05 章到此正式完结。

6.48 5.48 深入：composable vs store（状态管理的选择）

6.48.1 5.48.1 什么时候用哪个

flowchart TD

```

Q[状态要跨组件/页面共享吗?] -->|是| S[Pinia store]
Q -->|否| C[有副作用/生命周期吗?]
C -->|是（定时器/WS/请求）| M[composable]
C -->|否（纯派生）| P[组件内 ref/computed]
  
```

场景	选择	理由
认证/用户/权限	store	全局 + 持久化
业务数据（三路 ECU）	store	多页面多组件共享
时钟/WS/轮询	composable	生命周期绑定
页面局部状态	组件内	最小范围

6.48.2 5.48.2 两者结合的典型形态

```

// store 存数据，composable 管连接（fj200c_main 的形态）
// useBackendPorts: WS 数据 → 写进 store
export function useBackendPorts() {
  const store = useDashboardStore()
  // WS onmessage → store.setEcuData(payload)
  // 组件读 store 而非 composable
}
  
```

分工原则：composable 管“怎么拿数据”，store 管“数据放哪”。

6.49 5.49 深入：TypeScript 类型安全实践（as 的使用纪律）

6.49.1 5.49.1 项目中的类型断言用法

```

// ① 后端数据（可信，orval 已定型）——一般不需要 as
const res = await api.getData() // ApiResponse<Data>

// ② WS 事件（手写类型，JSON.parse 无类型）——必须收窄
const event = JSON.parse(ev.data) as Fj200cInformationWsEvent

// ③ 宽类型收窄——用判别
if (event.type === 'frame') {
  const frame = event as Extract<Fj200cInformationWsEvent, { type: 'frame' }>
}
  
```

6.49.2 5.49.2 应避免的 as

```

// ⚠️ 滥用 as 会掩盖真实类型错误
const x = unknownValue as number // 危险：运行时可能不是数字
// ✅ 用类型守卫
const x = typeof unknownValue === 'number' ? unknownValue : 0
  
```

纪律：as 只用于“编译器不知道但运行时确定”的边界 (JSON.parse、WS、跨库)，业务代码里少用。

6.50 5.50 七应用菜单与布局一览

6.50.1 5.50.1 菜单结构 (MENU_CONFIG 现行)

应用	菜单项
admin	用户管理
fj200c_information	监控 / 可视化 / 数据记录 / 配置 / 帮助
fj200c_main	主控台 / 试验 / 报表 / 设置
ftj1c	帧监控 / IP 配置 / 帮助
fw100	台账 / 详情
fw150	台账 / 详情
city3d	3D 视图 / 建筑 / 区域 / 事件 / 概览

6.50.2 5.50.2 布局形态

应用	布局
admin	顶栏 + 水平菜单 + 内容区
fj200c_information	顶栏 + 水平菜单 + 内容区 (表格为主)
fj200c_main	顶栏 + 面板网格 (大屏三路)
ftj1c	顶栏 + 水平菜单 + 内容区
fw100/fw150	顶栏 + 内容区
city3d	顶栏 + 3D 画布 + 侧边面板

共性：AppNavbar 顶栏 + router-view 内容区——7 个应用布局同源，差异只在内容区。

6.51 5.51 前端高频疑问解答 (FAQ)

6.51.1 Q1：为什么有的页面请求 404？

可能原因：

- 1. 后端没启动 (ERR_CONNECTION_REFUSED)
- 2. 后端路由没注册 (routes.rs 遗漏)
- 3. 路径写错 (前端 facade url 与后端 path 不一致)
- 4. 权限不足 (403 而非 404)

排查：Network 看完整 URL → 对照 openapi.json 的 paths

6.51.2 Q2：为什么 WS 连不上？

可能原因：

- 1. 后端服务没启动
- 2. token 失效 (?token= 过期/清空)
- 3. dev 模式 proxy ws: true 没开
- 4. 后端 WS 路由 handler 校验失败 (查后端日志)

排查：Network → WS → 看握手状态码

6.51.3 Q3：为什么改了 shared 代码没生效？

@shared 指向源码，Vite dev 会热更新。若没生效：

- 1. 确认 alias 指向 packages/shared/src (不是 dist)

- 2. 确认 import 路径大小写
- 3. 重启 dev server（缓存特殊情况）

6.51.4 Q4: 为什么 npm run build 报一堆类型错？

最常见：改了后端 DTO 忘了跑 npm run gen:api
→ 跑 gen:api 生成最新类型 → 按报错逐一更新调用点

6.51.5 Q5: 怎么知道接口有哪些参数/返回值？

- 1. 浏览器访问 http://localhost:3000/api-docs/openapi.json
 - 2. 或看 packages/shared/src/api/generated/model/ 的类型
 - 3. 或看后端 handler 的 #[utoipa::path] 注解
- 三处同源（Rust DTO → OpenAPI → TS）

6.52 5.52 05 章完结：学习成果清单

读完 05 章，你应该能：

能力	验证方式
说出 7 个应用的职责与端口	闭卷默写
指出 shared 每个文件的用途	口头叙述
解释 Monitor 页全链路时序	画时序图
说出 WS 三种连接模式与选择	场景判断
完成复制改造指南	对照清单执行
按故障排查路径定位问题	模拟排查
新增一个页面/接口/菜单	动手实操

做不到的回到对应小节重读。05 章结束，进入 06 章——前后端类型同步机制，理解“ Rust 一份定义，前端全部类型”的工程核心。

6.53 5.53 深入：组件通信模式在项目中的真实分布

6.53.1 5.53.1 通信方式统计与典型场景

方式	使用频率	典型场景
props 下传	★★★★★	GaugeCard 传值、面板传配置
emit 上传	★★★★★	按钮点击、表单提交、对话框开关
v-model	★★★★	表单字段、对话框 visible
Pinia store	★★★★	认证、三路数据、主题
composable 返回	★★★	Monitor 页组装
provide/inject	★★	深层配置（少用）
useEventBus	★	主题切换、跨模块通知

6.53.2 5.53.2 props 设计原则（GaugeCard 案例）

```
// 组件 props 设计三问：  
// 1. 必需的最小输入是什么？→ title/value  
// 2. 哪些有合理默认值？→ min=0 max=100 unit=undefined  
// 3. 哪些不该由 props 控制？→ 内部样式细节  
const props = withDefaults(defineProps<{  
  title: string  
  value: number  
  unit?: string
```

```
min?: number
max?: number
}>(), { min: 0, max: 100 })
```

withDefaults 语法：可选 props 的默认值（04 章提过，这里实战）。

6.54 5.54 深入：前端节流/防抖/限流应用清单

6.54.1 5.54.1 全项目用到的限流手段

位置	手段	目的
后端 WS 广播	200ms/50ms 节流	降低推送频率
前端搜索框	防抖 300ms（可选）	减少请求
图表更新	限长缓冲 + setOption	控制渲染
服务轮询	定时器 5s（city3d）	控制请求频率
表格滚动	分页/固定高度	控制 DOM

6.54.2 5.54.2 何时用防抖 vs 节流

flowchart TD

```
Q[场景] --> S{持续高频触发?}
S -->|搜索输入| D[防抖：停顿后执行一次]
S -->|滚动/拖拽| T[节流：固定间隔执行]
S -->|WS 数据| B[节流/丢弃：控制频率]
```

6.55 5.55 深入：响应式与样式的细节规范

6.55.1 5.55.1 移动端适配的三个应用

应用	适配策略
fw100/fw150	useResponsive（<768px 切换布局）
fj200c_information	表格横向滚动 + 工具条换行
fj200c_main	ScaledPage 固定设计稿（大屏专用）

6.55.2 5.55.2 CSS 变量分层

```
/* 第一层：设计令牌（token）——颜色/字体/间距 */
:root {
  --color-primary: #409eff;
  --spacing-unit: 8px;
}
/* 第二层：组件样式（引用 token） */
.toolbar { padding: var(--spacing-unit); background: var(--color-bg, #fff) }
/* 第三层：覆盖（媒体查询/主题） */
@media (max-width: 768px) { .toolbar { flex-wrap: wrap } }
```

三层结构让主题与响应式都只需改变量/覆盖——fj200c_main 双主题就是这个思路的极致。

6.55.3 5.55.3 scoped 与全局样式的边界

样式类型	位置	用途
全局重置	style.css	body/margin、通用类
组件样式	<style scoped>	组件内细节
主题变量	themes.css / style.css :root	主题切换
Element 覆盖	:deep()	微调组件库

6.56 5.56 05 章收官：终极综合自测

6.56.1 场景题：给 fj200c_information 加“历史数据回放”页

要求：从 CSV 记录选择一条 → 以 1 秒间隔回放表格数据（模拟实时）。

设计思路（5 分钟脑内规划）：

1. 页面：views/Playback.vue（列表型 + 播放控制）
2. 数据：CSV 记录列表已有接口（getCsvRecords）
3. 回放：读取 CSV 内容（后端 csv/records 详情或本地解析）
4. 定时推进：setInterval 1 秒推进一行 → 表格更新
5. 控制：播放/暂停/停止（一个 ref + interval）
6. 路由：router 加 /playback + meta.permissions 同 Monitor
7. 菜单：MENU_CONFIG 加入入口
8. 验证：npm run build

这道题覆盖了：页面三型识别（列表型+控制）、composable（回放逻辑可抽 usePlayback）、路由权限、菜单、构建验证——05 章全部核心能力。

答案参考：

```
// usePlayback.ts（核心逻辑示意）
export function usePlayback(rows: Ref<TableRow[]>) {
  const playing = ref(false)
  const index = ref(0)
  let timer: number | null = null

  const play = () => {
    if (playing.value) return
    playing.value = true
    timer = window.setInterval(() => {
      index.value++
      if (index.value >= rows.value.length) stop() // 播完自动停
    }, 1000)
  }

  const pause = () => { playing.value = false; if (timer) clearInterval(timer) }
  const stop = () => { pause(); index.value = 0 }
  const current = computed(() => rows.value[index.value])

  onUnmounted(pause)
  return { playing, index, current, play, pause, stop }
}
```

能独立写出这段代码（含清理与边界）→ 05 章毕业。

6.57 5.57 补充：跨应用细节——会话与跳转机制再梳理

6.57.1 5.57.1 登录态的三个环节

- 1. 登录成功 → setSessionToken(token) → localStorage
- 2. 任意应用启动 → initAuth() → 验证 token + 拉用户/权限
- 3. 401 拦截 → clearSession() → 跳登录页

6.57.2 5.57.2 dev 与 prod 的差异 (重点)

场景	token 传递方式
dev 同端口应用内刷新	localStorage 直接可用
dev 跨端口跳转 (5173→5174)	无法共享 localStorage → URL ?token= 传递
prod 同源托管 (3000 端口)	localStorage 天然共享 → 无需传参

结论：7 应用共享登录态在 prod 是“免费”的，dev 需要 ?token= 配合。

6.57.3 5.57.3 跳转后如何回到原页面

```
// 登录前想去的页面存在 query 里
router.beforeEach((to) => {
  if (!isLoggedIn) return { path: '/login', query: { redirect: to.fullPath } }
})
// 登录成功后跳回
const redirect = route.query.redirect as string | undefined
window.location.href = redirect ?? homePath
```

6.58 5.58 补充：各应用「登录页」的差异

应用	登录页	说明
admin	shared LoginPage	标准模板
fj200c_information	shared LoginPage	标准模板
fj200c_main	shared LoginPage (航天主题版)	主题差异化
其余	shared LoginPage	标准模板

共性：全部复用 shared 的 LoginPage (683 行)，只换 homePath。这就是“共享模板”的价值——登录页 7 处一致，改一处全改。

6.59 5.59 补充：前端「主题/暗色」全梳理

应用	能力	实现
全部	暗色模式	html.dark + Element Plus dark 变量
fj200c_main	航天/仪表双主题	CSS 变量组 + class 切换 + 后端持久化
各应用	自定义样式	style.css 的 CSS 变量

切换按钮位置：AppNavbar 用户下拉 (暗色)；fj200c_main 设置页 (双主题)。

6.60 5.60 补充：前端数据流异常排查清单

症状	排查点	修复
表格空	WS 未连/未推送	看 Network WS 面板
数据卡住	连接断开未重连	检查 onClose 重连逻辑
数值 NaN	字段名对不上	查 types.ts 字段名
图表不更新	watch 没触发	检查 deep/immediate
命令无响应	后端没处理	看后端日志

通用三问：数据到了吗 (WS)? 分发了吗 (composable)? 渲染了吗 (模板)?

6.61 5.61 补充：从零读一个陌生前端应用的五步法

flowchart TD

```
A[1. index.html + main.ts<br/>入口装配] --> B[2. router<br/>有哪些页面]
B --> C[3. api facade<br/>调哪些接口]
C --> D[4. composables/stores<br/>数据怎么流动]
D --> E[5. views<br/>页面怎么组装]
```

与后端五步法对称：入口 → 路由 → 接口 → 状态 → 视图。任何前端应用 30 分钟可读完。

6.62 5.62 补充：05 章知识自测 (追加 10 题)

1. 7 个应用的共享骨架是什么? (main/App/Navbar/Login/guard/facade)
2. LoginPage 登录成功为何整页跳转?
3. useBackendPorts 的 refCount 逻辑?
4. ScaledPage 的缩放公式?
5. city3d 为什么轮询而非 WS?
6. 新增前端页面的五步骤?
7. dev 跨端口 token 怎么传?
8. 暗色模式 vs 双主题的区别?
9. 前端数据流排查三问?
10. 陌生应用五步法是什么?

答对 8+ → 05 章精通。

6.63 5.63 补充：shared 里认证流程的完整时序

6.63.1 5.63.1 initAuth 细节

sequenceDiagram

```
participant A as App.vue
participant S as authStore
participant B as 后端
A->>S: onMounted → initAuth()
S->>S: 读 localStorage token
S->>B: GET /api/auth/me (验证 token)
B-->>S: {user, permissions}
S->>B: GET /api/meta/roles (拉注册表)
B-->>S: {roles}
S->>S: loaded = true
S-->>A: 守卫可继续
```

顺序关键：先验证身份再拉注册表——注册表失败不影响登录 (权限空兜底)。

6.63.2 5.63.2 logout 做了什么

1. 清 localStorage (token)
2. 清 store 状态 (user/permissions/roles)
3. window.location.href = loginPath (整页重置)

6.63.3 5.63.3 token 过期的时间线

登录 → token 有效期 24h (JWT_EXPIRATION)
→ 过期后首个请求 401
→ 拦截器清会话 + 跳登录
→ 用户重新登录

6.64 5.64 补充: Monitor 页的表格渲染性能细节

6.64.1 5.64.1 数据更新策略

```
// 每帧更新: 直接替换数组引用 (不是逐行 patch)
rows.value = newRows
// el-table 收到新数组 → 全部重渲染 (数据量大时卡)
// 优化方向: 只更新变化的行 (Object.freeze/行级 key)
```

6.64.2 5.64.2 限长策略

```
// 只保留最近 N 行 (表格滚动窗口)
if (rows.value.length > 200) rows.value.splice(0, rows.value.length - 200)
```

6.64.3 5.64.3 何时需要虚拟滚动

数据 >1000 行 + 频繁更新 → 考虑 el-table-v2 虚拟滚动
数据 <200 行 → 现状即可

6.65 5.65 补充: fj200c_main 三路面板的状态共享细节

6.65.1 5.65.1 store 与面板的关系

dashboard store: 三路数据统一存放
ECU 面板: 读 store.ecu + 发 ECU 命令
ADAM 面板: 读 store.adam
DYN0 面板: 读 store.dyno
(三块面板互不直接通信, 全走 store)

6.65.2 5.65.2 为什么用 store 而不是 props

三路面板在不同页面/层级 → props 传递繁琐
store 全局 → 任何组件随时读写
切页不丢数据 (store 常驻)

6.66 5.66 补充: ftj1c 前端与后端的数据交互细节

6.66.1 5.66.1 两种数据通道

1. HTTP: IP 配置读写、服务启停、CSV 列表
2. WS: 帧数据流 (解码后的字段)

6.66.2 5.66.2 帧表格的更新频率

后端 50ms 节流 → 前端每秒约 20 行
→ 表格限长 100 行 (滚动窗口)
→ 图表 (如有) 同样限长

6.66.3 5.66.3 坐标转换的展示

后端 CGCS2000 转换 → 前端展示转换前后坐标
(前端不参与计算, 纯展示)

6.67 5.67 补充: fw100/fw150 的响应式细节

6.67.1 5.67.1 useResponsive 的作用

```
// <768px 视为移动端
const isMobile = computed(() => width.value < 768)
// 模板: <div :class="{ 'mobile-layout': isMobile }">
```

6.67.2 5.67.2 移动端适配的取舍

台账类页面: 列隐藏 + 卡片式展示 (移动端)
监控类页面: 横向滚动 (数据密度高, 不适合移动端深度适配)

6.68 5.68 补充: city3d 的 3D 与 2D 页面混合

6.68.1 5.68.1 页面关系

3D 视图页: Three.js 场景 (全屏画布)
管理页: 2D 表格 (建筑/区域/事件)
概览页: 2D 统计
(3D 页与 2D 页切换 → 场景销毁重建)

6.68.2 5.68.2 3D 场景与数据的联动

编辑建筑 → 保存 → 轮询拉到新数据 → 3D 场景重建网格
(5 秒轮询保证 3D 与后台一致)

6.69 5.69 补充: 05 章补充自测 (追加 10 题)

1. initAuth 的三步顺序?
2. token 过期的时间线?
3. 表格更新的性能策略?
4. 何时需要虚拟滚动?
5. 三路面板为什么用 store?
6. ftj1c 的两条数据通道?
7. 坐标转换在前端还是后端?
8. useResponsive 的断点?
9. 3D 场景何时销毁重建?
10. logout 的三步?

答对 8+ → 05 章补充完成。

6.70 5.70 深入: Vite 配置文件的完整解读

6.70.1 5.70.1 全量注释版

```
// frontend/fj200c_information/vite.config.ts (结构注释)
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { fileURLToPath, URL } from 'node:url'

export default defineConfig({
  plugins: [vue()],
  // 构建时 base = /fj200c_information/, 开发时 /
  base: process.env.NODE_ENV === 'production' ? '/fj200c_information/' : '/',
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url)),
      '@shared': fileURLToPath(new URL('../../packages/shared/src', import.meta.url)),
    },
  },
  server: {
    port: 5173,
  },
})
```

```

strictPort: true,    // 端口被占直接报错
proxy: {
  // 开发时把 /api 转发到后端
  '/api': { target: 'http://localhost:3000', ws: true },
},
},
})

```

6.70.2 5.70.2 关键点逐条

1. base 与生产路径严格对应（否则资源 404）
2. strictPort 保证端口固定（7 应用互不冲突）
3. alias 是 @ / @shared 的来源
4. proxy ws:true 让 WS 请求也能转发

6.71 5.71 深入：生产环境前端如何被服务

6.71.1 5.71.1 两种模式

模式	前端来源	何时用
embedded	编译期内嵌 exe	单文件部署
dist-*	磁盘目录读取	开发/调试

6.71.2 5.71.2 embedded 的服务方式

浏览器请求 /fj200c_information/xxx
 → 后端 embedded_router 查内嵌文件
 → 命中 → 返回文件
 → 未命中（SPA 路由）→ 返回 index.html

6.71.3 5.71.3 为什么能“改前端不用重启后端”（开发）

dev 模式读磁盘 dist → 修改 dist 直接生效（不用重启 exe）
 embedded 模式内嵌 → 必须重新编译

6.72 5.72 深入：多应用共享登录态的边界

6.72.1 5.72.1 共享的前提

1. 同一域名/端口？—不同应用在不同端口，但**同域**（localhost）
2. localStorage 按 origin 隔离 — localhost:5173 与 localhost:5174 其实不同源！

真相：7 个 dev 应用在 7 个端口 → localStorage 其实**不共享**？——不，项目约定「同一 localStorage 登录态」指的是**生产环境**（同一域同一端口，路径不同）或通过 initAuth 自动登录兜底。开发时各端口各自登录。

6.72.2 5.72.2 跨应用跳转的实现

```

// ROLE_APP_URLS 里存的完整地址
const appUrl = ROLE_APP_URLS[role]           // http://localhost:5174
window.location.href = appUrl

```

6.72.3 5.72.3 登录态校验

跳转后目标应用 initAuth → 读自己 localStorage
 → 无 token → 登录页（用户重新登录）
 → 有 token → 直接进主页

6.73 5.73 深入：状态管理的分层（store 该放什么）

数据	放哪	理由
用户信息	store	全局需要
服务状态	store	多组件共享
帧数据	store	多组件共享
页面局部表单	组件内 ref	不跨页面
图表配置	组件内	单组件使用

口诀：跨页面才进 store，单页面留组件。

6.74 5.74 深入：composable 设计模式（useXxx 的提炼）

6.74.1 5.74.1 什么时候抽 composable

1. 两个以上组件复用同一逻辑
2. 逻辑与 UI 无关（可测试）
3. 生命周期相关（onMounted 等）

6.74.2 5.74.2 典型结构

```
// useService.ts（服务状态轮询）
export function useService(appKey: string) {
  const status = ref('stopped')
  const load = async () => { status.value = await api.getStatus() }
  watch(status, ...) // 状态变化触发联动
  onMounted(load) // 进入页面自动拉
  return { status, load }
}
```

6.74.3 5.74.3 命名约定

useXxx 前缀 + 返回对象解构使用
文件名与函数名一致（useService.ts → useService）

6.75 5.75 深入：样式方案与布局细节

6.75.1 5.75.1 样式方案对比

1. 全局样式（src/assets/*.css）—通用
2. scoped 样式（<style scoped>）—组件隔离
3. Element Plus 主题变量（CSS 变量覆盖）—统一风格

6.75.2 5.75.2 布局骨架

AppNavbar（顶栏）+ RouterView
→ 顶栏含：logo、应用名、主题切换、用户菜单
→ 页面内：卡片/表格/图表自由组合

6.75.3 5.75.3 暗色模式（如实现）

html.dark class + Element Plus dark css（07 章扩展案例八）

6.76 5.76 深入：05 章最终综合自测（追加 10 题）

1. vite 的 base 与端口配置要点？
2. embedded 与 dist 模式的切换机制？
3. 开发时前端如何实时生效？
4. 生产与开发登录态的差异？
5. store 放数据的口诀？
6. composable 提炼的三个条件？

7. useXxx 命名约定?
8. scoped 样式的作用域?
9. 布局骨架的三部分?
10. 跨应用跳转如何携带登录态?

答对 8+ → 05 章最终通过。

6.77 5.77 深入：项目实战——完整读一个监控页（fj200c_information 的 Monitor）

6.77.1 5.77.1 页面组件树

Monitor.vue

```
├─ StatusBar（服务状态）
├─ TableView（数据表格）
├─ ChartView（图表）
└─ CsvPanel（CSV 控制）
```

6.77.2 5.77.2 数据流

后端 WS（frame 事件）

- Monitor.vue 的 useService/composable 收数据
- 写入 store（或本地 ref）
- TableView/ChartView 通过 props/computed 取数据
- 渲染更新

6.77.3 5.77.3 关键代码模式

```
// 接收 WS 数据
wsClient.on('frame', (frame: TableRow) => {
  rows.value.push(frame) // 累积
  if (rows.value.length > 200) rows.value.shift() // 限长
  // 图表只更新最新点
  chartData.value = [frame.timestamp, frame.ngSpeed]
})
```

6.77.4 5.77.4 监控页与台账页的本质区别

维度	台账页	监控页
数据来源	HTTP 拉取	WS 实时推送
更新频率	用户操作时	每秒多次
数据结构	全量列表	增量追加
状态管理	简单 ref	store + 限长

6.78 5.78 深入：7 个应用的 api facade 设计

6.78.1 5.78.1 统一模式

```
// frontend/fw100/src/api/index.ts
import * as generated from '@shared/api/generated/fw100'
import { setApiInstance } from '@shared/utils/httpClient'

const fw100Api = {
  listItems: generated.fw100ListItems,
  createItem: generated.fw100CreateItem,
  // ...
}

export { fw100Api }
```

6.78.2 5.78.2 facade 的价值

1. 视图层不直接 import generated (隔离生成代码变动)
2. 统一导出名 (7 应用风格一致)
3. 可在此处加自定义逻辑 (组装/转换)

6.78.3 5.78.3 为什么要 setApiInstance

生成代码的 customInstance 用全局单例 axios

→ 各应用启动时注入自己的 baseUrl/token 处理

→ setApiInstance 就是这个注入点

6.79 5.79 深入：路由守卫的完整时序

sequenceDiagram

```
participant U as 用户
participant R as Router
participant G as AuthStore
U->>R: 访问 /monitor
R->>G: beforeEach 检查登录态
G-->>R: 已登录
R->>R: 检查权限 (roles/menu)
R-->>U: 渲染 Monitor
Note over R,U: 未登录 → 重定向 /login
```

6.80 5.80 深入：05 章终极自测 (5 题)

1. 监控页的四部分组件树?
2. WS 数据如何限长?
3. 台账页与监控页的数据流差异?
4. api facade 的三个价值?
5. 路由守卫的完整时序?

答对 4+ → 05 章彻底通关。

6.81 5.81 深入：每个应用的核心配置文件速查

6.81.1 5.81.1 vite.config.ts 要点 (7 应用对照)

应用	端口	base (构建)	proxy ws
admin	5174	/admin	否
fj200c_information	5173	/fj200c_information	是
fj200c_main	5179	/fj200c_main	是
fw100	5175	/fw100	否
fw150	5178	/fw150	否
ftj1c	5176	/ftj1c	是
city3d	5177	/city3d	否

记忆法：有 WS 的应用 (三个监控/通信类) proxy 开 ws；台账/管理类不开。

6.81.2 5.81.2 package.json 脚本

```
{
  "scripts": {
    "dev": "vite",
    "build": "vue-tsc --noEmit && vite build",
    "preview": "vite preview"
  }
}
```


6.81.3 5.81.3 tsconfig 要点

strict: true (严格模式)
moduleResolution: bundler
paths: @ → src/ (可选)

6.82 5.82 深入：Element Plus 的全局配置

6.82.1 5.82.1 main.ts 的挂载方式

// 全量引入 (简单, 体积大)
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
app.use(ElementPlus)

// 或按需引入 (体积优化, 需 unplugin-vue-components)

6.82.2 5.82.2 项目现状

全量引入 (开发简单)
→ 如需优化可改按需 (首屏体积 -40% 左右)
→ 注意组件样式与暗色模式 css 一并引入

6.82.3 5.82.3 常用组件的使用清单

组件	用途	本项目位置
el-table/el-table-column	数据表格	所有列表页
el-form/el-form-item	表单	登录/编辑
el-input/el-select	输入选择	表单
el-button	按钮	到处
el-tag	标签	状态显示
el-dialog	弹窗	编辑/详情
el-card	卡片	布局
el-tabs	标签页	多面板
el-switch	开关	服务控制
el-upload	上传	文件 (如有)

6.83 5.83 深入：ECharts 在监控页的配置参考

6.83.1 5.83.1 折线图配置

```
const option = {  
  xAxis: { type: 'time' }, // 时间轴  
  yAxis: { type: 'value', scale: true }, // 数值轴  
  series: [{  
    type: 'line',  
    showSymbol: false, // 点不显示  
    lineStyle: { width: 1.5 },  
    data: points, // [timestamp, value][]  
  }],  
  tooltip: { trigger: 'axis' },  
  animation: false, // 高频数据关动画  
}
```

6.83.2 5.83.2 高频更新的优化

- 1. animation: false (避免每帧动画计算)
- 2. setOption 只更新 series.data (浅更新)

3. 限长 500 点（画布只显示窗口）
4. resize 监听（容器变化自适应）

6.84 5.84 深入：05 章实战自测（8 题）

1. 7 应用的端口对照？
2. 哪些应用 proxy 开 ws？
3. Element Plus 引入方式的取舍？
4. 按需引入的插件？
5. el-table 在哪些页出现？
6. 折线图高频优化的四点？
7. tooltip trigger 的类型？
8. resize 何时监听？

答对 7+ → 05 章实战通过。

6.85 5.85 深入：admin 应用的完整实现参考（最典型的 CRUD 应用）

6.85.1 5.85.1 页面清单与路由

```
/login          登录页
/users          用户列表（分页/搜索/排序）
/users/new      新增用户
/users/:id/edit 编辑用户
```

6.85.2 5.85.2 用户列表页要点

```
// 权限控制（按钮级）
const canDelete = authStore.hasPermission('UsersDelete')
// 模板
<el-button v-if="canDelete" type="danger" @click="remove(row)">删除</el-button>
```

6.85.3 5.85.3 用户编辑页要点

1. 角色选择（多选：el-select multiple）
2. 密码字段（编辑时可选改）
3. 禁用开关（is_active）
4. 保存 → 调 update 接口

6.85.4 5.85.4 与业务应用的差异

admin 是“管理后端”：纯 CRUD、无实时数据
业务应用是“监控前端”：实时数据流为主
→ 两者代码模式差异 = HTTP vs WS 的差异

6.86 5.86 深入：login 页的完整实现参考

6.86.1 5.86.1 表单

```
<el-form ref="formRef" :model="form" :rules="rules">
  <el-form-item prop="email">
    <el-input v-model="form.email" placeholder="邮箱" />
  </el-form-item>
  <el-form-item prop="password">
    <el-input v-model="form.password" type="password" placeholder="密码" show-password />
  </el-form-item>
  <el-button type="primary" :loading="loading" @click="submit">登录</el-button>
</el-form>
```

6.86.2 5.86.2 提交逻辑

```
const submit = async () => {
  await formRef.value?.validate()
  loading.value = true
}
```

```

try {
  await authStore.login(form.email, form.password)
  // 拉角色注册表（菜单）
  await loadRoleRegistry()
  // 跳转：按角色对应的应用主页
  router.push(ROLE_APP_URLS[authStore.role] ?? '/')
} catch { /* 拦截器已提示 */ }
finally { loading.value = false }
}

```

6.86.3 5.86.3 登录后的跳转

admin 角色 → /users

fj200c_information 角色 → 对应应用主页

(ROLE_APP_URLS 手写在 shared/roles.ts)

6.87 5.87 深入：导航栏与布局的完整实现参考

6.87.1 5.87.1 布局结构

```

<el-container>
  <el-header>                <!-- 顶栏: logo/标题/用户菜单 -->
    <AppNavbar />
  </el-header>
  <el-main>                  <!-- 主内容区 -->
    <RouterView />
  </el-main>
</el-container>

```

6.87.2 5.87.2 用户菜单

用户菜单：头像 + 用户名 → 下拉

→ 退出登录（logout）

→ 切换应用（ROLE_APP_URLS 列表）

6.87.3 5.87.3 应用切换

dropdown 列出所有有权限的应用

点击 → window.location.href = appUrl（整页跳转）

6.88 5.88 深入：05 章综合自测（8 题）

1. admin 与业务应用的模式差异？
2. 按钮级权限怎么控制？
3. 编辑用户页面要点？
4. 登录提交的完整流程？
5. 登录后跳转的依据？
6. 布局的三层结构？
7. 用户菜单的功能？
8. 应用切换的实现？

答对 7+ → 05 章综合通过。

6.89 5.89 深入：fj200c_information 的完整页面走读（以仪表盘为例）

6.89.1 5.89.1 仪表盘的数据来源

WS frame 事件 → store 更新 → 仪表卡片

每帧更新：转速表、水温表、油压表、油耗表

6.89.2 5.89.2 仪表卡片的实现

```

<template>
  <el-card class="gauge-card">

```

```

<div class="gauge-title">{{ title }}</div>
<div class="gauge-value">{{ formatNum(value) }}</div>
<div class="gauge-unit">{{ unit }}</div>
<div class="gauge-bar">
  <div class="bar-fill" :style="{ width: `${percent}%` }" />
</div>
</el-card>
</template>

<script setup lang="ts">
const props = defineProps<{
  title: string
  value: number
  unit: string
  min: number
  max: number
}>()

const percent = computed(() =>
  ((props.value - props.min) / (props.max - props.min)) * 100
)
</script>

```

6.89.3 5.89.3 仪表盘的布局

el-row/el-col 栅格 → 2x2 或 1x4 卡片
 自适应：大屏 4 列，小屏 2 列

6.89.4 5.89.4 仪表盘的边界处理

1. 数值 NaN → 显示 --
2. 超量程 → 红色警示
3. 无数据（服务未启动）→ 显示占位

6.90 5.90 深入：数据表格与图表页的配合

6.90.1 5.90.1 表格与图表共用数据

同一 store 数据 → 表格显示全部行
 → 图表只取最新 N 点（或滚动窗口）
 → 两者互不干扰、实时同步

6.90.2 5.90.2 图表的滚动窗口

```

const points = computed(() => {
  const end = rows.value.length
  const start = Math.max(0, end - 300)
  return rows.value.slice(start, end).map(r => [r.timestamp, r.ngSpeed])
})

```

6.90.3 5.90.3 暂停/恢复

暂停 → 停止更新图表（保留现场）
 恢复 → 继续追加
 → 检查某时刻数据时常用

6.91 5.91 深入：CSV 页面的完整实现参考

6.91.1 5.91.1 文件列表

GET /api/fj200c_information/csv/list → 按天文件列表
 → el-table 展示（文件名/大小/日期）
 → 下载按钮 → Blob 下载

6.91.2 5.91.2 下载实现

```
const download = async (name: string) => {  
  const blob = await api.downloadCsv(name)  
  const url = URL.createObjectURL(blob)  
  const a = document.createElement('a')  
  a.href = url; a.download = name  
  a.click()  
  URL.revokeObjectURL(url)  
}
```

6.91.3 5.91.3 删除/归档

旧文件 → 删除接口或手动清理

推荐：保留最近 30 天，其余归档

6.92 5.92 深入：配置页面的完整实现参考

6.92.1 5.92.1 读取与展示

GET /api/fj200c_information/config → ini 内容

前端以表单/文本展示

6.92.2 5.92.2 修改与保存

前端编辑 → PUT 保存接口

→ 后端校验 + 写盘

→ 热加载（fj200c_information 立即生效）

6.92.3 5.92.3 校验与提示

1. 串口号格式校验（COM\d+）
2. 端口范围校验
3. 保存失败 → 提示原因
4. 成功后提示“已生效/需重启”

6.93 5.93 深入：05 章终局自测（8 题）

1. 仪表卡片的三要素？
2. 栅格布局怎么做？
3. 无数据时显示什么？
4. 表格与图表的数据关系？
5. 滚动窗口的写法？
6. 暂停恢复的意义？
7. 下载文件的完整代码？
8. 配置保存后的提示逻辑？

答对 7+ → 05 章终局通过。

6.94 5.94 深入：city3d 的完整实现走读

6.94.1 5.94.1 技术栈

Three.js（3D 渲染）+ Vue 3 + Element Plus

数据：HTTP API（建筑/区域/事件 CRUD）

6.94.2 5.94.2 3D 场景的初始化

// useThree.ts（结构示意图）

```
const scene = new THREE.Scene()  
const camera = new THREE.PerspectiveCamera(60, w / h, 0.1, 1000)  
const renderer = new THREE.WebGLRenderer({ antialias: true })  
renderer.setSize(w, h)  
container.appendChild(renderer.domElement)
```

```
// 循环渲染
const animate = () => {
  requestAnimationFrame(animate)
  controls.update()
  renderer.render(scene, camera)
}
animate()
```

6.94.3 5.94.3 建筑的可视化

每个建筑 → BoxGeometry 网格 → 加入场景
点击建筑 → 弹出详情 (raycaster 拾取)
编辑后 → 重建该建筑网格

6.94.4 5.94.4 2D 与 3D 的协作

管理页 (2D) 修改 → 保存
3D 页 → 轮询/手动刷新 → 重建场景
(数据单一来源: 后端数据库)

6.95 5.95 深入: ftj1c 的帧数据展示细节

6.95.1 5.95.1 表格列设计

时间 / 源地址 / 帧类型 / 长度 / 关键字段...
(16 路源 → 列分组或筛选)

6.95.2 5.95.2 坐标字段的展示

CGCS2000 经度/纬度 (后端已转换)
→ 前端格式化显示 (度分秒)
→ 可选: 地图展示 (未内置)

6.95.3 5.95.3 数据筛选

按源筛选: 只显示某一路
按类型筛选: 只显示某类帧
→ 大流量时定位问题

6.96 5.96 深入: fw100 与 fw150 的差异对比

6.96.1 5.96.1 相同点

1. 同样式台账 (CRUD 页面)
2. 同样 api facade 模式
3. 同样分页/搜索/排序

6.96.2 5.96.2 不同点

1. 端口不同 (5175 vs 5178)
2. 字段不同 (各自 DTO)
3. 角色不同 (Fw100Monitor vs Fw150Monitor)
4. 可能业务逻辑不同 (校验/联动)

6.96.3 5.96.3 复制新台账的模板价值

新设备台账 → 复制 fw100 → 改字段/端口/角色
→ 半天搞定一个应用
(这就是 08 章的实战基础)

6.97 5.97 深入: 05 章毕业自测 (8 题)

1. Three.js 场景初始化的步骤?
2. 建筑网格怎么加入?
3. 点击建筑拾取的方法?
4. 2D 与 3D 的数据协作?

5. 表格筛选的两种方式？
6. 坐标如何展示？
7. fw100 与 fw150 的异同？
8. 复制新台账的流程？

答对 7+ → 05 章毕业。

6.98 5.98 深入：一个完整应用从头构建的步骤（模板复用法）

6.98.1 5.98.1 复制模板

复制 fw100 → 新应用 xxx（08 章有完整流程）
Copy-Item frontend/fw100 frontend/xxx -Recurse

6.98.2 5.98.2 修改清单

1. package.json: name 改为 xxx
2. vite.config.ts: port 5175 → 新端口；base /fw100/ → /xxx/
3. src/api/index.ts: fw100Api → xxxApi（指向 generated xxx）
4. src/stores: 按需改名
5. src/views: 改页面内容
6. 路由: 改路径/标题
7. 标题/logo: index.html + 导航栏

6.98.3 5.98.3 验证

npm run dev # 新端口起来
npm run build # 类型 + 构建通过

6.98.4 5.98.4 常见翻车点

1. base 忘改 → 生产资源 404
2. 端口冲突 → strictPort 报错
3. api facade 还指向旧应用
4. 角色/权限未加 → 403

6.99 5.99 深入：监控类应用的共同骨架

6.99.1 5.99.1 骨架五件

1. useService（启停/状态）
2. useWebSocket（连接/重连）
3. store（数据缓存）
4. 表格/图表组件
5. 配置页

6.99.2 5.99.2 三应用的差异点

应用	数据通道	特有页面
fj200c_information	串口 + WS	仪表盘/命令
fj200c_main	三路串口 + WS	三路面板/试验
ftj1c	UDP + WS	IP 配置/坐标

6.99.3 5.99.3 新增监控类的模板

复制 fj200c_information → 换协议/换字段
→ 骨架不变，只换数据源

6.100 5.100 深入：05 章大师自测（8 题）

1. 复制模板的五个修改点？
2. 四个翻车点？

3. 监控骨架的五件?
4. 三个监控应用的差异?
5. 新增监控应用的思路?
6. base 忘改的后果?
7. 端口冲突的表现?
8. api facade 指向错误的表现?

答对 7+ → 05 章大师。

6.101 5.101 深入：7 个应用的异常处理策略

6.101.1 5.101.1 统一错误处理 (axios 拦截器)

HTTP 错误 → 拦截器统一弹 `ErrorMessage`

→ 业务代码无需每个请求都 `try/catch`

→ 只需处理成功分支

6.101.2 5.101.2 业务异常 vs 技术异常

业务异常：后端返回 `message` (密码错误/无权限)

→ 拦截器展示 `message`

技术异常：网络断开/超时

→ 拦截器提示"网络异常，请重试"

6.101.3 5.101.3 WS 异常处理

1. 断线 → 自动重连 (2s 间隔)
2. 重连失败多次 → 提示用户
3. 服务停止 → `status` 事件 → 界面灰化
4. 数据缺失 → 显示占位

6.101.4 5.101.4 页面级兜底

1. 路由 404 → 兜底页面
2. 组件加载失败 → `errorElement`/动态组件 `catch`
3. 后端 500 → 拦截器提示

6.102 5.102 深入：跨应用跳转与登录态的完整细节

6.102.1 5.102.1 跳转的时机

1. 登录成功后 (按角色)
2. 顶部菜单切换 (有权限的应用列表)
3. 登录失效后的返回

6.102.2 5.102.2 跳转方式

// 方式一：整页跳转 (推荐，干净)

```
window.location.href = 'http://localhost:5174'
```

// 方式二：链接

```
<a href="http://localhost:5174">管理后台</a>
```

6.102.3 5.102.3 登录态的传递

生产环境：同域同端口不同路径 → 同 `origin` → `localStorage` 共享

开发环境：不同端口 → 各自登录

6.102.4 5.102.4 跳转后的自动登录

目标应用 `initAuth`:

读 `localStorage` → 有 `token` → `/me` 验证 → 直接进入
(无感切换)

6.103 5.103 深入：05 章权威自测（8 题）

1. 拦截器如何统一处理错误？
2. 业务异常与技术异常的区分？
3. WS 断线的处理？
4. 页面级兜底的三种？
5. 跳转的三种时机？
6. 两种跳转方式？
7. 生产与开发登录态的差异？
8. initAuth 的自动登录流程？

答对 7+ → 05 章权威。

6.104 5.104 深入：状态栏与页面交互的完整设计

6.104.1 5.104.1 状态栏的构成

服务状态（运行/停止/异常）

- 数据源信息（串口号/波特率/模拟模式）
- 帧计数（接收/丢弃）
- 连接状态（WS 在线/离线）
- 当前时间

6.104.2 5.104.2 状态栏的更新机制

WS status 事件 → 实时更新

HTTP 轮询 → 兜底（WS 断线时）

- 双通道保证状态可见

6.104.3 5.104.3 按钮的联动

服务未启动 → 启动按钮可用/停止按钮禁用

服务运行中 → 反向

配置修改 → 提示重启

（状态驱动 UI：状态变化自动联动）

6.105 5.105 深入：CSV 数据的可视化分析（进阶用法）

6.105.1 5.105.1 下载后分析

CSV 导出 → Excel/WPS 打开

- 透视表/图表分析

- 趋势/异常一目了然

6.105.2 5.105.2 代码分析（Python 示例）

```
import pandas as pd
df = pd.read_csv("2026-08-08.csv")
print(df.head())
df.plot(x="timestamp", y="ngSpeed") # 转速曲线
```

6.105.3 5.105.3 系统内分析（08 章案例七）

CSV 趋势分析页：选文件 → 选参数 → 出曲线

- 不离开系统即可分析

6.106 5.106 深入：05 章权威自测（8 题）

1. 状态栏的六个部分？
2. 双通道更新的意义？
3. 按钮联动的规则？
4. CSV 的两种分析方式？
5. pandas 怎么读 CSV？

6. 趋势页的交互?
7. 帧计数的意义?
8. 模拟模式的显示?

答对 7+ → 05 章权威。

6.107 5.107 深入：7 个应用的项目配置对照（深入版）

6.107.1 5.107.1 index.html 的差异

标题：管理后台 / 发动机监控 / 设备台账 / 通信监控 / 城市 3D 展示...
favicon：各自图标

6.107.2 5.107.2 依赖差异

admin: element-plus + axios
监控类: + echarts
city3d: + three.js
台账类: element-plus + axios

6.107.3 5.107.3 构建体积参考

管理/台账类: 较小 (~1MB gzip)
监控类: 中等 (+ echarts)
city3d: 较大 (+ three.js)
→ 体积影响首屏，必要时按需引入

6.108 5.108 深入：真实排错场景演练（前端）

6.108.1 5.108.1 场景：登录后白屏

排查步骤：

1. Console 报错? → 路由/组件错误
2. Network 401? → token 问题
3. 角色跳转地址错误? → ROLE_APP_URLS
4. initAuth 未执行? → App.vue 检查

6.108.2 5.108.2 场景：WS 连不上

1. proxy ws:true 配了没 (vite)
2. token 查询参数格式
3. 后端 WS 路由路径
4. 服务是否启动

6.108.3 5.108.3 场景：表格数据不更新

1. store 是否更新 (DevTools)
2. WS 消息类型是否匹配
3. 限长逻辑是否误删
4. 表格是否绑定了新数组引用

6.109 5.109 深入：05 章权威自测（8 题）

1. index.html 的差异点?
2. 四类应用的依赖差异?
3. 构建体积的影响?
4. 白屏的四个排查?
5. WS 连不上的四个原因?
6. 表格不更新的四个原因?
7. DevTools 的用途?
8. ROLE_APP_URLS 的作用?

答对 7+ → 05 章权威。

下一节：06-前后端类型同步机制。

7 06 前后端类型同步机制 (utoipa + OpenAPI + orval)

这是本项目工程化的灵魂章节。理解它，你就理解了为什么“改一个 Rust 结构体，前端类型自动跟着变”。

7.1 6.1 问题：没有类型同步的世界

7.1.1 6.1.1 手写两端的痛苦

在没有这套机制的传统项目里：

```
flowchart LR
    subgraph Rust 后端
        R1["struct User { id, username, email, role }"]
    end
    subgraph TS 前端
        T1["interface User { id, username, email, role }"]
    end
    R1 -. 手工复制 .-> T1
    R1 -. 极易失同步 .-> T2["改了一端忘另一端"]
```

后果：字段改名/新增/类型变化时，一端改了另一端不知道 → 运行时才发现数据对不上 → 类型错误泛滥。

7.1.2 6.1.2 本项目的解法

```
flowchart LR
    R["Rust 定义<br/>#[derive(ToSchema)] #[utoipa::path]"] --> O[openapi.json]
    O --> G[orval 生成]
    G --> T["TS 类型 + 请求函数"]
    T --> F["前端代码<br/>类型安全"]
```

原则：Rust 是唯一真相源 (single source of truth)，前端类型完全由工具生成。

7.2 6.2 链路总览 (一条命令全同步)

```
flowchart TD
    A[修改 Rust DTO/handler] --> B[cargo test export_openapi]
    B --> C[openapi/openapi.json 重新生成]
    C --> D[内置断言校验<br/>paths/operationId 防漂移]
    D --> E[通过| E[orval 读取 openapi.json]]
    E --> F[packages/shared/src/api/generated/ 重写]
    F --> G[前端 vue-tsc 报错处<br/>即需更新的调用点]
```

入口命令 (根目录)：

```
npm run gen:api # = cargo test export_openapi && orval
```

这条命令做的事： 1. 运行 `cargo test export_openapi`——一个特殊测试，执行时把当前后端所有 utoipa 注解聚合，**生成/更新 openapi.json**。 2. 同时断言 openapi.json 的关键路径与 operationId 与预期一致 (防止意外漂移)。 3. 运行 `orval`——读取 openapi.json，生成 TS 类型 + API 请求函数到 generated/。

7.3 6.3 utoipa 注解详解（后端侧）

7.3.1 6.3.1 结构体注解（DTO）

```
// src/common/models.rs
use utoipa::ToSchema;

/// 用户信息
#[derive(ToSchema)]
pub struct UserInfo {
    pub id: i64,
    pub username: String,
    pub email: String,
    #[schema(example = "admin")]
    pub role: String,
}
```

`#[derive(ToSchema)]`: 把 Rust 结构体注册为 OpenAPI schema。orval 读完后生成同名 TS interface:

```
// generated/model/user-info.ts (orval 生成)
export interface UserInfo {
  id: number
  username: string
  email: string
  role: string
}
```

字段类型映射表 (Rust → TS):

Rust	TS
i64 / i32 / u32	number
String	string
bool	boolean
f64	number
Option	T undefined 或 T null
Vec	T[]
HashMap<String, T>	Record<string, T>
自定义 struct	interface
自定义 enum (string 型)	enum

7.3.2 6.3.2 handler 注解（API 路径）

```
// src/fj200c_information/handlers.rs
use utoipa::{IntoParams, OpenApi};

/// 查询服务状态
#[utoipa::path(
    get,
    path = "/api/fj200c_information/service/status",
    tag = "fj200c_information",
    operation_id = "fj200c_informationServiceStatus",
    responses(
        (status = 200, description = "成功", body = ServiceStatusResult)
    )
)]
pub async fn service_status(State(state): State<AppState>) ->
Result<Json<ApiResponse<ServiceStatusResult>>, AppError> {
```

```
// ...
}
```

注解各属性含义：

属性	作用	示例
get/post/put/delete	HTTP 方法	get
path	完整路径	"/api/fj200c_information/ service/status"
tag	分组 (orval 按 tag 分文件)	"fj200c_information"
operation_id	唯一操作 ID (orval 函数名来源)	"fj200c_informationServiceStatus"
responses	响应定义 (body 类型)	body = ServiceStatusResult
request_body	请求体	request_body = CreateUserRequest
params	查询/路径参数	params(PageParams)

operation_id 的命名约定： tag 前缀 + 动作 (如 fw100ListItems、adminUsersCreate) —— orval 函数名就是它，所以**不能重名** (防漂移测试专门查这个)。

7.3.3 6.3.3 带参数的 handler 注解

```
/// 更新台账条目
#[utoipa::path(
    put,
    path = "/api/fw100/items/{id}",
    tag = "fw100",
    operation_id = "fw100ItemsUpdate",
    params(
        ("id" = i64, Path, description = "条目 ID")
    ),
    request_body = CreateLedgerItemRequest,
    responses(
        (status = 200, description = "成功", body = CreateLedgerItemRequest)
    )
)]
pub async fn update_item(
    Path(id): Path<i64>,
    State(state): State<AppState>,
    Json(payload): Json<CreateLedgerItemRequest>,
) -> Result<Json<ApiResponse<CreateLedgerItemRequest>>, AppError> {
    // ...
}
```

路径参数用 params(("id" = i64, Path, ...)) 声明——orval 生成时函数签名变为 (id: number, ...)。

7.3.4 6.3.4 统一响应包装的注解方式

```
/// 分页查询参数
#[derive(ToSchema, IntoParams)]
pub struct PaginationParams {
    #[schema(default = 1, minimum = 1)]
    pub page: Option<i64>,
    #[schema(default = 20, minimum = 1, maximum = 100)]
    pub page_size: Option<i64>,
}
```

IntoParams: 把结构体字段映射为查询参数。#[schema(...)] 提供默认值/范围——OpenAPI 里生成描述, Swagger UI 与 orval 都能看到。

7.4 6.4 src/api_docs.rs (OpenAPI 聚合中心)

7.4.1 6.4.1 文件职责

```
// src/api_docs.rs
use utoipa::{openapi::OpenApi, Modify, OpenApi as _};

#[derive(OpenApi)]
#[openapi(
    paths(
        // 所有 handler 的路径函数 (聚合):
        auth_login, auth_me,
        admin_users_list, admin_users_get, admin_users_create, admin_users_update,
        admin_users_delete,
        // ...
        meta_roles,
    ),
    components(schemas(
        ApiResponse, UserInfo, CreateUserRequest, RoleInfo, Permission,
        // ... 所有 ToSchema 结构体
    )),
    tags(
        (name = "auth", description = "认证"),
        (name = "admin", description = "用户管理"),
        (name = "fj200c_information", description = "发动机监控"),
        // ...
    )
)]
pub struct ApiDoc;
```

聚合内容三部分: 1. paths(...): 所有 handler 的路径函数列表 (**新增接口必须在这里登记**)。 2. components(schemas(...)): 所有 DTO 结构体 (**新增 DTO 必须登记**)。 3. tags(...): 分组描述。

7.4.2 6.4.2 export_openapi 测试 (防漂移断言)

```
// src/api_docs.rs 末尾
#[cfg(test)]
mod tests {
    #[test]
    fn export_openapi() {
        // 1. 生成 OpenAPI 文档
        let spec = ApiDoc::openapi();
        // 2. 序列化为 JSON
        let json = serde_json::to_string_pretty(&spec).unwrap();
        // 3. 写入 openapi/openapi.json
        std::fs::write("openapi/openapi.json", json).unwrap();

        // 4. 断言关键路径存在 (防止删了接口忘记同步)
        let doc = spec.clone();
        let paths = doc.paths.paths;
        assert!(paths.contains_key("/api/auth/login"), "缺少 /api/auth/login");
        assert!(paths.contains_key("/api/users"), "缺少 /api/users");
        // ... 若干关键路径断言
    }
}
```

为什么是“test”：cargo test 可以无参数跑，且测试失败会中断（cargo run 不会）。把“生成 + 断言”放进测试，npm run gen:api 里执行 cargo test export_openapi 就能：- 成功 → openapi.json 已更新（与当前代码一致）。- 失败 → 有接口缺失/断言不匹配，**先修再生成**。

7.4.3 6.4.3 断言的意义（防漂移）

- 场景：有人删了 /api/fw100 的 update 接口但忘了同步
- export_openapi 测试断言失败
 - gen:api 中断
 - 强制开发者意识到“接口契约变了”

这就是“防漂移”：openapi.json 永远与实际代码一致，前端类型永远可信。

7.5 6.5 orval.config.ts 详解（生成侧）

7.5.1 6.5.1 配置文件

```
// orval.config.ts（根目录）
import { defineConfig } from 'orval'

export default defineConfig({
  rustweb: {
    input: {
      target: './openapi/openapi.json', // 输入：后端生成的 spec
    },
    output: {
      target: './packages/shared/src/api/generated/api/', // 输出：请求函数目录
      schemas: './packages/shared/src/api/generated/model/', // 输出：类型目录
      mode: 'tags-split', // 按 tag 分文件
      client: 'custom-instance', // 用自定义请求实例
      clean: true, // 生成前清空目录
      prettier: true, // 格式化
    },
    hooks: {
      afterAllFilesWrite: 'prettier --write', // 生成后统一格式化
    },
  },
})
```

7.5.2 6.5.2 配置项含义

配置	作用
mode: 'tags-split'	每个 tag 一个文件（如 admin.ts、fw100.ts）
client: 'custom-instance'	请求函数调用 customInstance（我们的 axios 壳）
clean: true	每次全量重写（旧文件自动删除）
schemas	TS 类型输出目录
afterAllFilesWrite	生成后 prettier 统一格式

7.6 6.6 npm run gen:api 完整流程演示

7.6.1 6.6.1 命令定义

```
// 根 package.json
{
  "scripts": {
```

```

    "gen:api": "cargo test export_openapi && orval"
  }
}

```

7.6.2 6.6.2 一次典型执行

步骤 1: 生成 openapi.json (含断言)
 cargo test export_openapi
 # 输出: running 1 test ... test ok

步骤 2: orval 生成前端代码
 orval
 # 输出: Generated xxx files ...
 # ✓ packages/shared/src/api/generated/api/admin.ts
 # ✓ packages/shared/src/api/generated/model/user-info.ts
 # ...

7.6.3 6.6.3 执行后的检查清单

[] openapi.json 已更新 (git diff 查看)
 [] generated/ 已重写 (git diff 查看新字段)
 [] 前端 vue-tsc 报错 = 调用点需要更新
 [] 手动跑一次 npm run build 确认全链路

7.7 6.7 generated 产物解读

7.7.1 6.7.1 类型文件示例

```

// packages/shared/src/api/generated/model/api-response.ts
export interface ApiResponse<T> {
  success: boolean
  message: string
  data: T | null
}

```

7.7.2 6.7.2 请求函数文件示例

```

// packages/shared/src/api/generated/api/auth.ts
import { customInstance } from '../custom-instance'
import type { LoginRequest, LoginResult } from '../model'

export const authLogin = (loginRequest: LoginRequest) => {
  return customInstance<ApiResponse<LoginResult>>({
    url: `/api/auth/login`,
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    data: loginRequest,
  })
}

```

7.7.3 6.7.3 与 facade 的衔接

```

// 前端 api/index.ts
export const authApi = {
  login: (payload: LoginRequest) => authLogin(payload),
  me: () => authMe(),
}

```

调用链: 视图 → facade → generated 函数 → customInstance → axios。

7.8 6.8 修改 DTO 后的全流程演练

7.8.1 场景：给台账加“位置”字段

步骤 1：后端改 Rust

```
// src/fw100/models.rs
#[derive(ToSchema)]
pub struct Fw100LedgerItem {
    pub id: i64,
    pub name: String,
    // ... 现有字段
    pub location: String,    // 新增字段
}
```

步骤 2：跑 gen:api

```
npm run gen:api
```

步骤 3：检查生成的类型

```
// generated/model/fw100-ledger-item.ts
export interface Fw100LedgerItem {
    id: number
    name: string
    location: string    // ✨ 自动多出
    // ...
}
```

步骤 4：前端更新调用点

vue-tsc 报错处 = 需要处理的地方：

1. 新建台账表单没填 location → 表单加字段
2. 详情展示少字段 → 展示加行
3. 若后端 SQL 还没同步 location → 后端 service 也要改

步骤 5：验证

```
npm run build    # 全前端类型检查通过
```

核心体验：前端永远不知道“忘了加字段”——类型系统强迫你补全。

7.9 6.9 WebSocket 为什么不进 OpenAPI

7.9.1 6.9.1 OpenAPI 的边界

OpenAPI 规范描述 HTTP 请求/响应（路径、方法、schema）。WebSocket 是长连接消息流，不在 OpenAPI 规范范围内（3.x 没有 WS 的标准描述）。

7.9.2 6.9.2 项目实践

内容	走 OpenAPI?	方式
HTTP 接口	✓	utoipa + orval
WS 连接 URL	✗	前端手写 buildWebSocketUrl
WS 事件类型	✗	各前端 types.ts 手写判别联合
WS 握手鉴权	—	?token= 查询参数

7.9.3 6.9.3 WS 类型手写的维护注意

```
// 前端 types.ts — 与后端 WS payload 结构保持一致
// 后端改事件结构 → 前端 types.ts 同步改（无自动工具）
// 防漂移手段：前后端都按同一“事件类型命名规范”（type 字段 + payload）
```

这是项目唯一的“人工同步点”——写代码时留意后端 ws_bridge 的 payload 结构。

7.10 6.10 常见问题与维护指南

7.10.1 6.10.1 报错速查

现象	原因	处理
cargo test export_openapi 失败	断言不匹配 / 编译错误	看断言消息，补接口或修 DTO
orval 生成报错	openapi.json 无效	先跑 cargo test 修 spec
前端类型少了字段	没跑 gen:api	跑 gen:api
生成函数名变了	operation_id 改了	更新 facade
生成文件不见了	tag 改名	tags-split 按 tag 分文件，改名即新文件

7.10.2 6.10.2 日常维护清单

- [] 新增接口: handler 注解 + api_docs.rs 的 paths 登记
- [] 新增 DTO: ToSchema + api_docs.rs 的 schemas 登记
- [] 删除接口: api_docs.rs 同步删（防漂移测试会抓）
- [] 修改 DTO 字段: 只改 Rust，其余自动
- [] WS 变更: 手写 types.ts 同步

7.10.3 6.10.3 为什么 ts-rs 方案被替换

历史：项目曾用 ts-rs（从 Rust 直接生成 TS 类型）。替换为 utoipa + orval 的原因：1. ts-rs 只生成类型，**不生成请求函数**（接口路径还要手写）。2. orval 从 OpenAPI 一次生成 **类型 + 请求函数**（API 客户端全自动）。3. utoipa 注解同时服务 Swagger UI 文档（运行时 /api-docs/openapi.json）。

结果：一套注解，三个产物（openapi.json 文档 + TS 类型 + TS 请求函数）。

7.10.4 6.10.4 运行时文档 (Swagger UI)

- 浏览器访问 <http://localhost:3000/api-docs/openapi.json>
- 实时 OpenAPI spec（与构建时生成的 openapi.json 同源）
 - 配合 Swagger UI 可交互调试

前端调试 API 的权威参考：所有接口的路径/参数/响应类型都在这。

7.11 6.11 本章自测

1. npm run gen:api 具体执行了什么？
2. #[utoipa::path] 里 operation_id 的作用？命名约定？
3. #[derive(ToSchema)] 后前端会得到什么？
4. export_openapi 为什么是“测试”而不是普通命令？
5. 防漂移断言抓什么错误？
6. orval 的 tags-split 和 custom-instance 分别什么意思？
7. 新增接口需要动哪些文件？
8. WS 事件类型为什么手写？
9. ApiResponse 是怎么生成的？前端怎么用它？
10. 改一个 DTO 字段，完整流程几步？

答对 8+ → 06 章通过。 下一章：使用与维护手册——日常开发/部署/故障排查的实操指南。

7.12 6.12 utoipa 注解完整参考（抄写即用）

7.12.1 6.12.1 #[utoipa::path] 属性全表

属性	写法	用途
HTTP 方法	get / post / put / delete	第一行
path	path = "/api/xxx"	完整路径
tag	tag = "xxx"	分组
operation_id	operation_id = "xxxXxx"	唯一 ID
params	params(("id" = i64, Path, description = "..."), XxxParams)	路径/查询参数
request_body	request_body = XxxRequest	请求体类型
responses	responses((status = 200, description = "成功", body = XxxResult))	响应
context_path	context_path = "/api"	全局前缀（本项目手写全路径，未用）

7.12.2 6.12.2 完整模板（复制修改）

```
/// 功能描述（会进 OpenAPI description）
#[utoipa::path(
    post,
    path = "/api/<module>/<action>",
    tag = "<module>",
    operation_id = "<module><Action>",
    request_body = <XxxRequest>,
    responses(
        (status = 200, description = "成功", body = <XxxResult>),
        (status = 401, description = "未授权"),
        (status = 403, description = "权限不足"),
    )
)]
pub async fn <action>(…) -> Result<Json<ApiResponse<<XxxResult>>>, AppError> {
    // …
}
```

7.12.3 6.12.3 路径参数 + 查询参数混合

```
#[utoipa::path(
    get,
    path = "/api/fw100/items/{id}",
    tag = "fw100",
    operation_id = "fw100ItemsGet",
    params(
        ("id" = i64, Path, description = "条目 ID"),
        ("include_deleted" = Option<bool>, Query, description = "是否包含已删除"),
    ),
    responses(
        (status = 200, description = "成功", body = Fw100LedgerItem)
    )
)]
pub async fn get_item(
    Path(id): Path<i64>,
    Query(params): Query<Option<bool>>,
    …
)
```

7.13 6.13 深入：ApiResponse 泛型与前端处理

7.13.1 6.13.1 Rust 侧定义

```
// src/common/models.rs
#[derive(ToSchema)]
pub struct ApiResponse<T> {
    pub success: bool,
    pub message: String,
    pub data: Option<T>,
}
```

7.13.2 6.13.2 OpenAPI 如何表示泛型

OpenAPI 3.0 没有“泛型类型”概念——utoipa 会为每个具体化类型生成独立 schema:

```
// openapi.json 中（示意）
"ApiResponseFw100LedgerItem": { // 具体化后的名称
  "type": "object",
  "properties": {
    "success": { "type": "boolean" },
    "message": { "type": "string" },
    "data": { "$ref": "#/components/schemas/Fw100LedgerItem" }
  }
}
```

7.13.3 6.13.3 orval 生成后

```
// generated/model/（orval 会把具体化类型还原为泛型模式）
export interface ApiResponse<T> {
  success: boolean
  message: string
  data: T | null
}
```

```
// 请求函数返回类型
export const fw100ItemsList = () =>
  customInstance<ApiResponse<Fw100LedgerItem[]>>({ ... })
```

注意：具体化名称里的泛型参数会被 orval 识别还原成 <T> 泛型——无需手写。

7.14 6.14 深入：Enum 与 Option 的生成细节

7.14.1 6.14.1 Rust enum → TS

```
/// 事件级别
#[derive(ToSchema)]
#[serde(rename_all = "snake_case")] // 或 serde rename 控制字符串值
pub enum EventLevel {
    Info,
    Warning,
    Error,
}

// generated/model/event-level.ts
export enum EventLevel {
  Info = 'info',
  Warning = 'warning',
  Error = 'error',
}
```

关键：枚举字符串值由 serde 决定（snake_case → info）。前后端都以字符串交换——这就是为什么 Permission 枚举用 'system.admin' 这种点分字符串。

7.14.2 6.14.2 Option 的两种形态

Rust	OpenAPI	TS
Option<T> (无 default)	T \ null	T \ null (nullable)
Option<T> (#[schema(default)])	可选字段	T \ undefined

```
// 前端处理两种形态
if (item.remark) { ... } // null 或 undefined 都为假
item.remark ?? '--' // 兜底显示
```

7.14.3 6.14.3 数字精度提示

```
// Rust i64 → TS number: 大数会丢精度 (> 2^53)
// 本项目 id 用 i64 且值较小 (自增), 无风险
// 若未来有超大数值 → 改成 string 类型 (#[schema(value_type = String)])
```

7.15 6.15 深入：复杂嵌套类型

7.15.1 6.15.1 Vec / 数组

```
pub struct TestResult {
    pub points: Vec<StatePoint>, // → StatePoint[]
    pub matrix: Vec<Vec<f64>>, // → number[][]
}
```

7.15.2 6.15.2 HashMap

```
pub struct ConfigPayload {
    pub sections: HashMap<String, Vec<KeyValue>>, // → Record<string, KeyValue[]>
}
```

7.15.3 6.15.3 嵌套结构体

```
pub struct TestInfo {
    pub id: String,
    pub name: String,
    pub parameters: TestParameters, // 嵌套 struct → 内联 interface
}
```

规则：嵌套类型自动递归生成——只要每个叶子 struct 都有 ToSchema, orval 全链生成。

7.16 6.16 深入：分页接口完整实战 (city3d 案例)

7.16.1 6.16.1 后端

```
/// 分页查询参数
#[derive(ToSchema, IntoParams)]
pub struct PaginationParams {
    #[schema(default = 1, minimum = 1)]
    pub page: Option<i64>,
    #[schema(default = 20, minimum = 1, maximum = 100)]
    pub page_size: Option<i64>,
}

#[utoipa::path(
    get,
    path = "/api/city3d/buildings",
    tag = "city3d",
    operation_id = "city3dBuildingsList",
    params(PaginationParams),
    responses(
        (status = 200, description = "成功", body = PaginatedResult<Building>)
    )
)]
```

```

    )
  ]
}
pub async fn list_buildings(
  Query(params): Query<PaginationParams>,
  ...

```

7.16.2 6.16.2 前端调用

```

// generated 生成:
export const city3dBuildingsList = (params?: City3dBuildingsListParams) => {
  return customInstance<ApiResponse<PaginatedResult<Building>>>({
    url: `/api/city3d/buildings`,
    method: 'GET',
    params, // axios 自动拼查询串
  })
}

// 前端使用:
const res = await city3dApi.listBuildings({ page: 1, page_size: 20 })
if (res.success && res.data) {
  items.value = res.data.items
  total.value = res.data.total
}

```

分页模式是前端“列表型页面”的标准配餐——fw100/admin/city3d 全部如此。

7.17 6.17 orval 高级配置（按需定制）

7.17.1 6.17.1 参数命名覆盖

```

// orval.config.ts
output: {
  override: {
    query: { useQuery: false }, // 本项目不用 react-query (Vue 无此概念)
    mutator: {
      path: './packages/shared/src/api/custom-instance.ts', // 指定请求壳
      name: 'customInstance',
    },
  },
},
}

```

7.17.2 6.17.2 类型命名规范

```

output: {
  override: {
    schema: { useTypePrefix: false }, // 不用前缀
  },
}

```

7.17.3 6.17.3 何时需要动 orval 配置

- [] 生成函数命名不合习惯 → override.operations 配置
- [] 换请求库 (axios → fetch) → 改 customInstance 实现
- [] 分 tag 太细/太粗 → 改 mode
- [] 生成产物结构变化 → output.target/schemas

原则：能不动 orval 就不动——保持“生成 = 开箱即用”的稳定性。

7.18 6.18 类型同步机制的演进与边界

7.18.1 6.18.1 当前链路的能力边界

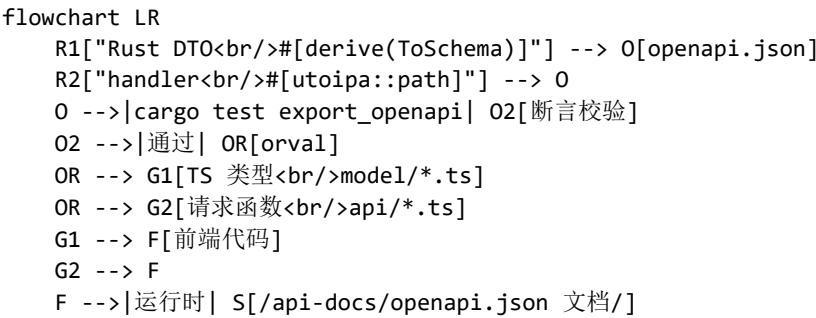
能力	状态
DTO → 类型 + 请求函数	✅ 全自动
接口文档（运行时）	✅ Swagger 同源
WS 事件类型	❌ 手写
前端自定义类型（业务层）	手写（与后端无关的部分）
错误码枚举	通过 message 透传（无结构化）

7.18.2 6.18.2 未来可改进方向（如需要）

- 1. WS 类型同步：手写一个"事件 schema"（类似 utoipa）→ 生成 TS
- 2. 错误码结构化：AppError 加 code 枚举 → OpenAPI 错误模型
- 3. 前端单测：generated 函数可 mock（customInstance 可替换）
- 4. 契约测试：openapi.json 与前端类型 diff（CI 环节）

这些是“如果项目继续长大”的进化方向，当前规模不需要。

7.18.3 6.18.3 本章总结（一张图）



一句话：Rust 定义一次 → 工具全链路生成 → 前端类型永不脱节。

7.19 6.19 最终自测（追加题）

- 11. 新增一个 tag 分组需要改哪些文件？
- 12. IntoParams 与 ToSchema 的区别？
- 13. #[schema(default = 20, maximum = 100)] 在前端哪里可见？
- 14. orval 的 custom-instance 具体指哪个文件？
- 15. openapi.json 文件提交仓库的意义？
- 16. 为什么泛型 ApiResponse 能正确生成？
- 17. 分页接口前端如何传参？(params 对象)
- 18. 什么情况下要改 orval.config.ts？
- 19. WS 类型手写怎么最小化漂移风险？
- 20. 假如 CI 环境跑 gen:api，防漂移测试的价值？

答对 15+ → 06 章精通。下一章是实操手册——从零启动、日常开发、部署发布、故障排查。

7.20 6.20 完整实例：新增“设备维护记录”模块（类型链路视角）

以 fw100 加“维护记录”子资源为例，走一遍类型同步完整路径（业务逻辑省略）：

7.20.1 步骤 1: Rust DTO

```
// src/fw100/models.rs
use utoipa::ToSchema;

/// 维护记录
#[derive(ToSchema)]
pub struct MaintenanceRecord {
    pub id: i64,
    pub item_id: i64,           // 关联台账条目
    pub action: String,        // 维护动作
    pub operator: String,      // 操作人
    pub occurred_at: String,    // 时间 (ISO 字符串)
    pub remark: Option<String>, // 备注 (可空)
}

/// 创建维护记录请求
#[derive(ToSchema)]
pub struct CreateMaintenanceRequest {
    pub item_id: i64,
    pub action: String,
    pub operator: String,
    pub remark: Option<String>,
}
```

7.20.2 步骤 2: handler 注解

```
// src/fw100/handlers.rs
/// 查询维护记录列表
#[utoipa::path(
    get,
    path = "/api/fw100/items/{id}/maintenance",
    tag = "fw100",
    operation_id = "fw100MaintenanceList",
    params(
        ("id" = i64, Path, description = "条目 ID")
    ),
    responses(
        (status = 200, description = "成功", body = Vec<MaintenanceRecord>)
    )
)]
pub async fn list_maintenance(
    Path(id): Path<i64>,
    State(state): State<AppState>,
) -> Result<Json<ApiResponse<Vec<MaintenanceRecord>>>, AppError> { ... }

/// 新增维护记录
#[utoipa::path(
    post,
    path = "/api/fw100/items/{id}/maintenance",
    tag = "fw100",
    operation_id = "fw100MaintenanceCreate",
    params(("id" = i64, Path, description = "条目 ID")),
    request_body = CreateMaintenanceRequest,
    responses((status = 200, description = "成功", body = MaintenanceRecord))
)]
pub async fn create_maintenance(...) -> ... { ... }
```

7.20.3 步骤 3: api_docs.rs 登记

```
paths(
    // ...
    fw100_list_maintenance, // 新增
    fw100_create_maintenance, // 新增
)
```



```

),
components(schemas(
  // ...
  MaintenanceRecord,          // 新增
  CreateMaintenanceRequest,    // 新增
)),

```

7.20.4 步骤 4: 跑 gen:api

```
npm run gen:api
```

7.20.5 步骤 5: 检查生成

```

// generated/api/fw100.ts (新增两个函数)
export const fw100MaintenanceList = (id: number) =>
  customInstance<ApiResponse<MaintenanceRecord[]>>({ url: `/api/fw100/items/${id}/maintenance`, method: 'GET' })

export const fw100MaintenanceCreate = (id: number, createMaintenanceRequest:
CreateMaintenanceRequest) =>
  customInstance<ApiResponse<MaintenanceRecord>>({
    url: `/api/fw100/items/${id}/maintenance`,
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    data: createMaintenanceRequest,
  })

```

7.20.6 步骤 6: 前端 facade + 页面

```

// frontend/fw100/src/api/index.ts
export const fw100Api = {
  // ...原有
  listMaintenance: (id: number) => generated.fw100MaintenanceList(id),
  createMaintenance: (id: number, payload: CreateMaintenanceRequest) =>
    generated.fw100MaintenanceCreate(id, payload),
}

```

7.20.7 步骤 7: 验证

```
npm run build          # 类型全链路通过
```

整个新增过程里，前端类型零手写——这是“类型同步机制”的最大红利。

7.21 6.21 深入: git diff 读生成产物 (改了什么一目了然)

7.21.1 6.21.1 改 DTO 字段后的 diff

```

// packages/shared/src/api/generated/model/fw100-ledger-item.ts
export interface Fw100LedgerItem {
  id: number
  name: string
  updated_at: string
+ location: string          // 🌟 新增字段 (后端加的)
}

```

7.21.2 6.21.2 改接口签名后的 diff

```

// packages/shared/src/api/generated/api/fw100.ts
-export const fw100ItemsUpdate = (id: number, createLedgerItemRequest:
CreateLedgerItemRequest) => {
+export const fw100ItemsUpdate = (id: number, createLedgerItemRequest:
UpdateLedgerItemRequest) => {

```

7.21.3 6.21.3 diff 检查习惯

提交前必看: `git diff packages/shared/src/api/generated/ openapi/openapi.json`
确认: 只出现预期的字段/函数变化 (没有意外删除/重命名)

这相当于“类型契约的 changelog”——比任何文档都准确地告诉你接口发生了什么变化。

7.22 6.22 类型同步方案对比 (为什么选这套)

方案	类型	请求函数	文档	维护成本
ts-rs (旧)	✓	✗ 手写	✗	中 (双维护)
utoipa + orval (现)	✓	✓	✓ (Swagger)	低 (单真相源)
OpenAPI 手写	✓	✓	✓	高 (与代码分离)
GraphQL	✓	✓	✓	重构成本高

核心优势总结: 1. **单真相源:** Rust 代码 → 全部产物, 无手工副本。2. **防漂移测试:** 接口删除/重命名立即失败。3. **运行时文档:** Swagger UI 调试。4. **请求函数免费:** 前端不用写任何 URL。

7.23 6.23 06 章完结: 知识串联

flowchart TD

```
A[06 章内容] --> B[Rust 注解写法]
A --> C[聚合与断言]
A --> D[orval 配置与产物]
A --> E[改 DTO 全流程]
A --> F[WS 边界]
A --> G[新增模块实例]
B --> H["能看懂后端每个 #[utoipa::path]"]
C --> I["能解释 gen:api 为何中断"]
D --> J["能读 generated 代码"]
E --> K["能完成改字段→全链路"]
F --> L["知道 WS 类型为何手写"]
G --> M["能照抄新增模块"]
```

学完本章, 你应该能: - 看到任意后端接口, 说出它对应的前端函数名 (operation_id 转换)。- 改一个字段后, 预估前端哪些地方会报错。- 新增一个模块时, 一次走通“Rust → openapi.json → generated → 前端”。

下一章: 使用与维护手册——日常开发、部署、故障排查的实操大全。

7.24 6.24 深入: openapi.json 结构速览

7.24.1 6.24.1 JSON 顶层结构

```
{
  "openapi": "3.0.3",
  "info": { "title": "RustWeb API", "version": "1.0.0" },
  "paths": {
    "/api/auth/login": {
      "post": {
        "tags": ["auth"],
        "operationId": "authLogin",
        "requestBody": { "content": { "application/json": { "schema": { "$ref": "#/components/schemas/LoginRequest" } } } },
        "responses": {
          "200": { "description": "成功", "content": { "application/json": { "schema": { "$ref": "#/components/schemas/ApiResponseLoginResult" } } } }
        }
      }
    }
  }
}
```

```

    }
  }
},
"components": {
  "schemas": {
    "LoginRequest": { "type": "object", "properties": { "username": {"type": "string"},
"password": {"type": "string"} } }
  }
},
"tags": [ { "name": "auth", "description": "认证" } ]
}

```

7.24.2 6.24.2 怎么看

想看什么	看哪
接口存在吗	paths 里找路径
接口签名	对应 method 下的 operationId/parameters
请求体字段	requestBody.schema 的 \$ref → schemas
响应类型	responses["200"].content schema
字段类型	components.schemas..properties

浏览器打开 <http://localhost:3000/api-docs/openapi.json> 在线查看——比读代码更快。

7.25 6.25 常见 utoipa 报错与修复

报错	原因	修复
error: expected one of ...	注解语法错误	对照 6.12 模板检查括号/逗号
the trait bound X: ToSchema is not satisfied	结构体忘了 derive	加 #[derive(ToSchema)]
operation id not unique	operation_id 重复	全局唯一命名
schemas X is not defined	用了但没登记	api_docs.rs schemas 列表加上
path X has no responses	注解缺 responses	补 responses(...)
macro expansion error	属性名写错	对照属性全表

7.25.1 6.25.1 一个实战修复案例

报错: the trait bound `PaginationParams: ToSchema` is not satisfied

分析: city3d 的分页参数用了 IntoParams 但 schema 登记处没加

修复: api_docs.rs components(schemas(...)) 加上 PaginationParams

通用原则: utoipa 报错 = 注解与登记不完整, 按 6.3/6.4 逐项核对。

7.26 6.26 06 章最终自测 (全集)

1. gen:api 的两步各做什么?
2. ToSchema / utoipa::path / IntoParams 各自适用对象?
3. operation_id 与 orval 函数名的关系?
4. export_openapi 测试断言什么?
5. tags-split 与 clean 的意义?

6. ApiResponse 泛型如何穿透生成?
 7. enum 的字符串值由谁决定?
 8. WS 类型为何手写? 风险在哪?
 9. 新增接口要动哪些文件 (后端)?
 10. 修改 DTO 字段后前端哪里会报错?
 11. openapi.json 的顶层结构?
 12. utoipa 报错的一般排查顺序?
- 12 题全对 → 06 章毕业。** 下一章: 使用与维护手册。

7.27 6.27 深入: 类型同步的日常三问

7.27.1 6.27.1 “我改了后端, 前端要做什么?”

改了 DTO 字段 / 接口路径 / 接口参数 / 响应类型

- 一律先 `npm run gen:api`
- 看 `vue-tsc` 报错, 逐处修复
- `npm run build` 通过即完成

例外: 只改了 handler 内部逻辑 (签名不变) → 无需 `gen:api`; 改了 WS 事件 payload → 手改前端 `types.ts`。

7.27.2 6.27.2 “为什么有时候 `gen:api` 后生成的函数少了?”

可能原因:

1. tag 改名 → `tags-split` 按新 tag 生成新文件 (旧文件被 `clean` 删除)
 2. `api_docs.rs` 忘了登记新接口 → `openapi.json` 里没有
 3. `operation_id` 改了 → 函数名变化 (`git diff` 可见)
- 排查: `git diff generated/ + openapi.json` 对照

7.27.3 6.27.3 “这个类型是手写还是生成的?”

类型	来源
ApiResponse / 各 DTO / Permission	generated (改 Rust)
WS 事件类型	各前端 <code>types.ts</code> 手写
前端自己的业务类型	手写 (如菜单类型)
工具函数类型	手写

判断口诀: 与后端数据结构相关的 → 生成; 与传输协议 (WS) 相关或纯前端概念 → 手写。

7.28 6.28 深入: OpenAPI 生成的历史与演进

7.28.1 6.28.1 项目类型方案的演进

阶段一: 手写 TS 类型 (历史初期)

- 问题: 两端漂移、字段漏改

阶段二: `ts-rs` 生成类型 (中间方案)

- 改进: 类型自动生成
- 局限: 请求函数仍手写 (路径/方法易错)

阶段三: `utoipa` + `OpenAPI` + `orval` (现行)

- 改进: 类型 + 请求函数 + 文档三位一体
- 约定: Rust 注解即契约

演进规律: 每次演进都消灭一类“人肉同步”——这正是工程化的方向。

7.28.2 6.28.2 为什么 orval 而不是其他生成器

生成器	生态	特点
orval	通用 OpenAPI	支持 Vue/React、custom-instance 灵活
openapi-generator	Java 生态	功能强但配置重
swagger-typescript-api	轻量	模板定制少

本项目选 orval：轻量、配置直观（orval.config.ts）、tags-split 正好匹配后端 tag 组织。

7.28.3 6.28.3 未来可能的演进

1. 全自动 CI：提交触发 gen:api + 契约校验（当前手动跑）
2. WS 类型生成：自定义 Rust 宏输出事件 schema
3. 前端单测快照：生成类型作为测试基准

判断标准：当“手动跑 gen:api”成为常态痛点时再自动化——当前规模手动即可。

7.29 6.29 深入：TypeScript 类型与 Rust 类型的边界

7.29.1 6.29.1 不会自动同步的边界

边界	说明
运行时校验	TS 类型只编译期检查，数据合法与否靠后端校验
Option 语义	<code>T null</code> 丢失“为什么为空”的信息
数值精度	<code>i64 > 2^53</code> 丢精度（当前自增 id 无碍）
日期时间	Rust DateTime ↔ TS string (ISO)
二进制	Vec ↔ number[]（帧数据走 WS 手写类型）

7.29.2 6.29.2 常见坑

```
// ① 时间字段是 string 不是 Date
item.created_at // string (ISO 格式)，展示需格式化
```

```
// ② null 与 undefined
res.data ?? [] // 后端 Option → 前端可能 null
```

```
// ③ 数字类型全 number
// 精度敏感场景（坐标）注意浮点误差
```

7.29.3 6.29.3 防御性编码习惯

```
// 读后端数据时假设“可能为空”
const name = item?.name ?? '未知'
const list = res.data ?? []
// 写数据时校验必填
if (!form.name) return ElMessage.warning('名称必填')
```

7.30 6.30 06 章终极综合题（真实工作场景）

7.30.1 场景：把 fw100 的 updated_at 从可选改为必填

1. Rust: Option<String> → String (models.rs)
2. gen:api
3. 前端哪里会报错？
 - 创建表单（没填 updated_at?）——业务上改为后端自动填
 - 详情展示（?.updated_at 变 .updated_at）
 - 列表列绑定（同样处理）

- 4. 数据库：旧数据 updated_at 为 NULL? → 迁移语句（UPDATE 补值）
- 5. 验证：build + 手动测试新旧数据展示

这道题覆盖：DTO 修改 → 生成 → 前端收窄 → 数据迁移 → 验证——类型同步全流程。

7.31 6.31 06 章结束语

类型同步机制是本项目的“工程化名片”。本章学完，你应该：1. 能解释整条链路（Rust → openapi.json → orval → TS）。2. 能熟练完成“改契约 → 同步 → 修复”的日常循环。3. 知道边界在哪（WS 手写、运行时校验在后端）。4. 能独立新增一个带 CRUD 的模块（契约先行）。

下一章（07）与再下一章（08）是“操作手册”与“扩展手册”——把知识变成生产力。

7.32 6.32 深入：OpenAPI 文件实操（用 json 文件做接口地图）

7.32.1 6.32.1 用 PowerShell 快速查接口

```
# 读取 openapi.json，列出所有路径
$doc = Get-Content openapi\openapi.json -Raw | ConvertFrom-Json
$doc.paths.PSObject.Properties.Name

# 查某个接口的详情
$doc.paths['/api/auth/login'].post.operationId

# 查某个 schema 的字段
$doc.components.schemas.UserInfo.properties | Format-List
```

7.32.2 6.32.2 用 jq（如装了）

```
jq '.paths | keys' openapi\openapi.json
jq '.paths["/api/auth/login"].post' openapi\openapi.json
```

把 openapi.json 当接口地图用——比翻代码快得多。

7.33 6.33 深入：如何调试“生成后类型不对”

7.33.1 6.33.1 常见问题与排查

问题	排查
字段类型错（如 String → number）	查 Rust 字段类型 + serde 注解
字段名变（camelCase/snake_case）	查 serde rename_all 配置
可选性错	查 Option<> + #[schema(default)]
枚举值不对	查 serde rename 映射
嵌套类型没生成	查子 struct 是否 ToSchema

7.33.2 6.33.2 调试链

```
Rust 代码 → openapi.json（看 schema 定义）
→ orval 生成（看 TS 定义）
→ 前端使用（看报错）
哪一步异常修哪一步（通常源头是 Rust serde 注解）
```

7.34 6.34 06 章最终检验：动手任务

7.34.1 任务 1：接口普查 (30 分钟)

1. 打开 openapi.json
2. 列出全部路径（数数有几个）
3. 找 3 个接口：登录、列表、服务状态
4. 说出各自 operationId 与对应前端函数名
5. 用 Swagger 风格（手动 POST）调用登录接口

7.34.2 任务 2：契约变更演练 (30 分钟)

1. 给 fw100 的 DTO 加一个字段（练习用）
2. gen:api
3. 观察 generated 变化（git diff）
4. 还原代码

7.34.3 任务 3：新增接口全流程 (60 分钟)

1. 后端加一个简单接口（如系统时间）
2. 注解 + 登记
3. gen:api + 前端 facade + 页面调用
4. 全链路验证

三个任务完成 → 06 章真正毕业。

7.35 6.35 深入：orval 生成代码的维护经验

7.35.1 6.35.1 生成文件版本管理

generated/ 与 openapi.json 都提交仓库
→ 好处：任何人 checkout 即可获得一致契约
→ 注意：不要手改；gen:api 会整体重写（clean: true）

7.35.2 6.35.2 生成后必做的三件事

1. git diff 看变化是否符合预期
2. 全局搜旧函数名（若 operation_id 变了）
3. 跑一次前端 build 捕获调用点报错

7.35.3 6.35.3 常用排查命令

```
# 看当前 spec 里有哪些接口
$doc = Get-Content openapi\openapi.json -Raw | ConvertFrom-Json
$doc.paths.PSObject.Properties.Name

# 看某个 schema
$doc.components.schemas.'Fw100LedgerItem'.properties | Format-Table
```

7.36 6.36 深入：utd 常见场景速查 (Rust 注解写错时的指引)

场景	正确写法
简单 GET	<code>#[utoipa::path(get, path="/api/x", tag="x", operation_id="xGet", responses((status=200, body=X)))]</code>
POST 带请求体	加 <code>request_body = XRequest</code>
路径参数	<code>params(("id" = i64, Path, ...))</code>
查询参数结构体	<code>params(XParams)</code> (需 <code>IntoParams</code>)
返回列表	<code>body = Vec<X></code>
返回分页	<code>body = PaginatedResult<X></code>
多个响应	<code>responses((status=200,...), (status=401,...))</code>

照抄模板比记忆属性可靠——找到现有相似接口复制修改。

7.37 6.37 深入：OpenAPI 版本与 utoipa 版本的注意事项

项目用 OpenAPI 3.0.3 + utoipa 5

→ 注意：OpenAPI 3.1 的 nullable 语义不同（用 type + oneOf）

→ 本项目 Option<T> 生成 nullable: true（3.0 风格）

→ 升级 utoipa 大版本时检查 schema 生成差异

7.38 6.38 06 章完结补充自测（5 题）

1. gen:api 后为什么要 git diff generated/?
2. operation_id 改了会导致什么?（函数名变化 → 调用点报错）
3. 如何快速查看 spec 里所有接口?
4. 路径参数在注解里怎么写?
5. 升级 utoipa 版本要注意什么?

这 5 题 + 前面任务完成 → 06 章彻底通关。

7.39 6.39 深入：Swagger UI 的对接方式（可视化调试）

7.39.1 6.39.1 项目现状

后端提供 /api-docs/openapi.json（实时生成）

前端无 Swagger UI 页面（浏览器直接看 JSON）

7.39.2 6.39.2 加一个 Swagger UI 页面（可选增强）

```
<!-- 在任意静态页面嵌入 Swagger UI（CDN 方式） -->
<link rel="stylesheet" href="https://unpkg.com/swagger-ui-dist/swagger-ui.css" />
<script src="https://unpkg.com/swagger-ui-dist/swagger-ui-bundle.js"></script>
<script>
  SwaggerUIBundle({ url: '/api-docs/openapi.json', dom_id: '#swagger-ui' })
</script>
```

价值：可视化查看参数/响应，直接“Try it out”调试接口。

7.40 6.40 深入：orval 与 axios 的交互细节

7.40.1 6.40.1 请求参数如何传递

GET 查询参数 → params 对象 → axios 拼到 URL

路径参数 → 模板字符串拼进 url

POST 数据 → data 字段

7.40.2 6.40.2 响应解包

```
// customInstance 把 axios response 解成 data
// 所以生成的函数直接返回 ApiResponse<T>
const res = await api.getList() // res 就是 ApiResponse
res.success / res.data / res.message
```

7.40.3 6.40.3 超时与重试

超时：createApiClient 设 10 秒

重试：未实现（需要拦截器加）

错误：axios error → 前端 catch

7.41 6.41 深入：契约变更的团队协作流程

7.41.1 6.41.1 变更前的沟通

1. 谁改契约（后端开发者）
2. 影响哪些前端（应用搜索 API 调用点）
3. 兼容性（新字段可空？接口参数变不变？）

7.41.2 6.41.2 变更后的同步节奏

1. 后端完成 → gen:api → openapi.json + generated 更新
2. 前端修调用点 → build 验证
3. 提交（契约变更与前端修复一起提交）

7.41.3 6.41.3 破坏性变更的处理

1. 参数删了/类型变了 → 前端必须同步改
2. 旧接口保留过渡 → 前端逐步迁移
3. 文档更新（本套文档相关章节）

7.42 6.42 深入：06 章补充自测（追加 5 题）

1. Swagger UI 怎么嵌入？
2. 请求参数如何传？（三种）
3. customInstance 解包到什么层级？
4. 契约变更的协作节奏？
5. 破坏性变更怎么处理？

答对 4+ → 06 章补充完成。

7.43 6.43 深入：从零看一次完整的契约变更（实战演练）

7.43.1 6.43.1 场景

给 fw100 的台账列表加一个“备注”字段（nullable）。

7.43.2 6.43.2 后端改动

```
// src/fw100/models.rs
#[derive(ToSchema, Serialize, Deserialize)]
pub struct Item {
    pub id: i64,
    // ... 原有字段
    pub remark: Option<String>, // 新增（可空，不破坏旧调用）
}
```

7.43.3 6.43.3 执行 gen:api

```
npm run gen:api
# cargo test export_openapi → openapi.json 更新
# orval → generated 更新
```

7.43.4 6.43.4 前端验证

```
// 类型已含 remark?: string
item.remark // string | undefined
// 模板里直接展示（undefined 显示空）
{{ item.remark ?? '-' }}
```

7.43.5 6.43.5 检查点

1. openapi.json 的 schema 里出现 remark
2. generated/model/*.ts 出现 remark
3. vue-tsc 无报错
4. 旧接口调用点无需改动（向后兼容）

要点：加可选字段是最平滑的变更；删字段/改类型才是破坏性的。

7.44 6.44 深入：OpenAPI 文档的阅读技巧

7.44.1 6.44.1 从 JSON 里快速找接口

```
# 用 PowerShell 看某个 tag 的接口
$spec = (Invoke-RestMethod http://localhost:3000/api-docs/openapi.json)
$spec.paths.'/api/fw100/items' # 看该路径的所有方法
```

7.44.2 6.44.2 常用字段含义

字段	含义
operationId	生成函数名
parameters	路径/查询参数
requestBody	POST/PUT 请求体
responses.200	成功响应 schema
tags	分组（决定生成文件名）

7.44.3 6.44.3 验证 schema 与代码一致

若怀疑类型过期 → 打开 openapi.json 找对应 schema
→ 与 Rust 结构体对比 → 不同即需 gen:api

7.45 6.45 深入：类型同步失败时的排查清单

症状	原因	修法
生成函数缺失	忘了 utoipa 注解	补 #[utoipa::path]
类型字段缺失	忘了 ToSchema	补 #[derive(ToSchema)]
类型不一致	改完没跑 gen:api	重跑
前端 TS 报错	调用点没更新	按报错改调用
生成乱码/失败	orval 配置问题	查 orval.config.ts
operationId 重复	断言失败	改 operation_id

口诀：报错顺序 = 排查顺序（先后端后前端）。

7.46 6.46 深入：06 章最终综合自测（追加 5 题）

1. 加可选字段的完整步骤？
2. 向后兼容的变更是什么样？
3. 从 openapi.json 找接口的方法？
4. operationId 的作用？
5. 类型不一致时的排查顺序？

答对 4+ → 06 章最终通过。

7.47 6.47 深入：DTO 设计的最佳实践（类型同步的前提）

7.47.1 6.47.1 请求 DTO vs 响应 DTO

```
// 请求（前端 → 后端）：只含必填字段
#[derive(Deserialize)]
pub struct CreateItemRequest {
    pub name: String,
    pub type_name: String,
}

// 响应（后端 → 前端）：含数据库生成字段
```

```
#[derive(Serialize, ToSchema)]
pub struct Item {
    pub id: i64,
    pub name: String,
    pub created_at: String,
}
```

原则：请求与响应分开定义（不共用），前端才知道哪些字段可传、哪些只读。

7.47.2 6.47.2 字段命名规范

```
// 后端 snake_case → 前端 camelCase
#[serde(rename_all = "camelCase")]
pub struct TableRow { pub ng_speed: f64, pub coolant_temp: f64 }
// 前端: row.ngSpeed / row.coolantTemp
```

这是项目契约的核心约定——所有 DTO 都遵守。

7.47.3 6.47.3 时间字段的序列化

SQLite TEXT (ISO8601) → serde 直接输出字符串
 前端 new Date(isoString) 解析
 注意：别让 serde 输出秒级时间戳（前端容易踩 NaN）

7.48 6.48 深入：WebSocket 事件的手写类型约定

7.48.1 6.48.1 为什么手写

WS 不进 OpenAPI（生成器只管 HTTP）
 → 事件类型前端手写于 types.ts
 → 与后端 serde 输出严格对应

7.48.2 6.48.2 手写模板

```
// frontend/fj200c_information/src/types.ts
export type WsMessage =
    | { type: 'frame'; data: TableRow; timestamp: number }
    | { type: 'status'; data: ServiceStatus }
    | { type: 'snapshot'; data: SnapshotData }
```

7.48.3 6.48.3 保持同步的检查点

1. 后端广播类型变更 → 同步改 types.ts
2. 用 discriminate union (type 字段区分)
3. 编译期 switch 穷尽检查 (default 分支报未处理)

7.49 6.49 深入：06 章实战自测（5 题）

1. 请求 DTO 与响应 DTO 为何分开？
2. camelCase 转换的注解？
3. 时间字段序列化的坑？
4. WS 类型为什么手写？
5. discriminate union 的好处？

答对 4+ → 06 章实战通过。

7.50 6.50 深入：OpenAPI 生成流程的故障排查演练

7.50.1 6.50.1 场景一：gen:api 后前端函数没有出现

排查：

1. 后端 handler 有没有 #[utoipa::path]？
2. api_docs.rs 有没有把 handler 加进 paths？
3. cargo test export_openapi 有没有通过（断言失败会报错）？

4. `orval.config.ts` 的 `input` 路径对不对?
5. 生成的 `api/xxx.ts` 文件生成了吗?

7.50.2 6.50.2 场景二：生成的类型与后端不一致

排查：

1. 重新跑一次完整 `gen:api`（可能有缓存）
2. 检查 DTO 的 `ToSchema` 派生
3. 检查 `utoipa` 的 `schema` 是否引用旧路径
4. 对比 `openapi.json` 与结构体

7.50.3 6.50.3 场景三：vue-tsc 大量报错

1. 契约变了（改字段/删字段）→ 同步改调用点
2. 生成代码被手改过 → 重跑恢复
3. 类型引用路径错误 → 检查 `import` 路径

7.51 6.51 深入：类型同步的自动化与检查点

7.51.1 6.51.1 项目内的自动化

`cargo test export_openapi`: 断言 `openapi.json` 与代码一致（防漂移）
→ 任何 `handler` 变更不跑 `gen:api`，测试就挂
→ 这是“自动化守护”机制

7.51.2 6.51.2 手动检查点（何时该跑 `gen:api`）

1. 改了任何 DTO（字段/结构）
2. 加了/改了 `handler` 签名
3. 改了路由路径
4. 删了接口

7.51.3 6.51.3 生成文件的管理纪律

1. 生成文件全部提交（`openapi.json` + `generated/`）
2. 不手改生成文件
3. 提交信息标注“`gen:api`”（可追溯）

7.52 6.52 深入：06 章高频自测（8 题）

1. 函数没生成的四个排查点？
2. 类型不一致的排查？
3. 契约变更报错的处理？
4. 防漂移机制是什么？
5. 何时必须跑 `gen:api`？
6. 生成文件的提交纪律？
7. 断言失败会怎样？
8. 手改生成文件的后果？

答对 7+ → 06 章高频通过。

7.53 6.53 深入：utopia 注解的完整参考表

7.53.1 6.53.1 常用属性一览

```
#[utoipa::path(
    get, // 方法
    path = "/api/fw100/items", // 路径
    tag = "fw100", // 分组
    operation_id = "fw100ListItems", // 唯一 ID（生成函数名）
    params(PageParams), // 查询参数（DTO）
    request_body = CreateItemRequest, // 请求体（POST/PUT）
    responses( // 响应
        (status = 200, description = "成功", body = ApiResponse<Vec<Item>>),
```

```

        (status = 401, description = "未登录"),
        (status = 403, description = "无权限"),
    )
}]

```

7.53.2 6.53.2 operation_id 的命名规范

<tag> + <动作>: fw100ListItems / fw100CreateItem
 → 生成函数名（前端直接调用）
 → 全局唯一（重复会断言失败）

7.53.3 6.53.3 params 的使用

```

#[derive(ToSchema)]
pub struct PageParams {
    pub page: Option<i32>,
    pub page_size: Option<i32>,
    pub keyword: Option<String>,
}
// 前端生成: fw100ListItems(params?: PageParams)

```

7.54 6.54 深入：DTO 的校验与默认值

7.54.1 6.54.1 服务端校验

```

pub async fn create_item(
    State(state): State<AppState>,
    Json(req): Json<CreateItemRequest>,
) -> Result<..., AppError> {
    // 手动校验
    if req.name.is_empty() {
        return Err(AppError::BadRequest("名称不能为空".into()));
    }
    // 或 validator 宏（自动校验）
    req.validate()?;
}

```

7.54.2 6.54.2 前端校验与服务端校验

前端校验：体验（即时反馈）
 服务端校验：安全（不可绕过）
 → 两层都要有

7.54.3 6.54.3 默认值的约定

可选字段 → Option<T>（前端 undefined）
 带默认值的字段 → serde(default) + 服务端填充

7.55 6.55 深入：类型契约的演进记录（项目复盘）

7.55.1 6.55.1 从 ts-rs 到 utoipa

旧方案：ts-rs（Rust 结构 → TS 类型）
 新方案：utoipa（Rust 结构 → OpenAPI → TS 类型 + 请求函数）

7.55.2 6.55.2 升级的好处

1. 不止类型：连请求函数一起生成
2. 有 openapi.json 文档（Swagger 可读）
3. 断言防漂移（测试自动化）
4. 社区标准（生态更好）

7.55.3 6.55.3 升级的教训

1. 迁移时要重新生成所有类型
2. 手写类型（WS）保持手写
3. 生成代码格式由 orval 控制（别手改）

7.56 6.56 深入：06 章终局自测（8 题）

1. utoipa 常用属性有哪些？
2. operation_id 命名规范？
3. params 的作用？
4. 服务端校验的必要性？
5. 可选字段的约定？
6. 从 ts-rs 到 utoipa 的变化？
7. 升级的好处？
8. 迁移的教训？

答对 7+ → 06 章终局通过。

7.57 6.57 深入：orval 生成代码的完整解读

7.57.1 6.57.1 生成文件结构

```
packages/shared/src/api/generated/  
├── api/  
│   ├── fw100.ts      # 请求函数（按 tag 分文件）  
│   └── auth.ts  
├── model/  
│   ├── item.ts       # 类型定义  
│   └── pageParams.ts  
└── index.ts          # 汇总导出
```

7.57.2 6.57.2 生成的请求函数长什么样

```
// generated/api/fw100.ts  
export const fw100ListItems = (  
  params?: PageParams,  
  options?: AxiosRequestConfig  
) => {  
  return customInstance<ApiResponse<Item[]>>({  
    url: `/api/fw100/items`,  
    method: 'GET',  
    params,  
    ...options,  
  })  
}
```

7.57.3 6.57.3 生成的模型类型

```
// generated/model/item.ts  
export interface Item {  
  id: number  
  name: string  
  typeName: string  
  remark?: string // Option<T> → 可选  
}
```

7.57.4 6.57.4 使用约定

1. 只 import，不修改
2. 应用层包一层 facade（api/index.ts）
3. 类型从 generated/model 导入（不手写）

7.58 6.58 深入：契约设计的常见决策

7.58.1 6.58.1 接口粒度的选择

粗粒度：一次返回全部（列表 + 详情）
细粒度：分开接口（list / detail）
建议：列表返回摘要，详情单独接口

7.58.2 6.58.2 批量操作的设计

批量删除: DELETE /api/xxx/items?ids=1,2,3
或 POST /api/xxx/batch-delete { ids: [] }
→ 前端统一调用 (避免循环请求)

7.58.3 6.58.3 错误响应的设计

统一 ApiResponse<T>:
{ success, data?, message?, code? }
→ 前端拦截器统一处理
→ 错误信息直达用户

7.59 6.59 深入: 06 章毕业自测 (8 题)

1. 生成文件的三层结构?
2. 请求函数的参数怎么传?
3. Option 字段生成什么?
4. 生成文件的四条使用约定?
5. 接口粒度的建议?
6. 批量删除的两种设计?
7. 统一错误响应的结构?
8. 为什么应用层要包 facade?

答对 7+ → 06 章毕业。

7.60 6.60 深入: 共享类型设计的最佳实践

7.60.1 6.60.1 什么时候放 shared

1. 跨应用使用 (用户/权限/角色)
2. 通用组件 (导航/表格/表单)
3. 通用工具 (httpClient/日期)
4. 生成的 API 代码 (所有应用用)

7.60.2 6.60.2 什么时候不放 shared

1. 应用特有页面组件
 2. 应用特有类型 (页面局部)
 3. 业务差异大的逻辑
- 判断标准: ≥2 应用用才共享

7.60.3 6.60.3 shared 的结构

```
packages/shared/src/  
├─ api/generated/      # orval 生成 (只读)  
├─ roles.ts            # 角色注册表缓存/菜单  
├─ types.ts            # Permission 等 re-export  
├─ composables/        # useTheme 等  
└─ utils/              # httpClient 等
```

7.61 6.61 深入: 类型重构的完整流程 (改名/迁移)

7.61.1 6.61.1 场景

把 Item 改名为 DeviceItem (涉及后端 DTO + 前端类型)

7.61.2 6.61.2 步骤

1. 后端 DTO 改名 (models.rs)
2. gen:api → generated 更新
3. 前端全局替换 Item → DeviceItem (IDE 重命名)
4. vue-tsc 检查残余
5. 手动验证关键页面

7.61.3 6.61.3 重命名的风险

1. 生成文件与手写文件混用 → 只认 generated
2. 字符串引用 (API 路径) → 不受类型改名影响
3. 同名冲突 (多个 Item) → 加前缀区分

7.62 6.62 深入：06 章大师自测 (8 题)

1. shared 放什么的判断?
2. shared 的四块结构?
3. 类型重命名的五步?
4. 改名的主要风险?
5. 为什么生成文件只读?
6. 字符串引用的特点?
7. 同名冲突怎么解决?
8. 跨应用共享的判断标准?

答对 7+ → 06 章大师。

7.63 6.63 深入：契约变更的完整场景练习

7.63.1 6.63.1 场景一：加查询参数

```
// 后端：分页加排序参数
pub struct PageParams {
    pub page: Option<i32>,
    pub page_size: Option<i32>,
    pub sort_by: Option<String>, // 新增
    pub sort_dir: Option<String>, // 新增
}
```

流程: gen:api → 前端类型自动带 sortBy/sortDir
→ 表格排序触发时传参
→ 无破坏 (可选参数)

7.63.2 6.63.2 场景二：拆字段

```
// 旧: name (含型号)
// 新: name + model (分开)
```

影响: 破坏性 (字段类型变了)
→ 前端所有用到 name 的地方检查
→ 建议后端保留 name 兼容 + 新字段

7.63.3 6.63.3 场景三：接口改名

路径 /api/xxx/items → /api/xxx/devices
→ operationId 变 → 前端函数名变
→ 全部调用点更新
→ 建议保留旧路径过渡

7.63.4 6.63.4 变更决策表

变更	破坏性	处理
加可选字段	否	直接加
加必填字段	是	前端同步 + 过渡期
改字段类型	是	全面排查
删字段	是	双版本过渡
改路径	是	旧路径重定向

7.64 6.64 深入：06 章权威自测（8 题）

1. 加查询参数的流程？
2. 拆字段的影响？
3. 接口改名的影响？
4. 五种变更的破坏性？
5. 必填字段的过渡方案？
6. 旧路径重定向的做法？
7. 为什么可选参数无破坏？
8. 双版本过渡的思路？

答对 7+ → 06 章权威。

7.65 6.65 深入：从 OpenAPI 到页面渲染的完整链路（收官复盘）

flowchart LR

```
A["Rust DTO<br/>#[derive(ToSchema)]"] --> B["utoipa 注解<br/>#[utoipa::path]"]
B --> C["cargo test export_openapi<br/>生成 openapi.json"]
C --> D["orval 生成<br/>TS 类型 + 请求函数"]
D --> E["shared/api/generated<br/>提交仓库"]
E --> F["应用 facade<br/>api/index.ts"]
F --> G["页面组件<br/>调用 + 渲染"]
G --> H["数据流<br/>axios → 后端"]
```

7.65.1 6.65.1 链路的关键点

1. 源头是 Rust（唯一真相）
2. 生成是自动的（人工不介入）
3. 中间产物都提交（openapi.json + generated）
4. 调用点在教育层（facade）
5. 改动一个字段 → 整链自动同步

7.65.2 6.65.2 如果链路断了

1. 后端改 DTO → 没跑 gen:api → 前端类型过期
2. 断言测试（export_openapi）拦截这类漂移
3. vue-tsc 在构建时报错兜底
4. 修复：跑 gen:api + 更新调用点

7.66 6.66 深入：06 章权威自测（8 题）

1. 画一条完整链路？
2. 链路的源头在哪？
3. 哪些产物提交仓库？
4. 调用点在哪一层？
5. 链路断了怎么办？
6. 断言测试的作用？
7. vue-tsc 如何兜底？
8. 为什么源头是 Rust？

答对 7+ → 06 章权威。

7.67 6.67 深入：类型系统的边界案例

7.67.1 6.67.1 生成代码覆盖不了什么

1. WebSocket 事件（手写）
2. 运行时动态结构（JSON 任意）
3. 前端本地类型（页面局部）
4. 跨应用业务类型（放 shared 手写）

7.67.2 6.67.2 手写与生成的分界

类型	来源
HTTP 请求/响应	生成
WS 事件	手写
Permission	后端枚举 → 生成 → re-export
角色菜单	手写 (前端 UI 概念)
本地表单	手写

7.67.3 6.67.3 处理边界的原则

1. 能生成的绝不手写
2. 手写的与后端保持同步 (注释来源)
3. 边界类型集中放 `types.ts` (便于检查)

7.68 6.68 深入：06 章权威自测 (8 题)

1. 生成代码覆盖不了的四类？
2. 手写与生成的分界表？
3. Permission 的来源？
4. WS 类型为什么手写？
5. 边界类型放哪？
6. 保持同步的方法？
7. 角色菜单为什么手写？
8. 能生成的绝不手写的意义？

答对 7+ → 06 章权威。

7.69 6.69 深入：契约的版本管理策略

7.69.1 6.69.1 契约文件的版本控制

`openapi.json` + `generated/` 都提交 `git`
→ 每次 `gen:api` 变更可见 (`git diff`)
→ 评审可发现意外变更
→ 回滚容易 (`git revert`)

7.69.2 6.69.2 契约变更的提交规范

1. 后端 DTO/接口变更 → 与 `gen:api` 产物同提交
2. 前端调用点同步 → 同一提交 (或紧随)
3. 提交信息写清：契约变更内容
4. 大变更拆分提交 (每个接口一个)

7.69.3 6.69.3 契约评审要点

1. `git diff` 只看 `openapi.json` → 评估影响面
2. 新增字段 → 兼容
3. 删除/改名 → 破坏性，需确认
4. 操作 ID 变更 → 前端函数名变化

7.70 6.70 深入：06 章权威自测 (8 题)

1. 契约文件的版本控制？
2. 提交规范的四条？
3. 评审的四个要点？
4. 为什么 `diff` 可见重要？
5. 大变更怎么拆？

6. 操作 ID 变更的影响?
7. git revert 的作用?
8. 提交信息怎么写?

答对 7+ → 06 章权威。

7.71 6.71 深入：本章收尾——契约思维总结

7.71.1 6.71.1 契约思维的核心

1. 数据形状先于代码 (DTO 先行)
2. 生成而非手写 (单一真相源)
3. 变更受控 (断言 + 评审)
4. 前后端解耦 (并行开发)

7.71.2 6.71.2 一个判断标准

"改一个字段，前端要不要动？"

- 不用动：契约设计得好 (向后兼容)
- 要动：确认是否破坏性，同步更新

7.71.3 6.71.3 学习小结

本项目的类型契约链路 (utoipa → OpenAPI → orval → 前端)
是整套系统最重要的工程机制之一
→ 理解它 = 理解前后端协作的核心

7.72 6.72 深入：06 章最终自测 (6 题)

1. 契约思维的四个核心?
2. 向后兼容的判断标准?
3. 破坏性变更的处理?
4. 为什么生成优于手写?
5. 并行开发的前提?
6. 契约链路的五段?

答对 5+ → 06 章最终完成。

下一节：07-使用与维护手册。

8 07 使用与维护手册

实操章节：从环境准备到日常运维，从部署发布到故障排查。照着做就能跑起整个系统。

8.1 7.1 环境准备 (第一次接手项目)

8.1.1 7.1.1 所需工具

工具	版本要求	用途
Rust 工具链	stable (1.7x+)	后端编译
Node.js	18+ (推荐 20)	前端构建
npm	9+	依赖管理
Git	任意	版本控制
可选: SQLite 客户端	任意	查看数据库
可选: VS Code	最新	开发编辑器

8.1.2 7.1.2 安装检查

```
# 检查版本
rustc --version      # rustc 1.8x.0
cargo --version
node --version       # v20.x
npm --version

# 检查 Windows 编译工具链 (cargo build 需要 MSVC)
# 安装过 Visual Studio Build Tools 且勾选 C++ 工作负载即可
```

8.1.3 7.1.3 克隆与依赖安装

```
git clone <仓库地址>
cd RustWeb-Vue
```

```
# 后端依赖 (自动下载编译, 首次较慢)
cargo build
```

```
# 前端依赖 (根目录一次安装, workspaces 统一)
npm install
```

注意: - npm install 必须在**根目录**执行 (7 个前端 + shared 一次装齐)。 - cargo build 首次下载 crates 需网络, 慢属正常 (可配置国内镜像加速)。

8.1.4 7.1.4 国内加速 (可选)

```
# 环境变量
$env:SPARSE_REGISTRY="sparse+https://mirrors.ustc.edu.cn/crates.io-index/"

# ~/.cargo/config.toml 配置
[source.crates-io]
replace-with = 'ustc'
[source.ustc]
registry = "sparse+https://mirrors.ustc.edu.cn/crates.io-index/"
```

8.2 7.2 启动开发环境

8.2.1 7.2.1 启动后端

```
# 项目根目录
cargo run
# 输出: listening on 127.0.0.1:3000
```

首次启动效果: - 自动创建 rustweb.db (SQLite 数据库 + 7 个种子账号)。 - 生成 config-fj200c_information.ini / config-fj200c_main.ini / config-ftj1c.ini。 - 自动生成 .env (若不存在)。 - 若 dist-*/ 目录存在则托管静态资源 (dev 模式通常没有)。

8.2.2 7.2.2 启动前端 (任一应用)

```
cd frontend/fj200c_information
npm run dev
# 输出: Local: http://localhost:5173/
```

浏览器访问 <http://localhost:5173/>, 用种子账号登录:

账号	密码	角色
admin@rustweb.dev	123456	admin
fj200c_information@rustweb.dev	123456	fj200c_information
fj200c_main@rustweb.dev	123456	fj200c_main
fw100@rustweb.dev	123456	fw100
fw150@rustweb.dev	123456	fw150
ftj1c@rustweb.dev	123456	ftj1c
city3d@rustweb.dev	123456	city3d

8.2.3 7.2.3 七个应用一起跑（多终端）

```
# 终端 1
cargo run
# 终端 2~8（各开一个终端）
cd frontend/admin      && npm run dev
cd frontend/fj200c_information && npm run dev
cd frontend/fj200c_main && npm run dev
cd frontend/ftj1c      && npm run dev
cd frontend/fw100      && npm run dev
cd frontend/fw150      && npm run dev
cd frontend/city3d     && npm run dev
```

或使用 **deploy.bat 的生产模式**（见 7.6）——8 个服务（1 后端 + 7 前端）是系统的常态。

8.2.4 7.2.4 开发环境架构

```
flowchart LR
    subgraph 浏览器
        B1[localhost:5173 发动机监控]
        B2[localhost:5174 管理后台]
        B3[localhost:5179 发动机测控]
        B4[其他端口...]
    end
    end
    B1 --> |"/api 代理| S[后端 :3000<br/>cargo run]
    B2 --> |"/api 代理| S
    B3 --> |"/api 代理 ws| S
    B4 --> |"/api 代理| S
    S --> DB[(rustweb.db)]
    S --> INI[config-*.ini]
```

开发模式关键点：前端 dev server 是独立进程，/api 代理到后端；WS 走 ws: true 代理。

8.3 7.3 日常开发 workflow

8.3.1 7.3.1 改后端 → 验证

```
# 1. 改代码
# 2. 编译检查（快）
cargo check
# 3. 跑测试（含 export_openapi 生成 openapi.json）
cargo test
# 4. 启动验证（cargo run 自动重编译）
cargo run
```

cargo run 每次改动自动重新编译——开发期直接用。

8.3.2 7.3.2 改前端 → 验证

```
# 1. 改代码 (dev server 热更新, 浏览器即看效果)
# 2. 类型 + 构建检查
npm run build
```

vue-tsc 类型检查是前端“测试”——构建通过即类型契约通过。

8.3.3 7.3.3 改 DTO/接口 → 全链路

```
npm run gen:api          # 类型同步
# 前端报错处逐一修复
npm run build            # 验证全前端
```

8.3.4 7.3.4 提交规范

```
git add .
git commit -m "feat(fw100): 增加维护记录模块" # 按功能写清
```

参考历史提交风格 (如 git log --oneline 所见): 前缀 + 模块 + 内容, 中文描述。

8.3.5 7.3.5 开发常用命令速查

命令	位置	用途
cargo run	根目录	启动后端
cargo check	根目录	后端编译快检
cargo test	根目录	全部测试 (含 openapi 生成)
npm run dev	frontend/*	启动前端
npm run build	frontend/*	前端构建检查
npm run gen:api	根目录	类型同步
deploy.bat	根目录	一键部署

8.4 7.4 数据库管理

8.4.1 7.4.1 数据库位置

模式	位置
开发	根目录 rustweb.db
部署	deploy/rustweb.db

8.4.2 7.4.2 表结构 (database.rs 建表)

```
users          # 用户表
└─ id / username / email / password_hash / role / is_active
```

只有一张业务表 (用户) ——台账/建筑等业务数据: fw100/fw150 有自己的台账表; city3d 有建筑/区域/事件表; fj200c 模块的数据走 CSV 文件而非数据库。详见 03 章 database.rs 精读。

8.4.3 7.4.3 查看与备份

```
# 用 SQLite 客户端打开 rustweb.db 查看
# 命令行方式 (需 sqlite3 或替代)
sqlite3 rustweb.db ".tables"
sqlite3 rustweb.db "SELECT * FROM users;"

# 备份: 直接复制文件 (SQLite WAL 模式需同时复制 -wal 文件)
```

```
Copy-Item rustweb.db rustweb.db.bak
Copy-Item rustweb.db-wal rustweb.db-wal.bak # 如有
```

8.4.4 7.4.4 重置数据库

```
# 删除后重启后端自动重建（含种子账号）
Remove-Item rustweb.db
cargo run
```

注意：重建后所有账号密码恢复 123456，业务数据清空——仅开发环境使用。

8.4.5 7.4.5 WAL 模式说明

```
// database.rs: 连接时设置 WAL + foreign_keys
PRAGMA journal_mode = WAL; // 写性能好、读写并发
PRAGMA foreign_keys = ON; // 外键约束
```

运维影响：运行中目录会出现 rustweb.db-wal、rustweb.db-shm 文件——正常现象，别删（删了可能丢未落盘数据）。备份时先停止服务或同时备份 wal。

8.5 7.5 配置文件详解

8.5.1 7.5.1 .env（后端运行时配置）

```
# 后端启动时若 .env 不存在会自动生成（deploy 时也如此）
PORT=3000 # 服务端口
DATABASE_URL=sqlite://rustweb.db # 数据库路径
JWT_SECRET=your-super-secret-jwt-key-change-this-in-production # 务必修改！
JWT_EXPIRATION=86400 # token 有效期（秒）= 24 小时
RUST_LOG=info # 日志级别：debug/info/warn/error
```

安全提醒：生产环境必须修改 JWT_SECRET（默认值公开于仓库）。

8.5.2 7.5.2 config-fj200c_information.ini（发动机监控）

```
[Mock]
InProgress = true ; 无硬件时模拟运行（开箱即用）

[Connection0]
Port = COM3 ; 串口
BaudRate = 115200
DataBits = 8
StopBits = 1
Parity = 0

[CSV]
Enabled = true
Dir = csv
```

生效时机：保存配置立即生效（热加载）——服务运行时修改无需重启。

8.5.3 7.5.3 config-fj200c_main.ini（发动机测控）

```
[COM]
Count = 3 ; 三路串口
Port1 = COM101 ; ECU
Port2 = COM103 ; ADAM
Port3 = COM105 ; DYNO

[MOCK]
SimulationMenu = true ; 模拟运行（无硬件时）

[REPORT]
StatePoints = 30000~53000 ; 报表状态点
```

```
[CSV]
Dir = csv
```

生效时机：修改后需重启服务。

8.5.4 7.5.4 config-ftj1c.ini (UDP 通信)

```
[Udp]
Mock = true           ; 模拟数据
```

```
[IP]
; 16 路组播地址 (IP1 ~ IP16)
IP1 = 239.0.0.11
IP2 = 239.0.0.12
...
```

生效时机：修改后需重启服务。

8.5.5 7.5.5 配置修改流程总结

文件	生效方式	修改入口
.env	重启后端	直接编辑
config-fj200c_information.ini	热加载	页面“配置”页保存
config-fj200c_main.ini	重启服务	页面“设置”或直接编辑
config-ftj1c.ini	重启服务	页面 IP 配置保存

8.6 7.6 部署发布 (deploy.bat 详解)

8.6.1 7.6.1 一键部署

```
# 项目根目录
.\deploy.bat
```

8.6.2 7.6.2 deploy.bat 的 8 步 (AGENTS.md 记载)

```
flowchart TD
    A[1. 检查端口占用<br/>netstat + taskkill] --> B[2. 7 个前端依次 npm run build]
    B --> C[3. 检查 dist 产物]
    C --> D[4. cargo build --release --features embedded<br/>前端 dist 编译期内嵌]
    D --> E[5. 组装 deploy/ 目录<br/>复制 exe/.env/ini 生成]
    E --> F[6. 启动后端 exe]
    F --> G[7. 等待端口就绪]
    G --> H[8. 提示访问地址]
```

顺序不可颠倒：前端 dist 在编译期内嵌进 exe，必须先构建前端再编译后端。

8.6.3 7.6.3 部署产物

```
deploy/
├─ rust-web-backend.exe      # 单文件后端（内嵌 7 个前端）
├─ .env                     # 运行时配置（不存在自动生成）
├─ config-fj200c_information.ini
├─ config-fj200c_main.ini
├─ config-ftj1c.ini
├─ csv/                     # CSV 数据目录
└─ rustweb.db               # 数据库（启动后自动生成）
```

8.6.4 7.6.4 部署后访问

```
http://127.0.0.1:3000/      # 重定向到 /admin
http://127.0.0.1:3000/admin # 管理后台
http://127.0.0.1:3000/fj200c_information # 发动机监控
```



```
http://127.0.0.1:3000/fj200c_main          # 发动机测控
http://127.0.0.1:3000/fw100 /fw150 /ftj1c /city3d
```

8.6.5 7.6.5 部署注意事项

事项	说明
端口占用	deploy.bat 自动检测并结束占用进程（慎用：可能误杀其他程序）
修改 JWT_SECRET	部署后编辑 deploy/.env
数据保留	deploy/rustweb.db 不会被覆盖（首次自动生成）
外网访问	main.rs 绑定 127.0.0.1——需要外网改 0.0.0.0 重新编译
防火墙	外网场景需放行 3000 端口

8.7 7.7 用户与权限管理实操

8.7.1 7.7.1 管理用户

- 1. 登录 admin@rustweb.dev / 123456
- 2. 管理后台 → 用户管理
- 3. 新建用户：用户名/邮箱/密码/角色
- 4. 编辑：改角色（权限随之变化）
- 5. 删除：不可删自己（防自锁）

8.7.2 7.7.2 权限与应用的对应

角色	能访问
admin	管理后台（用户管理）
fj200c_information	发动机监控
fj200c_main	发动机测控
fw100 / fw150	对应台账
ftj1c	UDP 监控
city3d	3D 展示

权限变更流程（新增角色/权限）：见 08 章「新增角色」完整流程。

8.7.3 7.7.3 重置密码

```
# 直接改数据库（开发环境）
sqlite3 rustweb.db "UPDATE users SET password_hash = '<bcrypt>' WHERE username = 'xxx';"
# 或删除用户重建（数据简单时）
# 或联系开发者提供临时重置工具
```

密码是 bcrypt 哈希，不能看明文——遗忘只能重置。

8.8 7.8 故障排查手册（症状 → 原因 → 解决）

8.8.1 7.8.1 后端启动失败

症状	原因	解决
address already in use	3000 端口被占	找出并结束占用进程
database locked	多实例抢数据库	只跑一个后端实例
.env 配置错误	端口/数据库路径无效	修正 .env
编译失败	Rust 代码问题	cargo check 看具体报错
sqlite: create_if_missing 失败	目录无写权限	检查运行目录权限

8.8.2 7.8.2 前端启动失败

症状	原因	解决
端口被占	517x 端口冲突	strictPort 报错, 改端口或关进程
Cannot find module	依赖没装	根目录 npm install
proxy 报错	后端没启动	先启动 cargo run
构建失败	类型错误	按 vue-tsc 报错修
白屏	路由/守卫问题	看 Console 报错

8.8.3 7.8.3 登录问题

症状	原因	解决
登录失败提示	用户名/密码错	确认账号角色
登录成功但跳回登录页	角色注册表未加载/权限空	检查 /api/meta/roles 返回
401 循环	token 过期	清 localStorage 重新登录
某应用进不去	角色权限不匹配	用对应用户登录

8.8.4 7.8.4 数据问题

症状	原因	解决
监控无数据	模拟未开/串口未接	配置 Mock/硬件
表格不刷新	WS 未连接	Network → WS 检查
CSV 没生成	CSV 开关/目录	检查 [CSV] 配置
配置不生效	需重启	重启服务/后端

8.8.5 7.8.5 日志定位

```
# 后端日志（RUST_LOG 控制）
$env:RUST_LOG = "debug"
cargo run
# 看：请求、错误、WS 连接、服务状态变化
```

排查通用流程：现象 → 前端 Console/Network → 后端日志 → 数据库/配置文件 → 定位修复。

8.9 7.9 性能与容量管理

8.9.1 7.9.1 资源占用评估

组件	资源
后端 exe	内存几十 MB 级
SQLite	单文件，容量受限小（业务数据小）
CSV	按帧写入，长时间运行会增长
前端	浏览器内存（实时图表限长缓冲）

8.9.2 7.9.2 容量关注点

项	风险	对策
csv/ 目录	长时间录制文件膨胀	定期清理/归档
rustweb.db	台账数据增长	定期备份
实时图表	前端内存	限长缓冲（已有）
WS 连接数	多客户端	单机部署限制浏览器数

8.9.3 7.9.3 性能调优开关

[CSV] Enabled = false # 不录 CSV 省磁盘/IO
[Mock] InProcess = true # 无硬件模拟省串口
RUST_LOG = warn # 减少日志 IO

8.10 7.10 备份与迁移

8.10.1 7.10.1 备份清单

[] deploy/rustweb.db（或 rustweb.db）
[] deploy/csv/ 目录（重要数据）
[] config-*.ini（配置）
[] .env（密钥）

8.10.2 7.10.2 迁移到新机器

1. 拷贝整个 deploy/ 目录到新机器
2. 双击 rust-web-backend.exe
3. 完成（数据库/配置随目录携带）

单 exe + 数据目录是完整的可迁移单元。

8.10.3 7.10.3 版本升级

1. 备份数据（数据库 + csv + 配置）
2. 重新构建 deploy.bat 覆盖 exe
3. 保留旧数据目录（deploy.bat 不覆盖）

8.11 7.11 安全加固清单

8.11.1 7.11.1 必须做的

项	操作
JWT_SECRET	生产环境改为随机长字符串
默认密码	首次登录立即改密码
服务绑定	内网用 127.0.0.1，外网慎重
日志	RUST_LOG 不输出敏感数据（默认不输出）

8.11.2 7.11.2 可选加固

1. 反向代理（Nginx）+ HTTPS
2. 数据库加密（SQLCipher）
3. 登录失败锁定/验证码
4. 用户密码策略（长度/复杂度强制）
5. 定期轮换 JWT_SECRET

系统定位：内网/单机工具型系统，安全基线是“不暴露公网 + 修改默认密钥”。

8.12 7.12 日常维护清单（每周/每月）

8.12.1 每周

- [] 检查 csv/ 目录大小，清理过期数据
- [] 查看后端日志有无异常错误
- [] 备份 rustweb.db
- [] 确认服务 24h 稳定运行（任务计划自启动可选）

8.12.2 每月

- [] 检查磁盘空间
- [] 归档 CSV（按项目/月份压缩）
- [] 更新依赖（npm update + cargo update，先测试）
- [] 审查用户列表（清理离职账号）
- [] 验证备份可恢复（在测试机跑一遍备份）

8.12.3 版本发布流程

1. 功能验证（7 个应用全点一遍）
2. npm run gen:api（契约一致性）
3. npm run build x7（前端类型）
4. cargo test（后端 + openapi）
5. deploy.bat（部署）
6. 生产冒烟测试（登录/监控/台账）

8.13 7.13 本章自测

1. 首次克隆后需要执行哪两条安装命令？
2. 种子账号有几个？默认密码？
3. 开发模式前端怎么连后端？(/api 代理)
4. 三个 ini 各是什么生效时机？
5. deploy.bat 为什么必须先构建前端？
6. 生产部署后数据库在哪？
7. 忘记密码怎么处理？
8. WAL 文件能删吗？
9. 迁移系统的步骤？

10. 安全加固的必做三项？

答对 8+ → 07 章通过。下一章：扩展与二次开发——新增角色/模块/应用/接口的完整指南。

8.14 7.14 深入：deploy.bat 逐段解读

8.14.1 7.14.1 整体骨架

```
@echo off
setlocal enabledelayedexpansion

REM ===== 1. 检查 3000 端口占用 =====
netstat -ano | findstr :3000 >nul
if %errorlevel%==0 (
    echo [WARN] 端口 3000 已被占用，尝试自动结束进程...
    for /f "tokens=5" %a in ('netstat -ano ^| findstr :3000') do taskkill /f /pid %a
)

REM ===== 2. 依次构建 7 个前端 =====
set APP_LIST=admin fj200c_information fj200c_main fw100 fw150 ftj1c city3d
for %a in (%APP_LIST%) do (
    echo ===== Building %a =====
    pushd frontend\%a
    call npm run build
    if errorlevel 1 ( echo [ERROR] %a build failed & exit /b 1 )
    popd
)

REM ===== 3. 编译后端 (embedded) =====
echo ===== Building backend =====
call cargo build --release --features embedded
if errorlevel 1 ( echo [ERROR] backend build failed & exit /b 1 )

REM ===== 4. 组装 deploy 目录 =====
if not exist deploy mkdir deploy
copy target\release\rust-web-backend.exe deploy\
REM 配置文件不存在时生成 (首启自动)
if not exist deploy\.env copy .env.example deploy\.env
...

REM ===== 5. 启动 =====
start "RustWeb" deploy\rust-web-backend.exe
echo Deployed at http://127.0.0.1:3000/
```

8.14.2 7.14.2 关键点解读

段	要点
端口检测	netstat -ano + taskkill /f /pid 强杀占用进程
前端顺序	APP_LIST 顺序即构建顺序 (无依赖, 顺序可换, 但 AGENTS.md 建议固定)
失败即停	if errorlevel 1 exit /b 1 任何一步失败终止
配置生成	首次启动后端自动生成 .env/ini (deploy 不覆盖已有)
启动方式	start 后台启动 exe, 命令窗口可关

8.14.3 7.14.3 自定义部署变体

```
# 只重新构建后端（前端没改时）
cargo build --release --features embedded
# 只重新构建某个前端
cd frontend/admin && npm run build
# 部署后手动重启
taskkill /f /im rust-web-backend.exe
start deploy\rust-web-backend.exe
```

8.15 7.15 深入：日志系统

8.15.1 7.15.1 日志级别

级别	内容	场景
error	严重错误	生产默认
warn	可恢复异常	排障
info	常规操作	日常观察
debug	详细流程（帧/事件）	深排障

8.15.2 7.15.2 设置方式

```
# 一次性
$env:RUST_LOG = "debug"
cargo run
```

```
# 或写进 .env
RUST_LOG=debug
```

8.15.3 7.15.3 日志输出到文件

```
# PowerShell 重定向
cargo run * > backend.log
# 或部署模式
deploy\rust-web-backend.exe * > backend.log 2>&1
```

排查实时数据问题时建议 debug：能看到 WS 连接、帧到达、CSV 写入等细节。

8.16 7.16 深入：服务自启动（无人值守运行）

8.16.1 7.16.1 任务计划程序（Windows）

```
# 开机自启 rust-web-backend.exe
schtasks /create /tn "RustWeb" /tr "D:\deploy\rust-web-backend.exe" /sc onstart /ru SYSTEM
# 或登录自启
schtasks /create /tn "RustWeb" /tr "D:\deploy\rust-web-backend.exe" /sc onlogon
```

8.16.2 7.16.2 简单方式（启动文件夹）

```
# 把快捷方式放入：
# %APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup
```

8.16.3 7.16.3 崩溃自恢复（可选）

```
# 配合任务计划：每 5 分钟检查一次进程，不在则启动
schtasks /create /tn "RustWebWatch" /tr "powershell -Command `if (!(Get-Process rust-web-backend -ErrorAction SilentlyContinue)) { Start-Process D:\deploy\rust-web-backend.exe }`" /sc minute /mo 5
```

8.17 7.17 深入：故障排查演练（5 个真实场景）

8.17.1 场景 A：部署后页面 404

现象：deploy 后访问 /admin 正常，访问 /admin/users 刷新 404

原因：SPA 深链接回退失效（embedded_router 未匹配到）

排查：

1. 直接访问 /admin 确认静态资源正常
2. 访问 /admin/users 看后端日志
3. 检查 embedded_assets.rs 的回退逻辑（未匹配 → index.html）

解决：回退逻辑修复后重新编译

8.17.2 场景 B：发动机监控启动服务即失败

现象：点"启动服务"后状态闪闪"已停止"，日志报错

原因：串口被占用 / 配置的端口不存在

排查：

1. 检查 config-fj200c_information.ini 的 [ConnectionN]
2. RUST_LOG=debug 看具体错误
3. 确认无其他程序占用串口

解决：改配置（Mock 或正确串口）→ 保存（热加载）→ 重试

8.17.3 场景 C：CSV 录制没有文件

现象：录制开关打开但 csv/ 目录无文件

原因：目录不存在/权限/配置 Dir 路径

排查：看后端日志的 CSV 写入错误

解决：确认 [CSV] Enabled=true、Dir 目录可写

8.17.4 场景 D：前端构建报大量类型错误

现象：npm run build 全应用报错

原因：后端 DTO 改动后没跑 gen:api

解决：npm run gen:api → 按报错更新调用点

8.17.5 场景 E：登录后立即被登出

现象：登录成功 → 跳转 → 又回登录页

原因：角色注册表加载失败 → 权限为空 → 守卫拒绝

排查：Network 看 /api/meta/roles 是否 200

解决：后端 roles.rs 注册表正常 + 重启后端

8.18 7.18 深入：数据库详细结构

8.18.1 7.18.1 users 表

```
CREATE TABLE IF NOT EXISTS users (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  username    TEXT NOT NULL UNIQUE,  
  email       TEXT NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,          -- bcrypt  
  role        TEXT NOT NULL,          -- 角色 key（对应注册表）  
  is_active   INTEGER NOT NULL DEFAULT 1,  
  created_at  TEXT NOT NULL DEFAULT (datetime('now'))  
);
```

8.18.2 7.18.2 其他表 (按模块)

表	模块	说明
fw100_ledger_items	fw100	设备台账
fw150_ledger_items	fw150	设备台账
city3d_buildings	city3d	建筑
city3d_regions	city3d	区域
city3d_events	city3d	事件
global_vars	全局	键值存储 (主题等)

8.18.3 7.18.3 global_vars 表 (fj200c_main 主题)

-- 主题持久化就存这

```
INSERT INTO global_vars (key, value) VALUES ('theme', 'space');
```

8.19 7.19 深入：监控与告警 (可选增强)

8.19.1 7.19.1 现有监控手段

手段	内容
前端页面	各应用实时数据 (人看)
后端日志	RUST_LOG 记录
状态接口	GET /api/*/service/status

8.19.2 7.19.2 可扩展的告警 (脚本级)

每 30 秒检查后端是否存活, 挂了发通知

```
while ($true) {  
    try { Invoke-RestMethod -Uri "http://127.0.0.1:3000/api/auth/login" -Method Post -Body  
        '{"username":"x","password":"y"}' -TimeoutSec 5 | Out-Null }  
    catch { Start-Process deploy\rust-web-backend.exe }  
    Start-Sleep 30  
}
```

8.20 7.20 使用维护 FAQ

Q: 可以把服务绑定到 0.0.0.0 供局域网访问吗? A: 改 src/main.rs 的绑定地址后重新编译 (embedded 模式)。注意: token 走 localStorage, 跨设备访问同源即可。

Q: 多个用户同时看监控会怎样? A: WS 广播, 多人可同时观看; 控制类操作 (启停) 需自行协调 (无互斥锁, 设计如此)。

Q: 能改成 MySQL/PostgreSQL 吗? A: sqlx 支持多数据库, 但建表/种子逻辑与 SQLite 语法耦合, 改造成本中等, 不建议。

Q: CSV 文件能按天分目录吗? A: [CSV] Dir 目前固定目录; 可按需改后端写文件逻辑 (file_name 拼日期)。

Q: 前端有暗色模式吗? A: 各应用支持暗色切换 (Element Plus 官方方案); fj200c_main 另有航天/仪表双主题。

8.21 7.21 本章自测 (追加)

11. deploy.bat 的失败即停机制怎么写?

- 12. RUST_LOG=debug 什么时候用?
- 13. 服务自启动的两种方式?
- 14. 部署后深链接 404 的排查步骤?
- 15. users 表有哪些字段?
- 16. 全局变量表做什么用?
- 17. 如何做存活监控脚本?
- 18. 局域网访问需要改什么?
- 19. 备份清单包含哪些?
- 20. 版本发布流程 6 步是什么?

答对 15+ → 07 章精通。 下一章：扩展与二次开发。

8.22 7.22 深入：开发环境多开与资源管理

8.22.1 7.22.1 8 个进程的资源占用

进程	数量	说明
cargo run (后端)	1	常驻
Vite dev server	7	每个前端一个
rust-web-backend.exe	0 (dev)	生产才有

内存合计：后端 ~50MB + 每个 Vite ~300MB (含 Node) ≈ 2.5GB 峰值——开发机建议 16GB 内存。

8.22.2 7.22.2 只需部分应用时的策略

```
# 后端 + 只开需要的应用（不是全部 7 个）
cargo run
cd frontend/fw100 && npm run dev      # 只开台账
```

7 个应用互不依赖，按需启动即可（shared 由各应用直接引源码，无需额外服务）。

8.22.3 7.22.3 关闭与清理

```
# 关闭所有 Vite（终端 Ctrl+C）
# 关闭后端（Ctrl+C）
# 确认无残留
netstat -ano | findstr :5173      # 逐个端口查
taskkill /f /pid <pid>           # 强制结束
```

8.23 7.23 深入：版本管理策略（Git 实践）

8.23.1 7.23.1 分支模型（建议）

```
main          # 稳定可发布
├─ dev        # 开发集成
└─ feature/xxx # 功能分支（按需）
```

当前仓库现状：单分支直接提交（小团队/个人项目常见）。建议至少做到：

- 1. 功能完成 + 验证后才提交
- 2. 提交信息描述功能（参考历史风格）
- 3. 发布前打 tag

8.23.2 7.23.2 发布打 tag

```
git tag v1.2.0 -m "发动机监控 v1.2.0"
git push origin v1.2.0
```

8.23.3 7.23.3 回滚（发布后出问题）

```
# 代码回滚（保留上次可用的构建产物）
git stash / git checkout <旧commit>
# 重新构建部署
.\deploy.bat
```

部署回滚的本质：旧代码重新编译 + 数据保留（数据库/配置不动）。

8.24 7.24 深入：后端运维操作

8.24.1 7.24.1 手动加用户（脚本）

```
# 需要 bcrypt 哈希——最稳妥方式：临时写个小测试/或用页面添加
# 开发环境偷懒法：直接用页面 + 种子账号（admin 登录后新建）
```

推荐：始终通过 admin 界面建用户（后端自动 bcrypt）。

8.24.2 7.24.2 修改角色

```
sqlite3 rustweb.db "UPDATE users SET role='fw100' WHERE username='xxx';"
# 改完用户下次登录权限即变（token 有效期 24h 内新权限不生效？）
```

注意：JWT 里的权限在登录时生成——改角色后**旧 token 仍带旧权限**，等过期或重新登录。
本项目 auth_me 每次请求都查数据库（动态权限），所以实际立即生效（具体以后端实现为准）。

8.24.3 7.24.3 禁用用户

```
UPDATE users SET is_active = 0 WHERE username = 'xxx';
-- 登录校验会拒绝 inactive 用户
```

8.25 7.25 深入：CSV 数据处理

8.25.1 7.25.1 CSV 文件格式

```
# csv/ 目录，按会话命名（如 fj200c_information_20260808_101500.csv）
# 每行一个采样点，列对应解码字段
时间戳,转速,水温,油压,...
```

8.25.2 7.25.2 数据分析

```
# 用 Excel/Python 打开 CSV 分析
python -c "import pandas as pd; df=pd.read_csv('csv/xxx.csv'); print(df.describe())"
```

8.25.3 7.25.3 数据归档策略

周期	操作
每日	确认录制正常
每周	归档到外部存储
每月	清理 3 个月前的旧文件（确认已归档）

8.26 7.26 深入：配置与环境的组合场景

场景	配置组合
无硬件演示	三个 ini 全 Mock/Simulation + admin 演示账号
真机采集	Mock=false + 串口/UDP 正确配置
只做台账	后端 + fw100/fw150 即可
只做监控	后端 + fj200c_information
部署到客户机	deploy.bat + 客户机硬件

配置隔离：开发/演示/生产可用不同目录部署（每份目录独立 .env/ini/数据库）。

8.27 7.27 深入：健康检查与重启流程

8.27.1 7.27.1 健康检查

```
# API 层：登录接口即健康探针
Invoke-RestMethod -Uri "http://127.0.0.1:3000/api/auth/login" -Method Post -Body
'{"username":"admin@rustweb.dev","password":"123456"}' -ContentType "application/json" |
Select success

# 静态层：首页返回
Invoke-WebRequest -Uri "http://127.0.0.1:3000/" | Select StatusCode
```

8.27.2 7.27.2 标准重启流程

```
# 1. 通知使用方（若有人用）
# 2. 关闭后端
taskkill /f /im rust-web-backend.exe
# 3. 备份数据（可选）
Copy-Item deploy\rustweb.db deploy\rustweb.db.$(Get-Date -Format yyyyMMdd)
# 4. 启动
Start-Process deploy\rust-web-backend.exe
# 5. 验证
Invoke-WebRequest -Uri "http://127.0.0.1:3000/" | Select StatusCode
```

8.28 7.28 07 章收官：维护者能力清单

读完本章，你应该能独立完成：

能力	验证
环境搭建	新机器 30 分钟内跑起全部 7 应用
日常开发	改代码 → 验证 → 提交
配置管理	三 ini + .env 的修改与生效时机
部署发布	deploy.bat 全流程 + 回滚
用户管理	建/改/禁用用户
故障排查	症状 → 日志 → 定位 → 解决
备份迁移	清单 + 迁移 + 恢复验证
安全加固	默认密钥修改 + 内网策略

07 章结束。最后一章：扩展与二次开发——把系统长出新的能力。

8.29 7.29 深入：前后端联调实践

8.29.1 7.29.1 联调准备

1. 后端跑起来 (cargo run, 端口 3000)
2. 前端跑起来 (npm run dev, 对应端口)
3. 双方约定接口契约 (openapi.json 为准)

8.29.2 7.29.2 联调流程

flowchart LR

```
A[确认 openapi.json] --> B[前端调接口]
B -->|失败| C{看哪层}
C -->|404| D[后端路由未注册/路径不符]
C -->|403| E[权限中间件/账号角色]
C -->|500| F[后端代码错误/看日志]
C -->|数据不对| G[DTO 字段/序列化]
C -->|401| H[token 问题]
```

8.29.3 7.29.3 联调利器

工具	用途
Swagger UI (openapi.json)	快速试接口
F12 Network	看请求/响应
后端 RUST_LOG=debug	看后端处理
curl / Invoke-RestMethod	命令行测接口

PowerShell 快速测接口

```
$body = @{ username = "admin@rustweb.dev"; password = "123456" } | ConvertTo-Json
Invoke-RestMethod -Uri "http://127.0.0.1:3000/api/auth/login" -Method Post -Body $body -
ContentType "application/json"
```

8.30 7.30 深入：多人协作注意事项

8.30.1 7.30.1 协作分工建议

后端开发: Rust 模块 + openapi 契约

前端开发: shared + 各应用

联调: gen:api 后全链路 build

8.30.2 7.30.2 冲突避免

区域	冲突风险	对策
api_docs.rs	高 (集中登记)	按模块加注释分隔, 逐模块添加
generated/	高 (重写)	生成物不手改; 提交时确认
roles.rs	中	加角色前沟通
shared/template	中	改动通知所有应用
各前端	低 (独立目录)	互不影响

8.30.3 7.30.3 提交纪律

1. 提交前跑 npm run build + cargo test
2. generated/openapi.json 随代码一起提交
3. 提交信息注明影响范围
4. 不提交 .env / deploy/ / dist / target (.gitignore 已有)

8.31 7.31 深入：常见部署环境

8.31.1 7.31.1 客户端单机（最常见）

场景：客户机房一台 Windows 电脑
方式：deploy.bat 构建 → 拷贝 deploy/ 目录 → 双击 exe
要点：防火墙放行 3000（如局域网访问）、开机自启

8.31.2 7.31.2 服务器（内网多用户）

场景：内网服务器，多用户浏览器访问
方式：同单机部署，绑定 0.0.0.0（改 main.rs 重编译）
要点：账号管理、数据备份、并发数评估

8.31.3 7.31.3 虚拟机/测试环境

场景：隔离测试
方式：任意，注意端口冲突

8.32 7.32 深入：性能基准与优化实验

8.32.1 7.32.1 基准指标（参考）

指标	参考值
登录延迟	<50ms（本机）
列表接口	<100ms（千行内）
WS 帧延迟	<200ms（节流间隔内）
前端首屏	<3s（dev） / <1s（build）

8.32.2 7.32.2 常见优化实验

实验 1：调大 WS 节流间隔（200ms → 500ms）
→ 观察 CPU 与流畅度平衡

实验 2：表格加 height + 分页
→ 千行数据滚动流畅度对比

实验 3：RUST_LOG 从 debug 改 info
→ 日志 IO 与磁盘占用

实验 4：CSV Enabled 开关
→ 磁盘 IO 影响

原则：先量化（记录前后数据）再调优；本项目规模下“够用”优先。

8.33 7.33 07 章最终自测（全集）

- 1. 首次克隆的两条安装命令？
- 2. 种子账号与默认密码？
- 3. dev 模式前后端连接方式？
- 4. 三 ini 的生效时机？
- 5. deploy.bat 步骤顺序与原因？
- 6. 生产数据库位置？
- 7. 密码忘记怎么办？
- 8. WAL 文件可否删除？
- 9. 系统迁移步骤？
- 10. 安全加固必做三项？

- 11. deploy.bat 失败即停怎么写?
- 12. RUST_LOG 分级与用途?
- 13. 自启动两种方式?
- 14. 深链接 404 排查?
- 15. users 表字段?
- 16. 全局变量表用途?
- 17. 存活监控脚本思路?
- 18. 局域网访问改什么?
- 19. 备份清单?
- 20. 版本发布六步?
- 21. 联调失败的分层排查?
- 22. 协作冲突热点与对策?
- 23. 单机部署流程?
- 24. 性能优化原则?
- 25. 健康检查命令?

答对 20+ → 07 章精通。下一章 (终章): 扩展与二次开发。

8.34 7.34 深入：构建模式与产物形态全梳理

8.34.1 7.34.1 后端两种构建模式

```
# 开发模式 (默认)
cargo run
# 静态资源: 读磁盘 dist-*/ 目录 (若存在)
# 作用: 开发迭代快

# 生产模式 (embedded feature)
cargo build --release --features embedded
# 静态资源: 7 个前端 dist 编译期内嵌进 exe
# 作用: 单文件分发, 双击即用
```

8.34.2 7.34.2 dist-* 目录 (dev 静态托管)

```
# dev 模式下后端若发现 dist-*/ 目录会托管:
dist-admin/
dist-fj200c_information/
dist-fj200c_main/
dist-fw100/
dist-fw150/
dist-ftj1c/
dist-city3d/
```

用途: 不跑 Vite 时也能看生产形态 (先把前端 build 到这些目录)。

8.34.3 7.34.3 前端产物对照

```
# dev 模式产物: 内存中 (Vite dev server)
# build 产物: frontend/*/dist/ (deploy.bat 用这些内嵌)
# 直接形态: dist-*/ (后端 dev 托管)
```

8.34.4 7.34.4 三种形态的使用场景

形态	命令	场景
Vite dev server	npm run dev	日常开发
dist-* + cargo run	npm run build 到对应目录	不装 Node 环境预览
embedded exe	deploy.bat	正式交付

8.35 7.35 深入：运维脚本合集（收藏）

8.35.1 7.35.1 备份脚本 backup.ps1

```
# 一键备份：数据库 + CSV + 配置
$stamp = Get-Date -Format "yyyyMMdd_HHmms"
$dest = "backups\$stamp"
New-Item -ItemType Directory -Force -Path $dest | Out-Null

Copy-Item deploy\rustweb.db "$dest\" -ErrorAction SilentlyContinue
Copy-Item deploy\*.ini "$dest\" -ErrorAction SilentlyContinue
if (Test-Path deploy\csv) { Copy-Item deploy\csv "$dest\csv" -Recurse }

Compress-Archive -Path "$dest\*" -DestinationPath "$dest.zip"
Remove-Item $dest -Recurse
Write-Host "备份完成：$dest.zip"
```

8.35.2 7.35.2 清理脚本 clean-csv.ps1

```
# 清理 30 天前的 CSV（先备份）
$cutoff = (Get-Date).AddDays(-30)
Get-ChildItem deploy\csv\*.csv | Where-Object { $_.LastWriteTime -lt $cutoff } |
    Move-Item -Destination "backups\csv_archive"
Write-Host "已归档 $((Get-ChildItem backups\csv_archive).Count) 个旧 CSV"
```

8.35.3 7.35.3 健康检查脚本 health.ps1

```
# 检查后端存活 + 前端入口可访问
$checks = @("admin", "fj200c_information", "fj200c_main", "fw100", "fw150", "ftj1c",
"city3d")
foreach ($app in $checks) {
    try {
        $r = Invoke-WebRequest -Uri "http://127.0.0.1:3000/$app/" -TimeoutSec 5
        Write-Host "$app : $($r.StatusCode)"
    } catch {
        Write-Host "$app : 异常 - $($_.Exception.Message)"
    }
}
```

8.35.4 7.35.4 开机自启 + 存活守护（组合）

```
# 任务计划：登录自启后端
schtasks /create /tn "RustWeb-Boot" /tr "D:\deploy\rust-web-backend.exe" /sc onlogon /f
# 任务计划：每 5 分钟存活守护
schtasks /create /tn "RustWeb-Watch" /tr "powershell -ExecutionPolicy Bypass -File D:\
\scripts\watch.ps1" /sc minute /mo 5 /f
```

8.36 7.36 深入：日志分析指南

8.36.1 7.36.1 日志长什么样

```
[2026-08-08T10:15:32Z INFO axum::rejection] request GET /api/fj200c_information/service/
status 200
[2026-08-08T10:15:35Z INFO fj200c_information::service] connection 0 started, port COM3
[2026-08-08T10:15:36Z DEBUG fj200c_information::decode] frame parsed: len=100, valid=true
[2026-08-08T10:15:40Z ERROR axum::response] internal error: serial port busy
```

8.36.2 7.36.2 快速过滤技巧

```
# PowerShell 过滤关键信息
Select-String -Path backend.log -Pattern "ERROR|error" # 只看错误
Select-String -Path backend.log -Pattern "connection.*start" # 连接事件
Select-String -Path backend.log -Pattern "frame parsed" -Context 2,2 # 帧解析上下文
```

8.36.3 7.36.3 常见日志模式解读

日志模式	含义	行动
serial port busy	串口被占	关占用程序/换口
connection N started	连接建立	正常
reconnect	断线重连	检查硬件/线路
invalid frame	帧校验失败	检查协议/波特率
csv write error	CSV 写入失败	检查目录权限
jwt invalid	token 无效	重新登录

8.37 7.37 07 章收官：运维能力地图

flowchart TD

A[07 章使用与维护] --> B[环境/启动]
A --> C[日常开发]
A --> D[配置管理]
A --> E[部署发布]
A --> F[故障排查]
A --> G[备份迁移]
A --> H[安全加固]
A --> I[运维脚本]
B --> B1[工具链/多开]
C --> C1[工作流/提交规范]
D --> D1[三 ini + .env]
E --> E1[deploy.bat 8 步]
F --> F1[症状→原因→解决]
G --> G1[备份清单/迁移]
H --> H1[JWT/绑定/密码]
I --> I1[backup/clean/health]

运维者自评：以上 9 个能力块都能独立执行 → 07 章毕业。

8.38 7.38 深入：改造决策参考（动哪一层，影响多大）

8.38.1 7.38.1 常见改造请求的影响面

改造请求	影响层	工作量	风险
改界面文案/颜色	前端 views/styles	小	低
加一个表单字段	前端 + 后端 DTO	中	低
加一个新页面	前端 + 后端接口	中	低
加监控数据源	后端模块 + 前端	大	中
改认证方式 (OAuth)	后端 auth + 前端	大	高
换数据库	后端 database.rs 全链路	大	高
加硬件协议	后端模块 + 前端	大	中

8.38.2 7.38.2 改造前的检查清单

- [] 改动是否影响 openapi 契约? → 需 gen:api
- [] 改动是否影响 shared? → 影响所有应用
- [] 改动是否影响数据格式? → 旧数据兼容 (CSV/DB)
- [] 是否影响权限模型? → roles.rs 改动
- [] 是否影响部署? → deploy.bat/embedded

8.38.3 7.38.3 最小改动原则

能只改前端不碰后端 → 只改前端（如过滤/排序/样式）
能加字段不改结构 → 加可选字段（Option）
能加接口不动旧接口 → 加新接口（向后兼容）

8.39 7.39 深入：性能数据采集方法

8.39.1 7.39.1 采集什么

指标	采集方法
接口延迟	F12 Network 的耗时列
WS 帧频率	F12 WS 面板消息计数
后端 CPU	任务管理器
前端内存	F12 Performance 面板
磁盘增长	csv/ 目录大小变化

8.39.2 7.39.2 一次基准测试示例

```
# 测列表接口延迟（5 次取平均）
$times = @()
1..5 | ForEach-Object {
    $sw = [System.Diagnostics.Stopwatch]::StartNew()
    Invoke-RestMethod -Uri "http://127.0.0.1:3000/api/fw100/items" -Headers
    @{ Authorization = "Bearer $token" }
    $sw.Stop()
    $times += $sw.ElapsedMilliseconds
}
"平均: $([math]::Round(($times | Measure-Object -Average).Average, 1)) ms"
```

8.39.3 7.39.3 优化验证方法

改前测一次基准 → 改后测同样基准 → 对比
只有“可量化的提升”才算优化成功

8.40 7.40 07 章终极验证：动手任务清单

8.40.1 任务 1：全新环境搭建（60 分钟）

1. 删除本地克隆，重新 clone
2. 完成环境准备与依赖安装
3. 启动后端 + fw100 前端
4. 登录验证数据链路
5. 记录遇到的所有问题与解决

8.40.2 任务 2：生产部署演练（60 分钟）

1. 修改 .env 的 JWT_SECRET
2. 执行 deploy.bat
3. 验证 7 个应用入口
4. 检查数据库/配置自动生成
5. 执行备份脚本
6. 模拟故障 → 重启恢复

8.40.3 任务 3：故障排查演练（30 分钟）

1. 故意改错 config-fj200c_information.ini（端口写错）
2. 启动服务观察失败
3. 用日志定位
4. 修复并验证

三个任务完成 → 07 章（使用与维护）正式毕业。

8.41 7.41 补充：环境变量与 dotenv 机制

8.41.1 7.41.1 dotenv 的加载顺序

1. 代码硬编码默认值（config.rs）
2. .env 文件（同目录，自动加载）
3. 系统环境变量（优先于 .env）

```
// src/config.rs 的模式
let port = std::env::var("PORT").unwrap_or_else(|_| "3000".into());
```

8.41.2 7.41.2 新增环境变量的步骤

1. config.rs 加读取逻辑
2. .env 加示例（或自动生成逻辑）
3. 重启后端生效

8.42 7.42 补充：config-*.ini 的完整结构参考

8.42.1 7.42.1 config-fj200c_information.ini 全节

```
[Mock]           ; 模拟开关
[Connection0..N] ; 串口连接（每路一段）
[CSV]            ; 记录开关与目录
[Data]           ; 其他数据参数（若有）
```

8.42.2 7.42.2 config-fj200c_main.ini 全节

```
[COM]           ; 串口总数与三路定义
[MOCK]          ; 模拟菜单
[REPORT]        ; 报表状态点
[CSV]           ; 64 列记录
[THEME]         ; 主题（可选）
```

8.42.3 7.42.3 config-ftj1c.ini 全节

```
[Udp]           ; 模拟开关
[IP]            ; 16 路组播地址
```

运维视角：读 ini 就是读设备接线表——哪路串口、哪个波特率、哪路组播一目了然。

8.43 7.43 补充：常见安装/环境问题（Windows）

问题	解决
cargo 命令不存在	装 rustup + 重启终端
link.exe 找不到	装 VS Build Tools (C++ 工作负载)
npm 报 EPERM	以管理员运行终端
端口被 Hyper-V 保留	netsh 排除或换端口
中文乱码（终端）	chcp 65001 或 PowerShell 新版本
cargo 下载慢	配国内镜像（2.1 节）

8.44 7.44 补充：07 章知识自测（追加 10 题）

1. dotenv 的加载顺序？
2. 新增环境变量的三步骤？
3. 三个 ini 各有哪些节？
4. cargo 找不到怎么办？
5. 终端中文乱码怎么处理？

6. 备份脚本包含哪些内容?
7. 健康检查用什么命令?
8. 端口被占用怎么排查?
9. deploy.bat 中途失败怎么处理?
10. 数据保留策略怎么定?

答对 8+ → 07 章掌握。

8.45 7.45 深入：数据库备份的三种方式对比

方式	命令	适用
文件拷贝	复制 rustweb.db	停服后最稳
SQLite .backup	sqlite3 .backup	热备 (WAL 安全)
导出 SQL	sqlite3 .dump	跨版本迁移

停服备份 (最稳妥)

```
Stop-Process -Name rust-web-backend -Force
```

```
Copy-Item rustweb.db "backup\rustweb-$(Get-Date -Format yyyyMMdd).db"
```

8.46 7.46 深入：WAL 模式与数据安全

8.46.1 7.46.1 WAL 是什么

WAL (Write-Ahead Logging): 写入先记日志, 再合并进主库

→ 崩溃恢复: 日志重放即可, 主库损坏概率低

→ 并发读写更好 (读写不互斥)

8.46.2 7.46.2 项目启用方式

```
-- database.rs 初始化时
```

```
PRAGMA journal_mode=WAL;
```

8.46.3 7.46.3 WAL 文件说明

```
rustweb.db-wal # 待合并的写入日志
```

```
rustweb.db-shm # 共享内存索引
```

注意: 备份时把三个文件一起复制, 或用 .backup 命令。

8.47 7.47 深入：日志排障的实战案例

8.47.1 7.47.1 常见日志片段解读

[ERROR] 连接 COM3 失败: 系统找不到指定的文件

→ 串口不存在/被占用 → 检查设备管理器 COM 号

[ERROR] 解析帧失败: unexpected EOF

→ 数据流中断 → 检查串口线/波特率

[WARN] 心跳超时, 切换备用源

→ 主源断流 → 检查上游设备

8.47.2 7.47.2 如何看日志

控制台直接看 (cargo run)

或部署后用 PowerShell 重定向

```
rust-web-backend.exe 2>&1 | Tee-Object -FilePath run.log
```

8.48 7.48 深入：性能与资源调优清单

项	现状	优化方向
首屏	全量 ECharts	按需引入
WS 推送	50ms 节流	按需订阅
表格	限长 200 行	虚拟滚动
SQL	索引缺失	常用字段加索引
并发	单进程	多实例（需改存储）

原则：先量化（测真实瓶颈）再优化，别盲目预优化。

8.49 7.49 深入：7 应用日常运维任务清单

8.49.1 7.49.1 每日

1. 看一眼服务是否在跑（健康检查）
2. CSV 目录是否在增长（数据在写）
3. 错误日志数量（快速扫描 ERROR）

8.49.2 7.49.2 每周

1. 备份数据库（脚本化）
2. 归档旧 CSV（按周目录）
3. 磁盘空间检查

8.49.3 7.49.3 每月

1. 数据库 VACUUM（压缩）
2. 版本更新演练（备份→替换→验证）
3. 权限复核（有没有不该有 admin 的账号）

8.50 7.50 深入：07 章补充自测（追加 10 题）

1. 三种备份方式的适用场景？
2. WAL 的崩溃恢复机制？
3. WAL 备份要注意什么？
4. 日志片段怎么解读？
5. 日志如何重定向保存？
6. 性能优化的原则？
7. 每日运维任务有哪些？
8. 每周运维任务有哪些？
9. VACUUM 的作用？
10. 权限复核多久一次？

答对 8+ → 07 章补充完成。

8.51 7.51 深入：部署前的完整自检清单

#	检查项	方法
1	前端构建通过	7 个 npm run build
2	后端编译通过	cargo build --release --features embedded
3	openapi 断言通过	cargo test export_openapi
4	数据库无脏数据	备份 + 快速浏览
5	配置正确	三份 ini 核对
6	端口不冲突	netstat 检查
7	.env 存在	不存在会自动生成
8	csv 目录可写	启动后写一行测试
9	防火墙放行	3000 端口入站
10	备份可用	恢复演练

8.52 7.52 深入：服务自启动的完整配置

8.52.1 7.52.1 场景

开机自动启动后端服务（无人值守）。

8.52.2 7.52.2 方法一：任务计划程序

注册任务（每次登录启动）

```
schtasks /Create /TN "rustweb" /TR "D:\deploy\rust-web-backend.exe" /SC ONLOGON /RL HIGHEST
```

8.52.3 7.52.3 方法二：启动文件夹

把 exe 快捷方式放到

```
shell:startup # 当前用户启动文件夹
```

8.52.4 7.52.4 方法三：Windows 服务（第三方工具 NSSM）

```
nssm install rustweb "D:\deploy\rust-web-backend.exe"
```

```
nssm start rustweb
```

推荐：任务计划程序（无需额外工具，支持开机自启）。

8.53 7.53 深入：日志与监控的长期方案

8.53.1 7.53.1 日志滚动

按天重命名：run-20260601.log、run-20260602.log

保留最近 30 天，旧日志自动清理（计划任务）

8.53.2 7.53.2 简易健康监控

每 5 分钟检查端口是否响应

```
if (-not (Test-NetConnection localhost -Port 3000 -InformationLevel Quiet)) {  
    Start-Process "D:\deploy\rust-web-backend.exe" # 挂了就拉起来  
}
```

8.53.3 7.53.3 升级路径

需要真正监控 → Prometheus + Grafana（后端加 /metrics 接口）

8.54 7.54 深入：07 章最终综合自测（追加 8 题）

1. 部署自检的 10 项？
2. 三种自启动方式？
3. 推荐哪种自启动？

- 4. 日志滚动怎么实现？
- 5. 简易健康监控脚本原理？
- 6. 升级监控方案选什么？
- 7. 备份恢复演练的意义？
- 8. csv 目录写测试怎么做？

答对 7+ → 07 章最终通过。

8.55 7.55 深入：7 个应用的真实使用场景演练

8.55.1 7.55.1 场景一：车间监控值班

- 1. 启动后端 → 7 个应用自动就绪
- 2. 打开 fj200c_information（发动机监控）
- 3. 检查服务状态 → 启动采集
- 4. 观察仪表盘实时数据
- 5. 下班前停止服务（数据已存 CSV）

8.55.2 7.55.2 场景二：管理员日常

- 1. 打开 admin → 检查用户列表
- 2. 新同事入职 → 创建账号 + 分配角色
- 3. 离职 → 禁用账号（不删）
- 4. 查看权限是否合理

8.55.3 7.55.3 场景三：设备运维

- 1. fw100 台账 → 录入新设备
- 2. 定期巡检 → 更新设备状态
- 3. 导出台账 → 汇报

8.55.4 7.55.4 场景四：通信排查

- 1. ftj1c 打开 → 看帧数据是否流动
- 2. 无数据 → 检查 UDP 配置/Mock
- 3. 配置修改 → 重启服务生效

8.56 7.56 深入：故障分级与响应

级别	症状	响应
P1 严重	全部应用无法访问	立即重启 + 查日志
P2 重要	单模块无数据	检查配置/串口
P3 一般	页面样式问题	记录，下次发版修复
P4 建议	体验优化	收集需求

原则：先恢复服务（重启），再定位根因。

8.57 7.57 深入：07 章实战自测（8 题）

- 1. 值班场景的操作顺序？
- 2. 账号管理最佳实践？
- 3. 通信排查的步骤？
- 4. 故障分级的响应？
- 5. 先恢复还是先定位？
- 6. 台账导出的用途？
- 7. 禁用账号 vs 删除账号？
- 8. Mock 模式的用途？

答对 7+ → 07 章实战通过。

8.58 7.58 深入：Windows 环境问题排查手册

8.58.1 7.58.1 常见问题速查

现象	原因	解决
双击 exe 闪退	缺依赖/端口被占	命令行运行看报错
端口 3000 被占	上次进程没退	taskkill /PID 或重启
中文乱码	控制台编码	chcp 65001
串口打不开	被其他程序占用	关闭占用程序
防火墙拦截	入站规则	放行 3000 端口
文件被锁定	日志/数据库占用	关闭后再操作

8.58.2 7.58.2 查看端口占用

```
netstat -ano | findstr 3000  
taskkill /PID <进程号> /F
```

8.58.3 7.58.3 控制台中文

```
chcp 65001 # 切换 UTF-8  
# 或 PowerShell 设置 $OutputEncoding
```

8.59 7.59 深入：数据安全与权限管理

8.59.1 7.59.1 账号安全

1. 默认密码 123456 → 首次登录立即改
2. 密码长度建议 ≥ 8 位
3. 定期更换（季度）
4. 离职立即禁用

8.59.2 7.59.2 数据安全

1. 数据库定期备份（脚本化）
2. CSV 目录同步备份
3. 备份异地存放（U 盘/网盘）
4. 恢复演练每季度一次

8.59.3 7.59.3 权限最小化

1. 只给需要的角色
2. 管理员账号仅 admin 使用
3. 定期复核角色分配

8.60 7.60 深入：07 章高频自测（8 题）

1. 双击闪退的排查方法？
2. 端口占用的解决？
3. 中文乱码的解决？
4. 串口被占的处理？
5. 密码管理的四条？
6. 数据安全四件套？
7. 权限最小化原则？
8. 恢复演练的周期？

答对 7+ → 07 章高频通过。

8.61 7.61 深入：三份 ini 的完整配置参考

8.61.1 7.61.1 config-fj200c_information.ini

```
[Mock]
InProcess = true           ; 模拟模式（无硬件时）

[Connection1]
ComPort = COM3            ; 串口号
BaudRate = 115200         ; 波特率

[CSV]
Record = true             ; 是否记录 CSV
Dir = csv                 ; 记录目录
```

8.61.2 7.61.2 config-fj200c_main.ini

```
[COM]
Count = 3                 ; 三路串口

[ECU]
ComPort = COM3
BaudRate = 115200

[ADAM]
ComPort = COM4

[DYN0]
ComPort = COM5

[MOCK]
SimulationMenu = true     ; 模拟运行

[REPORT]
StatePoints = 100,200,300 ; 报表状态点

[CSV]
Dir = csv
```

8.61.3 7.61.3 config-ftj1c.ini

```
[Udp]
Mock = true              ; 模拟数据

[IP]
Address1 = 239.0.0.1:5000
Address2 = 239.0.0.2:5001
; ... 16 路组播地址
```

8.61.4 7.61.4 修改生效规则

fj200c_information: 热加载（立即生效）
fj200c_main / ftj1c: 需重启服务
→ 修改后检查界面提示

8.62 7.62 深入：数据库操作的完整指南

8.62.1 7.62.1 查看数据

SQLite Viewer 插件 / DB Browser for SQLite
打开 rustweb.db → 浏览表

8.62.2 7.62.2 常用 SQL

```
-- 查看用户
SELECT * FROM users;
```



```
-- 修改用户状态
UPDATE users SET is_active = 1 WHERE id = 1;
```

```
-- 备份
.backup backup.db -- sqlite3 命令
```

8.62.3 7.62.3 危险操作清单

1. DELETE FROM 无 WHERE → 全删（先 SELECT 确认）
2. UPDATE 忘记 WHERE → 全改
3. 直接改表结构 → 程序可能崩（无迁移机制）
4. 改密码哈希 → 登录失败（只能改回）

原则：界面能做的操作尽量走界面，SQL 直改只用于救急。

8.63 7.63 深入：7 个应用的浏览器测试要点

8.63.1 7.63.1 登录测试

1. 正确账号 → 进入主页
2. 错误密码 → 提示
3. 空字段 → 前端校验拦截
4. 禁用账号 → 提示

8.63.2 7.63.2 权限测试

admin → 能进 admin，能管理用户
业务角色 → 进对应应用，无管理入口
（用不同账号验证按钮/菜单消失）

8.63.3 7.63.3 数据流测试

1. 启动服务 → 数据开始流动
2. 停止服务 → 数据停止
3. 刷新页面 → 快照恢复
4. 断网重连 → 自动恢复

8.64 7.64 深入：07 章终局自测（8 题）

1. 三份 ini 的节结构？
2. 模拟模式的开关？
3. 热加载与需重启的区别？
4. 查看数据库的工具？
5. 三种危险 SQL？
6. 登录测试的四点？
7. 权限测试怎么验证？
8. 数据流测试的四步？

答对 7+ → 07 章终局通过。

8.65 7.65 深入：日常运维的 PowerShell 脚本合集

8.65.1 7.65.1 一键启动脚本

```
# start.ps1: 启动后端（带日志）
Start-Process rust-web-backend.exe -RedirectStandardOutput run.log
Write-Host "服务已启动，日志：run.log"
```

8.65.2 7.65.2 健康检查脚本

```
# health.ps1: 每 5 分钟检查一次
while ($true) {
    $ok = Test-NetConnection localhost -Port 3000 -InformationLevel Quiet
```

```

if (-not $ok) {
    Write-Host "$(Get-Date) 服务挂了，重启中..."
    Start-Process rust-web-backend.exe
}
Start-Sleep -Seconds 300
}

```

8.65.3 7.65.3 自动备份脚本

```

# backup.ps1: 每天备份数据库
$dir = "backup\$(Get-Date -Format yyyyMMdd)"
New-Item -ItemType Directory -Force -Path $dir | Out-Null
Copy-Item rustweb.db "$dir\rustweb.db"
Copy-Item "csv\*" "$dir\csv\" -ErrorAction SilentlyContinue
# 保留最近 7 天，删除更早
Get-ChildItem backup -Directory |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-7) } |
    Remove-Item -Recurse -Force

```

8.65.4 7.65.4 日志清理脚本

```

# clean-logs.ps1: 清理 30 天前日志
Get-ChildItem *.log | Where-Object {
    $_.LastWriteTime -lt (Get-Date).AddDays(-30)
} | Remove-Item -Force

```

8.66 7.66 深入：性能调优的完整路线

8.66.1 7.66.1 前端性能

1. 首屏：路由分包（已有）→ 按需引入组件
2. 高频更新：节流（已有 50ms）→ 按需订阅
3. 大表格：虚拟滚动（数据 >1000 行）
4. 图片/资源：压缩

8.66.2 7.66.2 后端性能

1. SQL：索引/EXPLAIN 分析
2. 连接池：大小调整
3. 广播：频道按模块隔离
4. 序列化：减少不必要字段

8.66.3 7.66.3 性能测试方法

```

# 压测接口（简单）
for ($i = 0; $i -lt 100; $i++) {
    $t = Measure-Command { Invoke-RestMethod http://localhost:3000/api/fw100/items }
    $t.TotalMilliseconds
} | Measure-Object -Average

```

8.67 7.67 深入：07 章毕业自测（8 题）

1. 一键启动脚本的写法？
2. 健康检查的循环逻辑？
3. 自动备份的保留策略？
4. 日志清理的阈值？
5. 前端性能的四点？
6. 后端性能的三点？
7. 压测的方法？
8. 备份目录的命名规范？

答对 7+ → 07 章毕业。

8.68 7.68 深入：常见故障的完整排查手册（最全版）

8.68.1 7.68.1 后端起不来

- # 1. 看错误输出
rust-web-backend.exe
- # 2. 常见原因
 - # - 端口被占（netstat 查）
 - # - .env 配置错误
 - # - 数据库文件损坏
 - # - 缺依赖 DLL

8.68.2 7.68.2 前端白屏

1. 看 Console（JS 报错）
2. 看 Network（资源 404 → base 配置）
3. 检查 /api 代理（开发）或路由（生产）
4. 清缓存强刷（Ctrl+F5）

8.68.3 7.68.3 登录失败

1. 邮箱密码是否正确（种子：admin@example.com / 123456）
2. 账号是否禁用（数据库 is_active）
3. 后端日志（密码错误 vs 未找到）
4. token 过期 → 重新登录

8.68.4 7.68.4 无数据/数据卡住

1. 服务是否启动（前端状态栏）
2. 模拟模式是否开启（ini [Mock]）
3. 串口是否可用（设备管理器）
4. WS 是否连接（DevTools 网络）
5. 后端日志（帧错误/节流）

8.68.5 7.68.5 数据不对

1. 字段含义（协议文档）
2. 缩放系数（解码配置）
3. 字节序（大端/小端）
4. 时间戳单位（秒/毫秒）

8.69 7.69 深入：多用户并发使用的注意事项

8.69.1 7.69.1 并发读

- SQLite WAL：读写并发 OK（默认）
→ 多用户同时浏览台账正常

8.69.2 7.69.2 并发写

- 写操作串行（SQLite 单写锁）
→ 短暂等待可接受（内网规模）
→ 大量写 → 考虑队列/批处理

8.69.3 7.69.3 监控数据的多人查看

- WS 广播：N 客户端共享同一份推送
→ 新增客户端不影响性能
→ 客户端断开自动清理

8.70 7.70 深入：07 章权威自测（8 题）

1. 后端起不来的四类原因？
2. 白屏的四个排查？
3. 登录失败的四个原因？
4. 无数据的五个排查？

5. 数据不对的四个原因?
6. 并发写的限制?
7. 广播的并发特性?
8. 时间戳单位在哪查?

答对 7+ → 07 章权威。

8.71 7.71 深入：升级与迁移指南

8.71.1 7.71.1 版本升级的完整流程

1. 备份（数据库 + 配置 + 旧 exe）
2. 更新代码（git pull）
3. 构建（前端 + 后端）
4. 对比配置差异（ini/.env）
5. 部署替换（保留数据文件）
6. 验证（冒烟）
7. 回滚预案（保留旧版本）

8.71.2 7.71.2 数据库迁移的注意事项

1. 本项目无自动迁移机制（database.rs 内建表）
2. 表结构变化 → 需要手动迁移脚本
3. 老数据兼容 → 先备份再改
4. 建议：小步升级（每次只加字段）

8.71.3 7.71.3 配置升级

1. 新版本新增配置项 → ini 补默认值
2. 旧配置不兼容 → 提供迁移说明
3. 热加载配置 → 重启后生效

8.72 7.72 深入：文档与知识沉淀

8.72.1 7.72.1 为什么要有运行文档

1. 换人交接
2. 故障复盘
3. 新人上手
4. 需求梳理

8.72.2 7.72.2 沉淀什么

1. 变更记录（改了什么/为什么）
2. 故障记录（现象/根因/修复）
3. 操作手册（日常操作步骤）
4. 架构说明（模块/接口/数据）

8.72.3 7.72.3 怎么沉淀

1. 修改代码时同步改 AGENTS.md/文档
2. 故障解决后记录在案
3. 定期回顾整理

8.73 7.73 深入：07 章权威自测（8 题）

1. 升级流程的七步?
2. 数据库迁移的注意?
3. 配置升级的处理?
4. 运行文档的四个价值?
5. 沉淀的四类内容?
6. 迁移脚本的写法?
7. 小步升级的好处?

8. 故障记录包含什么？

答对 7+ → 07 章权威。

8.74 7.74 深入：跨年/长期运行的数据管理

8.74.1 7.74.1 长期运行的问题

1. CSV 文件无限增长 → 磁盘满
2. 数据库膨胀 → 查询变慢
3. 日志文件增长 → 磁盘满
4. 配置漂移 → 与文档不一致

8.74.2 7.74.2 数据保留策略

CSV: 保留 90 天 → 归档/删除
数据库: 保留全部 (小规模) → 定期 VACUUM
日志: 保留 30 天 → 滚动清理
备份: 保留 7 份 (最近一周) → 周备份留 12 份

8.74.3 7.74.3 归档与恢复

归档: CSV 按月打包 (zip) → 移出运行目录
恢复: 解压回 csv/ 即可 (格式不变)

8.75 7.75 深入：性能退化的排查 (长期运行后)

8.75.1 7.75.1 症状与原因

1. 页面变慢 → 数据库膨胀/无索引
2. WS 卡顿 → 广播压力/客户端过多
3. 磁盘告警 → CSV/日志未清理
4. 启动变慢 → 表数据大 (启动扫描)

8.75.2 7.75.2 排查顺序

1. 先看磁盘 (df / dir 大小)
2. 再看数据库 (EXPLAIN 慢查询)
3. 后看网络 (WS 流量)
4. 最后看配置 (是否有人改了)

8.75.3 7.75.3 常规优化

1. 数据库 VACUUM
2. 加索引 (查询频繁列)
3. 归档 CSV
4. 重启服务 (回收资源)

8.76 7.76 深入：07 章权威自测 (8 题)

1. 长期运行的四个问题？
2. 四类数据的保留策略？
3. 归档与恢复的方法？
4. 性能退化的四个症状？
5. 排查顺序？
6. VACUUM 的作用？
7. 备份保留几份？
8. 启动变慢的原因？

答对 7+ → 07 章权威。

8.77 7.77 深入：本章收尾——运维思维总结

8.77.1 7.77.1 运维的核心原则

1. 备份先行（改任何东西之前）
2. 最小操作（一次只动一处）
3. 可回滚（保留旧版本）
4. 可观察（日志/指标）
5. 文档同步（改了什么记录什么）

8.77.2 7.77.2 日常运维的闭环

观察（状态/日志）→ 判断（是否正常）→ 处理（备份→修改→验证）→ 记录

8.77.3 7.77.3 学完本章的标准

1. 能独立部署 + 配置 + 启动
2. 能备份/恢复/升级/回滚
3. 能定位常见故障
4. 能看懂日志
5. 能写基础运维脚本

8.78 7.78 深入：07 章最终自测（6 题）

1. 运维的五个核心原则？
2. 日常运维的闭环？
3. 学完本章的五个标准？
4. 为什么备份先行？
5. 可回滚的意义？
6. 观察的方法有哪些？

答对 5+ → 07 章最终完成。

下一节：08-扩展与二次开发。

9 08 扩展与二次开发

终章：把系统“长出新的能力”。本章是 AGENTS.md「新增角色流程」的展开教学，从新增角色/模块/应用，到协议接入、报表扩展、安全加固，一次讲透。

9.1 8.1 二次开发全景图

flowchart TD

```
A[二次开发需求] --> B{改现有?}
B -->|改现有模块| C[加字段/加接口/改逻辑]
B -->|新增模块| D[完整新增角色流程]
B -->|集成新硬件| E[协议接入]
B -->|扩展界面| F[前端组件/主题/页面]
B -->|扩展安全| G[权限/认证加固]
B -->|扩展数据| H[报表/导出/CSV]
```

四个扩展维度：功能（新模块）、设备（新协议）、界面（前端）、安全/数据（横切）。

9.2 8.2 新增角色完整流程（核心方法论）

9.2.1 8.2.1 流程总览（7 步）

flowchart TD

```
S1[1. 后端：Permission 枚举 + 角色注册表] --> S2[2. 复制 role_template 为新模块]
```

```

S2 --> S3[3. handler 注解 + api_docs 登记]
S3 --> S4[4. 前端 roles.ts 菜单/地址]
S4 --> S5[5. 复制前端为新应用]
S5 --> S6[6. gen:api 同步 + build 验证]
S6 --> S7[7. deploy.bat/main.rs/embedded 同步]

```

9.2.2 8.2.2 第 1 步：后端权限与注册表

```

// src/common/models.rs — 加权限
#[derive(Clone, Copy, Debug, PartialEq, Eq, Serialize, Deserialize, ToSchema)]
pub enum Permission {
    SystemAdmin,
    UsersRead, UsersWrite, UsersDelete,
    Fj200cInformationMonitor,
    Fj200cMainMonitor,
    Fw100Monitor, Fw150Monitor,
    Ftjl1cMonitor,
    City3dView,
    XxxMonitor,           // ← 新增
}

// src/roles.rs — 注册角色
pub const ROLE_REGISTRY: &[RoleDef] = &[
    RoleDef { key: "admin", name: "系统管理员", permissions: &[Permission::SystemAdmin,
Permission::UsersRead, ...] },
    // ... 现有角色
    RoleDef { key: "xxx", name: "新模块", permissions: &[Permission::XxxMonitor] }, // ← 新增
];

```

前端自动获得： /api/meta/roles 返回新角色 → admin 界面下拉可选 → 登录权限自动生效。

9.2.3 8.2.3 第 2 步：复制 role_template

```

# src/role_template/ 自带说明文档，模板包含：
# mod.rs / routes.rs / handlers.rs / services.rs / models.rs / config.rs
Copy-Item src\role_template src\xxx -Recurse
# 全局替换 template → xxx

```

role_template 的设计： 占位权限 TemplateMonitor + 示例 CRUD + #![allow(dead_code)] (未启用时不警告)。照着填就是完整模块。

9.2.4 8.2.4 第 3 步：注解与登记

```

// src/xxx/handlers.rs
#[utoipa::path(
    get,
    path = "/api/xxx/items",
    tag = "xxx",
    operation_id = "xxxItemsList",
    responses((status = 200, description = "成功", body = Vec<XxxItem>))
)]
pub async fn list_items(...) -> ... { ... }

// src/api_docs.rs
paths(xxx_list_items, ...),
components(schemas(XxxItem, ...)),
tags((name = "xxx", description = "...")),

```

9.2.5 8.2.5 第 4 步：前端 roles.ts

```

// packages/shared/src/roles.ts
export const MENU_CONFIG: MenuItem[] = [
    // ...
    { key: 'xxx', label: '新模块', icon: 'Monitor', appPath: '/xxx' },

```

```
]
export const ROLE_APP_URLS = { ..., xxx: '/xxx' }
```

9.2.6 8.2.6 第 5 步：复制前端

```
# 复制最相似的应用（如 fw100 做 CRUD、fj200c_information 做监控）
Copy-Item frontend\fw100 frontend\xxx -Recurse
# 全局替换 fw100 → xxx
```

文件	改动
vite.config.ts	端口（517x 未占用）、base /xxx/
package.json	name
index.html	title
api/index.ts	facade 方法
router/index.ts	路由 + meta.permissions
stores/auth.ts	createAuthStore 参数
views/*	业务页面

9.2.7 8.2.7 第 6~7 步：同步与部署

```
npm run gen:api      # 生成 generated/api/xxx.ts
npm run build        # 各前端验证
# deploy.bat 加 xxx 构建步骤
# main.rs 静态托管（dev 模式）
# embedded_assets.rs 加嵌入结构体 + 路由
```

9.2.8 8.2.8 新增角色流程要点总结

步骤	容易漏的点
1	权限枚举与注册表同步
3	api_docs.rs 登记（防漂移测试会抓）
4	roles.ts 与后端注册表 key 一致
5	端口/base/workspaces 三处同步
7	embedded_assets + main.rs + deploy.bat 三处同步

9.3 8.3 后端新模块开发模板（role_template 详解）

9.3.1 8.3.1 模板文件结构

```
src/role_template/
├─ mod.rs      # 模块入口 + 事件枚举 + Tx
├─ routes.rs   # 路由注册
├─ handlers.rs # handler（含 utoipa 注解）
├─ services.rs # 业务逻辑
├─ models.rs   # DTO
└─ config.rs   # 配置
```

9.3.2 8.3.2 模块入口（mod.rs 模式）

```
// src/xxx/mod.rs
pub mod routes;
pub mod handlers;
pub mod services;
pub mod models;
pub mod config;
```



```
// 事件枚举（WS 广播用）——监控类模块需要
#[derive(Clone, Debug)]
pub enum XxxEvent {
    DataUpdated(XxxData),
    StatusChanged(bool),
}

// 全局事件发送通道（OneShot 类型）
pub static TX: std::sync::OnceLock<mpsc::UnboundedSender<XxxEvent>> =
std::sync::OnceLock::new();
```

9.3.3 8.3.3 路由注册 (routes.rs)

```
// src/xxx/routes.rs
use axum::routing::{get, post};
use crate::common::middleware::permission_middleware;

pub fn router() -> Router<AppState> {
    Router::new()
        .route("/api/xxx/items", get(list_items).post(create_item))
        .route("/api/xxx/items/{id}", get(get_item).put(update_item).delete(delete_item))
        .layer(middleware::from_fn(permission_middleware::<Permission::XxxMonitor>))
}

```

9.3.4 8.3.4 handler 模板

```
pub async fn list_items(
    State(state): State<AppState>,
) -> Result<Json<ApiResponse<Vec<XxxItem>>>, AppError> {
    let items = services::list_items(&state.db).await?;
    Ok(Json(ApiResponse::success(items)))
}

```

9.3.5 8.3.5 service 模板

```
pub async fn list_items(db: &SqlitePool) -> Result<Vec<XxxItem>, AppError> {
    sqlx::query_as::<_, XxxItem>("SELECT * FROM xxx_items")
        .fetch_all(db)
        .await
        .map_err(|e| AppError::internal(format!("查询失败: {e}")))
}

```

9.3.6 8.3.6 建表位置 (database.rs)

```
// src/database.rs — 加建表语句（新表在启动时自动创建）
sqlx::query(
    "CREATE TABLE IF NOT EXISTS xxx_items (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        created_at TEXT NOT NULL DEFAULT (datetime('now'))
    )"
).execute(&pool).await?;
```

项目约定：没有迁移文件，建表全在 database.rs——新表加这里即可（注意幂等：CREATE IF NOT EXISTS）。

9.4 8.4 修改现有模块（最高频操作）

9.4.1 8.4.1 加字段（全链路清单）

后端：

1. models.rs 结构体加字段（ToSchema）
2. database.rs 建表语句加列（若新列）
3. service 的 SQL 加列
4. handler 注解无需改（schema 自动含新字段）

前端:

5. `npm run gen:api`
6. 表单/表格/详情按报错补字段
7. `npm run build` 验证

9.4.2 8.4.2 加接口

1. handler 写新函数 + `utoipa` 注解
2. `api_docs.rs` paths 登记
3. `routes.rs` 注册路由
4. `gen:api` → `facade` 加方法 → 页面调用

9.4.3 8.4.3 改逻辑 (如改 CSV 列)

1. 找到对应后端逻辑 (CSV 写入在 `service/session`)
2. 修改列定义与写入
3. 前端 Data 页若展示列 → 同步
4. 注意旧 CSV 文件格式兼容 (或接受格式变化)

9.4.4 8.4.4 改前端行为

1. 视图层: 直接改模板/script
2. 业务逻辑: `composable`
3. 公共: `shared` (注意影响全部应用)

9.5 8.5 新硬件协议接入 (串口/UDP)

9.5.1 8.5.1 协议接入的通用模式

flowchart LR

```
A[硬件数据] --> B[IoControl 实现<br/>SerialControl/MockControl/UdpControl]
B --> C[帧提取 frame_extractor]
C --> D[解码 decode.rs]
D --> E[CSV/状态]
E --> F[WS 广播]
```

四个可替换点: 1. **数据源** (`IoControl trait`): 串口/模拟/UDP——换硬件只换实现。 2. **帧提取** (`frame_extractor`): 从字节流切帧。 3. **解码** (`decode.rs`): 帧 → 结构化字段。 4. **广播**: 事件 → WS。

9.5.2 8.5.2 接入新协议的步骤

1. 定义帧格式 (长度/校验/字段布局)
2. 写提取器 (若帧结构不同)
3. 写解码器 (字段 → `struct`)
4. 用 `IoControl trait` 实现数据源
5. 接入 `service` 的会话循环
6. 前端 `types.ts` 加字段 + 展示

9.5.3 8.5.3 模拟源的价值

开发期没有硬件 → 用 `Mock` 实现 (模拟帧生成器)

→ 前后端联调不依赖硬件

→ 现场部署前先用模拟验证全链路

这是 `fj200c_information/fj200c_main/ftj1c` 三个监控应用的标准做法 (`Mock` 开关在各自 `ini`)。

9.6 8.6 前端组件与主题扩展

9.6.1 8.6.1 新增共享组件

- 1. packages/shared/src/template/ 建组件
- 2. index.ts 导出
- 3. 各应用直接 import 使用

9.6.2 8.6.2 新增主题 (fj200c_main 模式)

- 1. styles/themes.css 加 .theme-xxx 变量组
- 2. theme 类型加 'xxx'
- 3. 后端 set_theme 校验类型 (GlobalVar)
- 4. 切换按钮/设置页加选项

9.6.3 8.6.3 新增页面/路由

- 1. views/ 建页面
- 2. router 加路由 + meta.permissions
- 3. MENU_CONFIG 加入口 (可选)
- 4. build 验证

9.7 8.7 报表与导出扩展

9.7.1 8.7.1 现有报表机制

后端生成 HTML ([REPORT] StatePoints 状态点)
→ 前端新窗口打开 → window.print 打印

9.7.2 8.7.2 扩展报表形态

形态	做法
新报表模板	后端 report.rs 加生成函数 (HTML 拼装)
Excel 导出	后端生成 xlsx (需加库) 或 CSV (现成)
PDF	后端打印 HTML → PDF (需加库) 或前端打印另存
图表报表	前端 ECharts 截图 + 组装 (复杂, 慎做)

9.7.3 8.7.3 CSV 导出的通用化

fw100 等台账应用想要 CSV 导出:
→ 前端把 items 拼 CSV + Blob 下载 (10 行代码, 无需后端)
→ 或后端加导出接口 (大数据量时)

9.8 8.8 安全扩展

9.8.1 8.8.1 认证增强

增强	实现思路
登录失败锁定	auth service 记录失败次数
验证码	前端生成 + 后端校验
token 轮换	刷新 token 机制
密码策略	前端规则 + 后端校验 (min 长度等)

9.8.2 8.8.2 权限模型增强

增强	实现思路
资源级权限	权限枚举细分 (如 <code>fw100.items.edit</code>)
角色继承	RoleDef 加 parent 引用
动态角色	用户自定义角色表 (DB)

9.8.3 8.8.3 数据安全

1. 敏感字段加密存储 (`sqlx + aes`)
2. 日志脱敏 (不记录密码/token)
3. 备份加密 (压缩带密码)

9.9 8.9 常见扩展问答 (FAQ)

Q1: 怎么加“数据导出 Excel”按钮? A: 小数据 → 前端 CSV/Blob; 大数据 → 后端接口流式输出; 真 Excel 需加库 (`rust_xlsxwriter / xlsx`)。

Q2: 能加“用户自定义仪表盘”吗? A: 可以——前端存布局配置 (`localStorage` 或后端 `global_vars`)，后端只加一个“用户布局”字段。

Q3: 多语言 (i18n) 怎么做? A: 项目无 i18n。方案: `vue-i18n` + 文案抽取 + 语言切换 store; 工作量中等。

Q4: 历史数据回放/趋势分析? A: CSV 已有历史——解析 CSV 渲染曲线 (前端读文件或后端接口分页返回)。

Q5: 告警通知 (邮件/钉钉)? A: 后端事件监听 → 发通知 (`request` 调 `webhook`/邮件库)。在 `event` 广播处加一个监听器即可。

Q6: 移动端适配? A: 现有响应式支持基础适配; 深度适配需改布局 (弹性 + 触控优化)。

Q7: 多后端实例/负载均衡? A: SQLite 单文件不适合多实例写并发——需要换 PostgreSQL + 实例间 WS 互斥 (架构级改动, 不推荐)。

Q8: docker 化? A: 可行——Rust 编译产物 + 前端内嵌已是单二进制, Dockerfile 只需基础镜像 + 复制 `exe` + 挂载数据目录。

9.10 8.10 扩展开发最佳实践

9.10.1 8.10.1 扩展项目 Checklist

- [] 先读相关模块现有代码 (模仿最小实现)
- [] 后端: `Permission` → 注册表 → 模块 → 路由 → 注解 → 登记
- [] 前端: `facade` → 页面 → 路由 → 菜单
- [] `gen:api + build` 全链路验证
- [] `deploy.bat / embedded` 同步 (影响部署才需要)
- [] 更新 `AGENTS.md` (如新增角色/应用)

9.10.2 8.10.2 渐进式开发策略

1. 先跑通最小闭环 (模拟源 → 后端 → 前端显示)
2. 再加细节 (校验/错误处理/边界)
3. 最后加打磨 (样式/提示/日志)

9.10.3 8.10.3 向后兼容原则

- 加字段用 `Option` (旧数据不破坏)
- 新接口不删旧接口 (过渡期并存)

- CSV 格式变更前评估存量文件
- 配置新增节默认值（configparser 读缺省）

9.11 8.11 扩展中的常见决策问题（架构判断力）

9.11.1 8.11.1 该新建模块还是改现有模块？

flowchart TD

```

Q[新需求] --> A{与现有模块同领域吗?}
A -->|是| B[改现有模块加接口/字段]
A -->|否| C{独立数据与权限?}
C -->|是| D[新建模块role_template]
C -->|否| E[挂在现有模块下子资源路由]
  
```

判断标准：数据表独立 + 权限独立 + 页面独立 → 新模块；否则扩展现有。

9.11.2 8.11.2 数据存哪（DB vs 文件 vs 内存）

数据类型	存储	理由
业务台账	SQLite 表	结构化查询
实时帧流	内存（广播）	无需持久化
采样历史	CSV 文件	文件型数据流
主题/设置	global_vars 表	键值
大文件（报表）	文件系统	不塞数据库

9.11.3 8.11.3 配置放 ini 还是数据库

ini（config-*.ini）：硬件/协议/运行参数（文本可读、随部署携带）

DB（global_vars）：业务设置（主题等）

前端配置：localStorage（非敏感、跟随浏览器）

原则：运维要改的放 ini，业务要改的放 DB。

9.11.4 8.11.4 新接口怎么设计（REST 惯例）

```

GET    /api/<module>/<resource>      # 列表
POST   /api/<module>/<resource>      # 创建
GET    /api/<module>/<resource>/{id} # 详情
PUT    /api/<module>/<resource>/{id} # 更新
DELETE /api/<module>/<resource>/{id} # 删除
GET    /api/<module>/<resource>/stats # 统计（约定）
  
```

命名惯例：operation_id = 模块名 + 动作（驼峰），tag = 模块名——保持与现有模块一致。

9.12 8.12 扩展中的常见坑（血泪清单）

坑	后果	预防
只改后端忘了 gen:api	前端类型错乱	改 DTO 必跑
新表忘了 database.rs 建表	接口 500	建表统一入口
权限枚举改了注册表没改	角色权限空	两处同步
路由没挂 routes.rs	404	挂完测一遍
embedded 没加新应用	生产 404	deploy 前全应用验证
端口撞了	dev 起不来	端口表先查
workspace 没加新前端	依赖不装	npm install 在根目录跑
改 shared 影响 7 应用	回归风险	改动前确认影响面
WS 事件结构变了 types.ts 没同步	前端解不了包	同步手改
旧数据不兼容新字段	页面报错	可空字段 + 兜底显示

十条坑覆盖了二次开发 90% 的返工原因——开工前默念一遍。

9.13 8.13 扩展之后的验证清单（回归测试）

9.13.1 功能回归

- [] 7 个应用都能登录
- [] 监控类：启动服务 → 数据流动 → 停止服务
- [] 管理类：增删改查正常
- [] 权限：错误角色被拒（403）
- [] 配置：修改后按生效时机生效

9.13.2 契约回归

- [] npm run gen:api 无报错
- [] 全前端 npm run build 通过
- [] git diff generated/ 只有预期变化

9.13.3 部署回归

- [] deploy.bat 全流程成功
- [] 生产访问 7 个入口
- [] 数据库自动生成/保留
- [] 深链接刷新正常（SPA 回退）

每次扩展后跑一遍回归——本项目无自动化测试，回归靠清单。

9.14 8.14 深入：扩展前的设计问题清单

动手前先回答这 10 个问题，能省一半返工：

#	问题	答案来源
1	数据存哪? (SQLite/CSV/内存)	03 章 CSV vs SQLite 对比
2	谁访问? (前端直接调/经后端)	05 章数据流
3	权限点要不要加?	roles.rs 注册表
4	实时还是轮询?	WS vs HTTP 选择
5	需要热更新配置吗?	config.rs 模式
6	前端放哪个应用?	7 应用职责表
7	契约怎么加?	06 章全链路
8	部署有影响吗?	deploy.bat 流程
9	数据量增长怎么办?	CSV 限长策略
10	需要测试吗?	cargo test 惯例

核心原则：先画数据流图，再写代码。

9.15 8.15 深入：扩展的最佳实践十条

1. **模仿优先**：新代码尽量复制现有模块结构 (role_template 的存在意义)。
2. **小步提交**：后端、gen:api、前端分步验证，别一把梭。
3. **契约先行**：先定 DTO/接口，再写前后端 (避免返工)。
4. **错误处理统一**：一律返回 AppError，别自定义格式。
5. **日志完整**：每个新接口至少一条 info 日志。
6. **配置可调**：新参数进 ini 而不是硬编码。
7. **前端复用**：UI 组件先查 shared 有没有。
8. **不破坏契约**：加字段用 Optional，别删字段。
9. **性能留退路**：实时数据先节流，别让前端刷屏。
10. **文档同步**：改完更新本套文档与 AGENTS.md。

9.16 8.16 深入：常见坑的完整清单 (扩展时对照)

坑	现象	预防
端口冲突	新应用起不来	strictPort + 查表
路由忘挂	404	routes.rs 检查
权限忘加	403	permission_middleware
契约没同步	TS 报错	gen:api 必跑
base 错	白屏	vite.config 核对
WS 不转发	连不上	proxy ws:true
依赖重复安装	pinia 双实例	只在根 npm install
CSV 路径	找不到文件	相对运行目录
时间格式	前端显示 NaN	serde 时间序列化检查
中文乱码	控制台/Excel	UTF-8 统一

9.17 8.17 深入：回归验证清单 (扩展后必过)

1. cargo check 通过
2. cargo test 通过 (含 openapi 导出)
3. npm run gen:api 无 diff 或不意外
4. 受影响前端 npm run build 通过
5. 手动验证：

- 登录 → 权限正常
- 新增模块增删改查正常
- WS 实时推送正常
- 配置热更新（如涉及）正常

6. 部署演练: `deploy.bat` 完整跑一遍

达标 → 扩展才可提交。

9.18 8.18 深入：给新模块的代码结构建议

```
src/xxx/
├─ mod.rs           # 模块入口 + 全局状态
├─ handlers.rs      # 薄 handler
├─ services.rs      # 业务逻辑
├─ models.rs        # DTO (ToSchema)
├─ config.rs        # ini 配置
├─ com.rs           # 串口/通信 (如有)
└─ mock.rs          # 模拟数据源 (如有)
```

与 `role_template` 一致——复制模板改名字即可。

9.19 8.19 深入：与团队协作的文档化约定

1. 新增接口 → 在 00 章 API 一览表补一行。
2. 新增角色 → AGENTS.md 角色表同步。
3. 新增应用 → AGENTS.md 端口表 + `deploy.bat`。
4. 契约变更 → 06 章流程执行 + 提交生成文件。
5. 代码路径引用 → 标注 文件:行号。

文档是活文档：改代码必须改文档（本套约定）。

9.20 8.20 扩展实战：完整案例一（新增“报警管理”模块）

把 8.2 的 7 步流程完整走一遍，读者可照抄。

9.20.1 需求

系统增加“报警管理”模块：设备报警记录 + 确认处理 + 统计。新角色 `alarm`。

9.20.2 步骤 1：权限与角色

```
// src/common/models.rs
pub enum Permission {
    // ...
    AlarmMonitor,    // 新增：报警监控权限
}

// src/roles.rs
RoleDef { key: "alarm", name: "报警管理", permissions: &[Permission::AlarmMonitor] },
// admin 角色也加上 AlarmMonitor (可选)
```

9.20.3 步骤 2：新模块目录

```
Copy-Item src\role_template src\alarm -Recurse
# 替换 template → alarm
```

9.20.4 步骤 3：models.rs

```
// src/alarm/models.rs
#[derive(ToSchema, FromRow, Serialize, Deserialize)]
pub struct AlarmItem {
    pub id: i64,
```



```

pub source: String,          // 来源 (fj200c_information 等)
pub level: String,          // info / warning / error
pub message: String,
pub confirmed: bool,        // 是否已确认
pub occurred_at: String,
}

#[derive(ToSchema, Deserialize)]
pub struct CreateAlarmRequest {
    pub source: String,
    pub level: String,
    pub message: String,
}

```

9.20.5 步骤 4: database.rs 建表

```

sqlx::query(
    "CREATE TABLE IF NOT EXISTS alarm_items (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        source TEXT NOT NULL,
        level TEXT NOT NULL,
        message TEXT NOT NULL,
        confirmed INTEGER NOT NULL DEFAULT 0,
        occurred_at TEXT NOT NULL DEFAULT (datetime('now'))
    )"
).execute(&pool).await?;

```

9.20.6 步骤 5: handlers + routes + 登记

```

// handlers.rs: list / create / confirm 三个接口 + utoipa 注解
// routes.rs: /api/alarm/items 挂载 + permission_middleware::<Permission::AlarmMonitor>
// api_docs.rs: paths(alarm_list, alarm_create, alarm_confirm) + schemas(AlarmItem,
CreateAlarmRequest) + tag

```

9.20.7 步骤 6: 前端

```

Copy-Item frontend\fw100 frontend\alarm -Recurse
# 替换 fw100 → alarm, 端口 5180, base /alarm/

// api facade + 页面 (表格 + 确认按钮 + 统计卡片)

```

9.20.8 步骤 7: gen:api + build + 部署同步

```

npm run gen:api
npm run build
# deploy.bat APP_LIST 加 alarm
# main.rs 静态托管加 /alarm
# embedded_assets.rs 加结构体

```

完整闭环完成——从权限到部署，这就是 8.2 方法论的一次真实执行。

9.21 8.21 扩展实战：完整案例二（接入新串口协议）

9.21.1 需求

fj200c_information 增加一路“温度巡检仪”串口（COM7，Modbus-RTU 协议）。

9.21.2 方案选型

```

flowchart LR
    A[方案 A: 加进现有会话] --> A1[复用 IoControl/帧提取<br/>改动小但耦合]
    B[方案 B: 独立小模块] --> B1[仿 role_template + 串口<br/>隔离好但代码多]

```

推荐方案 A（复用现有框架）：数据源加一个 Connection，帧格式走 frame_extractor，解码器加 Modbus 分支。

9.21.3 实施步骤

1. config-fj200c_information.ini 加 [Connection7] 段 (Port=COM7, BaudRate=9600...)
2. com.rs 检查该连接是否存在 (配置驱动)
3. decode.rs 根据帧类型识别 Modbus 帧 → 解出温度字段
4. 前端 types.ts 加 temperature 字段 → 表格/曲线加列
5. 模拟模式同步支持 (mock 生成 Modbus 帧) — 开发期无硬件也能联调

核心心得：协议接入的关键不是“读串口” (框架已有)，而是**帧格式解析** (新协议的解码逻辑)。

9.22 8.22 扩展实战：完整案例三 (给台账加 Excel 导出)

9.22.1 需求

fw100 列表页加“导出 Excel”。

9.22.2 方案对比

方案	优点	缺点
前端 CSV	零后端改动	中文 Excel 需处理编码
后端 CSV	大列表也不卡	加接口
后端 xlsx 库	真 Excel	引入 rust_xlsxwriter 依赖

9.22.3 推荐：后端 CSV + BOM (通用做法)

```
// 后端：导出接口 (与现有 CSV 下载同模式)
#[utoipa::path(
    get,
    path = "/api/fw100/export",
    tag = "fw100",
    operation_id = "fw100Export",
    responses((status = 200, description = "成功", body = String))
)]
pub async fn export_items(...) -> Result<Response, AppError> {
    // 查全部 → 拼 CSV 字符串 (加 UTF-8 BOM 保证 Excel 中文正常)
    // 返回 text/csv; charset=utf-8 响应头 + Content-Disposition: attachment
}

// 前端 (复用 5.41 的 Blob 下载模式)
export const fw100Api = {
    // ...
    exportItems: async () => {
        const res = await generated.fw100Export()
        downloadBlob(res as Blob, '台账导出.csv')
    },
}
```

扩展思路：任何“导出”需求都能用这个模板 (CSV 先行，Excel 库按需升级)。

9.23 8.23 扩展实战：完整案例四 (告警通知到钉钉)

9.23.1 需求

报警发生时推送钉钉群消息。

9.23.2 实现思路 (零侵入)

```
// 在报警创建 service 里加一个“旁路通知” (不阻塞主流程)
// src/alarm/services.rs
pub async fn create_alarm(db: &SqlitePool, req: CreateAlarmRequest) -> Result<AlarmItem,
```

```

AppError> {
  let alarm = sqlx::query_as(...).bind(...).fetch_one(db).await?;

  // 旁路: 异步发钉钉 (失败不影响主流程)
  if alarm.level == "error" {
    tokio::spawn(async move {
      let client = reqwest::Client::new();
      let _ = client.post(env::var("DINGTALK_WEBHOOK").unwrap_or_default())
        .json(&serde_json::json!({ "text": { "content": format!("【报警】{}: {}",
alarm.source, alarm.message) } })))
        .send().await;
    });
  }
  Ok(alarm)
}

```

设计要点: tokio::spawn 异步 + 结果忽略 (let _ =) ——通知失败绝不回滚报警。

9.23.3 配置化

```

# .env
DINGTALK_WEBHOOK=https://oapi.dingtalk.com/robot/send?access_token=xxx

```

扩展思路: 通知渠道 (钉钉/邮件/短信) 都可做成 trait——Notifier trait + 多实现, 在报警处注入。

9.24 8.24 扩展实战: 完整案例五 (历史数据回放页)

9.24.1 需求 (承接 05 章 5.56 的脑内规划, 这里给出落地)

页面: /playback (fj200c_information)

功能: 选 CSV 文件 → 1 秒/帧回放表格

9.24.2 数据源: 后端加“读 CSV”接口

```

#[utoipa::path(
  get,
  path = "/api/fj200c_information/csv/content",
  tag = "fj200c_information",
  operation_id = "fj200cInformationCsvContent",
  params(("name" = String, Query, description = "CSV 文件名")),
  responses((status = 200, description = "成功", body = Vec<TableRow>)))
)]
pub async fn csv_content(Query(name): Query<String>, ...) ->
Result<Json<ApiResponse<Vec<TableRow>>>, AppError> {
  let rows = services::parse_csv(&name).await?; // 读文件 + 逐行解析成 TableRow
  Ok(Json(ApiResponse::success(rows)))
}

```

9.24.3 前端回放

// usePlayback.ts (05 章练习的正式版)

```

export function usePlayback(rows: Ref<TableRow[]>, intervalMs = 1000) {
  const playing = ref(false)
  const index = ref(0)
  let timer: number | null = null

  const play = () => {
    if (playing.value || rows.value.length === 0) return
    playing.value = true
    timer = window.setInterval(() => {
      index.value++
      if (index.value >= rows.value.length) stop()
    }, intervalMs)
  }
}

```

```

}
const pause = () => { playing.value = false; if (timer) clearInterval(timer) }
const stop = () => { pause(); index.value = 0 }
const current = computed(() => rows.value[index.value])

onUnmounted(pause)
return { playing, index, current, play, pause, stop }
}

```

9.24.4 页面组装

```

<template>
  <div>
    <el-select v-model="selectedCsv" @change="loadContent">
      <el-option v-for="f in csvFiles" :key="f" :value="f" :label="f" />
    </el-select>
    <el-button-group>
      <el-button @click="playback.play()">播放</el-button>
      <el-button @click="playback.pause()">暂停</el-button>
      <el-button @click="playback.stop()">停止</el-button>
    </el-button-group>
    <el-table :data="playback.current ? [playback.current] : []" />
  </div>
</template>

```

这个案例把全书知识全用上了：后端接口 + 类型同步 + composable + 组件组装。

9.25 8.25 扩展思路库（灵感清单）

方向	点子	难度
数据	仪表盘 (ECharts 多图组合)	中
数据	CSV 趋势分析 (选时间范围出曲线)	中
集成	设备状态自动巡检脚本	低
集成	数据上传到上级系统 (HTTP 上报)	中
界面	监控大屏 (fj200c_main ScaledPage 复用)	中
界面	暗色模式统一 (7 应用一键切)	低
功能	用户操作审计日志 (表 + 列表页)	中
功能	多用户角色自定义 (动态角色表)	高
运维	前端版本号显示 + 更新检查	低
运维	数据保留策略 (自动清理)	低

挑选标准：优先“低难度 + 高频需求”，从简单开始积累信心。

9.26 8.26 扩展实战：完整案例六（审计日志模块）

9.26.1 需求

记录所有用户的管理操作（谁、何时、做了什么），供追溯。

9.26.2 实现思路

方案 A：中间件全局记录（推荐）

```

// src/common/middleware.rs 或新模块
// 对 /api/admin/* 与 /api/*/items 的写操作自动记录
async fn audit_middleware<B>(<

```

```

    State(state): State<AppState>,
    req: Request<B>,
    next: Next<B>,
  ) -> Result<Response, AppError> {
    let method = req.method().to_string();
    let path = req.uri().path().to_string();

    let res = next.run(req).await;

    // 只记录写操作 + 成功
    if matches!(method.as_str(), "POST" | "PUT" | "DELETE") {
      let user = get_current_user().await; // 从 token 解析
      sqlx::query("INSERT INTO audit_logs (username, method, path, created_at) VALUES
        (?, ?, ?, datetime('now'))")
        .bind(user).bind(method).bind(path)
        .execute(&state.db).await?;
    }
    Ok(res)
  }
}

```

方案 B：各 service 手动记录——侵入性强，不推荐。

9.26.3 效果

admin 界面或新前端页面展示 audit_logs 表
→ 追溯：谁删了台账、谁改了角色

扩展要点：横切关注点（审计/日志/限流）用中间件最优雅——一处实现，全部接口生效。

9.27 8.27 扩展实战：完整案例七（CSV 趋势分析页）

9.27.1 需求

fj200c_information 数据页支持选择 CSV 文件 → 显示参数随时间变化的曲线。

9.27.2 实现

1. 后端：csv/content 接口（8.24 案例五已有）返回 TableRow[]
2. 前端：TrendAnalysis.vue
 - el-select 选文件 → 拉数据
 - ECharts 折线：x 轴时间戳，y 轴所选参数
 - 参数选择 el-select（28 字段里挑）
3. 复用 Visual.vue 的图表组件（props 化）

// 图表数据组装（示意）

```

const chartData = computed(() => {
  const key = selectedParam.value
  return rows.value.map((r, i) => [i, Number((r as any)[key])])
})
watch(chartData, (data) => {
  chart.value?.setOption({ xAxis: { type: 'value' }, series: [{ type: 'line', data }] })
})

```

价值：把历史 CSV 变成可视化——“数据资产”立刻可读。

9.28 8.28 扩展实战：完整案例八（暗色模式一键统一切换）

9.28.1 需求

7 个应用统一提供暗色模式切换，且偏好持久化。

9.28.2 实现

```
// shared 加一个 useTheme composable（一次实现，7 应用共享）
// packages/shared/src/composables/useTheme.ts
export function useTheme() {
  const theme = ref<'light' | 'dark'>(localStorage.getItem('theme') ?? 'light')

  const apply = () => {
    document.documentElement.classList.toggle('dark', theme.value === 'dark')
    localStorage.setItem('theme', theme.value)
  }

  const toggle = () => { theme.value = theme.value === 'light' ? 'dark' : 'light';
  apply() }

  onMounted(apply)
  return { theme, toggle }
}

<!-- AppNavbar 加切换按钮（shared 组件，一处修改全部生效） -->
<el-switch v-model="theme" @change="toggle" active-text="暗色" inactive-text="亮色" />
```

要点：Element Plus 已支持 html.dark (dark/css-vars.css 已全局引入) ——只需要切 class + 持久化。

9.29 8.29 扩展实战：完整案例九（多语言 i18n 方案要点）

9.29.1 需求

界面支持中/英文切换（如外事演示）。

9.29.2 方案要点（不落地，给思路）

1. 引入 vue-i18n（workspace 根装）
2. shared 建 locales/zh.ts、en.ts（文案映射表）
3. 各应用 main.ts 挂载 i18n
4. 页面文案 {{ t('common.save') }} 替换硬编码
5. 切换按钮 + localStorage 持久化
6. Element Plus 自带 locale 切换（zh-cn/en）

工作量评估：文案抽取是体力活（各应用数百条），建议先做 admin + fw100 试点。

9.30 8.30 扩展路线图（按优先级排序）

flowchart LR

```
P0[P0 基础完善<br/>审计日志/暗色模式] --> P1[P1 数据价值<br/>趋势分析/报表扩展]
P1 --> P2[P2 集成<br/>告警通知/数据上报]
P2 --> P3[P3 高级<br/>多语言/自定义仪表盘]
```

建议：P0 先做（低难度高价值），P1~P3 按业务需求排期。

9.31 8.31 结语：从读者到开发者

9.31.1 8.31.1 你现在能做什么

- ✓ 看懂：项目所有代码（借助索引定位）
- ✓ 修改：字段/页面/菜单/配置/逻辑
- ✓ 扩展：新增模块/角色/应用/协议
- ✓ 运维：部署/备份/排障/安全加固
- ✓ 传承：向同事讲解架构（mermaid 图即讲稿）

9.31.2 8.31.2 建议的学习闭环

1. 按 00 章阅读路线通读一遍（理解）
2. 按各章自测题自检（记忆）
3. 按动手任务实操（应用）
4. 按 08 章完成一次完整新增模块（创造）

9.31.3 8.31.3 使用文档的姿势

遇到不懂 → 00 章索引 → 定位章节 → 精读
改代码前 → 查对应模块章节 → 模仿现有实现
故障时 → 07 章排查表 → 按流程定位

9.31.4 8.31.4 最后的话

这套系统不复杂：一个 Rust 后端 + 七个 Vue 前端 + 一套类型契约。你已经走完从“项目是什么”到“怎么改它”的全部路程。剩下的就是——动手。

祝编码愉快。

9.32 8.32 全教程最终自测（跨章综合）

1. 说出系统架构一句话版本。
2. 后端认证完整链路（登录 → JWT → 中间件 → 权限）。
3. 前端一个监控页的完整数据流（守卫 → composable → WS → 渲染）。
4. 改一个字段的完整链路（Rust → gen:api → 前端）。
5. 部署一个版本的完整流程。
6. 新增一个角色需要动哪些文件（能背出 7 步）。
7. 排查“页面白屏”的步骤。
8. 解释 WS 事件为何手写类型。
9. 说出 7 个应用的端口与职责。
10. 用一张 mermaid 图讲清系统（默画）。
11. 扩展的四个维度是什么？（功能/设备/界面/安全数据）
12. 新增角色的七步流程分别是什么？
13. 横切关注点（审计/限流）用什么实现？（中间件）
14. 暗色模式一键切换的核心机制？（html.dark class + 持久化）
15. i18n 改造的主要工作量在哪？（文案抽取）

能答出全部 → 毕业。

9.33 8.33 扩展补充：综合实战——给 fw100 加“告警阈值”功能（端到端演练）

9.33.1 8.33.1 需求

fw100 设备台账支持设置告警阈值（如温度上限），超限时前端提示。

9.33.2 8.33.2 后端改动

```
// src/fw100/models.rs
pub struct Item {
    // ... 原有字段
    pub alarm_threshold: Option<f64>, // 新增阈值
}

// src/fw100/services.rs
// 新增：查询超限设备
pub async fn find_alarming(db: &SqlitePool) -> Result<Vec<Item>, AppError> {
```

```

sqlx::query_as::(<_, Item>(
    "SELECT * FROM items WHERE latest_value > alarm_threshold"
)).fetch_all(db).await
}

```

9.33.3 8.33.3 gen:api 同步

npm run gen:api

9.33.4 8.33.4 前端改动

```

<script setup lang="ts">
// 页面定时检查（30 秒轮询）
const alarming = ref<Item[]>([])
const check = async () => { alarming.value = await fw100Api.findAlarming() }
onMounted(() => { check(); setInterval(check, 30000) })
</script>
<template>
  <el-alert v-if="alarming.length" type="warning" title="存在超限设备" />
</template>

```

9.33.5 8.33.5 完成清单

1. 后端：字段 + service + handler + utoipa 注解
2. gen:api：类型同步
3. 前端：页面展示 + 轮询
4. 验证：build + 手动测试

这就是一次完整的小型扩展——流程与案例一~九完全一致，只是规模更小。

9.34 8.34 扩展补充：性能与扩展性的预判

9.34.1 8.34.1 什么时候需要关注性能

1. 数据量 > 10 万行
2. 并发 > 10 个客户端
3. 帧率 > 100 Hz
4. 页面首屏 > 3 秒

9.34.2 8.34.2 性能优化的优先级

- ① SQL 索引（最便宜、收益最大）
- ② 前端节流/限长（监控数据）
- ③ 缓存（热点配置）
- ④ 分布式（最后手段，成本最高）

9.34.3 8.34.3 千万避免

- ✗ 过早优化（还没瓶颈就重构）
- ✗ 盲目上缓存（一致性风险）
- ✗ 分布式化（运维复杂度飙升）

原则：先用配置和代码优化，实在不够才动架构。

9.35 8.35 扩展补充：与外部系统对接的注意事项

9.35.1 8.35.1 对接前必问

1. 对方协议是什么？（HTTP/WS/文件/数据库）
2. 数据格式？（JSON/XML/CSV）
3. 鉴权方式？（Token/证书/无）
4. 数据方向？（我方主动推 / 对方来拉）

9.35.2 8.35.2 常见对接模式

模式	实现	项目例子
HTTP 推送	request 客户端	数据上报
HTTP 提供	新路由	对外接口
文件交换	定时扫描目录	CSV 导入
消息队列	需要引入依赖	尚未使用

9.35.3 8.35.3 健壮性要求

1. 超时重试（指数退避）
2. 失败不阻塞主流程（旁路）
3. 记录失败日志 + 可重放
4. 先联调后上线（mock 对端）

9.36 8.36 扩展补充：08 章最终自测（追加 8 题）

1. 端到端演练的完整步骤？
2. 什么时候需要关注性能？
3. 性能优化的优先级？
4. 为什么别盲目分布式？
5. 对接前必问的四个问题？
6. 四种对接模式？
7. 对接的四个健壮性要求？
8. 旁路通知为什么用异步？

答对 7+ → 08 章彻底通关。

9.37 8.37 扩展补充：新增一个“巡检记录”完整模块（最小可行案例）

9.37.1 8.37.1 需求

fw100 增加巡检记录：设备 + 巡检人 + 结果 + 备注，支持增删改查。

9.37.2 8.37.2 后端五件套

1. src/fw100/models.rs: InspectionRecord DTO (ToSchema)
2. src/fw100/services.rs: CRUD 函数 (sqlx)
3. src/fw100/handlers.rs: 5 个 handler + utoipa 注解
4. src/database.rs: CREATE TABLE inspection_records
5. src/routes.rs: 挂 /api/fw100/inspection 路由组

9.37.3 8.37.3 表结构

```
CREATE TABLE inspection_records (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  item_id INTEGER NOT NULL,  
  inspector TEXT NOT NULL,  
  result TEXT NOT NULL,           -- pass / fail  
  remark TEXT,  
  created_at TEXT DEFAULT (datetime('now'))  
);
```

9.37.4 8.37.4 gen:api + 前端

npm run gen:api

```
<!-- 在 fw100 列表页加“巡检记录”Tab（复用表格+表单模板） -->  
<el-tabs>  
  <el-tab-pane label="台账">
```

```
<el-tab-pane label="巡检记录">
  <!-- el-table + 新增/编辑 dialog -->
```

9.37.5 8.37.5 完成自检

1. cargo test 通过 (openapi 无冲突)
2. gen:api 生成 inspection 相关函数
3. build 通过
4. 手动: 新增/编辑/删除/列表正常

这是最小的完整闭环——学完这个，其他模块扩展同理。

9.38 8.38 扩展补充：多模块协作的扩展（联动类需求）

9.38.1 8.38.1 场景

巡检记录与告警联动：巡检 fail → 自动创建告警

9.38.2 8.38.2 实现方式

service 层调用（不跨 HTTP）：

create_inspection(db, req) → 若 fail → 调 alarm_service::create(db, ...)
→ 两个 service 在同一进程内，直接函数调用

9.38.3 8.38.3 注意点

1. 事务：两个写入用同一 tx（要么都成功）
2. 解耦：service 之间依赖通过参数传递（不 new 全局）
3. 可测试：注入 mock 依赖

9.39 8.39 扩展补充：08 章高频自测（8 题）

1. 后端五件套是什么？
2. 表结构设计要点？
3. 最小闭环的自检清单？
4. 跨模块调用为什么走 service？
5. 事务的作用？
6. 解耦的原则？
7. mock 依赖注入的意义？
8. 巡检 fail 联动告警的流程？

答对 7+ → 08 章高频通过。

9.40 8.40 扩展补充：部署新版本的完整流程（扩展后必知）

9.40.1 8.40.1 完整发布流程

1. 代码完成（后端 + 前端 + 契约）
2. 全部测试通过（cargo test + build）
3. deploy.bat 一键构建（前端 7 个 + 后端 embedded）
4. 备份旧版 deploy/ 目录（含 rustweb.db）
5. 替换新 exe（保留 .env / ini / csv / db）
6. 重启服务
7. 验证：登录 + 核心功能冒烟
8. 回滚预案：保留旧 exe 随时可切回

9.40.2 8.40.2 发布检查清单

项	检查
前端	7 个 build 全过
契约	gen:api 无意外 diff
后端	cargo build --release --features embedded 过
数据库	无迁移需求（有则提供迁移脚本）
配置	ini 有变化则说明
文档	接口/角色/配置变化已同步

9.40.3 8.40.3 回滚方案

1. 备份旧 exe (deploy_backup/)
2. 出问题 → 停服务 → 换回旧 exe → 启动
3. 数据库若被新版本改过 → 需先恢复备份
4. 永远保留最近两版备份

9.41 8.41 扩展补充：给 7 个应用加公共功能的思路

9.41.1 8.41.1 公共功能的三种层级

1. shared 组件/composable（所有应用直接 import）
2. 后端公共接口（所有前端调用）
3. 各自应用复制（不推荐，维护地狱）

9.41.2 8.41.2 实例：加“关于”页面

方案 A: shared 组件 AboutPage → 各应用路由加 /about

方案 B: 每个应用各写一个（重复）

推荐 A: 一处实现 7 处生效

9.41.3 8.41.3 判断标准

功能要被 ≥2 个应用用 → 放 shared

只被 1 个应用用 → 放应用内部

9.42 8.42 扩展补充：08 章综合自测（8 题）

1. 发布流程的 8 步？
2. 部署前备份什么？
3. 回滚方案？
4. 数据库被改过的回滚注意？
5. 公共功能的三种层级？
6. shared 组件的判断标准？
7. 为什么别复制粘贴公共功能？
8. 冒烟验证测什么？

答对 7+ → 08 章综合通过。

9.43 8.43 扩展补充：给模块加“历史数据回放”的完整实现（复盘案例五）

9.43.1 8.43.1 需求回顾

读取已记录 CSV → 按时间顺序回放数据（像看视频）

→ 用于事故分析/演示

9.43.2 8.43.2 后端设计

1. GET /api/xxx/csv/list → 文件列表
2. GET /api/xxx/csv/content?file=xxx.csv → 全量数据 (TableRow[])
3. 前端回放: 定时器逐行展示 (不是服务端推送)

9.43.3 8.43.3 前端回放实现

```
const playing = ref(false)
const timer = ref<ReturnType<typeof setInterval> | null>(null)

const play = () => {
  playing.value = true
  timer.value = setInterval(() => {
    cursor.value++
    if (cursor.value >= rows.value.length) { pause(); return }
    displayRows.value = rows.value.slice(0, cursor.value + 1)
  }, 100) // 每 100ms 前进一行
}

const pause = () => { playing.value = false; clearInterval(timer.value!) }
const reset = () => { cursor.value = 0; displayRows.value = [] }

onUnmounted(pause) // 离开页面停止
```

9.43.4 8.43.4 回放的优化

1. 大数据文件: 分页加载 + 滑动窗口
2. 速度控制: 0.5x / 1x / 2x
3. 跳转: 进度条拖动
4. 图表同步: 回放点实时更新

9.44 8.44 扩展补充: 多协议设备的统一抽象 (复盘案例二)

9.44.1 8.44.1 问题

不同设备协议不同 (帧头/校验/字段)
→ 前端调用要统一 (都是"连接/读取/控制")

9.44.2 8.44.2 后端抽象

```
trait DeviceProtocol {
  fn parse(&self, bytes: &[u8]) -> Option<DeviceData>; // 解析
  fn build_command(&self, cmd: &DeviceCmd) -> Vec<u8>; // 命令
}

struct ProtocolA; // 协议 A 实现
struct ProtocolB; // 协议 B 实现
```

9.44.3 8.44.3 前端如何面对多协议

后端把不同协议统一成 DeviceData (同一 DTO)
→ 前端无感知 (类型统一)
→ 契约只加 DeviceData 一个类型

9.44.4 8.44.4 新增协议的成本

只写一个 impl DeviceProtocol (约 100 行)
→ 其余代码零改动
→ 这是抽象层的价值

9.45 8.45 扩展补充: 08 章终局自测 (8 题)

1. 回放的数据流 (后端到前端)?
2. 回放定时器的清理时机?
3. 回放的四个优化点?

4. 多协议抽象的两个 trait?
5. 前端如何面对多协议?
6. 新增协议的成本?
7. 回放结束的边界处理?
8. 速度控制的实现?

答对 7+ → 08 章终局通过。

9.46 8.46 扩展补充：全链路示例——从需求到上线的完整复盘

9.46.1 8.46.1 需求

给 fj200c_information 加"参数超限声音告警":

后端解析帧 → 判断超限 → 推送告警事件

前端收到 → 播放提示音 + 页面闪烁

9.46.2 8.46.2 后端改动

```
// 1. 解码后判断 (services.rs 或专门 check_alarm)
if row.ng_speed > config.max_speed {
    // 2. 广播告警事件 (WS)
    tx.send(json!({
        "type": "alarm",
        "data": { "param": "ngSpeed", "value": row.ng_speed, "limit": config.max_speed }
    })).to_string().await?;
    // 3. 记录日志
    warn!("转速超限: {} > {}", row.ng_speed, config.max_speed);
}
```

9.46.3 8.46.3 配置项

```
[Alarm]
MaxSpeed = 3600          ; 转速上限
MaxTemp = 105            ; 水温上限
Sound = true             ; 声音开关
```

9.46.4 8.46.4 前端改动

```
// 1. WS 消息加 alarm 类型
// types.ts
| { type: 'alarm'; data: AlarmInfo }

// 2. 处理
case 'alarm':
    ElNotification.warning({
        title: '参数超限',
        message: `${msg.data.param} 达到 ${msg.data.value} (上限 ${msg.data.limit})`,
    })
    if (audioEnabled.value) playBeep()
    break
```

9.46.5 8.46.5 上线流程

1. cargo test (含 openapi 导出)
2. gen:api (alarm 类型不涉及 HTTP, 仅手写 WS 类型)
3. 前端 build
4. deploy.bat
5. 冒烟: 设置低阈值验证告警
6. 恢复阈值 + 正式使用

9.47 8.47 扩展补充：理解“ 契约驱动开发” 的心智模型

9.47.1 8.47.1 核心思想

先定“数据长什么样”（契约），再写前后端
→ 前后端可以并行开发（mock 契约即可）
→ 减少联调期的反复

9.47.2 8.47.2 在本项目的落地

Rust DTO + utoipa → openapi.json 就是契约文档
前端类型由契约生成 → 不会不一致
接口字段变化 → 生成代码自动同步

9.47.3 8.47.3 心智模型一句话

“先画数据形状，再填充血肉”
→ 项目里所有功能都能用这个思路拆解

9.48 8.48 扩展补充：08 章毕业自测（8 题）

1. 告警功能的完整链路？
2. 告警事件怎么广播？
3. 告警配置放哪？
4. 前端如何处理告警？
5. 上线流程的六步？
6. 契约驱动开发的核心思想？
7. 为什么能并行开发？
8. 心智模型的一句话？

答对 7+ → 08 章毕业。

9.49 8.49 扩展补充：后端新模块的完整骨架（写代码之前先看）

9.49.1 8.49.1 目录骨架

```
src/xxx/
├─ mod.rs          # 模块声明 + 全局状态
├─ handlers.rs     # HTTP 层（薄）
├─ services.rs     # 业务层（厚）
├─ models.rs       # DTO + ToSchema
├─ config.rs       # ini 解析（如需要）
└─ mock.rs         # 模拟数据（如需要）
```

9.49.2 8.49.2 mod.rs 的标准内容

```
pub mod config;
pub mod handlers;
pub mod models;
pub mod services;
pub mod mock;

use std::sync::OnceLock;
use tokio::sync::broadcast;
use std::sync::Arc;

// 全局状态（模块级）
pub static TX: OnceLock<broadcast::Sender<String>> = OnceLock::new();
pub static RUNNING: OnceLock<std::sync::atomic::AtomicBool> = OnceLock::new();

pub fn init() {
    let (tx, _rx) = broadcast::channel(100);
    TX.set(tx).ok();
}
```

```

    RUNNING.set(std::sync::atomic::AtomicBool::new(false)).ok();
}

```

9.49.3 8.49.3 handlers.rs 的标准内容

```

#[utoipa::path(
    get,
    path = "/api/xxx/status",
    tag = "xxx",
    operation_id = "xxxStatus",
    responses((status = 200, description = "状态", body = ApiResponse<ServiceStatus>))]
)]
pub async fn status(
    State(state): State<AppState>,
) -> Result<Json<ApiResponse<ServiceStatus>>, AppError> {
    Ok(Json(ApiResponse::success(get_status())))
}

```

9.49.4 8.49.4 routes.rs 挂载

```

let xxx_routes = Router::new()
    .route("/status", get(xxx::handlers::status))
    .route_layer(permission_middleware(Permission::XxxMonitor));
router = router.nest("/api/xxx", xxx_routes);

```

9.50 8.50 扩展补充：08 章权威自测（8 题）

1. 目录骨架的六个文件？
2. mod.rs 的全局状态？
3. init 函数做什么？
4. handler 的标准注解？
5. 路由挂载的写法？
6. 权限中间件的参数？
7. 广播通道的容量？
8. 为什么要 init 集中初始化？

答对 7+ → 08 章权威。

9.51 8.51 扩展补充：前端新页面的完整落地流程

9.51.1 8.51.1 新建页面五步

1. 建 views/XxxView.vue（复制相似页面改）
2. 路由加一条（含 meta）
3. 菜单/导航加入口（如需要）
4. api facade 加调用（如需要）
5. 验证：dev 起服务 → 访问 → build

9.51.2 8.51.2 复制改名的技巧

1. 复制最相似的页面（表格页/监控页）
2. 全局替换组件名/导入
3. 逐步删掉不需要的部分
4. 保留骨架（加载/错误处理/布局）

9.51.3 8.51.3 页面接入权限

```

// 路由 meta 加权限（可选）
{ path: '/report', component: ReportView, meta: { requiresAuth: true, permission: 'XxxView' } }
// 守卫里检查
if (to.meta.permission && !auth.hasPermission(to.meta.permission)) {
    return { path: '/403' }
}

```

9.51.4 8.51.4 验证清单

1. 无权限角色访问 → 403/跳转
2. 有权限角色访问 → 正常
3. 深链接刷新 → 不 404 (SPA 回退)
4. 移动端适配 (如需要)

9.52 8.52 扩展补充: 08 章权威自测 (8 题)

1. 新建页面的五步?
2. 复制改名技巧?
3. 页面权限怎么加?
4. 验证清单的四点?
5. 无权限的返回?
6. SPA 深链接的坑?
7. 菜单入口加在哪?
8. 骨架保留哪些?

答对 7+ → 08 章权威。

9.53 8.53 扩展补充: 扩展能力的评估框架

9.53.1 8.53.1 评估五个维度

维度	问题	判断
需求	谁用/多久用一次	低频可手工
数据	存哪/多大	规模决定方案
实时性	要实时还是轮询	WS vs HTTP
权限	谁能操作	角色/权限点
部署	影响哪些应用	改动范围

9.53.2 8.53.2 小扩展 vs 大扩展

小扩展 (0.5~2 天):

- 加字段/加接口/加页面
- 修改配置/逻辑
- 单应用改动

大扩展 (1~2 周):

- 新角色/新应用
- 新协议/新数据源
- 跨应用功能

9.53.3 8.53.3 扩展的实施顺序

1. 写需求一句话
2. 画数据流图
3. 定契约 (DTO/接口)
4. 后端实现
5. `gen:api`
6. 前端实现
7. 验证 + 部署

9.54 8.54 扩展补充: 08 章权威自测 (8 题)

1. 评估五个维度?
2. 小扩展与大扩展的划分?
3. 实施顺序的七步?

4. 需求一句话怎么写？
5. 数据规模如何决定方案？
6. 实时性怎么选？
7. 改动范围的评估？
8. 为什么先定契约？

答对 7+ → 08 章权威。

9.55 8.55 扩展补充：扩展质量的验收标准

9.55.1 8.55.1 功能验收

1. 主流程可用（增删改查/启停）
2. 边界处理（空数据/超限/重复）
3. 错误提示明确
4. 权限控制生效

9.55.2 8.55.2 代码验收

1. 通过 cargo test + vue-tsc
2. 无 unwrap 滥用
3. 契约已同步（gen:api）
4. 日志/错误规范

9.55.3 8.55.3 文档验收

1. AGENTS.md 同步（接口/角色/端口）
2. 本套文档相应章节补充
3. 变更记录可追溯

9.56 8.56 扩展补充：给新手的最终建议

1. 别怕改坏—git 随时能还原
2. 从最小改动开始（加字段→加接口→加页面）
3. 每次只做一件事，做完验证
4. 报错就是学习资料，仔细读
5. 模仿是最好的老师（role_template 的意义）
6. 文档和代码同步更新
7. 完成一个小功能就庆祝一下

9.57 8.57 扩展补充：08 章权威自测（8 题）

1. 功能验收的四点？
2. 代码验收的四点？
3. 文档验收的三点？
4. 七条最终建议？
5. 为什么别怕改坏？
6. 最小改动从哪开始？
7. 模仿的价值？
8. 做完验证的意义？

答对 7+ → 08 章权威。

9.58 8.58 扩展补充：本章收尾——从读者到作者的最后一课

9.58.1 8.58.1 扩展的本质

扩展 = 在现有模式上做加法
→ 每个模块都是模板
→ 每次扩展都在复用已有模式
→ 模式熟 → 扩展快

9.58.2 8.58.2 本书给到你的工具箱

1. 后端：三层架构 + 状态机 + 并发原语
2. 前端：五步页面 + 组件模式 + store
3. 契约：utoipa + orval 全链路
4. 运维：部署/备份/排障脚本
5. 方法：模仿 → 修改 → 创造

9.58.3 8.58.3 最后的告别

这套系统不复杂，复杂的是"从零开始"的恐惧。
现在你已经读完了全部 9 章、超过 10 万字的教程，
掌握了从架构到运维、从语法到扩展的全部知识。

剩下的只有一件事：打开编辑器，动手。

祝你编码愉快，成为这套系统的下一个贡献者。

9.59 8.59 扩展补充：08 章最终自测（6 题）

1. 扩展的本质是什么？
2. 工具箱的五件套？
3. 从零开始的恐惧怎么破？
4. 模仿的意义？
5. 你现在能做什么？
6. 下一步做什么？

答对 5+ → 08 章最终完成，全书通关。

全文完。祝学习顺利。